book 2

mathematics for programmers

trigonometry to monads in c23, with a physics game capstone

Raeliosol

version 2.1, compiled 10 2026

Contents

1. floating point anatomy 7
   1.1. the bits of a double 7
   1.2. INFINITY, but 1.0 / −0.0 == -INFINITY“. The sample holds both halves 9
   1.3. one ulp 10
   1.4. rounding, pinned 11
   1.5. the subnormal floor 13
   1.6. what math.h promises 14
   1.7. the 2^53 frontier 16
2. error, conditioning, and stability 18
   2.1. machine numbers, ulp, and unit roundoff 18
   2.2. propagation and the condition of a subtraction 19
   2.3. the quadratic formula: cancellation in the wild 20
   2.4. conditioning versus stability 22
   2.5. compensated summation 23
   2.6. precision ladders and what more bits buy 26
3. trigonometry 30
   3.1. the unit circle and radians 30
   3.2. the identity kit 31
   3.3. atan2, the quadrant-honest inverse 32
   3.4. range reduction and a polynomial sin cos 34
   3.5. laws of sines and cosines 35
   3.6. the ambiguous ssa case 37
   3.7. large arguments and payne-hanek 38
   3.8. next 39
4. coordinate geometry 40
   4.1. points, vectors, direction 40
   4.2. orientation: one sign, exactly 41
   4.3. lines, rays, segments 43
   4.4. circles and conics 45
   4.5. affine transforms and order 46
   4.6. polar and parametric curves 48
5. vectors and vector spaces 51
   5.1. vectors and linear combinations 51
   5.2. span and subspaces 53
   5.3. linear independence and elimination 55
   5.4. basis and dimension 58
   5.5. norms, unit vectors, and equivalence 59
   5.6. what a basis buys a programmer 62
6. matrices and linear systems 64
   6.1. systems of linear equations 64
   6.2. matrix algebra 67
   6.3. elimination with partial pivoting 68
   6.4. lu factorization 71
   6.5. rank and the null space 73
   6.6. the inverse 76
7. inner products and projections 79
   7.1. the inner product 79
   7.2. lengths, distances, angles 80

7.3. orthonormal bases and the orthogonal complement ... 82
7.4. orthogonal projections ... 84
7.5. inner products of functions ... 88
7.6. rotations revisited ... 90
8. matrix decompositions ... 92
8.1. determinant and trace ... 92
8.2. eigenvalues and eigenvectors ... 94
8.3. symmetric matrices and cholesky ... 96
8.4. qr factorization ... 97
8.5. singular value decomposition ... 98
8.6. low-rank approximation and pca ... 100
9. single-variable calculus ... 104
9.1. limits and continuity ... 104
9.2. the derivative as a slope machine ... 106
9.3. the differentiation rules ... 108
9.4. taylor series ... 109
9.5. integration and the fundamental theorem ... 113
9.6. what survives in floats ... 116
10. multivariate calculus ... 119
10.1. partial derivatives as limits ... 119
10.2. the gradient and level sets ... 120
10.3. jacobians and the chain rule ... 122
10.4. the hessian and curvature ... 124
10.5. linearization and taylor ... 125
10.6. gradient identities that pay rent ... 128
11. continuous optimization ... 131
11.1. stationary points and basins ... 131
11.2. gradient descent ... 132
11.3. newton's method for optimization ... 136
11.4. constrained optimization and lagrange multipliers ... 138
11.5. convex sets and functions ... 141
11.6. programs: linear and quadratic ... 143
12. automatic differentiation ... 146
12.1. the accuracy ceiling of finite differences ... 146
12.2. dual numbers and forward mode ... 147
12.3. the tape and reverse mode ... 149
12.4. backpropagation on a layered function ... 151
12.5. the cost model: columns versus sweeps ... 152
12.6. verification discipline ... 153
13. logic, sets, and proof ... 156
13.1. propositions as bit patterns ... 156
13.2. quantifiers as bounded loops ... 157
13.3. sets as bitmasks and relations as matrices ... 159
13.4. functions and the pigeonhole principle ... 162
13.5. induction as a loop invariant ... 164
13.6. invariants that survive mutation ... 165
13.7. proofs as data ... 166
14. combinatorics ... 172
14.1. the sum and product rules as code ... 172
14.2. permutations and combinations ... 174
14.3. the binomial theorem ... 175

14.4. inclusion-exclusion and derangements ... 177
14.5. recurrences by iteration ... 178
14.6. generating functions as exact coefficients ... 180
14.7. from counting to probability ... 182
15. graphs and order theory ... 184
15.1. graph models and degree sums ... 184
15.2. walks, paths, and connectivity ... 186
15.3. trees ... 188
15.4. partial orders ... 191
15.5. lattices ... 192
15.6. equivalence relations and partitions ... 195
16. groups and subgroups ... 199
16.1. the axioms as a check suite ... 199
16.2. z mod n and its units ... 201
16.3. subgroups and the subgroup one element generates ... 203
16.4. cyclic groups and generator search ... 205
16.5. order, cosets, and lagrange ... 207
16.6. permutations, matrices, and the triangle ... 209
17. group actions and structure ... 213
17.1. homomorphisms and isomorphisms ... 213
17.2. cosets and quotients ... 215
17.3. direct products ... 217
17.4. rings and fields ... 219
17.5. finite fields ... 220
17.6. the dihedral group d4 acts on a square ... 222
17.7. burnside's lemma counts orbits ... 224
17.8. diffie-hellman and the discrete logarithm ... 226
18. probability and distributions ... 229
18.1. probability spaces ... 229
18.2. conditional probability and Bayes ... 231
18.3. discrete random variables ... 232
18.4. binomial and Poisson families ... 234
18.5. continuous densities and the gaussian ... 236
18.6. conjugacy, the exponential family, and change of variables ... 238
19. statistics and inference ... 241
19.1. sampling and the sample mean ... 241
19.2. the central limit theorem ... 244
19.3. estimators, bias, and variance ... 247
19.4. hypothesis tests ... 250
19.5. confidence intervals ... 253
19.6. least squares statistics ... 254
20. root finding ... 257
20.1. bisection and the halving guarantee ... 257
20.2. fixed points and contraction ... 259
20.3. newton's method and quadratic convergence ... 262
20.4. the secant method and the golden order ... 265
20.5. one error model, three speeds ... 266
20.6. safeguards: keep the bracket ... 267
21. interpolation and approximation ... 273
21.1. the interpolation problem ... 273
21.2. the lagrange and newton forms ... 274

21.3. the runge phenomenon 276
21.4. chebyshev nodes 277
21.5. cubic splines 278
21.6. least squares fitting 281
22. quadrature and differential equations 285
22.1. composite quadrature 285
22.2. adaptive quadrature 286
22.3. euler and the stability question 288
22.4. runge-kutta 4 and the fixed step 290
22.5. stability regions 292
22.6. symplectic integrators and the capstone core 293
23. iterative methods and conditioning 297
23.1. why iterate 297
23.2. splittings: jacobi and gauss-seidel 300
23.3. convergence: the spectral radius decides 303
23.4. power iteration and deflation 306
23.5. condition numbers 308
23.6. iterative refinement 310
24. strategic games 313
24.1. normal form and dominance 313
24.2. best response and pure nash 314
24.3. mixed strategies and nash 315
24.4. the minimax theorem 317
24.5. computing equilibria by support enumeration 319
24.6. regret and follow the regularized leader 321
25. sequential and repeated games 326
25.1. the tree comes first 326
25.2. backward induction 327
25.3. subgame perfection and credible threats 329
25.4. imperfect information 330
25.5. repetition and the discount factor 333
25.6. paying the second price 335
26. algebraic data types in c23 339
26.1. products and sums 339
26.2. the tagged union 340
26.3. exhaustive switch 343
26.4. product machinery: typeof, auto, constexpr 345
26.5. defer and ownership 347
26.6. from sums to option 349
27. option, result, and the railway 352
27.1. from null to option 352
27.2. result, the error channel 354
27.3. the railway, map and then 357
27.4. contracts the compiler can check 360
27.5. cleanup ownership on both tracks 361
27.6. where the railway goes 365
28. functors and monads in c 366
28.1. the shape railway generalized 366
28.2. map and bind over option 367
28.3. result with typed errors 369
28.4. the list monad 372

28.5. one spelling, many types ... 375
28.6. closures, purity, and the price of state ... 378
29. pipelines and effects ... 382
29.1. composition as data ... 382
29.2. the pipeline operator ... 385
29.3. threading state ... 389
29.4. rng discipline ... 391
29.5. the pure core and the impure shell ... 393
29.6. testing effectful code ... 394
30. capstone: the arena game, design ... 396
30.1. the game ... 396
30.2. modules and disciplines ... 399
30.3. the state is a value ... 399
30.4. the numerics budget ... 401
30.5. the opening book and the search ... 402
30.6. two lanes, one truth ... 403
31. capstone: the arena game, implementation ... 406
31.1. the fixed step ... 406
31.2. narrow phase: the separating axis test ... 408
31.3. sequential impulses ... 409
31.4. deterministic grid pathfinding ... 412
31.5. search over the group hash ... 412
31.6. the allegro host ... 414
32. capstone: the arena game, verification ... 416
32.1. the discipline-to-module matrix ... 416
32.2. the determinism hash and its three pins ... 417
32.3. perturbation, sensitivity, and the quantization caveat ... 419
32.4. ai strength, honestly ... 420
32.5. the allegro bench ... 422
32.6. audit totals ... 423
33. appendices: coverage and sources ... 425
33.1. topic coverage ... 425
33.2. the suites and the totals ... 426
33.3. pinned sources ... 427
33.4. the corpus view ... 429

1. floating point anatomy

Three samples carry the chapter: `anatomy.c` decodes bit patterns and asserts them, `rounding.c` pins all four rounding modes against fenv, and `frontier.c` walks the 2^53 integer wall and the subnormal floor. The chapter makes 6 moves: the layout of a double, one ulp, rounding pinned, the subnormal floor, what `math.h` promises, and the integer frontier where doubles stop counting. The dsa chapters treat floats as a tool with tolerances; here the format itself is opened and every bit is asserted. Every behavioral claim below is one of the 63 checks in the 3 samples of chapter 01 or a sentence quoted from a canonical source fetched 2026-09-21. The physics game capstone leans on all of it in book 14, ch 7 (game systems and architecture, deterministic math).

1.1. the bits of a double

Goldberg, fetched 2026-09-21: “Squeezing infinitely many real numbers into a finite number of bits requires an approximate representation.” The representation is 64 bits in 3 fields: a 1 bit sign s, an 11 bit biased exponent e, and a 52 bit fraction f. For every normal value the decode is

$$\text{value} = (-1)^s \cdot \left(1 + \frac{f}{2^{52}}\right) \cdot 2^{e-1023}$$

The exponent is stored biased by 1023 so that, within one sign, comparing two bit patterns as unsigned integers compares the values, ordering comes free with the layout. The free ordering stops at the sign bit: any negative value’s pattern compares greater than every nonnegative one as unsigned integers, the sign bit sits above the magnitude. The fraction stores everything right of the leading 1, which is never stored. The two worked examples of the section:

field	width	1.0	0.15625
sign s	1 bit	0	0
biased exponent e	11 bits	0x3ff	0x3fc
fraction f	52 bits	0x0000000000000	0x4000000000000

The dry run: 0.15625 is 5/32, which is 1.01_2 times 2^{-3}. The biased exponent must satisfy $e - 1023 = -3$, so $e = 1020$, stored as 0x3fc. The single fraction bit carries weight 2^{-2}, and scaled by 2^{52} it stores as $f = 2^{50}$, the pattern 0x4000000000000. Concatenated, the full 64 bits are 0x3fc4000000000000, and the sample asserts exactly that, then reassembles the 3 fields back into 0.15625 bit for bit with `ldexp`.

```c
static int checks = 0;
#define CHECK(cond, name)                                                  \
  do {                                                                     \
    if (cond) {                                                            \
      checks++;                                                            \
      printf("ok %d %s\n", checks, name);                                  \
    } else {                                                               \
      printf("FAIL %s (check %d)\n", name, checks + 1);                    \
      return 1;                                                            \
    }                                                                      \
  } while (0)

static_assert(sizeof(double) == 8, "binary64 assumed throughout this chapter");
static_assert(sizeof(uint64_t) == 8,
              "field extraction needs the full 64 bit view");

constexpr uint64_t frac_mask = 0xfffffffffffffull; /* the 52 fraction bits */
constexpr unsigned expo_mask = 0x7ffu; /* the 11 biased exponent bits */

/* union punning is defined behavior in c, memcpy is the portable spelling */
static uint64_t bits_of(double d) {
  union {
    double f;
    uint64_t u;
  } pun = {.f = d};
  return pun.u;
}
static uint64_t bits_via_memcpy(double d) {
  uint64_t u;
  memcpy(&u, &d, sizeof u);
  return u;
}
static unsigned sign_of(double d) { return (unsigned)(bits_of(d) >> 63); }
static unsigned expo_of(double d) {
  return (unsigned)((bits_of(d) >> 52) & expo_mask);
}
static uint64_t frac_of(double d) { return bits_of(d) & frac_mask; }

/* reassemble (-1)^s * (1 + f/2^52) * 2^(e-1023). f < 2^52 so the divide
 * is exact, ldexp only scales the exponent, and the rebuild is bit exact
 * for every triple this file feeds it */
static double rebuild(unsigned s, unsigned e, uint64_t f) {
  double base = ldexp(1.0 + (double)f / 0x1p52, (int)e - 1023);
  return s ? -base : base;
}
```

listing 1: the check harness, field extraction, and the bit exact rebuild (source: math/samples/src/Ch01/anatomy.c, lines 13-57)

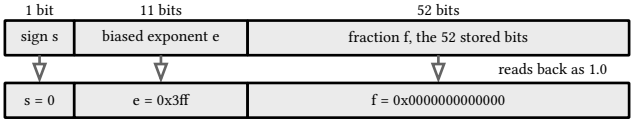


figure 1: the 64 bits of a double, and the decode of 1.0

NOTE: the hidden bit is a free bit

> The leading 1 of the significand is implied, never stored, so 52 stored fraction bits carry 53 bits of significand on every normal value. The bill arrives at the edges: when the exponent field is 0 the implied bit is gone and the value degenerates to $f \cdot 2^{-1074}$, which is exactly the subnormal floor of the fourth section.

Signed zero shows the sign bit doing real work. +0.0 and −0.0 compare equal yet divide differently: cppreference, fetched 2026-09-21, “1.0 / 0.0

1.2. INFINITY, but 1.0 / -0.0 == -INFINITY“. The sample holds both halves

as checks on volatile operands, comparison and division.

```
/* hand computed decode table (probe.py, exact fractions), reprinted live:
 *   1.0      3ff0000000000000 s=0 e=3ff f=0000000000000  2^0
 *   2.0      4000000000000000 s=0 e=400 f=0000000000000  2^1
 *   0.5      3fe0000000000000 s=0 e=3fe f=0000000000000  2^-1
 *   3.0      4008000000000000 s=0 e=400 f=8000000000000  1.5 * 2^1
 *   0.15625  3fc4000000000000 s=0 e=3fc f=4000000000000  1.01b * 2^-3
 *   -0.75    bfe8000000000000 s=1 e=3fe f=8000000000000  -1.5 * 2^-1
 *   +0.0     0000000000000000 s=0 e=000 f=0000000000000  zero
 *   -0.0     8000000000000000 s=1 e=000 f=0000000000000  signed zero
 *   2^-1074  0000000000000001 s=0 e=000 f=0000000000001  min subnormal
 *   2^-1022  0010000000000000 s=0 e=001 f=0000000000000  min normal
 *   inf      7ff0000000000000 s=0 e=7ff f=0000000000000  all-one exponent
 *   nan      7ff8000000000000 s=0 e=7ff f=8000000000000  quiet payload
 *   dbmax    7fefffffffffffff s=0 e=7fe f=fffffffffffff  (2 - 2^-52) * 2^1023
 */
static const struct {
  const char *name;
  double v;
} rows[] = {
    {"1.0", 1.0},           {"2.0", 2.0},         {"0.5", 0.5},
    {"3.0", 3.0},           {"0.15625", 0.15625}, {"-0.75", -0.75},
    {"+0.0", 0.0},          {"-0.0", -0.0},       {"2^-1074", 0x1p-1074},
    {"2^-1022", 0x1p-1022}, {"inf", INFINITY},    {"nan", NAN},
    {"dbmax", DBL_MAX},
};
puts("name     bits             s e   f");
for (auto r = rows; r < rows + sizeof rows / sizeof rows[0]; r++) {
  printf("%-8s %016llx s=%u e=0x%03x f=0x%013llx\n", r->name,
         (unsigned long long)bits_of(r->v), sign_of(r->v), expo_of(r->v),
         (unsigned long long)frac_of(r->v));
}
printf("observed: 13 decodes, every field above matches the hand computed "
       "table\n");
printf("%d checks passed\n", checks);
return 0;
```

listing 2: the decode table, one row per value, printed live (source: math/samples/src/Ch01/anatomy.c, lines 140-174)

Two numbers: 0x3ff0000000000000 is 1.0 and 0xbfe8000000000000 is −0.75. The pair looks unrelated, but 0x3fe8000000000000 is 0.75, and flipping the top bit alone is the whole difference. One sign bit, two zeros, and division keeps the zero’s sign.

1.3. one ulp

Goldberg, fetched 2026-09-21: “This rounding error is the characteristic feature of floating-point computation.” The unit of that error is the ulp, unit in the last place: the spacing between a value and its representable neighbor. `nextafter(v, INFINITY)` returns exactly that neighbor, so subtracting the original back out measures the spacing, and the sample walks 6 rungs from the smallest positive double to 2^{60}.

```
/* spacing table (probe.py, exact): one ulp at v is nextafter(v, inf) - v
 *   2^-1074  0000000000000001 -> 0000000000000002  gap 2^-1074
 *   2^-1022  0010000000000000 -> 0010000000000001  gap 2^-1074
 *   1.0      3ff0000000000000 -> 3ff0000000000001  gap 2^-52
 *   2.0      4000000000000000 -> 4000000000000001  gap 2^-51
 *   2^52     4330000000000000 -> 4330000000000001  gap 1
 *   2^60     43b0000000000000 -> 43b0000000000001  gap 2^8 = 256 */
static const struct {
  double v;
  double gap;
  const char *what;
} spacing[] = {
    {0x1p-1074, 0x1p-1074, "gap at the min subnormal is 2^-1074"},
    {0x1p-1022, 0x1p-1074, "gap at the min normal is still 2^-1074"},
    {1.0, 0x1p-52, "gap at 1.0 is 2^-52"},
    {2.0, 0x1p-51, "gap at 2.0 is 2^-51, one exponent step doubled it"},
    {0x1p52, 1.0, "gap at 2^52 is exactly 1"},
    {0x1p60, 256.0, "gap at 2^60 is 256"},
};
for (auto r = spacing; r < spacing + sizeof spacing / sizeof spacing[0];
     r++) {
  CHECK(nextafter(r->v, INFINITY) - r->v == r->gap, r->what);
}
```

listing 3: measuring one ulp with nextafter, six rungs (source: math/samples/src/Ch01/frontier.c, lines 59-81)

The spacing doubles at every exponent step. The bottom is dense: the minimum normal value 2^{-1022} still sits 2^{-1074} from its neighbor, because the subnormal rungs below it hold the floor at that spacing. At 2^{52} the spacing reaches 1, and at 2^{53} doubles stop counting every integer, section 6′s subject. At 2^{60} the spacing is 256, and nothing in the type records that neighbors a quintillion apart now step by whole hundreds.

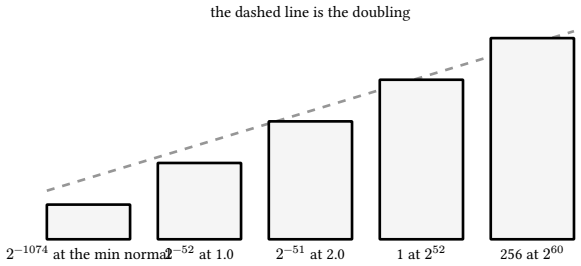


figure 2: one ulp doubles at every exponent step

The same doubling is what `DBL_EPSILON` measures from the middle of the range: at 1.0 the spacing is 2^{-52}, or 2.220446049250313e-16, the smallest step any computation near one can take or even request.

Two numbers: the spacing at the minimum normal is 2^{-1074}, about 5e-324, and at 2^{60} it is 256. Between those two rungs the spacing grows by a factor of 2^{1082}, and the format reports all of it with the same calm 8 byte pattern.

1.4. rounding, pinned

An exact result that does not fit must go somewhere. Goldberg, fetched 2026-09-21: “There are four rounding modes: round toward nearest, round toward ∞, round toward 0, and round toward −∞.” The startup mode is round to nearest, ties to even, and the tie rule is where it earns its keep.

The dry run: 1.0 + 2^(-53) is the exact midpoint between 1.0, whose stored tail is even, and its neighbor 0x3ff0000000000001, whose tail is odd. Ties to even keeps 1.0. Goldberg’s own example: “thus 12.5 rounds to 12 rather than 13 because 2 is even.”

```
int main(void) {
  /* neighbors of 1.0 (probe.py, exact):
   *   0x3ff0000000000000 = 4503599627370496/2^52, even tail
   *   0x3ff0000000000001 = 4503599627370497/2^52, odd tail
   * the exact midpoint is 1.0 + 2^-53 = 9007199254740993/9007199254740992.
   * goldberg 1991 (fetched 2026-09-21) on the tie rule: "thus 12.5 rounds
   * to 12 rather than 13 because 2 is even" */
  volatile double one = 1.0, step = 0x1p-53;
  double tie = one + step;
  /* neighbors of 0.5: 0x3fe0000000000000 even, 0x3fe0000000000001 odd,
   * exact midpoint 0.5 + 2^-54 = 9007199254740993/18014398509481984 */
  volatile double halfv = 0.5, halfstep = 0x1p-54;
  double tie05 = halfv + halfstep;

  CHECK(fegetround() == FE_TONEAREST, "startup mode is ties-to-even");
  CHECK(tie == 1.0, "1.0 + 2^-53, the exact midpoint, rounds down to 1.0");
  CHECK(bits_of(tie) == 0x3ff0000000000000ull,
        "the stored bits of the 1.0 tie are exactly 1.0's");
  CHECK(tie05 == 0.5, "0.5 + 2^-54, the exact midpoint, rounds down to 0.5");
  CHECK(bits_of(tie05) == 0x3fe0000000000000ull,
        "the stored bits of the 0.5 tie are exactly 0.5's");

  CHECK(fesetround(FE_UPWARD) == 0 && fegetround() == FE_UPWARD,
        "fesetround(FE_UPWARD) is accepted and reads back");
  /* probed 2026-09-21 under clang 23: upward moves the 1.0 tie up one ulp */
  CHECK(bits_of(one + step) == 0x3ff0000000000001ull,
        "upward rounds the 1.0 tie to 0x3ff0000000000001");
  CHECK(bits_of(halfv + halfstep) == 0x3fe0000000000001ull,
        "upward rounds the 0.5 tie to 0x3fe0000000000001");

  CHECK(fesetround(FE_DOWNWARD) == 0 && fegetround() == FE_DOWNWARD,
        "fesetround(FE_DOWNWARD) is accepted and reads back");
  CHECK(bits_of(one + step) == 0x3ff0000000000000ull,
        "downward keeps the positive tie at 1.0");

  CHECK(fesetround(FE_TOWARDZERO) == 0 && fegetround() == FE_TOWARDZERO,
        "fesetround(FE_TOWARDZERO) is accepted and reads back");
  CHECK(bits_of(one + step) == 0x3ff0000000000000ull,
        "toward zero keeps the positive tie at 1.0");

  CHECK(fesetround(FE_TONEAREST) == 0 && fegetround() == FE_TONEAREST,
        "restored to the default mode");
```

listing 4: the 1.0 tie recomputed under all four fenv modes (source: math/samples/src/Ch01/rounding.c, lines 35-76)

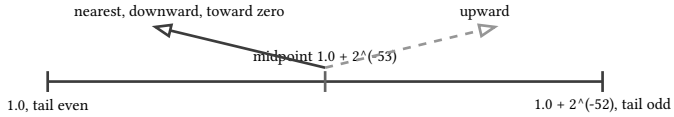


figure 3: the exact midpoint between neighbors goes to the even tail

PITFALL: the pragma that would make this clean is off by default

The c standard gates mode dependent code behind `#pragma STDC FENV_ACCESS ON`. Clang's users manual, fetched 2026-09-21, supports the pragma but leaves it off: "by default `FENV_ACCESS` is disabled", so an ordinary build is free to fold what looks like a runtime computation under the default mode. The samples stand on two measures instead: `volatile` operands force every sum through a runtime instruction that honors the installed mode, and each `fesetround` is verified with a `fegetround` readback before any tie is recomputed.

The other three modes are not curiosities. Interval arithmetic runs its lower bounds downward and its upper bounds upward, and directed rounding turns a wrong last digit into an interval that contains the true value. The sample walks the 1.0 tie through upward, downward, and toward zero, reads the stored bits back in each mode, and restores nearest last.

```
  /* 0.1 = 0x1.999999999999ap-4 = 3602879701896397/2^55 (probe.py, exact)
   * 0.2 = 0x1.999999999999ap-3 = 3602879701896397/2^54
   * exact sum = 10808639105689191/2^55, significand 0x2666666666667 on 54
   * bits, the exact tie between 0x1.3333333333333 and 0x1.3333333333334p-2;
   * ties-to-even picks the tail 4, so the sum is 0x1.3333333333334p-2
   * while 0.3 itself stores 0x1.3333333333333p-2, one ulp lower */
  volatile double a = 0.1, b = 0.2, c = 0.3;
  feclearexcept(FE_ALL_EXCEPT);
  double s = a + b;

  CHECK(s != c, "0.1 + 0.2 is not the double nearest 0.3");
  CHECK(bits_of(s) == 0x3fd3333333333334ull,
        "the sum stores 0x3fd3333333333334");
  CHECK(bits_of(c) == 0x3fd3333333333333ull, "0.3 stores 0x3fd3333333333333");
  CHECK(bits_of(s) - bits_of(c) == 1ull,
        "the two patterns differ by exactly one ulp");
  CHECK(fetestexcept(FE_INEXACT) != 0,
        "the hardware raised FE_INEXACT on that addition");

  printf("observed: 0.1 + 0.2 = 0x3fd3333333333334, 0.3 = 0x3fd3333333333333, "
         "one ulp apart, FE_INEXACT raised\n");
  printf("%d checks passed\n", checks);
  return 0;
}
```

listing 5: 0.1 + 0.2 lands one ulp above 0.3, flag and all (source: math/samples/src/Ch01/rounding.c, lines 78-101)

WARNING: 0.1 + 0.2 is not 0.3

Each literal stores a slight overestimate, the exact sum needs one bit more than 53, and the hardware rounds that sum up to `0x3fd3333333333334` while the literal 0.3 stores `0x3fd3333333333333`. The raised flag is `FE_INEXACT`, and it fires on nearly every interesting computation. Never test computed sums with ==, compare within a tolerance, which is chapter 02's whole subject.

Two numbers: the sum stores 0x3fd3333333333334 and 0.3 stores 0x3fd3333333333333. The bit patterns differ by exactly 1, one ulp, and the sample asserts that difference numerically.

1.5. the subnormal floor

Below 2^{-1022} the exponent field would be 0, and an implied leading 1 would have nowhere to live. The format spends the field differently: the value degenerates to $f \cdot 2^{-1074}$ with no implied 1, and the spacing holds at the floor instead of collapsing. This is gradual underflow, and its whole point is algebraic: with it, a subtraction rounds to zero only when the operands were equal. Flush to zero, the alternative still common in graphics code and on some gpus, breaks that: every subnormal becomes +0, distinct tiny values cancel into a fake zero, and the interval below 2^{-1022} turns into a cliff.

The dry run: the smallest subnormal, 5e-324, halved, lands exactly halfway between 0 and itself. The tie goes to the even significand 0, so the value drains to +0, and the hardware raises `FE_UNDERFLOW` and `FE_INEXACT`. The sample asserts the all zero bits and both flags.

```
/* the round trip betrays the source text: the stored value is 2^53, so
 * the cast reads 9007199254740992, not the 9007199254740993 written */
CHECK((uint64_t)(v53 + 1.0) == 9007199254740992ull,
      "the integer cast exposes the vanished increment");

/* cppreference nextafter (fetched 2026-09-21): a finite from with an
 * infinite result "raises FE_INEXACT and FE_OVERFLOW"; probed on this
 * machine the same day, both flags are really set */
feclearexcept(FE_ALL_EXCEPT);
CHECK(nextafter(DBL_MAX, INFINITY) == INFINITY,
      "nextafter(DBL_MAX, inf) is inf: the last step leaves the type");
CHECK(fetestexcept(FE_OVERFLOW) != 0,
      "FE_OVERFLOW raised by the walk past DBL_MAX");
CHECK(fetestexcept(FE_INEXACT) != 0, "FE_INEXACT raised by the same step");

/* downward from the min normal: all 52 fraction bits set, still
 * exponent field 0, so the classifier says subnormal */
double maxsub = nextafter(0x1p-1022, 0.0);
CHECK(bits_of(maxsub) == 0x000fffffffffffffull,
      "nextafter down from 2^-1022 lands on the max subnormal");
CHECK(fpclassify(maxsub) == FP_SUBNORMAL,
      "and the classifier calls it subnormal");

/* halving the min subnormal lands exactly halfway between 0 and itself:
 * the tie goes to the even significand 0, so gradual underflow drains
 * to +0 (probed 2026-09-21: bits 0, FE_UNDERFLOW and FE_INEXACT set) */
volatile double tiny = 0x1p-1074;
feclearexcept(FE_ALL_EXCEPT);
double halved = tiny / 2.0;
CHECK(bits_of(halved) == 0,
      "2^-1074 / 2 underflows to +0, never to a wrong small value");
CHECK(fetestexcept(FE_UNDERFLOW) != 0,
      "FE_UNDERFLOW raised by the drain to zero");
CHECK(fetestexcept(FE_INEXACT) != 0, "FE_INEXACT raised by the same drain");
```

listing 6: round trip betrayal, the overflow walk, the underflow drain (source: math/samples/src/Ch01/frontier.c, lines 83-116)

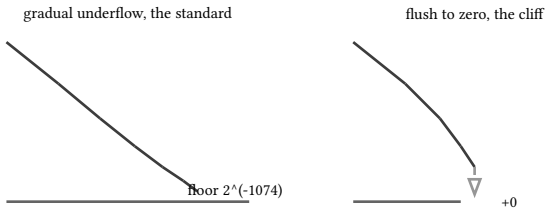


figure 4: gradual underflow drains smoothly, flush to zero cliffs

Two numbers: 5e-324 halved drains to +0 with `FE_UNDERFLOW` raised, while `nextafter(DBL_MAX, INFINITY)` jumps to ∞ with `FE_OVERFLOW` raised. The two edges of the type fail in opposite directions, and both leave a status flag as the only evidence that anything happened.

1.6. what math.h promises

The arithmetic operators, `sqrt`, and `fma` carry the strongest contract in the library: compute the true result exactly, round it once. cppreference, fetched 2026-09-21: "`sqrt` is required by the IEEE standard to be correctly rounded from the infinitely precise result. … The only other operations which require this are the arithmetic operators and the function `fma`. Other functions, including `pow`, are not so constrained." That sentence is the taxonomy of the whole header: one small required group, and everything else free to land an ulp or two off today and a different ulp or two after the next toolchain update.

```
/* the subnormal floor: exponent field 0, value f * 2^-1074, no hidden bit */
CHECK(bits_of(0x1p-1074) == 1ull,
      "the min subnormal is one unit of 2^-1074: bits 0x0000000000000001");
CHECK(fpclassify(0x1p-1074) == FP_SUBNORMAL,
      "the classifier calls 2^-1074 subnormal");
/* the min normal: exponent field 1, value 1.0 * 2^(1-1023) = 2^-1022 */
CHECK(bits_of(0x1p-1022) == 0x0010000000000000ull &&
          fpclassify(0x1p-1022) == FP_NORMAL,
      "2^-1022 is the min normal, exponent field 1, fraction empty");

/* specials: exponent field 0x7ff, the fraction picks infinity or nan */
CHECK(fpclassify(INFINITY) == FP_INFINITE &&
          bits_of(INFINITY) == 0x7ff0000000000000ull,
      "infinity is the all-one exponent with an empty fraction");
/* probed 2026-09-21: this toolchain prints the quiet nan payload
 * 0x8000000000000, bits 0x7ff8000000000000 */
CHECK(fpclassify(NAN) == FP_NAN && isnan(NAN) &&
          bits_of(NAN) == 0x7ff8000000000000ull,
      "nan is the all-one exponent with a nonzero fraction");

CHECK(fpclassify(0.0) == FP_ZERO,
      "zero is its own class despite the zero exponent");

/* DBL_MAX = (2 - 2^-52) * 2^1023: e=0x7fe=2046, every fraction bit set */
CHECK(bits_of(DBL_MAX) == 0x7fefffffffffffull &&
          expo_of(DBL_MAX) == 0x7feu &&
          frac_of(DBL_MAX) == 0xfffffffffffffull,
      "DBL_MAX is 0x7fefffffffffff, one ulp below the infinity pattern");

CHECK(FLT_EVAL_METHOD == 0,
      "FLT_EVAL_METHOD is 0: sse2 computes in binary64, no excess precision");

/* cppreference sqrt (fetched 2026-09-21): "sqrt is required by the IEEE
 * standard to be correctly rounded from the infinitely precise result."
 * squaring the rounded root is a fresh inexact operation: the exact
 * product needs more than 53 bits, lands one ulp high, and raises
 * FE_INEXACT (probe.py and fix-probe.c, exact, 2026-09-21) */
volatile double two = 2.0;
double root = sqrt(two);
CHECK(root == 0x1.6a09e667f3bcdp+0,
      "sqrt(2) stores the correctly rounded root 1.4142135623730951");
feclearexcept(FE_ALL_EXCEPT);
CHECK(root * root == 2.0000000000000004,
      "squaring it back lands one ulp high at 2.0000000000000004");
CHECK(fetestexcept(FE_INEXACT) != 0,
      "the square raised FE_INEXACT, exact product over 53 bits");
```

listing 7: classifiers, specials, boundary constants, and the rounded square (source: math/samples/src/Ch01/anatomy.c, lines 93-138)

Goldberg states the stakes for the required group: “The IEEE standard requires that the result of addition, subtraction, multiplication and division be exactly rounded.” `pow` makes no such promise, and nothing in the standard objects when its error moves between compiler versions.

The floating point environment is the observability layer. Every operation records what it did in status flags: cppreference pins `nextafter`’s contract, a finite `from` with an infinite result “raises FE_INEXACT and FE_OVERFLOW”, and the sample watches both flags set on the walk past `DBL_MAX`. Goldberg adds a mode dependent edge: “when round toward 0 or round toward $-\infty$ is in effect, an overflow of positive magnitude causes the default result to be the largest representable number, not ∞.” Toward zero, overflow saturates instead of escaping to infinity.

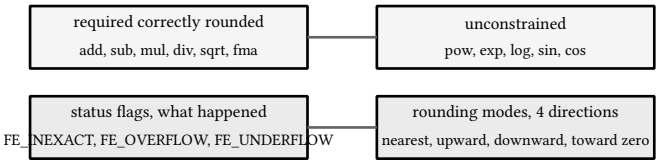


figure 5: what math.h promises, in 4 boxes

NOTE: FE_INEXACT fires on nearly everything

`FE_INEXACT` means the exact result did not fit, which is the ordinary case, not a failure. Treating it as an error indicator poisons every check built on it. The usable signals are the extreme pair, `FE_OVERFLOW` and `FE_UNDERFLOW`, and the clean rule that a correctly rounded operation sets `FE_INEXACT` exactly when it had to round.

Two numbers: `sqrt(2)` stores 1.4142135623730951, the correctly rounded true root, and squaring it back gives 2.0000000000000004, one ulp high. A required rounded operation composed with itself is still not an inverse, and this time the hardware says so: the square raised `FE_INEXACT`, and the sample asserts root, square, and flag together.

1.7. the 2^53 frontier

Doubles count integers exactly up to 2^{53}: below the wall the 52 stored fraction bits plus the implied 1 cover every integer bit. At 2^{53} the spacing becomes 2. One more than the wall is the exact midpoint between 2^{53} and $2^{53}+2$, ties to even keeps 2^{53}, and the increment vanishes.

The dry run: take 2^{53}, add 1, compare with the original, equal. Add 2, compare, different, one visible ulp. Add 3, it lands on $2^{53}+4$, the tie between +2 and +4 goes to the even tail. Then subtract the original back out of a nearby sum and it comes back exact, which is Sterbenz's lemma: when $y/2 \le x \le 2y$, the difference $x - y$ is representable, so nearby subtraction never rounds.

```
/* 2^53 = 0x4340000000000000: biased e=0x434=1076, 2^(1076-1023) = 2^53,
 * empty fraction. from here on the ulp is 2, so 2^53 + 1 is not
 * representable: it is the exact midpoint of 2^53 (even tail) and
 * 2^53 + 2, and ties-to-even keeps 2^53 (probe.py) */
constexpr double two53 = 0x1p53;
volatile double v53 = 0x1p53;
CHECK(bits_of(two53) == 0x4340000000000000ull,
      "2^53 stores as 0x4340000000000000");
CHECK(v53 + 1.0 == two53,
      "2^53 + 1, the exact midpoint, rounds back to 2^53");
CHECK(v53 + 2.0 == 0x1.0000000000001p53,
      "2^53 + 2 is exactly representable, one ulp up");
CHECK(v53 + 3.0 == 0x1.0000000000002p53,
      "2^53 + 3 ties between +2 and +4, the even tail 2 wins: rounds to +4");
CHECK(bits_of(v53 - 1.0) == 0x433fffffffffffull,
      "2^53 - 1 is exact, bits 0x433fffffffffff");

/* below the wall every integer lands exactly; at it, +1 vanishes while
 * +2 shows, and the nearby subtraction back out is exact (sterbenz) */
volatile double x = two53;
CHECK(x + 1.0 == x, "2^53 + 1 == 2^53: the increment vanished");
CHECK(x + 2.0 != x, "2^53 + 2 != 2^53: one visible ulp");
CHECK((x + 2.0) - x == 2.0, "sterbenz: subtracting nearby values is exact");
CHECK(nextafter(two53, INFINITY) == 0x1.0000000000001p53,
      "nextafter steps 2^53 by one ulp, straight to 2^53 + 2");
```

listing 8: the wall, the vanished increment, and sterbenz (source: math/samples/src/Ch01/frontier.c, lines 33-57)

The round trip betrays the source text. The stored value is 2^{53}, so the integer cast in section 4's listing reads 9007199254740992, not the 9007199254740993 written. The cast is where the lie becomes visible, and the integer view of the same bit patterns is book 1, ch 5 (c23: the language, the machine, and the cloud, the abstract machine and undefined behavior).

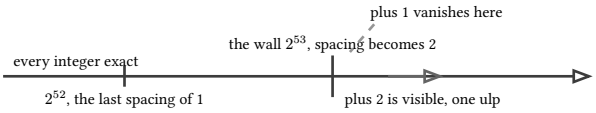


figure 6: the integer wall at 2^53

Two numbers: $2^{53} + 1$ is invisible, equal to 2^{53}, while $2^{53} + 2$ is the first integer past the wall. A counter that steps by 1 past the wall stops advancing, and a loop condition like `d < d + 1.0` is false on its first test.

Every later chapter lives downstream of these bits. The error chapter turns the ulp counting of section 2 and the tie rules of section 3 into tolerance discipline, forward to book 2, ch 2 (mathematics for programmers, error, conditioning, and stability).

sources: David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic", ACM Computing Surveys 23-1, docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html, fetched 2026-09-21. cppreference c pages fenv, FE_round, nextafter, math functions, sqrt, and arithmetic_types, en.cppreference.com, fetched 2026-09-21. Clang users manual, floating point behavior section, clang.llvm.org, fetched 2026-09-21. Probed on this machine the same day. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch01`, 63 checks in chapter 01 of the math suite.

2. error, conditioning, and stability

Floating point rounds every operation to a fixed number of significant bits, and this chapter is about what that rounding does to a computation you meant to perform exactly. Six moves: the machine-number grid of ulp and unit roundoff, error propagation and the condition number of a subtraction, the quadratic formula as the standard cancellation example, the line between an ill-conditioned problem and an unstable algorithm, compensated summation that buys back lost digits, and a precision ladder that measures what more significand bits pay. Every behavioral claim below is one of the 55 checks in the 4 samples of chapter 2, a sentence quoted from Goldberg 1991 or cppreference fetched 2026-09-21, or a toolchain fact probed on this machine the same day. Where the numerical chapter of the dsa book treats iteration as a contest driver (book 9, ch 36 (practical data structures and algorithms, numerical methods)), this chapter asks why the same driver in different arithmetic returns different answers.

2.1. machine numbers, ulp, and unit roundoff

IEEE 754 binary64 stores a number as a sign, an exponent, and a $p = 53$ bit significand. Every real input x becomes the nearest representable $\mathrm{fl}(x)$, and under round-to-nearest with ties to even the relative error of that storage obeys $|x - \mathrm{fl}(x)| \le u \cdot |x|$ with unit roundoff $u = 2^{-53}$. Machine epsilon is a different object: $\mathrm{DBL_EPSILON} = 2^{-52}$ is the gap between 1 and the next float, while the unit roundoff is half of it. The gap at a general x is the ulp, and it is not constant: the grid doubles at every power of two. The nextafter checks in propagate.c pin the three spacings this section needs, 2^{-52} at 1, 2^{-51} at 2, and 2^{-54} at 0.25, against integers, where representation is exact and the contrast with the fixed-width world of book 1, ch 5 (c23: the language, the machine, and the cloud, the abstract machine and undefined behavior) is sharpest.

Decimal literals meet this grid immediately, and storing a literal is the first rounded operation. 0.1 and 0.2 become 0x1.999999999999ap-4 and 0x1.999999999999ap-3, the same significand at adjacent exponents, so they carry the same relative input error.

The dry run: multiply and divide the stored pair by hand.

1. The exact product of the two stored significands needs 106 bits and rounds once, to 0x1.47ae147ae147cp-6.
2. The literal 0.02 stores one grid step lower, at 0x1.47ae147ae147bp-6, so the computed product sits exactly 1 ulp above the stored constant: relative error 1.7347e-16, which is 1.5625 units of u.
3. The quotient is exactly 0.5. Division is correctly rounded, and the identical significands cancel: the same mantissa at adjacent exponents divides to exactly 2^{-1}, with zero error from any source.

That is the two-number play of this section: the same two inputs, one multiplication that loses 1.56 units of u and one division that loses nothing, because error propagation is per-operation, not per-magnitude.

```c
// representation: 0.1 and 0.2 share the same significand, so they carry
// the same relative input error. fl(0.1) = 0x1.999999999999ap-4,
// fl(0.2) = 0x1.999999999999ap-3 (exact.py section A).
// their product rounds to 1 ulp above fl(0.02):
//   fl(0.1*0.2) = 0x1.47ae147ae147cp-6, fl(0.02) = 0x1.47ae147ae147bp-6
CHECK(0.1 * 0.2 == 0x1.47ae147ae147cp-6, "fl(0.1*0.2) pin");
CHECK(0.02 == 0x1.47ae147ae147bp-6, "fl(0.02) pin, the gap is exactly 1 ulp");

// relative error of the product against the 0.02 literal: exact.py gives
// 1.7347234759768070583e-16 = 1.5625 units of 2^-53
double relprod = (0.1 * 0.2 - 0.02) / 0.02;
CHECK(relprod > 1.73e-16 && relprod < 1.74e-16, "product rel err 1.7347e-16");

// division is correctly rounded and the identical input errors cancel:
// fl(0.1)/fl(0.2) = 0.5 with zero error (exact.py section A)
CHECK(0.1 / 0.2 == 0.5, "fl(0.1/0.2) is exactly 0.5");
```

listing 9: propagate.c, representation error born and moved, one check per pin (source: math/samples/src/Ch02/propagate.c, lines 23-38)



figure 7: ulp grid: tick density halves every binade, spacing doubles at each power of two

2.2. propagation and the condition of a subtraction

A problem's condition number is the worst-case ratio of relative output error to relative input error. For the subtraction $x - y$ with $x \approx .y$ it is $|x|/|x - y|$, and it is unbounded: subtracting nearly equal numbers can inflate a tiny input error without performing any new rounding at all. The companion fact is Sterbenz's lemma: when $1/2 \leq y/x \leq 2$, the difference $x - y$ is computed exactly. Exact arithmetic plus a huge condition number is precisely the trap.

The dry run: x = fl(1.0000001) and the subtraction x - 1.0.

1. Storing 1.0000001 rounds to 0x1.000001ad7f29bp+0 with relative error 5.84e-17, about half a unit of u. The 1e-7 part is simply not all there.
2. x - 1.0 is exact by Sterbenz: both operands are dyadic and within a factor of two. The result is 0x1.ad7f29bp-24, and the trailing zero hex digits are the visible tell of an exact result.
3. Measured against the intended 1e-7, that exact difference is off by 5.84e-10 relative. The amplification is 1.0e7, which is $|x|/|x - 1|$, the condition number, to the digit.

Nothing rounded in the subtraction. The input error was already there, and the condition number moved it to the output. The sum $\{10^{16}, 1, -10^{16}\}$ shows the same law on a bigger axis: the exact answer is 1, the condition number is $2 \cdot 10^{16} + 1$, and ulp at 10^{16} is 2, so both association orders return exactly 0.0. Reassociating cannot rescue a conditioned problem.

```
// sterbenz: for 0.5 ≤ y/x ≤ 2 the difference x - y is exact. here
// x = fl(1.0000001) = 0x1.000001ad7f29bp+0, and x - 1 lands exactly on
// 0x1.ad7f29bp-24; the trailing zero hex digits are the exactness tell.
double x = 1.0000001;
double d = x - 1.0;
CHECK(d == 0x1.ad7f29bp-24, "x - 1 exact by sterbenz");

// but exact subtraction of inexact operands only exposes the input error:
// rel err of d vs 1e-7 is 5.838671768287895e-10 while x itself was stored
// with rel err 5.838671184420777e-17. amplification 1.0e7 = |x|/|x-1|,
// the condition number of the subtraction.
double relsub = (d - 1e-7) / 1e-7;
CHECK(relsub > 5.83e-10 && relsub < 5.85e-10,
      "subtraction amplifies to 5.84e-10");
double kappa = x / 1e-7;
CHECK(kappa > 9.9e6 && kappa < 1.01e7, "condition |x|/|x-1| ~ 1e7");

// the sum {1e16, 1, -1e16} has exact answer 1 and condition 2e16+1: ulp
// at 1e16 is 2, so the lone 1 dies in either association order (exact.py)
CHECK((1.0e16 + 1.0) - 1.0e16 == 0.0, "left-to-right loses the 1");
CHECK(1.0e16 + (1.0 - 1.0e16) == 0.0, "regrouped still loses the 1");
```

listing 10: propagate.c, sterbenz exactness, condition-number amplification, and the sum no ordering can save (source: math/samples/src/Ch02/propagate.c, lines 40-60)

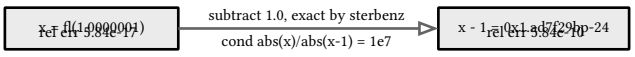


figure 8: the subtraction is exact, the condition number carries the input error out

2.3. the quadratic formula: cancellation in the wild

For $ax^2 + bx + c$ the textbook solution $x = \left(-b \pm \sqrt{b^2 - 4ac}\right)/(2a)$ hides two separate subtractions: the discriminant $b^2 - 4ac$ and, when $b < 0$, the small root's $-b - \sqrt{d}$. The stable rewrite forms $q = \left(-b - \sqrt{d}\right)/2$ for the sign of b that avoids cancellation, then returns the roots as q/a and c/q, trading one cancellation for two divisions.

Goldberg's coefficients a = 1.22, b = −3.34, c = 2.28 make the discriminant the crime scene. The dry run: trace the discriminant in float.

1. On the stored coefficients the exact products are $b^2 = 11.155599427$ and $4ac = 11.126400121$, pinned as doubles.
2. Each rounds to the 24-bit grid with its own error, in opposite directions: fl32(b*b) = 11.155599594 is 1.7e-7 high and fl32(4ac) = 11.126399994 is 1.3e-7 low.
3. The subtraction itself is exact, and the discriminant inherits the difference: 0.0291996002 versus the true 0.0291993053, off by 158 ulps.
4. The roots barely notice: against the root of the stored coefficients the naive small root sits 3.27 ulps low and the q-form 2.27 low, one better despite its extra division. A root's absolute sensitivity to the discriminant is $1/\left(4a\sqrt{d}\right)$, which is small here. Ulps of the discriminant are not ulps of the answer.

```
// goldberg's triple in f32. stored coefficients:
//   fa = 0x1.3851ecp+0f, -fb = 0x1.ab851ep+1f, fc = 0x1.23d70ap+1f
// the exact products on the stored inputs are b*b = 11.155599427 and
// 4ac = 11.126400121; fl32 rounds them to 11.155599594 (up 1.7e-7) and
// 11.126399994 (down 1.3e-7), and the exact subtraction exposes the
// difference as a discriminant off by 158 ulps.
float fa = 1.22f, fb = -3.34f, fc = 2.28f;
float disc32 = fb * fb - 4.0f * fa * fc;
CHECK(disc32 == 0x1.de68p-6f, "fl32 discriminant pin");
double dda = fa, ddb = -fb, ddc = fc;
double ddisc = ddb * ddb - 4.0 * dda * ddc; // exact on stored inputs
CHECK(ddb * ddb == 0x1.64faaba617c20p+3, "exact b*b on stored f32 coeffs");
CHECK(4.0 * dda * ddc == 0x1.640b784467380p+3,
      "exact 4ac on stored f32 coeffs");
float ubase = (float)ddisc;
float fulp = nextafterf(ubase, INFINITY) - ubase;
double disc_ulps = fabs((double)disc32 - ddisc) / (double)fulp;
CHECK(disc_ulps > 100.0, "disc off by ~158 ulps");
```

listing 11: quadratic.c, goldberg triple in f32: stored pins, exact products, the 158-ulp disc (source: math/samples/src/Ch02/quadratic.c, lines 35-52)

```
// and yet both root formulas stay within 4 ulps of the root of the stored
// coefficients: naive small 3.27 ulps low, q-form small 2.27 low, one
// better despite its extra division. ulps of the discriminant are not
// ulps of the root.
float sq32 = sqrtf(disc32);
float nb = -fb;
float r1n = (nb + sq32) / (2.0f * fa);
float r2n = (nb - sq32) / (2.0f * fa);
float q32 = (nb + sq32) / 2.0f;
float r1s = q32 / fa;
float r2s = fc / q32;
CHECK(r2n == 0x1.4c7f78p+0f && r2s == 0x1.4c7f7ap+0f, "both small root pins");
double rt_small = (ddb - sqrt(ddisc)) / (2.0 * dda); // double reference
float rbase = (float)rt_small;
float rulp = nextafterf(rbase, INFINITY) - rbase;
double errn = fabs((double)r2n - rt_small) / (double)rulp;
double errs = fabs((double)r2s - rt_small) / (double)rulp;
CHECK(errn > 3.2 && errn < 3.3, "naive small root 3.27 ulps low");
CHECK(errs > 2.2 && errs < 2.3, "q-form small root 2.27 ulps, one better");
CHECK(r1n == r1s, "big roots identical, no cancellation there");
```

listing 12: quadratic.c, both root forms pinned, 3.27 and 2.27 ulps against the double reference (source: math/samples/src/Ch02/quadratic.c, lines 54-73)

PITFALL: cancellation is a symptom, not the disease

After his worked example Goldberg states it directly: “The subtraction did not introduce any error, but rather exposed the error introduced in the earlier multiplications.” The fix is never to avoid subtraction. It is to notice which earlier operation lost the bits the subtraction then reveals.

The near-equal-roots triple (1, −1e9, 1) is where the formula choice decides everything. Its roots are 10^9 and 10^{-9}, and the discriminant is $10^{18} - 4$: ulp at 10^{18} is 128, the 4 dies in rounding, and sqrt(disc) comes back as exactly 10^9. The naive small root is then $(10^9 - 10^9)/2 = 0.0$, annihilated, while the q-form returns c/q with relative error 6.1e-17 against the true root, 6.2e-17 against the 1e-9 literal. Vieta’s formulas audit the damage in

one multiplication each: the naive pair multiplies to 0, the q-form pair to exactly 1. Two numbers carry the section: relative error 1.0 against 6.1e-17, same problem, same arithmetic, different algebra.

```
// near-equal roots (1, -1e9, 1): disc = 1e18 - 4, and ulp(1e18) = 128
// swallows the 4, so sqrt(disc) = 1e9 exactly. the naive small root is
// (1e9 - 1e9)/2 = 0 while the true root is 1e-9; the q-form keeps it.
double pa = 1.0, pb = -1.0e9, pc = 1.0;
double pdisc = pb * pb - 4.0 * pa * pc;
double psq = sqrt(pdisc);
double n_big = (-pb + psq) / (2.0 * pa);
double n_small = (-pb - psq) / (2.0 * pa);
double pq = (-pb + psq) / 2.0;
double s_big = pq / pa;
double s_small = pc / pq;
CHECK(n_small == 0.0, "naive small root annihilates to 0.0");
CHECK(s_small == 0x1.12e0be826d695p-30, "q-form small root: rel err 6.1e-17");
CHECK(n_big * n_small == 0.0 && s_big * s_small == 1.0,
      "vieta audit: products 0 vs 1");
```

listing 13: quadratic.c, the near-equal-roots triple: naive annihilation, q-form survival, vieta audit (source: math/samples/src/Ch02/quadratic.c, lines 75-89)

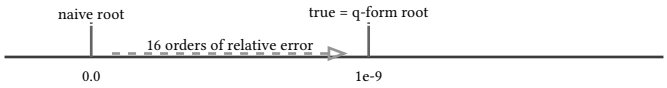


figure 9: small root of (1, −1e9, 1): the naive form returns 0.0, the q-form lands on the true 1e-9

Run the same goldberg triple in binary64 and the two formulas still agree bit for bit, now at 0x1.4c7f71ab5a24bp+0. The agreement is not accuracy: the computed discriminant carries a relative error of 3.3e-14 from its two rounded products, the square root halves it, and both root forms land 5.14 ulps (small) and 6.08 ulps (big) from the true roots of the stored coefficients. When the shared upstream error dominates each formula's own rounding, naive and stable coincide in the same wrong answer, and a bit-identical pair is no evidence of correctness.

```
// the same goldberg triple in f64 for contrast: both forms inherit the
// computed disc's 3.3e-14 relative error, halved by the sqrt, and agree
// bit-for-bit at 0x1.4c7f71ab5a24bp+0, 5.14 ulps below the true
// stored-coefficient root (the big root sits 6.08 above). shared
// upstream error dominates either formula's own rounding.
double dga = 1.22, dgb = -3.34, dgc = 2.28;
double ddd = dgb * dgb - 4.0 * dga * dgc;
double dsq = sqrt(ddd);
double dn2 = (-dgb - dsq) / (2.0 * dga);
double dq = (-dgb + dsq) / 2.0;
double ds2 = dgc / dq;
CHECK(dn2 == 0x1.4c7f71ab5a24bp+0 && ds2 == dn2,
      "f64: both forms coincide at the same 5.14-ulp error");
```

listing 14: quadratic.c, the f64 contrast: both forms coincide at the same 5.14-ulp error (source: math/samples/src/Ch02/quadratic.c, lines 108-120)

2.4. conditioning versus stability

Conditioning belongs to the problem: how much the exact answer moves when the inputs move. Stability belongs to the algorithm: how much avoidable error the method injects on top. The quadratic sections showed an unstable algorithm losing a well-conditioned root. Now the reverse corner: a problem no algorithm can save.

Take $a = 1$, $b = -(4 + 2^{-25})$, $c = 4 + 2^{-24}$. The polynomial factors as $(x - 2)(x - (2 + 2^{-25}))$: roots 2 and $2 + 2^{-25}$, a gap of 2^{-25}, and every coefficient is exactly representable. The exact discriminant is 2^{-50}. But $b^2 = 16 +$

$2^{-22} + 2^{-50}$ and $4ac = 16 + 2^{-22}$ both round to the same double, $16 + 2^{-22}$, because ulp at 16 is 2^{-48} and the 2^{-50} term is a quarter of it. So fl(disc) is exactly 0.0. The condition number of the discriminant, $|b^2|/|d|$, is 1.80e16: the discriminant of these stored coefficients genuinely is ambiguous at the double grid.

The dry run: feed fl(disc) = 0 to both formulas.

1. Naive: both roots come back $-b/2a = 2 + 2^{-26}$, the same double twice.
2. q-form: $q = 2 + 2^{-26}$, then q/a and the exact-quotient $c/q = 2 + 2/(2^{27} + 1)$, which rounds to $2 + 2^{-26}$ again.
3. Every root slot of both algorithms holds the double 0x1.0000002p+1, which is exactly the midpoint of the true pair: $(2 + (2 + 2^{-25}))/2$.

The information that separated the roots was never in the rounded coefficients, so no cleverer formula can recover it. Higham's remedy in Accuracy and Stability of Numerical Algorithms (2nd ed.) is to reformulate: compute the discriminant in wider arithmetic, or reparameterize the problem. Shopping for a stabler algorithm on an ill-conditioned input is a category error, and the two numbers of this section are its whole proof: condition $1.8 \cdot 10^{16}$, every computed root the identical midpoint.

```
// dyadic conditioning triple: a=1, b=-(4+2^-25), c=4+2^-24, true roots
// 2 and 2+2^-25 (gap 2^-25), exact disc = 2^-50. but b*b and 4ac both
// round to 16+2^-22 = 0x1.0000004p+4, so fl(disc) = 0 exactly.
// condition of the disc: |b^2|/|disc| = 1.8014399e16.
double ga = 1.0, gb = -(4.0 + 0x1p-25), gc = 4.0 + 0x1p-24;
double gdisc = gb * gb - 4.0 * ga * gc;
CHECK(gdisc == 0.0, "fl(disc) is exactly 0.0");
double gq = (-gb + sqrt(gdisc)) / 2.0;
double gn1 = (-gb + sqrt(gdisc)) / (2.0 * ga);
double gn2 = (-gb - sqrt(gdisc)) / (2.0 * ga);
double gs2 = gc / gq;
CHECK(gn1 == 0x1.0000002p+1 && gn2 == 0x1.0000002p+1, "naive pair collapses");
CHECK(gq == 0x1.0000002p+1 && gs2 == 0x1.0000002p+1,
      "q-form collapses to the same double");
CHECK(gn1 == (2.0 + 0x1.0000004p+1) / 2.0,
      "the shared answer is the exact midpoint");
```

listing 15: quadratic.c, the dyadic conditioning triple: both algorithms collapse to the shared midpoint (source: math/samples/src/Ch02/quadratic.c, lines 91-106)

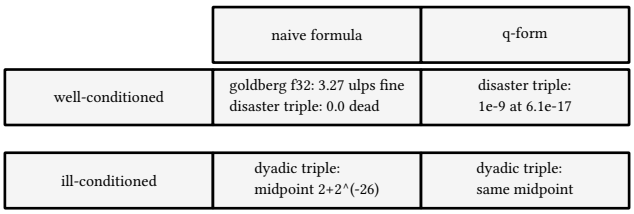


figure 10: conditioning versus stability: three measured triples place the quadrants

2.5. compensated summation

Summing many addends concatenates rounding error, and when the running sum outranges the addends, the small ones vanish entirely. Compensated summation carries a second variable that remembers what the last rounded addition dropped. Kahan's form feeds the compensation back into the next input, $y - c$, computes the rounded sum $t = s + y - c$, and updates $c = (t - s) - y$ computed exactly by Sterbenz. Neumaier's form instead accumulates the dropped parts separately and adds them once at the end.

The pattern for both lanes is one huge addend, then 999998 ones, then the negated huge addend: exact sum 999998. In float the lane is [1e8f, 1.0f, ..., −1e8f], in double [2^53, 1.0, ..., −2^53], chosen so the naive running sum cannot absorb a single 1.

The dry run: the f32 compensation dance, traced by the generator.

1. Naive: $10^8 + 1$ rounds back to 10^8 every time, since ulp at 10^8 is 8. After the final -10^8 the sum is exactly 0.
2. Kahan, first ones: after one add $c = -1$, the next uses $y - c = 2$, so c deepens to -2, then -3, -4.
3. At the fifth one the rounded sum itself ratchets to $10^8 + 8$ and the sign of c flips: 8 ones move the sum one grid step, and s tracks the true partial sum. At iteration 999999, $s = 101000000$ and $c = 2$.
4. Last step: $y - c$ rounds $-10^8 - 2$ to -10^8 (the 2 is under half of ulp 8), so the final sum lands on 1000000.0f. The exact answer 999998 plus the 2 the lane could not represent: 32 ulps off, measured, not guessed.

lane	naive	kahan	neumaier
f32 result	0.0f	1000000.0f	999998.0f
f32 error, ulps	15999968	32	0
f64 result	0.0	999998.0	999998.0
f64 error, ulps	8589917412130816	0	0

The two-number play: on the float lane naive loses 16.0 million ulps and compensation holds the damage to 32, on the double lane naive loses $8.6 \cdot 10^{15}$ ulps and both compensated forms are exact. Compensation cannot beat the grid, note the 32, but it removes the accumulation.

```
constexpr int nones = 999998;
float sn = 0.0f, sk = 0.0f, ck = 0.0f, sm = 0.0f, cm = 0.0f;
double dn = 0.0, dk = 0.0, dck = 0.0, dm = 0.0, dcm = 0.0;
float s6 = 0.0f, c6 = 0.0f, s13 = 0.0f, c13 = 0.0f, sfin = 0.0f, cfin = 0.0f;
for (int i = 0; i < nones + 2; i++) {
  float y = (i == 0) ? 1.0e8f : (i == nones + 1) ? -1.0e8f : 1.0f;
  double yd = (i == 0) ? 0x1p53 : (i == nones + 1) ? -0x1p53 : 1.0;
  float tf;
  double td;
  sn += y;           // naive: 1e8 + 1 rounds back to 1e8, forever
  float yy = y - ck; // kahan: the compensation dance
  tf = sk + yy;
  ck = (tf - sk) - yy;
  sk = tf;
  tf = sm + y; // neumaier: error into a second accumulator
  if (fabsf(sm) ≥ fabsf(y))
    cm += (sm - tf) + y;
  else
    cm += (y - tf) + sm;
  sm = tf;
  dn += yd; // the same three machines on the f64 lane
  double yyd = yd - dck;
  td = dk + yyd;
  dck = (td - dk) - yyd;
  dk = td;
  td = dm + yd;
  if (fabs(dm) ≥ fabs(yd))
    dcm += (dm - td) + yd;
  else
    dcm += (yd - td) + dm;
  dm = td;
  // dance checkpoints for the narrated trace (element index i, 0-based)
  if (i == 5) {
    s6 = sk;
    c6 = ck;
  }
  if (i == 12) {
    s13 = sk;
    c13 = ck;
  }
  if (i == nones) { // state entering the final element
    sfin = sk;
    cfin = ck;
  }
}
```

listing 16: kahan.c, three summation machines interleaved on both lanes, one loop with dance checkpoints (source: math/samples/src/Ch02/kahan.c, lines 26-70)

```
// f32 lane: naive 0, kahan 1000000.0f = 0x1.e848p+19f, neumaier exact
CHECK(sn == 0.0f, "naive f32 sum is exactly 0");
CHECK(sk == 0x1.e848p+19f, "kahan f32 lands on 1000000.0f");
CHECK(sm + cm == 999998.0f, "neumaier f32 is exact");
// the narrated dance trace, pinned: s ratchets one grid step while c
// cycles, and the state entering the final element carries c = 2, the
// two units the lane cannot represent
CHECK(s6 == 100000008.0f && c6 == 3.0f,
      "dance element 6: s ratchets, c flips");
CHECK(s13 == 100000016.0f && c13 == 4.0f, "dance element 13: next ratchet");
CHECK(sfin == 101000000.0f && cfin == 2.0f, "entering the final element");

float exact32 = 999998.0f;
float ulp32 = nextafterf(exact32, INFINITY) - exact32; // 2^-4
CHECK((double)(exact32 - sn) / (double)ulp32 == 15999968.0,
      "naive f32 misses by 15999968 ulps");
CHECK((double)fabsf(exact32 - sk) / (double)ulp32 == 32.0,
      "kahan f32 misses by exactly 32 ulps");

// f64 lane: naive 0, kahan and neumaier exact
CHECK(dn == 0.0, "naive f64 sum is exactly 0");
CHECK(dk == 999998.0, "kahan f64 is exact");
CHECK(dm + dcm == 999998.0, "neumaier f64 is exact");

double exact64 = 999998.0;
double ulp64 = nextafter(exact64, INFINITY) - exact64; // 2^-33
CHECK((exact64 - dn) / ulp64 == 8589917412130816.0,
      "naive f64 misses by 8.59e15 ulps");
```

listing 17: kahan.c, the pinned sums, the pinned dance states, and the ulp-scored errors (source: math/samples/src/Ch02/kahan.c, lines 72-99)

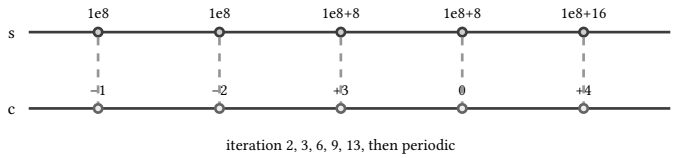


figure 11: the dance: s ratchets one grid step per 8 ones while c cycles, trace from the generator

2.6. precision ladders and what more bits buy

One computation, several rungs: newton for $\sqrt{2}$ from 1.5 with $x \leftarrow x - (x^2 - 2)/(2x)$, every operation rounded to the rung's precision, 20 updates. The ladder on this machine has two real rungs: float with 24 significand bits and double with 53. Long double is binary64 under the MSVC ABI, pinned by LDBL_MANT_DIG == 53, so that rung is a relabeling. The bottom rung has to be emulated: this lane's clang compiles `_Float16` but cannot link it, the conversion builtins `__extendhfsf2` and `__truncsfhf2` are undefined, probed 2026-09-21. The sample therefore rounds each double result to an 11-bit significand, which is how the hardware emulation would store it anyway.

WARNING: float16 compiles here but does not link

On this toolchain a `_Float16` program passes the frontend (clang documents x86 SSE2 support via float emulation) and then fails at link time on the missing conversion builtins. The 11-bit rung in precision.c is a frexp-and-ldexp rounder instead. Probe before you ship: the ladder you can compile is not always the ladder you can run.

The dry run: the double rung's first six updates.

1. From 1.5 the updates read 1.4166666666666667, 1.4142156862745099, 1.4142135623746899: quadratic convergence, the same ladder the dsa contest chapters pin.
2. Update 4 lands on 0x1.6a09e667f3bcdp+0, the double $\sqrt{2}$.
3. Update 5 steps one ulp down to 0x1.6a09e667f3bccp+0 and update 6 returns: from here the iterate oscillates between the two, pinned by checks at updates 4, 5, and 6.

Each rung stalls where its own grid locks the update map into a fixed point:

1. 11 bits: the iterate freezes at 1.4140625 after 2 updates, relative error 1.068e-4. The correction is under half an ulp and dies in rounding.
2. 24 bits: the iterate reaches 0x1.6a09e6p+0f in 3 updates and the rounded update maps that value to itself, verified by a 21st pass. Relative error 1.711e-8, which is the pure representation error of storing $\sqrt{2}$ in float.
3. 53 bits: the dry run above, oscillating between the stored value and one ulp below, pinned on the stored value at update 20. Relative error 6.84e-17, again the representation error of the rung.

rung	bits	stall value	relative error
emulated f16	11	1.4140625	1.068e-4
float	24	1.4142135	1.711e-8
double	53	1.4142135623730951	6.84e-17
long double	53	identical to double	6.84e-17

The two-number play: 11 bits to 24 bits is a factor 6200 in final error, 24 to 53 another 250000, and the driver is the same newton the dsa contest chapters tune (book 9, ch 36 (practical data structures and algorithms, numerical methods)). More bits do not make a better algorithm, they move the fixed point closer to the real number.

The last measurement is Dekker's fast two-sum on $10^{16} + 1$. The sum itself is an exact tie between 10^{16} and $10^{16} + 2$, and ties go to even: 10^{16} has an even significand, so the 1 dies in the rounded sum. Two-sum recovers it exactly, $z = a - s$ then $e = z + b$, both exact by Sterbenz, and the pair (s, e) satisfies $s + e = a + b$ over the reals while the single double $s + e$ still rounds back to s. When the error term matters it has to stay a separate variable, which is the whole design of the compensated summation above and of the autovectorization hazards in book 1, ch 9 (c23: the language, the machine, and the cloud, optimization: what -O actually does): a compiler that reassociates $s + e$ across your two variables quietly deletes the compensation.

```c
static double ladder16(int iters) {
  double x = r11(1.5);
  for (int i = 0; i < iters; i++) {
    double num = r11(r11(x * x) - 2.0);
    double den = r11(2.0 * x);
    x = r11(x - r11(num / den));
  }
  return x;
}

static float ladder32(int iters) {
  float x = 1.5f;
  for (int i = 0; i < iters; i++)
    x = x - (x * x - 2.0f) / (2.0f * x);
  return x;
}

static double ladder64(int iters) {
  double x = 1.5;
  for (int i = 0; i < iters; i++)
    x = x - (x * x - 2.0) / (2.0 * x);
  return x;
}

static long double ladder80(int iters) {
  long double x = 1.5L;
  for (int i = 0; i < iters; i++)
    x = x - (x * x - 2.0L) / (2.0L * x);
  return x;
}
```

listing 18: precision.c, the four ladder rungs, one update map, four roundings (source: math/samples/src/Ch02/precision.c, lines 42-71)

```c
  // dekker's fast two-sum on 1e16 + 1: the sum itself hits a tie that
  // rounds to even (1e16 has an even significand), so the 1 vanishes, and
  // the two-sum error term carries it back out exactly.
  double big = 1.0e16, one = 1.0;
  double s = big + one;
  CHECK(s == 1.0e16, "fl(1e16+1): the tie rounds to even, the 1 dies");
  double err = (big - s) + one; // |big| ≥ |one|: fast two-sum
  CHECK(err == 1.0, "two-sum recovers the lost 1 exactly");
  CHECK(s + err == s, "the pair still cannot share one double");
```

listing 19: precision.c, dekker fast two-sum: the lost 1 recovered exactly, and why it must stay separate (source: math/samples/src/Ch02/precision.c, lines 111-119)

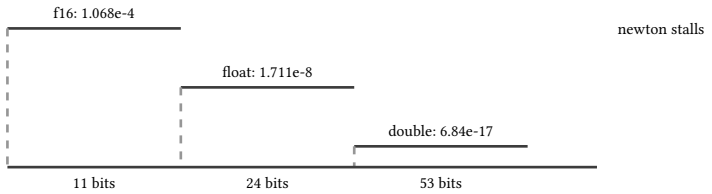


figure 12: precision ladder: each rung's stall error, three decades apart per rung

The next chapter, book 2, ch 3 (mathematics for programmers, trigonometry), takes these arithmetic rules onto the unit circle, where range reduction is the conditioning question and the identities are the stable reformulations.

sources: David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic", ACM Computing Surveys 23(1), 1991, cancellation example and quoted sentence from its cancellation section, oracle

text fetched 2026-09-21. Nicholas J. Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed, SIAM 2002, cited by name and edition for the conditioning-versus-stability remedy, no page text quoted. cppreference fenv, en.cppreference.com/w/c/numeric/fenv, fetched 2026-09-21. clang LanguageExtensions, `_Float16` on x86, clang.llvm.org/docs/LanguageExtensions.html, fetched 2026-09-21. `FLT_`, `DBL_`, `LDBL_` constants, the `_Float16` link failure, and long double as binary64 probed on this machine 2026-09-21, playground/math-ch02. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch02`, 55 checks in chapter 2 of the math suite.

3. trigonometry

Trigonometry enters programs as the pair of coordinates on a circle, and this chapter builds it in six moves: the unit circle and radians with the quadrant sign map, the identity kit checked to tight ulp tolerances, atan2 as the quadrant-honest inverse, Cody-Waite range reduction feeding a polynomial sin and cos measured against the C runtime, the laws of sines and cosines with the ambiguous SSA case, and the large-argument trap where argument reduction dominates accuracy. Every behavioral claim below is one of the 62 checks in the 3 samples of chapter 03 or a sentence quoted from a canonical source fetched 2026-09-21. The algorithm-first treatment of circles and angles on contest boards stays in book 9, ch 37 (practical data structures and algorithms, computational geometry ii), this chapter is the theory the algorithms stand on.

3.1. the unit circle and radians

Definition first. An angle in radians is the arc length it cuts on the unit circle, so a full turn is 2π, a half turn is π, and a right angle is $\pi/2$. The cosine and sine of that angle are the x and y coordinates of the point the arc ends on. That single sentence is the whole geometric content, and the mml-book says it through projections: “If ||x|| = 1, then x lies on the unit circle. It follows that the projection onto the horizontal axis spanned by b is exactly cos omega” (draft 2024-01-15, sec 3.8, eq 3.44, printed p 84). The angle between two vectors lives in $[0, \pi]$ because the normalized dot product (the componentwise sum chapter book 2, ch 4 (mathematics for programmers, coordinate geometry) writes $\boldsymbol{u}^T\boldsymbol{v}$) sits in $[-1, 1]$ (sec 3.4, eqs 3.24-3.25, printed p 76).

The dry run: walk four landmark angles around the circle and read the coordinates as sign plus magnitude. The values are pinned in `identities.c` and were produced by mpmath at 200 bits on the exact double inputs.

1. At $x = 0.7$ (about 40.1 degrees) the point is $(0.7648421872844885, 0.64421768723769102)$, both coordinates positive, quadrant 1.
2. At $x = 2.3$ (about 131.8 degrees) the point is $(-0.66627602127982399, 0.74570521217672026)$, cosine negative, sine positive, quadrant 2. The runtime value pins one ulp off the correctly rounded -0.6662760212798241.
3. At $x = 4.0$ (about 229.2 degrees) both go negative, $(-0.65364362086361194, -0.7568024953079282)$, quadrant 3.
4. At $x = 5.5$ (about 315.1 degrees) cosine recovers first, $(0.70866977429125999, -0.70554032557039192)$, quadrant 4.

angle rad	degrees	(cos, sin)	quadrant
0.7	40.1	$(0.765, 0.644)$	1: +, +
2.3	131.8	$(-0.666, 0.746)$	2: -, +
4.0	229.2	$(-0.654, -0.757)$	3: -, -
5.5	315.1	$(0.709, -0.706)$	4: +, -

Two numbers carry the section. The double nearest $\pi/6$ still has `sin(PI06) == 0.5` exactly, because the true sine of that double rounds to 0.5. The double nearest $\pi/2$ has `cos(PI02)` equal to 6.123233995736766e-17, not 0, because $\pi/2$ is irrational and the coordinate lands one ulp scale off the axis. The circle is continuous, binary64 is not, and every axis value below inherits that residue.

```
  // radians and the axis values. sin(pi/6_d) rounds to exactly 0.5;
  // cos(pi/3_d) lands one ulp off 0.5 (true 0.49999999999999989, and
  // one ulp of 0.5 is 1.11e-16).
  CHECK(sin(PI06) == 0.5, "sin(pi/6) is 0.5 exactly");
  CHECK(fabs(cos(PI03) - 0.5) ≤ 1.2e-16, "cos(pi/3) within 1 ulp of 0.5");
  // pi/2 itself is not representable: cos(pi/2_d) is 6.123233995736766e-17,
  // not 0, while sin(pi/2_d) still rounds to exactly 1.
  CHECK(sin(PI02) == 1.0, "sin(pi/2) rounds to 1");
  CHECK(cos(PI02) == 6.123233995736766e-17, "cos(pi/2) is the pi/2 residue");

  // the quadrant sign map on four landmark angles, values pinned:
  // 0.7 in q1, 2.3 in q2, 4.0 in q3, 5.5 in q4.
  static const double marks[4] = {0.7, 2.3, 4.0, 5.5};
  static const double want_s[4] = {0.64421768723769102, 0.74570521217672026,
                                   -0.7568024953079282, -0.70554032557039192};
  // cos(2.3) pins the runtime value -0.66627602127982399, one ulp off
  // the correctly rounded -0.6662760212798241 from mpmath.
  static const double want_c[4] = {0.7648421872844885, -0.66627602127982399,
                                   -0.65364362086361194, 0.70866977429125999};
  for (int i = 0; i < 4; i++) {
    unit2 p = circle(marks[i]);
    // signs are the point: sin>0 above the x axis, cos>0 right of the y axis.
    CHECK(p.y == want_s[i] && p.x == want_c[i], "quadrant walk lands the pair");
  }
```

listing 20: identities.c, the axis pins and the quadrant sign walk over the four landmarks (source: math/samples/src/Ch03/identities.c, lines 35-58)

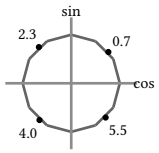


figure 13: the unit circle with the four landmark points at their pinned coordinates

3.2. the identity kit

The identities are the working set. Pythagorean: $\sin^2 x + \cos^2 x = 1$. Angle sum and difference: $\sin(a+b) = \sin a \cos b + \cos a \sin b$, $\cos(a+b) = \cos a \cos b - \sin a \sin b$. Double angle by setting $b = a$: $\sin 2a = 2 \sin a \cos a$ and three cosine forms, $\cos^2 a - \sin^2 a$, $2\cos^2 a - 1$, $1 - 2\sin^2 a$. Half angle by solving the second form: $\cos^2(a/2) = (1 + \cos a)/2$. All four families are one theorem deep, the angle sum, and DLMF sections 4.14, 4.17, and 4.21 carry the definitions, symmetries, and multiple-angle formulas.

The dry run: fix $a = 0.7$, $b = 0.4$, take `sin` and `cos` from the runtime, and evaluate both sides of each identity in the exact order the sample does.

1. Pythagorean over the four landmarks: the sum is 1.0 at 0.7 and 5.5, and 0.99999999999999978 at 2.3, one eps low, zero deviation elsewhere. Worst case 2.2204460492503131e-16.
2. Sum: `sin(1.1)` is 0.89120736006143542 against the expansion's 0.89120736006143531, gap 1.1102230246251565e-16. Difference: `sin(0.3)` matches `sa*cb - ca*sb` bit for bit.
3. Double: `sin(1.4)` against $2s_a c_a$, gap 1.1102230246251565e-16. The three cosine forms land at gaps 8.3e-17, 1.4e-16, 2.8e-17 from `cos(1.4)`, so the $1 - 2\sin^2 a$ form is the closest here.
4. Half: `cos(0.35)^2` against $(1 + c_a)/2$, gap 1.1102230246251565e-16.

The two-number play: at this one angle the three algebraically identical cosine forms differ from each other by up to 1.4e-16 and the winner is the one that subtracts nothing, $1 - 2\sin^2 a$ at 2.8e-17. An identity is exact in real arithmetic and a choice among rounding patterns in floating point, so pick the form whose inputs you already hold.

```
// pythagorean identity: at 2.3 the sum drifts to 1 - 1 ulp, elsewhere 0.
double worst_pyth = 0.0;
for (int i = 0; i < 4; i++) {
  unit2 p = circle(marks[i]);
  double v = p.y * p.y + p.x * p.x;
  if (fabs(v - 1.0) > worst_pyth)
    worst_pyth = fabs(v - 1.0);
}
// pinned: max deviation 2.2204460492503131e-16 (exactly 1 eps) at x=2.3.
CHECK(worst_pyth ≤ 2.2205e-16, "sin^2+cos^2 within 1 eps of 1");

// angle sum and difference at a=0.7, b=0.4. pinned deviations:
// sum 1.1102230246251565e-16, difference 0 (bit-identical), and the
// expansion is computed in the same order as the generator.
double sa = sin(0.7), ca = cos(0.7), sb = sin(0.4), cb = cos(0.4);
CHECK(fabs(sin(0.7 + 0.4) - (sa * cb + ca * sb)) ≤ 1.2e-16,
      "sin(a+b) matches the sum formula");
CHECK(sin(0.7 - 0.4) == sa * cb - ca * sb, "sin(a-b) matches bit for bit");

// double angle: sin 2a off by 1.1e-16; the three cos 2a forms differ,
// pinned per form 8.3e-17, 1.4e-16, 2.8e-17: the 1-2 sin^2 form wins
// here because it cancels nothing.
CHECK(fabs(sin(2.0 * 0.7) - 2.0 * sa * ca) ≤ 1.2e-16,
      "sin(2a) matches 2 sin a cos a");
double c2a = cos(2.0 * 0.7);
double f1 = ca * ca - sa * sa, f2 = 2.0 * ca * ca - 1.0,
       f3 = 1.0 - 2.0 * sa * sa;
CHECK(fabs(c2a - f1) ≤ 8.4e-17 && fabs(c2a - f2) ≤ 1.4e-16 &&
          fabs(c2a - f3) ≤ 2.8e-17,
      "all three cos(2a) forms within their pinned gaps");
// half angle from the same family: cos^2(a/2) = (1+cos a)/2, gap 1.1e-16.
CHECK(fabs(cos(0.7 / 2.0) * cos(0.7 / 2.0) - (1.0 + ca) / 2.0) ≤ 1.2e-16,
      "half-angle formula closes");
```

listing 21: identities.c, the pythagorean, sum, difference, double, and half checks with the pinned gaps (source: math/samples/src/Ch03/identities.c, lines 60-92)

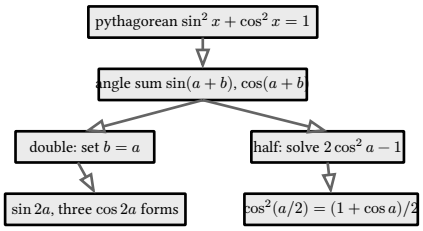


figure 14: the identity kit as one derivation: pythagorean at the base, angle sum above, double and half by substitution and solving

PITFALL: IDENTICAL IDENTITIES ROUND DIFFERENTLY

The three $\cos 2a$ forms are the same real number and differ by up to 1.4e-16 in binary64 at $a = 0.7$. The $\cos^2 a - \sin^2 a$ form squares and subtracts two rounded terms, so it is the one to avoid near $a = \pi/4$ where they nearly cancel. Compute from what you hold, and when in doubt measure the gap the way the sample does.

3.3. atan2, the quadrant-honest inverse

Every inverse trigonometric function has to throw away information, the question is how much. `acos` returns $[0, \pi]$, matching the mml-book's angle between vectors (sec 3.4, printed p 76), and loses the sign of the sine entirely. `atan(y/x)` loses the quadrant of the ratio: the division maps $(1, 1)$ and $(-1, -1)$ to the same value. `atan2(y, x)`

takes both coordinates separately and returns the angle of the point (x, y) in $[-\pi, \pi]$, which is the argument of the complex number $x + iy$ and the reason cppreference describes it as the angle of the vector. Both endpoints are live: positive zero over the negative x axis gives π, negative zero gives $-\pi$, and the sample pins both. The mml-book reaches the same place with rotations: the map that sends e_1 to $(\cos\theta, \sin\theta)$ and e_2 to $(-\sin\theta, \cos\theta)$ is the rotation matrix $\boldsymbol{R}(\theta)$ (sec 3.9, eqs 3.75 and 3.76, printed pp 91-92), and atan2 is its inverse on points. For the angle between vectors, mml-book example 3.6 (printed p 77) has $\cos\omega = 3/\sqrt{10}$ for $[1, 1]$ against $[1, 2]$, about 0.32 rad, and the atan2 form of the 2d cross over the dot returns 0.32175055439664219 directly, with no clamp guarding a rounded ratio that drifts outside $[-1, 1]$ before an `acos`.

The dry run: the four axis crossings, then the quadrant lie.

1. Axis walk: `atan2(0,1)` is 0, `atan2(1,0)` is the double $\pi/2$, `atan2(0,-1)` is the double π, `atan2(-1,0)` is $-\pi/2$. All four are exact equality checks in the sample.
2. The lie: `atan(-1.0 / -1.0)` returns 0.78539816339744828, the same as `atan(1.0)`, while `atan2(-1.0, -1.0)` returns −2.3561944901923448, and the sample checks the two differ by exactly the double π.
3. Infinities follow the coordinate map: `atan2(inf, -inf)` is $3\pi/4$, `atan2(inf, 0)` is $\pi/2$.
4. Folding: for an angle already outside the principal range, `atan2(sin(4), cos(4))` returns −2.2831853071795867, within 4.4e-16 of $4 - 2\pi$.

The two-number play: 0.7854 versus −2.3562, the same ratio, two different points, and only the two-argument call can tell them apart.

```
// the atan2 axis walk: y first, x second, all four crossings pinned.
CHECK(atan2(0.0, 1.0) == 0.0, "atan2(0,1) is 0");
CHECK(atan2(1.0, 0.0) == 1.5707963267948966, "atan2(1,0) is pi/2");
CHECK(atan2(0.0, -1.0) == 3.1415926535897931, "atan2(0,-1) is pi");
CHECK(atan2(-0.0, -1.0) == -3.1415926535897931, "atan2(-0,-1) is -pi");
CHECK(atan2(-1.0, 0.0) == -1.5707963267948966, "atan2(-1,0) is -pi/2");
// the quadrant-honest diagonals against the atan(y/x) lie: the ratio
// hides the quadrant, atan(-1/-1) = atan(1) = 0.78539816339744828
// while atan2(-1,-1) = -2.3561944901923448 = atan(1) - pi exactly.
CHECK(atan(-1.0 / -1.0) == 0.78539816339744828, "atan sees only the ratio");
CHECK(atan2(-1.0, -1.0) - atan(1.0) == -3.1415926535897931,
      "atan2(-1,-1) is atan(1) minus pi");
// infinities follow the same map: diagonal infinity pairs give the
// diagonal angles, y infinity dominates a finite x.
CHECK(atan2(INFINITY, -INFINITY) == 2.3561944901923448,
      "atan2(inf,-inf) is 3pi/4");
CHECK(atan2(INFINITY, 0.0) == 1.5707963267948966, "atan2(inf,0) is pi/2");
// the principal value of an angle already outside (-pi, pi]:
// atan2(sin 4, cos 4) = -2.2831853071795867, within 4.4e-16 of 4-2pi.
CHECK(fabs(atan2(sin(4.0), cos(4.0)) - (4.0 - 2.0 * PI)) <= 4.5e-16,
```

listing 22: triangles.c, the atan2 axis walk, the atan versus atan2 quadrant lie, infinity cases, and principal folding (source: math/samples/src/Ch03/triangles.c, lines 23-42)

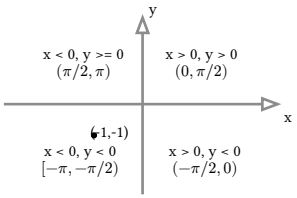


figure 15: the atan2 return map: four quadrants, four ranges, both coordinates kept

3.4. range reduction and a polynomial sin cos

A polynomial approximates sine well only near the origin, so a production sin starts by reducing its argument: find the integer n nearest to $2x/\pi$, subtract n copies of $\pi/2$ from x to get a reduced argument r in $[-\pi/4, \pi/4]$, evaluate short polynomials $s(r)$ and $c(r)$ there, then dispatch on n mod 4 for the signs and the swap. This is the Cody-Waite scheme, and its whole difficulty is that $n \cdot (\pi/2)$ must be subtracted without rounding error proportional to n.

The fix is to split $\pi/2$ into pieces whose products with n stay exact. C_1 is the double $\pi/2$ with its low 29 bits cleared, C_2 is the rest of the double, and C_3 is the 6.123233995736766e-17 tail of true $\pi/2$ beyond the double. The split sums to true $\pi/2$ within 1.5e-33, against −6.1e-17 for the unsplit double, and for the $|n| \le 13$ this file ever sees, every product nC_1, nC_2 is exact.

The dry run: reduce $x = 5.5$.

1. $t = 5.5 \cdot 0.63661977236758138 = 3.5014087480\ldots$, so $n = \text{round}(t) = 4$ and the quadrant index is $4 \bmod 4 = 0$.
2. $r = ((5.5 - 4C_1) - 4C_2) - 4C_3 = -0.78318530717958645$, inside $[-\pi/4, \pi/4]$ with room to spare.
3. Quadrant 0 means no swap: $\sin(5.5) = s(r) = -0.70554032557039181$ against the runtime's −0.70554032557039192, and $\cos(5.5) = c(r) = 0.7086697742912591$ against 0.70866977429125999.
4. The axis cases come out bit-identical to the runtime: $\text{red_sin}(\pi/2) = 1$ and $\text{red_cos}(\pi/2) = 6.123233995736766e-17$, exactly the libm values.

$n \bmod 4$	$\sin(x)$	$\cos(x)$
0	$s(r)$	$c(r)$
1	$c(r)$	$-s(r)$
2	$-s(r)$	$-c(r)$
3	$-c(r)$	$s(r)$

```
// pi/2 split cody-waite style: C1 has its low 29 bits clear so n*C1 stays
// exact for the |n| ≤ 13 this file ever sees, C2 carries the rest of the
// double pi/2, C3 carries the tail of true pi/2 beyond it. the split sums
// to true pi/2 within 1.5e-33 (mpmath), versus -6.1e-17 for the plain
// double, a phase bias that would grow linearly with n.
static constexpr double TWO_OVER_PI = 0.63661977236758138;
static constexpr double C1 = 1.5707962512969971;
static constexpr double C2 = 7.5497899487686482e-08;
static constexpr double C3 = 6.123233995736766e-17;
static constexpr double PIO2 = 1.5707963267948966;
static constexpr double TWOPI = 6.2831853071795862;
```

listing 23: sincos.c, the cody-waite split of pi over 2 in three exact pieces (source: math/samples/src/Ch03/sincos.c, lines 22-32)

```
// reduce x mod pi/2: n = round(2x/pi), r = x - n*(C1+C2+C3).
// valid for |x| ≤ 20, where |n| ≤ 13 keeps every product exact.
static double reduce(double x, int *q) {
  double nd = round(x * TWO_OVER_PI);
  double r = ((x - nd * C1) - nd * C2) - nd * C3;
  *q = (int)nd & 3; // two's complement AND: the true residue mod 4
  return r;
}

static void poly(double r, double *s, double *c) {
  double z = r * r;
  double ps =
      SC[0] +
      z * (SC[1] +
           z * (SC[2] +
                z * (SC[3] +
                     z * (SC[4] + z * (SC[5] + z * (SC[6] + z * SC[7]))))));
  double pc =
      CD[0] +
      z * (CD[1] +
           z * (CD[2] +
                z * (CD[3] +
                     z * (CD[4] + z * (CD[5] + z * (CD[6] + z * CD[7]))))));
  *s = r * ps;
  *c = pc;
}
```

listing 24: sincos.c, the reduction and the odd taylor pair evaluated by horner in z = r^2 (source: math/samples/src/Ch03/sincos.c, lines 54-79)

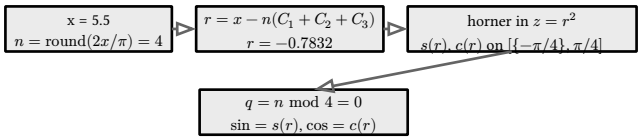


figure 16: the reduction pipeline for x = 5.5: nearest multiple, three-piece subtraction, polynomial, quadrant dispatch

VERIFY: THE SWEEP CONTRACT

The sample walks 327681 dyadic grid points $i \cdot 2^{-13}$ over $[-20, 20]$ and measures the ulp distance against the runtime sin and cos with the monotone-integer key. Measured maxima, identical in the binary and in the python mirror: 10 ulp for sin, 10 ulp for cos, absolute gap 1.1102230246251565e-15 for both, and the reduced argument never leaves $[-0.78539593616989667, 0.78539593616989667]$. The asserts carry headroom at 16 ulp and 2e-15.

Deterministic fixed-point rotations take this another way, CORDIC instead of polynomials, and book 14, ch 7 (game systems and architecture, deterministic math) builds that engine for lockstep multiplayer.

3.5. laws of sines and cosines

A triangle is three sides and three angles with two laws tying them together. The law of sines: $a/\sin A = b/\sin B = c/\sin C = 2R$ with R the circumradius. The law of cosines: $c^2 = a^2 + b^2 - 2ab\cos\gamma$, the Pythagorean theorem with a correction term that dies on right angles. Both rules and Heron's formula, area from the three sides alone, $K = \sqrt{s(s-a)(s-b)(s-c)}$ with s the semiperimeter, are DLMF section 4.42, solution of triangles. Heron is the independent cross-check, and the area also equals $(1/2)ab\sin\gamma$.

The dry run: two triangles, one exact, one measured.

1. The 3-4-5 right triangle: atan2(3,4) gives A = 0.64350110879328437, atan2(4,3) gives B = 0.92729521800161219, and A + B equals the double $\pi/2$ exactly. The three sine ratios agree to 2.8e-17. Heron's product under the square root is exactly 36, so K = 6 and the circumradius abc/4K = 2.5, exactly the hypotenuse over 2.
2. The 7-8-1 triangle: sides $a = 7$, $b = 8$ flanking $\gamma = 1$ rad. The law of cosines gives $c^2 = 52.486141742768346$ and $c = 7.2447319993750181$. Feeding c^2 back through the recovered cosine returns gamma = 1.0 bit-exactly through acos. Heron and $(1/2)ab\sin\gamma$ agree to 7.1054273576010019e-15.

The two-number play: the right triangle closes on exact doubles, K = 6 with nothing after the decimal, while the 7-8-1 triangle carries a 7.1e-15 gap between its two area routes. Integer sides plus a right angle are the only free lunch here, every other triangle pays rounding.

```
// the 3-4-5 triangle: right angle at the origin legs, hypotenuse 5.
// A = atan2(3,4) = 0.64350110879328437, B = atan2(4,3) =
// 0.92729521800161219, and the two acute angles sum to the double
// pi/2 exactly (pinned: A+B-pi/2 = 0).
double A = atan2(3.0, 4.0), B = atan2(4.0, 3.0);
CHECK(A == 0.64350110879328437 && B == 0.92729521800161219,
      "3-4-5 angles pinned");
CHECK(A + B == 1.5707963267948966, "acute pair sums to pi/2 exactly");
// law of sines: all three ratios equal to 2.78e-17 (pinned max gap).
double rA = sin(A) / 3.0, rB = sin(B) / 4.0, rC = sin(PI / 2.0) / 5.0;
CHECK(fabs(rA - rB) <= 2.8e-17 && fabs(rA - rC) <= 2.8e-17,
      "law of sines ratios agree");
// heron K = 6 exactly for the 3-4-5, so the circumradius abc/4K is
// 2.5, exactly c/2: the right-triangle rule.
double s = (3.0 + 4.0 + 5.0) / 2.0;
double K = sqrt(s * (s - 3.0) * (s - 4.0) * (s - 5.0));
CHECK(K == 6.0, "heron gives 3-4-5 area 6 exactly");
```

listing 25: triangles.c, the 3-4-5 angles, sine ratios, heron area, and the circumradius rule (source: math/samples/src/Ch03/triangles.c, lines 44-61)

```
// law of cosines, a=7 b=8 gamma=1 rad: c^2 = 52.486141742768346,
// c = 7.2447319993750181 (both pinned), the acos recovery of gamma
// returns 1.0 bit-exactly, and heron agrees with (1/2)ab sin gamma
// to 7.1054273576010019e-15.
double a = 7.0, b = 8.0, g = 1.0;
double c2 = a * a + b * b - 2.0 * a * b * cos(g);
double c = sqrt(c2);
CHECK(c2 == 52.486141742768346, "cosine law c^2 pinned");
CHECK(c == 7.2447319993750181, "cosine law side pinned");
double cosg = (a * a + b * b - c2) / (2.0 * a * b);
CHECK(acos(cosg) == 1.0, "acos recovery is bit-exact here");
double ss = (a + b + c) / 2.0;
double Kh = sqrt(ss * (ss - a) * (ss - b) * (ss - c));
```

listing 26: triangles.c, the law of cosines round trip on the 7-8-1 triangle with the heron cross-check (source: math/samples/src/Ch03/triangles.c, lines 63-76)

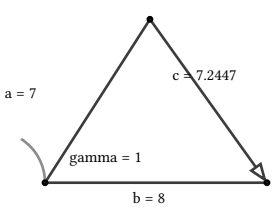


figure 17: the 7-8-1 triangle to scale: gamma = 1 rad between a = 7 and b = 8, c = 7.2447 opposite

3.6. the ambiguous ssa case

Given one angle, the side opposite it, and one more side, the SSA configuration does not determine a triangle. The law of sines gives $\sin B = b \sin A / a$ and the sine is symmetric under $B \to \pi - B$, so whenever both roots are geometrically valid, two triangles answer the same data. The sample fixes $A = \pi/6$ and $b = 7$ and sweeps the opposite side a through the three regimes.

The dry run:

1. $a = 3$: the ratio is $7 \cdot 0.5/3 = 1.1667$, above 1, no triangle exists, the side is too short to close.
2. $a = 8$: the ratio is 0.4375 and $a > b$, so only the acute root is consistent, one triangle.
3. $a = 5$: the ratio is 0.7 (the double 0.6999999999999996), and $b \sin A = 3.5 < 5 < 7 = b$ puts the case inside the ambiguity band. `asin` returns B1 = 0.77539749661075297, the supplement B2 = π − B1 = 2.3661951569790403, and `sin(B1) == sin(B2)` holds bit for bit. The two third angles give the two sides, $c_1 = 9.6328920407624956$ and $c_2 = 2.4914636122196407$ against the mpmath truths 9.6328920407624938 and 2.4914636122196443.
4. Heron on the first triangle reproduces the area of $(1/2)bc_1 \sin A$ exactly, both 16.857561071334366.

The two-number play: one ratio, 0.7, and sides 9.633 against 2.491, a factor of 3.87 between the two triangles that share it.

```c
// the ambiguous SSA case: angle A = pi/6 known, side opposite it a,
// side b = 7 known. sin B = b sin A / a decides everything.
double As = 0.52359877559829893;
CHECK(7.0 * sin(As) / 3.0 > 1.0, "a=3: ratio 7/6 exceeds 1, no triangle");
CHECK(7.0 * sin(As) / 8.0 == 0.4375, "a=8: ratio 0.4375, a>b so unique");
// a=5 sits in the ambiguity band: sin B = 0.7 exactly (7 sinA/5 is
// 0.69999999999999996 as a double), two angles carry that sine.
double sinB = 7.0 * sin(As) / 5.0;
double B1 = asin(sinB), B2 = PI - B1;
CHECK(B1 == 0.77539749661075297, "first root pinned");
CHECK(B1 + B2 == PI, "supplementary pair sums to pi exactly");
CHECK(sin(B1) == sin(B2), "both roots share one sine");
// the two triangles: C = pi - A - B, c = a sin C / sin A.
double C1 = PI - As - B1, C2 = PI - As - B2;
double cA = 5.0 * sin(C1) / sin(As), cB = 5.0 * sin(C2) / sin(As);
// pinned: cA = 9.6328920407624956 (true 9.6328920407624938),
// cB = 2.4914636122196407 (true 2.4914636122196443).
CHECK(fabs(cA - 9.6328920407624938) <= 4e-15, "first triangle side");
CHECK(fabs(cB - 2.4914636122196443) <= 4e-15, "second triangle side");
// heron cross-check on the first triangle: sides 5, 7, cA give the
// same area as (1/2) b c sin A, bit for bit (pinned diff 0).
double st = (5.0 + 7.0 + cA) / 2.0;
double Kt = sqrt(st * (st - 5.0) * (st - 7.0) * (st - cA));
```

listing 27: triangles.c, the three SSA regimes and the supplementary pair with the heron cross-check (source: math/samples/src/Ch03/triangles.c, lines 78-101)

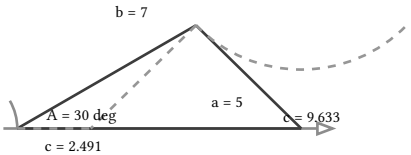


figure 18: the ambiguous case: a = 5 swings an arc through the base ray at two points, two triangles share A = 30 degrees and b = 7

PITFALL: ASIN RETURNS ONE ROOT

> asin covers $[-\pi/2, \pi/2]$ only. When the SSA ratio lands strictly between sin A and 1, the mirror root $\pi - B$ is a second valid angle and the program must add it by hand, the way the sample does. Forgetting it is the classic wrong-answer in any solver that reads two sides and a non-included angle.

3.7. large arguments and payne-hanek

The reduction, not the polynomial, is where a sin gains or loses its accuracy. The polynomial works on $r \in [-\pi/4, \pi/4]$ and is accurate to about 4.2e-17 for sine and 1.0e-15 for cosine by truncation, but r can only be as good as the $\pi/2$ used to compute it. The double 2π sits 2.4492935982947064e-16 below true 2π, and a naive reduction `fmod(x, 2*pi)` subtracts a wrong period every turn, so the phase error grows as $k \cdot 2.45 \cdot 10^{-16}$ with k the number of turns.

The dry run: ask for `sin(1e16)`.

1. $k = 10^{16}/2\pi \approx 1.59 \cdot 10^{15}$ turns. The accumulated phase error is $1.59 \cdot 10^{15} \cdot 2.45 \cdot 10^{-16} = 0.38981718325193748$ rad.
2. `fmod(1e16, 2*pi_d)` returns 2.637242432414304, but the true reduced phase is 2.2474252491623665. The phases differ by the predicted 0.3898 rad.
3. sin of the wrong phase is 0.48323866838796636. The runtime `sin(1e16)` returns 0.77968800660697879, which mpmath confirms as correctly rounded: the C runtime reduces exactly, and the sample checks the naive shortcut is off by at least 0.29 in sine.

The exact reducers use Payne-Hanek: instead of a precomputed constant of 53 bits, they read the binary expansion of $2/\pi$ from a table at the bit position the argument demands, so the subtraction recovers the phase to full precision no matter how large x is. This chapter states the scheme and does not implement it, the three-piece Cody-Waite split in `sincos.c` is documented for $|x| \le 20$ where $|n| \le 13$ keeps the products exact, and the sweep asserts that domain.

The two-number play: 0.39 rad of phase error from a 2.45e-16 constant error, and 0.29644933821901243 of error in the sine, all from one `fmod` against the wrong 2π.

```
    CHECK(rmax ≤ 0.7854, "reduced argument stays inside pi/4");

    // the large-argument trap: reduce 1e16 with the naive fmod against the
    // double 2pi. the runtime sin(1e16) is correctly rounded at
    // 0.77968800660697879 (it pays for exact reduction internally), but
    // the naive phase 2.637242432414304 is off by 0.38981718325193748 rad
    // because the double 2pi sits 2.45e-16 below true 2pi and 1.59e15
    // turns multiply that gap out. sin of the wrong phase: 0.48323866838796636.
    double naive_phase = fmod(1e16, TWOPI);
    CHECK(fabs(sin(1e16) - 0.77968800660697879) ≤ 1e-15,
          "libm sin(1e16) is the true value");
    CHECK(naive_phase == 2.637242432414304, "naive phase pinned");
    CHECK(fabs(sin(naive_phase) - sin(1e16)) ≥ 0.29,
          "naive reduction is off by 0.29 in sine");
```

listing 28: sincos.c, the 1e16 trap: the runtime is right, the naive double-2pi reduction is 0.39 rad out of phase (source: math/samples/src/Ch03/sincos.c, lines 180-193)

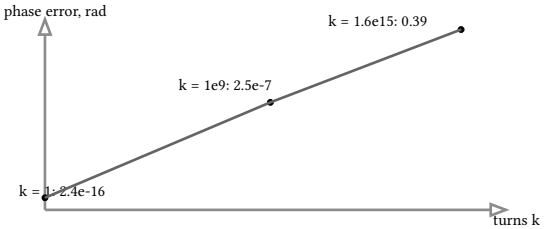


figure 19: phase error of the naive double-2pi reduction against turns, one decade per step

WARNING: NEVER REDUCE WITH THE DOUBLE 2 PI

Any loop of the form `x - k*2*M_PI` or `fmod(x, 2*M_PI)` inherits a 2.45e-16 period error and multiplies it by the turn count. At 1e16 that is 0.39 rad. Either keep arguments inside a domain your split covers, the way this chapter does, or call a sin that reduces exactly.

3.8. next

Points, lines, and the transforms that move them, including the rotation matrix this chapter only quoted, are chapter 4.

sources: mml-book draft 2024-01-15, ch 3, printed pp 76-78 (angles, orthogonality, example 3.6), p 81 (sin and cos as orthogonal functions), pp 83-85 (projection length and eq 3.44), pp 91-94 (rotation matrices, eqs 3.75-3.76, the rotation group), read from the cached text layer and cross-verified against the pdf pages. NIST DLMF, sections 4.14, 4.17, and 4.21 for the definitions, symmetries, and multiple-angle formulas, 4.23 for the inverse functions, 4.42 for the solution of triangles, https://dlmf.nist.gov/4.14 and neighbors, fetched 2026-09-21. cppreference, `sin` and `atan2`, special value tables, https://en.cppreference.com/w/c/numeric/math/sin and .../atan2, fetched 2026-09-21. Cody and Waite 1980, the split-reduction scheme, and Payne and Hanek 1971, stated not implemented, as named in Muller, "Elementary Functions", 2nd ed. Probed on this machine the same day. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch03`, 62 checks in chapter 03 of the math suite.

4. coordinate geometry

A plane of coordinates is where geometry becomes arithmetic, and this chapter builds the arithmetic carefully enough to trust. The moves: separate points from vectors and give both a length and a direction, take the 2D cross product as one exact orientation predicate, solve parametric lines, rays, and segments through that predicate with no division, cut circles with lines by a squared discriminant and pass a circle through three points by bisectors, compose translate, rotate, and scale as matrices where order is the meaning, and walk polar and parametric curves point by point. Every behavioral claim below is one of the 51 checks in the 4 samples of chapter 4 or a sentence quoted from the mml draft fetched 2026-09-21. Where book 9, ch 23 (practical data structures and algorithms, computational geometry) and book 9, ch 37 (practical data structures and algorithms, computational geometry ii) hunt winning contest algorithms, this chapter builds the coordinate algebra underneath them and leaves the algorithmics out.

4.1. points, vectors, direction

A point is a location, a vector is a displacement, and two operations connect them, only two. The difference of two points is a vector, $\boldsymbol{v} = p - q$, and a point plus a vector is a point, $q + \boldsymbol{v} = p$. The sum of two points is on neither list. It compiles, it runs, and it answers a question nobody asked, which is how it breaks code: averaging positions is legal only as an affine combination whose weights sum to 1, so the centroid $\frac{p_1+p_2+p_3}{3}$ is fine while a bare $p_1 + p_2$ is a type error the compiler cannot see.

Length comes from an inner product. The mml draft opens its analytic geometry chapter by saying "we equip the vector space with an inner product that induces the geometry of the vector space" (draft 2024-01-15, ch 3 opener). With the dot product $\boldsymbol{u}^T\boldsymbol{v} = \sum u_i v_i$ (its eq 3.5) the default ruler is the Euclidean norm $\|\boldsymbol{v}\|_2 = \sqrt{\boldsymbol{v}^T\boldsymbol{v}}$ (eq 3.4), with the Manhattan norm $\|\boldsymbol{v}\|_1 = \sum|v_i|$ (eq 3.3) as the other classic. Both satisfy the three axioms of its definition 3.1: absolute homogeneity, the triangle inequality, positive definiteness. The distance between points rides on the norm, $d(x, y) = \|x - y\|$ (eq 3.21), and is a metric: positive definite, symmetric, triangle. Direction is the vector with its length divided out, $\hat{\boldsymbol{v}} = \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|}$, and the angle between two directions is read off

$$\cos\omega = \frac{\boldsymbol{u}^T\boldsymbol{v}}{\|\boldsymbol{u}\|\|\boldsymbol{v}\|}$$

which is the mml eq 3.25, valid because the ratio is pinned between −1 and 1 (its eq 3.24) for a unique $\omega \in [0, \pi]$.

The dry run: from (1, 2) to (4, 6) the difference is the vector (3, 4), the distance is $\sqrt{9+16} = 5$, and the unit direction is (0.6, 0.8) = $\left(\frac{3}{5}, \frac{4}{5}\right)$. Stepping twice as far, (1, 2) + 2 (3, 4) = (7, 10), scales the displacement and never the start. The two-number play for length: the same vector (3, 4) measures 7 under the Manhattan norm and 5 under the Euclidean one, two axioms-compliant rulers that disagree, which is why a codebase picks one and names it.

```c
typedef struct {
  long long x, y;
} V;

[[nodiscard]] static long long cross(V a, V b) { return a.x * b.y - a.y * b.x; }
[[nodiscard]] static long long dot(V a, V b) { return a.x * b.x + a.y * b.y; }
static V sub(V a, V b) { return (V){a.x - b.x, a.y - b.y}; }
```

listing 29: the vector substrate: integer points, difference, dot, cross (source: math/samples/src/Ch04/lines.c, lines 19-25)

PITFALL: POINTS ARE NOT VECTORS

> Storing both as a pair of numbers is fine, but the operations must respect the split. $p - q$ is a vector, $p + v$ is a point, and $p + q$ is undefined. Any formula that adds two locations, or scales a location by 2, is wrong in a way no compiler catches. The fix travels with the types: think "point minus point" before writing a single plus.

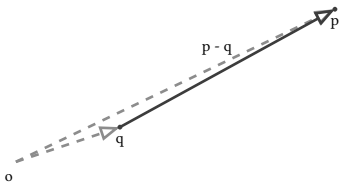


figure 20: point difference as the one bridge between points and vectors

Vectors as a space, with bases, coordinates, and the full algebra, are the subject of book 2, ch 5 (mathematics for programmers, vectors and vector spaces), and this chapter only needs the plane.

4.2. orientation: one sign, exactly

The 2D cross product of two vectors is the scalar

$$a \times b = a_x b_y - a_y b_x,$$

the z component of the 3D cross product, and also twice the signed area of the triangle with corners 0, a, b. One subtraction of point differences turns it into a predicate on triples:

```
orient(p0, p1, p2) = sign(cross(p1 - p0, p2 - p0))
```

fragment, illustrative, not executed

The sign is +1 when p0, p1, p2 turn counterclockwise, −1 clockwise, and 0 when collinear, the right-hand-rule convention with z pointing out of the plane. That one integer is the workhorse of computational geometry: inside-or-outside, left-or-right, hull order, and the intersection tests of the next section all reduce to it.

On an integer grid the predicate is exact. With coordinates bounded by 10^9, differences reach $2 \cdot 10^9$, products $4 \cdot 10^{18}$, and their difference $8 \cdot 10^{18}$, under the signed 64-bit ceiling of about $9.22 \cdot 10^{18}$: the sample pins the bound with a `static_assert` at compile time, so the exactness is a checked property, not a hope.

The dry run: for the triple (0,0), (1,0), (1,1) the differences are $a = (1, 0)$ and $b = (1, 1)$, the cross is $1 \cdot 1 - 0 \cdot 1 = 1$, and the verdict is +1, counterclockwise. For the near-degenerate pair $b_1 = (2^{53} + 3, 1)$ and $b_2 = (2^{53} + 5, 1)$ the exact cross is $b_1 - b_2 = -2$, a small but real clockwise turn. In doubles both coordinates round to the same value $2^{53} + 4 = 9007199254740996.0$ (spacing is 2 at 2^{53}, and both ties round to even), so the double cross is exactly 0.0 and the verdict flips to collinear. The two-number play of the chapter: exact cross −2 against double cross 0.0, one triple, two answers, and only one of them true.

triple	exact cross	exact verdict	double cross	double verdict
$(0,0), (b_1,1), (b_2,1)$	−2	clockwise, −1	0.0	collinear
$(0,0), (b_2,1), (b_1,1)$	+2	counterclockwise, +1	0.0	collinear

```
typedef struct {
  long long x, y;
} Pt;

// a x b, the z component of the 2d cross product: twice the signed triangle
// area
[[nodiscard]] static long long cross(Pt a, Pt b) {
  return a.x * b.y - a.y * b.x;
}

// sign of cross(p1-p0, p2-p0): +1 ccw, -1 cw, 0 collinear
[[nodiscard]] static int orient(Pt p0, Pt p1, Pt p2) {
  const Pt a = {p1.x - p0.x, p1.y - p0.y};
  const Pt b = {p2.x - p0.x, p2.y - p0.y};
  const long long area = cross(a, b);
  return (area > 0) - (area < 0);
}

// grid coords up to 1e9: differences 2e9, products 4e18, their difference 8e18
static_assert(2000000000LL * 4000000000LL < INT64_MAX,
              "1e9 coordinate grid leaves long long headroom");

constexpr long long BIG1 = (1LL << 53) + 3;
constexpr long long BIG2 = (1LL << 53) + 5;
```

listing 30: cross, orient, and the compile-time grid bound with the big pair declared (source: math/samples/src/Ch04/orientation.c, lines 19-42)

```
  // the two-number play: one triple through exact and double predicates
  CHECK(BIG1 - BIG2 == -2, "exact cross of the near-degenerate pair is -2");
  CHECK(orient((Pt){0, 0}, (Pt){BIG1, 1}, (Pt){BIG2, 1}) == -1,
        "exact predicate calls the triple clockwise");
  CHECK(orient((Pt){0, 0}, (Pt){BIG2, 1}, (Pt){BIG1, 1}) == 1,
        "reversed triple flips to counterclockwise");
  CHECK((double)BIG1 == (double)BIG2,
        "both big coordinates collapse to one double");
  CHECK((double)BIG1 == 9007199254740996.0,
        "the shared double is 2^53+4, ties to even");
  const double dcross = (double)BIG1 * 1.0 - 1.0 * (double)BIG2;
  CHECK(dcross == 0.0, "double cross reports 0.0, collinear");
  CHECK(dcross != -2.0 && dcross != 2.0, "the double cross lost both signs");
```

listing 31: the two-number play: one near-degenerate triple through both predicates (source: math/samples/src/Ch04/orientation.c, lines 59-71)

VERIFY: EXACTNESS IS A COMPILE-TIME FACT

The sample's `static_assert(2000000000LL * 4000000000LL < INT64_MAX, ...)` encodes the worst case of the 10^9 grid. Raise the grid bound without re-deriving the inequality and the build stops, which is the correct place for a geometry bug to die. The rounding wall at 2^{53} is the float spacing story of book 2, ch 1 (mathematics for programmers, floating point anatomy).

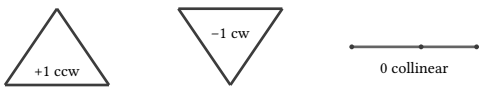


figure 21: the orientation sign: counterclockwise +1, clockwise −1, collinear 0

The algorithms this predicate feeds, convex hulls, point-in-polygon, sweeps, are the business of book 9, ch 23 (practical data structures and algorithms, computational geometry), and here it stays a definition.

4.3. lines, rays, segments

One parametric form covers all three shapes: $l(t) = p + t\boldsymbol{r}$ with p a point and $\boldsymbol{r}$ a direction vector. The line is every t, the ray is $t \geq 0$, the segment is $t \in [0, 1]$. Ranges, not formulas, are what separate them, so one solver serves all three by changing the bound on t.

Two such curves meet where $p + t\boldsymbol{r} = q + u\boldsymbol{s}$. Rearranged, $t\boldsymbol{r} - u\boldsymbol{s} = \boldsymbol{w}$ with $\boldsymbol{w} = q - p$. Cross both sides with $\boldsymbol{s}$: the u term dies because $\boldsymbol{s} \times \boldsymbol{s} = 0$, leaving $t = \frac{\boldsymbol{w} \times \boldsymbol{s}}{\boldsymbol{r} \times \boldsymbol{s}}$. Cross with $\boldsymbol{r}$ the same way and $u = \frac{\boldsymbol{w} \times \boldsymbol{r}}{\boldsymbol{r} \times \boldsymbol{s}}$. No square roots, no trigonometry, and a zero denominator is exactly the parallel case.

The dry run: the X crossing fixture has $p = (0, 0)$, $\boldsymbol{r} = (4, 4)$, $q = (0, 4)$, $\boldsymbol{s} = (4, -4)$. The denominator is $\boldsymbol{r} \times \boldsymbol{s} = 4(-4) - 4 \cdot 4 = -32$, and with $\boldsymbol{w} = (0, 4)$ both numerators are $0(-4) - 4 \cdot 4 = -16$, so $t = u = -16/-32 = \frac{1}{2}$ and the shared point is $(2, 2)$, mid of both segments. The extended-lines fixture $(0, 0)$ to $(1, 1)$ against $(3, 0)$ to $(4, 1)$ solves to $t = 0$, $u = -3$: the infinite lines cross at the origin, but u falls outside $[0, 1]$ and the segments miss. Two numbers, 1/2 against −3, are the whole difference between hit and miss.

fixture	denominator	t	u	verdict
x crossing	−32	1/2	1/2	hit at (2,2)
parallel	0	–	–	abstain
collinear overlap	0	–	–	abstain
endpoint touch	−8	1	0	hit at (2,2)
ray against segment	4	3	1/2	hit at (3,1)
extended miss	−1	0	−3	miss

The same solve projects a point onto a line: the foot of the perpendicular from x to $p + t\boldsymbol{r}$ sits at $t^* = ((x - p) \cdot \boldsymbol{r})/(\boldsymbol{r} \cdot \boldsymbol{r})$, and the distance from x to the line is $|\ (x - p) \times \boldsymbol{r}\ |\ /\|\boldsymbol{r}\|$. Verified on the diagonal fixture $p = (0, 0)$, $\boldsymbol{r} = (4, 4)$, $x = (4, 0)$: $t^* = 16/32 = \frac{1}{2}$, the foot is (2,2), and the distance squared is $16^2/32 = 8$, a distance of $2\sqrt{2}$. The collinear overlap of the callout below is the same projection read twice: the pinned dots give $t^* = 8/16 = \frac{1}{2}$ and $t^* = 24/16 = \frac{3}{2}$, the second foot landing past the segment end, which is what overlap means numerically.

```
typedef struct {
  int hit;
  V pt;
} Xing;

// 0 ≤ num/den ≤ 1 by cross multiplication, no division
[[nodiscard]] static int in_unit(long long num, long long den) {
  return den > 0 ? (num ≥ 0 && num ≤ den) : (num ≤ 0 && num ≥ den);
}

[[nodiscard]] static Xing seg_x(V p, V r, V q, V s) {
  const long long denom = cross(r, s);
  const V w = sub(q, p);
  const long long tn = cross(w, s);
  const long long un = cross(w, r);
  if (denom == 0 || !in_unit(tn, denom) || !in_unit(un, denom))
    return (Xing){0, (V){0, 0}};
  const long long px = p.x * denom + tn * r.x;
  const long long py = p.y * denom + tn * r.y;
  if (px % denom ≠ 0 || py % denom ≠ 0)
    return (Xing){0, (V){0, 0}};
  return (Xing){1, (V){px / denom, py / denom}};
}
```

listing 32: the segment solver: cross-multiplied bounds, scaled intersection point (source: math/samples/src/Ch04/lines.c, lines 27-49)

```
  // endpoint touch at the shared (2,2): denom -8, t = 1, u = 0
  CHECK(seg_x((V){0, 0}, (V){2, 2}, (V){2, 2}, (V){2, -2}).hit == 1 &&
            seg_x((V){0, 0}, (V){2, 2}, (V){2, 2}, (V){2, -2}).pt.x == 2 &&
            seg_x((V){0, 0}, (V){2, 2}, (V){2, 2}, (V){2, -2}).pt.y == 2,
        "endpoint touch hits at the shared corner");

  // ray (0,1) dir (1,0) against segment (3,-1)-(3,3): denom 4, t = 3, u = 1/2
  CHECK(cross((V){1, 0}, (V){0, 4}) == 4 &&
            cross((V){3, -2}, (V){0, 4}) == 12 &&
            cross((V){3, -2}, (V){1, 0}) == 2,
        "ray numbers: denom 4, t = 3, u = 1/2");
  const V rp = {0 + 3 * 1, 1 + 3 * 0};
  CHECK(rp.x == 3 && rp.y == 1, "the ray hits the segment at (3,1)");
```

listing 33: endpoint touch and the ray with its bound lifted, exact t and u (source: math/samples/src/Ch04/lines.c, lines 76-88)

WARNING: PARALLEL IS AN ANSWER, NOT AN ERROR

When the denominator is 0 the solver abstains, and the collinear overlap of (0,0) to (4,0) against (2,0) to (6,0) shows why: the parametric solve has no unique meeting point. Overlap then lives in dot-product projections, $8/16 = \frac{1}{2}$ against $24/16 = \frac{3}{2}$ of $r^2 = 16$ in the sample. The full case analysis, touching, overlapping, disjoint, belongs to book 9, ch 37 (practical data structures and algorithms, computational geometry ii), which owns the contest-grade template.

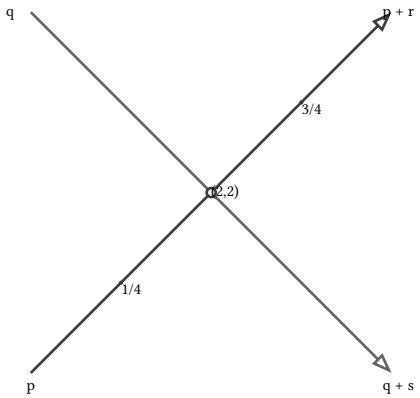


figure 22: the x crossing: two parametric segments meeting at t = u = 1/2

4.4. circles and conics

A circle is the set of points at a fixed squared distance r^2 from a center, and the squared form is the honest one: keep $r^2 = 10$ as an integer, do not open the `sqrt` door, because `isqrt` shows it only floors, `isqrt(10) = 3`, a one-way loss of information.

The vertical line $x = k_x$ against $x^2 + y^2 = r^2$ gives $y^2 = r^2 - k_x^2$ on sight, and the sign of that one integer is the discriminant: positive cuts a chord, zero is tangent, negative misses. In general the half chord is $\sqrt{r^2 - h^2}$ where h is the perpendicular distance from center to line, by the right triangle from center to foot to hit point, and tangency is exactly $h = r$. For a line $ax + by = c$ the nearest point to the origin lies along the normal (a, b) at parameter $c/(a^2 + b^2)$, so the distance is $|c| \ /\sqrt{a^2 + b^2}$.

The dry run: with $r = 5$, the line $x = 3$ yields $y^2 = 25 - 9 = 16$ and cuts at $(3, 4)$ and $(3, -4)$, $x = 5$ yields 0 and touches at $(5, 0)$, $x = 6$ yields -11 and misses. The tangent at the circle point $(3, 4)$ is $3x + 4y = 25$: its distance from the origin is $25/\sqrt{9 + 16} = 5 = r$, and its direction $(-4, 3)$ dots with the radius to 0, perpendicular as tangency demands.

A circle through three points comes from perpendicular bisectors, the set of points equidistant from two given ones. For (0,0), (6,0), (2,4): equidistance from (0,0) and (6,0) is $x = 3$, and expanding $x^2 + y^2 = (x - 2)^2 + (y - 4)^2$ gives $x + 2y = 5$. The bisectors meet at $(3, 1)$, which sits at squared distance 10 from all three corners, so the circle is center $(3, 1)$ with $r^2 = 10$. Three collinear points, orientation 0 back in the orientation section, have parallel bisectors and carry no circle at all.

The general second-degree curve $ax^2 + bxy + cy^2 + dx + ey + f = 0$ is a conic, sorted by the sign of $b^2 - 4ac$: negative is an ellipse, the circle being the special case $b = 0, a = c$. Zero is a parabola, positive a hyperbola. Substituting canonical members confirms the sort, the unit circle gives -4, $xy = 1$ gives $+1$, $y = x^2$ gives 0, a one-off fraction pinned in the playground.

The two-number play sits near the tangent: with $r = 10^8$ and $k_x = 10^8 - 1$ the exact half chord squared is $r^2 - k_x^2 = 199999999$, while in doubles $(10^8)^2 = 10^{16}$ is exact but $(10^8 - 1)^2 = 9999999800000001$ rounds to 9999999800000000.0, and the subtraction drifts to 200000000.0. One unit of drift on a quantity whose sign is the answer, the same wall as the orientation pair, and the same cure: exact integers on a bounded grid.

```
[[nodiscard]] static long long half_chord2(long long kx, long long r) {
  return r * r - kx * kx;
}

// exact floor sqrt by scan, fine for n up to a few thousand
static long long isqrt(long long n) {
  long long k = 0;
  while (k * k ≤ n)
    k++;
  return k - 1;
}
```

listing 34: squared half chord and exact floor sqrt, no floating point (source: math/samples/src/Ch04/circles.c, lines 19-29)

```
  // big near-tangent: r = 1e8 against x = 99999999, one unit inside the rim
  constexpr long long R = 100000000LL;
  constexpr long long X = 99999999LL;
  CHECK(R * R - X * X == 199999999,
        "exact half chord squared 199999999 near the tangent");
  CHECK((double)10000000000000000LL == 10000000000000000.0,
        "1e16 is exactly a double");
  CHECK((double)9999999800000001LL == 9999999800000000.0,
        "(1e8-1)^2 rounds to 9999999800000000.0");
  CHECK((double)(R * R) - (double)(X * X) == 200000000.0,
        "the double half chord squared drifts to 200000000.0");
```

listing 35: the big near-tangent: exact 199999999 versus double 200000000.0 (source: math/samples/src/Ch04/circles.c, lines 55-65)

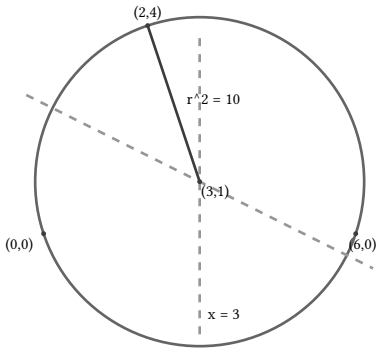


figure 23: circumcircle of (0,0), (6,0), (2,4): bisectors meet at (3,1), r^2 = 10

4.5. affine transforms and order

Translate, rotate, and scale all have the shape $x' = ax + by + c$, $y' = dx + ey + f$: an affine map, storable as 6 numbers, the 2x3 matrix of the sample. Rotation by θ counterclockwise is

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

and the mml draft asks exactly this object of the reader as a linear map $\mathbb{R}^2 \to \mathbb{R}^2$ in its exercise 2.16e, there written $x \mapsto \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} x$ for $\theta \in [0, 2\pi)$, which is the same family read clockwise, rotation by $-\theta$. Rotation preserves dot products, so it preserves lengths and the angle measure of the first section, and reading an angle from coordinates is trigonometry, book 2, ch 3 (mathematics for programmers, trigonometry).

The rotation that a 3-4-5 triangle knows, $\cos\theta = 3/5$ and $\sin\theta = 4/5$, maps the integer grid onto itself: (5, 0) goes to (3, 4) and (0, 5) goes to (-4, 3), exactly, in the sample's integer routine.

The dry run: composition order, starting from (1,1). Translate by (1,2) first, giving (2,3), then rotate 90 degrees counterclockwise, giving (-3,2). Rotate first instead, giving (-1,1), then translate, giving (0,3). Same two maps, same start, two different answers: the two-number play of the section, (-3,2) against (0,3). In matrix language the maps do not commute, $\boldsymbol{AB} \neq \boldsymbol{BA}$, while associativity holds, $(\boldsymbol{AB})\boldsymbol{C} = \boldsymbol{A}(\boldsymbol{BC})$, pinned entry by entry in the sample with the pinned products $\boldsymbol{AB} = \begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ and $\boldsymbol{ABC} = \begin{bmatrix} 0 & -3 & -2 \\ 2 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$, which sends (1,1) to (-5,3).

order	after the first map	after the second
translate (1,2), then rotate 90	(2,3)	(-3,2)
rotate 90, then translate (1,2)	(-1,1)	(0,3)

The 3x3 form is the homogeneous trick: append the row $(0, 0, 1)$ and translation stops being a special case, composition becomes a plain matrix product, and the last row stays $(0, 0, 1)$ across every product of transforms.

One more read of the pinned composite closes the loop with the first section. The composite sends (0,0) to $(-2, 1)$ by the same pinned entries, so the difference of the two images is $(-5, 3) - (-2, 1) = (-3, 2)$: the translation column cancels and the linear part alone carries differences of points, which is the point-minus-vector rule walking in matrix clothes.

```c
// 2x3 affine matrix: x' = a x + b y + c, y' = d x + e y + f
typedef struct {
  double a, b, c, d, e, f;
} Aff;

[[nodiscard]] static V2 apply(Aff m, V2 p) {
  return (V2){m.a * p.x + m.b * p.y + m.c, m.d * p.x + m.e * p.y + m.f};
}
[[nodiscard]] static Aff translate(double dx, double dy) {
  return (Aff){1, 0, dx, 0, 1, dy};
}
[[nodiscard]] static Aff rot90(void) { return (Aff){0, -1, 0, 1, 0, 0}; }
[[nodiscard]] static Aff scale2(double sx, double sy) {
  return (Aff){sx, 0, 0, 0, sy, 0};
}
```

listing 36: affine constructors: translate, rotate 90, scale as 2x3 matrices (source: math/samples/src/Ch04/transforms.c, lines 26-40)

```
constexpr M3 ROT90 = {{{0, -1, 0}, {1, 0, 0}, {0, 0, 1}}};
constexpr M3 TR12 = {{{1, 0, 1}, {0, 1, 2}, {0, 0, 1}}};
constexpr M3 SC23 = {{{2, 0, 0}, {0, 3, 0}, {0, 0, 1}}};
constexpr M3 AB = {{{0, -1, -2}, {1, 0, 1}, {0, 0, 1}}};
constexpr M3 ABC = {{{0, -3, -2}, {2, 0, 1}, {0, 0, 1}}};

int main(void) {
  // hand-computed: translate(2,3) sends (1,1) to (3,4); rot90 sends
  // (x,y) to (-y,x) so (2,0) lands on (0,2); scale(2,3) stretches (1,1)
  // to (2,3)
  const V2 t = apply(translate(2, 3), (V2){1, 1});
  CHECK(t.x == 3.0 && t.y == 4.0, "translate(2,3) moves (1,1) to (3,4)");
  const V2 r = apply(rot90(), (V2){2, 0});
  CHECK(r.x == 0.0 && r.y == 2.0, "rot90 turns +x onto +y");
  const V2 sc = apply(scale2(2, 3), (V2){1, 1});
  CHECK(sc.x == 2.0 && sc.y == 3.0, "scale(2,3) stretches (1,1) to (2,3)");

  // the product ROT90*TR12 applies translate(1,2) first, then the
  // rotation, per the right-to-left convention
  CHECK(eq3(mul3(ROT90, TR12), AB),
        "translate(1,2) then R90 composes to the pinned AB");
  CHECK(eq3(mul3(mul3(ROT90, TR12), SC23), mul3(ROT90, mul3(TR12, SC23))),
        "3x3 multiply associates");
  CHECK(eq3(mul3(mul3(ROT90, TR12), SC23), ABC),
        "(AB)C equals the pinned [[0,-3,-2],[2,0,1],[0,0,1]]");
  const I2 w = mv3(ABC, 1, 1);
  CHECK(w.x == -5 && w.y == 3, "the composite sends (1,1) to (-5,3)");
```

listing 37: homogeneous 3x3 constants, associativity, and the pinned composite (source: math/samples/src/Ch04/transforms.c, lines 79-105)

NOTE: THE HOMOGENEOUS ROW IS A PREVIEW

Stacking $(x, y, 1)$ and closing with $(0, 0, 1)$ buys composition as multiplication and nothing else yet. Why that trade pays off, inverses, ranks, the whole matrix algebra, is the next stop in book 2, ch 6 (mathematics for programmers, matrices and linear systems). Read the product right to left: in $\boldsymbol{ABCp}$, the map $\boldsymbol{C}$ touches the point first.

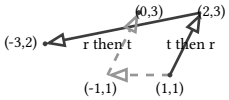


figure 24: translate then rotate versus rotate then translate from the same start

4.6. polar and parametric curves

Polar coordinates trade the pair (x, y) for a radius and an angle:

$$x = r\cos\theta, \quad y = r\sin\theta,$$

and the way back is `atan2(y, x)`. At $r = 6$, $\theta = \pi/3$ the point is $(6\cos(\pi/3), 6\sin(\pi/3)) = (3.000000000000001, 5.196152422706632)$, because the double nearest $\pi/3$ has a cosine one ulp above $1/2$. The roundtrip stays honest: `atan2(4, 3)` followed by multiplying by 5 returns $(3, 4)$ to within 10^{-12}, pinned in the sample. The angle conventions and the ulp arithmetic of π belong to book 2, ch 3 (mathematics for programmers, trigonometry).

A parametric curve is a point-valued function of one parameter, and the closing example walks one: the ellipse

$$E(t) = (1 + 3\cos t, \quad 2 + 2\sin t),$$

which is the unit circle of this chapter's fourth section pushed through the machine of the fifth: scale by $(3, 2)$, then translate by (1, 2). A circle is the special case of equal axes.

The dry run: four cardinal values of t. $t = 0$ gives (4, 2), $t = \pi/2$ gives (1.0000000000000002, 4), $t = \pi$ gives (-2, 2.0000000000000004), and $t = 3\pi/2$ gives (0.9999999999999994, 0): $\sin(3\pi/2)$ lands exactly on -1 while $\cos(3\pi/2)$ carries $-1.8 \cdot 10^{-16}$, the largest residue of the four. The residues are the story: three of the four zero-expected values ($\cos(\pi/2)$, $\sin(\pi)$, $\cos(3\pi/2)$, against an exact $\sin(0)$) are not quite 0 in doubles, so each "exact" cardinal point lands one rounding step off. The two-number play: 2.0000000000000004 against 2, and the table's exact-arithmetic (1, 0) at $3\pi/2$ against the pinned 0.9999999999999994. The figure walks the same curve at twelve values of t, the hand-computed points of the playground one-off joined by segments, and the table lists the walk in exact arithmetic: the cardinal entries are the ones whose double residues the listing pins.

t	0	pi/6	pi/3	pi/2	2pi/3	5pi/6
(x, y)	(4, 2)	(3.598, 3)	(2.5, 3.732)	(1, 4)	(-0.5, 3.732)	(-1.598, 3)

t	pi	7pi/6	4pi/3	3pi/2	5pi/3	11pi/6
(x, y)	(-2, 2)	(-1.598, 1)	(-0.5, 0.268)	(1, 0)	(2.5, 0.268)	(3.598, 1)

```
// polar pins: 6*cos(pi/3) = 3.000000000000001 (cos one ulp above 1/2),
// 6*sin(pi/3) = 5.196152422706632; atan2(4,3) roundtrips 5*r back to
// (3,4) within 1e-12
const double th = PI / 3.0;
CHECK(6.0 * cos(th) == 3.000000000000001 &&
          6.0 * sin(th) == 5.196152422706632,
      "polar r=6 theta=pi/3: cos(pi/3) is one ulp above 1/2");
const double th2 = atan2(4.0, 3.0);
CHECK(fabs(5.0 * cos(th2) - 3.0) ≤ 1e-12 &&
          fabs(5.0 * sin(th2) - 4.0) ≤ 1e-12,
      "polar roundtrip of (3,4) lands within 1e-12");
// ellipse (1,2) axes 3,2 cardinal pins from the probe: t=0 (4,2);
// t=pi/2 (1.0000000000000002, 4); t=pi (-2, 2.0000000000000004);
// t=3pi/2 (0.9999999999999994, 0), sin exact at -1, cos residue
// -1.8369701987210297e-16 the largest of the four
CHECK(1.0 + 3.0 * cos(0.0) == 4.0 && 2.0 + 2.0 * sin(0.0) == 2.0 &&
          1.0 + 3.0 * cos(PI / 2.0) == 1.0000000000000002 &&
          2.0 + 2.0 * sin(PI / 2.0) == 4.0 && 1.0 + 3.0 * cos(PI) == -2.0 &&
          2.0 + 2.0 * sin(PI) == 2.0000000000000004 &&
          1.0 + 3.0 * cos(3.0 * PI / 2.0) == 0.9999999999999994 &&
          2.0 + 2.0 * sin(3.0 * PI / 2.0) == 0.0,
      "ellipse (1,2) axes 3,2: cos(3pi/2) leaves -1.8e-16, sin exact");
```

listing 38: polar pins and the ellipse residues at the four cardinal t (source: math/samples/src/Ch04/transforms.c, lines 125-146)

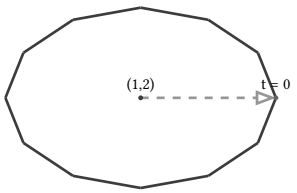


figure 25: the parametric ellipse walked at twelve values of t

One parametric curve walked, the coordinate plane now carries points, lines, circles, and transforms that compose. Vectors as a space, with bases and the algebra that generalizes this plane, are next in book 2, ch 5 (mathematics for programmers, vectors and vector spaces).

sources: mml-book draft 2024-01-15, ch 3 opener and section 3.1 norms (pp 70-72 printed, eq 3.3 Manhattan, eq 3.4 Euclidean), section 3.2 dot product (p 72 printed, eq 3.5), section 3.3 distance (p 75 printed, eq 3.21), section 3.4 angles (p 76 printed, eq 3.24 and eq 3.25), and ch 2 exercise 2.16e rotation as a linear map (p 68 printed), read from ref/mml-book.pdf pages 74, 76-78, 81, and 82, cross verified page-scoped on 2026-09-21 by text-layer extraction (this box has no pdf renderer, and the inherited working notes cover pdf 74 and 76-82 the same way). Parametric segment intersection, the line-circle discriminant, circumcenter by bisectors, the conic sort, affine and homogeneous matrices, and the polar and ellipse pins are derived in the text and pinned exactly by the checks. The trig values behind them are probed on this machine the same day, pins in the listings. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch04`, 51 checks in chapter 4 of the math suite.

5. vectors and vector spaces

Chapter 4 moved single points through coordinates. This chapter works on the space those points live in, in six moves: vectors as tuples with an addition and a scaling that obey the vector space axioms, span and subspace recognized by closure tests, linear independence decided by counting the pivots elimination leaves behind, basis and dimension as the unique coordinates theorem, the three workhorse norms with the triangle inequality and the constants that compare them, and a first change of basis that shows what coordinates really are. Every behavioral claim below is one of the 55 checks in the 3 samples of chapter 05 or a sentence quoted from the mml book draft fetched 2026-09-21. The dsa book runs elimination as a solver over GF(p), this chapter only borrows its pivot count as a recognition tool, see book 9, ch 34 (practical data structures and algorithms, linear algebra).

5.1. vectors and linear combinations

A real vector space is a set V where vector addition forms an abelian group (commutative and associative, with a zero and negatives, the axioms chapter book 2, ch 16 (mathematics for programmers, groups and subgroups) states as loops) and scaling by reals distributes over both additions and fixes $1x = x$ (mml def 2.9, p 37). The space this book computes in is $\mathbb{R}^n$, written as column vectors $\boldsymbol{x} = (x_1, \ldots, x_n)^T$, with the convention that $\boldsymbol{a}\boldsymbol{b}^T$ is the only outer product and $\boldsymbol{a}^T\boldsymbol{b}$ the inner one (mml ex 2.11 with equation 2.64, p 38). A linear combination of vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k$ is any sum $\sum_i \lambda_i \boldsymbol{x}_i$ with real coefficients (mml def 2.11 with equation 2.65, p 40).

Floating point blurs every one of those axioms, so the samples of this chapter run the algebra on exact rationals: a numerator and denominator stored as long long, reduced by gcd, denominator positive by construction. Thirty-five lines of C, and no coefficient ever rounds.

The dry run: $\boldsymbol{a} = (1, 2, 0)^T$, $\boldsymbol{b} = (0, 1, 2)^T$, $\boldsymbol{v} = (2, 5, 2)^T$.

1. The combination $2\boldsymbol{a} + \boldsymbol{b}$ is coordinatewise: $(2 \cdot 1 + 0, 2 \cdot 2 + 1, 2 \cdot 0 + 2) = (2, 5, 2) = \boldsymbol{v}$, exactly.
2. The membership probe $\det[\boldsymbol{a}\ \boldsymbol{b}\ \boldsymbol{v}]$ expands $\boldsymbol{b} \times \boldsymbol{v} = (-8, 4, -2)^T$ and $\boldsymbol{a} \cdot (-8, 4, -2)^T = 0$: zero, so $\boldsymbol{v}$ lies in the plane of $\boldsymbol{a}$ and $\boldsymbol{b}$.
3. The same probe on $\boldsymbol{w} = (1, 0, 0)^T$ reads $\boldsymbol{a} \cdot (0, 2, -1)^T = 4$: nonzero, so $\boldsymbol{w}$ is off the plane.

The two numbers of the section: 0 versus 4. One determinant sign decision separates inside the plane from outside it.

```c
typedef struct {
  ll num;
  ll den;
} rat;

[[nodiscard]] static rat rat_make(ll num, ll den) {
  if (den < 0) {
    num = -num;
    den = -den;
  }
  ll g = gcd_ll(num, den);
  return (rat){num / g, den / g};
}

[[nodiscard]] static rat rat_add(rat a, rat b) {
  return rat_make(a.num * b.den + b.num * a.den, a.den * b.den);
}

[[nodiscard]] static rat rat_sub(rat a, rat b) {
  return rat_make(a.num * b.den - b.num * a.den, a.den * b.den);
}

[[nodiscard]] static rat rat_mul(rat a, rat b) {
  return rat_make(a.num * b.num, a.den * b.den);
}

[[nodiscard]] static rat rat_div(rat a, rat b) {
  return rat_make(a.num * b.den, a.den * b.num);
}

[[nodiscard]] static bool rat_eq(rat a, rat b) {
  return a.num == b.num && a.den == b.den;
}

[[nodiscard]] static bool rat_is_zero(rat a) { return a.num == 0; }
```

listing 39: the exact rational core, every fraction reduced by gcd (source: math/samples/src/Ch05/span.c, lines 23-57)

```c
int main(void) {
  const vec3 a = vec3_make(1, 2, 0);
  const vec3 b = vec3_make(0, 1, 2);
  const vec3 v = vec3_make(2, 5, 2);

  // 2a + b = 2(1,2,0) + (0,1,2) = (2,5,2) = v, all coordinates exact
  CHECK(vec3_eq(vec3_add(vec3_scale(rat_make(2, 1), a), b), v),
        "combination reconstructs v");
  // det[a b v]: b x v = (-8,4,-2) and a . (-8,4,-2) = 0, so v sits in span{a,b}
  CHECK(rat_is_zero(det3(a, b, v)), "det zero puts v in the plane of a and b");
  const vec3 w = vec3_make(1, 0, 0);
  // det[a b w] = a . (b x w) = a . (0,2,-1) = 4 ≠ 0, w is off the plane
  CHECK(rat_eq(det3(a, b, w), rat_make(4, 1)), "det 4 keeps w out of span a b");
```

listing 40: the combination rebuild and the determinant membership probes (source: math/samples/src/Ch05/span.c, lines 115-127)

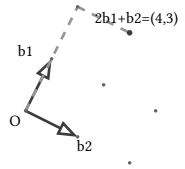


figure 26: linear combinations of b1=(1,2) and b2=(2,-1) as a lattice through the origin, 2b1+b2 marked

5.2. span and subspaces

The span of a set is the collection of all its linear combinations, and it is the smallest subspace containing that set (mml def 2.13, p 44). A subset U of a vector space is a subspace when U contains the zero vector and is closed under addition and under scaling (mml def 2.10, p 39). Closure is the recognition tool: you never enumerate a subspace of $\mathbb{R}^n$, you check that its defining condition survives both operations. Three one-sided tests, and failing any one ends the question.

The dry run: the plane U of $\mathbb{R}^3$ defined by $x + y - z = 0$, and the rival set W of vectors with third coordinate 1.

1. Zero: $0 + 0 - 0 = 0$, so $\mathbf{0}$ is in U.
2. $\boldsymbol{u} = (1, 2, 3)^T$ and $\boldsymbol{s} = (4, -1, 3)^T$ each read 0, so both are in U.
3. Closure: $\boldsymbol{u} + \boldsymbol{s} = (5, 1, 6)^T$ reads $5 + 1 - 6 = 0$, $-3\boldsymbol{u} = (-3, -6, -9)^T$ reads $-3 - 6 + 9 = 0$, and $7\boldsymbol{s} = (28, -7, 21)^T$ reads $28 - 7 - 21 = 0$.
4. W fails twice: the zero vector has third coordinate 0, and doubling $(0, 0, 1)^T$ gives $(0, 0, 2)^T$, third coordinate 2, off the set.

Two structural facts frame the test (mml ex 2.12 and the remark on p 39-40): the solution set of a homogeneous system $\boldsymbol{A}\boldsymbol{x} = \mathbf{0}$ is always a subspace, an inhomogeneous one with a nonzero right side never is, and every subspace of $\mathbb{R}^n$ arises as a homogeneous solution set. The sample pins the homogeneous side of the ledger: the plane U is the solution set of $x + y - z = 0$, it is $\operatorname{span}\big((1, -1, 0)^T, (1, 0, 1)^T\big)$, and $2\boldsymbol{p} + 3\boldsymbol{q} = (5, -2, 3)^T$ reads $5 - 2 - 3 = 0$. Scaling a generator moves nothing: $\operatorname{span}\big((1, 2)^T\big) = \operatorname{span}\big((2, 4)^T\big)$ because each generator is a multiple of the other, $(3, 6)^T = \frac{3}{2} \cdot (2, 4)^T$ sits on the line, $(3, 7)^T$ misses it.

The two numbers of the section: 0 against 2. Every combination probed inside U reads 0 on the form $x + y - z$, and doubling the seed $(0, 0, 1)^T$ of W reads 2 on the third coordinate, the exact step where closure breaks.

```
// the plane U = {x + y - z = 0}
const vec3 zero = vec3_make(0, 0, 0);
const vec3 u = vec3_make(1, 2, 3);
const vec3 s = vec3_make(4, -1, 3);
// plane_val reads 1+2-3 = 0 and 4-1-3 = 0
CHECK(rat_is_zero(plane_val(zero)), "the zero vector satisfies x+y-z=0");
CHECK(rat_is_zero(plane_val(u)), "u=(1,2,3) lies in the plane");
CHECK(rat_is_zero(plane_val(s)), "s=(4,-1,3) lies in the plane");
// u+s = (5,1,6) reads 5+1-6 = 0
CHECK(rat_is_zero(plane_val(vec3_add(u, s))),
      "u+s=(5,1,6) stays in the plane");
// -3u = (-3,-6,-9) reads -3-6+9 = 0
CHECK(rat_is_zero(plane_val(vec3_scale(rat_make(-3, 1), u))),
      "-3u=(-3,-6,-9) stays in the plane");
// 7s = (28,-7,21) reads 28-7-21 = 0
CHECK(rat_is_zero(plane_val(vec3_scale(rat_make(7, 1), s))),
      "7s=(28,-7,21) stays in the plane");

// W = {z = 1} is not a subspace
// the zero vector has third coordinate 0, not 1
CHECK(!rat_eq(zero.x[2], rat_make(1, 1)), "0 is not in W={z=1}");
const vec3 p1 = vec3_make(0, 0, 1);
// p+p = (0,0,2), third coordinate 2, off W
CHECK(!rat_eq(vec3_add(p1, p1).x[2], rat_make(1, 1)),
      "doubling (0,0,1) escapes W");

// the homogeneous solution set is span{(1,-1,0),(1,0,1)} inside U
const vec3 hp = vec3_make(1, -1, 0);
const vec3 hq = vec3_make(1, 0, 1);
// 2p + 3q = (5,-2,3) reads 5-2-3 = 0
CHECK(rat_is_zero(plane_val(vec3_add(vec3_scale(rat_make(2, 1), hp),
                                     vec3_scale(rat_make(3, 1), hq)))),
      "2p+3q stays in the solution plane");
```

listing 41: closure probes on the plane U, the counterexample W, and the homogeneous solution set (source: math/samples/src/Ch05/span.c, lines 141-173)

```
// span{(1,2)} = span{(2,4)} in the z = 0 plane
const vec3 b21 = vec3_make(1, 2, 0);
const vec3 b22 = vec3_make(2, 4, 0);
CHECK(vec3_eq(vec3_scale(rat_make(2, 1), b21), b22) &&
          vec3_eq(vec3_scale(rat_make(1, 2), b22), b21),
      "(2,4)=2(1,2) and (1,2)=(1/2)(2,4) give equal spans");
const vec3 hit = vec3_make(3, 6, 0);
// (3/2)(2,4) = (3,6)
CHECK(vec3_eq(vec3_scale(rat_make(3, 2), b22), hit),
      "(3,6)=(3/2)(2,4) is on the line");
const vec3 off = vec3_make(3, 7, 0);
// lambda = 3/2 predicts (3,6), which misses (3,7)
CHECK(!vec3_eq(vec3_scale(rat_div(off.x[0], b22.x[0]), b22), off),
      "(3,7) is not on the line");
```

listing 42: equal spans, scaling a generator leaves the line in place (source: math/samples/src/Ch05/span.c, lines 175-188)

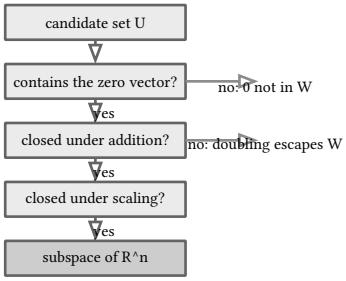


figure 27: the subspace test as a decision flow, the W counterexamples on the fail branches

5.3. linear independence and elimination

Vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k$ are linearly independent when the only combination $\sum_i \lambda_i \boldsymbol{x}_i = \mathbf{0}$ is the one with every coefficient zero; otherwise the set is dependent (mml def 2.12, p 40). For a fixed finite set the question is decidable by elimination: put the vectors side by side as columns, reduce to echelon form, and count pivots. The set is independent exactly when every column carries a pivot, the pivot columns are independent on their own, and every non-pivot column is a combination of the pivot columns before it (mml pp 41-42). Three shortcuts come from the same pages: a zero vector in the set forces dependence, a duplicate forces dependence, and one vector being a combination of the others is equivalent to the whole set being dependent.

The counting bound turns the test into a corollary: m linear combinations of k vectors are linearly dependent whenever m > k (mml p 43). Take k = n and the standard basis of $\mathbb{R}^n$, which reaches every vector: n + 1 vectors in $\mathbb{R}^n$ always lose, because elimination on n rows cannot leave more than n pivots.

The dry run: three fixed sets plus the mml example 2.14 (p 42) with $x_1 = (1, 2, -3, 4)^T$, $x_2 = (1, 1, 0, 2)^T$, $x_3 = (-1, -2, 1, 1)^T$.

1. Example 2.14: elimination on the 4 by 3 matrix leaves a pivot in every column, rank 3 of 3, independent. The echelon form (2.69) is the identity block.
2. The pair $(1, 2)^T, (2, 4)^T$: the second column is twice the first, one pivot, and $2\boldsymbol{a} - \boldsymbol{b} = \mathbf{0}$ exhibits the dependency outright.
3. Three vectors in $\mathbb{R}^2$: rank 2 of 3, and $\boldsymbol{e}_1 + \boldsymbol{e}_2 - (1, 1)^T = \mathbf{0}$. Four vectors in $\mathbb{R}^3$: rank 3 of 4, and $2\boldsymbol{e}_1 - 3\boldsymbol{e}_2 + 5\boldsymbol{e}_3 - (2, -3, 5)^T = \mathbf{0}$.
4. A zero vector or a duplicate in a 2 column set each collapse the rank to 1.

The two numbers of the section: 3 pivots in 3 columns against 2 pivots in 3 columns, independent against dependent, decided by one integer.

```
// fraction gaussian elimination, rank = number of pivots
[[nodiscard]] static int rank_of(mat m) {
  int r = 0;
  for (auto c = 0; c < m.cols && r < m.rows; c++) {
    int p = -1;
    for (auto i = r; i < m.rows; i++) {
      if (!rat_is_zero(m.a[i][c])) {
        p = i;
        break;
      }
    }
    if (p < 0) {
      continue;
    }
    if (p != r) {
      for (auto j = 0; j < m.cols; j++) {
        rat t = m.a[r][j];
        m.a[r][j] = m.a[p][j];
        m.a[p][j] = t;
      }
    }
    for (auto i = r + 1; i < m.rows; i++) {
      rat f = rat_div(m.a[i][c], m.a[r][c]);
      for (auto j = c; j < m.cols; j++) {
        m.a[i][j] = rat_sub(m.a[i][j], rat_mul(f, m.a[r][j]));
      }
    }
    r++;
  }
  return r;
}
```

listing 43: fraction gaussian elimination that returns the pivot count (source: math/samples/src/Ch05/independence.c, lines 125-155)

```
int main(void) {
  // mml ex 2.14 verbatim
  static constexpr ll x14[3][MAXR] = {
      {1, 2, -3, 4}, {1, 1, 0, 2}, {-1, -2, 1, 1}};
  // mml ex 2.15 with b_i = e_i: x1 = b1-2b2+b3-b4, x2 = -4b1-2b2+4b4,
  // x3 = 2b1+3b2-b3-3b4, x4 = 17b1-10b2+11b3+b4
  static constexpr ll x15[4][MAXR] = {
      {1, -2, 1, -1}, {-4, -2, 0, 4}, {2, 3, -1, -3}, {17, -10, 11, 1}};
  // mml ex 2.16, the set A of (2.80)
  static constexpr ll seta[3][MAXR] = {
      {1, 2, 3, 4}, {2, -1, 0, 2}, {1, 1, 0, -4}};

  // ex 2.14: elimination leaves pivots in all three columns
  CHECK(rank_of(mat_from_cols(x14, 4, 3)) == 3,
        "ex 2.14 vectors are independent");
```

listing 44: the fixtures, mml examples 2.14, 2.15 and the set A as fixed columns (source: math/samples/src/Ch05/independence.c, lines 166-180)

```
    // a dependent pair in R^2: (2,4) = 2(1,2), one pivot survives
    static constexpr ll pair[2][MAXR] = {{1, 2}, {2, 4}};
    CHECK(rank_of(mat_from_cols(pair, 2, 2)) == 1,
          "duplicated direction collapses rank to 1");
    const vec4 a = vec4_make(1, 2, 0, 0);
    const vec4 b = vec4_make(2, 4, 0, 0);
    // 2(1,2) - (2,4) = (0,0)
    CHECK(vec4_is_zero(vec4_sub(vec4_scale(rat_make(2, 1), a), b)),
          "2a-b=(0,0) exhibits the dependency");

    // {0, (3,4)}: the zero column adds no pivot
    static constexpr ll zv[2][MAXR] = {{0, 0}, {3, 4}};
    CHECK(rank_of(mat_from_cols(zv, 2, 2)) == 1,
          "a zero vector forces dependence");

    // {(3,4),(3,4)}: the duplicate adds no pivot
    static constexpr ll dup[2][MAXR] = {{3, 3}, {4, 4}};
    CHECK(rank_of(mat_from_cols(dup, 2, 2)) == 1,
          "a duplicate vector forces dependence");

    // 3 vectors in R^2, the n+1 case: rank 2 < 3
    static constexpr ll trio2[3][MAXR] = {{1, 0}, {0, 1}, {1, 1}};
    CHECK(rank_of(mat_from_cols(trio2, 2, 3)) == 2,
          "3 vectors in R^2 lose, rank 2 of 3");
    const vec4 e1 = vec4_make(1, 0, 0, 0);
    const vec4 e2 = vec4_make(0, 1, 0, 0);
    const vec4 t = vec4_make(1, 1, 0, 0);
    // e1 + e2 - (1,1) = (0,0)
    CHECK(vec4_is_zero(vec4_sub(vec4_add(e1, e2), t)),
          "e1+e2=(1,1) exhibits the dependency");

    // 4 vectors in R^3, the n+1 case: rank 3 < 4
    static constexpr ll quad3[4][MAXR] = {
        {1, 0, 0}, {0, 1, 0}, {0, 0, 1}, {2, -3, 5}};
    CHECK(rank_of(mat_from_cols(quad3, 3, 4)) == 3,
          "4 vectors in R^3 lose, rank 3 of 4");
    const vec4 e3 = vec4_make(0, 0, 1, 0);
    const vec4 last = vec4_make(2, -3, 5, 0);
    // 2e1 - 3e2 + 5e3 - (2,-3,5) = (0,0,0)
    CHECK(
        vec4_is_zero(vec4_sub(vec4_add(vec4_scale(rat_make(2, 1), e1),
                                       vec4_add(vec4_scale(rat_make(-3, 1), e2),
                                                vec4_scale(rat_make(5, 1), e3))),
                              last)),
        "2e1-3e2+5e3=(2,-3,5) exhibits the dependency");
```

listing 45: fixed-set verdicts, duplicates, the zero vector, and n+1 vectors losing in R^n (source: math/samples/src/Ch05/independence.c, lines 182-226)

set	ambient	columns	pivots	verdict
ex 2.14	$\mathbb{R}^4$	3	3	independent
$(1,2),(2,4)$	$\mathbb{R}^2$	2	1	dependent, $2a - b = 0$
$\mathbf{0},(3,4)$	$\mathbb{R}^2$	2	1	dependent, zero column
$e_1, e_2, (1,1)$	$\mathbb{R}^2$	3	2	dependent, 3 in 2
$e_1, e_2, e_3, (2,-3,5)$	$\mathbb{R}^3$	4	3	dependent, 4 in 3
set A (2.80)	$\mathbb{R}^4$	3	3	independent, short

PITFALL: NEVER DECIDE INDEPENDENCE IN DOUBLES

Elimination in floating point turns exact zeros into residues like 1e-17, and the pivot search then answers a question you did not ask. The samples run the fractions exactly. When doubles are unavoidable, as in the physics chapters, compare each residual against a stated epsilon the way book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) teaches, and pin the epsilon in a comment.

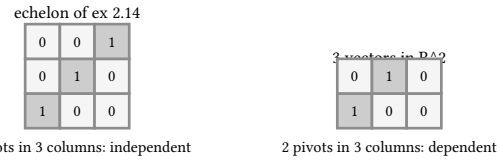


figure 28: pivot patterns, every column carrying a pivot against one column left over

5.4. basis and dimension

A basis of a vector space is a generating set that is also linearly independent (mml def 2.14, p 44): every vector is a combination of the basis, and no basis vector is a combination of the others. Coordinates in a basis are then unique (equation 2.77, p 45): if two coefficient vectors both rebuilt $\boldsymbol{x}$, their difference would be a nonzero combination giving zero, which independence forbids. Every vector space has a basis, all bases of one space have the same size, and that size is the dimension (mml pp 45-46). For subspaces, $\dim U \leq \dim V$ with equality exactly when $U = V$. To build a basis of a span, run elimination and keep the pivot columns; mml example 2.17 (pp 46-47) does exactly that and keeps $\{x_1, x_2, x_4\}$.

The dry run: the set A of mml example 2.16, equation (2.80), p 45, with $a_1 = (1, 2, 3, 4)^T$, $a_2 = (2, -1, 0, 2)^T$, $a_3 = (1, 1, 0, -4)^T$ in $\mathbb{R}^4$.

1. Elimination: rank 3, the set is independent.
2. Reachability of $\boldsymbol{e}_1 = (1, 0, 0, 0)^T$: the leading 3 by 3 block of $[a_1\ a_2\ a_3]$ has determinant 9, and Cramer on it returns $\lambda_1 = 0$, $\lambda_2 = \frac{1}{3}$, $\lambda_3 = \frac{1}{3}$.
3. The fourth equation reads $4 \cdot 0 + 2 \cdot \frac{1}{3} - 4 \cdot \frac{1}{3} = -\frac{2}{3}$, so $\boldsymbol{e}_1$ is not in the span. Independent but short: a basis of $\mathbb{R}^4$ needs exactly 4 vectors.

The companion example 2.15 (mml pp 43-44) shows the other direction: from four vectors whose first three carry rank 3, back substitution reads the fourth off the others as $x_4 = -7x_1 - 15x_2 - 18x_3$, and $\{x_1, x_2, x_3\}$ is then a basis of the span. The sample verifies that relation coordinate by coordinate, all four rows reading 0.

The two numbers of the section: 3 pivots against the residue −2/3. The rank says the set is independent, the residue says it stops short of generating, and the gap between them is exactly one basis vector.

```
// ex 2.15: x4 + 7x1 + 15x2 + 18x3 = 0 coordinate by coordinate
const vec4 y1 = vec4_make(1, -2, 1, -1);
const vec4 y2 = vec4_make(-4, -2, 0, 4);
const vec4 y3 = vec4_make(2, 3, -1, -3);
const vec4 y4 = vec4_make(17, -10, 11, 1);
const vec4 rel =
    vec4_add(y4, vec4_add(vec4_scale(rat_make(7, 1), y1),
                          vec4_add(vec4_scale(rat_make(15, 1), y2),
                                   vec4_scale(rat_make(18, 1), y3))));
CHECK(vec4_is_zero(rel), "x4+7x1+15x2+18x3=0 in ex 2.15");
// the first three of ex 2.15 carry rank 3, x4 hangs off them
CHECK(rank_of(mat_from_cols(x15, 4, 3)) == 3,
      "x1,x2,x3 of ex 2.15 are independent");
```

listing 46: ex 2.15, the fourth vector hangs off the first three (source: math/samples/src/Ch05/independence.c, lines 228-240)

```
// ex 2.16, set A: three pivots in R^4
CHECK(rank_of(mat_from_cols(seta, 4, 3)) == 3, "set A (2.80) is independent");

// is (1,0,0,0) reachable? cramer on the leading 3x3 block of A
const vec4 a1 = vec4_make(1, 2, 3, 4);
const vec4 a2 = vec4_make(2, -1, 0, 2);
const vec4 a3 = vec4_make(1, 1, 0, -4);
const rat d = det3_cols(a1, a2, a3);
// det[[1,2,1],[2,-1,1],[3,0,0]] = 3*(2*1 - 1*(-1)) = 9
CHECK(rat_eq(d, rat_make(9, 1)), "leading 3x3 determinant is 9");
const vec4 e1f = vec4_make(1, 0, 0, 0);
const rat l1 = rat_div(det3_cols(e1f, a2, a3), d);
const rat l2 = rat_div(det3_cols(a1, e1f, a3), d);
const rat l3 = rat_div(det3_cols(a1, a2, e1f), d);
// lambda1 = 0
CHECK(rat_eq(l1, rat_make(0, 1)), "cramer lambda1 is 0");
// lambda2 = 3/9 = 1/3 and lambda3 = 3/9 = 1/3
CHECK(rat_eq(l2, rat_make(1, 3)) && rat_eq(l3, rat_make(1, 3)),
      "cramer lambda2 and lambda3 are both 1/3");
// the 4th equation reads 4*0 + 2*(1/3) - 4*(1/3) = -2/3, so e1 is not in span
// A
const rat resid4 =
    rat_add(rat_add(rat_mul(rat_make(4, 1), l1), rat_mul(rat_make(2, 1), l2)),
            rat_mul(rat_make(-4, 1), l3));
CHECK(rat_eq(resid4, rat_make(-2, 3)), "the 4th coordinate reads -2/3");
CHECK(!rat_is_zero(resid4),
      "set A does not generate e1, independent but short");
```

listing 47: set A, cramer coordinates 0, 1/3, 1/3 and the fourth-row residue −2/3 (source: math/samples/src/Ch05/independence.c, lines 242-268)

NOTE: DIMENSION COUNTS DIRECTIONS, NOT ELEMENTS

The line $\text{span}((0,1)^T)$ has infinitely many vectors and dimension 1, and $\mathbb{R}^4$ has infinitely many vectors and dimension 4 (mml pp 45-46). A basis of $\mathbb{R}^n$ always has exactly n vectors: fewer independent ones do not generate, and any n + 1 lose by the counting bound of the previous section.

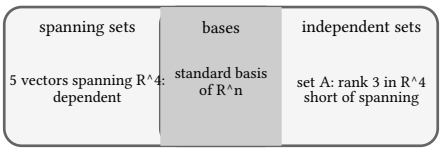


figure 29: spanning sets against independent sets, bases in the overlap

5.5. norms, unit vectors, and equivalence

A norm on $\mathbb{R}^n$ is a function with absolute homogeneity $\|\lambda \boldsymbol{v}\| = |\lambda| \, \|\boldsymbol{v}\|$, the triangle inequality $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$, and positive definiteness, $\|\boldsymbol{v}\| = 0$ only at $\boldsymbol{v} = \boldsymbol{0}$ (mml def 3.1, p 71). Three workhorses carry all the numerics of this book. The manhattan norm $\|\boldsymbol{v}\|_1 = \sum_i |v_i|$ (mml example 3.1 with equation 3.3, p 71), the euclidean norm $\|\boldsymbol{v}\|_2 = \sqrt{\boldsymbol{v}^T \boldsymbol{v}}$ (mml example 3.2 with equation 3.4, p 72), and the max norm $\|\boldsymbol{v}\|_\infty = \max_i |v_i|$. The first two are the draft's, with its remark to use the euclidean norm by default (p 72). The max norm appears nowhere in the draft: its three axioms are defined here and verified by the sample.

The dry run on the fixed vector $\boldsymbol{v} = (3, -4, 12)^T$.

1. Values: $19 = 3 + 4 + 12$, $13 = \sqrt{169}$ from $9 + 16 + 144$, $12 = \max(3, 4, 12)$. The chain $12 < 13 < 19$ orders all three norms on one vector.

2. Homogeneity: $7\boldsymbol{v}$ has $\mathtt{norm}_1 = 133 = 7 \cdot 19$ and $\mathtt{norm}_\infty = 84 = 7 \cdot 12$, and $-2\boldsymbol{v}$ has $\mathtt{norm}_2 = \sqrt{676} = 26 = 2 \cdot 13$.
3. Unit vectors: $\mathtt{norm}_2(\boldsymbol{v}) = 13$ is exact because 169 is a perfect square under the root, and the three squares of $\boldsymbol{v}/13$ then round to a sum of exactly 1, so the scaled vector sits on the l2 shell. $\boldsymbol{v}/19$ has $\mathtt{norm}_1$ exactly 1. The l1 sum of $\boldsymbol{v}/13$ lands one ulp above the rational $\frac{19}{13}$: 1.4615384615384617 against 1.4615384615384615.
4. Triangle equality: with $\boldsymbol{p} = (1, 2, 2)^T$ and $\boldsymbol{q} = 2\boldsymbol{p}$, all three norms hit equality, $9 = 3 + 6$, $15 = 5 + 10$, $6 = 2 + 4$. With $\boldsymbol{r} = (2, 0, -1)^T$, which is not a nonnegative multiple of $\boldsymbol{p}$, all three are strict: $\sqrt{14} = 3.7416573867739413 < 3 + \sqrt{5} = 5.23606797749979$, $6 < 8$, $3 < 4$.

Equality in the triangle inequality holds when one side is a nonnegative multiple of the other. For the euclidean norm that is equality in Cauchy-Schwarz. For the manhattan norm the exact condition is derived, not cited: per coordinate, $|a + b| = |a| + |b|$ holds exactly when $ab \geq 0$, so equality means no coordinatewise sign cancellation, and a nonnegative multiple guarantees it. The sample shows the guarantee and one strict counterexample.

Norm equivalence: on a fixed $\mathbb{R}^n$ any two norms compare through constants, so convergence or boundedness questions get the same answers in every one of them. On $\mathbb{R}^3$ the chain is $\mathtt{norm}_\infty \leq \mathtt{norm}_2 \leq \mathtt{norm}_1$ termwise, $\mathtt{norm}_2 \leq \sqrt{3}\mathtt{norm}_\infty$ and $\mathtt{norm}_1 \leq \sqrt{3}\mathtt{norm}_2$, the sum bound $\sum|v_i| \leq \sqrt{n}\mathtt{norm}_2$ on $|\boldsymbol{v}|$ against the all-ones vector that chapter book 2, ch 7 (mathematics for programmers, inner products and projections) names Cauchy-Schwarz, and $\mathtt{norm}_1 \leq 3\mathtt{norm}_\infty$ termwise. The draft does not state equivalence. On the fixed vector: 19 sits under $\sqrt{3} \cdot 13 = 22.516660498395403$, 13 under $\sqrt{3} \cdot 12 = 20.784609690826528$, and 19 under 36.

```c
// manhattan norm: sum of absolute coordinates
[[nodiscard]] static double norm1(vec3 v) {
  return fabs(v.x[0]) + fabs(v.x[1]) + fabs(v.x[2]);
}

// euclidean norm: square root of the dot product with itself
[[nodiscard]] static double norm2(vec3 v) {
  return sqrt(v.x[0] * v.x[0] + v.x[1] * v.x[1] + v.x[2] * v.x[2]);
}

// max norm: largest absolute coordinate
[[nodiscard]] static double normi(vec3 v) {
  double m = fabs(v.x[0]);
  if (fabs(v.x[1]) > m) {
    m = fabs(v.x[1]);
  }
  if (fabs(v.x[2]) > m) {
    m = fabs(v.x[2]);
  }
  return m;
}
```

listing 48: the three norms as three straight functions over fabs and sqrt (source: math/samples/src/Ch05/norms.c, lines 37-57)

```
int main(void) {
  constexpr vec3 v = {{3.0, -4.0, 12.0}};

  // |3| + 4 + 12 = 19, an integer sum, exact in doubles
  CHECK(norm1(v) == 19.0, "l1 of (3,-4,12) is 19");
  // sqrt(9 + 16 + 144) = sqrt(169) = 13, a perfect square, exact
  CHECK(norm2(v) == 13.0, "l2 of (3,-4,12) is 13");
  // max(3, 4, 12) = 12
  CHECK(normi(v) == 12.0, "linf of (3,-4,12) is 12");
  // 12 < 13 < 19 orders the three norms on this v
  CHECK(normi(v) < norm2(v) && norm2(v) < norm1(v),
        "12 < 13 < 19 orders the norms");

  // absolute homogeneity: |7|*19 = 133, |-2|*13 = 26 via sqrt(676), |7|*12 = 84
  const vec3 v7 = vec3_scale(7.0, v);
  const vec3 vm2 = vec3_scale(-2.0, v);
  CHECK(norm1(v7) == 133.0 && norm2(vm2) == 26.0 && normi(v7) == 84.0,
        "scaling by 7 and -2 scales every norm");

  // equivalence upper bounds on R^3, all with slack on this v:
  // 19 ≤ sqrt(3)*13 = 22.516660498395403, 13 ≤ sqrt(3)*12
  // = 20.784609690826528, 19 ≤ 3*12 = 36
  const double s3 = sqrt(3.0);
  CHECK(norm1(v) ≤ s3 * norm2(v) && norm2(v) ≤ s3 * normi(v) &&
            norm1(v) ≤ 3.0 * normi(v),
        "the sqrt(n) upper bounds hold with slack");
  // and the lower chain 12 ≤ 13 ≤ 19 always holds, in any dimension
  CHECK(normi(v) ≤ norm2(v) && norm2(v) ≤ norm1(v),
        "the lower chain linf ≤ l2 ≤ l1 holds");

  // unit vectors: dividing by the norm is the only move that lands on the shell
  const vec3 u = vec3_scale(1.0 / 13.0, v);
  // (3/13)^2 + (4/13)^2 + (12/13)^2 rounds back to exactly 1
  CHECK(norm2(u) == 1.0, "v/13 has l2 exactly 1");
  // the l1 sum lands one ulp up: 1.4615384615384617 against 19/13
  // = 1.4615384615384615
  CHECK(fabs(norm1(u) - 19.0 / 13.0) < 1e-12,
        "v/13 has l1 19/13 within an ulp");
  CHECK(normi(u) == 12.0 / 13.0, "v/13 has linf 12/13");
```

listing 49: values, homogeneity, the equivalence bounds, and unit vectors (source: math/samples/src/Ch05/norms.c, lines 59-97)

```
// triangle inequality, equality case: q = 2p points the same way
const vec3 p = vec3_make(1.0, 2.0, 2.0);
const vec3 q = vec3_scale(2.0, p);
const vec3 pq = vec3_add(p, q);
// l2: sqrt(81) = 9 = 3 + 6
CHECK(norm2(pq) == norm2(p) + norm2(q), "same direction: l2 equality 9=3+6");
// l1: 15 = 5 + 10
CHECK(norm1(pq) == norm1(p) + norm1(q),
      "same direction: l1 equality 15=5+10");
// linf: 6 = 2 + 4
CHECK(normi(pq) == normi(p) + normi(q),
      "same direction: linf equality 6=2+4");

// strict case: r = (2,0,-1) is not a nonnegative multiple of p
const vec3 r = vec3_make(2.0, 0.0, -1.0);
const vec3 pr = vec3_add(p, r);
// l2: sqrt(14) = 3.7416573867739413 < 3 + sqrt(5) = 5.23606797749979
CHECK(norm2(pr) < norm2(p) + norm2(r),
      "opposite-side r: l2 strict 3.74 < 5.24");
// l1: |3|+|2|+|1| = 6 < 5 + 3 = 8
CHECK(norm1(pr) < norm1(p) + norm1(r), "opposite-side r: l1 strict 6 < 8");
// linf: 3 < 2 + 2 = 4
CHECK(normi(pr) < normi(p) + normi(r), "opposite-side r: linf strict 3 < 4");

// positive definiteness: all three norms vanish exactly at the zero vector
constexpr vec3 z = {{0.0, 0.0, 0.0}};
CHECK(norm1(z) == 0.0 && norm2(z) == 0.0 && normi(z) == 0.0,
      "all three norms vanish only at zero");
```

listing 50: triangle equality, the strict case, and positive definiteness (source: math/samples/src/Ch05/norms.c, lines 104-131)

VERIFY: THE EQUIVALENCE CONSTANTS ON ONE VECTOR

Every bound in the chain is checked against the fixed $\boldsymbol{v} = (3, -4, 12)^T$ in norms.c: the lower chain 12 <= 13 <= 19 exactly, and the three upper bounds with their slack, 19 <= 22.516660498395403, 13 <= 20.784609690826528, 19 <= 36. The constants $\sqrt{3}$ and 3 depend on the dimension, not on the vector.

PITFALL: NORMALIZE WITH THE MATCHING NORM

Dividing by one norm lands on that norm's unit shell only. $\boldsymbol{v}/13$ has $\texttt{norm}_2 = 1$ but $\texttt{norm}_1 = \frac{19}{13}$ still, one ulp from the rational value in doubles. A distance test written with $\texttt{norm}_2$ against a threshold tuned for $\texttt{norm}_1$ is wrong by up to the equivalence constant, here a factor of $\sqrt{3}$.

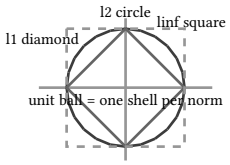


figure 30: unit balls of the three norms, diamond, circle, square

5.6. what a basis buys a programmer

An ordered basis $B = (\boldsymbol{b}_1, \ldots, \boldsymbol{b}_n)$ assigns to every vector $\boldsymbol{x}$ its unique coordinates $\lambda_1, \ldots, \lambda_n$ with $\boldsymbol{x} = \sum_i \lambda_i \boldsymbol{b}_i$ (mml def 2.18 with equations 2.89-2.91, p 50). Mml's figure 2.8 draws the punchline: the same vector $x = [2, 2]^T$ reads $[1.09, 0.72]^T$ in a tilted basis. The vector does not move, the name does.

The purchase is a lossless compression. Fix a basis and every vector in an n dimensional space is exactly n numbers, and every linear map is exactly n^2 numbers: the transformation matrix whose columns are the images of the basis vectors (mml def 2.19, p 51). Games store positions in the standard basis and physics in whatever frame the level designer tilted, and both are the same data wearing different names.

The dry run: mml example 2.20 (p 51), $\boldsymbol{x} = (2,3)^T$ in the basis $\boldsymbol{b}_1 = (1,-1)^T$, $\boldsymbol{b}_2 = (1,1)^T$.

1. Solve $\alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2 = \boldsymbol{x}$: the two equations $\alpha + \beta = 2$ and $-\alpha + \beta = 3$ add to $2\beta = 5$, so $\beta = \frac{5}{2}$ and $\alpha = -\frac{1}{2}$.
2. Rebuild exactly: $-\frac{1}{2}(1,-1)^T + \frac{5}{2}(1,1)^T = \left(-\frac{1}{2} + \frac{5}{2}, \frac{1}{2} + \frac{5}{2}\right) = (2,3)^T$.
3. Packaged as matrices: the basis matrix $\boldsymbol{B} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$ with columns $\boldsymbol{b}_1, \boldsymbol{b}_2$ carries tilted coordinates to standard ones, $\boldsymbol{B}\left(-\frac{1}{2}, \frac{5}{2}\right)^T = (2,3)^T$, and its inverse $\frac{1}{2}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$ is the solve as a table, reading $\left(-\frac{1}{2}, \frac{5}{2}\right)^T$ back off $(2,3)^T$. Mml example 2.21 (pp 51-52) builds such a matrix column by column from the images of the basis vectors, and the text uses it to map coordinates in one ordered basis to coordinates in another.

The two numbers of the section: the same vector is named $(2,3)$ in the standard basis and $\left(-\frac{1}{2}, \frac{5}{2}\right)$ in the tilted one, and both names rebuild it exactly.

```c
// change of basis, mml ex 2.20: x=(2,3) in b1=(1,-1), b2=(1,1)
const vec3 cb1 = vec3_make(1, -1, 0);
const vec3 cb2 = vec3_make(1, 1, 0);
const vec3 cx = vec3_make(2, 3, 0);
// (-1/2)(1,-1) + (5/2)(1,1) = (-1/2+5/2, 1/2+5/2) = (2,3) exactly
CHECK(vec3_eq(vec3_add(vec3_scale(rat_make(-1, 2), cb1),
                       vec3_scale(rat_make(5, 2), cb2)),
              cx),
      "coordinates (-1/2,5/2) rebuild x=(2,3)");
```

listing 51: change of basis, mml ex 2.20 in exact fractions (source: math/samples/src/Ch05/span.c, lines 190-198)

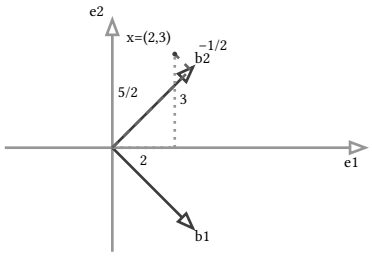


figure 31: the same vector read in two bases, standard coefficients 2 and 3, tilted coefficients −1/2 and 5/2

The transformation matrix is the subject of the next chapter, book 2, ch 6 (mathematics for programmers, matrices and linear systems), where this 2 by 2 preview becomes the general theory of linear maps.

sources: mml-book draft 2024-01-15, ch 2.4-2.5 pp 37-47 (vector spaces, subspaces, linear independence, elimination test, basis and dimension), ch 2 pp 50-52 (coordinates in a basis, transformation matrix, examples 2.20 and 2.21), ch 3.1-3.2 pp 71-72 (norm definition, manhattan and euclidean norms, dot product, the default-euclidean remark), cross-checked 2026-09-21 against the verbatim extracted text of pdf pp 43-58 and 77-80 of ref/mml-book.pdf, the page renders themselves being unavailable on this machine. The max norm and the norm equivalence constants are not in the draft, they are defined and derived inline and pinned by the samples. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch05`, 55 checks in chapter 05 of the math suite, 19 in span, 17 in independence, 19 in norms, zero skipped, format and asan clean.

6. matrices and linear systems

A system of linear equations, the matrix notation that compresses it, and the elimination algorithm that solves it. This chapter runs six moves: the row picture and the column picture of one fixed 2x2 system with its three possible outcomes, the algebra of matrices with non-commutativity pinned on one pair, gaussian elimination with partial pivoting on doubles judged against exact rational precomputation, the elimination record reused as an LU factorization with two triangular solves, rank and the null space with the rank-nullity count, and the inverse with its adjugate shortcut and its round trip. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 06 or a sentence quoted from the mml-book draft of 2024-01-15, fetched 2026-09-21 and pinned in the anchor sheet ref/mml/anchors-la.md. The sibling dsa book runs elimination modulo a prime for contest determinism (book 9, ch 34 (practical data structures and algorithms, linear algebra)), this chapter runs it on binary64 doubles with python Fractions as the exact referee, the same split between measurement and arithmetic that chapter book 2, ch 1 (mathematics for programmers, floating point anatomy) set up for single numbers.

6.1. systems of linear equations

A factory produces n products from m resources, one unit of product j consumes a_{ij} units of resource i, and a production plan $x_1, \ldots, x_n$ must exactly consume the stock b_i. That bookkeeping demand, mml Example 2.1 [printed pp 19-21 / pdf pp 25-27], is the system

$$a_{i1}x_1 + a_{i2}x_2 + \ldots + a_{in}x_n = b_i, \quad i = 1, \ldots, m,$$

and mml states the payoff plainly: “In general, for a real-valued system of linear equations we obtain either no, exactly one, or infinitely many solutions.” This section pins all three outcomes on one family and reads them twice, once as rows and once as columns.

The fixed 2x2 fixture for the whole chapter is mml’s worked illustration [printed p 21 / pdf p 27]:

$$4x_1 + 4x_2 = 5, \quad 2x_1 - 4x_2 = 1$$

The row picture reads each equation as a line in the x_1x_2 plane, and the solution as where the lines meet. mml’s remark on the geometry: the solution set “can be a line (if the linear equations describe the same line), a point, or empty (when the lines are parallel)”. The column picture reads the same system as one vector equation, how much of column $(4, 2)$ plus how much of column $(4, -4)$ builds the right side $(5, 1)$.

The dry run: eliminate x_2 from the second equation using the first. The multiplier is $\frac{2}{4} = \frac{1}{2}$, the second row becomes $\left(0, -6 \mid -\frac{3}{2}\right)$, so $x_2 = \frac{-\frac{3}{2}}{-6} = \frac{1}{4}$, and back in the first equation $4x_1 = 5 - 1 = 4$ gives $x_1 = 1$. The two lines cross in exactly one point, $\left(1, \frac{1}{4}\right)$. The column reading of the same numbers: $1(4, 2) + \frac{1}{4}(4, -4) = (5, 1)$, one whole first column and a quarter of the second.

1. The multiplier $\frac{1}{2}$ is exact in binary, so the doubles trace the fractions bit for bit.
2. Both intermediate values stay integers or halves: -6 and $-\frac{3}{2}$ are exactly representable, no rounding enters the trace.
3. The answer $\left(1, \frac{1}{4}\right)$ satisfies both original equations with residual exactly 0, checks 1 through 4 of systems.c.

The three outcomes come from mml Example 2.2 [printed pp 19-21 / pdf pp 25-27], one family with the same first two equations $x_1 + x_2 + x_3 = 3$ and $x_1 - x_2 + 2x_3 = 2$ and three different thirds. Elimination with no row swaps classifies all three, and because every value is an integer or a half, binary64 holds the trace exactly:

third equation	last row after elimination	outcome	solution set
$2x_1 + 3x_3 = 1$	$0 = -4$	none	empty
$x_2 + x_3 = 2$	$\frac{3}{2}x_3 = \frac{3}{2}$	unique	$(1, 1, 1)$
$2x_1 + 3x_3 = 5$	$0 = 0$	infinite	$\left(\frac{5}{2}, \frac{1}{2}, 0\right) + t\left(-\frac{3}{2}, \frac{1}{2}, 1\right)$

The no-solution case is a derived contradiction: adding the first two equations yields $2x_1 + 3x_3 = 5$, and a third equation claiming $2x_1 + 3x_3 = 1$ says $5 = 1$, which elimination compresses to the dead row $0 = -4$. The infinite case is the mirror image, the third equation is exactly the sum of the first two so it adds no information, one variable runs free and the solution set is a line through $\left(\frac{5}{2}, \frac{1}{2}, 0\right)$ with direction $\left(-\frac{3}{2}, \frac{1}{2}, 1\right)$. The two-number play of this section: the family's unique member has 1 solution point, its contradiction member has 0, and nothing in between exists, because a linear system cannot have 2.

```c
// forward elimination on an augmented 3x4 system, no row swaps: the fixtures
// are integer matrices whose pivots are 1, -2, 2, so every intermediate value
// is an integer or a half and binary64 holds it exactly
static enum outcome elim3(double a[3][4], double x[3]) {
  for (int k = 0; k < 2; k++) {
    for (int i = k + 1; i < 3; i++) {
      double m = a[i][k] / a[k][k];
      for (int j = k; j < 4; j++)
        a[i][j] -= m * a[k][j];
    }
  }
  bool dead_row =
      fabs(a[2][0]) < 1e-12 && fabs(a[2][1]) < 1e-12 && fabs(a[2][2]) < 1e-12;
  if (dead_row)
    // 0 = 0 buys a free variable, 0 = c with c ≠ 0 buys a contradiction
    return near(a[2][3], 0.0, 1e-12) ? OUT_INFINITE : OUT_NONE;
  x[2] = a[2][3] / a[2][2];
  x[1] = (a[1][3] - a[1][2] * x[2]) / a[1][1];
  x[0] = (a[0][3] - a[0][1] * x[1] - a[0][2] * x[2]) / a[0][0];
  return OUT_UNIQUE;
}
```

listing 52: the outcome classifier, elimination without swaps on exact fixtures (source: math/samples/src/Ch06/systems.c, lines 27-47)

```
// the Example 2.2 family: same first two equations, three different thirds
double none[3][4] = {{1, 1, 1, 3}, {1, -1, 2, 2}, {2, 0, 3, 1}};
double uniq[3][4] = {{1, 1, 1, 3}, {1, -1, 2, 2}, {0, 1, 1, 2}};
double inf[3][4] = {{1, 1, 1, 3}, {1, -1, 2, 2}, {2, 0, 3, 5}};
double inf_pristine[3][4];
memcpy(inf_pristine, inf, sizeof inf);
double x[3];

// (2.4): the bottom row reduces to 0 = -4, no solution
CHECK(elim3(none, x) == OUT_NONE, "system (2.4) has no solution");
CHECK(near(none[2][3], -4.0, 1e-12), "contradiction reads 0 = -4");

// (2.5): unique solution (1, 1, 1)
CHECK(elim3(uniq, x) == OUT_UNIQUE, "system (2.5) has one solution");
CHECK(near(x[0], 1.0, 1e-12) && near(x[1], 1.0, 1e-12) &&
          near(x[2], 1.0, 1e-12),
      "solution is (1, 1, 1)");

// (2.6): the bottom row reduces to 0 = 0, one free parameter
CHECK(elim3(inf, x) == OUT_INFINITE, "system (2.6) has infinitely many");

// pinned members of the solution line x = (5/2, 1/2, 0) + t (-3/2, 1/2, 1)
double t0[3] = {2.5, 0.5, 0.0};
double t2[3] = {-0.5, 1.5, 2.0};
CHECK(residual3(inf_pristine, t0) < 1e-12, "t = 0 member satisfies all rows");
CHECK(residual3(inf_pristine, t2) < 1e-12, "t = 2 member satisfies all rows");
```

listing 53: the Example 2.2 family, three thirds, three outcomes, two pinned members of the solution line (source: math/samples/src/Ch06/systems.c, lines 89-114)

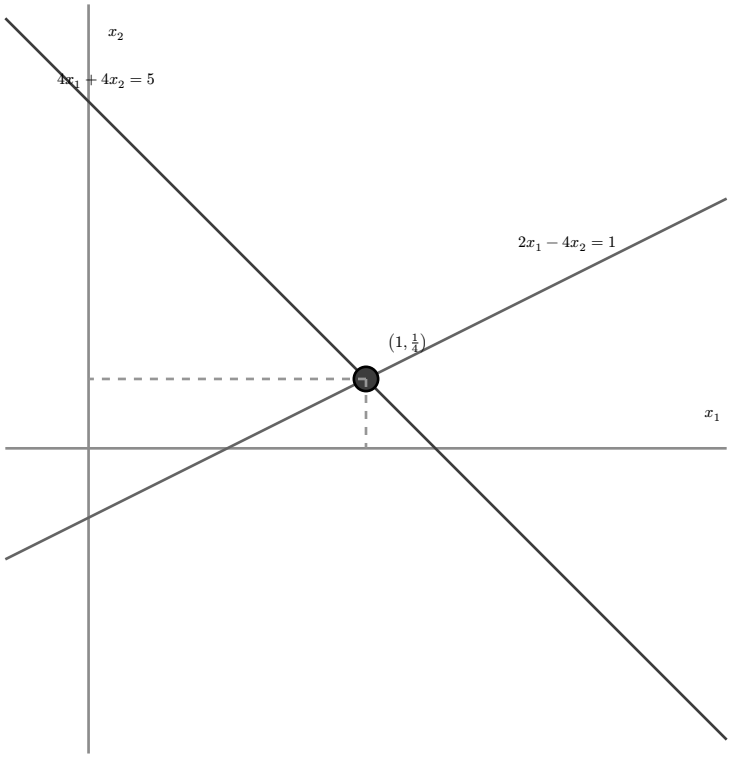


figure 32: two lines, one crossing point at (1, 1/4), the row picture of the fixed system

6.2. matrix algebra

Stack the coefficients of the system into a rectangle and the notation collapses. mml Definition 2.1 [printed p 22 / pdf p 28]: "With $m, n \in \mathbb{N}$ a real-valued (m, n) matrix A is an $m \cdot n$-tuple of elements a_{ij}, $i = 1, \ldots, m$, $j = 1, \ldots, n$, which is ordered according to a rectangular scheme consisting of m rows and n columns". Rows are $(1, n)$ matrices, columns are $(m, 1)$ matrices, and both are just vectors with a shape, the same objects chapter book 2, ch 5 (mathematics for programmers, vectors and vector spaces) tested for independence. The system becomes $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ and, in mml's words from the compact representations of section 2.2.4 [printed p 26 / pdf p 32], "the product Ax is a (linear) combination of the columns of A", which is the column picture of the previous section promoted to notation.

The operations are deliberately boring. Addition and scalar multiplication touch every entry independently, $a_{ij} + b_{ij}$ and λa_{ij}. The transpose, mml Definition 2.4 [printed p 25 / pdf p 31], flips rows with columns: "For $A \in \mathbb{R}^{m \times n}$ the matrix $B \in \mathbb{R}^{n \times m}$ with $b_{ij} = a_{ji}$ is called the transpose of A." The identity, mml Definition 2.2 [printed pp 23-24 / pdf pp 29-30], is the matrix of doing nothing: "In $\mathbb{R}^{n \times n}$, we define the identity matrix I_n as the $n \times n$-matrix containing 1 on the diagonal and 0 everywhere else", and it satisfies $I_m \boldsymbol{A} = \boldsymbol{A} I_n = \boldsymbol{A}$. The one operation with structure is the product, mml eq 2.13 [printed pp 22-24 / pdf pp 28-30], an all-pairs grid of dot products:

$$c_{ij} = \sum_{l=1}^{n} a_{il} b_{lj}, \quad i = 1, \ldots, m, \quad j = 1, \ldots, k$$

Entry c_{ij} is row i of $\boldsymbol{A}$ dotted against column j of $\boldsymbol{B}$, and the shapes must chain: $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ times $\boldsymbol{B} \in \mathbb{R}^{n \times k}$ lands in $\mathbb{R}^{m \times k}$, the neighboring dimensions must agree or the dots have nothing to sum over.

The dry run: mml Example 2.3 [printed p 23 / pdf p 29] pins the whole structure on

$$\boldsymbol{A} = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{pmatrix} \in \mathbb{R}^{2 \times 3}, \quad \boldsymbol{B} = \begin{pmatrix} 0 & 2 \\ 1 & -1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

1. c_{11} is row 1 of $\boldsymbol{A}$ against column 1 of $\boldsymbol{B}$: $1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0 = 2$.
2. c_{12} is row 1 against column 2: $1 \cdot 2 + 2 \cdot (-1) + 3 \cdot 1 = 3$.
3. The remaining entries follow the same grid, $\boldsymbol{A}\boldsymbol{B} = \begin{pmatrix} 2 & 3 \\ 2 & 5 \end{pmatrix}$ in $\mathbb{R}^{2 \times 2}$, mml eq 2.15.
4. The reverse order multiplies a different grid, $\boldsymbol{B}\boldsymbol{A} = \begin{pmatrix} 6 & 4 & 2 \\ -2 & 0 & 2 \\ 3 & 2 & 1 \end{pmatrix}$ in $\mathbb{R}^{3 \times 3}$, mml eq 2.16.

The two-number play: the same two matrices produce 4 entries in one order and 9 in the other. mml draws the moral: "From this example, we can already see that matrix multiplication is not commutative, i.e., $AB \neq BA$." What does hold is the rest of the algebra, associativity $(\boldsymbol{A}\boldsymbol{B})\boldsymbol{C} = \boldsymbol{A}(\boldsymbol{B}\boldsymbol{C})$ and distributivity over addition, mml eqs 2.18-2.20, so long chains of products parse without parentheses ambiguity and factoring works as usual. The triple loop below is the dot grid made literal, running mml Example 2.4′s inverse pair [printed p 25 / pdf p 31] and confirming the product lands on I_3.

```
// mml Example 2.4: two 3x3 matrices that are inverse to each other
double P[3][3] = {{1, 2, 1}, {4, 4, 5}, {6, 7, 7}};
double Q[3][3] = {{-7, -7, 6}, {2, 1, -1}, {4, 5, -4}};
double pd = 0.0;
for (int i = 0; i < 3; i++)
  for (int j = 0; j < 3; j++) {
    double s = 0.0;
    for (int k = 0; k < 3; k++)
      s += P[i][k] * Q[k][j];
    double want = i == j ? 1.0 : 0.0;
    if (fabs(s - want) > pd)
      pd = fabs(s - want);
  }
CHECK(pd < 1e-12, "Example 2.4 pair multiplies to I");
```

listing 54: the dot grid as a triple loop, Example 2.4 pair multiplied to the identity (source: math/samples/src/Ch06/inverse.c, lines 100-113)

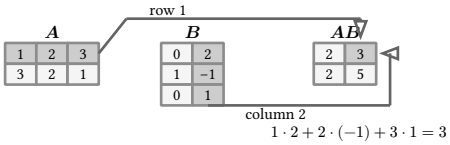


figure 33: one grid entry is one dot product, row 1 of $\boldsymbol{A}$ against column 2 of $\boldsymbol{B}$

NOTE: the exponent ladder

The dot grid costs n^3 multiply-add units, and the exponent itself is negotiable. Strassen 1969 replaces each 2 by 2 block product with 7 scalar multiplications plus recursion, landing at $n^{\log_2 7} \approx 2.8074$. The laser method line pushed lower from there: the public record stood at $\omega < 2.371177$, Dupont et al 2026, arXiv 2608.16884. The openai/math release then went below it in 2 tiers, in the arithmetic-operation model: Lean-formalized, $\omega < 2.371054886006746$ over every fixed field and $\omega \le \frac{9}{4}$ over the complex numbers, with dual exponent $\alpha > 0.465$ and rectangular products below 2.092, and paper tier beside them, $\omega < 2.258$ in characteristic zero, the preprint headline whose complex case follows trivially from the formalized 9/4. Read the terms strictly: these are existence bounds, the model counts arithmetic operations and ignores bit cost, the 9/4 proof states no competitive finite matrix size, and no practical algorithm follows. What runs on real hardware is still this chapter's elimination and Strassen-style blocking: the runnable 7-product kernel and its recurrence live in book 9, ch 43 (practical data structures and algorithms, fine-grained speedups), and book 2, ch 13 (mathematics for programmers, logic, sets, and proof) reruns the release's exact-rational verifier for the every-field bound.

6.3. elimination with partial pivoting

Solving is row bookkeeping. mml section 2.3.2 [printed pp 28-32 / pdf pp 34-38] names the three elementary transformations that preserve the solution set: exchange of two rows, multiplication of a row by a nonzero constant, and addition of one row to another. Repeated in the right order they drive a matrix to the staircase shape of mml Definition 2.6 [printed p 30 / pdf p 36]: "A matrix is in row-echelon form if (i) All rows that contain only zeros are at the bottom of the matrix ... (ii) Looking at nonzero rows only, the first nonzero number from the left (also called the pivot or the leading coefficient) is always strictly to the right of the pivot of the row above it." mml's own definition of the algorithm aims one step further, to the reduced form: "Gaussian elimination is an algorithm that performs elementary transformations to bring a system of linear equations into reduced row-echelon form" [printed p 31 / pdf p 37]. Numerical code stops at the echelon form and finishes with back substitution, the same answers for a fraction of the sweeps.

The classification logic falls out of the staircase. mml Example 2.6 [printed pp 29-30 / pdf pp 35-36] runs a 4x5 system whose echelon form ends in the row $(0, 0, 0, 0, 0 \mid a + 1)$, and "only for $a = -1$ this system can be solved":

a dead row with a live right side is a contradiction, a fully dead row is a free variable, a pivot in every column is a unique solution.

On doubles the bookkeeping acquires an ordering rule. The multiplier $m = a_{ik}/a_{kk}$ divides by the pivot, and a tiny pivot inflates m, which then multiplies every rounding error already sitting in the pivot row. Partial pivoting swaps the largest magnitude entry of the elimination column into the pivot seat, which bounds every multiplier by 1 and keeps the error growth tame.

The dry run: the fixture is mml's compact system of eq 2.36 [printed p 26 / pdf p 32], $2x_1 + 3x_2 + 5x_3 = 1$, $4x_1 - 2x_2 - 7x_3 = 8$, $9x_1 + 5x_2 - 3x_3 = 2$.

step	pivot	multipliers
1	9, row 3 swaps up	$\frac{4}{9}$, $\frac{2}{9}$
2	$-\frac{38}{9}$	$-\frac{17}{38}$
3	$\frac{119}{38}$	none left

Back substitution up the staircase gives the exact answer $x = (\frac{291}{119}, -\frac{23}{7}, \frac{142}{119})$, approximately $(2.4454, -3.2857, 1.1933)$, computed once with python Fractions and pinned. The double run reproduces all three entries to under 10^{-12}, checks 2 through 4 of gauss.c, and the largest residual against the original rows measures $4.4 \cdot 10^{-16}$, check 5, the referee pattern of chapter book 2, ch 1 (mathematics for programmers, floating point anatomy) scaled up from one arithmetic operation to a whole algorithm.

The pivot rule earns its keep on the fixture $\begin{pmatrix} 10^{-16} & 1 \\ 1 & 1 \end{pmatrix} \boldsymbol{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. The exact solution is $x_1 = 10^{16}/(10^{16} - 1)$ and $x_2 = 1 - 1/(10^{16} - 1)$, which equals the shorthand $1 - 10^{-16}$ only after binary64 rounds, both indistinguishable from 1. Eliminating in the given order computes the multiplier $1/10^{-16} = 10^{16}$, the updated second diagonal $1 - 10^{16} \cdot 1$ cannot be represented (the odd integer below 10^{16} is not a binary64 value, it rounds back to -10^{16}), and the damage propagates to $x_1 = (1 - x_2)/10^{-16} = 2.220446049250313$. That number is machine epsilon at scale 1, the cancellation of two almost equal numbers divided by something tiny. Swapping the rows first makes the multiplier 10^{-16} and every stored quantity small, and the solve returns $(1, 0.9999999999999999)$ with error exactly 0. The two-number play: 1.220446049250313 of error without pivoting, 0.0 with it, on the same four coefficients in the same precision.

WARNING: THE DISEASE IS THE MULTIPLIER, NOT THE SMALL NUMBER

A pivot of 10^{-16} is harmless by itself, the system above is barely sensitive to its data and the pivoted solve is exact. What kills the plain solve is the multiplier 10^{16} amplifying rounding error that already exists. Partial pivoting is a bound on multipliers, every one at most 1 in magnitude, and that is the whole mechanism.

```
// augmented 3x4 solve with partial pivoting: the pivot row is the row with
// the largest magnitude entry in the elimination column
[[nodiscard]] static bool solve3(double a[3][4], double x[3]) {
  for (int k = 0; k < 2; k++) {
    int p = k;
    for (int i = k + 1; i < 3; i++)
      if (fabs(a[i][k]) > fabs(a[p][k]))
        p = i;
    if (fabs(a[p][k]) < 1e-12)
      return false;
    if (p != k)
      for (int j = 0; j < 4; j++) {
        double tmp = a[k][j];
        a[k][j] = a[p][j];
        a[p][j] = tmp;
      }
    for (int i = k + 1; i < 3; i++) {
      double m = a[i][k] / a[k][k];
      for (int j = k; j < 4; j++)
        a[i][j] -= m * a[k][j];
    }
  }
  if (fabs(a[2][2]) < 1e-12)
    return false;
  for (int i = 2; i >= 0; i--) {
    double s = a[i][3];
    for (int j = i + 1; j < 3; j++)
      s -= a[i][j] * x[j];
    x[i] = s / a[i][i];
  }
  return true;
}
```

listing 55: partial pivoting, the largest magnitude entry leads the column, back substitution up the staircase (source: math/samples/src/Ch06/gauss.c, lines 25-56)

```
  }
  CHECK(worst < 1e-12, "max residual under 1e-12, measured 4.4e-16");

  // part 2, the pivot disaster: [[1e-16, 1], [1, 1]] x = (1, 2), exact
  // solution x1 = 10^16 / (10^16 - 1), x2 = 1 - 1 / (10^16 - 1), equal to
  // the 1 - 1e-16 shorthand only after binary64 rounds, both next to 1
  double plain[2][3] = {{1e-16, 1, 1}, {1, 1, 2}};
  double xp[2];
  solve2_plain(plain, xp);
  CHECK(near(xp[0], 2.220446049250313, 1e-12), "no-pivot x1 = 2.2204...");
  CHECK(fabs(xp[0] - 1.0) > 1.0, "no-pivot error above 1.0");
  double swapped[2][3] = {{1, 1, 2}, {1e-16, 1, 1}};
```

listing 56: the disaster and the repair, one function, two row orders, errors 1.22 and 0 (source: math/samples/src/Ch06/gauss.c, lines 138-149)

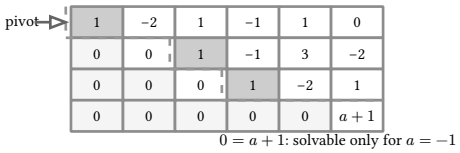


figure 34: row echelon form of Example 2.6, pivots shaded, zeros below the staircase, the last row decides solvability

6.4. lu factorization

Elimination on $\boldsymbol{A}$ alone, right sides forgotten, produces two triangular matrices, and that factorization is worth more than any single solve. Every multiplier m that cleared a column position is an entry of a lower triangular $\boldsymbol{L}$ with unit diagonal, the surviving staircase is the upper triangular $\boldsymbol{U}$, and the row swaps collect into a permutation $\boldsymbol{P}$, so the whole elimination record reads $\boldsymbol{PA} = \boldsymbol{LU}$. Solving $\boldsymbol{Ax} = \boldsymbol{b}$ becomes two triangular sweeps, forward through $\boldsymbol{L}$ for $\boldsymbol{Ly} = \boldsymbol{Pb}$, then backward through $\boldsymbol{U}$ for $\boldsymbol{Ux} = \boldsymbol{y}$, each sweep just a running sum because the matrices are triangular. Chapter book 9, ch 34 (practical data structures and algorithms, linear algebra) factors the same way over $GF(p)$, the multipliers are modular inverses there and exact, here they are divisions and the pivoting rule of the previous section carries over unchanged.

The dry run: the eq 2.36 fixture again, factored in place with the multipliers stored exactly where the zeros came out.

1. The permutation is $(2, 1, 0)$, row 3 leads, mml's coefficients reordered before the first sweep.
2. $\boldsymbol{L} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{4}{9} & 1 & 0 \\ \frac{2}{9} & -\frac{17}{38} & 1 \end{pmatrix}$, the three multipliers of the pivot ledger below the diagonal.
3. $\boldsymbol{U} = \begin{pmatrix} 9 & 5 & -3 \\ 0 & -\frac{38}{9} & -\frac{17}{3} \\ 0 & 0 & \frac{119}{38} \end{pmatrix}$, the staircase, and $\boldsymbol{PA} = \boldsymbol{LU}$ holds entrywise to under 10^{-12}, check 14 of gauss.c.
4. Forward: $\boldsymbol{y} = \left(2, \frac{64}{9}, \frac{71}{19}\right)$. Backward: $\boldsymbol{x}$ reproduces $\left(\frac{291}{119}, -\frac{23}{7}, \frac{142}{119}\right)$, the same answer as the direct solve.

The counting argument is what makes the record worth keeping. The house flop currency is the fused multiply-subtract, one multiply and one subtract per unit, the unit lu3 counts. In that currency the factorization costs $(2n^3 - 3n^2 + n)/6$ units, 5 at $n = 3$, 328350 at $n = 100$, and each additional right side costs $n^2 - n$ units through both sweeps, 9900 at $n = 100$. Trefethen and Bau III, Numerical Linear Algebra, SIAM 1997, lecture 20, count the operations singly, which doubles the factorization to $(2n^3 - 3n^2 + n)/3$, 656700 at $n = 100$, the number factor_flops checks. The two-number play: at $n = 100$ the factorization costs as much as 33 extra right sides, so the second through thirty-third solves are free riders, and the ratio widens like $n^3/n^2 = n$: at $n = 1000$ the one factorization is worth 333 fresh right sides.

```
// factor a 3x3 in place: the strict lower triangle holds the multipliers of
// L (unit diagonal implied), the upper triangle holds U, perm records the row
// order of P A
[[nodiscard]] static bool lu3(double a[3][3], int perm[3]) {
  for (int i = 0; i < 3; i++)
    perm[i] = i;
  for (int k = 0; k < 2; k++) {
    int p = k;
    for (int i = k + 1; i < 3; i++)
      if (fabs(a[i][k]) > fabs(a[p][k]))
        p = i;
    if (fabs(a[p][k]) < 1e-12)
      return false;
    if (p != k) {
      for (int j = 0; j < 3; j++) {
        double tmp = a[k][j];
        a[k][j] = a[p][j];
        a[p][j] = tmp;
      }
      int tp = perm[k];
      perm[k] = perm[p];
      perm[p] = tp;
    }
    for (int i = k + 1; i < 3; i++) {
      double m = a[i][k] / a[k][k];
      a[i][k] = m;
      for (int j = k + 1; j < 3; j++) {
        a[i][j] -= m * a[k][j];
        lu_units++;
      }
    }
  }
  return fabs(a[2][2]) > 1e-12;
```

listing 57: lu3, the multipliers stored where the zeros came out, the permutation recorded alongside (source: math/samples/src/Ch06/gauss.c, lines 71-104)

```
      prod_diff = d;
  }
CHECK(prod_diff < 1e-12, "P A = L U entrywise under 1e-12");

// two triangular sweeps: L y = P b with y = (2, 64/9, 71/19), then U x = y
double y[3];
long long fwd_units = 0;
for (int i = 0; i < 3; i++) {
  double s = b0[perm[i]];
  for (int j = 0; j < i; j++) {
    s -= lu[i][j] * y[j];
    fwd_units++;
  }
  y[i] = s;
}
CHECK(near(y[0], 2.0, 1e-12) && near(y[1], 64.0 / 9.0, 1e-12) &&
          near(y[2], 71.0 / 19.0, 1e-12),
      "forward sweep y = (2, 64/9, 71/19)");
double xlu[3];
long long bwd_units = 0;
for (int i = 2; i ≥ 0; i--) {
  double s = y[i];
  for (int j = i + 1; j < 3; j++) {
    s -= lu[i][j] * xlu[j];
    bwd_units++;
  }
  xlu[i] = s / lu[i][i];
}
CHECK(near(xlu[0], 291.0 / 119.0, 1e-12) &&
          near(xlu[1], -23.0 / 7.0, 1e-12) &&
          near(xlu[2], 142.0 / 119.0, 1e-12),
      "LU solve matches the direct solve");
```

listing 58: two triangular sweeps and the flop ledger, counted units pinned at 5 and 3 (source: math/samples/src/Ch06/gauss.c, lines 187-218)

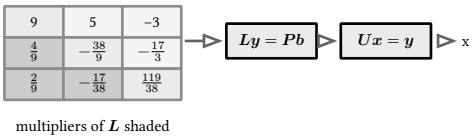


figure 35: one array holds the record, multipliers below the diagonal, staircase above, two sweeps out

6.5. rank and the null space

How many independent directions does a matrix actually carry. mml section 2.6.2 [printed p 47 / pdf p 53] defines it: “The number of linearly independent columns of a matrix $A \in \mathbb{R}^{m \times n}$ equals the number of linearly independent rows and is called the rank of A and is denoted by $rk(A)$.” The same page collects the working facts: $rk(\boldsymbol{A}) = rk(\boldsymbol{A}^\top)$, $\boldsymbol{A}$ square is invertible exactly when $rk(\boldsymbol{A}) = n$, and $\boldsymbol{Ax} = \boldsymbol{b}$ is solvable exactly when $rk(\boldsymbol{A}) = rk(\boldsymbol{A} \mid \boldsymbol{b})$, rank of the coefficient matrix and of the augmented one. Elimination computes it: the rank is the pivot count, because pivot columns are independent and every non-pivot column is a combination of the pivot columns to its left.

The dry run: mml Example 2.18 [printed pp 47-48 / pdf pp 53-54],

$$\boldsymbol{A} = \begin{pmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{pmatrix}$$

1. Two pivots survive, columns 1 and 2, so $rk(\boldsymbol{A}) = 2$ and the third column is a combination of the first two.
2. Reading the combination off the staircase: column 3 equals 3 copies of column 2 minus 5 copies of column 1, that is $5\boldsymbol{c}_1 - 3\boldsymbol{c}_2 + \boldsymbol{c}_3 = 0$, a non-trivial dependence, exactly what the independence test of chapter book 2, ch 5 (mathematics for programmers, vectors and vector spaces) refuses to find in an independent set.
3. The witness vector $(5, -3, 1)$ maps to the zero vector, rank.c check 4, and the same elimination run on $\boldsymbol{A}^\top$ also counts 2, column rank equals row rank as the definition promises.

The null space is where those witnesses live. mml section 2.7.3 [printed p 59 / pdf p 65]: "the kernel/null space $\ker(\Phi)$ is the general solution to the homogeneous system of linear equations $Ax = 0$". For mml Example 2.7′s echelon matrix [printed pp 31-32 / pdf pp 37-38], $\begin{pmatrix} 1 & 3 & 0 & 0 & 3 \\ 0 & 0 & 1 & 0 & 9 \\ 0 & 0 & 0 & 1 & -4 \end{pmatrix}$ with pivots in columns 1, 3 and 4, the free columns 2 and 5 each generate one witness, $(3, -1, 0, 0, 0)$ and $(3, 0, 9, -4, -1)$, both mapped to zero by rank.c. The dimension budget that ties it together is mml Theorem 2.24 [printed p 60 / pdf p 66], the rank-nullity theorem: "For vector spaces V, W and a linear mapping $\Phi : V \rightarrow W$ it holds that $\dim(\ker(\Phi)) + \dim(\mathfrak{I}(\Phi)) = \dim(V)$." For a matrix with n columns the reading is $n = rk(\boldsymbol{A}) + \dim(\ker(\boldsymbol{A}))$, every column is either a pivot or a free parameter, never both, never neither. The fixtures pin both instances: $3 = 2 + 1$ for the 3x3 above, $5 = 3 + 2$ for the 3x5.

NOTE: RANK-NULLITY IS A BUDGET, NOT AN IDENTITY

The theorem does not compute anything, it audits. Count the pivots elimination found, subtract from the column count, and you know how many free parameters any solution set carries before you look at the right side. A full rank square matrix spends the whole budget on pivots, nullity 0, unique solutions. A rank deficient one owes you witnesses, and every free column is an IOU the null space pays.

```
// rank of an m x n matrix: forward elimination with partial pivoting on a
// scratch copy, rank is the number of nonzero pivots left standing
static int rank_of(int m, int n, double a[m][n]) {
  double tmp[8][8];
  for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
      tmp[i][j] = a[i][j];
  int rank = 0;
  int row = 0;
  for (int col = 0; col < n && row < m; col++) {
    int p = row;
    for (int i = row + 1; i < m; i++)
      if (fabs(tmp[i][col]) > fabs(tmp[p][col]))
        p = i;
    if (fabs(tmp[p][col]) < 1e-12)
      continue; // dead column, no pivot here
    if (p != row)
      for (int j = 0; j < n; j++) {
        double t = tmp[row][j];
        tmp[row][j] = tmp[p][j];
        tmp[p][j] = t;
      }
    for (int i = row + 1; i < m; i++) {
      double f = tmp[i][col] / tmp[row][col];
      for (int j = col; j < n; j++)
        tmp[i][j] -= f * tmp[row][j];
    }
    row++;
    rank++;
  }
  return rank;
}
```

listing 59: rank as the pivot count, elimination with the same partial pivoting rule on a scratch copy (source: math/samples/src/Ch06/rank.c, lines 21-53)

```
// null space witness (5, -3, 1): 5 col1 - 3 col2 + col3 = 0, exactly the
// non-trivial dependence the independence test of chapter 5 looks for
double w[3] = {5, -3, 1};
CHECK(image_norm(3, 3, A, w) < 1e-12, "witness maps to the zero vector");
CHECK(3 - rk == 1, "rank 2 plus nullity 1 spans 3 columns");

// mml Example 2.7 in REF: 3x5, pivots in columns 1, 3, 4, rank 3
double B[3][5] = {{1, 3, 0, 0, 3}, {0, 0, 1, 0, 9}, {0, 0, 0, 1, -4}};
int rk_b = rank_of(3, 5, B);
CHECK(rk_b == 3, "rank of the 3x5 REF matrix is 3");

double w1[5] = {3, -1, 0, 0, 0};
double w2[5] = {3, 0, 9, -4, -1};
CHECK(image_norm(3, 5, B, w1) < 1e-12, "first witness maps to zero");
CHECK(image_norm(3, 5, B, w2) < 1e-12, "second witness maps to zero");
CHECK(5 - rk_b == 2, "rank 3 plus nullity 2 spans 5 columns");

// full rank: the 3x3 of chapter gauss.c, rank 3, trivial null space
double F[3][3] = {{2, 3, 5}, {4, -2, -7}, {9, 5, -3}};
int rk_f = rank_of(3, 3, F);
CHECK(rk_f == 3, "the elimination fixture has full rank");
CHECK(3 - rk_f == 0, "full rank leaves nullity 0");

// rank below size means singular: the 2.18 matrix has no inverse
CHECK(rk < 3, "rank 2 in a 3x3 means no inverse");
```

listing 60: witness vectors mapped to zero and the two rank-nullity counts, 3 = 2 + 1 and 5 = 3 + 2 (source: math/samples/src/Ch06/rank.c, lines 79-105)

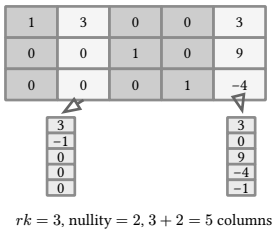


figure 36: pivot columns shaded, free columns light, each free column generates one null space witness

6.6. the inverse

The inverse is the undo button, when it exists. mml Definition 2.3 [printed p 24 / pdf p 30]: “Consider a square matrix $A \in \mathbb{R}^{n\times n}$. Let matrix $B \in \mathbb{R}^{n\times n}$ have the property that $AB = I_n = BA$. B is called the inverse of A and denoted by A^{-1}.” The follow-up sets the vocabulary and the stakes: “Unfortunately, not every matrix A possesses an inverse A^{-1}. If this inverse does exist, A is called regular/invertible/nonsingular, otherwise singular/noninvertible. When the matrix inverse exists, it is unique.” The rank section just gave the test: $rk(\boldsymbol{A}) = n$ or nothing, and two chapters ahead the determinant carries the same verdict (book 2, ch 8 (mathematics for programmers, matrix decompositions)), Sarrus and Laplace there.

For 2x2 the inverse is a closed formula, mml’s remark [printed p 24 / pdf p 30] multiplying $\boldsymbol{A}$ by the adjugate $\begin{pmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{pmatrix}$ to obtain $(a_{11}a_{22} - a_{12}a_{21})\boldsymbol{I}$, so

$$\boldsymbol{A}^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{pmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{pmatrix}$$

whenever that scalar is nonzero. The dry run: the chapter fixture $\begin{pmatrix} 4 & 4 \\ 2 & -4 \end{pmatrix}$ has determinant $-16 - 8 = -24$, so the inverse is $\begin{pmatrix} \frac{1}{6} & \frac{1}{6} \\ \frac{1}{12} & -\frac{1}{6} \end{pmatrix}$, and multiplying back, $\boldsymbol{A}\boldsymbol{A}^{-1}$ and $\boldsymbol{A}^{-1}\boldsymbol{A}$ both land on $\boldsymbol{I}$ to under 10^{-12}, inverse.c check 7. The inverse also solves the opening system in one stroke, $\boldsymbol{A}^{-1}(5, 1) = (1, \frac{1}{4})$, the same crossing point the row picture found, recovered by pure algebra.

For larger matrices the inverse is a batch of eliminations. mml [printed p 33 / pdf p 39] sets it up as one augmented sweep, $(\boldsymbol{A} \mid \boldsymbol{I}_n) \to (\boldsymbol{I}_n \mid \boldsymbol{A}^{-1})$, because "determining the inverse of a matrix is equivalent to solving systems of linear equations", one system per unit vector. mml Example 2.9 [printed p 34 / pdf p 40] runs it on $\begin{pmatrix} 1 & 0 & 2 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 2 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$ and pins the integer answer $\begin{pmatrix} -1 & 2 & -2 & 2 \\ 1 & -1 & 2 & -2 \\ 1 & -1 & 1 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}$, which inverse.c reproduces by gauss-jordan with partial pivoting and verifies by the round trip $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}_4$. The singular cases refuse on both paths: the zero determinant $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ dies in the adjugate formula, the 4x4 embedding of the rank 2 Example 2.18 fixture dies in elimination when a column arrives with no pivot left.

PITFALL: SOLVE, DO NOT INVERT

mml's own caveat from the algorithms section [printed p 35 / pdf p 41]: "for reasons of numerical precision it is generally not recommended to compute the inverse or pseudo-inverse." Computing $\boldsymbol{A}^{-1}\boldsymbol{b}$ costs the elimination plus a full matrix product, carries more rounding than the two triangular sweeps of LU, and feeds the temptation to reuse the inverse as if it were free. The round-trip checks here exist to characterize the object, the workflow they certify is the factorization of the previous section.

```
// adjugate inverse of a 2x2: swap the diagonal, negate the off diagonal,
// divide by the determinant, mml eq 2.24
[[nodiscard]] static bool inv2(const double a[2][2], double out[2][2]) {
  double det = a[0][0] * a[1][1] - a[0][1] * a[1][0];
  if (det == 0.0)
    return false; // exact on these integer fixtures
  out[0][0] = a[1][1] / det;
  out[0][1] = -a[0][1] / det;
  out[1][0] = -a[1][0] / det;
  out[1][1] = a[0][0] / det;
  return true;
}
```

listing 61: inv2, the adjugate shortcut with the determinant as the gatekeeper (source: math/samples/src/Ch06/inverse.c, lines 24-35)

```c
// mml Example 2.9: 4x4 inverse by gauss-jordan, pinned integer result
// [[-1, 2, -2, 2], [1, -1, 2, -2], [1, -1, 1, -1], [-1, 0, -1, 2]]
static const double pin[4][4] = {
    {-1, 2, -2, 2}, {1, -1, 2, -2}, {1, -1, 1, -1}, {-1, 0, -1, 2}};
double aug[4][8] = {{1, 0, 2, 0, 1, 0, 0, 0},
                    {1, 1, 0, 0, 0, 1, 0, 0},
                    {1, 2, 0, 1, 0, 0, 1, 0},
                    {1, 1, 1, 1, 0, 0, 0, 1}};
CHECK(inv4(aug), "4x4 gauss-jordan succeeds");
double md = 0.0;
for (int i = 0; i < 4; i++)
  for (int j = 0; j < 4; j++)
    if (fabs(aug[i][j + 4] - pin[i][j]) > md)
      md = fabs(aug[i][j + 4] - pin[i][j]);
CHECK(md < 1e-12, "computed inverse matches the pinned integers");
```

listing 62: the 4x4 inverse by gauss-jordan, checked against Example 2.9′s pinned integers (source: math/samples/src/Ch06/inverse.c, lines 115-129)

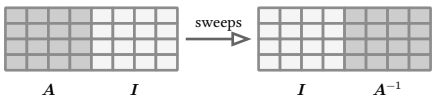


figure 37: gauss-jordan moves the identity across, the right block becomes the inverse

Chapter book 2, ch 7 (mathematics for programmers, inner products and projections) adds the inner product to these bare columns and turns matrices into geometry, lengths, angles and the projections that the rest of the book builds on.

sources: mml-book draft 2024-01-15, Deisenroth, Faisal, Ong, ch 2: systems of linear equations [printed pp 19-21 / pdf pp 25-27], matrices [printed pp 22-26 / pdf pp 28-32], solving systems of linear equations [printed pp 27-35 / pdf pp 33-41] including Example 2.6 and Definition 2.6, rank [printed pp 47-48 / pdf pp 53-54], image and kernel with Theorem 2.24 [printed pp 58-60 / pdf pp 64-66], read from the anchor sheet ref/mml/anchors-la.md, numpy-verified there, fetched 2026-09-21. Flop counts $(2n^3 - 3n^2 + n)/3$ and $n^2 - n$ are the standard elimination counts, Trefethen and Bau III, Numerical Linear Algebra, SIAM 1997, cited by name and edition. Expected values computed by the playground one-off playground/math-ch06/expected.py, exact Fractions plus a binary64 simulation of the sample loops. Probed on this machine the same day. Exponent ladder bounds from the openai/math release, overview.tex entry 107 with the family 107 cache ref/openai-math/family-107/ and its doc-107.md stating the Lean scope, fetched 2026-10-11, and the prior public record Dupont et al 2026, arXiv 2608.16884, $\omega < 2.371177$. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch06`, 60 checks in chapter 06 of the math suite.

7. inner products and projections

Chapter book 2, ch 5 (mathematics for programmers, vectors and vector spaces) measured vectors with norms and chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems) moved them with matrices. This chapter installs the machine that generates both: the inner product, a symmetric bilinear positive definite form whose single definition produces lengths, distances, angles, orthogonal bases, projections, and the least squares solution of an overdetermined system. Six moves: the axioms with the dot product as one instance among many, Cauchy-Schwarz and the angle it licenses, Gram-Schmidt with the orthogonal complement, projections as closest points, the same theory over functions where sums become integrals, and rotations as the maps that preserve the whole structure. Every behavioral claim below is one of the 53 checks in the 4 samples of chapter 7 or a sentence quoted from mml-book draft 2024-01-15. The dsa book's linalg chapter runs elimination as an algorithm, this chapter only borrows its solved systems to build geometry.

7.1. the inner product

A norm can say how long a vector is but not what an angle is. The missing structure is a function of two vectors. mml definition 3.3 [printed p 73 / pdf p 79]: a bilinear mapping $\Omega : V \times V \to \mathbb{R}$ that is symmetric, $\Omega(x, y) = \Omega(y, x)$, and positive definite, $\Omega(x, x) > 0$ for every $x \neq 0$ and $\Omega(0, 0) = 0$, is an inner product, written $\langle \cdot, \cdot \rangle$. The pair $(V, \langle \cdot, \cdot \rangle)$ is an inner product space.

The dot product of chapter 5 is one instance, $x^\top y = \sum_{i=1}^{n} x_i y_i$ (mml eq 3.5) [printed p 72 / pdf p 78], and a space carrying it is called Euclidean. It is not the only one. mml example 3.5 [printed p 75 / pdf p 81] weights the coordinates instead:

$$\langle x, y \rangle := x^\top \begin{pmatrix} 1 & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} y = x_1 y_1 - \frac{1}{2}(x_1 y_2 + x_2 y_1) + x_2 y_2$$

The dry run: the vector (1,1) under both products.

1. Dot product: $\langle x, x \rangle = 1 + 1 = 2$, so the length is $\sqrt{2} = 1.414214$.
2. Weighted product: $\langle x, x \rangle = 1 - \frac{1}{2}(1 + 1) + 1 = 1$, so the length is 1.

Same vector, two lengths. mml theorem 3.5 [printed p 74 / pdf p 80] makes the correspondence exact: for a finite dimensional space with an ordered basis, every inner product is $\langle x, y \rangle = \hat{x}^\top A \hat{y}$ for a symmetric positive definite matrix A and conversely, so picking an inner product is picking a metric matrix. The sample pins the three axioms on fixed vectors, $x = (1, 2, 2)$, $y = (2, 1, 0)$, $z = (0, 3, 1)$, with exact integer arithmetic.

```
int main(void) {
  const double x[3] = {1, 2, 2}; // ||x|| = 3
  const double y[3] = {2, 1, 0}; // ||y|| = sqrt(5)
  const double z[3] = {0, 3, 1};
  const double zero[3] = {0};

  // symmetry: dot(x,y) = dot(y,x). pinned: 4
  CHECK(dot3(x, y) == 4.0 && dot3(y, x) == 4.0, "symmetry dot(x,y) = 4");

  // linearity in the second argument: x.(2y + z) = 2(x.y) + x.z.
  // pinned: 2y + z = (4, 5, 1), x.(2y+z) = 16 = 2*4 + 8
  const double lin[3] = {2 * y[0] + z[0], 2 * y[1] + z[1], 2 * y[2] + z[2]};
  CHECK(dot3(x, lin) == 16.0 && 2 * dot3(x, y) + dot3(x, z) == 16.0,
        "linearity x.(2y+z) = 16");

  // positive definiteness. pinned: <x,x> = 9, <0,0> = 0
  CHECK(dot3(x, x) == 9.0 && dot3(zero, zero) == 0.0,
        "positivity <x,x> = 9, <0,0> = 0");

  // the induced norm. pinned: sqrt(<x,x>) = 3 exactly
  CHECK(sqrt(dot3(x, x)) == 3.0, "norm sqrt(<x,x>) = 3");

  // the same vector (1,1) under two inner products: <(1,1),(1,1)> = 2 with
  // the dot product but 1 with the weighted inner product (mml example 3.5),
  // so its length is sqrt(2) vs 1
  CHECK(dot2((double[]){1, 1}, (double[]){1, 1}) == 2.0 &&
            wip2(1, 1, 1, 1) == 1.0,
        "two inner products: 2 vs 1");
```

listing 63: inner.c, the three axioms and the induced norm pinned on fixed vectors (source: math/samples/src/Ch07/inner.c, lines 47-74)

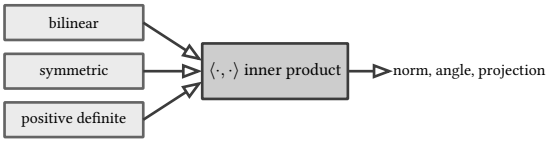


figure 38: three axioms feed one inner product, which generates the geometry

PITFALL: ONLY TWO VECTOR PRODUCTS EXIST

For column vectors exactly two multiplications are defined, the outer product $ab^\top \in \mathbb{R}^{n\times n}$ and the inner product $a^\top b \in \mathbb{R}$, a remark mml attaches to its vector space definition [printed pp 37-38 / pdf pp 43-44]. In C there is no such restriction: the samples always contract with an explicit loop over $a[i] * b[i]$, because a componentwise product of two vectors is the Hadamard product, a different object that never produces a scalar.

7.2. lengths, distances, angles

An inner product induces a norm, $\|x\| := \sqrt{\langle x, x\rangle}$ (mml eq 3.16) [printed p 75 / pdf p 81], and the norm induces a distance, $d(x, y) := \|x - y\|$ (mml definition 3.6) [printed p 75 / pdf p 81], called Euclidean when the product is the dot product. The converse fails: the Manhattan norm of chapter 5 is a norm no inner product induces [mml, printed p 75 / pdf p 81]. The bridge from products to angles is one inequality.

Cauchy-Schwarz (mml eq 3.17) [printed p 75 / pdf p 81]: for an inner product space with induced norm,

$$|\langle x, y\rangle| \leq \|x\|\|y\|$$

Equality holds exactly for collinear vectors. Both sides are pinned: $x = (1, 2, 2)$ against $y = (2, 1, 0)$ gives the strict case $|x^\top y| = 4$ versus $3\sqrt{5} = 6.708204$, while the collinear pair $(1, 2, 2)$, $(2, 4, 4)$ hits $18 = 3 \cdot 6$ exactly. Because the quotient is trapped in $[-1, 1]$, the angle is well defined (mml eq 3.25) [printed p 76 / pdf p 82]:

$$\cos\omega = \frac{\langle x, y\rangle}{\|x\|\|y\|}, \quad \omega \in [0, \pi]$$

The dry run: mml example 3.6 [printed p 77 / pdf p 83], $x = (1, 1)$, $y = (1, 2)$.

1. The product is 3, the norms are $\sqrt{2}$ and $\sqrt{5}$.
2. $\cos\omega = \frac{3}{\sqrt{10}} = 0.948683$, so $\omega = 0.321751$ rad, printed as 18.434949 degrees, which the book rounds to 0.32 rad at about 18 degree.

pair	metric	cos omega	omega
$(1, 1), (1, 2)$	dot	0.948683	0.321751 rad
$(1, 1), (-1, 1)$	dot	0	[$\frac{\pi}{2}$ = 1.570796 rad]
$(1, 1), (-1, 1)$	diag(2,1)	$-\frac{1}{3}$	1.910633 rad

Orthogonality (mml definition 3.7) [printed p 77 / pdf p 83]: $x \perp y$ iff $\langle x, y\rangle = 0$, and if both are unit vectors they are orthonormal. The zero vector is orthogonal to everything. The table's last row is the two-number play: $(1, 1)$ and $(-1, 1)$ sit at 90 degrees under the dot product, but under $\langle x, y\rangle = x^\top \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} y$ (mml example 3.7) [printed pp 77-78 / pdf pp 83-84] the cosine becomes $-\frac{1}{3}$ and the angle opens to 109.471221 degrees. Orthogonality is a property of the pair and the metric together, never of the pair alone.

```
// Cauchy-Schwarz, strict side. pinned: |x.y| = 4 < ||x|| ||y|| =
// 3*sqrt(5) = 6.708203932499369
CHECK(fabs(dot3(x, y)) == 4.0 && 4.0 < 3.0 * sqrt(5.0),
      "cauchy-schwarz strict 4 < 6.708204");

// Cauchy-Schwarz, equality case: (2,4,4) = 2x is collinear with x.
// pinned: x.(2x) = 18 = 3 * 6
const double xc[3] = {2, 4, 4};
CHECK(dot3(x, xc) == 18.0 && 18.0 == 3.0 * sqrt(dot3(xc, xc)),
      "cauchy-schwarz equality 18 = 18");

// angle between (1,1) and (1,2) (mml example 3.6).
// pinned: cos omega = 3/sqrt(10) = 0.9486832980505138
const double c =
    dot2((double[]){1, 1}, (double[]){1, 2}) / (sqrt(2.0) * sqrt(5.0));
CHECK(fabs(c - 0.9486832980505138) < 1e-15, "cos omega = 3/sqrt(10)");

// pinned: omega = 0.3217505543966423 rad = 18.434948822922017 deg
const double w = acos(c);
CHECK(fabs(w - 0.3217505543966423) < 1e-12 &&
          fabs(w * (180.0 / PI) - 18.434948822922017) < 1e-9,
      "omega = 0.3218 rad = 18.435 deg");

// orthogonality under the dot product. pinned: (1,1).(-1,1) = 0, acos(0)
// = pi/2
CHECK(dot2((double[]){1, 1}, (double[]){-1, 1}) == 0.0 &&
          fabs(acos(0.0) - PI / 2) < 1e-15,
      "(1,1) orthogonal to (-1,1), omega = pi/2");

// the same pair under diag(2,1) (mml example 3.7).
// pinned: cos = -1/3, omega = 1.9106332362490186 rad
const double ca =
    aip2(1, 1, -1, 1) / (sqrt(aip2(1, 1, 1, 1)) * sqrt(aip2(-1, 1, -1, 1)));
const double wa = acos(ca);
CHECK(fabs(ca + 1.0 / 3.0) < 1e-15 && fabs(wa - 1.9106332362490186) < 1e-12,
      "diag(2,1): cos = -1/3, omega = 1.9106 rad");
```

listing 64: inner.c, Cauchy-Schwarz on both sides, the angle, and the metric that moves it (source: math/samples/src/Ch07/inner.c, lines 76-111)

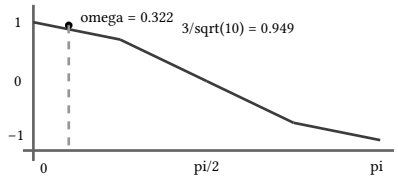


figure 39: cos on [0, pi] is bijective, so every cosine is one angle

7.3. orthonormal bases and the orthogonal complement

A basis is orthonormal (mml definition 3.9) [printed p 79 / pdf p 85] when $\langle b_i, b_j \rangle = 0$ for $i \neq j$ and $\langle b_i, b_i \rangle = 1$ for all i. Orthonormality is what makes coefficient reading trivial: for an orthonormal basis matrix B the projection needs no inverse, $\pi_U(x) = BB^\top x$ with $\lambda = B^\top x$ (mml eqs 3.65-3.66) [printed p 88 / pdf p 94].

Gram-Schmidt builds such a basis from any basis (mml eqs 3.67-3.68) [printed p 89 / pdf p 95]:

$$u_1 := b_1, \quad u_k := b_k - \pi_{\mathrm{span}[u_1,\ldots,u_{k-1}]}(b_k), \quad k = 2, \ldots, n$$

Each new vector sheds its projection onto the directions already built. In code, projecting u onto a single direction w costs one quotient, $\left(\frac{\langle u,w\rangle}{\langle w,w\rangle}\right)w$, and if the inputs are rational the whole run stays rational: no square root appears before normalization, so the sample computes in exact fractions.

```
[[nodiscard]] static rat dot_r(const vec3 a, const vec3 b) {
  rat s = rat_mk(0, 1);
  for (int i = 0; i < 3; i++)
    s = r_add(s, r_mul(a[i], b[i]));
  return s;
}

// one Gram-Schmidt step: u <- u - (<u,w>/<w,w>) w (mml eq 3.67)
static void orthogonalize(vec3 u, const vec3 w) {
  const rat coef = r_div(dot_r(u, w), dot_r(w, w));
  for (int i = 0; i < 3; i++)
    u[i] = r_add(u[i], r_neg(r_mul(coef, w[i])));
}

// full pass: u_k orthogonalized against u_1..u_(k-1)
static void gram_schmidt(const vec3 b[3], vec3 u[3]) {
  for (int k = 0; k < 3; k++) {
    for (int i = 0; i < 3; i++)
      u[k][i] = b[k][i];
    for (int j = 0; j < k; j++)
      orthogonalize(u[k], u[j]);
  }
}
```

listing 65: gramschmidt.c, the rational inner product and the two Gram-Schmidt loops (source: math/samples/src/Ch07/gramschmidt.c, lines 83-105)

The dry run: the basis $(1,0,1)$, $(1,1,1)$, $(0,1,2)$ of $\mathbb{R}^3$.

1. $u_1 = (1,0,1)$ passes through, $\langle u_1, u_1\rangle = 2$.
2. $\langle b_2, u_1\rangle = 2$ over 2 subtracts one copy: $u_2 = (1,1,1) - (1,0,1) = (0,1,0)$.
3. The third vector first pays $\frac{2}{2}$ against u_1, landing $(-1,1,1)$, then $\frac{1}{1}$ against u_2: $u_3 = (-1,0,1)$, with $\langle u_3, u_3\rangle = 2$.

All three outputs are integer vectors, all three cross products are exactly 0, and the coordinate loop rebuilds every input from the u basis. mml example 3.12 [printed pp 89-90 / pdf pp 95-96] runs the same subtraction on $(2,0)$, $(1,1)$ and lands $(2,0)$, $(0,1)$.

k	b_k	u_k	$\langle u_k, u_k\rangle$
1	$(1,0,1)$	$(1,0,1)$	2
2	$(1,1,1)$	$(0,1,0)$	1
3	$(0,1,2)$	$(-1,0,1)$	2

The orthogonal complement (mml section 3.6) [printed pp 79-80 / pdf pp 85-86]: for an M dimensional subspace U of a D dimensional V, the complement $U^\perp$ is the $(D-M)$ dimensional subspace of vectors orthogonal to every vector of U, $U \cap U^\perp = \{0\}$, and every $x \in V$ splits uniquely (mml eq 3.36):

$$x = \sum_{m=1}^{M} \lambda_m b_m + \sum_{j=1}^{D-M} \psi_j b_j^\perp$$

The dry run: the line $U = \mathrm{span}[(1,2,2)]$ in $\mathbb{R}^3$, so $M = 1$ and the complement is a 2 dimensional plane.

1. Two seeds orthogonal to the line: $(2,-1,0)$ and $(2,0,-1)$.

2. One Gram-Schmidt step against the first: $(2,0,-1) - \frac{4}{5}(2,-1,0) = \left(\frac{2}{5}, \frac{4}{5}, -1\right)$, and 5 times that is the integer $(2,4,-5)$.
3. The three vectors $(1,2,2)$, $(2,-1,0)$, $(2,4,-5)$ are pairwise orthogonal, an orthogonal basis of $\mathbb{R}^3$ split as 1 + 2 = 3 dimensions.

A plane's complement is even shorter. The residual $(1,-2,1)$ of the next section's projection spans the complement of $\text{span}[(1,1,1),(0,1,2)]$, exactly the normal direction mml draws for a plane in $\mathbb{R}^3$.

```
// orthogonal complement of the line U = span[(1,2,2)]: two seed vectors
// orthogonal to (1,2,2), then one Gram-Schmidt step.
// pinned: (2,-1,0) stays, (2,0,-1) - (4/5)(2,-1,0) = (2/5,4/5,-1),
// and 5 times that is the integer (2,4,-5)
vec3 w[2];
vset(w[0], 2, -1, 0);
vset(w[1], 2, 0, -1);
orthogonalize(w[1], w[0]);
CHECK(r_eq(w[1][0], rat_mk(2, 5)) && r_eq(w[1][1], rat_mk(4, 5)) &&
          r_eq(w[1][2], rat_mk(-1, 1)),
      "complement step: (2/5, 4/5, -1)");

vec3 line, s1, s2;
vset(line, 1, 2, 2);
for (int i = 0; i < 3; i++) {
  s1[i] = w[0][i];
  s2[i] = rat_mk(w[1][i].num * 5, w[1][i].den * 5); // (2,4,-5)
}
CHECK(r_zero(dot_r(s1, line)) && r_zero(dot_r(s2, line)) &&
          r_zero(dot_r(s1, s2)),
      "orthogonal basis (1,2,2),(2,-1,0),(2,4,-5) of R^3, dims 1+2 = 3");
```

listing 66: gramschmidt.c, the complement of a line as one exact subtraction step (source: math/samples/src/Ch07/gramschmidt.c, lines 188-208)

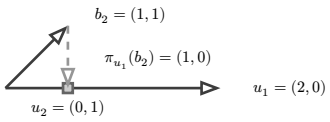


figure 40: gram-schmidt as subtraction, the shadow leaves and the residual stays

NOTE: GRAM-SCHMIDT KEEPS THE SPAN

The algorithm only subtracts projections, which are themselves combinations of the vectors built so far, so $\text{span}[b_1,\ldots,b_k] = \text{span}[u_1,\ldots,u_k]$ at every step k. mml states the span equality alongside its equations 3.67-3.68 and the samples pin it by rebuilding each input from the computed u coordinates in exact arithmetic. The payoff arrives in the next section: an orthonormal basis turns projection into a plain product with $B^\top$.

7.4. orthogonal projections

A projection (mml definition 3.10) [printed p 82 / pdf p 88] is a linear mapping $\pi : V \to U$ onto a subspace with $\pi^2 = \pi$, idempotence as the defining property. The orthogonal projection of x onto U is the point of U closest to x, and its residual $x - \pi_U(x)$ is orthogonal to U.

A line first. With $U = \text{span}[b]$ and $\pi_U(x) = \lambda b$, the orthogonality condition $\langle x - \lambda b, b\rangle = 0$ solves for the coefficient (mml eqs 3.39-3.46) [printed pp 83-85 / pdf pp 89-91]:

$$\lambda = \frac{\langle x, b\rangle}{\|b\|^2}, \quad P_\pi = \frac{bb^\top}{b^\top b}$$

The dry run: mml example 3.10 [printed p 85 / pdf p 91], $b = (1, 2, 2)$, $x = (1, 1, 1)$.

1. $\lambda = \frac{5}{9}$ and $\pi_U(x) = \left(\frac{5}{9}, \frac{10}{9}, \frac{10}{9}\right)$, with the rank 1 projector $P_\pi = \frac{1}{9}\begin{pmatrix} 1 & 2 & 2 \\ 2 & 4 & 4 \\ 2 & 4 & 4 \end{pmatrix}$.
2. The residual $\left(\frac{4}{9}, -\frac{1}{9}, -\frac{1}{9}\right)$ is orthogonal to b and carries squared norm $\frac{2}{9}$.

A subspace of dimension m needs the normal equations. With $B = [b_1, \ldots, b_m]$ (mml eqs 3.55-3.59) [printed pp 85-87 / pdf pp 91-93]:

$$B^\top (x - B\lambda) = 0 \iff B^\top B\lambda = B^\top x \iff \lambda = \left(B^\top B\right)^{-1} B^\top x$$

The dry run: mml example 3.11 [printed pp 87-88 / pdf pp 93-94], $U = \operatorname{span}[(1, 1, 1), (0, 1, 2)]$, $x = (6, 0, 0)$.

1. $B^\top B = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$ with determinant 6, and $B^\top x = (6, 0)$, so $\lambda = (5, -3)$.
2. $\pi_U(x) = 5(1, 1, 1) - 3(0, 1, 2) = (5, 2, -1)$. The residual $(1, -2, 1)$ is orthogonal to both columns and has norm $\sqrt{6} = 2.449490$.
3. $P_\pi = \frac{1}{6}\begin{pmatrix} 5 & 2 & -1 \\ 2 & 2 & 2 \\ -1 & 2 & 5 \end{pmatrix}$, and $P_\pi P_\pi = P_\pi$ holds exactly in rational arithmetic.

```
// ---- plane projection, mml example 3.11 ----
// B columns (1,1,1) and (0,1,2), x = (6,0,0). pinned: B^T B =
// [[3,3],[3,5]], det = 6, B^T x = (6,0), lambda = (5,-3)
const double bx[3] = {1, 1, 1}, by[3] = {0, 1, 2};
const double xs[3] = {6, 0, 0};
const double a11 = dot3(bx, bx), a12 = dot3(bx, by), a22 = dot3(by, by);
const double r1 = dot3(bx, xs), r2 = dot3(by, xs);
const double det = a11 * a22 - a12 * a12;
const double l1 = (a22 * r1 - a12 * r2) / det;
const double l2 = (-a12 * r1 + a11 * r2) / det;
CHECK(det == 6.0 && l1 == 5.0 && l2 == -3.0, "normal equations lam = (5,-3)");

// pinned: pi_U(x) = 5*(1,1,1) - 3*(0,1,2) = (5,2,-1), all integers
double pis[3];
for (int i = 0; i < 3; i++)
  pis[i] = l1 * bx[i] + l2 * by[i];
CHECK(pis[0] == 5.0 && pis[1] == 2.0 && pis[2] == -1.0, "pi_U(x) = (5,2,-1)");

// pinned: residual (1,-2,1) orthogonal to both basis vectors, error
// sqrt(6) = 2.449489742783178
const double rs[3] = {xs[0] - pis[0], xs[1] - pis[1], xs[2] - pis[2]};
CHECK(dot3(rs, bx) == 0.0 && dot3(rs, by) == 0.0 &&
          fabs(sqrt(dot3(rs, rs)) - 2.449489742783178) < 1e-15,
      "residual (1,-2,1), error sqrt(6)");

// the projection matrix route: P = 1/6 [[5,2,-1],[2,2,2],[-1,2,5]] sends
// x to the same point
const double P[3][3] = {{5.0 / 6, 2.0 / 6, -1.0 / 6},
                        {2.0 / 6, 2.0 / 6, 2.0 / 6},
                        {-1.0 / 6, 2.0 / 6, 5.0 / 6}};
double px[3] = {0};
for (int i = 0; i < 3; i++)
  for (int j = 0; j < 3; j++)
    px[i] += P[i][j] * xs[j];
CHECK(fabs(px[0] - 5) < 1e-12 && fabs(px[1] - 2) < 1e-12 &&
          fabs(px[2] + 1) < 1e-12,
      "P x = (5,2,-1)");
```

listing 67: proj.c, the normal equations, the closest point, and the projection matrix route (source: math/samples/src/Ch07/proj.c, lines 58-94)

Two routes, two coefficient vectors, one point. The normal equations return $\lambda = (5, -3)$. The Gram-Schmidt basis of the previous section, $u_1 = (1, 1, 1)$ and $u_2 = (-1, 0, 1)$, returns the orthogonal coordinates $\frac{\langle x, u_j \rangle}{\langle u_j, u_j \rangle} = (2, -3)$, and $2u_1 - 3u_2 = (5, 2, -1)$ again, with no inverse anywhere.

```
  // the orthonormal-basis route: Gram-Schmidt turns (1,1,1),(0,1,2) into
  // u1 = (1,1,1), u2 = (-1,0,1). pinned: coefficients (<x,u>/<u,u>) =
  // (6/3, -6/2) = (2,-3), and 2*u1 - 3*u2 = (5,2,-1), same point, no
  // matrix inverse anywhere
  const double u1[3] = {1, 1, 1}, u2[3] = {-1, 0, 1};
  const double c1 = dot3(xs, u1) / dot3(u1, u1);
  const double c2 = dot3(xs, u2) / dot3(u2, u2);
  double pio[3];
  for (int i = 0; i < 3; i++)
    pio[i] = c1 * u1[i] + c2 * u2[i];
  CHECK(dot3(u1, u2) == 0.0 && c1 == 2.0 && c2 == -3.0 && pio[0] == 5.0 &&
            pio[1] == 2.0 && pio[2] == -1.0,
        "onb coefficients (2,-3), same pi_U(x)");

  // closest point: every other point pi + t*(1,1,1) on U is farther.
  // pinned squared distances for t = -1, -1/2, 1/2, 1: 9, 6.75, 6.75, 9,
  // all above the projection's 6
  bool farthest = true;
  const double ttab[4] = {-1, -0.5, 0.5, 1};
  const double dtab[4] = {9, 6.75, 6.75, 9};
  for (int k = 0; k < 4; k++) {
    const double q[3] = {pis[0] + ttab[k], pis[1] + ttab[k], pis[2] + ttab[k]};
    const double d2 =
        dot3((double[3]){xs[0] - q[0], xs[1] - q[1], xs[2] - q[2]},
             (double[3]){xs[0] - q[0], xs[1] - q[1], xs[2] - q[2]});
    if (!(d2 > 6.0 && fabs(d2 - dtab[k]) < 1e-12))
      farthest = false;
  }
  CHECK(farthest, "projection is the closest point, d^2 = 9, 6.75, 6.75, 9");
```

listing 68: proj.c, the inverse-free route and the closest point walk along U (source: math/samples/src/Ch07/proj.c, lines 96-124)

route	coefficients	pi_U (x)
normal equations	$(5, -3)$	$(5, 2, -1)$
orthogonal basis	$(2, -3)$	$(5, 2, -1)$
projection matrix		$(5, 2, -1)$

The closest point property is pinned by walking: from $x = (6, 0, 0)$ to any other point of U, say $\pi_U(x) + t(1, 1, 1)$ for t in $\left(-1, -\frac{1}{2}, \frac{1}{2}, 1\right)$, the squared distance climbs 6 to 6.75 to 9 while the projection holds

6. For affine subspaces the same computation applies after subtracting the

support point, $\pi_L(x) = x_0 + \pi_U(x - x_0)$ (mml eq 3.72) [printed p 90 / pdf p 96].

Least squares falls out. When $A\lambda = y$ has no solution, projecting y onto the column space of A gives the closest attainable right hand side and the coefficients of that projection are the least squares solution (mml [printed p 88 / pdf p 94]).

The dry run: fit $y = a + ct$ through $(0, 1)$, $(1, 3)$, $(2, 3)$.

1. The design matrix has columns $(1, 1, 1)$ and $(0, 1, 2)$, the same B as example 3.11, so the same normal equations run with $A^\top y = (7, 9)$.
2. $\lambda = \left(\frac{4}{3}, 1\right)$, the line $y = \frac{4}{3} + t$.
3. Residuals $\left(-\frac{1}{3}, \frac{2}{3}, -\frac{1}{3}\right)$, exactly orthogonal to both columns, with squared norm $\frac{2}{3}$ and norm 0.816497.

```
// ---- least squares: fit y = a + c t through (0,1),(1,3),(2,3) ----
// design matrix columns (1,1,1) and (0,1,2): the same normal equations
// as above. pinned: A^T y = (7,9), lambda = (4/3, 1)
const double yv[3] = {1, 3, 3};
const double s1 = yv[0] + yv[1] + yv[2];
const double s2 = 0 * yv[0] + 1 * yv[1] + 2 * yv[2];
const double la = (a22 * s1 - a12 * s2) / det;
const double lc = (-a12 * s1 + a11 * s2) / det;
CHECK(s1 == 7.0 && s2 == 9.0 && fabs(la - 4.0 / 3.0) < 1e-15 && lc == 1.0,
      "least squares line a = 4/3, c = 1");

// pinned: residuals (-1/3, 2/3, -1/3), orthogonal to both columns,
// squared norm 2/3, norm = 0.816496580927726
const double rr[3] = {yv[0] - (la + lc * 0), yv[1] - (la + lc * 1),
                      yv[2] - (la + lc * 2)};
CHECK(fabs(rr[0] + 1.0 / 3.0) < 1e-12 && fabs(rr[1] - 2.0 / 3.0) < 1e-12 &&
          fabs(rr[2] + 1.0 / 3.0) < 1e-12,
      "residuals (-1/3, 2/3, -1/3)");
CHECK(fabs(rr[0] + rr[1] + rr[2]) < 1e-12 &&
          fabs(rr[1] + 2 * rr[2]) < 1e-12 &&
          fabs(dot3(rr, rr) - 2.0 / 3.0) < 1e-12,
      "residual orthogonal to columns, ||r||^2 = 2/3");
```

listing 69: proj.c, an overdetermined fit solved as a projection (source: math/samples/src/Ch07/proj.c, lines 126-147)

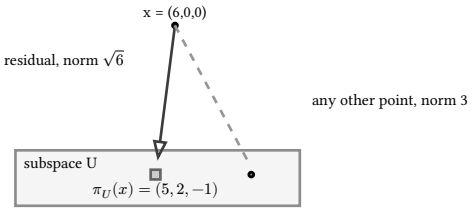


figure 41: the projection is the foot of the perpendicular, everything else on U is farther

WARNING: DO NOT INVERT IN PRACTICE

mml closes its projection section with two numerical caveats: computing the pseudo-inverse $(B^\top B)^{-1} B^\top$ is discouraged for precision reasons, and practitioners add a jitter term εI to $B^\top B$ for stability, a ridge that also falls out of Bayesian inference [printed pp 87-88 / pdf pp 93-94]. The samples invert a 2 by 2 matrix with determinant 6 because the pedagogy is exact fractions, while chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) handles the factorizations real solvers use.

7.5. inner products of functions

Nothing in the definition used finiteness. For functions u, v on a finite interval (mml eq 3.37) [printed p 80 / pdf p 86]:

$$\langle u, v \rangle := \int_a^b u(x) v(x) \, \mathrm{d}x$$

Symmetry and bilinearity are the linearity of the integral, positive definiteness holds because a nonnegative continuous function integrates to 0 only by being 0, and every construction of this chapter carries over: norms, Cauchy-Schwarz, orthogonality, projection. Chapter book 2, ch 22 (mathematics for programmers, quadrature and differential equations) owns quadrature itself, this sample only borrows composite Simpson with 4000 panels as its measuring device.

The dry run: mml example 3.9 [printed p 81 / pdf p 87], $u = \sin$, $v = \cos$ on $[-\pi, \pi]$. The product $\sin(x)\cos(x)$ is odd, so the integral is 0 and sine is orthogonal to cosine. Measured: 1.1e-16. The wider family is pinned the same way, $\{1, \cos t, \cos 2t, \cos 3t\}$ orthogonal as a system, $\langle \sin, \sin \rangle = \langle \cos, \cos \rangle = \pi = 3.141593$ and $\langle 1, 1 \rangle = 2\pi = 6.283185$.

```
// composite Simpson for the inner product integral <u, v> on [a, b],
// n even (mml eq 3.37)
[[nodiscard]] static double simpson_ip(fn u, fn v, double a, double b, int n) {
  const double h = (b - a) / n;
  double s = u(a) * v(a) + u(b) * v(b);
  for (int i = 1; i < n; i++) {
    const double w = (i % 2) ? 4.0 : 2.0;
    s += w * u(a + i * h) * v(a + i * h);
  }
  return s * h / 3.0;
}

// projection coefficient <f, e>/<e, e> for an orthogonal basis element
[[nodiscard]] static double coeff(fn f, fn e) {
  return simpson_ip(f, e, -PI, PI, N) / simpson_ip(e, e, -PI, PI, N);
}
```

listing 70: funcinner.c, the integral inner product and one projection coefficient, both by Simpson (source: math/samples/src/Ch07/funcinner.c, lines 43-58)

Projection now runs in infinitely many dimensions. The sine modes are mutually orthogonal, so projecting $f(x) = x$ onto the first three modes uses the inverse-free formula of the previous section, one coefficient per mode, and integration by parts gives the closed form:

$$a_k = \frac{\langle f, \sin(k\,\cdot) \rangle}{\langle \sin(k\,\cdot), \sin(k\,\cdot) \rangle} = \frac{2(-1)^{k+1}}{k}$$

The dry run: $a_1 = 2$, $a_2 = -1$, $a_3 = \frac{2}{3}$, measured by Simpson as 2.000000000, −1.000000000 and 0.666666667. The 3 mode approximation $2\sin t - \sin 2t + \frac{2}{3}\sin 3t$ tracks the line closely mid interval and misses at the endpoints, where the periodic extension of x jumps: at $t = +-\pi$ the projection returns 0 against the function's $+-3.141593$.

```
  // projection of f(t) = t onto the first sine modes, coefficients
  // a_k = <f, sin kt>/<sin kt, sin kt> = 2(-1)^(k+1)/k.
  // pinned simpson values: 2.000000000000071, -1.000000000000541,
  // 0.6666666666684915
  CHECK(fabs(coeff(f_x, f_sin1) - 2.0) < 1e-9, "a_1 = 2");
  CHECK(fabs(coeff(f_x, f_sin2) + 1.0) < 1e-9, "a_2 = -1");
  CHECK(fabs(coeff(f_x, f_sin3) - 2.0 / 3.0) < 1e-9, "a_3 = 2/3");

  // pinned: ||f||^2 = 2pi^3/3 = 20.670851120199877, simpson gives
  // 20.670851120199842
  const double fnorm2 = simpson_ip(f_x, f_x, -PI, PI, N);
  CHECK(fabs(fnorm2 - 20.670851120199877) < 1e-8, "||f||^2 = 2pi^3/3");
```

listing 71: funcinner.c, the Fourier coefficients of f = x against their closed forms (source: math/samples/src/Ch07/funcinner.c, lines 87-98)

Energy accounting is the two-number play. The whole function holds $\langle f, f \rangle = 2\frac{\pi^3}{3} = 20.670851$. One mode captures $4\pi = 12.566371$, a fraction of 0.607927. Three modes capture $\left(4 + 1 + \frac{4}{9}\right)\pi = 17.104227$, a fraction of 0.827456. mml remarks that projecting functions onto the orthogonal cosine system "is the fundamental idea behind Fourier series" [printed p 81 / pdf p 87].

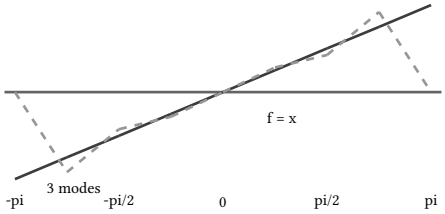


figure 42: f = x against its 3 mode projection, hand-plotted at multiples of pi/4

PITFALL: INTEGRALS CAN DIVERGE

mml is careful about the function inner product: limits must be finite, the product must be integrable, and "precise treatment requires measures and Hilbert spaces" beyond the book's scope [printed pp 80-81 / pdf pp 86-87]. The sample stays on safe ground, smooth products of sines, cosines and polynomials on a closed interval, where Simpson converges fast enough to pin coefficients to 12 digits.

7.6. rotations revisited

Chapter book 2, ch 3 (mathematics for programmers, trigonometry) built the rotation matrix from the unit circle. The inner product explains why that matrix is special. A square matrix is orthogonal (mml definition 3.8) [printed p 78 / pdf p 84] when its columns are orthonormal, $AA^\top = I = A^\top A$, so $A^{-1} = A^\top$. Then lengths and angles survive the map (mml eqs 3.31-3.32) [printed p 78 / pdf p 84]:

$$\|Ax\|_2^2 = (Ax)^\top (Ax) = x^\top A^\top A x = \|x\|_2^2, \quad \cos\omega = \frac{(Ax)^\top (Ay)}{\|Ax\|\|Ay\|} = \frac{x^\top y}{\|x\|\|y\|}$$

The dry run: $R\left(\frac{\pi}{3}\right)$ in gramschmidt.c.

1. $R^\top R = I$ to $1e-15$ and $\det R = \cos^2 + \sin^2 = 1$.
2. $\langle (1,2), (3,-1) \rangle = 1$ before the rotation and 1 after, and $\|R(1,1)\| = \sqrt{2}$.

mml states the conclusion directly: "orthogonal matrices define transformations that are rotations (with the possibility of flips)" [printed p 78 / pdf p 84]. Section 3.9 assembles $R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$ (mml eq 3.76) [printed p 92 / pdf p 98] from the images of the standard basis vectors, which is the orthonormal-columns condition in action, and lists the group properties: rotations preserve distances and angles, and they commute only in 2 dimensions [mml section 3.9.4, printed p 94 / pdf p 100]. The two-number play is the pair of images: e_1 moves to $(0.5, 0.866)$ and e_2 to $(-0.866, 0.5)$, both still unit, still at a right angle, determinant +1 where a reflection would carry −1.

```
// the ch 03 rotation by pi/3 is an orthogonal matrix: R^T R = I,
// det = 1, and it preserves inner products (mml eqs 3.29-3.32)
const double th = PI / 3;
const double R[2][2] = {{cos(th), -sin(th)}, {sin(th), cos(th)}};
double rtr[2][2] = {0};
for (int i = 0; i < 2; i++)
  for (int j = 0; j < 2; j++)
    for (int k = 0; k < 2; k++)
      rtr[i][j] += R[k][i] * R[k][j];
bool eye = fabs(rtr[0][0] - 1.0) < 1e-15 && fabs(rtr[0][1]) < 1e-15 &&
           fabs(rtr[1][0]) < 1e-15 && fabs(rtr[1][1] - 1.0) < 1e-15;
CHECK(eye, "R(pi/3): R^T R = I");

// pinned: det = cos^2 + sin^2 = 1
CHECK(fabs(R[0][0] * R[1][1] - R[0][1] * R[1][0] - 1.0) < 1e-15, "det R = 1");

// pinned: <(1,2),(3,-1)> = 1 and the rotated pair still has 1
const double ax = 1, ay = 2, bx = 3, by = -1;
const double rax = R[0][0] * ax + R[0][1] * ay;
const double ray = R[1][0] * ax + R[1][1] * ay;
const double rbx = R[0][0] * bx + R[0][1] * by;
const double rby = R[1][0] * bx + R[1][1] * by;
CHECK(fabs((rax * rbx + ray * rby) - (ax * bx + ay * by)) < 1e-15,
      "R preserves <x,y> = 1");

// pinned: ||R(1,1)|| = sqrt(2), lengths preserved
const double rx = R[0][0] + R[0][1], ry = R[1][0] + R[1][1];
CHECK(fabs(sqrt(rx * rx + ry * ry) - sqrt(2.0)) < 1e-15,
      "||R(1,1)|| = sqrt(2)");
```

listing 72: gramschmidt.c, the rotation matrix certified orthogonal, determinant 1, inner product preserved (source: math/samples/src/Ch07/gramschmidt.c, lines 220-248)

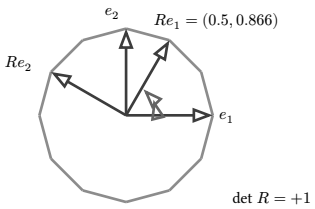


figure 43: rotation by pi/3, the basis stays orthonormal

The next chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) breaks matrices into products of simpler ones, and its central actors, the orthogonal factors of the eigendecomposition and the singular value decomposition, are exactly the maps this chapter certified as inner product preservers.

sources: mml-book draft 2024-01-15, ch 3 analytic geometry, sections 3.1 to 3.9, definitions 3.1 to 3.11, examples 3.5 to 3.12 and equations 3.5 to 3.76, worked from the verified anchor sheet ref/mml/anchors-la.md (printed pp 71-94 / pdf pp 77-100), read 2026-09-21, with chapter 2 background on elimination and vector spaces (printed pp 27-38 / pdf pp 33-44). Expected sample values computed by the playground one-off playground/math-ch07/pin.py, python fractions, run 2026-09-21. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch07`, 53 checks in chapter 7 of the math suite, 4 files, zero failures.

8. matrix decompositions

A matrix decomposition factors one matrix into simpler pieces whose shape tells you what the matrix does. This chapter builds six of them on paper and pins each one in c: the determinant and trace as the two scalar fingerprints of a matrix, eigenvalues and eigenvectors from the characteristic polynomial, the Cholesky square root of a symmetric positive definite matrix, QR from Gram-Schmidt, the singular value decomposition, and the Eckart-Young rank-k approximation that low-rank compression and pca rest on. The sibling dsa book calls a factorization library inside its solvers, this chapter derives each factor by hand on fixed matrices and verifies it against exact precomputed values. Every behavioral claim below is one of the 52 checks in the 4 samples of chapter 08 or a sentence quoted from the mml-book draft 2024-01-15, read 2026-09-21. Chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems) supplies Gaussian elimination and the inverse, chapter book 2, ch 7 (mathematics for programmers, inner products and projections) supplies the projection machinery that Gram-Schmidt uses.

8.1. determinant and trace

The determinant maps a square matrix to one number: for a 2x2 matrix it is $a_{11}a_{22} - a_{12}a_{21}$ (mml Example 4.1, eq 4.4), and a matrix is invertible exactly when that number is nonzero (mml Theorem 4.1). Geometric reading, quoted: "the determinant $\det(A)$ is the signed volume of an n-dimensional parallelepiped formed by columns of the matrix A" (mml Example 4.2, printed pp 101-102). The sign carries orientation, swapping two columns flips it. For a 3x3 matrix Sarrus' rule (mml eq 4.7) expands the determinant into six triple products, and Laplace expansion (mml Theorem 4.2) reduces an $n \times n$ determinant to n determinants of size $(n-1) \times (n-1)$, so both give the same number on our fixtures. The trace is the other scalar fingerprint: $\operatorname{tr}(A) := \sum_{i=1}^{n} a_{ii}$, the sum of the diagonal (mml Definition 4.4). Its one non-obvious identity is cyclic: $\operatorname{tr}(AB) = \operatorname{tr}(BA)$ even for rectangular $A \in \mathbb{R}^{n \times k}$, $B \in \mathbb{R}^{k \times n}$ where the two products have different sizes, and the two counts meet the same eigenvalues in section 2: $\det(A) = \prod_{i=1}^{n} \lambda_i$ (mml Theorem 4.16) and $\operatorname{tr}(A) = \sum_{i=1}^{n} \lambda_i$ (mml Theorem 4.17).

The dry run: the volume fixture is the mml Example 4.2 matrix $A = \begin{pmatrix} 2 & 6 & 1 \\ 0 & 1 & 4 \\ -8 & 0 & -1 \end{pmatrix}$.

1. Sarrus: $2 \cdot 1 \cdot (-1) + 0 \cdot 0 \cdot 1 + (-8) \cdot 6 \cdot 4 = -194$ against the counter-diagonal $(-8) \cdot 1 \cdot 1 + 2 \cdot 0 \cdot 4 + 0 \cdot 6 \cdot (-1) = -8$, and $-194 + 8 = -186$. The volume is $|\det(A)| = 186$, the negative sign says the column triple is orientation flipping.
2. Laplace along the first row gives the same −186 by three cofactors: $2(-1 - 0) - 6(0 + 32) + 1(0 + 8)$.
3. The trace pair: with the mml Example 2.3 rectangles $AB = \begin{pmatrix} 2 & 3 \\ 2 & 5 \end{pmatrix}$ has trace $2 + 5 = 7$ and $BA = \begin{pmatrix} 6 & 4 & 2 \\ -2 & 0 & 2 \\ 3 & 2 & 1 \end{pmatrix}$ has trace $6 + 0 + 1 = 7$. Different shapes, one number.
4. Eigenvalue handshake on $\begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$: eigenvalues 5 and 2 give $\det = 5 \cdot 2 = 10$ and $\operatorname{tr} = 5 + 2 = 7$, both matching direct computation.

PITFALL: DETERMINANT NOTATION

mml warns that the alternative notation $|A|$ "must not be confused with the absolute value". The determinant of $\begin{pmatrix} 2 & 6 & 1 \\ 0 & 1 & 4 \\ -8 & 0 & -1 \end{pmatrix}$ is −186 while its absolute value, the volume, is 186. This book writes $\det(A)$.

The characteristic polynomial packages both fingerprints into one object, $p_A(\lambda) = \det(A - \lambda I) = c_0 + c_1\lambda + \ldots + (-1)^n\lambda^n$ with $c_0 = \det(A)$ and $c_{n-1} = (-1)^{n-1}\operatorname{tr}(A)$ (mml Definition 4.5). Faddeev-LeVerrier computes all coefficients from traces of matrix powers, no symbolic algebra: $M_1 = A$, $p_1 = -\operatorname{tr}(M_1)$, then $M_k = A(M_{k-1} + p_{k-1}I)$ and $p_k = -\frac{\operatorname{tr}(M_k)}{k}$.

```
// Faddeev-LeVerrier: coefficients c[0..3] of det(A - lam I) ascending,
// c0 + c1 lam + c2 lam^2 - lam^3 for n = 3. Recurrence on
// det(lam I - A) = lam^3 + p1 lam^2 + p2 lam + p3:
// M1 = A, p1 = -tr(M1), Mk = A (M(k-1) + p(k-1) I), pk = -tr(Mk)/k.
static void leverrier3(const double a[N][N], double c[4]) {
  double m[N][N];
  double p[4] = {0, 0, 0, 0};
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      m[i][j] = a[i][j];
  p[1] = -trace3(m);
  for (int k = 2; k ≤ N; k++) {
    double shifted[N][N];
    for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
        shifted[i][j] = m[i][j] + (i == j ? p[k - 1] : 0.0);
    mm3(a, shifted, m);
    p[k] = -trace3(m) / k;
  }
  // det(A - lam I) = -(det(lam I - A)) for odd n
  c[0] = -p[3];
  c[1] = -p[2];
  c[2] = -p[1];
  c[3] = -1.0;
}
```

listing 73: dettrace.c, faddeev-leverrier turns traces of powers into polynomial coefficients (source: math/samples/src/Ch08/dettrace.c, lines 69-93)

On the symmetric fixture $\begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & 2 \\ 2 & 2 & 3 \end{pmatrix}$ the recurrence yields $p_A(\lambda) = -\lambda^3 + 9\lambda^2 - 15\lambda + 7$, with $c_0 = 7 = \det(A)$ and $c_2 = 9 = \operatorname{tr}(A)$, both integers exactly representable in doubles. The sample then pins volume, sign flip under a column swap, multiplicativity $\det(AB) = \det(A)\det(B)$ as $49 = 7^2$, the trace identities $\operatorname{tr}(A^2) = 51$ and $\operatorname{tr}(A^3) = 345$, and both eigenvalue handshakes.

```
  // 3x3 volume matrix, mml Example 4.2 eqs 4.9-4.11
  const double vol[N][N] = {{2, 6, 1}, {0, 1, 4}, {-8, 0, -1}};
  // Sarrus: 2*1*(-1) + 0*0*1 + (-8)*6*4 - ((-8)*1*1 + 2*0*4 + 0*6*(-1))
  //       = -2 - 192 + 8 = -186, volume |det| = 186
  CHECK(sarrus3(vol) == -186.0, "sarrus determinant -186");
  CHECK(laplace_row1(vol) == -186.0, "laplace row 1 determinant -186");

  double swap[N][N];
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      swap[i][j] = vol[i][j == 0 ? 1 : (j == 1 ? 0 : 2)];
  // swapping columns 1 and 2 flips the sign: +186
  CHECK(sarrus3(swap) == 186.0, "column swap flips sign to +186");
```

listing 74: dettrace.c, sarrus and laplace agree on the signed volume −186 (source: math/samples/src/Ch08/dettrace.c, lines 103-115)

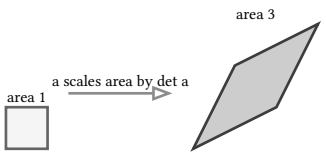


figure 44: unit square and its image, area scales by det a = 3

The two-number play: the same column triple scores −186 and +186, identical volume, opposite orientation. And one matrix owns two scalar fingerprints, here 7 and 9, whose factorization and sum the eigenvalues reproduce.

8.2. eigenvalues and eigenvectors

An eigenvalue λ with eigenvector $x \neq 0$ solves $Ax = \lambda x$ (mml Definition 4.6, eq 4.25): A leaves the line through x invariant and stretches it by λ. The eigenvalues are exactly the roots of $p_A(\lambda)$ (mml Theorem 4.8), and the eigenvectors for a given λ span the eigenspace E_λ, the null space of $A - \lambda I$ (mml Definition 4.10). When A has n linearly independent eigenvectors they form a basis (mml Theorem 4.12), $P = [p_1, \ldots, p_n]$ is invertible, and the diagonalization $A = PDP^{-1}$ holds with the eigenvalues on the diagonal of D (mml Definition 4.19, Theorem 4.20). The key lemma is columnwise: $AP = PD$ holds if and only if $Ap_i = \lambda_i p_i$ for every column (mml eq 4.50).

The dry run: the fixture is the mml Example 4.5 matrix $A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$, trace 7, determinant 10.

1. Characteristic polynomial: $(4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10$. Quadratic formula: $\frac{7 \pm \sqrt{49-40}}{2} = \frac{7 \pm 3}{2}$, eigenvalues 5 and 2.
2. Eigenvector for 5: $A - 5I = \begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix}$, rows collinear, null direction $(2, 1)$. Check: $A(2,1) = (10, 5) = 5(2,1)$.
3. Eigenvector for 2: $A - 2I = \begin{pmatrix} 2 & 2 \\ 1 & 1 \end{pmatrix}$, null direction $(1, -1)$, and $A(1,-1) = (2, -2)$.
4. Diagonalization: $P = \begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}$ with $\det(P) = -3$, so $P^{-1} = \begin{pmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & -\frac{2}{3} \end{pmatrix}$. Then $AP = \begin{pmatrix} 10 & 2 \\ 5 & -2 \end{pmatrix} = PD$ entry by entry, and PDP^{-1} rebuilds A to 10^{-12}.

```c
// ——— 2x2: mml Example 4.5 ———
const double a2[2][2] = {{4, 2}, {1, 3}};
// characteristic polynomial lam^2 - 7 lam + 10, quadratic formula
// disc = 49 - 40 = 9, roots (7 +- 3)/2 = 5 and 2, generator exact
double tr = a2[0][0] + a2[1][1];
double det = a2[0][0] * a2[1][1] - a2[0][1] * a2[1][0];
double disc = tr * tr - 4.0 * det;
double l1 = (tr + sqrt(disc)) / 2.0;
double l2 = (tr - sqrt(disc)) / 2.0;
CHECK(fabs(l1 - 5.0) < 1e-12 && fabs(l2 - 2.0) < 1e-12,
      "2x2 eigenvalues 5 and 2 by quadratic formula");

// eigenvectors: null vector of A - lam I, perpendicular to a nonzero row
// lam = 5: rows (-1, 2) and (1, -2), null direction (2, 1), mml eq 4.35
// lam = 2: rows (2, 2) and (1, 1), null direction (1, -1), mml eq 4.34
const double v5[2] = {2, 1};
const double v2[2] = {1, -1};
// A v = lam v exactly on these integer fixtures
CHECK(a2[0][0] * v5[0] + a2[0][1] * v5[1] == 5.0 * v5[0] &&
          a2[1][0] * v5[0] + a2[1][1] * v5[1] == 5.0 * v5[1],
      "A (2,1) = 5 (2,1), eigenspace E_5 = span(2,1)");
CHECK(a2[0][0] * v2[0] + a2[0][1] * v2[1] == 2.0 * v2[0] &&
          a2[1][0] * v2[0] + a2[1][1] * v2[1] == 2.0 * v2[1],
      "A (1,-1) = 2 (1,-1), eigenspace E_2 = span(1,-1)");
```

listing 75: eigen.c, quadratic formula for the eigenvalue pair, eigenvector checks against exact integer arithmetic (source: math/samples/src/Ch08/eigen.c, lines 83-106)

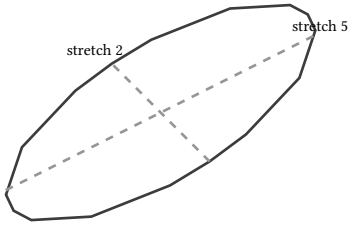


figure 45: unit circle mapped to an ellipse, semi-axes 5 and 2 along the eigen-directions

The ellipse is the point of the section: the unit circle maps to an ellipse whose semi-axes are the eigenvalues, each pointing along its eigenvector. The two-number play: 5 and 2, the stretch factors along $(2, 1)$ and $(1, -1)$, multiply to the area scale 10 and add to the trace 7.

The symmetric case is where the theory closes. The spectral theorem (mml Theorem 4.15) says a symmetric $A \in \mathbb{R}^{n \times n}$ has an orthonormal basis of eigenvectors and all real eigenvalues, so a symmetric matrix is always diagonalizable (mml Theorem 4.21) and P can be taken orthogonal with $P^{-1} = P^\top$. The 3x3 symmetric fixture $\begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & 2 \\ 2 & 2 & 3 \end{pmatrix}$ from section 1 has $p_A(\lambda) = -(\lambda - 1)^2(\lambda - 7)$, a repeated root. The sample finds 7 among the integer divisors of $\det = 7$, deflates to the quotient $\lambda^2 - 2\lambda + 1$ with discriminant 0, and reads off the double root 1. Algebraic multiplicity 2 for $\lambda = 1$, and the geometric multiplicity is honest too: $A - I$ is the all-2s matrix of rank 1, so E_1 is 2-dimensional with basis $(-1, 1, 0)$ and $(-1, 0, 1)$ (mml Example 4.8, eq 4.39). Inside E_1 one Gram-Schmidt step orthogonalizes them to $(-1, 1, 0)$ and $\frac{1}{2}(-1, -1, 2)$ (mml eq 4.41), and appending $\frac{1,1,1}{\sqrt{3}}$ from E_7 gives the orthogonal P with $A = PDP^\top$ verified to 10^{-12}.

```
// integer root candidates are the divisors of c0 = 7: p(1) = 0 and
// p(7) = 0 both hold (eigenvalues 7, 1, 1), take 7 first.
// synthetic division of lam^3 - 9 lam^2 + 15 lam - 7 by (lam - 7):
// quotient lam^2 - 2 lam + 1, remainder 0
double monic[4] = {-c[3], -c[2], -c[1], -c[0]}; // lam^3 - 9lam^2 + 15lam - 7
double root = 7.0; // first divisor of 7 with p(root) = 0
double pv =
    ((monic[0] * root + monic[1]) * root + monic[2]) * root + monic[3];
CHECK(fabs(pv) < 1e-12, "p(7) = 0, root found among divisors of det = 7");

double q[2]; // quotient lam^2 + q[0] lam + q[1]
q[0] = monic[1] + root;
q[1] = monic[2] + q[0] * root;
// remainder check folded into round trip below: q = lam^2 - 2 lam + 1
double discr = q[0] * q[0] - 4.0 * q[1];
CHECK(fabs(q[0] - (-2.0)) < 1e-12 && fabs(q[1] - 1.0) < 1e-12 &&
          fabs(discr) < 1e-12 && fabs(-q[0] / 2.0 - 1.0) < 1e-12,
      "deflation gives lam^2 - 2lam + 1, double root 1");

// geometric multiplicity: A - I is all 2s, rank 1, nullity 2
double ai[N][N];
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    ai[i][j] = a3[i][j] - (i == j ? 1.0 : 0.0);
CHECK(rank3(ai) == 1, "rank(A - I) = 1 so E_1 has dimension 2");
```

listing 76: eigen.c, integer root, deflation, and the rank count that certifies the eigenspace dimension (source: math/samples/src/Ch08/eigen.c, lines 160-184)

NOTE: SIGN FREEDOM

> If x is an eigenvector so is cx for any $c \neq 0$, mml's collinearity remark after Definition 4.7. The checks pin one representative per line, $(2, 1)$ and $(1, -1)$, and the sample verifies the eigen equation on exactly those representatives with integer arithmetic.

8.3. symmetric matrices and cholesky

Symmetric positive definite matrices are the square matrices of positive numbers: $A = A^\top$ and $x^\top Ax > 0$ for every $x \neq 0$ (mml Definition 3.4). Their square root is the Cholesky factorization, quoted: "A symmetric, positive definite matrix A can be factorized into a product $A = LL^\top$, where L is a lower-triangular matrix with positive diagonal elements" (mml Theorem 4.18). The factor L is unique, and matching entries of $A = LL^\top$ gives the computing pattern directly (mml Example 4.10, eqs 4.47-4.48): the diagonal entries are nested square roots $l_{jj} = \sqrt{a_{jj} - \sum_{k<j} l_{jk}^2}$ and the below-diagonal entries are nested divisions, each computable top to bottom from entries already known.

The dry run: the fixture is $A = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 5 & 5 \\ -2 & 5 & 14 \end{pmatrix}$, built as $LL^\top$ for $L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ -1 & 3 & 2 \end{pmatrix}$ so the factorization must return exactly that L.

1. First column: $l_{11} = \sqrt{4} = 2$, $l_{21} = 2/2 = 1$, $l_{31} = -2/2 = -1$.
2. Second column: $l_{22} = \sqrt{5 - 1^2} = 2$, then $l_{32} = (5 - (-1) \cdot 1)/2 = 3$.
3. Third column: $l_{33} = \sqrt{14 - (-1)^2 - 3^2} = \sqrt{4} = 2$.
4. Determinant bonus, mml Section 4.3: $\det(A) = \det(L)^2 = \prod_i l_{ii}^2 = (2 \cdot 2 \cdot 2)^2 = 64$, and Sarrus on A agrees.
5. Failure trace on the mml Example 3.4 matrix $\begin{pmatrix} 9 & 6 \\ 6 & 3 \end{pmatrix}$: symmetric, but $l_{11} = 3$, $l_{21} = 2$, then the radicand $3 - 2^2 = -1$. A negative radicand is not a rounding artifact, it is the proof that $x^\top Ax$ takes negative values, here with witness $x = (2, -3)$.

```c
// Cholesky A = L L^T, mml Theorem 4.18 with the entry patterns of eqs
// 4.47-4.48. Returns false and leaves the offending radicand in *bad when A
// is not positive definite.
static bool cholesky3(const double a[N][N], double l[N][N], double *bad) {
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      l[i][j] = 0.0;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j <= i; j++) {
      double sum = a[i][j];
      for (int k = 0; k < j; k++)
        sum -= l[i][k] * l[j][k];
      if (i == j) {
        if (sum <= 0.0) {
          *bad = sum;
          return false;
        }
        l[i][i] = sqrt(sum);
      } else {
        l[i][j] = sum / l[j][j];
      }
    }
  }
  return true;
}
```

listing 77: factor.c, the nested cholesky recurrence, returning false on a negative radicand (source: math/samples/src/Ch08/factor.c, lines 27-51)

```
// failure: [[9,6],[6,3]] is symmetric but x^T A x = (3x1 + 2x2)^2 - x2^2
// takes negative values, mml Example 3.4 A2, eigenvalues 6 +- 3 sqrt5
const double notspd[2][2] = {{9, 6}, {6, 3}};
double l2d[3][3], bad2 = 0.0;
double a2[3][3] = {{notspd[0][0], notspd[0][1], 0},
                   {notspd[1][0], notspd[1][1], 0},
                   {0, 0, 1}}; // embed in 3x3, kernel dim 1 is harmless
bool failc = cholesky3(a2, l2d, &bad2);
// hand trace: l11 = 3, l21 = 2, radicand for l22 = 3 - 2^2 = -1
CHECK(!failc && fabs(bad2 - (-1.0)) < 1e-12,
      "cholesky detects non-SPD input, radicand -1 at l22");
```

listing 78: factor.c, the non-spd input fails at l22 with radicand −1 (source: math/samples/src/Ch08/factor.c, lines 98-108)

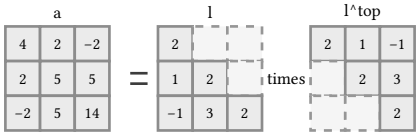


figure 46: cholesky splits a into two triangular shadows of itself

The two-number play: the honest matrix hits radicand 4 at its second pivot while the impostor hits −1 at the same spot. The decomposition is also the cheap determinant: 64 as 8^2 from three diagonal entries of L, against 64 from a full Sarrus pass.

8.4. qr factorization

Gram-Schmidt orthogonalization, met in chapter book 2, ch 7 (mathematics for programmers, inner products and projections) as eqs 3.67-3.68 of mml Section 3.8.3, subtracts from each new vector its projection onto the span of the previous ones. Run it on the columns of a matrix A and you get the QR factorization: $A = QR$ with Q orthogonal and R upper triangular. The entries fall out of the construction, $r_{ij} = q_i^\top a_j$ for the projections and r_{jj} the length of what is left over. Orthogonal Q means $Q^\top Q = I$ and $|\det(Q)| = 1$, and R carries the entire determinant in its diagonal: $\det(A) = \det(Q) \prod_i r_{ii}$, with the sign of $\det(Q)$ deciding orientation. This is the pattern the numerical solvers in the dsa book lean on, Q preserves lengths and angles (mml eqs 3.31-3.32) while R records the elimination. The orthogonalization itself is mml's, eqs 3.67-3.68, the packaging into QR and the determinant split are this book's own derivation, carried by the sample checks.

The dry run: the fixture is $A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$, determinant −2 by Sarrus.

1. Column 1: $u_1 = (1, 1, 0)$, $r_{11} = \sqrt{2}$, $q_1 = \frac{1,1,0}{\sqrt{2}}$.
2. Column 2: $r_{12} = q_1^\top a_2 = 1/\sqrt{2}$, and $a_2 - r_{12} q_1 = (1, 0, 1) - \frac{1}{2}(1, 1, 0) = \left(\frac{1}{2}, -\frac{1}{2}, 1\right)$ of length $\sqrt{\frac{3}{2}}$, so $q_2 = (1, -1, 2)/\sqrt{6}$.
3. Column 3: $r_{13} = 1/\sqrt{2}$, $r_{23} = 1/\sqrt{6}$, remainder $\left(-\frac{2}{3}, \frac{2}{3}, \frac{2}{3}\right)$ of length $2/\sqrt{3}$, $q_3 = (-1, 1, 1)/\sqrt{3}$.
4. Determinant: $\det(Q) = -1$ exactly (the closed form is $1/\sqrt{36}$ times a determinant of −6), $\prod_i r_{ii} = \sqrt{2} \cdot \sqrt{\frac{3}{2}} \cdot 2/\sqrt{3} = 2$, and $-1 \cdot 2 = -2$ matches Sarrus.

```
// ——— qr by gram-schmidt, mml Section 3.8.3 eqs 3.67-3.68 ———
const double a[N][N] = {{1, 1, 0}, {1, 0, 1}, {0, 1, 1}};
double q[N][N] = {0}, r[N][N] = {0};
for (int j = 0; j < N; j++) {
  double v[N];
  for (int i = 0; i < N; i++)
    v[i] = a[i][j];
  for (int i = 0; i < j; i++) {
    r[i][j] = q[0][i] * a[0][j] + q[1][i] * a[1][j] + q[2][i] * a[2][j];
    for (int k = 0; k < N; k++)
      v[k] -= r[i][j] * q[k][i];
  }
  r[j][j] = sqrt(v[0] * v[0] + v[1] * v[1] + v[2] * v[2]);
  for (int k = 0; k < N; k++)
    q[k][j] = v[k] / r[j][j];
}
// generator: r11 = sqrt2 = 1.4142135623730951, r22 = sqrt(3/2) =
// 1.224744871391589, r33 = 2/sqrt3 = 1.1547005383792515
CHECK(fabs(r[0][0] - 1.4142135623730951) < 1e-12 &&
          fabs(r[1][1] - 1.224744871391589) < 1e-12 &&
          fabs(r[2][2] - 1.1547005383792515) < 1e-12,
      "R diagonal sqrt2, sqrt(3/2), 2/sqrt3 pinned");
```

listing 79: factor.c, gram-schmidt as qr, projections into r and leftover lengths on the diagonal (source: math/samples/src/Ch08/factor.c, lines 110-131)

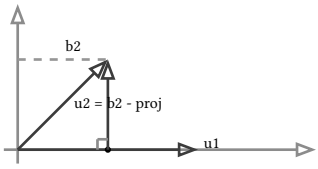


figure 47: gram-schmidt subtracts the projection, mml example 3.12 in two dimensions

The two-number play: det Q = −1 and diagonal product 2, the sign and the magnitude of one determinant split across the two factors. Sample pins the irrational diagonal $\sqrt{2}, \sqrt{3/2}, 2/\sqrt{3}$ to 12 decimals and verifies $Q^\top Q = I$, the triangular shape of R, and the $QR = A$ round trip.

8.5. singular value decomposition

The SVD drops every restriction the eigendecomposition needs. Quoted from mml Theorem 4.22: "The SVD of A is a decomposition of the form $A = U\Sigma V^\top$ with an orthogonal matrix $U \in \mathbb{R}^{m\times m}$ ... and an orthogonal matrix $V \in \mathbb{R}^{n\times n}$", Σ rectangular with nonnegative diagonal $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$, and it "exists for any matrix $A \in \mathbb{R}^{m\times n}$". The construction runs through the symmetric theory of section 3: the eigenvectors of $A^\top A$ are the right-singular vectors V, its eigenvalues are the squared singular values, $\sigma_i^2 = \lambda_i$ (mml eq 4.75), and the left-singular vectors come from normalizing the images, $u_i = 1/\sigma_i A v_i$ (mml eq 4.78). Geometrically $V^\top$ rotates the domain onto its principal axes, Σ scales axis i by σ_i, and U rotates into the codomain (mml Section 4.5.1, Figure 4.8).

The dry run: the fixture is $A = \begin{pmatrix} 2 & 3 \\ 0 & 2 \end{pmatrix}$.

1. $A^\top A = \begin{pmatrix} 4 & 6 \\ 6 & 13 \end{pmatrix}$, trace 17, determinant 16, eigenvalues $(17 \pm 15)/2 = 16$ and 1, so $\sigma_1 = 4$ and $\sigma_2 = 1$.
2. Right-singular vectors: eigenvector of $A^\top A$ for 16 is $(1, 2)/\sqrt{5}$, for 1 it is $(-2, 1)/\sqrt{5}$.
3. Left-singular vectors by eq 4.78: $u_1 = 1/4A(1, 2)/\sqrt{5} = (8, 4)/\left(4\sqrt{5}\right) = (2, 1)/\sqrt{5}$, and $u_2 = A(-2, 1)/\sqrt{5} = (-1, 2)/\sqrt{5}$.
4. The full statement: $\begin{pmatrix} 2 & 3 \\ 0 & 2 \end{pmatrix} = 1/\sqrt{5}\begin{pmatrix} 2 & -1 \\ 1 & 2 \end{pmatrix}\begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}1/\sqrt{5}\begin{pmatrix} 1 & 2 \\ -2 & 1 \end{pmatrix}$, verified to 10^{-12}, and $\det(A) = 4 = \sigma_1\sigma_2$, orientation preserved since both factors are rotations.

```
  // ---- 2x2 full svd ----
  const double a[2][2] = {{2, 3}, {0, 2}};
  // singular values derived here: A^T A = [[4,6],[6,13]], trace 17, det 16,
  // eigenvalues (17 +- 15)/2 = 16, 1, sigma = sqrt = 4, 1
  double att[2][2] = {{4, 6}, {6, 13}};
  double tr = att[0][0] + att[1][1];
  double det = att[0][0] * att[1][1] - att[0][1] * att[1][0];
  double disc = sqrt(tr * tr - 4.0 * det);
  double s1 = sqrt((tr + disc) / 2.0);
  double s2 = sqrt((tr - disc) / 2.0);
  CHECK(fabs(s1 - 4.0) < 1e-12 && fabs(s2 - 1.0) < 1e-12,
        "singular values 4 and 1 from eigenvalues of A^T A, eq 4.75");

  // pinned factors, numpy == closed form bitwise, 1/sqrt5 = 0.4472135954999579
  const double u[2][2] = {{0.8944271909999159, -0.4472135954999579},
                          {0.4472135954999579, 0.8944271909999159}};
  const double vt[2][2] = {{0.4472135954999579, 0.8944271909999159},
                           {-0.8944271909999159, 0.4472135954999579}};
  double sig[2][2] = {{4, 0}, {0, 1}};
  double recon = 0.0, ortho = 0.0;
  for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
      double s = 0.0;
      for (int k = 0; k < 2; k++)
        for (int m = 0; m < 2; m++)
          s += u[i][k] * sig[k][m] * vt[m][j];
      recon = fmax(recon, fabs(s - a[i][j]));
    }
  for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
      double su = 0.0, sv = 0.0;
      for (int k = 0; k < 2; k++) {
        su += u[k][i] * u[k][j];
        sv += vt[i][k] * vt[j][k];
      }
      ortho = fmax(ortho, fabs(su - (i == j ? 1.0 : 0.0)));
      ortho = fmax(ortho, fabs(sv - (i == j ? 1.0 : 0.0)));
    }
  CHECK(recon < 1e-12 && ortho < 1e-12,
        "U sigma V^T = A and both factors orthogonal, generator residual "
        "4.4e-16");

  // det A = 2*2 - 3*0 = 4 = sigma1 sigma2, both sides computed here
  CHECK((a[0][0] * a[1][1] - a[0][1] * a[1][0]) == 4.0 && s1 * s2 == 4.0,
        "det = sigma1 sigma2 = 4, orientation preserved");
```

listing 80: svd.c, singular values from the eigenvalues of a transposed a, then the pinned u and v round trip (source: math/samples/src/Ch08/svd.c, lines 36-80)

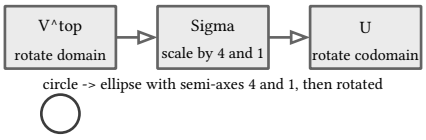


figure 48: svd as rotate, scale, rotate

Rectangular matrices get the same treatment. The mml Example 4.13 rectangle $\begin{pmatrix} 1 & 0 & 1 \\ -2 & 1 & 0 \end{pmatrix}$ has rank 2 and singular values $\sqrt{6}$ and 1, and the third right-singular vector $(-1, -2, 1)/\sqrt{6}$ comes with $Av_3 = 0$: the SVD hands you an orthonormal basis of the kernel for free (mml Section 4.5.2). The sample pins Av_3 to zero and counts the rank as

the number of nonzero singular values. For symmetric matrices the two decompositions coincide, the SVD of an SPD matrix is its eigendecomposition (mml Section 4.5.2 remark).

The two-number play: singular values 4 and 1 against eigenvalues 2 and 2. The same determinant 4 splits both ways, $4 \cdot 1 = 2 \cdot 2$, but the repeated eigenvalue leaves $\begin{pmatrix} 2 & 3 \\ 0 & 2 \end{pmatrix}$ defective, its only eigendirection is $(1, 0)$, so no diagonalization exists while the SVD sails through.

PITFALL: DO NOT BUILD THE SVD THIS WAY IN PRODUCTION

mml warns at the end of Example 4.13: "on a computer the approach illustrated here has poor numerical behavior, and the SVD of A is normally computed without resorting to the eigenvalue decomposition of $A^\top A$." The sample follows the textbook route because its job is to pin the definitions, library svd routines use variants of the Golub-Reinsch iteration.

8.6. low-rank approximation and pca

An outer product $u_i v_i^\top$ is a rank-1 matrix, and the SVD rewrites A as a sum of them weighted by singular values, $A = \sum_{i=1}^{r} \sigma_i u_i v_i^\top$ (mml eq 4.91). Keep only the first k terms and you get the rank-k approximation $\tilde{A}^{(k)}$ (mml eq 4.92), whose error is exactly the tail: the dropped terms are themselves an SVD, so $\left\| A - \tilde{A}^{(k)} \right\|_2 = \sigma_{k+1}$ in the spectral norm, the largest singular value (mml Definition 4.23, Theorem 4.24). Eckart-Young makes the truncation optimal, quoted: the rank-k SVD truncation is the argmin of $\|A - B\|_2$ over all matrices B of rank k (mml Theorem 4.25). The same tail controls the Frobenius norm, $\left\| A - \tilde{A}^{(k)} \right\|_F = \sqrt{\sum_{i>k} \sigma_i^2}$, and $\|A\|_F = \sqrt{\sum_i \sigma_i^2}$ is the whole spectrum in one number.

The dry run: the fixture is the mml Example 4.14 movie ratings matrix, 4 movies as rows, 3 viewers as columns, $A = \begin{pmatrix} 5 & 4 & 1 \\ 5 & 5 & 0 \\ 0 & 0 & 5 \\ 1 & 0 & 4 \end{pmatrix}$ with singular values $9.6438, 6.3639, 0.7056$.

1. Rank-1 reconstruction $\sigma_1 u_1 v_1^\top$ puts 4.7673 in the top-left cell where A has 5, science fiction captured, everything else smeared.
2. Rank-2 reconstruction (mml eq 4.102) repairs row 1 to $4.7801, 4.2419, 1.0244$ against the true $5, 4, 1$.
3. Errors: rank 1 misses by $\sqrt{6.3639^2 + 0.7056^2} = 6.4029$ in Frobenius, rank 2 misses by exactly $\sigma_3 = 0.7056$, both the direct residual and the tail sum agreeing to 10^{-9}.
4. Whole-matrix scale: $\|A\|_F = \sqrt{9.6438^2 + 6.3639^2 + 0.7056^2} = 11.5758$.

```
// ---- eckart-young on the movie matrix, mml eq 4.103 ----
const double mv[4][3] = {{5, 4, 1}, {5, 5, 0}, {0, 0, 5}, {1, 0, 4}};
const double sm[3] = {9.64381090011831, 6.363890888787544,
                      0.7055523214943976};
const double um[4][3] = {
    {-0.6709687841335595, 0.023628984619974036, -0.46466033666628836},
    {-0.7197424975501419, 0.20541998143184664, 0.47585715831065356},
    {-0.09389374348336019, -0.7704808395713496, 0.5268237283837064},
    {-0.15151103218346207, -0.6029955042683129, -0.529251826284588}};
const double vtm[3][3] = {
    {-0.7367489381728545, -0.6514631704576322, -0.18109870137154832},
    {0.08520720035382322, 0.17624686928798378, -0.9806511989866976},
    {-0.6707762183538074, 0.7379246429029692, 0.07434034091589163}};
// generator sigmas agree with the 4-decimal values printed in mml
// Figure 4.10 and Section 4.6
CHECK(fabs(sm[0] - 9.6438) < 5e-5 && fabs(sm[1] - 6.3639) < 5e-5 &&
          fabs(sm[2] - 0.7056) < 5e-5,
      "sigmas 9.6438, 6.3639, 0.7056 match mml Figure 4.10");

// rank-1 and rank-2 reconstructions, mml eqs 4.100-4.102
double a1[4][3] = {0}, a2k[4][3] = {0};
for (int i = 0; i < 4; i++)
  for (int j = 0; j < 3; j++) {
    a1[i][j] = sm[0] * um[i][0] * vtm[0][j];
    a2k[i][j] = a1[i][j] + sm[1] * um[i][1] * vtm[1][j];
  }
// generator: a1[0][0] = 4.767278461807643, rank-2 row 0 =
// (4.780091262791671, 4.241922823045146, 1.024371899371405), mml 4.7801
CHECK(fabs(a1[0][0] - 4.767278461807643) < 1e-9 &&
          fabs(a2k[0][0] - 4.780091262791671) < 1e-9 &&
          fabs(a2k[0][1] - 4.241922823045146) < 1e-9 &&
          fabs(a2k[0][2] - 1.024371899371405) < 1e-9,
      "rank-1 entry 4.7673 and rank-2 row 4.7801, 4.2419, 1.0244");

// frobenius errors: direct vs tail sum of squared singular values,
// generator: rank-1 6.402883047718368 vs 6.402883047718367, rank-2
// 0.7055523214943978
double e1[4][3], e2[4][3];
for (int i = 0; i < 4; i++)
  for (int j = 0; j < 3; j++) {
    e1[i][j] = mv[i][j] - a1[i][j];
    e2[i][j] = mv[i][j] - a2k[i][j];
  }
double fro1 = fro4(e1), fro2 = fro4(e2);
double tail1 = sqrt(sm[1] * sm[1] + sm[2] * sm[2]);
CHECK(fabs(fro1 - 6.402883047718368) < 1e-9 && fabs(fro1 - tail1) < 1e-9,
      "rank-1 frobenius error 6.4028830477 = sqrt(sigma2^2 + sigma3^2)");
CHECK(fabs(fro2 - 0.7055523214943978) < 1e-9 && fabs(fro2 - sm[2]) < 1e-9,
      "rank-2 frobenius error = sigma3 = 0.7055523214943976");

// ||A||_F = sqrt(sum sigma^2) = 11.575836902790225
CHECK(fabs(fro4(mv) - 11.575836902790225) < 1e-9 &&
          fabs(sqrt(sm[0] * sm[0] + sm[1] * sm[1] + sm[2] * sm[2]) -
               11.575836902790225) < 1e-9,
      "||A||_F = 11.575836902790225 = sqrt(sum of squared sigmas)");
```

listing 81: svd.c, rank-1 and rank-2 reconstructions with frobenius errors checked against the sigma tail (source: math/samples/src/Ch08/svd.c, lines 102-156)

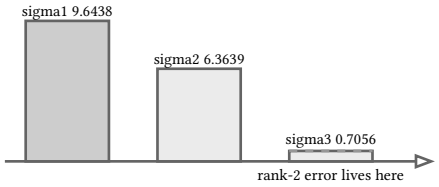


figure 49: singular values of the ratings matrix, the rank-2 tail is one small bar

Principal component analysis is this machinery turned around on data. Take the centered ratings, subtract each viewer's mean, here $(2.75, 2.25, 2.5)$, form the covariance $1/3 C^\top C$, and eigendecompose it. Its eigenvectors are exactly the right-singular vectors of C and its eigenvalues are $\sigma_i^2/3$: the generator pins $19.2612 = 7.6016^2/3$ for the top axis, a clean $1/6$ from $\sigma_2 = 1/\sqrt{2}$ in the middle, and the projections of the data onto the first axis carry $19.2612/(19.2612 + 0.1667 + 0.0721) = 0.9878$ of the variance. The ratings matrix is effectively two-dimensional, one science fiction axis and one French art house axis, exactly the reading mml draws from the same numbers in Example 4.15. Chapter book 2, ch 19 (mathematics for programmers, statistics and inference) builds the full inference story on this teaser.

```
// ---- pca teaser: mean-center, covariance eigendecomposition ----
// column means 11/4, 9/4, 5/2
double means[3] = {0, 0, 0};
for (int j = 0; j < 3; j++)
  for (int i = 0; i < 4; i++)
    means[j] += mv[i][j] / 4.0;
CHECK(means[0] == 2.75 && means[1] == 2.25 && means[2] == 2.5,
      "viewer means 2.75, 2.25, 2.5");

double cent[4][3];
for (int i = 0; i < 4; i++)
  for (int j = 0; j < 3; j++)
    cent[i][j] = mv[i][j] - means[j];
double cov[3][3];
for (int i = 0; i < 3; i++)
  for (int j = 0; j < 3; j++) {
    cov[i][j] = 0.0;
    for (int k = 0; k < 4; k++)
      cov[i][j] += cent[k][i] * cent[k][j] / 3.0;
  }
// generator eigenvalues 19.261225309749502, 0.16666666666666996,
// 0.0721080235838305 with principal axis
// w1 = (-0.5951686673983099, -0.5951686673983101, 0.5399523263165367)
const double w1[3] = {-0.5951686673983099, -0.5951686673983101,
                      0.5399523263165367};
const double lam1 = 19.261225309749502;
double cw[3] = {0, 0, 0};
for (int i = 0; i < 3; i++)
  for (int k = 0; k < 3; k++)
    cw[i] += cov[i][k] * w1[k];
CHECK(fabs(cw[0] - lam1 * w1[0]) < 1e-9 &&
          fabs(cw[1] - lam1 * w1[1]) < 1e-9 &&
          fabs(cw[2] - lam1 * w1[2]) < 1e-9,
      "covariance w1 = 19.2612253097 w1, eigenvalue equation");

// projection variance along the principal axis equals lam1, and the
// centered data has singular values (7.601557467338421, 0.7071067811865476,
// 0.46510651549026) whose squares over 3 reproduce all three eigenvalues,
// the middle one exactly 1/6 from sigma = 1/sqrt2
```

```
  double proj[4] = {0, 0, 0};
  for (int i = 0; i < 4; i++)
    for (int k = 0; k < 3; k++)
      proj[i] += cent[i][k] * w1[k];
  double var = 0.0;
  for (int i = 0; i < 4; i++)
    var += proj[i] * proj[i] / 3.0;
  CHECK(fabs(var - lam1) < 1e-9, "projected variance = lam1 = 19.2612253097");
  const double sc[3] = {7.601557467338421, 0.7071067811865476,
                        0.46510651549026};
  CHECK(fabs(sc[1] * sc[1] / 3.0 - 1.0 / 6.0) < 1e-12 &&
            fabs(sc[0] * sc[0] / 3.0 - lam1) < 1e-9,
        "covariance eigenvalues = sigma^2/3, middle pair 1/6 from 1/sqrt2");

  // variance explained by the first axis, generator ratio 0.9877551440897179
  CHECK(fabs(lam1 / (19.261225309749502 + 0.16666666666666996 +
                     0.0721080235838305) -
             0.9877551440897179) < 1e-9,
        "first axis explains 0.9877551440897 of the variance");
```

listing 82: svd.c, mean-center, covariance eigen check, projected variance and explained ratio (source: math/samples/src/Ch08/svd.c, lines 158-215)

The two-number play: 6.4029 down to 0.7056, the Frobenius error dropping by a factor of 9 when the second singular pair is kept, and 0.9878, the share of variance one axis of the centered ratings already explains.

A map of the whole chapter is mml Section 4.7, the matrix phylogeny of Figure 4.13: every rectangular matrix has the SVD, square invertible matrices are marked by $\det \neq 0$, non-defective ones diagonalize, symmetric ones diagonalize orthogonally, positive definite ones own a unique Cholesky factor, and orthogonal matrices are the rotations sitting inside all of it. Determinant and trace, the two scalars of section 1, label the same territory from above.

Calculus begins next in chapter book 2, ch 9 (mathematics for programmers, single-variable calculus), and every derivative there is a linear map whose decompositions are these.

sources: mml-book draft 2024-01-15, Deisenroth, Faisal, Ong, ch 4 matrix decompositions, determinant and trace [printed pp 99-105, pdf pp 105-111], eigenvalues and eigenvectors [printed pp 105-114, pdf pp 111-120], cholesky decomposition [printed pp 114-115, pdf pp 120-121], eigendecomposition and diagonalization [printed pp 115-119, pdf pp 121-125], singular value decomposition [printed pp 119-129, pdf pp 125-135], matrix approximation [printed pp 129-133, pdf pp 135-139], matrix phylogeny [printed pp 133-135, pdf pp 139-141], plus ch 2 Example 2.3 [printed p 23, pdf p 29] and ch 3 Gram-Schmidt [printed pp 89-90, pdf pp 95-96] and Definition 3.4 [printed p 74, pdf p 80], read from the verified anchor sheet and cross checked against the pdf pages on 2026-09-21, our own words and code throughout. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch08`, 52 checks in chapter 08 of the math suite, zero skipped.

9. single-variable calculus

Everything in this book eventually differentiates or integrates something, and this chapter builds that machinery for one variable at a time, from the limit up. Six moves: limits computed on converging ladders, the derivative as the slope the secants walk toward, the four differentiation rules verified against hand-coded derivative pairs, taylor series with pinned remainders and term counts, integration with riemann ladders and the fundamental theorem, and finally what survives of all this in floats, where the central difference has an optimum step near $\sqrt[\varepsilon]{3}$. Every behavioral claim below is one of the 70 checks in the 4 samples of chapter 09 or a sentence quoted from the mml anchor sheet for chapter 5. Where book 9, ch 36 (practical data structures and algorithms, numerical methods) drives newton iteration and simpson as contest tools, this chapter builds the limit definitions those drivers stand on, and the sister chapters on floats (book 2, ch 1 (mathematics for programmers, floating point anatomy), book 2, ch 2 (mathematics for programmers, error, conditioning, and stability)) own the representation story.

9.1. limits and continuity

A limit is the value a sequence or function settles toward, and the honest way to compute one in code is a ladder: evaluate at $\frac{1}{2^k}$, watch the gap to the candidate limit shrink at the rate the algebra predicts. The sequence $\left(1+\frac{1}{k}\right)^k$ rises to e with a gap of about $\frac{e}{2k}$, and $\frac{\sin(x)}{x}$ at $x = 2^{-}k$ sits below 1 with a gap of about $\frac{x^2}{6}$. Both predictions are checks, not prose: at $k = 2^{20}$ the sequence gap is 1.296e-6 against the predicted 1.296e-6, and at $x = 2^{-}10$ the sinc gap is 1.589457e-07 against $2^{-}\frac{20}{6} = 1.58946e-7$. The epsilon-delta definition underneath is the trade the ladder makes concrete: for every tolerance ε on the output there is a tolerance δ on the input that keeps you inside it.

Why rungs of $\frac{1}{2^k}$ and not $1/10^k$: every rung is exact in binary, so the only rounding in the ladder is the function's own arithmetic, never the evaluation points. A decimal ladder mixes representation error into every rung and the gap column stops meaning anything below $10^{-}15$. book 2, ch 1 (mathematics for programmers, floating point anatomy) owns this distinction, the ladder here just exploits it, which is also why so many checks below can assert exact equality on rung values.

The dry run: the fixtures are $\left(1+\frac{1}{k}\right)^k$ on $k = 1, 4, 2^{20}$, $\frac{\sin(x)}{x}$ on $x = 0.5, 2^{-}10$, the squeeze $\cos(x) \le \frac{\sin(x)}{x} \le 1$ at $x = 0.25$, and $f(x) = x^2$ at $a = 3$ with $L = 9$ and $\varepsilon = 1e-3$.

1. The ladder rungs land on exact binary values: $\left(1+\frac{1}{1}\right)^1 = 2$ and $\left(1+\frac{1}{4}\right)^4 = \frac{625}{256} = 2.44140625$.
2. The sinc rung at $x = 0.5$ is 0.9588510772084060, still 4 percent from 1 with the gap $\frac{x^2}{6} = 0.0417$ large, while at $2^{-}10$ the value is 0.9999998410542882.
3. For the epsilon-delta machine: $|x^2 - 9| = |x-3| \cdot |x+3|$, and $|x+3| < 7$ whenever $|x-3| < 1$, so any $\delta < \frac{\varepsilon}{7}$ works. With $\delta = \frac{\varepsilon}{7} = 1.4286e-4$ the residual above 3 is 8.5716e-4 and below 3 it is −8.5712e-4, both inside ε. With $\delta = \frac{\varepsilon}{100}$ the residual drops to 6.0000e-5.

Continuity is then one sentence: f is continuous at a when the limit equals the value, $\lim_{x \to a} f(x) = f(a)$. The holed expression $g(x) = \frac{x^2-4}{x-2}$ is the standard counterexample, undefined at $x = 2$ because the fraction reads 0/0, while its ladder closes on 4, the value of the filled function $\tilde{g}(x) = x + 2$. The check pins both: the rung at $2 + 2^{-}5$ evaluates to exactly 4.03125 and the rung at $2 + 2^{-}10$ to 4.0009765625, one rung from the fill. This removable discontinuity pattern, expression undefined at the point but limit existing, returns every time a formula divides by something that vanishes.

```c
// (1 + 1/k)^k on the k = 2^j ladder, converging to e from below.
static double seq(double k) { return pow(1.0 + 1.0 / k, k); }

// sin(x)/x, undefined at 0 but with limit 1.
static double sinc(double x) { return sin(x) / x; }

// (x^2 - 4)/(x - 2), removable hole at x = 2 with limit 4.
static double hole(double x) { return (x * x - 4.0) / (x - 2.0); }

int main(void) {
  // Ladder rungs are exact in binary: (1+1/1)^1 = 2, (1+1/4)^4 = 625/256.
  // gen: seq k=1 -> 2, seq k=4 -> 2.44140625
  CHECK(seq(1.0) == 2.0, "seq k=1 is 2");
  CHECK(seq(4.0) == 2.44140625, "seq k=4 is 2.44140625");
  // gen: seq k=2^20 minus e = -1.296e-06
  CHECK(fabs(seq(1048576.0) - 2.718281828459045) < 2e-6 &&
            fabs(seq(1048576.0) - 2.718281828459045) > 1e-6,
        "seq k=2^20 sits within 2e-6 of e");

  // sin(x)/x on x = 2^-j: gen 0.5 -> 0.9588510772084060,
  // 2^-10 -> 0.9999998410542882 with 1 - value = 1.589457e-07.
  CHECK(fabs(sinc(0.5) - 0.9588510772084060) < 1e-15, "sinc at 0.5");
  CHECK(fabs((1.0 - sinc(1.0 / 1024.0)) - 1.589457e-07) < 1e-12,
        "sinc gap at 2^-10 is x^2/6 to 7 digits");
```

listing 83: sequence and function ladders with pinned rungs, limits.c (source: math/samples/src/Ch09/limits.c, lines 22-45)

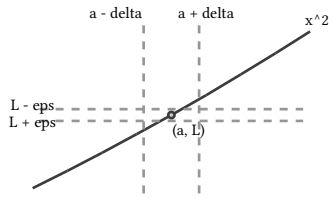


figure 50: the delta window inside the epsilon band, not to scale

```
// Delta for epsilon, f(x) = x^2 at a = 3, L = 9. |x^2 - 9| = |x-3||x+3| and
// |x + 3| < 7 whenever |x - 3| < 1, so delta = eps/7 works for eps = 1e-3.
double eps = 1e-3;
double delta = eps / 7.0;
// gen: resid at 3+delta = 8.571633e-04, at 3-delta = -8.571224e-04,
// at 3+eps/100 = 6.000010e-05. Pins are two-sided on |residual|: the
// pinned magnitude must match within tol, not merely stay under it.
CHECK(fabs(fabs((3.0 + delta) * (3.0 + delta) - 9.0) - 8.571633e-04) < 1e-9 &&
          fabs((3.0 + delta) * (3.0 + delta) - 9.0) < eps,
      "delta = eps/7 works above 3");
CHECK(fabs(fabs((3.0 - delta) * (3.0 - delta) - 9.0) - 8.571224e-04) < 1e-9 &&
          fabs((3.0 - delta) * (3.0 - delta) - 9.0) < eps,
      "delta = eps/7 works below 3");
CHECK(fabs(fabs((3.0 + eps / 100.0) * (3.0 + eps / 100.0) - 9.0) -
           6.000010e-05) < 1e-10,
      "any smaller delta also works");

// Continuity: the filled function g(x) = x + 2 has g(2) = 4 equal to the
// limit of the holed expression. gen (x^2-4)/(x-2) at 2+2^-5 -> 4.03125
// exactly, and at 2+2^-10 -> 4.0009765625, one rung from the fill.
CHECK(hole(2.0 + 1.0 / 32.0) == 4.03125, "hole ladder at 2+2^-5 is x+2");
CHECK(hole(2.0 + 1.0 / 1024.0) == 4.0 + 1.0 / 1024.0 &&
          fabs(hole(2.0 + 1.0 / 1024.0) - 4.0) < 1.0 / 512.0,
      "ladder closes on the filled value 4");
// The expression itself is undefined at 2: 0/0 is NaN, the limit is not.
```

listing 84: delta for epsilon on x^2 and the removable hole, limits.c (source: math/samples/src/Ch09/limits.c, lines 51-75)

The two-number play: $\delta = \frac{\varepsilon}{7} = 1.43 \times 10^{-}4$ leaves residual 8.57e-4, while $\delta = \frac{\varepsilon}{100}$ leaves 6.0e-5, the same epsilon bought two ways.

9.2. the derivative as a slope machine

mml Definition 5.1 (Difference Quotient) defines $\delta y/\delta x := (f(x+\delta x) - f(x))/\delta x$ as "the slope of the secant line through two points on the graph of f" [printed p 141 / pdf p 147], and Definition 5.2 (Derivative) takes the limit, $\frac{\mathrm{d}f}{\mathrm{d}x} := \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$, where "the secant becomes a tangent" and "the derivative of f points in the direction of steepest ascent" [printed p 141 / pdf p 147]. In code the limit is never taken, it is approached: three quotients on a fixed polynomial, forward $(f(x+h) - f(x))/h$, backward $(f(x) - f(x-h))/h$, and central $(f(x+h) - f(x-h))/(2h)$, each with its own error law. The fixture is $f(x) = x^4 - 3x^2 + 7x - 2$ with coded derivative $f'(x) = 4x^3 - 6x + 7$, so $f(1) = 3$ and $f'(1) = 5$ exactly, and the taylor expansion of each quotient around $h = 0$ reads the error laws straight off: forward misses by $f''(1)/2 \cdot h + f'''(1)/6 \cdot h^2 + h^3 = 3h + 4h^2 + h^3$, backward by the same with the sign of the linear term flipped, central by $f'''(1)/6 \cdot h^2 = 4h^2$ with the linear term cancelled.

The dry run: one step of the secant walk at $h = 0.5$, all values exact in binary.

1. $f(1.5) = 5.0625 - 6.75 + 10.5 - 2 = 6.8125$ and $f(0.5) = 0.0625 - 0.75 + 3.5 - 2 = 0.8125$.
2. Forward: $(6.8125 - 3)/0.5 = 7.625$. Backward: $(3 - 0.8125)/0.5 = 4.375$. Central: $(6.8125 - 0.8125)/1 = 6$. The true slope is 5 and the central quotient already carries only $4h^2 = 1$.
3. One coarse secant across the whole interval, from 1 to 3, has slope $(f(3) - f(1))/2 = 35$ where the tangent reads 5.

```
static double f(double x) { return x * x * x * x - 3 * x * x + 7 * x - 2; }
static double fd(double x) { return 4 * x * x * x - 6 * x + 7; }

static double fwd(double x, double h) { return (f(x + h) - f(x)) / h; }
static double bwd(double x, double h) { return (f(x) - f(x - h)) / h; }
static double ctr(double x, double h) {
  return (f(x + h) - f(x - h)) / (2 * h);
}
```

listing 85: the fixed polynomial and its three quotients, deriv.c (source: math/samples/src/Ch09/deriv.c, lines 26-33)

The order law is the chapter's first pinned convergence rate. On the ladder $h = 2^{-}k$ the central error is exactly $4h^2$ while the rungs last, 1.5625e-2 at $k = 4$ and 3.90625e-3 at $k = 5$, and the ratio of consecutive errors is exactly 4: halve the step, quarter the error. The forward error carries its $3h$ term, 2.9335e-3 at $h = 2^{-}10$, and the check pins it against the full cubic prediction $3h + 4h^2 + h^3$ to 1e-12.

Two derivative facts ride inside those error laws. First, the derivative is itself a function: the sample codes f' and f'' and f''' as separate expressions, $f'(x) = 4x^3 - 6x + 7$, $f''(x) = 12x^2 - 6$, $f'''(x) = 24x$, with $f''(1) = 6$ and $f'''(1) = 24$ supplying the error-law coefficients 3 and 4. Differentiating the derivative is just applying the machinery again, and the coefficients in the error laws are how the higher derivatives first pay rent. Second, the expansion that produced the error laws is itself the next section's subject: writing $f(x \pm h)$ as a polynomial in h and cancelling is exactly what a taylor series does, so the derivative's own analysis tool becomes the next construction.

```
int main(void) {
  CHECK(f(1.0) == 3.0 && fd(1.0) == 5.0, "f(1) = 3, f'(1) = 5");
  // gen: h = 0.5 -> forward 7.625, backward 4.375, central 6 exactly
  CHECK(fwd(1.0, 0.5) == 7.625, "forward quotient at h = 0.5 is 7.625");
  CHECK(bwd(1.0, 0.5) == 4.375, "backward quotient at h = 0.5 is 4.375");
  CHECK(ctr(1.0, 0.5) == 6.0, "central quotient at h = 0.5 is 6");
  CHECK((f(3.0) - f(1.0)) / 2.0 == 35.0, "secant walk 1 -> 3 has slope 35");

  // Order pinning on the h = 2^-k ladder. Central error = f'''(1)/6 h^2
  // = 4h^2 exactly at these h (gen: err minus 4h^2 = 0 at 2^-10).
  double h4 = ldexp(1.0, -4), h5 = ldexp(1.0, -5), h10 = ldexp(1.0, -10);
  CHECK(fabs(ctr(1.0, h4) - 5.0) == 4 * h4 * h4, "central error is 4h^2");
  CHECK(fabs(ctr(1.0, h4) - 5.0) / fabs(ctr(1.0, h5) - 5.0) == 4.0,
        "halving h quarters the central error");
  CHECK(fabs(ctr(1.0, ldexp(1.0, -8)) - 5.0) /
                fabs(ctr(1.0, ldexp(1.0, -9)) - 5.0) ==
            4.0,
        "order 2 holds deeper on the ladder");
  // Forward error = f''(1)/2 h + f'''/6 h^2 + f''''/24 h^3 = 3h + 4h^2 + h^3.
  CHECK(fabs(fwd(1.0, h10) - 5.0 -
             (3 * h10 + 4 * h10 * h10 + h10 * h10 * h10)) < 1e-12,
        "forward error is 3h + 4h^2 + h^3");
```

listing 86: dry-run checks and the order pinning ladder, deriv.c (source: math/samples/src/Ch09/deriv.c, lines 59-80)

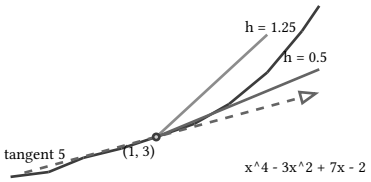


figure 51: three secants walk toward the tangent at (1, 3)

The two-number play: forward misses by 2.03e-1 at $h = 2^{-}4$ while central misses by 1.56e-2, thirteen times closer for the same two evaluations.

9.3. the differentiation rules

mml section 5.1.2 states the four rules [printed pp 145-146 / pdf pp 151-152]: sum $(f+g)' = f' + g'$ (eq 5.31), product $(fg)' = f'g + fg'$ (eq 5.29), quotient $(f/g)' = (f'g - fg')/g^2$ (eq 5.30), chain $(g \circ f)' = g'(f(x))f'(x)$ (eq 5.32). This chapter refuses to trust them symbolically. Each rule is checked numerically against a hand-coded pair, a function and its derivative both written out explicitly, with the rule's value compared to a central difference of the composed expression at $h = 1e-6$. The fixtures are $u(x) = x^3$ with $u'(x) = 3x^2$ and $v(x) = e^x$ with $v'(x) = e^x$, evaluated at $x = 0.7$. This is exactly the gradient- implementation check mml recommends for code that differentiates, finite differences against the analytic derivative at small h [printed p 149 / pdf p 155], run here on scalars.

The dry run: the product rule at $x = 0.7$, where $u = 0.343$, $u' = 1.47$, $v = v' = 2.0137527074704766$.

1. Rule value: $u'v + uv' = 1.47 \cdot 2.0138 + 0.343 \cdot 2.0138 = 3.6509336586439738$.
2. Central difference of the product at $h = 1e-6$: 3.6509336587275953, a gap of 8.4e-11, which is the $O(h^2)$ truncation of the central quotient plus a few ulps.
3. The quotient rule on the same pair: $(1.47 \cdot 2.0138 - 0.343 \cdot 2.0138)/2.0138^2 = 0.5596516373729186$ against central 0.5596516373801430.

```
// Sum rule eq 5.31 at x = 0.7, h = 1e-6.
// gen: central of sum 3.4837527074718122 vs rule 3.4837527074704764
double x = 0.7, hh = 1e-6;
CHECK(fabs(ud(x) + vd(x) - central_of(sum_fg, x, hh)) < 1e-8,
      "sum rule matches coded pair");
// Product rule eq 5.29. gen: central 3.650933658651812 vs rule
// 3.6509336586439738
CHECK(fabs(ud(x) * v(x) + u(x) * vd(x) - central_of(prod_fg, x, hh)) < 1e-8,
      "product rule matches coded pair");
// Quotient rule eq 5.30. gen: central 0.5596516373727088 vs rule
// 0.5596516373729186
CHECK(fabs((ud(x) * v(x) - u(x) * vd(x)) / (v(x) * v(x)) -
           central_of(quot_fg, x, hh)) < 1e-8,
      "quotient rule matches coded pair");
// Power rule from mml Example 5.2: (x^5)' = 5x^4 at 1.3.
// gen: 5*1.3^4 = 14.2805, central 14.2805000000169
CHECK(fabs(5 * 1.3 * 1.3 * 1.3 * 1.3 - central_of(p5, 1.3, hh)) < 1e-8,
      "power rule (x^5)' = 5x^4");

// Chain rule eq 5.32 on mml Example 5.5: coded composition derivative
// g'(f(x)) f'(x) = 8(2x+1)^3. gen: 0.3 -> 32.768, 1.4 -> 438.976.
double c3 = outer4_d(inner(0.3)) * inner_d(0.3);
double c14 = outer4_d(inner(1.4)) * inner_d(1.4);
CHECK(fabs(c3 - 8 * inner(0.3) * inner(0.3) * inner(0.3)) < 1e-12 &&
          fabs(c14 - 8 * inner(1.4) * inner(1.4) * inner(1.4)) < 1e-9,
      "coded chain equals mml eq 5.38 closed form");
// gen: central of (2x+1)^4 at 0.3 = 32.768000000051202
CHECK(fabs(c3 - central_of(comp54, 0.3, hh)) < 1e-8,
      "chain rule matches central difference");
```

listing 87: each rule asserted against a central difference of the coded pair, deriv.c (source: math/samples/src/Ch09/deriv.c, lines 82-110)

The chain rule gets the full mml Example 5.5 treatment [printed pp 145-146 / pdf pp 151-152]: $h(x) = (2x+1)^4$ as the composition $g \circ f$ with coded inner $f(x) = 2x + 1$, $f' = 2$ and coded outer $g(t) = t^4$, $g'(t) = 4t^3$. The composed derivative $g'(f(x))f'(x)$ evaluates to 32.768 at $x = 0.3$, the closed form $8(2x+1)^3$ of eq 5.38 also

gives 32.768, and the central difference of the composed function gives 32.768000000051202. Three routes, one number to 11 digits. The power rule $\mathrm{d}/\mathrm{d}x x^n = nx^{n-1}$, derived from the limit definition via the binomial expansion in mml Example 5.2 [printed p 142 / pdf p 148], rides along at $x = 1.3$: $5 \cdot 1.3^4 = 14.2805$ against central 14.2805000000169.

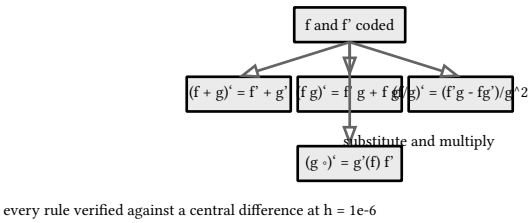


figure 52: the four rules as one pattern, outer derivatives times inner ones

VERIFY: rules are checked, not trusted

Each CHECK pairs the rule's closed form with a central difference of the composed expression. If a rule were miscoded the two sides diverge at $O(1)$, if the fixture pair were wrong they diverge at $O(h)$, and only a correct pair agrees to $O(h^2)$. The gap you should see is about 1e-10, and it is.

The quotient rule deserves its check more than the others. The sign in the numerator, $f'g - fg'$, is the most commonly flipped sign in applied differentiation, and a flipped sign fails the CHECK at $O(1)$, a gap of about 1.1 on this fixture rather than 1e-10. The chain rule deserves its check for a different reason: it is the only rule whose operands are functions of different variables, g' evaluated at $f(x)$ rather than at x, and wiring the argument through the composition is where code drifts from math.

The two-number play: the chain at $x = 0.3$ gives 32.768 in exact form and 32.7680000000512 by difference, while at $x = 1.4$ it gives 438.976, the same agreement scaled up.

9.4. taylor series

mml Definition 5.3 (Taylor Polynomial) fixes the object, $T_n(x) := \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k$ [printed p 142 / pdf p 148], and Definition 5.4 (Taylor Series) extends the sum to infinity, calling f analytic when $f(x) = T_\infty(x)$ [printed pp 142-143 / pdf pp 148-149]. At $x_0 = 0$ these are maclaurin series. The four working series are $e^x = \sum x^k/k!$, sin and cos alternating over odd and even factorials (mml eqs 5.26-5.27, printed with the sin + cos combination in Example 5.4 [printed p 144 / pdf p 150], and book 2, ch 3 (mathematics for programmers, trigonometry) builds them again from the angle identities), and $\ln(1 + x) = \sum (-1)^{k+1} x^k/k$.

The dry run: e^x at $x = 1$, term by term, with the lagrange remainder bound $|R_n| \le 3/(n+1)!$ using $e^\xi < 3$ for $\xi \in (0, 1)$.

1. $T_2(1) = 1 + 1 + \frac{1}{2} = 2.5$ exactly. $T_5(1) = 2.7166666666666668$, remainder 1.615e-3, bound $\frac{3}{720} = 4.17e-3$.
2. The bound crosses 1e-12 between $n = 14$ (3/15! = 2.29e-12, fails) and $n = 15$ (3/16! = 1.43e-13, passes), so 16 terms suffice by the bound. The actual remainder at $n = 15$ is 5.08e-14, nearly three times inside the bound.
3. sin at $x = 1$ is cheaper per digit because its terms carry $1/(2k+1)!$: $T_9(1) = 0.8414710097001764$ with remainder 2.489e-8, and $n = 14$ already clears 1e-12 by the bound.

```c
// Exact integer factorial as a double, safe through 22!.
static double fact(int n) {
  double acc = 1.0;
  for (int i = 2; i ≤ n; i++) {
    acc *= (double)i;
  }
  return acc;
}

// T_n(1) of exp: sum 1/k!, k = 0..n.
static double exp_taylor(int n) {
  double s = 0.0;
  for (int k = 0; k ≤ n; k++) {
    s += 1.0 / fact(k);
  }
  return s;
}

// T_n(1) of sin: terms x^(2k+1)/(2k+1)! with alternating signs.
static double sin_taylor(int n) {
  double s = 0.0, x = 1.0;
  for (int k = 0; 2 * k + 1 ≤ n; k++) {
    s += ((k % 2 == 0) ? 1.0 : -1.0) * pow(x, 2 * k + 1) / fact(2 * k + 1);
  }
  return s;
}

// T_n(1) of cos.
static double cos_taylor(int n) {
  double s = 0.0, x = 1.0;
  for (int k = 0; 2 * k ≤ n; k++) {
    s += ((k % 2 == 0) ? 1.0 : -1.0) * pow(x, 2 * k) / fact(2 * k);
  }
  return s;
}

// Partial sum of ln(1+x) = sum (-1)^(k+1) x^k / k, k = 1..n.
static double ln_taylor(double x, int n) {
  double s = 0.0;
  for (int k = 1; k ≤ n; k++) {
    s += ((k % 2 == 1) ? 1.0 : -1.0) * pow(x, k) / (double)k;
  }
  return s;
}

// mml eq 5.17: T_6 of x^4 at x_0 = 1 in powers of (x - 1).
static double t6_x4(double x) {
  double z = x - 1.0;
  return 1.0 + 4.0 * z + 6.0 * z * z + 4.0 * z * z * z + z * z * z * z;
}
```

listing 88: the four series machines and the exact polynomial fixture, taylor.c (source: math/samples/src/Ch09/taylor.c, lines 24-73)

```
int main(void) {
  // exp at x = 1. gen: T_2 = 2.5, T_5 = 2.7166666666666668 with remainder
  // 1.6151617924e-03, Lagrange bound e/(6!) = 3/720 = 4.1667e-03.
  CHECK(exp_taylor(2) == 2.5, "T_2(1) of exp is 2.5");
  CHECK(fabs(exp_taylor(5) - 2.7166666666666668) < 5e-15, "T_5(1) of exp");
  double r5 = fabs(exp(1.0) - exp_taylor(5));
  CHECK(r5 > 1.5e-3 && r5 < 1.7e-3, "R_5(1) of exp is 1.615e-3");
  CHECK(3.0 / fact(6) > r5, "Lagrange bound 3/(n+1)! covers R_5");
  // Term count for 1e-12: smallest n with 3/(n+1)! < 1e-12 is n = 15
  // (3/15! = 2.29e-12 fails, 3/16! = 1.43e-13 passes), and the actual
  // remainder there is 5.08e-14. gen: T_15 = 2.7182818284589945.
  int n_min = 0;
  while (3.0 / fact(n_min + 1) ≥ 1e-12) {
    n_min++;
  }
  CHECK(n_min == 15, "bound needs n = 15 for 1e-12 at x = 1");
  CHECK(fabs(exp(1.0) - exp_taylor(15)) < 1e-12, "T_15 hits 1e-12");
```

listing 89: remainder bound and term count for exp at x = 1, taylor.c (source: math/samples/src/Ch09/taylor.c, lines 75-91)

series	at x	T_n	remainder	bound
e^x	1	T_5	1.615e-3	4.17e-3
e^x	1	T_{15}	5.08e-14	1.43e-13
$\sin x$	1	T_9	2.489e-8	2.76e-7
$\ln(1+x)$	0.5	s_{20}	1.537e-8	2.27e-8

Every bound in the table is a pinned check and every actual remainder sits inside its bound, which is the whole method: the bound is computable before you sum, the remainder only after. When the remainder goes to zero for every x in some interval, $T_\infty(x) = f(x)$ there and f is analytic on that interval, and the largest interval where this happens is set by the term ratio $|a_{k+1}(x-x_0)^{k+1}| \,/\, |a_k(x-x_0)^k|$: the series converges where the ratio's limit stays under 1. The $\ln(1+x)$ row shows the alternating-series bound $x^{n+1}/(n+1)$, and the radius of convergence is part of the fixture. The term ratio of the ln series is $x \cdot k/(k+1)$, which tends to x, so at $x = 0.5$ the ratio at $k = 20$ is 0.476 and the partial sums settle, while at $x = 2$ it is 1.905 and they blow up, 5.07 at 5 terms, −64.8 at 10, −34359.7 at 20. Outside the radius the series is not slow, it is wrong.

```
  // sin and cos at x = 1. gen: T_9(1) of sin = 0.84147100970017641 with
  // remainder 2.4892279860e-08, T_8(1) of cos = 0.54030257936507942 with
  // remainder 2.7349693965e-07, minimal n with 1/(n+1)! < 1e-12 is n = 14.
  CHECK(fabs(sin_taylor(9) - 0.84147100970017641) < 1e-14, "T_9(1) of sin");
  CHECK(fabs(fabs(sin(1.0) - sin_taylor(9)) - 2.4892279860e-08) < 1e-13,
        "R_9(1) of sin is 2.489e-8");
  CHECK(fabs(cos_taylor(8) - 0.54030257936507942) < 1e-14 &&
            fabs(cos(1.0) - cos_taylor(8)) < 1e-6,
        "T_8(1) of cos and its remainder");
  n_min = 0;
  while (1.0 / fact(n_min + 1) ≥ 1e-12) {
    n_min++;
  }
  CHECK(n_min == 14, "sin bound needs n = 14 for 1e-12 at x = 1");
  CHECK(fabs(sin(1.0) - sin_taylor(15)) < 1e-14, "T_15(1) of sin at 3e-15");

  // ln(1+x). gen: s_20 at 0.5 = 0.40546509273417702, true ln(1.5) =
  // 0.40546510810816438, alternating bound 0.5^21/21 = 2.270653e-08 vs actual
  // error 1.5373987349e-08.
  double s20 = ln_taylor(0.5, 20);
  CHECK(fabs(s20 - 0.40546509273417702) < 1e-14, "s_20 of ln(1.5)");
  CHECK(fabs(log(1.5) - s20) < pow(0.5, 21) / 21.0,
        "alternating bound covers the remainder");
  CHECK(fabs(log(1.5) - s20) > 1e-8 && fabs(log(1.5) - s20) < 2e-8,
        "ln remainder after 20 terms is 1.54e-8");
  // Radius of convergence 1: at x = 2 the partial sums diverge (gen:
  // s_5 = 5.0667, s_10 = -64.825, s_20 = -34359.7) while the term ratio
  // x k/(k+1) at k = 20 is 0.476 below 1 for x = 0.5 and 1.905 above for
  // x = 2.
  CHECK(fabs(ln_taylor(0.5, 10) - 0.4054346478174603) < 1e-14,
        "s_10 of ln(1.5) still converging");
  CHECK(fabs(ln_taylor(2.0, 20)) > fabs(ln_taylor(2.0, 10)) &&
            fabs(ln_taylor(2.0, 10)) > fabs(ln_taylor(2.0, 5)),
        "partial sums blow up at x = 2");
  CHECK(0.5 * 20.0 / 21.0 < 1.0 && 2.0 * 20.0 / 21.0 > 1.0,
        "term ratio crosses 1 between x = 0.5 and x = 2");

  // mml Example 5.3: the degree-6 Taylor polynomial of x^4 at x_0 = 1 is
  // exact. gen: T_6(1.5) = 5.0625 = 1.5^4, T_6(0.5) = 0.0625 = 0.5^4.
  CHECK(t6_x4(1.5) == 5.0625 && 1.5 * 1.5 * 1.5 * 1.5 == 5.0625,
        "T_6 of x^4 at 1.5 equals f");
  CHECK(t6_x4(0.5) == 0.0625, "T_6 of x^4 at 0.5 equals f");
```

listing 90: sin and cos remainders, the ln radius, and the exact x^4 case, taylor.c (source: math/samples/src/Ch09/taylor.c, lines 93-134)

The polynomial case closes the loop with mml Example 5.3 [printed p 143 / pdf p 149]: for $f(x) = x^4$ at $x_0 = 1$, the derivatives at 1 are $1, 4, 12, 24, 24, 0, 0$ and the divisions by $k!$ turn eq 5.17 into $T_6(x) = 1 + 4(x-1) + 6(x-1)^2 + 4(x-1)^3 + (x-1)^4$, which multiplied out is exactly x^4. A degree-n taylor polynomial of a polynomial of degree $k \leq n$ is the polynomial itself, and the check evaluates T_6 in powers of $(x-1)$ at 1.5 and 0.5 and gets $5.0625 = 1.5^4$ and $0.0625 = 0.5^4$, both exact in binary.

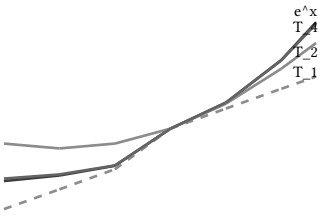


figure 53: partial sums of e^x straighten out to the curve as n grows

NOTE: bounds before sums

The lagrange bound $3/(n+1)!$ for e^x on $(0,1)$ costs one factorial to evaluate and one comparison to turn into a term count. The measured remainder 5.08e-14 sits almost three times inside the bound 1.43e-13 because the bound evaluates e^ξ at the worst point. Use the bound to choose n, then spend the actual remainder only if you have the true value to compare against.

The two-number play: five terms of e^x at 1 leave error 1.6e-3, fifteen terms leave 5.1e-14, ten extra terms buy ten digits.

9.5. integration and the fundamental theorem

The definite integral is the area signature of a function, computed in code as riemann sums on a lattice of n panels: left and right sums stamp the height at the panel edges, the midpoint sum at the center. The fixture is $g(x) = x^3 + 2x$ on $[0, 2]$ with antiderivative $G(x) = \frac{x^4}{4} + x^2$ and exact integral $G(2) - G(0) = 8$. The convergence orders are pinned by doubling n: left and right sums carry $O(1/n)$ error, so doubling halves it, while the midpoint sum carries $O(1/n^2)$, so doubling quarters it. For this cubic the midpoint error is exactly $2/n^2$, 0.125 at $n = 4$ and 0.03125 at $n = 8$, ratios exactly 4, and the left error ratio is 1.913 from 4 to 8 panels, closing on 2 as the higher Euler-Maclaurin terms fade.

The dry run: the $n = 4$ lattice on $[0, 2]$, panel width 0.5, all sums exact in binary.

1. Edge values $g(0), g(0.5), g(1), g(1.5), g(2) = 0, 1.125, 3, 6.375, 12$.
2. Left sum: $0.5 \cdot (0 + 1.125 + 3 + 6.375) = 5.25$, undershooting 8 by 2.75 because g rises.
3. Right sum: $0.5 \cdot (1.125 + 3 + 6.375 + 12) = 11.25$, overshooting by 3.25.
4. Midpoint sum: $0.5 \cdot (0.516 + 1.922 + 4.453 + 8.859) = 7.875$, off by 0.125 with the same four evaluations.

```
static double g(double x) { return x * x * x + 2 * x; }
static double G(double x) {
  return x * x * x * x / 4 + x * x;
} // antiderivative

static double p(double x) { return x * x * x * x - 3 * x * x + 7 * x - 2; }
static double P_ant(double x) {
  return x * x * x * x * x / 5 - x * x * x + 3.5 * x * x - 2 * x;
}

typedef double (*fn1)(double);

static double left_sum(fn1 f, double a, double b, int n) {
  double h = (b - a) / n, s = 0.0;
  for (int i = 0; i < n; i++) {
    s += f(a + i * h);
  }
  return s * h;
}

static double right_sum(fn1 f, double a, double b, int n) {
  double h = (b - a) / n, s = 0.0;
  for (int i = 1; i ≤ n; i++) {
    s += f(a + i * h);
  }
  return s * h;
}

static double mid_sum(fn1 f, double a, double b, int n) {
  double h = (b - a) / n, s = 0.0;
  for (int i = 0; i < n; i++) {
    s += f(a + (i + 0.5) * h);
  }
  return s * h;
}
```

listing 91: the three riemann sums as one loop each, ftc.c (source: math/samples/src/Ch09/ftc.c, lines 25-59)

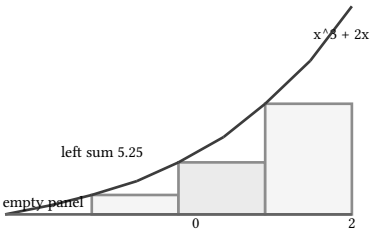


figure 54: left rectangles under x^3 + 2x, the n = 4 lattice

```
int main(void) {
  // Exact by antiderivative: G(2) - G(0) = 4 + 4 = 8.
  CHECK(G(2.0) - G(0.0) == 8.0, "antiderivative gives exact integral 8");
  // gen n = 4: left 5.25 (err 2.75), right 11.25 (err -3.25), mid 7.875
  // (err 0.125)
  CHECK(left_sum(g, 0, 2, 4) == 5.25, "left sum n=4 is 5.25");
  CHECK(right_sum(g, 0, 2, 4) == 11.25, "right sum n=4 is 11.25");
  CHECK(mid_sum(g, 0, 2, 4) == 7.875, "midpoint n=4 is 7.875");
  // Order pinning by doubling n. gen: left error ratio 4->8 = 1.913 and
  // 8->16 = 1.957 (first order), midpoint ratios exactly 4 (second order).
  double lr =
      fabs(8.0 - left_sum(g, 0, 2, 4)) / fabs(8.0 - left_sum(g, 0, 2, 8));
  CHECK(lr > 1.9 && lr < 2.0, "left error halves as n doubles");
  CHECK(fabs(8.0 - mid_sum(g, 0, 2, 4)) / fabs(8.0 - mid_sum(g, 0, 2, 8)) ==
            4.0,
        "midpoint error quarters as n doubles");
  CHECK(fabs(8.0 - mid_sum(g, 0, 2, 8)) / fabs(8.0 - mid_sum(g, 0, 2, 16)) ==
            4.0,
        "midpoint order 2 holds deeper");
  // gen: mid n=64 = 7.99951171875, double mid n=512 err = 7.63e-06.
  CHECK(mid_sum(g, 0, 2, 64) == 7.99951171875, "midpoint n=64 exact rung");
  CHECK(fabs(mid_sum(g, 0, 2, 512) - 8.0) < 1e-5, "midpoint n=512 at 7.6e-6");
```

listing 92: the ladder of sums with order ratios pinned, ftc.c (source: math/samples/src/Ch09/ftc.c, lines 79-100)

The $n = 4$ row also shows a free error certificate: on a rising integrand the left sum undershoots and the right sum overshoots, 5.25 and 11.25 bracketing the true 8, so the pair bounds the answer without knowing the antiderivative. The bracket width is $(g(b) - g(a)) \cdot (b - a)/n = 12 \cdot 2/4 = 6$, shrinking as $1/n$, and the midpoint sits inside it at 7.875. A bracket plus a rate is often all a caller needs, and book 9, ch 36 (practical data structures and algorithms, numerical methods) turns exactly this shape into the simpson drivers the contest book uses.

The fundamental theorem of calculus is the bridge, and it is checked from both directions on a fixed case. Part 2, integrate then differentiate: the numeric integrator $G_{\text{num}(x)}$, a composite midpoint with 4096 panels from 0 to x, is differentiated by a central difference with $h = 1e - 5$, and the result matches $g(1.5) = 6.375$ to 1.0e-7 and $g(0.9) = 2.529$ to 2.2e-8. Part 1, differentiate then integrate: the exact antiderivative G differenced centrally at 1.1 lands within 2.8e-10 of $g(1.1)$, the gap being cancellation noise in the difference of two nearly equal values, not truncation, which is only $G'''(1.1)/6 \cdot h^2 = g''(1.1)/6 \cdot h^2 = 1.1e - 12$, two orders under the noise. Linearity rides along on a second pair, x^2 and $3x$ on $[0, 2]$ with exact total $\frac{26}{3}$: the sum of the two midpoint sums equals the midpoint sum of the sum to the last bit at $n = 128$, both 8.6666259765625.

```
// Table rows: exact antiderivative vs midpoint, both near the true value.
for (unsigned i = 0; i < sizeof table / sizeof table[0]; i++) {
  double exact = table[i].F(table[i].b) - table[i].F(table[i].a);
  CHECK(fabs(exact - table[i].exact) < 1e-12 &&
            fabs(mid_sum(table[i].f, table[i].a, table[i].b, 512) - exact) <
                1e-4,
        i == 0 ? "table row 1 agrees" : "table row 2 agrees");
}

// FTC identity on a fixed case: d/dx of the numeric integral of g equals g.
// gen: G'(1.5) = 6.374999899567162 vs g(1.5) = 6.375 (gap 1.0e-07),
// G'(0.9) = 2.5289999783641068 vs g(0.9) = 2.529 (gap 2.2e-08).
double d = 1e-5;
double gp15 = (gnum(1.5 + d) - gnum(1.5 - d)) / (2 * d);
double gp09 = (gnum(0.9 + d) - gnum(0.9 - d)) / (2 * d);
CHECK(fabs(gp15 - g(1.5)) < 1e-6, "FTC holds at x = 1.5");
CHECK(fabs(gp09 - g(0.9)) < 1e-6, "FTC holds at x = 0.9");
// And the exact antiderivative differentiates back: central difference of
// G at 1.1 with h = 1e-6. Truncation is G'''(1.1)/6 h^2 = g''(1.1)/6 h^2
// = 1.1e-12 in exact arithmetic (gen mp), the double gap is -2.765e-10
// (cancellation in the G difference, 10 significant digits survive).
CHECK(fabs((G(1.1 + 1e-6) - G(1.1 - 1e-6)) / 2e-6 - g(1.1)) < 1e-9,
      "G' = g on the exact antiderivative");

// Linearity: the integral of a sum is the sum of the integrals.
// gen: both lattices give 8.6666259765625 at n = 128, exact 26/3.
CHECK(fabs(mid_sum(sq, 0, 2, 128) + mid_sum(tri, 0, 2, 128) -
           mid_sum(sq_plus_tri, 0, 2, 128)) <= 1e-12,
      "integral of the sum is the sum of the integrals");
CHECK(fabs(mid_sum(sq_plus_tri, 0, 2, 128) - (8.0 / 3.0 + 6.0)) < 5e-5,
      "linearity target is 26/3 at n = 128");
```

listing 93: antiderivative table, the FTC identity both ways, linearity, ftc.c (source: math/samples/src/Ch09/ftc.c, lines 112-142)

PITFALL: exact ratios are a fixture property

The midpoint error ratios reading exactly 4.0 hold because the error of this cubic on this interval is exactly $2/n^2$ with no lower-order correction. For a generic smooth integrand the ratio approaches 4 from below as n grows, so a check that demands exactly 4.0 on arbitrary fixtures fails. Pin the order with a tolerance, or pick a fixture whose error law you have derived.

The two-number play: at $n = 4$ the left sum is off by 2.75 and the midpoint by 0.125, twenty-two times closer for the same four evaluations, and by $n = 512$ the midpoint error is 7.6e-6.

9.6. what survives in floats

The limit $h \to 0$ is a mathematical fiction. In doubles the central difference error is the sum of truncation ch^2 and a rounding floor of a few ulps of $f(x)$ divided by h, and that sum has a minimum. Solving for the minimum of the two laws, $ch^2 \approx u/h$ gives $h^* \approx \sqrt[u/c]{3}$, which for $\varepsilon = 2^{-}52$ and $c = O(1)$ lands near $6 \times 10^{-}6$, between $2^{-}18$ and $2^{-}17$. The sweep in the sample confirms it on the fixed polynomial at the generic point $x = 1.1$: the error rides the truncation law $4.4h^2$ (the $f'''(1.1)/6$ coefficient) while the rungs are clean, 6.56e-8 against the law's 6.56e-8 at $h = 2^{-}13$, but the sweep bottom 4.56e-11 at $h = 2^{-}18$ already sits below the law's 6.40e-11 there, because rounding starts subtracting from the truncation term, and beyond it the error rises to 5.36e-10 at $2^{-}22$ where cancellation eats the difference $f(x + h) - f(x - h)$. The argmin over the whole sweep is exactly $k = 18$. This is the same

cancellation mechanism book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) dissects at the bit level, seen here through a calculus operation.

The same algebra run on the forward quotient, whose error law is $ch + u/h$, puts its optimum at $h^* \approx \sqrt{u/c}$, around $10^{-}8$ for doubles, an order tighter than the central one and two orders less accurate at its best. The central difference is the right default precisely because its truncation term is squared: same two function values, one extra power of h bought.

A second, quieter fact rides along. At $x = 1$ with $h = 2^{-}k$, the points $1 \pm h$ are exact, the polynomial values round symmetrically, and once $4h^2$ drops under half an ulp of 5 the central quotient collapses to exactly 5.0, with error 0.0, all the way down to $h = 2^{-}21$. The U curve is not a property of the formula, it is a property of the formula at points whose arithmetic does not cancel. Power-of-two offsets from a clean point are the lucky case, generic offsets like 1.1 are the honest one, and the two checks sit next to each other to make the contrast explicit.

The dry run: the sweep at $x = 1.1$, target $f'(1.1) = 5.724$.

1. $k = 13$: error 6.56e-8, clean $4.4h^2$ truncation territory.
2. $k = 18$: error 4.56e-11, the bottom of the U, $h = 3.81 \times 10^{-}6$.
3. $k = 22$: error 5.36e-10, twelve times worse than the bottom while h is sixteen times smaller.

```
// The float wall. At x = 1 with h = 2^-k the arithmetic stays exact and the
// quotient collapses to exactly 5 once 4h^2 falls under half an ulp.
CHECK(fabs(ctr(1.0, ldexp(1.0, -21)) - 5.0) == 0.0,
      "power-of-two ladder is exact at h = 2^-21");
// At the generic point x = 1.1 cancellation bites: gen sweep argmin at
// k = 18, err 4.563372e-11 at the bottom, 5.364429e-10 at k = 22.
double xg = 1.1, best_err = 1e9;
int best_k = -1;
for (int k = 6; k ≤ 26; k++) {
  double err = fabs(ctr(xg, ldexp(1.0, -k)) - fd(xg));
  if (err < best_err) {
    best_err = err;
    best_k = k;
  }
}
CHECK(best_k == 18, "argmin of central error is h = 2^-18");
CHECK(fabs(ctr(xg, ldexp(1.0, -18)) - fd(xg)) <
          fabs(ctr(xg, ldexp(1.0, -22)) - fd(xg)),
      "error grows below the optimum");
// gen: the sweep bottom is 4.563372e-11 at k = 18, already below the
// truncation law 4.4h^2 = 6.40e-11 because rounding subtracts there.
CHECK(fabs(fabs(ctr(xg, ldexp(1.0, -18)) - fd(xg)) - 4.563372e-11) < 1e-13,
      "sweep bottom is 4.563e-11 at h = 2^-18");
CHECK(ldexp(1.0, -18) < pow(2.0, -52.0 / 3.0) &&
```

listing 94: the exact power-of-two ladder and the generic-point sweep to the argmin, deriv.c (source: math/samples/src/Ch09/deriv.c, lines 112-135)

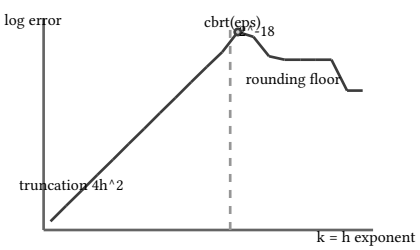


figure 55: central difference error against h, the U with its floor

WARNING: never push h below cbrt of epsilon

> A central difference at $h = 10^{-}8$ on values of size 1 discards all but the top few bits of $f(x + h) - f(x - h)$ and returns noise with a straight face. The check pins the failure: the error at $2^{-}22$ is twelve times the error at the optimum. The workable rule for doubles is $h \approx 10^{-}6$ for central differences, $h \approx 10^{-}8$ never. When you need exact derivatives at machine precision, the next chapters' chain-rule machinery, book 2, ch 12 (mathematics for programmers, automatic differentiation), computes them without any h at all.

The two-number play: at the lucky point $x = 1$ the error at $h = 2^{-}21$ is exactly 0, at the generic point $x = 1.1$ the best error over the whole sweep is 4.6e-11 and pushing further makes it worse.

The derivative of a function of several variables is the next chapter, book 2, ch 10 (mathematics for programmers, multivariate calculus), where the single derivative becomes a gradient, a row vector of partials, and every rule verified here composes one variable at a time.

sources: mml-book draft 2024-01-15, ch 5, differentiation of univariate functions, Definitions 5.1 and 5.2 with eqs 5.3-5.4, Example 5.2 polynomial power rule with eqs 5.5-5.6d, Definition 5.3 and 5.4 with eqs 5.7-5.8, Example 5.3 exact polynomial taylor case with eq 5.17, Example 5.4 sin plus cos series with eqs 5.25-5.27, differentiation rules eqs 5.29-5.32, Example 5.5 chain rule with eq 5.38 [printed pp 141-146 / pdf pp 147-152], the gradient-implementation check remark with $h = 10^{-}4$ [printed p 149 / pdf p 155], and the automatic differentiation forward and reverse modes with eqs 5.119-5.121 [printed pp 161-164 / pdf pp 167-170], all quoted from the verified anchor sheet ref/mml/anchors-calc.md (53 numpy recomputes) rather than re-opened from the pdf. Epsilon-delta, riemann sums, and the fundamental theorem are standard single-variable calculus, our own words and code throughout, with book 9, ch 36 (practical data structures and algorithms, numerical methods) owning the algorithmic drivers these definitions feed. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch09`, 70 checks in chapter 09 of the math suite.

10. multivariate calculus

A function of several variables arrives in C as a function of several arguments, and its derivative arrives as a vector or a matrix rather than a scalar. This chapter builds those objects in six moves: partial derivatives as the single-variable limit already owned from book 2, ch 9 (mathematics for programmers, single-variable calculus), the gradient with its orthogonality to level sets, the jacobian of a vector-valued map with the chain rule as matrix multiplication, the hessian with its definiteness test, the linearization and second-order taylor model, and the short identity list that keeps hand gradients cheap. Where book 9, ch 36 (practical data structures and algorithms, numerical methods) consumes derivatives as oracles inside search drivers, this chapter produces the derivative objects themselves. Every behavioral claim below is one of the 59 checks in the 4 samples of chapter 10 or a sentence from the mml-book draft 2024-01-15 anchor sheet read 2026-09-21.

10.1. partial derivatives as limits

Definition first. For $f : \mathbb{R}^n \to \mathbb{R}$ the partial derivative with respect to x_1 holds every other variable fixed and runs the ordinary difference quotient, mml definition 5.5 eq 5.39 [printed p 146 / pdf p 152]:

$$\frac{\partial f}{\partial x_1} = \lim_{h \to 0} \frac{f(x_1 + h, x_2, ..., x_n) - f(x_1, x_2, ..., x_n)}{h}$$

Each partial is a derivative with respect to a scalar, so the rules from the univariate chapter transfer one variable at a time. The univariate chain rule even works inside a partial: for $f(x, y) = (x + 2y^3)^2$ the mml example 5.6 [printed pp 146-147 / pdf pp 152-153] reads off $\frac{\partial f}{\partial x} = 2(x + 2y^3)$ and $\frac{\partial f}{\partial y} = 12(x + 2y^3)y^2$. The worked function for the checks is the mml example 5.7 polynomial [printed p 147 / pdf p 153], $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$, whose analytic partials are $2x_1x_2 + x_2^3$ and $x_1^2 + 3x_1x_2^2$.

The dry run: pin $x_2 = 3$ and the surface collapses to the slice $g(x) = 3x^2 + 27x$, an ordinary parabola in one variable.

1. The slice derivative is $g'(x) = 6x + 27$, so $g'(2) = 39$, and the coded partial returns exactly 39.0 at $(2, 3)$.
2. The forward quotient $\frac{f(2+h,3)-f(2,3)}{h}$ walks 39.02999999999821, 39.003000000008115, 39.000300000111565 as h steps through 10^{-2}, 10^{-3}, 10^{-4}, each value pinned to a billionth.
3. The error column 3.0e-2, 3.0e-3, 3.0e-4 drops tenfold per decade of h. That is the $O(h)$ truncation of the one-sided quotient, the ladder shape the univariate chapter already pinned on its own functions.

h	(f(2 + h, 3) - f(2, 3)) / h	error against 39
10^{-2}	39.02999999999821	3.0e-2
10^{-3}	39.003000000008115	3.0e-3
10^{-4}	39.000300000111565	3.0e-4

Second derivatives arrive the same way, and the one theorem worth quoting is that differentiation order does not matter, mml eq 5.146 [printed p 164 / pdf p 170]: if f is twice continuously differentiable then $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$. On the fixed polynomial $f_2 = x^3y^2 + 2x^2y^3$ the chain x first then y gives $6x^2y + 12xy^2$, the chain y first then x gives the same expression, and at $(2, 1)$ both evaluate to 48, both analytically and through nested central differences at $h = 10^{-5}$.

```
// f(x1, x2) = x1^2 x2 + x1 x2^3, mml example 5.7 [printed p 147 / pdf p 153]
[[nodiscard]] static double f1(double x1, double x2) {
  return x1 * x1 * x2 + x1 * x2 * x2 * x2;
}

// slice with x2 pinned to 3: g(x) = 3x^2 + 27x, one variable survives
[[nodiscard]] static double slice(double x) { return 3.0 * x * x + 27.0 * x; }

// analytic partials, 2 x1 x2 + x2^3 and x1^2 + 3 x1 x2^2
[[nodiscard]] static double df1_dx1(double x1, double x2) {
  return 2.0 * x1 * x2 + x2 * x2 * x2;
}
[[nodiscard]] static double df1_dx2(double x1, double x2) {
  return x1 * x1 + 3.0 * x1 * x2 * x2;
}
```

listing 95: the coded function, its x2 pinned slice, and its two analytic partials in partials.c (source: math/samples/src/Ch10/partials.c, lines 21-35)

```
  // limit ladder, forward difference (f(x1+h, x2) - f(x1, x2)) / h of eq 5.39
  // h=1e-2 -> 39.02999999999821, h=1e-3 -> 39.003000000008115,
  // h=1e-4 -> 39.000300000111565
  double q2 = (f1(px + 1e-2, py) - f1(px, py)) / 1e-2;
  double q3 = (f1(px + 1e-3, py) - f1(px, py)) / 1e-3;
  double q4 = (f1(px + 1e-4, py) - f1(px, py)) / 1e-4;
  CHECK(fabs(q2 - 39.02999999999821) < 1e-9, "forward quotient h=1e-2 pinned");
  CHECK(fabs(q3 - 39.003000000008115) < 1e-9, "forward quotient h=1e-3 pinned");
  CHECK(fabs(q4 - 39.000300000111565) < 1e-9, "forward quotient h=1e-4 pinned");
  // errors 2.999999999821057e-2, 3.0000000081145117e-3, 3.0000001115651685e-4,
  // each tenfold drop: the O(h) truncation of the one-sided quotient
  double e2 = fabs(q2 - 39.0), e3 = fabs(q3 - 39.0), e4 = fabs(q4 - 39.0);
  CHECK(fabs(e3 / e2 - 0.1) < 5e-3, "error drops tenfold h=1e-2 to 1e-3");
  CHECK(fabs(e4 / e3 - 0.1) < 5e-3, "error drops tenfold h=1e-3 to 1e-4");
```

listing 96: the limit ladder with pinned quotients and the tenfold error drop in partials.c (source: math/samples/src/Ch10/partials.c, lines 57-70)

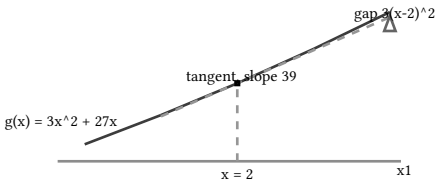


figure 56: the slice x2 pinned to 3 is the parabola g(x) = 3x^2 + 27x, tangent slope 39 at x = 2

VERIFY: gradient checking

Before trusting any coded gradient, mml remark 5.2.2 [printed p 149 / pdf p 155] compares it against central differences at $h = 10^{-4}$ with the criterion $\sqrt{\frac{\sum_i (dh_i - df_i)^2}{\sum_i (dh_i + df_i)^2}} < 10^{-6}$, where dh_i is the finite-difference value and df_i the analytic one. partials.c pins both difference values, 39.00000000008674 and 58.000000020115294, and the criterion evaluates to 1.4390243727509478e-10, five orders inside the gate. The same test guards every hand gradient in the rest of the book.

10.2. the gradient and level sets

Collect the partials into a row and you have the gradient, mml eq 5.40 [printed p 146 / pdf p 152]:

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right] \in \mathbb{R}^{1 \times n}$$

For the worked polynomial the gradient at $(2, 3)$ is the row $(39, 58)$ and at $(1, -1)$ it is $(-3, 4)$, all four numbers exact in gradient.c. The gradient answers two questions a slice cannot.

First, rate of climb in an arbitrary unit direction $\boldsymbol{d}$: the directional derivative is $D_{\boldsymbol{d}} f = \nabla f \cdot \boldsymbol{d}$, a dot product you can evaluate by hand. Second, which direction climbs fastest: by Cauchy-Schwarz $\nabla f \cdot \boldsymbol{d}$ is at most $|\nabla f|$, with equality exactly when $\boldsymbol{d}$ points along the gradient.

The dry run: stand at $(2, 3)$ on $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$ with $\nabla f = (39, 58)$ and read the table.

1. Due east along $(1, 0)$ the climb rate is 39, due north along $(0, 1)$ it is 58.
2. Along the diagonal $\frac{3,4}{5}$ it is 69.8, and reversing the direction gives exactly −69.8.
3. Along $\nabla \frac{f}{|} \nabla f|$ it is $\sqrt{4885} = 69.8927750200262$, the unique maximum, beating 69.8 and 68.53846153846155 along $\frac{5,12}{13}$.

unit direction	directional derivative at (2, 3)
$(1, 0)$	39
$(0, 1)$	58
$\frac{3,4}{5}$	69.8
$\frac{5,12}{13}$	68.53846153846155
$\nabla \frac{f}{\|} \nabla f\|$	69.8927750200262

The gradient also knows where the level sets run. On the quadratic $q(x, y) = x^2 + 4y^2$ the level curve $q = 8$ is an ellipse through $(2, 1)$, the gradient there is $(4, 8)$, and a tangent direction of the curve at that point is $(2, -1)$. The dot product $4 \cdot 2 + 8 \cdot (-1)$ is exactly 0: the gradient is perpendicular to the level set, pointing straight out of the ellipse. gradient.c pins the same zero at a second curve point $\left(1.2, \sqrt{1.64}\right)$ where the tangent is $(-8y, 2x)$, and the dot vanishes again. Flip the sign and the same perpendicular points into the level set: the negative gradient is the steepest descent direction, the workhorse of book 2, ch 11 (mathematics for programmers, continuous optimization).

```
// level curve q = 8 through (2, 1): grad q there is (4, 8), and the curve's
// tangent direction (2, -1) has dot product exactly 0
CHECK(fabs(q(2.0, 1.0) - 8.0) < 1e-15, "q(2,1) sits on the level 8");
CHECK(fabs(dot2((double[2]){2.0 * 2.0, 8.0 * 1.0}, (double[2]){2.0, -1.0})) <
          1e-15,
      "grad q orthogonal to the level tangent at (2,1)");
// second point of the same level curve, (1.2, sqrt(1.64)) with
// sqrt(1.64) = 1.2806248474865698: tangent (-8y, 2x) again kills the dot
constexpr double yv = 1.2806248474865698;
CHECK(fabs(q(1.2, yv) - 8.0) < 1e-12,
      "(1.2, sqrt(1.64)) sits on the same level curve");
CHECK(fabs(dot2((double[2]){2.0 * 1.2, 8.0 * yv},
                (double[2]){-8.0 * yv, 2.0 * 1.2})) < 1e-12,
      "grad q orthogonal to the level tangent at the second point");
```

listing 97: level curve membership and gradient-tangent orthogonality pinned at two points in gradient.c (source: math/samples/src/Ch10/gradient.c, lines 46-59)

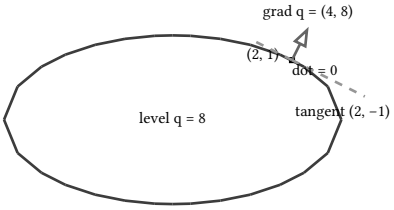


figure 57: the ellipse x^2 + 4y^2 = 8 with grad q = (4, 8) leaving the level curve at a right angle at (2, 1)

PITFALL: the gradient is a row

The mml book keeps ∇f as a $1 \times n$ row, the numerator layout, so that the gradient of $f : \mathbb{R}^n \to \mathbb{R}^m$ generalizes to an $m \times n$ matrix and the chain rule becomes plain matrix multiplication with no transposes [mml remark, printed p 147 / pdf p 153]. Treat the gradient as a column and the dimension bookkeeping flips silently. The samples store gradients as flat arrays and document each one as row or column before multiplying.

10.3. jacobians and the chain rule

A vector-valued map $f : \mathbb{R}^n \to \mathbb{R}^m$ stacks m scalar functions, and its first derivative stacks their gradients into a matrix, mml definition 5.6 eqs 5.57-5.59 [printed p 150 / pdf p 156]:

$$\boldsymbol{J} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$$

The entry rule is one line: $J(i,j) = \frac{\partial f_i}{\partial x_j}$. The scalar gradient of the previous section is the $m = 1$ special case, a single row. The full shape inventory, after the mml dimension summary [printed p 152 / pdf p 158]:

map	derivative object	shape
$f : \mathbb{R} \to \mathbb{R}$	scalar	1×1
$f : \mathbb{R}^D \to \mathbb{R}$	gradient	$1 \times D$ row
$f : \mathbb{R} \to \mathbb{R}^E$	gradient	$E \times 1$ column
$f : \mathbb{R}^D \to \mathbb{R}^E$	jacobian	$E \times D$ matrix

Two fixed maps carry the section in jacobian.c.

The linear one is the mml basis change $y_1 = -2x_1 + x_2$, $y_2 = x_1 + x_2$ [eqs 5.63-5.66, printed p 151 / pdf p 157]. Its jacobian is the constant matrix $\left(\begin{smallmatrix} -2 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ with determinant −3, and the columns are the images $c_1 = (-2, 1)$ and $c_2 = (1, 1)$ of the standard basis. The unit square, area 1, maps to the parallelogram those columns span, area $|\det J| = 3$. The same holds for $y = \boldsymbol{A}x$ with any $\boldsymbol{A}$: the jacobian of $\boldsymbol{A}x$ is $\boldsymbol{A}$ itself, mml example 5.9, and the sample pins all four entries through forward differences at $h = 10^{-4}$.

The nonlinear one is $f(x_1, x_2) = (x_1 x_2 + x_1^2, 3x_1 - x_2^2)$ evaluated at $(2, 3)$: the analytic jacobian is $\left(\begin{smallmatrix} 7 & 2 \\ 3 & -6 \end{smallmatrix}\right)$ with determinant −48, and the finite-difference columns 7.000100000027487 and 1.9999999999953388 agree to a part in a billion.

The chain rule is where the row convention pays. For a composition $h(t) = f(g(t))$ with $g : \mathbb{R} \to \mathbb{R}^2$ and $f : \mathbb{R}^2 \to \mathbb{R}$, mml eqs 5.51-5.53 and example 5.10 [printed pp 148-153 / pdf pp 154-159]:

$$\frac{dh}{dt} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial t}$$

a 1×2 row times a 2×1 column, and the middle dimension cancels. The mml book offers the reading aid that ∂f shows up in the denominator of the first factor and the numerator of the second, so the dimensions match and the symbol seems to cancel, then immediately warns that this is intuition rather than mathematics since a partial

derivative is not a fraction [printed p 148 / pdf p 154]. The row convention is what makes the compact matrix form legal in the first place.

The dry run: take $g(t) = (t^2, t^3)$ and $f(u, v) = uv + 2u$, so $h(t) = t^5 + 2t^2$.

1. At $t = 2$ the inner point is $g(2) = (4, 8)$ and the inner jacobian is the column $(2t, 3t^2) = (4, 12)$.
2. The outer gradient at $(4, 8)$ is the row $(v + 2, u) = (10, 4)$.
3. The product $10 \cdot 4 + 4 \cdot 12 = 88$ equals the analytic $h'(2) = 5t^4 + 4t = 88$, and a central difference of the coded h pins 88.00000040007916.

```
// forward-difference jacobian, column j is (f(x + h e_j) - f(x)) / h, row-major
static void jac_fd(scalar2 f0, scalar2 f1, double x, double y, double h,
                   double j[static 4]) {
  double b0 = f0(x, y), b1 = f1(x, y);
  j[0] = (f0(x + h, y) - b0) / h;
  j[2] = (f1(x + h, y) - b1) / h;
  j[1] = (f0(x, y + h) - b0) / h;
  j[3] = (f1(x, y + h) - b1) / h;
}
```

listing 98: the forward-difference jacobian helper, one column per input variable, in jacobian.c (source: math/samples/src/Ch10/jacobian.c, lines 57-65)

```
  // chain rule: g(2) = (4, 8), jacobian of g at 2 is the column (4, 12),
  // grad f at (4, 8) is the row (10, 4), product 10*4 + 4*12 = 88
  double gpt[2] = {2.0 * 2.0, 2.0 * 2.0 * 2.0}; // (t^2, t^3) at t = 2
  double jg[2] = {2.0 * 2.0, 3.0 * 2.0 * 2.0};  // (2t, 3t^2) at t = 2
  CHECK(gpt[0] == 4.0 && gpt[1] == 8.0 && jg[0] == 4.0 && jg[1] == 12.0,
        "g(2) = (4,8) and its jacobian column is (4, 12)");
  double gfr[2] = {gpt[1] + 2.0, gpt[0]}; // grad f = (v + 2, u) at g(2)
  CHECK(fabs(gfr[0] - 10.0) < 1e-15 && fabs(gfr[1] - 4.0) < 1e-15,
        "grad f at (4,8) pins the row (10, 4)");
  double chain = gfr[0] * jg[0] + gfr[1] * jg[1];
  double direct = 5.0 * 2.0 * 2.0 * 2.0 * 2.0 + 4.0 * 2.0;
  CHECK(fabs(chain - 88.0) < 1e-15 && fabs(direct - 88.0) < 1e-15,
        "jacobian product equals analytic h'(2) = 88");
  double hd = (hv(2.0 + 1e-4) - hv(2.0 - 1e-4)) / 2e-4;
  CHECK(fabs(hd - 88.00000040007916) < 1e-9,
        "central difference of h at 2 pinned near 88");
```

listing 99: the two-stage composition checked as a row times a column against the analytic 88 in jacobian.c (source: math/samples/src/Ch10/jacobian.c, lines 135-151)

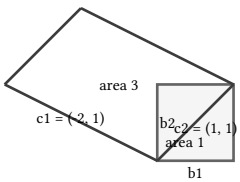


figure 58: the basis change matrix sends the unit square, area 1, to the column parallelogram, area 3 = |det J|

WARNING: the area scaling is local

For a linear map $|\det \boldsymbol{J}|$ scales areas exactly. For a nonlinear map the same number is the magnification in an infinitesimal neighborhood, because the jacobian approximates the nonlinear transformation locally with a linear one [mml, printed pp 151-152 / pdf pp 157-158]. The pinned determinant -48 of the nonlinear map at $(2, 3)$ is that local factor at that point only, and it varies from point to point. The same determinant reappears as

> the change of variables correction in probability and as the reparametrization trick in deep network samplers [mml, printed p 152 / pdf p 158].

10.4. the hessian and curvature

Why collect second derivatives at all: Newton's method for optimization needs them, and so does any multivariate taylor model beyond first order, the two motivations the mml sec 5.7 opening names [printed p 164 / pdf p 170]. Second derivatives collect into a matrix too. For $f : \mathbb{R}^n \to \mathbb{R}$ the hessian is the $n \times n$ matrix of all second-order partials, mml eq 5.147 [printed p 164 / pdf p 170], and the mixed-partials equality of the first section makes it symmetric whenever f is twice continuously differentiable. The hessian measures the curvature of f locally around the point of evaluation. For a vector field $f : \mathbb{R}^n \to \mathbb{R}^m$ the same collection is an $m \times n \times n$ tensor, one hessian per output component, mml remark [printed p 165 / pdf p 171], and the samples stay with scalar f where the matrix picture holds.

The worked function from here to the end of the chapter is the mml example 5.15 cubic [printed pp 167-169 / pdf pp 173-175]:

$$F(x, y) = x^2 + 2xy + y^3, \quad H(x, y) = \begin{pmatrix} 2 & 2 \\ 2 & 6y \end{pmatrix}, \quad H(1, 2) = \begin{pmatrix} 2 & 2 \\ 2 & 12 \end{pmatrix}$$

The dry run: classify four fixed 2×2 matrices with the sylvester test and confirm each verdict against eigenvalues, the machinery of book 2, ch 8 (mathematics for programmers, matrix decompositions).

1. The $H(1, 2)$ above has determinant 20 and trace 14, so both eigenvalues are positive, and the closed forms $\frac{\mathrm{tr} \pm \sqrt{\mathrm{tr}^2 - 4 \det}}{2}$ give 12.385164807134505 and 1.6148351928654963, which is $7 \pm \sqrt{29}$. Positive definite: a bowl.
2. $\left(\begin{smallmatrix} 2 & 4 \\ 4 & 2 \end{smallmatrix}\right)$ has determinant −12 and eigenvalues 6 and −2 of opposite sign. Indefinite: a saddle.
3. $\left(\begin{smallmatrix} -4 & 2 \\ 2 & -4 \end{smallmatrix}\right)$ has determinant 12, negative diagonal, eigenvalues −2 and −6. Negative definite: a dome.
4. $\left(\begin{smallmatrix} 4 & 4 \\ 4 & 4 \end{smallmatrix}\right)$ has determinant 0 and eigenvalues 8 and 0. Semidefinite with a flat direction.

matrix	det	eigenvalues	class
$\left(\begin{smallmatrix} 2 & 2 \\ 2 & 12 \end{smallmatrix}\right)$	20	12.385, 1.615	bowl
$\left(\begin{smallmatrix} 2 & 4 \\ 4 & 2 \end{smallmatrix}\right)$	−12	6, −2	saddle
$\left(\begin{smallmatrix} -4 & 2 \\ 2 & -4 \end{smallmatrix}\right)$	12	−2, −6	dome
$\left(\begin{smallmatrix} 4 & 4 \\ 4 & 4 \end{smallmatrix}\right)$	0	8, 0	flat

jacobian.c codes the sylvester classifier and the closed-form eigenvalues side by side, pins every determinant and eigenvalue above, and asserts the two routes agree on all four verdicts.

```c
typedef enum : unsigned char { k_pd, k_nd, k_indef, k_semi } defn_t;

// 2x2 sylvester test: sign of det separates the saddle, then sign of m00
static defn_t classify2(const double m[static 4]) {
  double det = m[0] * m[3] - m[1] * m[2];
  if (det < 0.0) {
    return k_indef;
  }
  if (det == 0.0) {
    return k_semi;
  }
  return m[0] > 0.0 ? k_pd : k_nd;
}

// closed-form eigenvalues (tr +/- sqrt(tr^2 - 4 det)) / 2
static void eig2(const double m[static 4], double e[static 2]) {
  double tr = m[0] + m[3];
  double det = m[0] * m[3] - m[1] * m[2];
  double disc = sqrt(tr * tr - 4.0 * det);
  e[0] = (tr + disc) / 2.0;
  e[1] = (tr - disc) / 2.0;
}
```

listing 100: the sylvester classifier and the closed-form 2x2 eigenvalues cross-checking each other in jacobian.c (source: math/samples/src/Ch10/jacobian.c, lines 67-88)

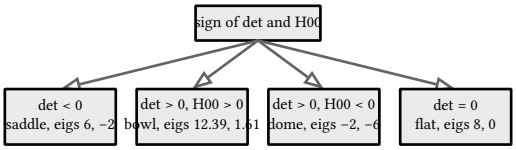


figure 59: definiteness of a 2x2 hessian read off det and the diagonal, each verdict confirmed by eigenvalues

NOTE: symmetry is a hypothesis

The mixed-partials equality that makes the hessian symmetric needs f twice continuously differentiable, mml eq 5.146 [printed p 164 / pdf p 170]. Every function in this chapter is a polynomial, so the hypothesis holds and the samples pin the symmetry directly. A function with a kink or a jump in a second derivative can break it, and with it every downstream conclusion.

10.5. linearization and taylor

The gradient gives the best first-order model of f near a point x_0, mml eq 5.148 [printed p 165 / pdf p 171]:

$$f(x) \approx f(x_0) + \nabla f(x_0)(x - x_0)$$

The model is locally accurate and degrades as you walk away from x_0, the behavior the mml Figure 5.12 discussion pins on its own example [printed p 165 / pdf p 171]. Everything below quantifies that degradation: first order dies as $|\delta|^2$, second order as $|\delta|^3$.

The taylor series stacks higher-order corrections on top, and the second-order term is the one that earns its keep, mml definition 5.7-5.8 and eqs 5.156-5.158 [printed p 166 / pdf p 172]. With the displacement $\delta = x - x_0$ and the hessian $H(x_0)$:

$$T_2(x) = f(x_0) + \nabla f(x_0)\delta + \frac{1}{2}\delta^\top H(x_0)\delta$$

For the cubic $F(x, y) = x^2 + 2xy + y^3$ at $(1, 2)$ the mml example 5.15 numbers are $F(1, 2) = 13$, $\nabla F(1, 2) = (6, 14)$, and the quadratic term $\frac{1}{2}\delta^\top H\delta = dx^2 + 2dxdy + 6dy^2$. Because the only live third derivative of F is $\frac{\partial^3 F}{\partial y^3} = 6$, the remainder is exactly $(y-2)^3$ and the full expansion of eq 5.180 [printed p 169 / pdf p 175] reproduces the polynomial term for term:

$$F(x, y) = 13 + 6(x-1) + 14(y-2) + (x-1)^2 + 2(x-1)(y-2) + 6(y-2)^2 + (y-2)^3$$

taylor.c pins that identity at three probe points to the last bit.

The series continues past second order: the k-th term multiplies the k-th total derivative, a k-dimensional tensor, against the k-fold outer product δ^k, so $D_x^3 f(x_0)\delta^3$ is a triple sum over all third partials, mml eqs 5.153-5.155 and 5.159 [printed p 166 / pdf p 172]:

$$\delta^2 = \delta\delta^\top, \quad \delta^2[i, j] = \delta[i]\delta[j]$$

For the cubic above only one of those partials survives, which is why the remainder collapses to a single cube.

The dry run: walk the ray $(1, 2) + t(0.1, 0.1)$ and watch what each order of model costs in error.

1. The linear model misses F by 0.09099999999999753 at $t = 1$, by 0.0090010000000007068 at $t = 0.1$, and by 9.000999998676207e-06 at $t = 0.01$.
2. Each decade of t divides the linear error by about 100, ratios 0.009901098901106938 and 0.009990011097302048: quadratic shrinkage, the signature of a first-order model.
3. The second-order model at $(1.1, 2.1)$ is off by exactly $(y-2)^3 = 0.001$, and 0.001 stays under the bound $|\delta|^3 = 0.0028284271247461983$ from the third-derivative magnitude.

t on the ray	linear error	ratio to previous
1	9.1e-2	
0.1	9.0e-4	0.0099
0.01	9.0e-6	0.0100

```
// first-order model at (1, 2): F = 13, grad F = (6, 14)
[[nodiscard]] static double lin(double a, double b) {
  return 13.0 + 6.0 * (a - 1.0) + 14.0 * (b - 2.0);
}

// second-order model, quadratic part is 1/2 d^T H d with H = [[2,2],[2,12]]
[[nodiscard]] static double taylor2(double a, double b) {
  double dx = a - 1.0, dy = b - 2.0;
  return 13.0 + 6.0 * dx + 14.0 * dy + dx * dx + 2.0 * dx * dy + 6.0 * dy * dy;
}

[[nodiscard]] static double quad_form(double dx, double dy) {
  return 0.5 * (dx * (2.0 * dx + 2.0 * dy) + dy * (2.0 * dx + 12.0 * dy));
}
```

listing 101: the first and second-order models with the hessian quadratic form in taylor.c (source: math/samples/src/Ch10/taylor.c, lines 24-37)

```
// linearization error along the ray (1, 2) + t (0.1, 0.1) shrinks as t^2:
// t=1 -> 0.09099999999999753, t=0.1 -> 0.0009010000000007068,
// t=0.01 -> 9.000999998676207e-06 (points computed as 1 + 0.1 t, the
// playground's own expression, so the pins match bit for bit)
const double ts[3] = {1.0, 0.1, 0.01};
double e1 = Fv(1.0 + 0.1 * ts[0], 2.0 + 0.1 * ts[0]) -
            lin(1.0 + 0.1 * ts[0], 2.0 + 0.1 * ts[0]);
double e01 = Fv(1.0 + 0.1 * ts[1], 2.0 + 0.1 * ts[1]) -
             lin(1.0 + 0.1 * ts[1], 2.0 + 0.1 * ts[1]);
double e001 = Fv(1.0 + 0.1 * ts[2], 2.0 + 0.1 * ts[2]) -
              lin(1.0 + 0.1 * ts[2], 2.0 + 0.1 * ts[2]);
CHECK(fabs(e1 - 0.09099999999999753) < 1e-9, "linear error at t=1 pinned");
CHECK(fabs(e01 - 0.0009010000000007068) < 1e-12,
      "linear error at t=0.1 pinned");
CHECK(fabs(e001 - 9.000999998676207e-06) < 1e-14,
      "linear error at t=0.01 pinned");
// ratios 0.009901098901106938 and 0.009990011097302048: quartering per
// decimal decade is the quadratic order signature
CHECK(fabs(e01 / e1 - 0.009901098901106938) < 1e-12,
      "error ratio t=0.1 over t=1 near 0.01");
CHECK(fabs(e001 / e01 - 0.009990011097302048) < 1e-12,
      "error ratio t=0.01 over t=0.1 near 0.01");
```

listing 102: the linearization error ladder with pinned values and pinned ratios in taylor.c (source: math/samples/src/Ch10/taylor.c, lines 49-71)

```
// second-order remainder at (1.1, 2.1) is exactly (y - 2)^3: fd error
// 0.0009999999999976694 against dy^3 = 0.0010000000000000026
double r1 = Fv(1.1, 2.1) - taylor2(1.1, 2.1);
CHECK(fabs(r1 - 0.0009999999999976694) < 1e-9 &&
          fabs(r1 - 0.0010000000000000026) < 1e-12,
      "second-order remainder equals (y-2)^3");

// F is a cubic polynomial, so F = T2 + (y - 2)^3 holds identically:
// residuals -1.7763568394002505e-15, 0.0, 0.0 at the three probe points
double pts[3][2] = {{1.1, 2.1}, {1.5, 2.5}, {0.7, 1.3}};
bool identified = true, bounded = true;
for (int i = 0; i < 3; i++) {
  double dy = pts[i][1] - 2.0;
  double resid = Fv(pts[i][0], pts[i][1]) -
                 (taylor2(pts[i][0], pts[i][1]) + dy * dy * dy);
  identified = identified && fabs(resid) < 1e-12;
}
CHECK(identified, "cubic identity F = T2 + (y-2)^3 at three points");

// remainder bound: the only live third derivative is d3F/dy3 = 6, so
// |F - T2| = |dy|^3 ≤ |delta|^3; pins 0.001 ≤ 0.0028284271247461983,
// 0.125 ≤ 0.35355339059327384, 0.34299999999999997 ≤ 0.4417148401401066
for (int i = 0; i < 3; i++) {
  double dx = pts[i][0] - 1.0, dy = pts[i][1] - 2.0;
  double err = fabs(Fv(pts[i][0], pts[i][1]) - taylor2(pts[i][0], pts[i][1]));
  double norm = sqrt(dx * dx + dy * dy);
  bounded = bounded && err ≤ norm * norm * norm;
}
CHECK(bounded, "remainder bounded by |delta|^3 at all three points");
```

listing 103: the exact cubic remainder and its bound against |delta|^3 at three points in taylor.c (source: math/samples/src/Ch10/taylor.c, lines 72-101)

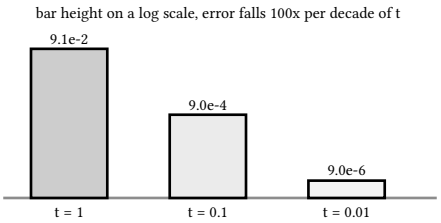


figure 60: linearization error at t = 1, 0.1, 0.01 on a log height scale, each decade of t cutting the error by 100

10.6. gradient identities that pay rent

Hand differentiation of a composition needs a small rule set, not creativity. The scalar rules of mml sec 5.2.1 [printed pp 147-148 / pdf pp 153-154] keep their shape, and the vector rules add the caveat that matrix multiplication does not commute, so the order of the factors is part of the rule: $\frac{\partial}{\partial x} fg = \frac{\partial f}{\partial x} g + f \frac{\partial g}{\partial x}$ for the product, termwise addition for the sum, and $\frac{\partial}{\partial x} g(f(x)) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial x}$ for the chain, mml eqs 5.46-5.48:

$$\frac{\partial}{\partial x} fg = \frac{\partial f}{\partial x} g + f \frac{\partial g}{\partial x}, \quad \frac{\partial}{\partial x}(f + g) = \frac{\partial f}{\partial x} + \frac{\partial g}{\partial x}$$

The dry run: the worked polynomial at $(2, 3)$ splits as $f_1 = u + v$ with $u = x_1^2 x_2$ and $v = x_1 x_2^3$.

1. Sum: $\nabla u = (12, 4)$ and $\nabla v = (27, 54)$, and the componentwise sum is $(39, 58)$, the pinned gradient of f_1 from the second section.

2. Product: $w = uv = x_1^3 x_2^4$ has $\nabla w = (972, 864)$, and $\nabla u \cdot v + u \cdot \nabla v = (648, 216) + (324, 648)$ lands on the same pair.
3. Chain: the composition of the third section gave $(10, 4) \cdot (4, 12) = 88$, matching the analytic derivative to the last integer.

When the argument itself is a matrix the derivative becomes a four-dimensional tensor, $J_{ijkl} = \frac{\partial A_{ij}}{\partial B_{kl}}$, mml sec 5.4 [printed pp 155-158 / pdf pp 161-164]. The practical route is the one C programmers would pick anyway: matrices form a vector space isomorphic to R^{mn}, so flatten by stacking columns and differentiate against the flattened argument, which turns the chain rule back into plain matrix multiplication. The worked cases are $f = Ax$ against A, where the partial of f_i with respect to row i of A is the row $x^\top$ and zero elsewhere, eqs 5.89-5.92, and $K = R^\top R$ against R, where $\frac{\partial K_{pq}}{\partial R_{ij}}$ is R_{iq}, R_{ip}, $2R_{iq}$, or 0 depending on which indices coincide, eqs 5.96-5.98.

For matrix arguments the same discipline produces the identities programmers actually reach for, mml sec 5.5 eqs 5.99-5.108 [printed pp 158-159 / pdf pp 164-165], after Petersen and Pedersen, the matrix cookbook, 2012: $\frac{\partial a^\top x}{\partial x} = a^\top$, $\frac{\partial a^\top X b}{\partial X} = ab^\top$, $\frac{\partial x^\top B x}{\partial x} = x^\top (B + B^\top)$, and $\frac{\partial f(X)^{-1}}{\partial X} = -f(X)^{-1} \frac{\partial f(X)}{\partial X} f(X)^{-1}$. Two more carry the pattern through the trace and determinant: $\frac{\partial}{\partial X} \operatorname{tr}(f(X)) = \operatorname{tr}\left(\frac{\partial f(X)}{\partial X}\right)$ and $\frac{\partial}{\partial X} \det(f(X)) = \det(f(X)) \operatorname{tr}\left(f(X)^{-1} \frac{\partial f(X)}{\partial X}\right)$. The computational payoff is the least squares loss of mml example 5.11 [printed p 154 / pdf p 160]: for $L(\theta) = \|y - \Phi\theta\|^2$ the chain rule through the residual collapses to $\frac{\partial L}{\partial \theta} = -2e^\top \Phi$, one matrix product instead of a symbolic expansion, and that single identity is where book 2, ch 11 (mathematics for programmers, continuous optimization) picks up the trail with descent methods and the hessian-based newton step. book 2, ch 12 (mathematics for programmers, automatic differentiation) then mechanizes the chain identity so no hand gradient is needed at all.

```
// sum rule at (2, 3): f1 = u + v with u = x1^2 x2, v = x1 x2^3, and
// grad u = (12, 4), grad v = (27, 54), so grad u + grad v = (39, 58)
double gu[2] = {2.0 * 2.0 * 3.0, 2.0 * 2.0};
double gv[2] = {3.0 * 3.0 * 3.0, 3.0 * 2.0 * 3.0 * 3.0};
CHECK(gu[0] == 12.0 && gu[1] == 4.0 && gv[0] == 27.0 && gv[1] == 54.0 &&
          gu[0] + gv[0] == 39.0 && gu[1] + gv[1] == 58.0,
      "sum rule: grad u + grad v pins grad f1 = (39, 58)");
// product rule at (2, 3): w = u v = x1^3 x2^4 has grad w = (972, 864), and
// u' v + u v' = (648, 216) + (324, 648) lands on the same pair
double uv23 = 12.0, v23 = 54.0;
double upv[2] = {gu[0] * v23, gu[1] * v23};
double uvp[2] = {uv23 * gv[0], uv23 * gv[1]};
double gw[2] = {3.0 * 2.0 * 2.0 * 3.0 * 3.0 * 3.0 * 3.0,
                4.0 * 2.0 * 2.0 * 2.0 * 3.0 * 3.0 * 3.0};
CHECK(upv[0] == 648.0 && upv[1] == 216.0 && uvp[0] == 324.0 &&
          uvp[1] == 648.0 && gw[0] == 972.0 && gw[1] == 864.0 &&
          upv[0] + uvp[0] == gw[0] && upv[1] + uvp[1] == gw[1],
      "product rule: u'v + uv' pins grad w = (972, 864)");
```

listing 104: sum and product rules pinned against the exact gradients at (2, 3) in gradient.c (source: math/samples/src/Ch10/gradient.c, lines 85-102)

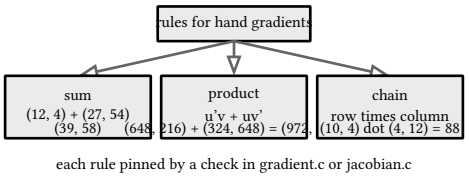


figure 61: the three rules that keep hand gradients cheap, each pinned by a chapter 10 check

sources: mml-book draft 2024-01-15, ch 5, def 5.5-5.8, examples 5.6-5.7, 5.9-5.11, 5.15, eqs 5.39-5.53, 5.57-5.66, 5.99-5.108, 5.146-5.180 [printed pp 146-169 / pdf pp 152-175], read from the verified anchor sheet ref/mml/anchors-calc.md. Matrix identity list after Petersen and Pedersen, the matrix cookbook, 2012, cited by name through mml

sec 5.5. Probed on this machine the same day. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch10`, 59 checks in chapter 10 of the math suite.

11. continuous optimization

Training a model means picking parameters that make an objective small, and the picking is done by an algorithm, not by algebra: mml-book opens its optimization chapter with the sentence “Training a machine learning model often boils down to finding a good set of parameters” (draft 2024-01-15, ch 7 opener [printed pp 225-226 / pdf pp 231-232]). This chapter builds the machinery in six moves: the terrain of local and global minima on one pinned quartic, gradient descent with its step-size bound and momentum correction, newton’s method as the hessian correction, constrained optimization through lagrange multipliers and kkt, convexity as the property that turns stationary into global, and the two program families (linear and quadratic) that close the story. Every behavioral claim below is one of the 81 checks in the 4 samples of chapter 11 or a definition quoted from the mml anchor sheet (mml-book draft 2024-01-15, every sheet number reverified by 53 numpy recomputes). Where the book 9, ch 36 (practical data structures and algorithms, numerical methods) chapter of the sibling book times these loops for contest boards, this chapter stays theory first: why the steps shrink, why the ball remembers, and why convexity removes the search. Gradients and Hessians come from book 2, ch 10 (mathematics for programmers, multivariate calculus).

11.1. stationary points and basins

Optimization is minimization by convention. A point is stationary when its gradient vanishes, and on a 1D slice that means the derivative’s real roots: “Stationary points are the real roots of the derivative” (mml ch 7 opener). The chapter’s running example is the quartic

$$\ell(x) = x^4 + 7x^3 + 5x^2 - 17x + 3,$$

with derivative $4x^3 + 21x^2 + 10x - 17$ and second derivative $12x^2 + 42x + 10$ (mml eqs 7.1-7.3). Abel-Ruffini says degree 5 and up has no root formula in general, so even finding stationary points is an iterative job, and the derivative here is only cubic, which is why this quartic makes a fair test fixture: its three stationary points can be pinned once and reused all chapter. The second-derivative test sorts them: positive curvature means a minimum, negative means a maximum.

The dry run: the three roots of $4x^3 + 21x^2 + 10x - 17$ pin as −4.48026848037364, −1.43211239686132, and 0.662380877234966.

1. At $x = -4.48026848037364$ the value is -47.0747900996365 and $12x^2 + 42x + 10$ gives $62.7023916991 > 0$: the global minimum.
2. At $x = -1.43211239686132$ the curvature is $-25.5373696612 < 0$: a local maximum, value 21.2467239746817.
3. At $x = 0.662380877234966$ the curvature is $43.0849779622 > 0$: a second, shallower local minimum, value -3.83990262504519.

The recognition in two numbers: the global minimum sits at -47.07 while the local trap two basins away bottoms out at -3.84. An iterative method that only walks downhill lands in whichever basin it starts in, and the mml opener notes that starting this same function at $x_0 = 0$ “would have led us to the wrong minimum”. Convexity, previewed in the last third of this chapter, is the property that deletes the word local from that sentence.

```
// the chapter's running quartic, mml ch 7 opener (eqs 7.1-7.3)
[[nodiscard]] static double ell(double t) {
  return t * t * t * t + 7 * t * t * t + 5 * t * t - 17 * t + 3;
}
[[nodiscard]] static double elld1(double t) {
  return 4 * t * t * t + 21 * t * t + 10 * t - 17;
}
[[nodiscard]] static double elld2(double t) { return 12 * t * t + 42 * t + 10; }
```

listing 105: the quartic fixture with its two derivatives, the chapter's pinned terrain (source: math/samples/src/Ch11/newtonopt.c, lines 20-27)

```
  // the other two stationary points from the other two basins
  const double r2 = newton_min(-1.5, 11, lad, &n);
  const double r3 = newton_min(0.5, 11, lad, &n);
  CHECK(close_to(r2, -1.43211239686132),
        "from -1.5 pins the local max -1.43211239686132");
  CHECK(close_to(r3, 0.662380877234966),
        "from 0.5 pins the local min 0.662380877234966");
  CHECK(elld2(r2) < 0 && elld2(r1) > 0 && elld2(r3) > 0,
        "second-derivative signs read max, min, min");
```

listing 106: the sign test reading max, min, min from the curvature at the three roots (source: math/samples/src/Ch11/newtonopt.c, lines 82-90)

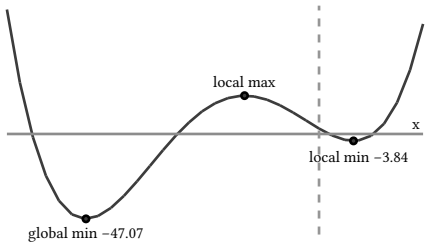


figure 62: the quartic terrain: one global minimum, one local maximum, one local trap

PITFALL: THE STARTING POINT CHOOSES THE BASIN

Downhill-only methods are blind to the far valley. On this quartic, a start at $x_0 = 0$ walks into the -3.84 trap and never sees -47.07. The fix is not a smarter step, it is either many starts or, when the objective is convex, the guarantee developed in the convexity section below that no trap exists.

11.2. gradient descent

The gradient points uphill, so we step against it. With $f : \mathbb{R}^d \to \mathbb{R}$ differentiable and no closed-form minimizer, mml eq 7.6 iterates

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i - \gamma_i((\nabla f)(\boldsymbol{x}_i))^\top$$

for a step-size $\gamma_i \geq 0$ (the transpose is the book's row-vector gradient convention, and the margin names γ the learning rate). The pinned fixture is mml example 7.1: $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x} - \boldsymbol{b}^\top \boldsymbol{x}$ with $\boldsymbol{Q} = \begin{pmatrix} 2 & 1 \\ 1 & 20 \end{pmatrix}$ and $\boldsymbol{b} = (5, 3)$, whose analytic minimum is $\boldsymbol{Q}^{-1}\boldsymbol{b} = \left(\frac{97}{39}, \frac{1}{39}\right) \approx (2.487, 0.026)$. The condition number of $\boldsymbol{Q}$ is $20.0554/1.9446 \approx 10.3$: one flat valley direction and one steep one, the shape that makes plain descent zigzag.

The dry run: from $\boldsymbol{x}_0 = (-3, -1)$ with $\gamma = 0.085$ the gradient is $(-12, -26)$, so the step adds 1.02 and 2.21.

1. One step lands at $(-1.98, 1.21)$, and mml prints exactly these coordinates in its figure 7.3 walk-through.

2. Recomputing the gradient there, $(-7.75, 19.22)$, the second step lands at $(-1.32125, -0.4237)$, already oscillating across the valley: the second coordinate flips sign every step.
3. After 8 steps the iterate is $(1.20693630992, 0.0156794647305)$ at distance 1.28028193209 from the minimum, and f fell on every single step.

k	x_1	x_2	f	distance
0	−3	−1	40	5.5822
1	−1.98	1.21	22.4356	4.6215
2	−1.32125	−0.4237	11.9781	3.8348
4	−0.158998	−0.155699	1.55455	2.6524
8	1.206936	0.015679	−4.60364	1.2803

```
  double x1 = -3.0, x2 = -1.0; // x0
  double g1 = q11 * x1 + q12 * x2 - b1;
  double g2 = q12 * x1 + q22 * x2 - b2;
  // grad f(x0) = (-12, -26), f(x0) = 40
  CHECK(g1 == -12.0 && g2 == -26.0, "grad at x0 pins (-12, -26)");
  CHECK(fval(-3.0, -1.0) == 40.0, "f(x0) pins 40");

  // fixed step gamma = 0.085, the mml figure 7.3 ladder
  constexpr double gam = 0.085;
  double fprev = fval(x1, x2);
  int monotone = 1;
  for (int k = 1; k <= 8; k++) {
    g1 = q11 * x1 + q12 * x2 - b1;
    g2 = q12 * x1 + q22 * x2 - b2;
    x1 = x1 - gam * g1;
    x2 = x2 - gam * g2;
    if (fval(x1, x2) > fprev)
      monotone = 0;
    fprev = fval(x1, x2);
    if (k == 1) {
      // x1 = (-1.98, 1.21)
      CHECK(close_to(x1, -1.98) && close_to(x2, 1.21),
            "step 1 pins (-1.98, 1.21)");
    }
    if (k == 2) {
      // x2 = (-1.32125, -0.4237)
      CHECK(close_to(x1, -1.32125) && close_to(x2, -0.4237),
            "step 2 pins (-1.32125, -0.4237)");
    }
    if (k == 4) {
      // x4 = (-0.15899827125, -0.15569942)
      CHECK(close_to(x1, -0.15899827125) && close_to(x2, -0.15569942),
            "step 4 pins (-0.15899827125, -0.15569942)");
    }
  }
  CHECK(monotone, "f decreases on every 0.085 step");
```

listing 107: the fixed-step ladder on the example 7.1 quadratic, iterates pinned against the precomputed column (source: math/samples/src/Ch11/descent.c, lines 42-77)

The step-size question has a hard edge. The eigenvalues of $\boldsymbol{Q}$ are $11 \pm \sqrt{82}$, so the largest curvature is $L = 20.0554$ and any fixed $\gamma > 2/L = 0.0997238$ amplifies the error in the steep direction each step: the error $\boldsymbol{x} - \boldsymbol{x}^*$ (with $\boldsymbol{x}^*$ the minimum) is mapped by $\boldsymbol{I} - \gamma\boldsymbol{Q}$, which then has an eigenvalue outside the unit circle. Pinning it: at $\gamma = 2.2/L = 0.1097$ the 30th iterate reaches magnitude $|\boldsymbol{x}_{30}| = 314.967$ with $f = 995753$, monotonically worse.

The same edge explains the two heuristics mml quotes from Toussaint (2012): when the value increases after a step, undo and decrease, and when it decreases, try growing the step, which "guarantees monotonic convergence". A schedule that decays on its own, $\gamma_i = 0.15/(i+1)$, buys the same safety differently: fixed 0.15 explodes past 10^{17} within 60 steps, while the decayed ladder from the same unsafe 0.15 start is alive at distance 1.27588449602 after 60 steps, and the honest fixed 0.085 is far ahead at 1.06226e-4. Decay trades speed for survival when L is unknown.

```
CHECK(close_to(x1, 1.20693630992) && close_to(x2, 0.0156794647305),
      "step 8 pins (1.20693630992, 0.0156794647305)");
CHECK(close_to(dist_to_min(x1, x2), 1.28028193209),
      "step 8 distance to minimum pins 1.28028193209");

// curvature: eigenvalues of Q are 11 +- sqrt(82), so 2/L = 0.0997238...
const double L = 11.0 + sqrt(82.0);
const double l = 11.0 - sqrt(82.0);
CHECK(close_to(2.0 / L, 0.0997238390698761), "2/L pins 0.0997238390698761");

// divergence: gamma = 2.2/L steps past the 2/L bound
const double gdiv = 2.2 / L;
x1 = -3.0;
x2 = -1.0;
for (int k = 1; k <= 30; k++) {
  g1 = q11 * x1 + q12 * x2 - b1;
  g2 = q12 * x1 + q22 * x2 - b2;
  x1 = x1 - gdiv * g1;
  x2 = x2 - gdiv * g2;
  if (k == 30) {
    // |x_30| = 314.967147957, f = 995753.21826
    CHECK(close_to(hypot(x1, x2), 314.967147957),
          "gamma 2.2/L after 30 steps pins |x| = 314.967147957");
```

listing 108: the 2/L bound and the divergence ladder past it (source: math/samples/src/Ch11/descent.c, lines 79-101)

```
            "gamma 2.2/L after 30 steps pins f = 995753.21826");
  }
}
CHECK(fval(x1, x2) > 1e5, "diverged f exceeds 1e5");

// fixed 0.15 (past 2/L) explodes; the decay schedule gamma_i = 0.15/(i+1)
// survives because the steps shrink
x1 = -3.0;
x2 = -1.0;
for (int i = 0; i < 60; i++) {
  g1 = q11 * x1 + q12 * x2 - b1;
  g2 = q12 * x1 + q22 * x2 - b2;
  x1 = x1 - 0.15 * g1;
  x2 = x2 - 0.15 * g2;
}
CHECK(dist_to_min(x1, x2) > 1e17, "fixed 0.15 diverges past 1e17");
x1 = -3.0;
x2 = -1.0;
for (int i = 0; i < 60; i++) {
  const double gi = 0.15 / (i + 1);
  g1 = q11 * x1 + q12 * x2 - b1;
  g2 = q12 * x1 + q22 * x2 - b2;
  x1 = x1 - gi * g1;
```

listing 109: the decaying schedule rescuing an unsafe first step (source: math/samples/src/Ch11/descent.c, lines 103-125)

NOTE: STEP-SIZE IS THE LEARNING RATE

mml's margin on section 7.1.1: "The step-size is also called the learning rate." Too small crawls, too large overshoots or diverges as pinned above, and adaptive methods rescale it per iteration. The 2/L analysis here is exact only for quadratics, but the qualitative edge is universal: curvature times step must stay under 2.

Momentum is the heavy-ball correction, mml eq 7.11:

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i - \gamma_i((\nabla f)(\boldsymbol{x}_i))^\top + \alpha\Delta\boldsymbol{x}_i, \quad \Delta\boldsymbol{x}_i = \boldsymbol{x}_i - \boldsymbol{x}_{i-1},$$

with $\alpha \in [0, 1]$: "This memory dampens oscillations and smoothes out the gradient updates", a heavy ball reluctant to change direction. On a quadratic with curvatures l and L, the theoretically optimal pair is $\alpha = \left(\left(\sqrt{L} - \sqrt{l}\right)/\left(\sqrt{L} + \sqrt{l}\right)\right)^2 = 0.275732$ and $\gamma = \left(2/\left(\sqrt{L} + \sqrt{l}\right)\right)^2 = 0.115976$, and note the second number exceeds $2/L$: momentum extends the stability region. The sample derives both constants from L and l in code, no magic numbers, then pins the ladder.

```
}
// decayed ladder after 60 steps: dist = 1.27588449602
CHECK(close_to(dist_to_min(x1, x2), 1.27588449602),
      "decay 0.15/(i+1) after 60 steps pins dist = 1.27588449602");
// the honest fixed 0.085 over the same 60 steps: dist = 1.06226e-4
x1 = -3.0;
x2 = -1.0;
for (int i = 0; i < 60; i++) {
  g1 = q11 * x1 + q12 * x2 - b1;
  g2 = q12 * x1 + q22 * x2 - b2;
  x1 = x1 - 0.085 * g1;
  x2 = x2 - 0.085 * g2;
}
CHECK(close_to(dist_to_min(x1, x2), 0.000106226044652),
      "fixed 0.085 after 60 steps pins dist = 0.000106226044652");

// optimal fixed step 2/(L+l) = 2/22 versus heavy ball with
// alpha = ((sqrt(L)-sqrt(l))/(sqrt(L)+sqrt(l)))^2,
// gamma = (2/(sqrt(L)+sqrt(l)))^2, both derived from L and l
x1 = -3.0;
x2 = -1.0;
const double gam_gd = 2.0 / (L + l);
for (int k = 1; k ≤ 30; k++) {
  g1 = q11 * x1 + q12 * x2 - b1;
  g2 = q12 * x1 + q22 * x2 - b2;
  x1 = x1 - gam_gd * g1;
  x2 = x2 - gam_gd * g2;
  if (k == 30) {
    // plain dist@30 = 0.016301722
    CHECK(close_to(dist_to_min(x1, x2), 0.016301722),
          "plain optimal step dist at 30 pins 0.016301722");
  }
}
const double sl = sqrt(L), sml = sqrt(l);
const double alpha_hb = ((sl - sml) / (sl + sml)) * ((sl - sml) / (sl + sml));
const double gam_hb = (2.0 / (sl + sml)) * (2.0 / (sl + sml));
CHECK(gam_hb > 2.0 / L, "heavy-ball step exceeds 2/L and still converges");
double xc1 = -3.0, xc2 = -1.0, xp1 = -3.0, xp2 = -1.0;
for (int k = 1; k ≤ 30; k++) {
  g1 = q11 * xc1 + q12 * xc2 - b1;
```

listing 110: optimal fixed step versus heavy ball, both parameterized by the eigenvalues (source: math/samples/src/Ch11/descent.c, lines 127-166)

The two-number play: at step 30, plain descent with its own optimal fixed step sits 0.0163 from the minimum while the heavy ball sits 4.19×10^{-7} away, five orders for one line of memory.

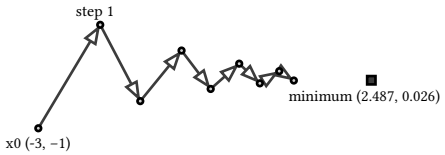


figure 63: the 0.085 zigzag across the valley toward (2.487, 0.026)

11.3. newton's method for optimization

Newton for minimization is book 2, ch 9 (mathematics for programmers, single-variable calculus) root-finding applied to the derivative, with curvature in the denominator. In 1D the update is $x_{i+1} = x_i - f'(x_i)/f''(x_i)$, and

in d dimensions the gradient gets premultiplied by the inverse Hessian, the matrix of second partials from book 2, ch 10 (mathematics for programmers, multivariate calculus). On the running quartic from $x_0 = -6$:

The dry run: at $x_0 = -6$ the residual is $f' = -185$ on curvature $f'' = 190$.

1. The first step is one fraction: $-6 - (-185)/190 = -6 + 0.973684 = -5.02631578947$.
2. The errors then square: 1.520, 5.460×10^{-1}, 1.085×10^{-1}, 5.658×10^{-3}, 1.665×10^{-5}, 1.448×10^{-10}, 8.9×10^{-16}, six steps to machine precision on the derivative.
3. The squaring constant pins at 0.5225: each error is that multiple of the previous error squared, the measured face of quadratic convergence.

k	x_k	$e_k = x_k - r_1$	e_k / e_{k-1}^2
0	−6	1.520	–
1	−5.026316	0.5460	0.236
2	−4.588747	0.1085	0.364
3	−4.485926	5.658e-3	0.481
4	−4.480285	1.665e-5	0.520
5	−4.48026848052	1.448e-10	0.523

```
int main(void) {
  // ladder from x0 = -6 toward the global minimum r1 = -4.48026848037364
  double lad[12];
  int n = 0;
  const double r1 = newton_min(-6.0, 11, lad, &n);
  // k = 1..3: -5.02631578947, -4.58874722157, -4.48592605961
  CHECK(close_to(lad[1], -5.02631578947), "newton step 1 pins -5.02631578947");
  CHECK(close_to(lad[2], -4.58874722157), "newton step 2 pins -4.58874722157");
  CHECK(close_to(lad[3], -4.48592605961), "newton step 3 pins -4.48592605961");
  CHECK(n == 6, "six newton steps reach |ell'| ≤ 1e-13");
  CHECK(close_to(r1, -4.48026848037364), "converged x pins -4.48026848037364");
  CHECK(close_to(ell(r1), -47.0747900996365),
        "ell at the global minimum pins -47.0747900996365");
  CHECK(close_to(elld2(r1), 62.7023916991),
        "curvature at the minimum pins 62.7023916991");

  // quadratic convergence: each error is about 0.5225 times the square of the
  // previous; e4 = 1.66494742224e-05, e5 = 1.44844136685e-10
  const double e4 = fabs(lad[4] - r1), e5 = fabs(lad[5] - r1);
  CHECK(close_to(e4, 1.66494742224e-05),
        "error at step 4 pins 1.66494742224e-05");
  CHECK(close_to(e5, 1.44844136685e-10),
        "error at step 5 pins 1.44844136685e-10");
  CHECK(fabs(e5 / (e4 * e4) - 0.5225) < 0.001,
        "error ratio e5/e4^2 pins the squaring constant 0.5225");
```

listing 111: the newton ladder with the error-squaring pins (source: math/samples/src/Ch11/newtonopt.c, lines 56-80)

On a quadratic the Hessian is constant, so one newton step is exact: $x_1 = x_0 - Q^{-1}(Qx_0 - b) = Q^{-1}b$, and the sample pins the landing on $(\frac{97}{39}, \frac{1}{39})$ to 10^{-12} from $(-3, -1)$, where fixed-step descent after 8 steps was still 1.28 away. That one-step exactness is also the weakness: the inverse Hessian is $d \times d$ and only local, so the workhorse middle ground is a descent direction with a searched step length. Along $d = -\nabla f$ on a quadratic, the exact minimizer of $f(x + td)$ is closed form, $t^* = (d \cdot d)/(d^\top Q d)$: from x_0 it pins at $t^* = 820/14432 = 5/88 = 0.0568182$, landing on $(-\frac{51}{22}, \frac{21}{44})$ with $f = 735/44 = 16.7045$. Backtracking approximates the same thing with evaluations only: halve t from 1 until the armijo inequality $f(x + td) \le f(x) + ct \cdot \nabla f \cdot d$ holds, here 4 rejections down to $t = 1/16$ and $f = 271/16 = 16.9375$, above the exact search but found without knowing Q.

```
  // the contrast series: fixed-step gradient walk on ell' at gamma = 0.008
  // from the same x0 = -6, 8 steps land at error 2.9118566664207235e-4
  double tg = -6.0;
  for (int k = 0; k < 8; k++) {
    tg = tg - 0.008 * elld1(tg);
  }
  CHECK(close_to(fabs(tg - r1), 2.9118566664207235e-4),
        "gamma 0.008 walk pins error 2.9118566664207235e-4 at step 8");

  // 2d newton on the quadratic: x1 = x0 - Q^-1 grad lands on Q^-1 b in one step
  const double gx = q11 * -3.0 + q12 * -1.0 - b1;
  const double gy = q12 * -3.0 + q22 * -1.0 - b2;
  const double det = q11 * q22 - q12 * q12;
  const double nx = -3.0 - (q22 * gx - q12 * gy) / det;
  const double ny = -1.0 - (-q12 * gx + q11 * gy) / det;
  CHECK(close_to(nx, 97.0 / 39.0) && close_to(ny, 1.0 / 39.0),
        "one hessian step pins the minimum (97/39, 1/39)");
  CHECK(fabs(nx - 97.0 / 39.0) < 1e-12 && fabs(ny - 1.0 / 39.0) < 1e-12,
        "the one-step landing is exact to 1e-12");

  // exact line search along -grad from x0: t* = (g.g)/(g^T Q g) = 820/14432
  // = 5/88 = 0.0568181818181818
  const double gg = gx * gx + gy * gy;
  const double gqg = q11 * gx * gx + 2 * q12 * gx * gy + q22 * gy * gy;
```

listing 112: the one-step hessian solve and the closed-form line search (source: math/samples/src/Ch11/newtonopt.c, lines 92-115)

```
  CHECK(close_to(gg, 820.0) && close_to(gqg, 14432.0),
        "g.g pins 820 and g^T Q g pins 14432");
  CHECK(fabs(tstar - 5.0 / 88.0) < 1e-12, "exact line search t* = 5/88");
  CHECK(close_to(-3.0 - tstar * gx, -51.0 / 22.0) &&
            close_to(-1.0 - tstar * gy, 21.0 / 44.0),
        "line-searched point pins (-51/22, 21/44)");
  CHECK(close_to(fval(-3.0 - tstar * gx, -1.0 - tstar * gy), 735.0 / 44.0),
        "line-searched value pins 735/44 = 16.7045454545");

  // armijo backtracking: start t = 1, halve while
  // f(x + t d) > f(x) + c t grad.d with c = 1e-4 and d = -grad
  double t = 1.0;
  int rejections = 0;
  while (fval(-3.0 + t * 12.0, -1.0 + t * 26.0) > 40.0 + 1e-4 * t * -820.0) {
```

listing 113: armijo backtracking, 4 halvings to the accepted step (source: math/samples/src/Ch11/newtonopt.c, lines 117-130)

The two-number play: after 6 newton steps the log error is -15, after 8 fixed-gradient steps at $\gamma = 0.008$ it is -3.5.

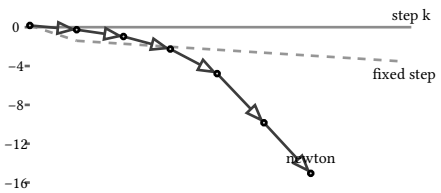


figure 64: log10 error per step: newton's squaring cliff versus the fixed-step straight line

11.4. constrained optimization and lagrange multipliers

Reality adds side conditions: minimize $f(\boldsymbol{x})$ subject to $g_i(\boldsymbol{x}) \leq 0$ (mml eq 7.17). The book's move is to replace the infinite penalty for violated constraints with a linear one, the lagrangian (mml eq 7.20)

$$L(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) + \sum_{i=1}^{m} \lambda_i g_i(\boldsymbol{x}), \quad \lambda_i \geq 0,$$

and then trade variables: the primal problem in $\boldsymbol{x}$ becomes the dual problem $\max_{\lambda \geq 0} D(\lambda)$ with $D(\lambda) = \min_{\boldsymbol{x}} L(\boldsymbol{x}, \lambda)$. Weak duality is one line: L with $\lambda \geq 0$ never exceeds the penalized original, so maximizing the lower bound and then minimizing can only understate the optimum, and the gap closes exactly for the convex programs of the next section. Equality constraints $h(\boldsymbol{x}) = 0$ join as two inequalities, which leaves their multipliers unconstrained.

The pinned fixture is $\min x^2 + 2y^2$ subject to $2x + y = 5$. The lagrangian is $L = x^2 + 2y^2 + \lambda(2x + y - 5)$ and the stationarity system is three linear equations.

The dry run: eliminate by hand before any code.

1. $\partial L/\partial x = 2x + 2\lambda = 0$ gives $x = -\lambda$.
2. $\partial L/\partial y = 4y + \lambda = 0$ gives $y = -\lambda/4$.
3. The constraint then reads $-2\lambda - \lambda/4 = 5$, so $\lambda = -\frac{20}{9}$, $x = \frac{20}{9}$, $y = \frac{5}{9}$, and $f^* = \frac{50}{9} = 5.5556$.

```
// cramer solve of the 3x3 stationarity system for (x, y, lambda):
// 2x + 2 lambda = 0, 4y + lambda = 0, 2x + y = 5
static void stationarity(double v[static 3]) {
  const double m[3][3] = {{2, 0, 2}, {0, 4, 1}, {2, 1, 0}};
  const double d = det3(m);
  double r[3][3];
  r[0][0] = 0;
  r[0][1] = 0;
  r[0][2] = 2;
  r[1][0] = 0;
  r[1][1] = 4;
  r[1][2] = 1;
  r[2][0] = 5;
  r[2][1] = 1;
  r[2][2] = 0;
  v[0] = det3(r) / d;
  r[0][0] = 2;
  r[0][1] = 0;
  r[0][2] = 2;
  r[1][0] = 0;
  r[1][1] = 0;
  r[1][2] = 1;
  r[2][0] = 2;
  r[2][1] = 5;
  r[2][2] = 0;
  v[1] = det3(r) / d;
  r[0][0] = 2;
  r[0][1] = 0;
  r[0][2] = 0;
  r[1][0] = 0;
  r[1][1] = 4;
  r[1][2] = 0;
  r[2][0] = 2;
  r[2][1] = 1;
  r[2][2] = 5;
  v[2] = det3(r) / d;
}
```

listing 114: the 3x3 stationarity system solved by cramer determinants (source: math/samples/src/Ch11/lagrange.c, lines 41-77)

```
int main(void) {
  double v[3];
  stationarity(v);
  // x = 20/9, y = 5/9, lambda = -20/9
  CHECK(close_to(v[0], 20.0 / 9.0), "stationarity x pins 20/9");
  CHECK(close_to(v[1], 5.0 / 9.0), "stationarity y pins 5/9");
  CHECK(close_to(v[2], -20.0 / 9.0), "stationarity lambda pins -20/9");
  // f* = 400/81 + 50/81 = 50/9
  CHECK(close_to(fxy(v[0], v[1]), 50.0 / 9.0),
        "constrained optimum pins f* = 50/9");
  // the constraint is satisfied exactly
  CHECK(fabs(2 * v[0] + v[1] - 5.0) < 1e-12, "the optimum sits on 2x + y = 5");

  // geometric reading: grad f = (2x, 4y) = (40/9, 20/9) aligns with
  // grad h = (2, 1), and grad f = -lambda grad h
  const double gf1 = 2 * v[0], gf2 = 4 * v[1];
  CHECK(close_to(gf1, 40.0 / 9.0) && close_to(gf2, 20.0 / 9.0),
        "grad f at the optimum pins (40/9, 20/9)");
  CHECK(fabs(gf1 * 1.0 - gf2 * 2.0) < 1e-12,
        "grad f is parallel to grad h (cross product 0)");
  CHECK(close_to(gf1, -v[2] * 2.0) && close_to(gf2, -v[2] * 1.0),
        "grad f equals -lambda grad h");

  // sliding along the constraint line (x + t, y - 2t) costs 9 t^2 exactly
  CHECK(close_to(fxy(v[0] + 1.0, v[1] - 2.0), 131.0 / 9.0),
        "t = 1 along the line pins 131/9");
  CHECK(close_to(fxy(v[0] + 1.0, v[1] - 2.0) - 50.0 / 9.0, 9.0),
        "the cost of sliding pins 9 t^2");
  CHECK(close_to(fxy(v[0] + 0.5, v[1] - 1.0), 50.0 / 9.0 + 9.0 * 0.25),
        "t = 1/2 along the line pins f* + 9/4");
```

listing 115: the rational pins, the gradient alignment, and the cost of sliding along the constraint (source: math/samples/src/Ch11/lagrange.c, lines 84-113)

The geometric reading is where the multiplier stops being bookkeeping: at the optimum $\nabla f = (2x, 4y) = \left(\frac{40}{9}, \frac{20}{9}\right)$ and $\nabla h = (2, 1)$, and the cross product is exactly zero, pinned to 10^{-12}. Level sets of the objective are ellipses, the constraint is a line, and the constrained optimum is the tangency: moving along the line to first order changes nothing, which is only possible when the objective's gradient is perpendicular to the line's direction, that is, parallel to ∇h. The relation $\nabla f = -\lambda \nabla h$ with $\lambda = -\frac{20}{9}$ is that alignment as an equation, and sliding $(x + t, y - 2t)$ along the line costs exactly $9t^2$: $\frac{131}{9}$ at $t = 1$, $f^* + \frac{9}{4}$ at $t = \frac{1}{2}$.

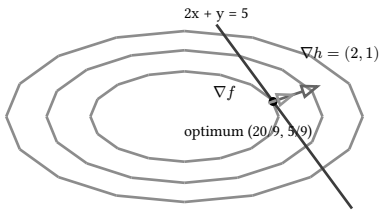


figure 65: the tangency: level ellipses of the objective against the constraint line, gradients aligned at the optimum

Inequalities get the same machinery with a sign condition, the kkt system: stationarity of the lagrangian, feasibility, multipliers $\lambda \geq 0$, and complementary slackness $\lambda_i g_i = 0$, meaning only tight constraints may carry a nonzero multiplier. The pinned example is mml example 7.6, the quadratic $\frac{1}{2}\boldsymbol{x}^\top \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix} \boldsymbol{x} + (5, 3)^\top \boldsymbol{x}$ on the box $|x_i| \leq 1$. Its unconstrained minimum sits at $-\boldsymbol{Q}^{-1}\boldsymbol{c} = \left(-\frac{17}{7}, -\frac{1}{7}\right)$ with value $-\frac{44}{7}$, which is also the dual value $D(0)$, and the box clips it to $\left(-1, -\frac{1}{2}\right)$ with value $-\frac{9}{2}$. There the gradient pins at $\left(\frac{5}{2}, 0\right)$: the free coordinate has zero slope, and the active bound $-x_1 \leq 1$ absorbs the remaining $\frac{5}{2}$ through its multiplier, exactly the kkt ledger with all other multipliers at zero.

```
// QP: unconstrained minimum -Q^-1 c = (-17/7, -1/7) with value -44/7,
// which is also the dual value D(0)
const double det = q11 * q22 - q12 * q12; // 7
const double ux1 = -(q22 * c1 - q12 * c2) / det;
const double ux2 = -(-q12 * c1 + q11 * c2) / det;
CHECK(close_to(ux1, -17.0 / 7.0) && close_to(ux2, -1.0 / 7.0),
      "unconstrained QP minimum pins (-17/7, -1/7)");
CHECK(close_to(fqp(ux1, ux2), -44.0 / 7.0),
      "unconstrained QP value pins -44/7, the dual D(0)");

// the box clips it to (-1, -1/2) with value -9/2
CHECK(close_to(fqp(-1.0, -0.5), -4.5), "clipped optimum pins f = -9/2");
const double g1 = q11 * -1.0 + q12 * -0.5 + c1;
const double g2 = q12 * -1.0 + q22 * -0.5 + c2;
CHECK(close_to(g1, 2.5) && g2 == 0.0,
      "gradient at the clipped optimum pins (5/2, 0)");

// kkt: the active constraint is -x1 ≤ 1 (the x1 ≥ -1 side, since the
// unconstrained argmin sits at x1 = -17/7), grad g = (-1, 0), its multiplier
// lambda = g1 = 5/2 ≥ 0 restores stationarity, the inactive x2 bounds get 0
const double lam = g1 / 1.0;
CHECK(close_to(lam, 2.5), "the active multiplier pins lambda = 5/2");
CHECK(g1 - lam * 1.0 == 0.0 && g2 == 0.0,
      "grad + lambda grad g vanishes at the clipped optimum");
CHECK(lam ≥ 0.0, "the multiplier is nonnegative");
CHECK(-44.0 / 7.0 < -4.5, "the constraint costs 44/7 - 9/2 = 25/14");
```

listing 116: the box qp: clip, gradient readout, and the kkt multiplier ledger (source: math/samples/src/Ch11/lagrange.c, lines 115-140)

The two-number play: the constraint costs $\frac{25}{14}$, unconstrained $-\frac{44}{7} = -6.286$ versus clipped $-\frac{9}{2} = -4.5$.

11.5. convex sets and functions

Convexity is the property that removes local traps. A set is convex when it contains every segment between its points, and a function is convex when its domain is such a set and chords never dip below the graph, mml definitions 7.2 and 7.3: for $0 \le \theta \le 1$,

$$f(\theta \boldsymbol{x} + (1-\theta)\boldsymbol{y}) \le \theta f(\boldsymbol{x}) + (1-\theta) f(\boldsymbol{y}).$$

The same inequality read at a weighted average is Jensen's inequality in mml's phrasing, "a whole class of inequalities for taking nonnegative weighted sums of convex functions are all called Jensen's inequality", and it is the one-line consequence of the definition. Two more equivalences make it checkable: the first-order one, $f(\boldsymbol{y}) \ge f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x})$, every tangent plane a global lower bound (mml eq 7.31), and the second-order one: a twice-differentiable f is convex if and only if its Hessian is positive semidefinite everywhere. The descent quadratic's Hessian $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 20 \end{smallmatrix}\right)$ has eigenvalues $11 \pm \sqrt{82} = 1.9446$ and 20.0554, both positive, so that bowl is convex, which is why its only stationary point, the $\left(\frac{97}{39}, \frac{1}{39}\right)$ every method in this chapter walked toward, is its global minimum.

The dry run: negative entropy $f(x) = x \log_2 x$ is convex for $x > 0$ (mml example 7.3), and both tests pin on it at $x = 2, y = 4$.

1. Midpoint: $f(3) = 3 \log_2 3 = 4.75489$ while the chord reads $0.5 f(2) + 0.5 f(4) = 1 + 4 = 5$, so the definition holds by 0.245.
2. Tangent: $f(2) + f'(2)(4-2) = 2 + 2(1 + 1/\ln 2) = 6.88539 \le 8 = f(4)$, with $f'(x) = \log_2 x + 1/\ln 2$ pinning at 2.44270.

3. The quartic fails the same midpoint test by 43.71: at the midpoint of its two minima the graph sits at 18.257 over a chord at −25.457, the measured face of nonconvexity.

```
// convexity fixtures: negative entropy x log2 x against definition 7.3 at
// x = 2, y = 4, theta = 1/2: f(3) = 4.75488750216 ≤ 5
const double lo2 = 1.0 / log(2.0);
CHECK(close_to(3.0 * log(3.0) * lo2, 4.75488750216),
      "3 log2 3 pins 4.75488750216");
CHECK(3.0 * log(3.0) * lo2 ≤ 5.0,
      "the midpoint of the chord pins f(3) ≤ 5");
// tangent bound (eq 7.31) at x = 2: f'(2) = 1 + 1/ln2 = 2.44269504089,
// f(2) + f'(2) * 2 = 6.88539008178 ≤ f(4) = 8
const double df2 = 1.0 + lo2;
CHECK(close_to(df2, 2.44269504089), "f'(2) pins 2.44269504089");
CHECK(close_to(2.0 + df2 * 2.0, 6.88539008178) && 2.0 + df2 * 2.0 ≤ 8.0,
      "the tangent line stays below the curve");

// nonconvexity of the quartic: midpoint of the two minima sits 43.71 above
// the chord; m = (r1 + r3)/2 = -1.90894380157 with r1, r3 the two minima
const double r1 = -4.48026848037364, r3 = 0.662380877234966;
const double mid = 0.5 * (r1 + r3);
CHECK(close_to(mid, -1.90894380157), "the midpoint pins -1.90894380157");
CHECK(close_to(ell(mid), 18.2573707464),
      "ell at the midpoint pins 18.2573707464");
CHECK(close_to(0.5 * (ell(r1) + ell(r3)), -25.4573463623),
      "the chord midpoint pins -25.4573463623");
CHECK(ell(mid) > 0.5 * (ell(r1) + ell(r3)),
      "the quartic violates definition 7.3 by 43.71");

// the hessian [[2, 1], [1, 20]] of the descent quadratic has eigenvalues
// 11 +- sqrt(82) = 1.94461486186258 and 20.0553851381374, both positive
const double e1 = 11.0 - sqrt(82.0), e2 = 11.0 + sqrt(82.0);
CHECK(close_to(e1, 1.94461486186258) && close_to(e2, 20.0553851381374),
      "hessian eigenvalues pin 11 +- sqrt(82)");
CHECK(e1 > 0 && e2 > 0, "both curvatures positive, so the bowl is convex");
```

listing 117: midpoint and tangent tests for the entropy, the quartic violation, and the hessian eigenvalues (source: math/samples/src/Ch11/programs.c, lines 125-156)

Closure extends the class: "A nonnegative weighted sum of convex functions is convex" (mml example 7.4), so sums of squared errors and nonnegative combinations inherit the property for free. And the payoff for optimization: if f is convex and differentiable and $\nabla f(\boldsymbol{x}^*) = 0$, then the tangent bound at that point says $f(\boldsymbol{y}) \geq f(\boldsymbol{x}^*)$ for every $\boldsymbol{y}$, so a stationary point of a convex differentiable function is automatically a global minimum. For convex programs, where the objective and inequality constraints are convex and the equality constraints cut convex sets, mml states strong duality: the dual optimum equals the primal optimum, and the search that opened this chapter, over basins and starts, collapses to finding one stationary point.

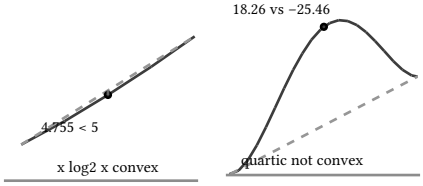


figure 66: the chord test: entropy passes by 0.245, the quartic fails by 43.71

11.6. programs: linear and quadratic

Two program families close the chapter, both convex. A linear program minimizes a linear objective over linear inequalities, $\min \boldsymbol{c}^\top \boldsymbol{x}$ subject to $\boldsymbol{A}\boldsymbol{x} \leq \boldsymbol{b}$ (mml eq 7.39), and mml's margin calls it "one of the most widely used approaches in industry". Its dual (mml eq 7.43) is another linear program in the multipliers, $\max -\boldsymbol{b}^\top \lambda$ subject to $\boldsymbol{c} + \boldsymbol{A}^\top \lambda = 0$ and $\lambda \geq 0$, so the solver can pick whichever side has fewer variables. The geometry: a feasible polytope, and an optimum at a vertex, because a linear objective has no interior curvature to reward. In 2 variables, vertex enumeration is the whole solver: cross every pair of constraint lines, keep the feasible crossings, read off the best.

The dry run: our own fixture (the mml draft's example 7.5 figure carries a known sign inconsistency in one constraint row, so this chapter rolls its own): minimize $-2x_1 - 3x_2$ subject to $x_1 + x_2 \leq 5$, $x_1 \leq 3$, $x_2 \leq 4$, $x_1 \geq 0$, $x_2 \geq 0$.

1. The 5 constraint rows pair into 10 combinations, 8 of which cross (rows 2 and 4, and rows 3 and 5, are parallel), and 5 of those crossings are feasible corners: $(0, 0)$, $(3, 0)$, $(3, 2)$, $(1, 4)$, $(0, 4)$.
2. The corner $(3, 4)$ is crossed out by the budget row, $3 + 4 = 7 > 5$, cut off by exactly 2.
3. Evaluating the objective at the survivors, $0, -6, -12, -14, -12$, the optimum is the vertex $(1, 4)$ with value -14, the crossing of rows 1 and 3.

vertex	$-2x_1 - 3x_2$	status
(0, 0)	0	feasible
(3, 0)	−6	feasible
(3, 2)	−12	feasible
(1, 4)	−14	optimal
(0, 4)	−12	feasible
(3, 4)	–	cut by row 1

```c
int main(void) {
  // enumerate all 10 pairs of constraints, keep feasible intersections
  int feasible = 0;
  double bestx1 = 0, bestx2 = 0, bestv = 1e30;
  int bestpair = -1;
  for (int i = 0; i < M; i++) {
    for (int j = i + 1; j < M; j++) {
      const double det = A[i][0] * A[j][1] - A[i][1] * A[j][0];
      if (fabs(det) < 1e-12)
        continue;
      const double x1 = (bv[i] * A[j][1] - A[i][1] * bv[j]) / det;
      const double x2 = (A[i][0] * bv[j] - bv[i] * A[j][0]) / det;
      int ok = 1;
      for (int k = 0; k < M && ok; k++)
        if (A[k][0] * x1 + A[k][1] * x2 > bv[k] + 1e-9)
          ok = 0;
      if (!ok)
        continue;
      feasible++;
      if (lpobj(x1, x2) < bestv) {
        bestv = lpobj(x1, x2);
        bestx1 = x1;
        bestx2 = x2;
        bestpair = i * 10 + j;
      }
    }
  }
  CHECK(feasible == 5, "the polytope has exactly 5 feasible vertices");
  CHECK(close_to(bestx1, 1.0) && close_to(bestx2, 4.0),
        "the lp optimum pins the vertex (1, 4)");
  CHECK(close_to(bestv, -14.0), "the lp optimal value pins -14");
  CHECK(bestpair == 2, "the optimum is the crossing of rows 1 and 3");
  // the infeasible crossing x1 = 3, x2 = 4 violates x1 + x2 ≤ 5 by 2
  CHECK(3.0 + 4.0 - 5.0 == 2.0,
        "the corner (3, 4) is cut off by the budget row");
```

listing 118: the lp solved by pair crossing and feasibility filtering (source: math/samples/src/Ch11/programs.c, lines 49-84)

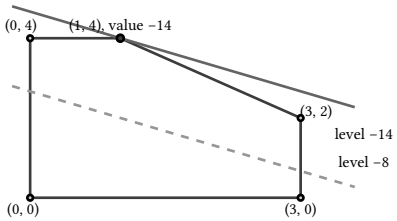


figure 67: the feasible pentagon with level lines of the objective sliding to the optimal vertex

A quadratic program keeps the linear constraints but minimizes $\frac{1}{2}\boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x} + \boldsymbol{c}^\top \boldsymbol{x}$ with $\boldsymbol{Q}$ positive definite (mml eq 7.45), convex by the second-order test since its Hessian is $\boldsymbol{Q}$ itself. The unconstrained minimizer is the closed form $-\boldsymbol{Q}^{-1}\boldsymbol{c}$ from the lagrange section, and the whole objective rearranges by completion of squares into $\frac{1}{2}(\boldsymbol{x}-\boldsymbol{m})^\top \boldsymbol{Q}(\boldsymbol{x}-\boldsymbol{m}) - \frac{44}{7}$ with $\boldsymbol{m} = \left(-\frac{17}{7}, -\frac{1}{7}\right)$: the constant is the unconstrained value, the quadratic term is nonnegative, and the sample pins the identity at 4 probe points. On the unit box the clipped point $\left(-1, -\frac{1}{2}\right)$ carries the kkt certificate from the previous section, and a 401 by 401 grid scan of the box confirms no feasible point beats $-\frac{9}{2}$.

```
// completion of squares: f(x) = 1/2 (x - m)^T Q (x - m) - 44/7 with
// m = -Q^-1 c = (-17/7, -1/7)
const double det = q11 * q22 - q12 * q12; // 7
const double m1 = -(q22 * c1 - q12 * c2) / det;
const double m2 = -(-q12 * c1 + q11 * c2) / det;
struct Probe {
  double x1, x2;
};
const struct Probe probes[] = {{0, 0}, {1, 2}, {-3, -1}, {2.5, 0.5}};
int squares_ok = 1;
for (unsigned k = 0; k < sizeof probes / sizeof probes[0]; k++) {
  const double d1 = probes[k].x1 - m1, d2 = probes[k].x2 - m2;
  const double sq =
      0.5 * (q11 * d1 * d1 + 2 * q12 * d1 * d2 + q22 * d2 * d2) - 44.0 / 7.0;
  if (!close_to(sq, fqp(probes[k].x1, probes[k].x2)))
    squares_ok = 0;
}
CHECK(close_to(m1, -17.0 / 7.0) && close_to(m2, -1.0 / 7.0),
      "the square center pins (-17/7, -1/7)");
CHECK(squares_ok, "squares form equals f at 4 probe points");
CHECK(close_to(fqp(0, 0) - (-44.0 / 7.0), 44.0 / 7.0),
      "f(0) - (-44/7) pins the 1/2 m^T Q m constant");
```

listing 119: the qp as completed squares, the center and constant pinned (source: math/samples/src/Ch11/programs.c, lines 85-107)

VERIFY: THE GRID IS THE LIE DETECTOR

Algebra says the clipped point is optimal, and the kkt multipliers certify it, but a 401 by 401 sweep of the box is the check that needs no theory: its minimum pins at exactly −9/2 with the argmin within 0.01 of (-1, −1/2). When a certificate and a brute-force sweep agree, the fixture is trustworthy. This is the same discipline as book 2, ch 10 (mathematics for programmers, multivariate calculus) gradient checking by finite differences.

The two-number play: the lp reads its answer off 5 vertices, the qp pays $\frac{25}{14}$ for its box. Where these programs meet adversaries, zero-sum games solve to the same linear structure, which is book 2, ch 24 (mathematics for programmers, strategic games) territory, and where objectives stop being hand-differentiable, the gradients that feed every method in this chapter come from book 2, ch 12 (mathematics for programmers, automatic differentiation), the next chapter.

sources: mml-book draft 2024-01-15 (Deisenroth, Faisal, Ong, “Mathematics for Machine Learning”), ch 7 continuous optimization, sections 7.1-7.3 [printed pp 227-246 / pdf pp 233-252] and the ch 7 opener [printed pp 225-226 / pdf pp 231-232], read through the verified anchor sheet ref/mml/anchors-calc.md (53 numpy recomputes), no further pdf cross-verification per the coordinator dispatch naming the sheet as this chapter’s authority, all definitions quoted from that sheet, our own words and code throughout. The mml draft’s example 7.5 (eq 7.44) carries the sheet’s documented sign inconsistency, so all lp fixtures here are our own. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch11`, 81 checks in chapter 11 of the math suite (descent 19, lagrange 19, newtonopt 23, programs 20), zero skipped, format and asan clean.

12. automatic differentiation

Gradients drive everything downstream of this point in the book: descent in book 2, ch 11 (mathematics for programmers, continuous optimization) consumes one ∇L per step and the quality of the step is the quality of the gradient. This chapter builds the two machines that produce gradients exactly, at machine precision, from the same code that computes the value: forward mode over dual numbers, one seeded pass per input, and reverse mode over an expression tape, one forward evaluation plus one backward sweep for every input at once, which is what backpropagation is. The route runs through the accuracy ceiling of finite differences, the dual number algebra, the tape and its adjoint sweep, a 2-layer network with every weight gradient pinned, the cost model that decides which mode wins, and the two-gate verification discipline that keeps all of it honest. Every behavioral claim below is one of the 57 checks in the 3 samples of chapter 12 or a sentence quoted from the mml-book draft 2024-01-15 anchor sheet. Where book 9, ch 36 (practical data structures and algorithms, numerical methods) consumes slopes as driver ingredients, this chapter computes them exactly.

12.1. the accuracy ceiling of finite differences

Chapter 9 differentiated by quotient: perturb, reevaluate, divide. Definition 5.1 of the mml book defines the difference quotient $\frac{f(x+\delta x)-f(x)}{\delta}x$ and its limit as the derivative [printed p 141 / pdf p 147]. Two quotients are standard, the forward one with truncation error $O(h)$ and the central one $\frac{f(x+h)-f(x-h)}{2h}$ with truncation error $O(h^2)$, and both carry a rounding term of order $\varepsilon\ |f\frac{|}{h}$ where $\varepsilon = 2^{-52}$ is the machine precision of book 2, ch 2 (mathematics for programmers, error, conditioning, and stability). Balancing truncation against rounding puts the forward optimum near $\sqrt{\varepsilon} \approx 1.49 \times 10^{-8}$ and the central optimum near $\varepsilon^{\frac{1}{3}} \approx 6.06 \times 10^{-6}$, the cbrt rule derived in book 2, ch 9 (mathematics for programmers, single-variable calculus). Below those step sizes the rounding term dominates and the quotient degrades, above them the truncation term does.

```
double fd = (f(x + h) - f(x)) / h;         // O(h) + eps|f|/h
double cd = (f(x + h) - f(x - h)) / (2*h); // O(h^2) + eps|f|/h
```

fragment, illustrative, not executed

Measured on the fixed function that carries this chapter, the mml book's own eq (5.109) $f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2))$ [printed p 159 / pdf p 165], at $x = 0.5$ where the true slope is -1.3604310785982792:

h	forward error	central error
1e-3	2.18e-3	2.23e-7
1e-4	2.18e-4	2.23e-9
1e-5	2.18e-5	3.07e-11
1e-6	2.18e-6	8.03e-11
1e-8	2.25e-8	3.02e-10
1e-10	6.89e-7	4.22e-7
1e-12	1.86e-4	7.48e-5
1e-15	2.82e-2	2.82e-2
1e-17	1.36e0	1.36e0

The dry run: at $h = 10^{-17}$ the machine cannot represent the perturbed point at all. One ulp of 0.5 is 1.11×10^{-16}, half an ulp is 5.55×10^{-17}, so $0.5 + h$ rounds back to exactly 0.5, both function evaluations return the same double, both quotients return exactly 0, and the error is 1.36, the entire slope. One decade up, at $h = 10^{-8}$, the forward quotient lands 2.25×10^{-8} from the truth while the central quotient at $h = \varepsilon^{\frac{1}{3}}$ lands 1.91×10^{-12} away,

the measured floor on this function, right where the cbrt analysis put the optimum. The two columns fall and rise in a V because the two error terms exchange roles, and no choice of h buys more than roughly 11 correct digits.

PITFALL: h below half an ulp is invisible

at $x = 0.5$ any $h < 5.55 \times 10^{-17}$ rounds $x + h$ back to x, so the quotient is exactly 0 and the reported slope is 0, not a small refinement of the previous estimate. shrinking the step past the representable increment deletes the measurement. the same cliff exists at every scale: near $x = 1000$ the ulp is 1.14×10^{-13} and the central difference is already unreliable at $h = 10^{-7}$.

The alternative to differencing is symbolic manipulation. Applied to (5.109) it produces eq (5.110), a two line closed form whose factored shape $2x(1 + \exp(x^2))\left(\frac{1}{2\sqrt{x^2+\exp(x^2)}} - \sin(x^2 + \exp(x^2))\right)$ took care to derive, and the book's own verdict is that writing out gradients this way "is often impractical since it often results in a very lengthy expression" [printed p 159 / pdf p 165]. Hand derived gradients also rot: change one term of the function and every derivative line must be redone by hand, with nothing but discipline catching the stale ones. What is wanted is a mechanism that differentiates whatever code computes the value, with the derivative correct to the last representable digit.

The play: the best central difference lands 1.91×10^{-12} from the true slope at $h = 6.06 \times 10^{-6}$, the best forward difference lands 2.25×10^{-8} away at $h = 10^{-8}$, and the dual number pass in the next section lands within one part in 10^{12} of the closed form at the cost of one evaluation.

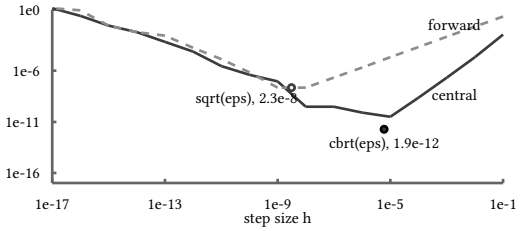


figure 68: measured absolute error of the two difference quotients on f at x = 0.5, log-log axes, h from 1e-1 down to 1e-17

12.2. dual numbers and forward mode

Make the derivative a passenger. A dual number is $a + b\varepsilon$ with $\varepsilon^2 = 0$, an algebra cousin of the complex numbers with a nilpotent unit instead of an imaginary one. Evaluate a function built from addition, multiplication, division and the elementary functions at the seeded pair $(x, 1)$ and the second component of the result is $f'(x)$, because every arithmetic rule propagates tangents by exactly the rules of chapter 9:

$$(u + u'\varepsilon)(v + v'\varepsilon) = uv + (u'v + uv')\varepsilon$$

The ε^2 term is zero by construction, not by neglect, so no information is lost to cancellation and the tangent is exact up to the rounding of the value arithmetic itself. This is forward mode, the grouping $(df/db)((db/da)(da/dx))$ of the mml chain rule (5.121), in which "gradients flow with the data" [printed p 161 / pdf p 167].

The dry run: seed $x = 0.5$ with tangent 1 and walk the intermediate variables of the mml example 5.14, eqs (5.123) to (5.128). Each row carries its value and its tangent, and the tangent column is df/dx at every row.

node	rule	value	tangent
x	seed	0.5	1
$a = x^2$	product	0.25	1
$b = \exp(a)$	chain	1.284025	1.284025
$c = a + b$	sum	1.534025	2.284025
$d = \sqrt{c}$	chain	1.238558	0.922050
$e = \cos(c)$	chain	0.036763	−2.282481

node	rule	value	tangent
$f = d + e$	sum	1.275320	−1.360431

The last row is the payoff: value 1.2753204214157405 and slope −1.3604310785982792 from one pass, the slope matching the sympy pinned closed form of eq (5.110) to 12 digits (check 13 of the sample).

```
[[nodiscard]] static dual d_var(double x) {
  return (dual){.val = x, .dot = 1.0}; // seeded input
}

[[nodiscard]] static dual d_const(double x) {
  return (dual){.val = x, .dot = 0.0}; // unseeded input or constant
}

// sum rule, mml eq (5.31)
[[nodiscard]] static dual dual_add(dual u, dual v) {
  return (dual){.val = u.val + v.val, .dot = u.dot + v.dot};
}

// product rule, mml eq (5.29): (u v)' = u' v + u v'
[[nodiscard]] static dual dual_mul(dual u, dual v) {
  return (dual){.val = u.val * v.val, .dot = u.dot * v.val + u.val * v.dot};
}

// quotient rule, mml eq (5.30): (u/v)' = (u' v - u v') / v^2
[[nodiscard]] static dual dual_div(dual u, dual v) {
  return (dual){.val = u.val / v.val,
                .dot = (u.dot * v.val - u.val * v.dot) / (v.val * v.val)};
}
```

listing 120: dual.c, the seeded constructors and the four arithmetic rules with their mml equation numbers (source: math/samples/src/Ch12/dual.c, lines 40-62)

Seeding is the whole interface. `d_var` turns on the derivative with respect to that input, `d_const` leaves it off, and an unseeded pass returns a tangent lane of exactly 0 (check 15). The function itself is written once, in the intermediate variable order of the book, and it serves both evaluations:

```c
// mml eq (5.109): f(x) = sqrt(x^2 + exp(x^2)) + cos(x^2 + exp(x^2)),
// written exactly as the intermediate variables of example 5.14,
// eqs (5.123)-(5.128)
[[nodiscard]] static dual f_mml(dual x) {
  dual a = dual_mul(x, x); // a = x^2
  dual b = dual_exp(a);    // b = exp(a)
  dual c = dual_add(a, b); // c = a + b
  dual d = dual_sqrt(c);   // d = sqrt(c)
  dual e = dual_cos(c);    // e = cos(c)
  return dual_add(d, e);   // f = d + e
}

// closed-form derivative, mml eq (5.110) second form, for cross-checking
static double df_mml_closed(double x) {
  double a = x * x;
  double b = exp(a);
  double c = a + b;
  return 2.0 * x * (1.0 / (2.0 * sqrt(c)) - sin(c)) * (1.0 + b);
}
```

listing 121: dual.c, the mml function and its closed-form derivative written side by side (source: math/samples/src/Ch12/dual.c, lines 84-102)

One seeded pass produces one column of the jacobian of book 2, ch 10 (mathematics for programmers, multivariate calculus). On the two input function of example 5.7, $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$ at $(2, 3)$, the pass seeded on x_1 returns 39 and the pass seeded on x_2 returns 58, both exact because every intermediate is an integer below 2^{53} (checks 16 to 19). A function of n inputs needs n seeded passes for all n columns, and each pass costs the same as one evaluation of the function plus one lane of arithmetic.

The play: the same function, one pass, one column. Seed x_1, get 39, seed x_2, get 58, and notice the price tag: two full traversals for a two dimensional gradient.

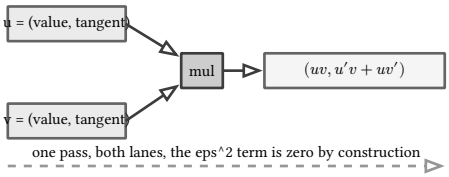


figure 69: the two lanes of one dual multiply, the product rule rides along in the tangent lane

12.3. the tape and reverse mode

Reverse mode keeps the graph instead of the passenger. Evaluate once, recording for every intermediate its value, its parents and the local partials, then sweep adjoints backward. The mml book formalizes the computation graph as $x_i = g_i\left(x_{\mathrm{Pa}(x_i)}\right)$ for the computed nodes (5.143) and the sweep as (5.145):

$$\frac{\partial f}{\partial x_i} = \sum_{x_j : x_i \in \mathrm{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial g_j}{\partial x_i}$$

with the seed $\partial f / \partial x_D = 1$ at the output (5.144). Writing the adjoint of node j as $|x|_j$, the sweep says: when node j is reached, its adjoint is final, and it contributes $|x|_j \times \left(\partial g_j / \partial x_i\right)$ to each parent i. This is the grouping $((df/db)(db/da))(da/dx)$ of (5.120), "gradients backward, opposite to data flow" [printed p 161 / pdf p 167].

The dry run: the same seven nodes as the dual table, values filled left to right, then adjoints swept right to left. The fan-in at c is the whole story of eq (5.135): c feeds both d and e, so its adjoint is a sum of two contributions.

node	adjoint formula	value
f	seed (5.144)	1
d	$\lvert f\rvert \cdot 1$	1
e	$\lvert f\rvert \cdot 1$	1
c	$1 \cdot \frac{1}{2\sqrt{c}} + 1 \cdot (-\sin c)$ (5.139)	−0.595629
b	$\lvert c\rvert \cdot 1$ (5.140)	−0.595629
a	$\lvert b\rvert \cdot \exp(a) + \lvert c\rvert \cdot 1$ (5.141)	−1.360431
x	$\lvert a\rvert \cdot 2x$ (5.142)	−1.360431

The last row matches the closed form of eq (5.110) to 12 digits (check 14 of the sample). Every intermediate gradient, not just the input gradient, is available after the sweep, which is what training diagnostics read.

```
typedef struct {
  double val; // forward value of this node
  double bar; // adjoint df/d(node), filled by the reverse sweep
  double p0;  // local partial d(node)/d(parent 0)
  double p1;  // local partial d(node)/d(parent 1)
  int p0i;    // parent 0 index, -1 when unused
  int p1i;    // parent 1 index, -1 when unused
  enum op op;
} node;

static_assert(sizeof(node) == 48,
              "node packs to 48 bytes: 4 doubles, 2 ints, 1 tag");

static node tape[64];
static int n = 0;
```

listing 122: tape.c, the node: value, adjoint, two local partials, two parent indices, one op tag (source: math/samples/src/Ch12/tape.c, lines 15-29)

```
// the reverse sweep: seed the root with 1, mml eq (5.144), then walk the
// tape backwards accumulating bar_j * d(node_j)/d(node_i) into each
// parent, mml eq (5.145). creation order guarantees every child is
// visited before its parents are consumed.
static void reverse(int root) {
  for (int i = 0; i < n; i++)
    tape[i].bar = 0.0;
  tape[root].bar = 1.0;
  for (int i = root; i ≥ 0; i--) {
    if (tape[i].p0i ≥ 0)
      tape[tape[i].p0i].bar += tape[i].bar * tape[i].p0;
    if (tape[i].p1i ≥ 0)
      tape[tape[i].p1i].bar += tape[i].bar * tape[i].p1;
  }
}
```

listing 123: tape.c, the reverse sweep, mml eq (5.145) as a flat array walk (source: math/samples/src/Ch12/tape.c, lines 92-106)

NOTE: creation order is the sweep schedule

the reverse loop walks indices from the root down, and that is valid because every parent has a smaller index than its children: a node is pushed only after its parents exist. that single invariant lets one flat array replace a pointer graph, since no child contributes to a parent before the child's adjoint is final. push a node whose parent is not on the tape yet and the sweep silently reads a stale bar.

On the two input function $x_1^2 x_2 + x_1 x_2^3$ at $(2, 3)$ the tape needs one forward pass plus one sweep, and both partials, 39 and 58, fall out together (checks 15 to 17 of the sample), where forward mode spent two seeded passes. For a scalar valued function of n inputs this is the difference between n traversals and two.

The play: two columns, two traversals forward, two traversals reverse. The reverse pair grows no matter how many inputs arrive, which is the entire economic argument of the next sections.

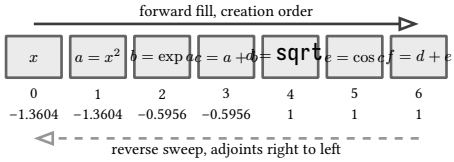


figure 70: the seven-node tape for example 5.14 at x = 0.5, values fill left to right, adjoints sweep right to left

12.4. backpropagation on a layered function

The mml book introduces backpropagation for deep compositions $y = (f_K \circ f_{K-1} \circ \ldots \circ f_1)(x)$ (5.111) built from layers $f^i = \sigma_i(A_{i-1} f^{i-1} + b_{i-1})$ (5.112) under a squared loss $L(\theta) = \left\| y - f^K(\theta, x) \right\|^2$ (5.114), and calls it "an efficient way to compute the gradient of an error function with respect to the parameters of the model" [printed pp 159-161 / pdf pp 165-167]. Section 5.6.2 then folds it into the general theory: "backpropagation is a special case of a general technique in numerical analysis called automatic differentiation" [printed p 161 / pdf p 167]. Read through the tape lens, the layer bookkeeping of eq (5.118) is adjacency reuse: the adjoint of a layer node is computed once and read by every weight that feeds the layer.

The fixed network: input $x = 0.5$, hidden layer $h_1 = \sigma(w_{11} x + b_1)$, $h_2 = \sigma(w_{12} x + b_2)$ with $(w_{11}, b_1, w_{12}, b_2) = (2, -0.5, -1, 0.25)$, output $y = \sigma(w_{21} h_1 + w_{22} h_2 + b_3)$ with $(w_{21}, w_{22}, b_3) = (1, 2, -0.75)$, label $t = 0.5$, loss $L = (y - t)^2$. That is eq (5.112) twice with σ the logistic sigmoid.

The dry run, forward first, all values pinned by sympy:

quantity	expression	value
z_1	$w_{11} x + b_1$	0.5
h_1	$\sigma(z_1)$	0.622459
z_2	$w_{12} x + b_2$	−0.25
h_2	$\sigma(z_2)$	0.437823
u	$w_{21} h_1 + w_{22} h_2 + b_3$	0.748106
y	$\sigma(u)$	0.678766
L	$(y - t)^2$	0.031957

Then backward, each row one application of the sweep at a node:

gradient	rule	value
$\partial L / \partial y$	$2(y - t)$	0.357532
$\partial L / \partial u$	$\lvert y \rvert \cdot y(1 - y)$	0.077957
$\partial L / \partial w_{21}$	$\lvert u \rvert \cdot h_1$	0.048525
$\partial L / \partial w_{22}$	$\lvert u \rvert \cdot h_2$	0.034132
$\partial L / \partial b_3$	$\lvert u \rvert \cdot 1$	0.077957
$\partial L / \partial h_1$	$\lvert u \rvert \cdot w_{21}$	0.077957
$\partial L / \partial h_2$	$\lvert u \rvert \cdot w_{22}$	0.155914
$\partial L / \partial w_{11}$	$\lvert h_1 \rvert \cdot h_1(1 - h_1) \cdot x$	0.009160
$\partial L / \partial b_1$	$\lvert h_1 \rvert \cdot h_1(1 - h_1)$	0.018320
$\partial L / \partial w_{12}$	$\lvert h_2 \rvert \cdot h_2(1 - h_2) \cdot x$	0.019188

gradient	rule	value
$\partial L/\partial b_2$	$\lvert h_2\rvert \cdot h_2(1-h_2)$	0.038376
$\partial L/\partial t$	$-\lvert y\rvert$	−0.357532

All 12 rows come out of one forward pass and one sweep, each pinned to 12 digits by checks 5 to 12 of the sample. The sigmoid's local partial $s(1-s)$ is computed once on the forward pass and stored in the node:

```
static int push_sig(int i) {
  ops_fwd++;
  double s = 1.0 / (1.0 + exp(-tape[i].val));
  tape[n] = (node){.val = s, .p0 = s * (1.0 - s), .p0i = i, .op = OP_SIG};
  return n++;
}
```

listing 124: backprop.c, the sigmoid node carries its own local partial for the sweep (source: math/samples/src/Ch12/backprop.c, lines 85-90)

```
int main(void) {
  // the tape: 9 input leaves (x, 7 weights, label t), 13 computed nodes
  int xi = push(0.5);
  int w11 = push(2.0), b1 = push(-0.5), w12 = push(-1.0), b2 = push(0.25);
  int w21 = push(1.0), w22 = push(2.0), b3 = push(-0.75);
  int ti = push(T);
  int m1 = push_mul(w11, xi);
  int z1 = push_add(m1, b1);
  int h1 = push_sig(z1);
  int m2 = push_mul(w12, xi);
  int z2 = push_add(m2, b2);
  int h2 = push_sig(z2);
  int m21 = push_mul(w21, h1);
  int m22 = push_mul(w22, h2);
  int s = push_add(m21, m22);
  int u = push_add(s, b3);
  int y = push_sig(u);
  int r = push_sub(y, ti);
  int L = push_mul(r, r);
  reverse(L);
```

listing 125: backprop.c, the network as 9 input leaves and 13 computed nodes, then one sweep (source: math/samples/src/Ch12/backprop.c, lines 160-179)

The play: one sweep hands over all 12 gradients at once. The largest in magnitude is $\partial L/\partial t = -0.3575$ and the smallest is $\partial L/\partial w_{11} = 0.009160$, a 39 to 1 spread that book 2, ch 11 (mathematics for programmers, continuous optimization) would otherwise pay 12 inaccurate finite difference evaluations to feel.

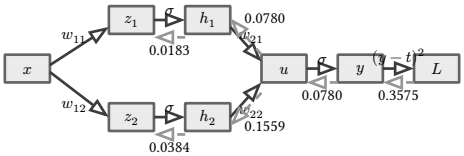


figure 71: forward pass solid, adjoint sweep dashed, every number measured in backprop.c at the pinned weights

12.5. the cost model: columns versus sweeps

Forward mode prices by the number of seeded directions, reverse mode prices at one forward plus one sweep regardless of input count. The mml book states the consequence for learning systems: "In the context of neural

networks, where the input dimensionality is often much higher than the dimensionality of the labels, the reverse mode is computationally significantly cheaper than the forward mode" [printed p 161 / pdf p 167].

The dry run: the fixture is the 13-node network of the previous section and its 7 weights, and the counters come out 91 elementary evaluations for forward mode against 26 for reverse.

The network of the previous section is a 13-node graph, and the counts are measured by the sample, not asserted. Forward mode needs one seeded pass per weight: 7 weights times 13 nodes is 91 elementary evaluations. Reverse mode needs 13 forward plus 13 sweep visits, 26 in total. The two curves tie at exactly 2 seeded inputs and reverse wins everywhere beyond, while forward mode keeps an advantage the plot does not show: it stores no graph, so its memory is $O(1)$ extra per intermediate.

```
// traversal counts, measured: 13 computed nodes forward, 13 visited by
// the sweep, and forward mode needs 7 seeded passes = 91 node
// evaluations against 13 + 13 = 26 for reverse
CHECK(ops_fwd == 13, "forward pass computes 13 nodes");
CHECK(ops_rev == 13, "reverse sweep visits 13 nodes");
for (int seed = 0; seed < 7; seed++)
  eval_dual(seed); // each seeded pass counts 13 elementary ops
CHECK(ops_dual == 91, "forward mode costs 7 passes x 13 nodes = 91");
CHECK(ops_fwd + ops_rev == 26, "reverse mode costs 13 + 13 = 26");
```

listing 126: backprop.c, the measured traversal counts for both modes on the same graph (source: math/samples/src/Ch12/backprop.c, lines 199-207)

WARNING: the tape is memory, not magic

reverse mode buys 26 versus 91 by remembering every intermediate, 48 bytes per node in this chapter's tapes. a 10-layer network over a million activations holds hundreds of megabytes of nodes and adjoints. production frameworks checkpoint layer boundaries and recompute the forward pass in chunks during the sweep, trading a bounded factor of time for memory that no longer grows with depth.

The rule of thumb falls out of the same counting. A scalar loss with many parameters is the learning shape and belongs to reverse mode. A function of few inputs, or a request for directional derivatives along a handful of directions, belongs to forward mode, which also vectorizes naturally when one pass carries a vector of tangents.

The play: 91 versus 26 elementary evaluations for the same 7 gradients, and the tie point sits at 2 inputs, which is exactly where the two-input function of the earlier sections sat when forward mode spent 2 passes and reverse spent 2 traversals.

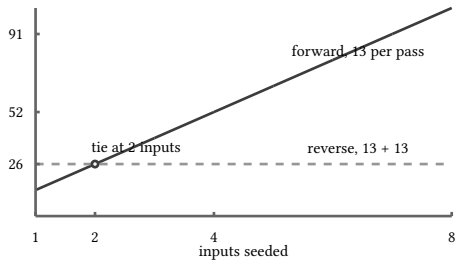


figure 72: elementary evaluations against the number of seeded inputs, forward grows as 13 per input, reverse stays flat at 26

12.6. verification discipline

An autodiff implementation is a small interpreter for the chain rule, and interpreters get wiring bugs: a dropped parent edge, a missed fan-in accumulation, a stale adjoint read. The mml book prescribes the gradient check in a remark in section 5.2 [printed p 149 / pdf p 155]: compare the analytic gradient df_i against the finite-difference approximation dh_i from (5.39) at a small h, the book's example being $h = 10^{-4}$, componentwise through the criterion

$$\sqrt{\frac{\sum_i (dh_i - df_i)^2}{\sum_i (dh_i + df_i)^2}} < 10^{-6}$$

The loose gate catches structural errors, which typically shift gradients by whole factors. It cannot catch drift, because the central difference itself is only good to about 10^{-9} on this network, per the ceiling of the first section. So this chapter pins every gradient twice, and the samples enforce both gates on every number: the sympy exact value at relative 10^{-12} and the mml criterion against central differences.

The dry run: the mml criterion over all 7 weights at $h = 10^{-4}$ measures 1.02×10^{-9} on the fixed network, three decades inside the 10^{-6} gate (check 18). The one-weight spelling puts the central difference at 0.0091601191772 against the pinned 0.0091601191794 (check 19).

```
// verification discipline, mml remark in section 5.2: central
// differences at h = 1e-4 against the analytic gradient, criterion
// sqrt(sum (dh - df)^2 / sum (dh + df)^2) < 1e-6. sympy measures
// 1.02e-9 for this network, well inside the gate
double df[7] = {tape[w11].bar, tape[b1].bar,  tape[w12].bar, tape[b2].bar,
                tape[w21].bar, tape[w22].bar, tape[b3].bar};
double num = 0.0, den = 0.0;
for (int i = 0; i < 7; i++) {
  double w0[7] = {2.0, -0.5, -1.0, 0.25, 1.0, 2.0, -0.75};
  double dh =
      (eval_plain(i, w0[i] + HFD) - eval_plain(i, w0[i] - HFD)) / (2 * HFD);
  num += (dh - df[i]) * (dh - df[i]);
  den += (dh + df[i]) * (dh + df[i]);
}
double crit = sqrt(num / den);
CHECK(crit < 1e-6 && crit > 1e-12,
      "central-difference criterion 1.0e-9 < 1e-6");
```

listing 127: backprop.c, the mml criterion computed over all 7 weights at h = 1e-4 (source: math/samples/src/Ch12/backprop.c, lines 213-229)

The exact pins run at 10^{-12} relative throughout, which finite differences can never meet at any step size.

VERIFY: pin the gradient twice

every gradient in this chapter answers to two independent authorities: the central difference through the mml section 5.2 criterion below 10^{-6}, which catches wiring bugs, and the sympy pinned exact value at relative 10^{-12}, which catches drift and regressions. a gradient that passes only the loose gate can still be wrong in its fourth digit, and one that passes only the tight gate may be pinned to a formula that no longer matches the code.

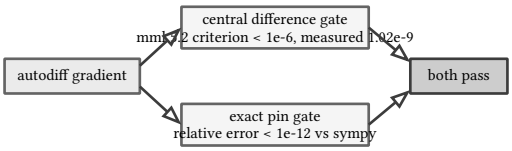


figure 73: the two-gate rule, loose agreement with differences and tight agreement with the pinned exact value

This discipline is the contract the rest of the book leans on. The capstone planner's cost model needs gradients the moment its weights become tunable, and the machinery drops in unchanged: the objective is just another tape, forward once, sweep once, pin every number twice. The continuous half of the book ends here. book 2, ch 13 (mathematics for programmers, logic, sets, and proof) opens the discrete half with the proof vocabulary these pinned checks have been using informally.

sources: mml-book draft 2024-01-15, ch 5, sec 5.6 backpropagation and automatic differentiation [printed pp 159-164 / pdf pp 165-170] including eqs (5.109)-(5.145) and example 5.14, sec 5.5 gradient identities [printed pp 158-159 / pdf pp 164-165], the gradient-check remark of sec 5.2 [printed p 149 / pdf p 155], and eqs (5.29)-(5.32), (5.110), (5.123)-(5.128), (5.135)-(5.142) as cited, all read from the verified anchor sheet ref/mml/anchors-calc.md. Griewank and Walther, Evaluating Derivatives, 2nd ed, cited by name via mml sec 5.9 further reading. Expected values computed with sympy in the one-off playground/math-ch12/gen.py. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch12`, 57 checks in chapter 12 of the math suite, probed on this machine 2026-09-21.

13. logic, sets, and proof

The discrete half of this book starts here: propositions as bit patterns a machine can exhaust, quantifiers as bounded loops that return evidence, sets and relations as masks and boolean matrices, functions with injections and the pigeonhole principle, induction written as a loop invariant, invariants that must survive mutation, and proofs as step tables a checker recomputes. Every behavioral claim below is one of the 52 checks in the 5 samples of chapter 13 or a sentence quoted from a canonical source fetched 2026-09-21. The dsa book's book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) chapter measures what algorithms cost; this chapter builds the proof patterns that establish what a program is true of, with counting left to book 9, ch 35 (practical data structures and algorithms, combinatorics) and to the next chapter.

The one idea that carries all seven sections: a finite mathematical claim is data. A proposition over 3 variables is 8 bits, a subset of a fixed universe is one integer, a relation on 5 points is 5 integers, a function between finite sets is a short array. Once the claim is data, checking it is a loop, and a proof by exhaustion is a mask comparison.

13.1. propositions as bit patterns

A proposition is a claim with a truth value. Connectives build compound propositions: $\neg p$, $p \wedge q$, $p \vee q$, $p \oplus q$, implication $p \to q$, biconditional $p \leftrightarrow q$. Over k variables the truth table has 2^k rows, and stacking the output column of row i into bit i of an integer turns the whole table into one machine word. Variable j reads bit $(k - 1 - j)$ of the row index: with (p, q, r) and $k = 3$, row 5 is 101, so p holds and r holds.

Every connective becomes one bit operation on the whole vector at once. With `p = 0xF0`, `q = 0xCC`, `r = 0xAA` over the 8-row universe:

connective	vector form	mask	true rows
$\neg p$	`~p & m`	`0x0F`	4
$p \wedge q$	`p & q`	`0xC0`	2
$p \vee q$	`p | q`	`0xFC`	6
$p \oplus q$	`p ^ q`	`0x3C`	4
$p \to q$	`(~p | q) & m`	`0xCF`	6
$p \leftrightarrow q$	`~(p ^ q)`	`0xC3`	4

Implication is the one worth a second look: $p \to q$ is $\neg p \vee q$, false in exactly the rows where the antecedent holds and the consequent does not. Over two variables the vector is `0xB`: true in 3 rows of 4, false only at $(p, q) = (1, 0)$. That single false row is why $\oplus$ and $\to$ are different connectives even though both feel like "differs".

A tautology is now a mask comparison: the vector is all ones over the universe, $vm == m$. A contradiction is the zero vector. De Morgan $\neg(p \wedge q) \leftrightarrow \neg(p \vee q)$, distributivity, the contrapositive $(p \to q) \leftrightarrow (\neg q \to \neg p)$, and modus ponens each collapse to eight-bit checks, which is what proof by exhaustion looks like when the universe is 8 rows.

```
int main(void) {
  // base vectors over the 8-row universe, row i with p = i>>2, q = i>>1,
  // r = i&1. expected (gen.py): p 0xf0, q 0xcc, r 0xaa, ~p 0x0f
  unsigned p = 0xF0, q = 0xCC, r = 0xAA;
  CHECK(p == 0xF0 && q == 0xCC && r == 0xAA && (~p & M) == 0x0F,
        "base truth vectors p q r not-p");

  // connectives are single bit ops (gen.py: PandQ 0xc0 PorQ 0xfc PxorQ 0x3c)
  CHECK((p & q) == 0xC0 && (p | q) == 0xFC && (p ^ q) == 0x3C,
        "and or xor as bit ops");
  // p -> q = ~p | q = 0xcf, biconditional = ~(p ^ q) = 0xc3 (gen.py)
  CHECK(imp(p, q, M) == 0xCF && agrees(~(p ^ q), 0xC3, M),
        "implication and biconditional masks");
```

listing 128: truthvec.c, base vectors and the connective masks over the 8-row universe (source: math/samples/src/Ch13/truthvec.c, lines 50-62)

```
  // proof by exhaustion: a named law is checked against all 8 rows at once.
  // gen.py laws: every one of these vectors is all-ones over its universe
  CHECK(agrees(imp(p & imp(p, q, M), q, M), M, M),
        "modus ponens (p and p->q) -> q is a tautology");
  CHECK(agrees(imp(p, q, M), imp(~q, ~p, M), M),
        "contrapositive: p->q equals ~q->~p");
  CHECK(agrees(~(p & q), ~p | ~q, M), "de morgan: ~(p and q) iff ~p or ~q");
  CHECK(agrees(p & (q | r), (p & q) | (p & r), M),
        "distributivity: p and (q or r) iff (p and q) or (p and r)");
  CHECK(agrees(p | ~p, M, M) && (p & ~p & M) == 0,
        "excluded middle tautology, contradiction zero");
```

listing 129: truthvec.c, named laws checked against every row at once (source: math/samples/src/Ch13/truthvec.c, lines 64-74)

The dry run: the implication column below is the mask `0xCF` built row by row. Row 7 is $(1, 1, 1)$, antecedent true and consequent true, so the row reads true. Row 5 is $(1, 0, 1)$: p holds, q fails, so the row is false, and row 4 $(1, 0, 0)$ is the other false row. Six true rows, two false rows, matching `popcount(0xCF) = 6`.

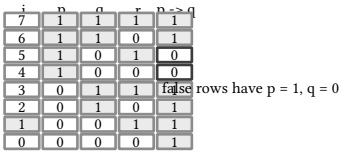


figure 74: truth vector of implication p -> q over the 8-row universe, false only where p holds and q fails

PITFALL: implication is not xor

$p \to q$ fails only at $(p, q) = (1, 0)$, two false rows out of 8 with r free, while $p \oplus q$ fails in 4. treating "differs" as "implies" in a bitmask flag test inverts exactly the rows where both flags are set. the census in truthvec.c is sharper still: of the 16 binary connectives, 1 is a tautology, 1 a contradiction, 4 ignore one variable (p, q, $\neg p$, $\neg q$), and only 10 depend on both.

13.2. quantifiers as bounded loops

The quantifiers $\forall x, P(x)$ and $\exists x, P(x)$ range over a domain. On a fixed finite domain each is a loop, and the honest loop returns evidence with its bit: a failing $\forall$ returns the first counterexample, a passing $\exists$ returns the first witness. A claim and its negation are then the same loop read two ways, the duality $\neg(\forall x, P(x)) \leftrightarrow \exists x, \neg P(x)$.

The workhorse example: over the domain 1 to 100, does $x^2 > 50$ hold for all x? The $\forall$ scan fails immediately at $x = 1$, the dual $\exists$ scan stops at the first witness $x = 8$, and the census says 93 of 100 rows satisfy the predicate. The boundary pair is the two-number play: $7^2 = 49$ misses, $8^2 = 64$ clears, so the witness is 8.

```
// quantifiers as bounded loops over the fixed domain 1..100
constexpr int N = 100;
int dom[N];
for (int i = 0; i < N; i++) {
  dom[i] = i + 1;
}

// squares over 50: the for-all fails with counterexample x = 1, exactly
// 93 of 100 rows satisfy, the dual exists finds first witness 8
// (7*7 = 49 ≤ 50 < 64 = 8*8) (gen.py quantifiers)
int sq_true = 0, sq_first_false = 0, sq_first_true = 0;
for (int i = 0; i < N; i++) {
  if (dom[i] * dom[i] > 50) {
    sq_true++;
    if (sq_first_true == 0) {
      sq_first_true = dom[i];
    }
  } else if (sq_first_false == 0) {
    sq_first_false = dom[i];
  }
}
CHECK(sq_true == 93 && sq_first_false == 1 && sq_first_true == 8,
      "forall x^2 > 50 fails, exists witness 8");
// de morgan for quantifiers: not forall P iff exists not P
CHECK((sq_true != N) && (sq_first_false > 0),
      "not-forall and exists-not agree");
```

listing 130: truthvec.c, a for-all that fails with counterexample 1 and its dual exists with witness 8 (source: math/samples/src/Ch13/truthvec.c, lines 105-130)

Quantifier order is not free. Over Z_4: $\forall x \exists y, y = x + 1$ holds, because the witness y may depend on x. Swap the prefixes and $\exists y \forall x, y = x + 1$ fails: every fixed y candidate is broken by some x, and the sample pins the first breaker of each, x values 0, 1, 0, 0 for $y = 0, 1, 2, 3$. Dependent choice is exactly what the inner loop of a nested scan expresses.

The strongest loop in the file is a bounded Goldbach check: every even n from 4 to 100 is a sum of two primes. The outer $\forall$ spans 49 values of n, and the inner $\exists$ carries the smallest witness pair, $4 = 2 + 2$, $6 = 3 + 3$, up to $98 = 19 + 79$.

```
// bounded goldbach: every even n in 4..100 is a sum of two primes, and
// the for-all carries the smallest witness pair per n. gen.py: 49
// values, witnesses 4 = 2+2, 6 = 3+3, 98 = 19+79
int all_gold = 1, w4 = 0, w98 = 0, count_even = 0;
for (int n = 4; n ≤ 100; n += 2) {
  count_even++;
  int hit = 0;
  for (int a = 2; a ≤ n && hit == 0; a++) {
    if (is_prime(a) && is_prime(n - a)) {
      hit = a;
    }
  }
  if (n == 4) {
    w4 = hit;
  }
  if (n == 98) {
    w98 = hit;
  }
  if (hit == 0) {
    all_gold = 0;
  }
}
CHECK(all_gold == 1 && count_even == 49 && w4 == 2 && w98 == 19,
      "bounded goldbach for-all with witness pairs");
```

listing 131: truthvec.c, a bounded for-all whose every instance carries a witness pair (source: math/samples/src/Ch13/truthvec.c, lines 171-194)

The dry run: for $n = 98$ the inner scan tries $a = 2$ ($98 - 2 = 96$, not prime), walks up through $a = 19$ where 79 is prime, and records $19 + 79$. No even n in range reaches the end of the scan without a hit, so the $\forall$ holds with 49 witnesses in hand.

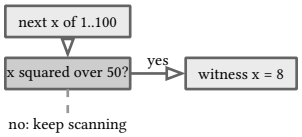


figure 75: the exists scan with early exit: the first row that satisfies the predicate becomes the witness

VERIFY: bounded is not general

the goldbach check proves the statement for 4 to 100 and says nothing about the unbounded conjecture, which stays open. a for-all loop over a finite domain is a proof of exactly that domain, no wider. when this chapter says every, the universe of the every is the domain array in the sample.

13.3. sets as bitmasks and relations as matrices

Fix the universe $\{\{0, .., 7\}\}$. A set is one `uint8_t`, element e is bit e. Union is `|`, intersection is `&`, complement is xor with the universe mask, difference is `a & ~b`, and $A \subseteq B$ is the single expression `(a & ~b) == 0`. Cardinality counts set bits. With $A = \{\{0, 2, 4, 6\}\}$ as mask `0x55` and $B = \{\{1, 2, 3\}\}$ as mask `0x0E`: $|A| = 4$, $|B| = 3$, $|A \cup B| = 6$, $|A \cap B| = 1$, and inclusion-exclusion holds as $4 + 3 - 1 = 6$.

The power set of an 8-element universe is every mask from 0 to 255, and two census facts fall out of one 256-iteration loop: exactly 128 masks have even cardinality, and the sum of all cardinalities is $8 \cdot 2^7 = 1024$, because each element appears in exactly half the masks. Counting the same total two ways is the set-theoretic version of the two-number play.

```
// fixed sets, element e is bit e (gen.py sets):
// A = {0,2,4,6} = 0x55, B = {1,2,3} = 0x0e
uint8_t a = 0x55, b = 0x0E;

// gen.py: |A| 4, |B| 3, |A u B| 6, |A n B| 1, and the
// inclusion-exclusion identity |A| + |B| - |A n B| = |A u B| holds
CHECK(card(a) == 4 && card(b) == 3 && card(a | b) == 6 && card(a & b) == 1 &&
          card(a) + card(b) - card(a & b) == card(a | b),
      "union intersection cardinalities, inclusion-exclusion");
// gen.py: A - B = 0x51, A delta B = 0x5b with |A delta B| 5 = 4 + 3 - 2,
// complement of B in the universe = 0xf1
CHECK((a & ~b) == 0x51 && (a ^ b) == 0x5B && card(a ^ b) == 5 &&
          (uint8_t)~b == 0xF1,
      "difference symmetric-difference complement");
// subset is one and-not: {2,6} = 0x44 is inside A, {0,3} = 0x09 is not
CHECK(subset(0x44, a) && !subset(0x09, a), "subset test as a and-not");
// de morgan for sets: ~(A u B) = ~A n ~B = 0xa0 (gen.py)
auto both = (uint8_t)(U8 ^ (a | b)); // typeof would spell it uint8_t
CHECK(both == 0xA0 && both == (uint8_t)((U8 ^ a) & (U8 ^ b)),
      "de morgan for sets");
```

listing 132: sets.c, the set algebra identities as bit operations with pinned masks (source: math/samples/src/Ch13/sets.c, lines 111-130)

```
// the power set of the 8-element universe is every mask 0..255: gen.py
// says 256 masks, 128 of even cardinality, and the sum of all
// cardinalities is 1024 = 8 * 2^7, each element in half the masks
int even = 0, total = 0;
for (int m = 0; m < 256; m++) {
  uint8_t s = (uint8_t)m;
  even += (card(s) % 2 == 0);
  total += card(s);
}
CHECK(even == 128 && total == 1024 && total == 8 * (1 << 7),
      "power set census: half even, element in half the masks");
```

listing 133: sets.c, power set census, each element in half of the 256 masks (source: math/samples/src/Ch13/sets.c, lines 132-142)

A binary relation on $\{0,..,4\}$ is a boolean matrix, stored as one mask per row: aRb is bit b of row a. The four defining properties are short loops over the matrix:

property	statement	loop shape
reflexive	$\forall a : aRa$	own bit set in every row
symmetric	$\forall a, b : aRb \leftrightarrow bRa$	matrix equals its transpose
antisymmetric	$aRb \wedge bRa \rightarrow a = b$	no mirrored pair off the diagonal
transitive	$aRb \wedge bRc \rightarrow aRc$	triple loop, fail on any gap

An equivalence relation is reflexive, symmetric, and transitive, and its classes partition the universe. The sample's R_1 has rows `07 07 07 18 18`: classes $\{\{0,1,2\}\}$ and $\{\{3,4\}\}$, 13 pairs total, and the class scan marks both classes while covering all 5 points exactly once. The order R_2 with $b \geq a$ is reflexive, antisymmetric, and transitive but not symmetric: a partial order, 15 pairs. And R_3 = $\{\{(0,1),(1,2)\}\}$ is the warning case: antisymmetric yet not transitive, since $0R1$ and $1R2$ hold while $0R2$ does not.

Warshall's algorithm repairs exactly that gap: for each intermediate k, every row that reaches k also takes in row k's neighborhood. On R_3 the closure adds the single missing pair $(0,2)$, pair count 2 to 3, and the result is transitive. On R_1 the closure changes nothing: transitive relations are already closed.

```
// relations on 5 points as boolean matrices, one row mask per point.
// R1: classes {0,1,2} and {3,4} (gen.py rows 07 07 07 18 18)
uint8_t r1[P] = {0x07, 0x07, 0x07, 0x18, 0x18};
// gen.py R1: refl 1, sym 1, antisym 0, trans 1: an equivalence relation
CHECK(refl(r1) && sym(r1) && trans(r1) && !antisym(r1),
      "R1 is an equivalence relation");
// gen.py: 13 pairs (9 + 4), 2 classes, class of 0 is {0,1,2} = 0x07
uint8_t seen = 0;
int classes = 0;
uint8_t class0 = 0;
for (int i = 0; i < P; i++) {
  if (!(seen >> i & 1)) {
    classes++;
    if (i == 0) {
      class0 = r1[i];
    }
    seen |= r1[i];
  }
}
CHECK(pairs(r1) == 13 && classes == 2 && class0 == 0x07 && seen == UP,
      "equivalence classes partition the universe");
```

listing 134: sets.c, equivalence classes partition the 5-point universe (source: math/samples/src/Ch13/sets.c, lines 144-164)

```
// R2: the order b ≥ a (gen.py rows 1f 1e 1c 18 10): reflexive,
// antisymmetric, transitive, not symmetric: a partial order
uint8_t r2[P] = {0x1F, 0x1E, 0x1C, 0x18, 0x10};
CHECK(refl(r2) && antisym(r2) && trans(r2) && !sym(r2) && pairs(r2) == 15,
      "R2 is a partial order, 15 pairs");

// R3: {(0,1), (1,2)}: antisymmetric but not transitive (0R1, 1R2, and
// 0R2 is missing), so antisymmetry does not imply transitivity
uint8_t r3[P] = {0x02, 0x04, 0x00, 0x00, 0x00};
CHECK(antisym(r3) && !trans(r3) && pairs(r3) == 2,
      "R3 antisymmetric yet not transitive");

// warshall adds exactly the pair (0,2): gen.py closure rows
// {0x06, 0x04, 0, 0, 0}, pair count 2 -> 3, closure is transitive
typeof(r3) c3 = {0};
for (int i = 0; i < P; i++) {
  c3[i] = r3[i];
}
closure(c3);
CHECK(c3[0] == 0x06 && c3[1] == 0x04 && c3[2] == 0 && c3[3] == 0 &&
          c3[4] == 0 && pairs(c3) == 3 && trans(c3),
      "warshall closure adds the missing pair");
```

listing 135: sets.c, partial order, the non-transitive warning case, and the warshall closure (source: math/samples/src/Ch13/sets.c, lines 166-187)

The dry run: the closure of R_3 walks intermediates $k = 0, 1, 2, 3, 4$. At $k = 1$, row 0 has bit 1 set, so row 0 takes in row 1's mask `0x04`, becoming `0x06`: the pair $(0, 2)$ appears. No other row reaches anything new, so the loop ends with 3 pairs, transitive by exhaustion.

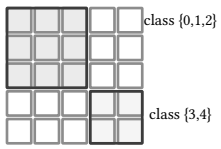


figure 76: the equivalence relation as a boolean matrix: two shaded blocks on the diagonal are the two classes

13.4. functions and the pigeonhole principle

A function f from $\{0, .., n-1\}$ to $\{0, .., m-1\}$ is an array with $f[i] < m$. It is injective when no two inputs share an output, a scan over all pairs. It is surjective when the image has size m, a census over outputs. Bijective is both, and a bijection on a finite set has a two-sided inverse: composing with the inverse array yields the identity.

Composition $(f \circ g)[i] = f[g[i]]$ is associative, which the chapter states as prose and the sample checks on three fixed maps, $(f \circ g) \circ h = f \circ (g \circ h)$ entry by entry on all 6 points.

The pigeonhole principle is the first counting theorem with teeth: a map from $n+1$ pigeons to n holes cannot be injective, because $n+1$ outputs among n holes force a repeat. Stated constructively it is an algorithm: scan pairs, return the first collision. For the fixed map {1, 3, 4, 4, 5, 0, 0} from 7 pigeons to 6 holes, the scan returns the pair $(2, 3)$, both landing in hole 4, and the image misses hole 2: not injective and not surjective in one pass.

```
int main(void) {
  // f7: a fixed map from 7 pigeons into 6 holes (gen.py f7):
  // values {1,3,4,4,5,0,0}, image {0,1,3,4,5} of size 5: neither
  // injective (4 repeats at 2,3 and 0 repeats at 5,6) nor surjective
  // onto {0..5} (2 is missed)
  int f7[7] = {1, 3, 4, 4, 5, 0, 0};
  CHECK(!injective(f7, 7) && image_size(f7, 7, 6) == 5,
        "7 into 6: not injective, image misses a hole");
  // gen.py colliding pairs: (2,3) and (5,6); the scan returns the first
  const int *w = nullptr; // witness: left element of the first collision
  int wj = -1;
  for (int i = 0; i < 7 && w == nullptr; i++) {
    for (int j = i + 1; j < 7; j++) {
      if (f7[i] == f7[j]) {
        w = &f7[i];
        wj = j;
        break;
      }
    }
  }
  CHECK(w != nullptr && *w == 4 && wj == 3 && w - f7 == 2,
        "pigeonhole witness pair (2,3)");
```

listing 136: functions.c, the 7 into 6 map with the collision witness found by scan (source: math/samples/src/Ch13/functions.c, lines 73-94)

The dry run: the collision scan walks pairs in index order. It clears the 6 pairs that start at pigeon 0 and the 5 that start at pigeon 1, then stops on its 12th pair $(2, 3)$: both entries read 4, the witness is in hand after 12 comparisons out of the 21 a full triangle would cost.

The average form is just as useful: 10 pigeons into 3 holes force some hole to hold at least $\lceil 10/3 \rceil = 4$. The sample's fixed load array {0, 1, 2, 0, 1, 2, 0, 1, 2, 0} has loads 4, 3, 3, the busiest hole exactly at the ceiling, and the bound itself is a compile-time `static_assert`.

Counting functions is where exact integers start to matter. There are m^n maps from an n-set to an m-set, so the set of maps from a 20-set to a 20-set has 20^{20} elements, a 27-digit number that needs 87 bits. `uint64_t` holds 64, so it wraps to
1634530577365755 4944. An `unsigned _BitInt(128)` holds the exact value,

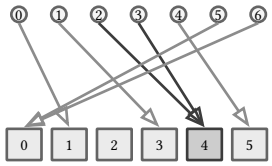
and the sample computes it by 20 multiplications and compares against the digit string parsed back in. The two numbers side by side are the point of the section: 87 bits exact versus a 64-bit wrap that looks plausible and is wrong.

```
// exact counting. 20^20 = 104857600000000000000000000 needs 87 bits,
// past the 64 of uint64_t (gen.py: exact value, bit length 87, wrapped
// 16345305773657554944)
unsigned _BitInt(128) val = 1;
for (int i = 0; i < 20; i++) {
  val *= 20;
}
CHECK(val == parse128("104857600000000000000000000"),
      "20^20 exactly in _BitInt(128)");
int bits = 0;
for (unsigned _BitInt(128) t = val; t != 0; t >>= 1) {
  bits++;
}
CHECK(bits == 87, "20^20 needs 87 bits, past uint64");
// the uint64 wrap is the same value mod 2^64 (gen.py mod 2^64): mod a
// power of two is a mask, and a mask avoids the runtime divide helper
unsigned _BitInt(128) low = val & (((unsigned _BitInt(128))1 << 64) - 1);
uint64_t wrapped_fixed = 16345305773657554944ULL; // pinned by gen.py
CHECK((uint64_t)low == wrapped_fixed,
      "uint64 arithmetic wraps to the value mod 2^64");

// the falling product 20 * 19 * ... * 11 counts the injections of a
// 10-set into a 20-set: 670442572800 = C(20,10) * 10!, still 64-bit
unsigned long long inj = 1;
for (int k = 0; k < 10; k++) {
  inj *= (unsigned long long)(20 - k);
}
CHECK(inj == 670442572800ULL, "injections of 10 into 20: 20!/10!");
```

listing 137: functions.c, exact 20^20 in a 128-bit integer and the falling product for injections (source: math/samples/src/Ch13/functions.c, lines 143-170)

The falling product $20 \cdot 19 \cdot \ldots \cdot 11 = 20!/10! = 670442572800$ counts the injections of a 10-set into a 20-set, and it still fits in 64 bits: the boundary between counting that fits and counting that does not sits inside one sample file.



f(2) = f(3) = 4, the scan witness

figure 77: seven pigeons into six holes: the scan witness is the pair sharing hole 4

WARNING: 128-bit remainder with a runtime divisor

on this toolchain `%` on `_BitInt(128)` with a divisor known only at runtime lowers to a compiler-rt `__umodti3` call that the msvc-linked build cannot resolve. a compile-time constant divisor is different: the compiler expands it to inline multiply-shift and the link stays clean. reducing a power of two is a mask either way: `v & ((1 << 64) - 1)` gives the low 64 bits with native instructions, and that is what the sample does. probed 2026-09-21

> with the pinned clang 23: the runtime-divisor object carries an undefined `__umodti3`, the constant-divisor one does not.

13.5. induction as a loop invariant

Induction proves a claim $P(n)$ for all $n \geq 0$ from two steps: the base case $P(0)$, and the inductive step $P(k) \to P(k+1)$. The program twin is exact. A loop that has finished k iterations holds some state satisfying an invariant, one more iteration preserves the invariant, and when the loop exits at $k = n$ the invariant is the conclusion $P(n)$. The `assert` at the loop top is the induction hypothesis, checked at every rung instead of assumed.

The canonical run: $1 + 2 + .. + n = n(n+1)/2$. The loop adds k then asserts the closed form $k(k+1)/2$ holds of the running sum, 100 times for one answer, $S(100) = 5050$, and 1000 times for $S(1000) = 500500$. The odd-number identity $1 + 3 + .. + (2n-1) = n^2$ gets the same treatment and lands on $20^2 = 400$.

```c
int main(void) {
  // gauss: sum 1..n = n(n+1)/2 by a loop that asserts the closed form at
  // the top of every iteration (gen.py: S(100) = 5050, S(1000) = 500500)
  int sum = 0; // invariant: sum == k*(k+1)/2 after k iterations
  for (int k = 1; k ≤ 100; k++) {
    sum += k;
    assert(sum == k * (k + 1) / 2); // inductive step preserved the claim
  }
  CHECK(sum == 5050, "sum 1..100 by invariant-checked loop");
  int sum1000 = 0;
  for (int k = 1; k ≤ 1000; k++) {
    sum1000 += k;
    assert(sum1000 == k * (k + 1) / 2);
  }
  CHECK(sum1000 == 500500, "sum 1..1000");
```

listing 138: induction.c, the gauss loop asserting the closed form at every rung (source: math/samples/src/Ch13/induction.c, lines 82-96)

C23 moves the base case to compile time: `static_assert` with a message checks a constant claim before the program runs, and since C23 the message is optional and the spelling without underscore is the non-deprecated one, per the cppreference page fetched 2026-09-21.

```c
static_assert(0 * 101 == 0, "base case n = 0: S(0) = 0 and S(1) = S(0) + 1");
static_assert((1u ≪ 11) - 1 == 2047, "geometric base: 2^11 - 1 = 2047");
```

listing 139: induction.c, compile-time base cases as static_assert with messages (source: math/samples/src/Ch13/induction.c, lines 27-28)

The other inductive staples ride the same pattern. The geometric sum $1 + 2 + 4 + .. + 2^{10} = 2^{11} - 1 = 2047$ is asserted per exponent. The divisibility claim $3 \mid 4^n - 1$ is an invariant on a running power: $v = 4^n$ starts at 1 with $v \bmod 3 = 1$, and multiplying by 4 preserves it because $4 = 1 \pmod 3$, checked for $n = 0$ to 15. Repeated squaring computes $3^{13} = 1594323$ and the plain loop agrees, two algorithms proving the same arithmetic statement.

The dry run: after 3 iterations of the odd loop the sum is $1 + 3 + 5 = 9 = 3^2$ and the assert holds, after 4 it is $16 = 4^2$. Each rung is one instance of the same check, which is exactly why the loop version of the proof is trusted: it fails loudly at the first rung that would break the induction.

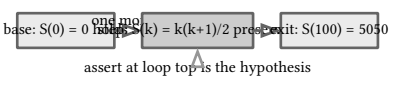


figure 78: induction as the loop: base case, preservation step, exit value

NOTE: where the dsa book meets this

the book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) chapter uses loop invariants and amortized sums to reason about cost; this chapter builds the proof discipline itself and keeps every invariant assert-checked at each rung. the next chapter counts structures, and every count there is proved by the induction patterns fixed here.

13.6. invariants that survive mutation

An invariant is a claim about a data structure that every operation must preserve. The chapter's final samples run fixed mutation sequences over three structures born in the sections above and assert one invariant after every single mutation, 13 asserts per sequence, zero violations.

The first structure is the bitmask set with a parity counter kept alongside: the invariant is `parity == popcount(mask) mod 2`. A fixed sequence of 13 add, remove, and toggle operations, exactly one a no-op (the remove of the absent element 7), ends at mask `0xE7` with 6 elements and parity 0, and the assert runs after each of the 13 steps.

```
// invariant 1: the parity counter inside the bitset structure always
// equals popcount(mask) mod 2, checked after every one of 13 mutations.
// gen.py: final mask 0xe7 = {0,1,2,5,6,7}, popcount 6, parity 0
struct bitset s = {.mask = 0, .parity = 0};
const struct step {
  enum op kind;
  int elem;
} steps[] = {
    {ADD, 3},    {ADD, 5}, {ADD, 6},    {REMOVE, 3}, {TOGGLE, 1},
    {TOGGLE, 1}, {ADD, 1}, {REMOVE, 7}, {ADD, 7},    {ADD, 2},
    {REMOVE, 5}, {ADD, 5}, {TOGGLE, 0},
};
for (unsigned i = 0; i < sizeof steps / sizeof steps[0]; i++) {
  bs_apply(&s, steps[i].kind, steps[i].elem);
  assert(s.parity == card(s.mask) % 2);
}
CHECK(s.mask == 0xE7 && card(s.mask) == 6 && s.parity == 0,
      "bitset parity invariant survives 13 mutations");
```

listing 140: induction.c, the bitset parity invariant asserted after every mutation (source: math/samples/src/Ch13/induction.c, lines 147-164)

The second is a permutation under swaps. A swap of distinct entries flips the permutation's sign, and the invariant is that the running sign equals the sign recomputed from scratch by cycle decomposition. The fixed sequence of 13 swaps, 4 of them self-swaps that change nothing, ends at the permutation [6, 4, 0, 2, 7, 3, 1, 5] with sign −1, and both methods agree after every step. This is the honest form of the argument: the structure could drift, the fresh recomputation cannot.

```
  // invariant 2: a swap sequence keeps a permutation a permutation, and
  // the running sign equals the cycle parity recomputed from scratch.
  // gen.py: 13 swaps (4 of them no-ops) end at [6,4,0,2,7,3,1,5], sign -1
  int perm[8] = {0, 1, 2, 3, 4, 5, 6, 7};
  int sign = 1;
  const int swaps[][2] = {{0, 3}, {1, 5}, {2, 2}, {4, 7}, {0, 6},
                          {1, 1}, {3, 5}, {2, 6}, {0, 0}, {4, 4},
                          {1, 7}, {2, 5}, {3, 6}};
  for (unsigned i = 0; i < sizeof swaps / sizeof swaps[0]; i++) {
    int a = swaps[i][0], b = swaps[i][1];
    int t = perm[a];
    perm[a] = perm[b];
    perm[b] = t;
    if (a != b) {
      sign = -sign;
    }
    assert(sign == perm_sign(perm, 8));
  }
  int expect[8] = {6, 4, 0, 2, 7, 3, 1, 5};
  int same = 1;
  for (int i = 0; i < 8; i++) {
    same &= perm[i] == expect[i];
  }
  CHECK(same && sign == -1 && perm_sign(perm, 8) == -1,
        "permutation sign invariant survives the swap sequence");
```

listing 141: induction.c, permutation sign by running flip versus cycle decomposition (source: math/samples/src/Ch13/induction.c, lines 166-190)

The third is rotation on the fixed array [3, 1, 4, 1, 5, 9, 2, 6, 5]: the multiset is unchanged by a rotation, so the sum 36 is invariant across all 9 rotations, after which the array is restored to its original order. Mutation changes arrangement, never membership.

The dry run: the bitset sequence starts empty, adds 3, 5, 6, removes 3, toggles 1 twice, adds 1 back, removes absent 7, adds 7, adds 2, removes 5, adds 5, and finally toggles 0. Tracing the cardinalities gives 1, 2, 3, 2, 3, 2, 3, 3, 4, 5, 4, 5, 6, and each parity matches the counter, ending at even. Every step had a witness, and the final state matches the playground prediction bit for bit.

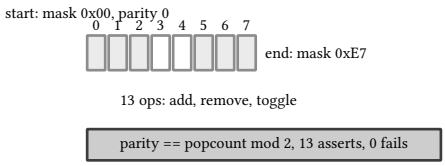


figure 79: before and after 13 mutations, the parity band asserted at each step

13.7. proofs as data

The chapter closes by turning its machinery on itself. A proof is a table of steps over the 8-row universe: each step carries a claimed truth vector and a justification, either a base rule naming one of p, q, r or a derivation citing two prior steps with a bit operation or implication. The checker recomputes every step from its justification, requires the recomputation to agree with the claim row by row, and accepts the proof only when the final step equals the goal mask. It trusts nothing the prover says, only what it can recompute, which makes it the whole-vector descendant of the exhaustion checks in the first section: there the claim was one mask, here the claim is the whole derivation.

The worked proof is hypothetical syllogism, $(p \to q) \wedge (q \to r)$ together give $(p \to r)$, in 8 steps. Steps s0, s1, s2 are the base vectors `0xF0`, `0xCC`, `0xAA`, s3 is `imp(s0, s1)` = `0xCF`, s4 is `imp(s1, s2)` = `0xBB`, s5 is their conjunction `0x8B`, s6

is the conclusion's own mask `imp(s0, s2)` = `0xAF`, and s7 applies implication to s5 and s6, landing on the tautology `0xFF`, the goal. The planted forgery changes one byte of that table: s4 claims `0xCF`, the mask of $p \to q$, in the $q \to r$ slot, exactly the shape of a copy-paste slip, and the checker rejects the table at index 4 with the disagree mask `0x74`.

```
int main(void) {
  // base vectors over the 8-row universe (gen.py: p 0xf0 q 0xcc r 0xaa)
  static const unsigned base_vecs[3] = {0xF0, 0xCC, 0xAA};
  // hypothetical syllogism: (p -> q) and (q -> r) give (p -> r).
  // gen.py step masks: s0 f0 s1 cc s2 aa s3 cf s4 bb s5 8b s6 af s7 ff
  static const struct step good[] = {
      {0xF0, VP, 0, 0},    // s0 p
      {0xCC, VQ, 0, 0},    // s1 q
      {0xAA, VR, 0, 0},    // s2 r
      {0xCF, D_IMP, 0, 1}, // s3 p -> q = imp(s0, s1)
      {0xBB, D_IMP, 1, 2}, // s4 q -> r = imp(s1, s2)
      {0x8B, D_AND, 3, 4}, // s5 s3 and s4
      {0xAF, D_IMP, 0, 2}, // s6 p -> r = imp(s0, s2)
      {0xFF, D_IMP, 5, 6}, // s7 imp(s5, s6), the tautology
  };
  enum { NSTEPS = sizeof good / sizeof good[0] };

  CHECK(recompute(good, 0, base_vecs) == 0xF0 &&
            recompute(good, 1, base_vecs) == 0xCC &&
            recompute(good, 2, base_vecs) == 0xAA,
        "base steps s0 s1 s2 recompute to p q r");
  // gen.py: s3 0xcf, s4 0xbb, s5 0x8b, s6 0xaf, s7 0xff
  CHECK(recompute(good, 3, base_vecs) == 0xCF &&
            recompute(good, 4, base_vecs) == 0xBB,
        "s3 p->q 0xcf and s4 q->r 0xbb");
  CHECK(recompute(good, 5, base_vecs) == 0x8B &&
            recompute(good, 6, base_vecs) == 0xAF,
        "s5 s3-and-s4 0x8b and s6 p->r 0xaf");
  CHECK(recompute(good, 7, base_vecs) == 0xFF,
        "s7 recomputes to the tautology 0xff");

  struct verdict v = check_proof(good, NSTEPS, base_vecs);
  CHECK(v.index == -1 && good[NSTEPS - 1].claim == GOAL,
        "good proof accepted, goal mask 0xff reached");

  // the planted forgery: s4 claims p->q's mask 0xcf in q->r's slot, the
  // exact shape of a plausible copy-paste slip. gen.py: rejected at
  // index 4, disagree (0xbb ^ 0xcf) & 0xff = 0x74
  static const struct step bad[] = {
      {0xF0, VP, 0, 0},    {0xCC, VQ, 0, 0},    {0xAA, VR, 0, 0},
      {0xCF, D_IMP, 0, 1}, {0xCF, D_IMP, 1, 2}, {0x8B, D_AND, 3, 4},
      {0xAF, D_IMP, 0, 2}, {0xFF, D_IMP, 5, 6},
  };
  struct verdict w = check_proof(bad, NSTEPS, base_vecs);
  CHECK(w.index == 4 && w.disagree == 0x74,
        "bad proof rejected at step 4 with disagree 0x74");

  CHECK(NSTEPS == 8, "the proof table holds 8 steps");

  printf("%d checks passed\n", checks);
  return 0;
}
```

listing 142: checker.c, the syllogism as a step table, the planted forgery, and both verdicts (source: math/samples/src/Ch13/checker.c, lines 102-153)

The dry run: the checker walks the good table in order. The first three steps return the base vectors, then each derivation recomputes from the cited claims, `0xCF`, `0xBB`, `0x8B`, `0xAF`, and `0xFF` at s7, every step agreeing, the final

claim equal to the goal, the proof accepted. The same walk over the forged table dies at index 4: the recomputation says `0xBB`, the claim says `0xCF`, and the disagree mask `(0xBB ^ 0xCF) & 0xFF = 0x74` names rows 2, 4, 5, and 6, the rows where the forged claim misstates $q \to r$.

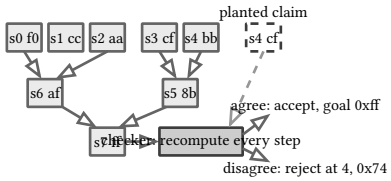


figure 80: the proof as data: steps s0 to s7 carry their claim masks, citations flow downward into the checker, and the flagged variant of s4 routes to reject

PITFALL: citations must point backward

the checker validates steps in order and reads only the claims of the cited steps, so the contract that citations name earlier steps is what keeps every recomputation resting on already-validated ground. a table that cites forward, or cycles, can be self-consistent and still prove nothing: recompute the cycle and it agrees with itself. the sample tables cite backward by construction, and a checker outside the trust boundary would reject any forward citation before recomputing a single mask.

The same shape runs at research scale in the openai/math release. An internal model was pointed at about 4,000 open research problems, and the 2026-10-06 publication carries 719 manuscripts in 372 result families, with about 42 percent of top-line results backed by Lean 4 formalizations and trust graded per claim: formal, comparator-checked supporting, and warned unformalized. The proof is the artifact handed to a small checker that decides acceptance on its own recomputation, which is what makes the output auditable claim by claim.

Family 107, matrix multiplication, is the release's worked instance of both tiers at once, and this book reran its numerical checker. The staggered-extraction preprint packages `verify_parameters.py`, which recomputes the every-field bound $\omega < 2.371054886006746$ from exact rationals: all 104 printed enclosures are rebuilt from exact Fraction weights with outward rounding, an assertion over an interval requires the whole interval to satisfy the bound, the widths obey caps of 10^{-76} in native log mode and $3 \cdot 10^{-29}$ in series mode, and the script writes its output only after every internal assertion passes. The two commands and their manifest live in `ref/openai-math/family-107/README.md`, and the two local runs of 2026-10-11, python 3.14.7 with mpmath 1.3.0, one in native log mode and one in rational-series mode, are cached there under `recheck/` and copied byte-exact into `books/math/samples/fixtures/` with their provenance. What the rerun certifies is the numerical layer: the enclosures, the 13 structural checks, and the strict gap. What it does not certify sits above that layer, the Lean proofs are separate comparator targets and the extraction theorem lifting the numerics to the omega bound is proved in the accompanying paper. Same shape as the step table: the checker trusts only what it recomputes.

log mode	strict gap lo	exponent upper bound hi
native	$4.67682972596832730177081 27 \cdot 10^{-6}$	2.37105488600674568486111187559740030536 09...
rational series	$4.67682972596832730177080 94 \cdot 10^{-6}$	2.37105488600674568486111187559819079279 68...

The two modes agree on the gap through 23 digits and on the bound through 30, then the native and series enclosures diverge while both stay below the claimed threshold. The full hi strings, 92 characters carrying 91 significant digits and 90 decimals, are the lane's oracle, compared byte for byte against the values recorded in the family README.

The book's own checker for these fixtures is `mmbound.c`, built by the same pinned clang 23 lane as the chapter samples. It parses the two decimals under `exponent_upper_bound` and the two under `strict_gap`, and compares them exactly with no floating point anywhere: `lo < hi` holds exactly when lo's digits followed by hi's number of zeros fall short of hi's digits followed by lo's number of zeros, the cross-multiplication by the other side's power of ten carried out as digit-string padding, and two integers written as digit strings compare by length then place value. The verdict set is the claim in miniature: the exponent interval is nonempty, its top sits strictly below

2.371054886006746 the same way, the gap is strictly positive, and any tampered or malformed fixture exits nonzero. `make verify-math-f107` drives it: 16 field checks against the README oracle, the 8 checker verdicts over both fixtures, 16 tamper probes covering swapped, raised, equal-to-threshold, garbled, negative, and zero-valued bounds plus duplicated-key and duplicated-member decoys, and 64 seeded byte mutations, all green.

```c
static int cmp_digits(const char *a, const char *b) {
  size_t la = strlen(a), lb = strlen(b);
  if (la != lb)
    return la < lb ? -1 : 1;
  int c = strcmp(a, b);
  return c < 0 ? -1 : (c > 0 ? 1 : 0);
}

static int pad_to(const char *digits, size_t extra, char *out, size_t cap) {
  size_t n = strlen(digits);
  if (n + extra + 1 > cap)
    return 0;
  memcpy(out, digits, n);
  memset(out + n, '0', extra);
  out[n + extra] = '\0';
  return 1;
}
```

listing 143: mmbound.c, the exact compare, digit strings by length then place value, and the power-of-ten padding, no floating point (source: math/samples/checker/mmbound.c, lines 69-85)

```c
static int dec_cmp(const struct dec *x, const struct dec *y) {
  char xn[MAX_DIGITS * 2], yn[MAX_DIGITS * 2];
  char px[MAX_DIGITS * 3 + 2], py[MAX_DIGITS * 3 + 2];
  if (!join_digits(x, xn, sizeof xn) || !join_digits(y, yn, sizeof yn))
    return 2;
  int xz = digits_zero(xn), yz = digits_zero(yn);
  if (xz || yz) {
    if (xz && yz)
      return 0;
    if (xz)
      return y->neg ? 1 : -1;
    return x->neg ? -1 : 1;
  }
  if (x->neg != y->neg)
    return x->neg ? -1 : 1;
  size_t fx = strlen(x->fp), fy = strlen(y->fp);
  if (!pad_to(xn, fy, px, sizeof px) || !pad_to(yn, fx, py, sizeof py))
    return 2;
  int m = cmp_digits(px, py);
  return x->neg ? -m : m;
}
```

listing 144: mmbound.c, the cross-multiplication, each side's digits followed by the other side's number of zeros (source: math/samples/checker/mmbound.c, lines 97-117)

The release's families touch several Clay millennium problems, and the trust grades decide what a book may claim from them. The stance rows:

family	claim	tier	what this book states
002	full BSD formula over Q in Selmer corank 0 and 1, with Sha finiteness	report, paper only	absent from the formalized catalogue, no formal backing claimed

family	claim	tier	what this book states
032	rational Hodge conjecture for CM abelian varieties	report, paper only	same gap, one of the release's two procedure exceptions
003	quasi-RH, Dirichlet L-functions zero-free for $\Re(s) > \frac{7}{8}$, companion proof at $\frac{11}{12}$	Lean	formalized with comparator `DirichletSevenEighths`, and the Riemann hypothesis itself stays open

No P versus NP claim exists anywhere in the release. The complexity-side headline is family 102, the Unique Games Conjecture at Lean tier, which carries the approximation thresholds the book 9, ch 42 (practical data structures and algorithms, approximation algorithms) chapter restates with their tier named.

With propositions, quantifiers, sets, relations, functions, induction, invariants, and proofs in hand as data plus loops, the next step is counting the structures themselves, which is book 2, ch 14 (mathematics for programmers, combinatorics).

sources: Hammack, Book of Proof, 3rd ed., 2018, and Rosen, Discrete Mathematics and Its Applications, 8th ed., 2019, cited by name and edition for the definitions of connectives, quantifiers, sets, relations, functions, induction, and the pigeonhole principle, restated here in our own words and verified by the checks below (the former people.vcu.edu hosting of the open Hammack text redirected to a university landing page on access 2026-09-21, so no page text is quoted). cppreference, "static_assert declaration", en.cppreference.com/w/c/language/static_assert, fetched 2026-09-21: `static_assert(expression, message)` since C23, the message optional since C23, `_Static_assert` the deprecated spelling kept for compatibility. ISO/IEC 9899:2024 draft N3220 (C23) cited by name as the authority for bit-precise integer types `_BitInt(N)`. Probed on this machine the same day: `_BitInt(128)` multiply, shift, and mask link cleanly under the pinned clang 23 with the MSVC toolset, while `%` with a runtime divisor lowers to a compiler-rt `__umodti3` call that fails the link (a constant divisor expands to multiply-shift and links), so the sample masks by `2^64 - 1` instead, and `<stdbit.h>` is absent from the toolset headers, so popcount is the clear-lowest-bit loop. OpenAI, "Sharing AI progress in mathematics", https://openai.com/index/sharing-ai-progress-in-mathematics/, published 2026-10-06, fetched 2026-10-09, an internal model pointed at about 4,000 open research problems whose release publishes 719 manuscripts in 372 result families with trust graded formal, comparator-checked, and warned. The repository github.com/openai/math, fetched 2026-10-09, publishes the ComparatorChallenges verification targets run via `lake env comparator <file>`. Family 107 sources, fetched 2026-10-11 through the cache ref/openai-math/family-107/: the staggered-extraction verifier `verify_parameters.py`, sha256 d7d11c2ff81ebb535a80f61da1b68928743ab6d782ffb2f23442045a0342cb32, run locally in both modes (python 3.14.7, mpmath 1.3.0) over inputs pinned by parameter sha256 538deff25fb30b68b01f7ff43b2a62ce0719afba25891ff38fda7027a31c2b65, both outputs copied byte-exact into books/math/samples/fixtures/ with provenance in the README there. Millennium stance rows from overview.tex entries 002, 003, 032, and 102, with formalization.yaml as the tier arbiter, its Dirichlet entry carrying declaration `OAI.DirichletCharacter.LFunction_ne_zero_of_seven_eighths_lt_re` and comparator config `ComparatorChallenges/DirichletSevenEighths.json`. The f107 lane verified by `pwsh -NoProfile -File tools/verify-math-f107.ps1`: 16 fixture field checks against the family README oracle, 8 mmbound verdicts over both fixtures, 16 tamper probes with the right verdict each, 64 seeded byte mutations exited clean, format leg clean. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch13`, 52 checks in chapter 13 of the math suite, format leg clean, zero failures.

14. combinatorics

Counting is arithmetic with structure, and the structure is what makes a count trustworthy: the product rule composes independent choices, the binomial coefficient collapses orderings that do not matter, and every identity in this chapter is a statement that two different counting arguments return the same integer. The route runs through the sum and product rules as exact integer code paths, permutations and combinations built by the Pascal recurrence with the uint64 overflow boundary pinned and the wide ladder moved onto C23 `_BitInt`, the binomial theorem verified against schoolbook expansion, inclusion-exclusion with the derangement ladder walking into $\frac{n!}{e}$, linear recurrences from Fibonacci to the Tower of Hanoi with their closed forms pinned beside them, generating functions as exact coefficient arithmetic, and finally the bridge where counting becomes probability. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 14 or a sentence quoted from the mml-book draft 2024-01-15 anchor sheet and canonical references fetched 2026-09-21. Where book 9, ch 35 (practical data structures and algorithms, combinatorics) counts to bound algorithm work, this chapter is the theory underneath: what the identities are and why they hold in exact arithmetic.

14.1. the sum and product rules as code

Definition first. The product rule says a choice made in k independent stages with n_i options at stage i has $n_1 n_2 \cdots n_k$ outcomes, the size of a cartesian product. The sum rule says mutually exclusive cases add: if A and B are disjoint then $|A \cup B| = |A| + |B|$. Both are theorems about finite sets, and both run as plain loops: an odometer for the product rule, a bucket counter for the sum rule.

The mml book's probability space of section 6.1 is built on exactly these objects [printed pp 172-178 / pdf pp 178-184]: the sample space Ω is "the set of all possible outcomes of the experiment", and two successive coin tosses give $\Omega = \{hh, tt, ht, th\}$, four outcomes because the product rule composes two 2-element stages [printed p 175 / pdf p 181]. Example 6.1 runs the same construction at a funfair: draw a coin with replacement from a bag of USA $ and UK £ coins where "a draw returns at random a $ with probability 0.3", and let X count dollars [printed p 176 / pdf p 182]. The lookup table eqs (6.1)-(6.4) maps $\Omega = \{(\$, \$), (\$, £), (£, \$), (£, £)\}$ onto target states $\mathcal{T} = \{0, 1, 2\}$.

The dry run: the 4 outcomes of Ω land on 3 states, and the pre-image sizes eq (6.8) are 1, 2, 1.

1. `($,$)` maps to 2: one outcome, so the pre-image of state 2 has size 1.
2. `($,£)` and `(£,$)` both map to 1: pre-image size 2.
3. `(£,£)` maps to 0: pre-image size 1, and $1 + 2 + 1 = 4$ closes the count.

Those sizes 1, 2, 1 are row 2 of Pascal's triangle, which is where the next two sections are headed. Weight each outcome by its draw probability, held exactly as the rational $\frac{3}{10}$, and the numerators of eqs (6.5)-(6.7) fall out as $7 \cdot 7 = 49$, $3 \cdot 7 + 7 \cdot 3 = 42$, $3 \cdot 3 = 9$ over the common denominator 100, summing to exactly $\frac{100}{100}$.

outcome	X	weight	numerator
($, $)	2	$3 \cdot 3$	9/100
($, £), (£, $)	1	$3 \cdot 7$ each	42/100
(£, £)	0	$7 \cdot 7$	49/100

```c
// mml example 6.1: omega for two draws with replacement has
// 2*2 = 4 outcomes (the two coin tosses of section 6.1.2 use the
// same product), X collapses them onto states {0, 1, 2}
uint64_t states[3] = {0};
for (coin_face a = 0; a ≤ 1; a++) {
  for (coin_face b = 0; b ≤ 1; b++) {
    states[dollar_count(a, b)]++;
  }
}
CHECK(states[0] + states[1] + states[2] == 4, "omega has 4 outcomes");
CHECK(states[0] == 1 && states[1] == 2 && states[2] == 1,
      "pre-image sizes 1, 2, 1 (eq 6.8)");

// the same outcomes weighted: P($) = 3/10 held exactly as the
// rational 3/10, so each outcome carries weight 3^k * 7^(2-k) and
// the pmf numerators over the common denominator 100 are exactly
// eqs (6.5)-(6.7): 49/100, 42/100, 9/100
static constexpr int P_NUM = 3; // P($) = 3/10, exactly
static constexpr int P_DEN = 10;
int num[3] = {0};
for (coin_face a = 0; a ≤ 1; a++) {
  for (coin_face b = 0; b ≤ 1; b++) {
    int wa = (a == USD) ? P_NUM : (P_DEN - P_NUM);
    int wb = (b == USD) ? P_NUM : (P_DEN - P_NUM);
    num[dollar_count(a, b)] += wa * wb;
  }
}
CHECK(num[0] == 49 && num[1] == 42 && num[2] == 9,
      "pmf numerators 49, 42, 9 over 100");
CHECK(num[0] + num[1] + num[2] == 100, "pmf sums to 100/100");
```

listing 145: counting.c, example 6.1 as pure counting: 4 outcomes, pre-image sizes 1, 2, 1, then the same outcomes weighted exactly as the rational 3/10 (source: math/samples/src/Ch14/counting.c, lines 74-103)

The play: the sample space has 4 outcomes but the target space has 3 states, and the 1, 2, 1 multiplicity between them is already a binomial row. The sum rule on subset sizes says the same thing one level up: the 16 subsets of a 4-element set split $1 + 4 + 6 + 4 + 1$ by cardinality, each bucket counted by bitmask scan (checks 4 and 5).

PITFALL: count the right space

the two classic counting bugs are both space bugs: enumerating ordered tuples when the question wants unordered (every hand counted 5! = 120 times), and forgetting that the sum rule needs disjointness. the fix is mechanical, enumerate the sample space by odometer or bitmask, bucket by the property asked, and let the loop disagree with the formula until they agree for the right reason.

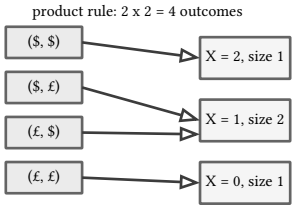


figure 81: the example 6.1 mapping, 4 outcomes of the product space collapsing onto 3 states with pre-image sizes 1, 2, 1

14.2. permutations and combinations

An ordered selection of k from n is a permutation count, $P(n,k) = n(n-1)\cdots(n-k+1) = n!/(n-k)!$. Forgetting order divides out the $k!$ orderings of each selection and leaves the binomial coefficient $\binom{n}{k} = n!/(k!(n-k)!)$.

The factorial ladder is the first thing to pin because it sets every width decision after it: $20! = 2432902008176640000$ has 62 bits and is the last exact factorial in uint64, while $21! = 51090942171709440000$ needs 66 bits and wraps to 14197454024290336768 (checks 1 and 2). Unsigned wraparound is defined behavior, the mod 2^{64} arithmetic of book 1, ch 5 (c23: the language, the machine, and the cloud, the abstract machine and undefined behavior), so the wrap is a measurement, not a crash. Computing $\binom{n}{k}$ through factorials inherits that ceiling immediately, and the floating-point route $n!/(k!(n-k)!)$ in doubles loses the integer character of the answer entirely.

The way out is the Pascal recurrence:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}, \quad \binom{0}{0} = 1, \quad \binom{n}{k} = 0 \text{ outside } 0 \le k \le n$$

Additions only: no division, no cancellation, no rounding, and every intermediate value is itself a binomial coefficient. On a rolling row of $k+1$ entries the whole triangle costs $O(nk)$ additions and stays exact until a value leaves the type.

The dry run: row 5 of the triangle, built by the recurrence from row 4's `1 4 6 4 1`.

1. Outside entries copy: $\binom{5}{0} = \binom{5}{5} = 1$.
2. Interior entries add two parents: $\binom{5}{1} = 1+4 = 5$, $\binom{5}{2} = 4+6 = 10$, $\binom{5}{3} = 6+4 = 10$, $\binom{5}{4} = 4+1 = 5$.
3. Row 5 reads `1 5 10 10 5 1` and sums to $2^5 = 32$.

```c
// C(n,k) by the pascal recurrence on a rolling row: only true
// binomials are ever added, so the value is exact until it leaves the
// type, and the single wrap at the ceiling is visible, not silent
static uint64_t binom_u64(int n, int k) {
  if (k < 0 || k > n) {
    return 0;
  }
  if (n - k < k) {
    k = n - k;
  }
  uint64_t row[35] = {1}; // k ≤ 34 after the symmetry clamp
  for (int i = 1; i ≤ n; i++) {
    int top = (i < k) ? i : k;
    for (int j = top; j ≥ 1; j--) {
      row[j] += row[j - 1];
    }
  }
  return row[k];
}
```

listing 146: binomial.c, the rolling row: only true binomials are ever added, so the value is exact until it leaves the type (source: math/samples/src/Ch14/binomial.c, lines 52-70)

The identities the rest of the book leans on, each CHECKed in both directions against the pinned value:

identity	instance	pinned value
symmetry $\binom{n}{k} = \binom{n}{n-k}$	$\binom{50}{47} = \binom{50}{3}$	19600
pascal $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$	$\binom{48}{12}$	69668534468
hockey stick $\sum_{j=k}^{n}\binom{j}{k} = \binom{n+1}{k+1}$	$\sum_{j=3}^{30}\binom{j}{3}$	31465

identity	instance	pinned value
vandermonde $\sum_j \binom{m}{j}\binom{n}{r-j} = \binom{m+n}{r}$	$\binom{20+20}{10}$	847660528
absorption $k\binom{n}{k} = n\binom{n-1}{k-1}$	$7\binom{30}{7}$	14250600
row sum $\sum_k \binom{n}{k} = 2^n$	row 30	1073741824

The play: a 5-card poker deal has $P(52,5) = 311875200$ orderings but only $C(52,5) = 2598960$ hands, and the ratio of the two pins is exactly $120 = 5!$, the orderings each hand was carrying (checks 4 to 6).

```
// the uint64 ceiling of the central ladder: C(66,33) =
// 7219428434016265740 exact, one row up C(68,34) = 28453041475240576740
// leaves 64 bits and the pascal addition wraps to 10006297401531025124
CHECK(binom_u64(66, 33) == 7219428434016265740ULL,
      "C(66,33) = 7219428434016265740 exact");
CHECK(binom_u64(68, 34) == 10006297401531025124ULL,
      "C(68,34) wraps to 10006297401531025124");

// _BitInt(128): C(100,50) = 100891344545564193334812497256 (97
// bits), and at the top of the ladder C(128,64) =
// 23951146041928082866135587776380551750 (125 bits) still splits by
// pascal's rule into the two 127-row entries
// 119755730209640414330677938881902758 75
CHECK(binom128(100, 50) == 100891344545564193334812497256uwb,
      "C(100,50) exact in _BitInt(128)");
CHECK(binom128(128, 64) == 23951146041928082866135587776380551750uwb,
      "C(128,64) exact in _BitInt(128)");
CHECK(binom128(127, 63) + binom128(127, 64) == binom128(128, 64),
      "pascal holds at row 128 in _BitInt(128)");
```

listing 147: binomial.c, the width ladder: uint64 dies between C(66,33) and C(68,34), the bit-precise types carry the ladder to row 128 (source: math/samples/src/Ch14/binomial.c, lines 182-200)

The width ladder continues past 64 bits on the C23 bit-precise integers, with widths stated: the central column is exact in uint64 through $\binom{66}{33} = 7219428434016265740$, but $\binom{68}{34} = 28453041475240576740$ leaves 64 bits and the final Pascal addition wraps to 10006297401531025124 (checks 12 and 13). `_BitInt(128)` carries $\binom{100}{50} = 100891344545564193334812497256$ (97 bits) and tops out of this chapter's needs at $\binom{128}{64}$ with 125 bits, and `_BitInt(256)` holds $\binom{200}{100}$ with 196 bits, written as the `uwb` literal the C23 bit-precise suffix gives (cppreference, integer constants, fetched 2026-09-21). The Pascal recurrence holds at row 128 in the wide type (check 16), and since printf has no `_BitInt` conversion the digits come out by repeated division and round-trip against the decimal string (checks 17 and 18).

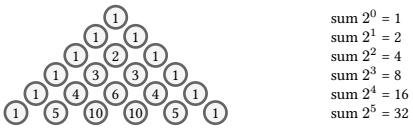


figure 82: pascal's triangle rows 0 to 5 with row sums doubling

14.3. the binomial theorem

The theorem: $(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$, the statement that the expansion coefficients are counted, each coefficient the number of ways to pick which k of the n factors contribute their x. Concrete Mathematics proves it by induction and calls it "the most important fact about the binomial coefficients" (Graham, Knuth, Patashnik, 2nd ed, chapter 5).

The verification discipline of this chapter refuses to trust either side: multiply $(1+z)$ by itself n times schoolbook style and compare every coefficient against the Pascal row.

```
// expansion vs theorem: every coefficient of (1+z)^12 equals
// C(12,k): 1, 12, 66, 220, 495, 792, 924, 792, 495, 220, 66, 12, 1
uint64_t c[33];
expand(12, c);
int all12 = 1;
for (int k = 0; k ≤ 12; k++) {
  all12 = all12 && (c[k] == binom_u64(12, k));
}
CHECK(all12, "(1+z)^12 coefficients match C(12,k) at every k");

// the theorem at integer arguments: (3 + (-2))^7 = 1 by the
// coefficient sum, matching 1^7 computed directly
int64_t v = 0;
for (int k = 0; k ≤ 7; k++) {
  int64_t xk = 1, yk = 1;
  for (int i = 0; i < 7 - k; i++) {
    xk *= 3;
  }
  for (int i = 0; i < k; i++) {
    yk *= -2;
  }
  v += (int64_t)binom_u64(7, k) * xk * yk;
}
CHECK(v == 1, "(3-2)^7 = 1 through the expansion coefficients");
```

listing 148: theorem.c, the expansion checked against the pascal row at n = 12, then the theorem at integer arguments, (3 - 2)^7 = 1 (source: math/samples/src/Ch14/theorem.c, lines 81-104)

The dry run: at $n = 2$, $(1 + z)(1 + z) = 1 + 2z + z^2$, and the coefficients `1 2 1` are row 2 of the triangle. One more multiplication gives `1 3 3 1`, and the pattern continues to row 12, where the expansion and $\binom{12}{k}$ agree at all 13 positions: `1 12 66 220 495 792 924 792 495 220 66 12 1` (check 1). At integer arguments the coefficient sum evaluates $(3 + (-2))^7$ to exactly 1, matching the direct 1^7 (check 2).

Three sum identities live on the same row, each a different weighting:

$$\sum_k \binom{n}{k} = 2^n, \quad \sum_k (-1)^k \binom{n}{k} = 0, \quad \sum_k k\binom{n}{k} = n2^{n-1}$$

On row 30 they pin to 1073741824, 0, and $16106127360 = 30 \cdot 2^{29}$ (checks 3 to 5). The row-sum identity has an exact overflow boundary: every entry of row 64 fits uint64, $\binom{64}{32} = 1832624140942590534$, but the row sums to exactly 2^{64}, so the uint64 accumulator lands on 0 (check 6). The alternating identity is the binomial theorem at $x = 1, y = -1$.

The probability side of the same theorem is already in the counting section: eqs (6.5)-(6.7) of the mml book weight the $\binom{2}{k}$ pre-images by $0.3^k 0.7^{2-k}$, which is the binomial distribution, the theorem doing double duty as a counting statement and a weighting statement.

The play: row 30 sums to 1073741824 with all plus signs and to exactly 0 with alternating signs, the same 31 numbers read two ways.

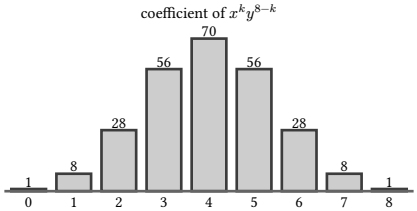


figure 83: coefficients of (x + y)^8, the binomial row as a symmetric bar profile

14.4. inclusion-exclusion and derangements

Two sets: $|A \cup B| = |A| + |B| - |A \cap B|$, subtracting the outcomes counted twice. Three sets add back the double-subtracted triple:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Worked example, the one the sample runs both ways: integers in 1..1000 divisible by 3, 5, or 7. The floors are $333, 200, 142$, the pairwise $66, 47, 28$, the triple 9, and the formula gives $333 + 200 + 142 - 66 - 47 - 28 + 9 = 543$. A 1000-step scan of the same question returns 543 (checks 7 to 10). The formula is 7 divisions, the scan is 1000 tests, and the gap grows with the universe: at 10^{18} the scan is dead and the formula still costs 7 divisions.

The dry run: which numbers die at each correction? 15 is divisible by 3 and by 5, so it enters in $|A|$ and $|B|$, leaves in $-|A \cap B|$, and never returns, net count 1. 105 is divisible by all three, so it enters three times, leaves three times, and re-enters once in the triple, net count 1.

```
// inclusion-exclusion, 1..1000 divisible by 3, 5, or 7: the floors
// are 333, 200, 142, the pairwise 66, 47, 28, the triple 9
uint64_t d3 = 0, d105 = 0;
for (int i = 1; i ≤ 1000; i++) {
  d3 += (i % 3 == 0);
  d105 += (i % 105 == 0);
}
CHECK(d3 == 333, "floor(1000/3) = 333 by scan");
CHECK(d105 == 9, "floor(1000/105) = 9 by scan");
uint64_t formula = 1000 / 3 + 1000 / 5 + 1000 / 7 - 1000 / 15 - 1000 / 21 -
                   1000 / 35 + 1000 / 105;
uint64_t scan = 0;
for (int i = 1; i ≤ 1000; i++) {
  scan += (i % 3 == 0 || i % 5 == 0 || i % 7 == 0);
}
CHECK(formula == 543, "inclusion-exclusion union = 543");
CHECK(scan == formula, "the 1000-step scan agrees with the formula");
```

listing 149: theorem.c, inclusion-exclusion by 7 divisions against a full scan of 1..1000 (source: math/samples/src/Ch14/theorem.c, lines 131-147)

A derangement is a permutation with no fixed point, written $!n$, and it is inclusion-exclusion in its purest form: exclude each fixed point, re-include pairs, and so on, which lands

$$!n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} \quad \sim \quad n!/e$$

The ladder satisfies the recurrence $!n = (n-1)(!(n-1) + !(n-2))$ with $!0 = 1, !1 = 0$, and the two forms agree exactly: at $n = 4$ the alternating sum is $24 - 24 + 12 - 4 + 1 = 9 = !4$, and at $n = 20$ both give 895014631192902121 (checks 11 to 14).

n	$!n$	$!n/n!$	distance to $1/e$
4	9	0.375	7.1e-3
6	265	0.368056	1.8e-4
8	14833	0.367882	2.5e-6
10	1334961	0.3678795	2.3e-8
12	176214841	0.367879441	1.5e-10
20	895014631192902121	0.3678794412	1.9e-20

The ratio column walks into $1/e = 0.36787944117144233$, and the shortcut "round $\frac{n!}{e}$" is tempting but floats cannot cash it: at $n = 20$ the double $20!/e$ lands on 895014631192902144, which is 23 past the exact !20, because

one ulp at 8.95×10^{17} is 128 and the float path cannot see single digits (check 15). The integer path has no such ceiling inside this ladder.

VERIFY: two authorities per count

every union size in this section answers to both the formula and the scan, and every derangement to both the recurrence and the alternating sum. when the two disagree the bug is in the overlap structure you modeled, not in the arithmetic, because the arithmetic is exact integers end to end.

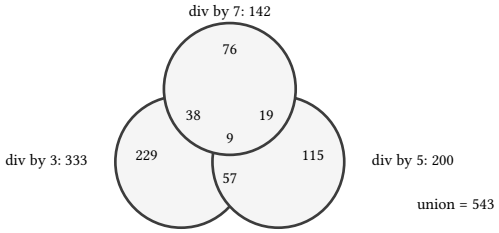


figure 84: the three-set venn for 1..1000 divisible by 3, 5, 7, region counts netting to 543

The play: 543 survivors from 7 divisions or from a 1000-step scan, and 895014631192902121 exact against 895014631192902144 from the float shortcut.

14.5. recurrences by iteration

A recurrence defines a sequence by its history, and the honest first implementation is iteration: keep the last few terms, add, step. The closed form earns trust only by matching the ladder.

Fibonacci, $F(n) = F(n-1) + F(n-2)$ with $F(0) = 0, F(1) = 1$: the uint64 ladder ends at $F(93) = 12200160415121876738$, and one more step wraps, $F(94) = 19740274219868223167$ collapsing to 1293530146158671551 (checks 1 to 3). Two identities pin the structure without any closed form: the telescoping sum $\sum_{i=0}^{n} F(i) = F(n+2) - 1$, at $n = 10$ both sides 143, and Cassini's identity $F(n-1)F(n+1) - F(n)^2 = (-1)^n$, at $n = 40$ the exact products are $102372279279564026 - 102372279279564025 = 1$ (checks 4 and 5).

The dry run: the ladder through F(12) reads 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, and the telescoping identity closes on it by hand: $0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + 21 + 34 + 55 = 143 = 144 - 1 = F(12) - 1$, the eleven terms $F(0)$ through $F(10)$ summing to one less than the next-but-one rung.

The growth ratio converges to the golden ratio $\varphi = \left(1 + \sqrt{5}\right)/2 = 1.618033988749895$:

n	$F(n+1)/F(n)$	distance to φ
10	1.6181818181818182	1.48e-4
30	1.6180339887505408	6.5e-13
90	1.618033988749895	below double resolution

By $n = 90$ the ratio of two exact integers rounds to precisely the double of φ, the check comparing `f[91]/f[90]` against `(1 + sqrt(5))/2` for bit equality (checks 6 and 7). Binet's closed form $F(n) = (\varphi^n - \psi^n)/\sqrt{5}$ with $\psi = 1 - \varphi$ evaluated in doubles lands 6765.000000000005 at $n = 20$, five trillionths off the exact 6765 and close only because ψ^{20} is already 6.6×10^{-5} (check 8).

```c
// fibonacci, F(0) = 0, F(1) = 1: the uint64 ladder ends at F(93) =
// 12200160415121876738; one step further F(94) = 19740274219868223167
// wraps to 1293530146158671551
uint64_t f[95];
f[0] = 0;
f[1] = 1;
for (int n = 2; n <= 94; n++) {
  f[n] = f[n - 1] + f[n - 2];
}
CHECK(f[20] == 6765, "F(20) = 6765");
CHECK(f[93] == 12200160415121876738ULL, "F(93) = last exact uint64");
CHECK(f[94] == 12935301461586715551ULL, "F(94) wraps to 1293530146158671551");

// telescoping sum: F(0) + ... + F(10) = 143 = F(12) - 1
uint64_t s = 0;
for (int n = 0; n <= 10; n++) {
  s += f[n];
}
CHECK(s == 143 && f[12] - 1 == 143, "sum F(0..10) = F(12) - 1 = 143");

// cassini at n = 40: F(39) F(41) - F(40)^2 = 10472279279564026 -
// 10472279279564025 = 1
CHECK(f[39] * f[41] - f[40] * f[40] == 1, "cassini F(39)F(41) - F(40)^2 = 1");

// growth ratio: F(11)/F(10) = 1.6181818... is 1.48e-4 from phi, but
// F(91)/F(90) rounds to exactly the double of phi = (1+sqrt5)/2
auto phi = (1.0 + sqrt(5.0)) / 2.0;
CHECK((double)f[11] / (double)f[10] == 1.6181818181818182,
      "F(11)/F(10) = 1.6181818181818182");
CHECK((double)f[91] / (double)f[90] == phi,
      "F(91)/F(90) equals the double of phi");

// binet's closed form in doubles: 20 terms of powers of phi and psi
// land 6765.000000000005, six picos from the exact integer
double psi = 1.0 - phi;
double binet20 = (pow(phi, 20.0) - pow(psi, 20.0)) / sqrt(5.0);
CHECK(fabs(binet20 - 6765.0) < 1e-11, "binet F(20) within 1e-11 of 6765");
```

listing 150: recurrence.c, the exact ladder to F(93), two identities, the ratio pins, and binet in doubles (source: math/samples/src/Ch14/recurrence.c, lines 70-106)

The Tower of Hanoi, $T(n) = 2T(n-1) + 1$ with $T(1) = 1$: unfold it and each disk-doubling nests, $T(n) = 2^{n-1}T(1) + (2^{n-1} - 1) = 2^n - 1$. The induction that makes the last step legal is the vocabulary of book 2, ch 13 (mathematics for programmers, logic, sets, and proof). The sample checks the closed form $2^n - 1$ against the recurrence for every n from 1 to 63, and then the endpoint: $T(64) = 2^{64} - 1 = 18446744073709551615$, exactly uint64 max, the 64-disk tower is the entire unsigned range (checks 9 and 10).

Catalan numbers count balanced structures (nested parentheses, binary trees) by the convolution recurrence $C_n = \sum_i C_i C_{n-1-i}$ with $C_0 = 1$, and the closed form $\binom{2n}{n}/(n+1)$ agrees with the convolution at every pinned point: $C_{15} = \binom{30}{15}/16 = 9694845$ and $C_{20} = \binom{40}{20}/21 = 6564120420$ (checks 11 and 12).

```
// tower of hanoi, T(n) = 2 T(n-1) + 1: the closed form 2^n - 1 holds
// for every n = 1..63, and T(64) = 2^64 - 1 = UINT64_MAX exactly
uint64_t t[65];
t[0] = 0;
for (int n = 1; n ≤ 64; n++) {
  t[n] = 2 * t[n - 1] + 1;
}
int closed_all = 1;
for (int n = 1; n ≤ 63; n++) {
  closed_all = closed_all && (t[n] == ((uint64_t)1 << n) - 1);
}
CHECK(closed_all, "T(n) = 2^n - 1 for n = 1..63");
CHECK(t[64] == UINT64_MAX, "T(64) = 18446744073709551615 = 2^64 - 1");

// catalan numbers two ways: the convolution recurrence
// C_n = sum_i C_i C_(n-1-i) and the closed form C(2n,n)/(n+1) agree
// at n = 15 (9694845) and n = 20 (6564120420)
uint64_t cat[31];
cat[0] = 1;
for (int n = 1; n ≤ 20; n++) {
  cat[n] = 0;
  for (int i = 0; i < n; i++) {
    cat[n] += cat[i] * cat[n - 1 - i];
  }
}
CHECK(cat[15] == 9694845 && binom_u64(30, 15) / 16 == 9694845,
      "catalan(15) = C(30,15)/16 = 9694845");
CHECK(cat[20] == 6564120420 && binom_u64(40, 20) / 21 == 6564120420,
      "catalan(20) = C(40,20)/21 = 6564120420");
```

listing 151: recurrence.c, hanoi's closed form checked for all n = 1..63 with T(64) = uint64 max, catalan two ways (source: math/samples/src/Ch14/recurrence.c, lines 108-136)

The play: $F(93) = 12200160415121876738$ is the last honest Fibonacci in 64 bits while $T(64)$ fits its entire $2^{64} - 1$ range exactly, two recurrences with two different ceilings on the same type.

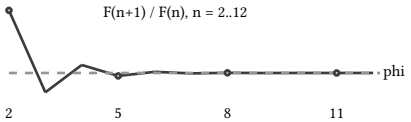


figure 85: the ratio F(n+1)/F(n) alternating into the golden ratio, dashed

14.6. generating functions as exact coefficients

The ordinary generating function of a sequence $a_0, a_1, a_2, \ldots$ is the formal power series $G(z) = \sum_{n \geq 0} a_n z^n$. "Formal" is the whole point for a programmer: z is a position marker, never a number, multiplication of two series is the convolution $\sum_{i+j=n} a_i b_j$, and everything in this section is exact integer arithmetic on truncated coefficient arrays. Concrete Mathematics opens its generating function chapter with this exact attitude: the series is a clothesline on which the sequence hangs (Graham, Knuth, Patashnik, 2nd ed, chapter 7).

The technique that makes generating functions compute: find the polynomial that annihilates the series. Multiplying by $1 - z$ shifts and subtracts, so $(1 - z) \cdot \frac{1}{1-z} = 1$, the geometric series killed by one factor.

The dry run: $(1 - z - z^2)$ against the Fibonacci series $1, 1, 2, 3, 5, \ldots$, coefficient by coefficient.

1. Constant term: $a_0 = 1$, so the product starts with 1.
2. z^1: $a_1 - a_0 = 1 - 1 = 0$.

3. z^2: $a_2 - a_1 - a_0 = 2 - 1 - 1 = 0$, and from here on each term is the sum of the two before it, so the subtraction always cancels.
4. The product is exactly 1: $(1 - z - z^2)F(z) = 1$, which says $F(z) = 1/(1 - z - z^2)$.

```
// ordinary generating functions by convolution:
// (1 - z - z^2) * (1 + z + 2z^2 + 3z^3 + ...) = 1, so every
// coefficient past the constant must vanish: c_n = a_n - a_(n-1) - a_(n-2)
uint64_t a[30];
a[0] = 1;
a[1] = 1;
for (int n = 2; n < 30; n++) {
  a[n] = a[n - 1] + a[n - 2];
}
int fib_unit = (a[0] == 1) && (a[1] - a[0] == 0);
for (int n = 2; n < 30; n++) {
  fib_unit = fib_unit && (a[n] - a[n - 1] - a[n - 2] == 0);
}
CHECK(fib_unit, "(1 - z - z^2) F(z) = 1: all interior coefficients 0");

// (1 - 3z + 2z^2) * (0 + z + 3z^2 + 7z^3 + ...) = z: the hanoi
// series is annihilated except for the single z
uint64_t h[30];
h[0] = 0;
for (int n = 1; n < 30; n++) {
  h[n] = 2 * h[n - 1] + 1;
}
int hanoi_z = (h[0] == 0) && (h[1] - 3 * h[0] == 1);
for (int n = 2; n < 30; n++) {
  hanoi_z = hanoi_z && (h[n] - 3 * h[n - 1] + 2 * h[n - 2] == 0);
}
CHECK(hanoi_z, "(1 - 3z + 2z^2) T(z) = z exactly");

// (1 - z)^2 * (1 + 2z + 3z^2 + ...) = 1: the triangular series is
// the square of the geometric series, c_n = u_n - 2u_(n-1) + u_(n-2)
// with u_j = j + 1
int tri_unit = (2 - 2 * 1 == 0); // c_1 = u_1 - 2 u_0 = 0
for (int n = 2; n < 30; n++) {
  uint64_t un = (uint64_t)(n + 1);
  tri_unit = tri_unit && (un - 2 * (un - 1) + (un - 2) == 0);
}
CHECK(tri_unit, "(1 - z)^2 annihilates 1 + 2z + 3z^2 + ...");
```

listing 152: recurrence.c, three annihilations checked by convolution: fibonacci, hanoi, triangular (source: math/samples/src/Ch14/recurrence.c, lines 138-174)

The sample runs three annihilations, each a loop over exact uint64 coefficients:

series	annihilator	product
fibonacci $1, 1, 2, 3, 5, \ldots$	$1 - z - z^2$	1
hanoi $0, 1, 3, 7, 15, \ldots$	$1 - 3z + 2z^2$	z
triangular $1, 2, 3, 4, \ldots$	$(1 - z)^2$	1

The hanoi identity factors as $T(z) = \frac{1}{1-2z} - \frac{1}{1-z}$, two geometric series whose coefficients 2^n and 1 subtract termwise to $2^n - 1$, the closed form of the previous section read off the clothesline. And the binomial theorem of two sections back is itself a coefficient extraction in this language: $[z^k](1 + z)^n = \binom{n}{k}$, the expansion verified by schoolbook multiplication is exactly a statement that multiplying the series $1 + z$ by itself n times has $\binom{n}{k}$ ways to pick which factors contribute their z.

NOTE: formal, not convergent

nothing here evaluates z. a formal power series is an infinite-tail integer array with finite head, the operations are convolution and shift, and the identity $F(z) = \frac{1}{1-z-z^2}$ means only "the convolution of both sides is 1", which is what the check verifies over 30 coefficients. convergence questions belong to analysis, not to counting.

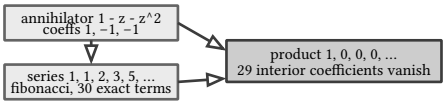


figure 86: convolution as the mechanism: annihilator times series leaves a finite product

The play: one two-term-deep polynomial kills all 29 interior coefficients of the Fibonacci series, and factoring its hanoi counterpart re-derives $2^n - 1$ without touching the recurrence.

14.7. from counting to probability

Everything above counts equally likely outcomes. Probability starts when the outcomes carry weights: the mml book's section 6.1 attaches to each event A a number $P(A) \in [0, 1]$ with $P(\Omega) = 1$ [printed p 175 / pdf p 181], and on a finite space with uniform weights $P(A) = |A| \;/\; |\Omega|$, which is why the dice table in the first sample is already a probability computation. Its section 6.2.1 defines the joint probability of two discrete variables as exactly a count ratio, $P(X = x_i, Y = y_j) = n_{ij}/N$ eq (6.9) [printed p 178 / pdf p 184], and the marginalization is the sum rule over a row or column of the table.

Two fair dice make the whole story a 6×6 grid, $N = 36$: the diagonal $\sum = 7$ holds 6 pairs and the half-plane $\sum \geq 10$ also holds 6, so $P(\sum = 7) = P(\sum \geq 10) = \frac{6}{36}$, a tie between one exact value and a tail event that only counting sees.

```c
// joint table by counting, mml eq (6.9) P = n_ij / N: two fair
// dice, N = 36 pairs, the sum column is a diagonal count
uint64_t sum7 = 0;
uint64_t sum_ge_10 = 0;
for (int i = 1; i ≤ 6; i++) {
  for (int j = 1; j ≤ 6; j++) {
    sum7 += (i + j == 7);
    sum_ge_10 += (i + j ≥ 10);
  }
}
CHECK(sum7 == 6, "P(sum = 7) = 6/36 by scan");
```

listing 153: counting.c, the joint table of two dice as counts over N = 36, mml eq 6.9 (source: math/samples/src/Ch14/counting.c, lines 105-115)

The mml book's two fundamental rules are the weighted image of the two counting rules. The sum rule eq (6.20) $p(x) = \sum_{y \in \mathcal{Y}} p(x, y)$ marginalizes a joint table, the weighted sum rule. The product rule eq (6.22) $p(x, y) = p(y \mid x)p(x)$ factorizes a joint, the weighted product rule, both stated in section 6.3, eq (6.20) [printed p 184 / pdf p 190]. Dividing one product-rule reading by the other gives Bayes' theorem eq (6.23):

$$\underbrace{p(x \mid y)}_{\text{posterior}} = \frac{\overbrace{p(y \mid x)}^{\text{likelihood}}\overbrace{p(x)}^{\text{prior}}}{\underbrace{p(y)}_{\text{evidence}}}$$

The dry run: a two-urn table small enough to hold in your head. Urn A holds 2 red and 1 blue, urn B holds 1 red and 2 blue, the urn is picked fair, and the draw comes up red.

1. Joint for A: $\frac{1}{2} \cdot \frac{2}{3} = \frac{2}{6}$. Joint for B: $\frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$.
2. Evidence: $\frac{2}{6} + \frac{1}{6} = \frac{3}{6}$, the sum rule over the two hypotheses.
3. Posterior: $p(A|\text{red}) = \left(\frac{2}{6}\right)/\left(\frac{3}{6}\right) = \frac{2}{3}$, computed in exact integer rationals, unreduced $\frac{72}{108}$, cross-multiplied against $\frac{2}{3}$ (check 16).

```c
// bayes on a finite table (mml eq 6.23): urn A holds 2 red 1 blue,
// urn B holds 1 red 2 blue, the urn is fair, the draw is red.
// posterior P(A|red) = (1/2 * 2/3) / (1/2 * 2/3 + 1/2 * 1/3) = 2/3
frac prior_a = {.num = 1, .den = 2};
frac prior_b = {.num = 1, .den = 2};
frac lik_red_a = {.num = 2, .den = 3};
frac lik_red_b = {.num = 1, .den = 3};
frac joint_a = frac_mul(prior_a, lik_red_a);
frac joint_b = frac_mul(prior_b, lik_red_b);
frac post_a = frac_div(joint_a, frac_add(joint_a, joint_b));
// unreduced 72/108, cross-multiplied against 2/3
CHECK(post_a.num * 3 == post_a.den * 2, "bayes posterior P(A|red) = 2/3");
```

listing 154: recurrence.c, bayes eq 6.23 on the two-urn table as exact rational arithmetic (source: math/samples/src/Ch14/recurrence.c, lines 176-187)

The play: the posterior is $\frac{2}{3}$, three exact integers, where the same table in floating point would print 0.6666666666666666 and hide that it is a ratio of small counts. This is the whole bridge: book 2, ch 18 (mathematics for programmers, probability and distributions) builds the continuous theory on top of these finite tables, and every distribution it introduces is a weighting scheme over a counted space.

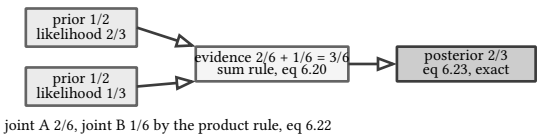


figure 87: bayes on the two-urn table, joints by the product rule, evidence by the sum rule

sources: mml-book draft 2024-01-15, ch 6, sec 6.1 construction of a probability space [printed pp 172-178 / pdf pp 178-184] including the sample space, event space and probability definitions and the two coin toss omega [printed p 175 / pdf p 181], example 6.1 funfair draws with lookup table eqs (6.1)-(6.4) and pmf eqs (6.5)-(6.6) [printed p 176 / pdf p 182], eq (6.7) and pre-image eq (6.8) [printed p 177 / pdf p 183], sec 6.2.1 discrete probabilities eq (6.9) [printed p 178 / pdf p 184], and sec 6.3 sum rule, product rule, bayes eqs (6.20), (6.22) [printed p 184 / pdf p 190], (6.23) [printed p 185 / pdf p 191], all read from the verified anchor sheet ref/mml/anchors-prob.md. Concrete Mathematics, Graham, Knuth, Patashnik, 2nd ed 1994, ch 5 binomial coefficients, ch 6 special numbers, ch 7 generating functions, cited by name. Rosen, Discrete Mathematics and Its Applications, 8th ed 2019, ch 6, cited by name. cppreference, c integer constants, en.cppreference.com/w/c/language/integer_constant.html, wb and uwb suffixes, fetched 2026-09-21. Clang 23 language extensions, `_BitInt` section, clang.llvm.org/docs/LanguageExtensions.html, fetched 2026-09-21. Expected values computed with python exact integers in the one-off playground/math-ch14/gen.py. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch14`, 60 checks in chapter 14 of the math suite, probed on this machine 2026-09-21.

15. graphs and order theory

Graphs are the bare structure of things and their connections, and this chapter works the theory a programmer actually leans on: the models and degree identities, walks and connectivity, the tree characterization, partial orders with hasse diagrams and topological sort, lattices, and equivalence relations as partitions. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 15 or a sentence quoted from a canonical source fetched 2026-09-21. Where the sibling treatment is algorithmic, bfs, dfs, shortest paths, flows in book 9, ch 10 (practical data structures and algorithms, graphs), book 9, ch 11 (practical data structures and algorithms, shortest paths), and book 9, ch 12 (practical data structures and algorithms, minimum spanning trees and flows), this chapter keeps to structure and proof, and cross references the algorithm when it appears.

15.1. graph models and degree sums

A graph $G = (V, E)$ is a vertex set and an edge set of unordered pairs, and the first programming decision is the encoding. The adjacency matrix holds an $n \times n$ grid with $A_{ij} = 1$ exactly when $ij \in E$, an edge test in constant time at n^2 cells of storage. The adjacency list holds one neighbor set per vertex, $\sum_v \deg(v)$ slots in total, and a row bitmask is the same list as one machine word per vertex when $n \le 64$. The degree $\deg(v)$ is the number of edge ends at v, and counting edge ends two ways gives the handshake lemma:

$\sum_{v \in V} \deg(v) = 2|E|$

Each edge contributes one end at each of its two vertices, so the degree column of any real graph adds up to an even number, twice the edge count. The consequence worth quoting: a finite graph has an even number of odd-degree vertices, since the sum is even. For a digraph the same argument split by direction gives $\sum_v \deg^+(v) = \sum_v \deg^-(v) = |E|$, out-degrees and in-degrees both summing to the arc count (en.wikipedia.org/wiki/Handshaking_lemma, fetched 2026-09-21).

The dry run: G1 is the undirected fixture of this chapter, 6 vertices and 8 edges, (0,1), (0,2), (1,2), (1,3), (2,4), (3,4), (3,5), (4,5).

1. Degrees read off the edges: $d(0) = 2$, $d(1) = 3$, $d(2) = 3$, $d(3) = 3$, $d(4) = 3$, $d(5) = 2$, so the vector is (2,3,3,3,3,2).
2. The handshake: $2 + 3 + 3 + 3 + 3 + 2 = 16 = 2 \cdot 8$. Four vertices carry odd degree 3, an even count, as the corollary demands.
3. The matrix costs 36 cells, the bitmask 6 words holding 16 set bits between them. That 16 is $2|E|$ again, the lemma as a storage number.

vertex	neighbors	degree	bitmask
0	{1, 2}	2	0x06
1	{0, 2, 3}	3	0x0D
2	{0, 1, 4}	3	0x13
3	{1, 4, 5}	3	0x32
4	{2, 3, 5}	3	0x2C
5	{3, 4}	2	0x18

```
int main(void) {
  // three encodings of G1 built independently: matrix from edges, row
  // bitmask from edges, degrees from the matrix
  unsigned char a[G1_N][G1_N] = {0};
  unsigned char nbr[G1_N] = {0};
  for (int e = 0; e < G1_M; e++) {
    int u = g1_edges[e][0], v = g1_edges[e][1];
    a[u][v] = 1;
    a[v][u] = 1;
    nbr[u] = (unsigned char)(nbr[u] | 1u << v);
    nbr[v] = (unsigned char)(nbr[v] | 1u << u);
  }
  int deg[G1_N];
  for (int i = 0; i < G1_N; i++) {
    deg[i] = 0;
    for (int j = 0; j < G1_N; j++) {
      deg[i] += a[i][j];
    }
  }

  // gen.py: G1 degrees: [2, 3, 3, 3, 3, 2]
  static const int deg_pinned[G1_N] = {2, 3, 3, 3, 3, 2};
  bool deg_ok = true;
  for (int i = 0; i < G1_N; i++) {
    deg_ok = deg_ok && deg[i] == deg_pinned[i] && popcnt(nbr[i]) == deg[i];
  }
  CHECK(deg_ok, "degrees [2,3,3,3,3,2] agree across matrix and bitmask");

  // gen.py: degree sum 16 = 2*m = 2*8 (handshake lemma)
  int dsum = 0;
  for (int i = 0; i < G1_N; i++) {
    dsum += deg[i];
  }
  CHECK(dsum == 2 * G1_M, "handshake: degree sum 16 == 2m");

  // gen.py: four vertices carry odd degree 3, an even count
  int odd = 0;
  for (int i = 0; i < G1_N; i++) {
    odd += deg[i] % 2;
  }
  CHECK(odd == 4 && odd % 2 == 0, "odd-degree vertices: 4, even");
```

listing 155: models.c builds the edge list, the matrix, and the bitmask independently and checks them against the degree vector (source: math/samples/src/Ch15/models.c, lines 65-105)

The matrix also counts walks. The entry $(A^k)_{ij}$ of the k-th matrix power is the number of walks of length k from i to j, because a k-walk chooses a neighbor per step and the product sums exactly those choices. Two entries carry the chapter: the diagonal of A^2 is the degree vector, a walk out and back per edge end, and the trace of A^3 counts closed 3-walks, six per triangle, two rotations times three starting vertices. On G1 that gives $\operatorname{tr}(A^3) = 12$ and $12/6 = 2$ triangles, 012 and 345, with the total of 2-walks $44 = \sum_v \deg(v)^2 = 4 + 9 + 9 + 9 + 9 + 4$.

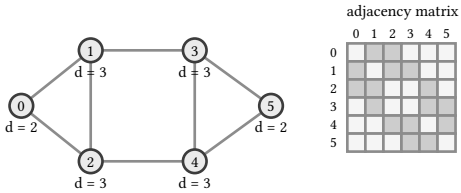


figure 88: G1 with its degrees and adjacency matrix, 36 cells holding 8 edges

NOTE: PICK THE ENCODING BY THE OPERATION

The matrix answers "is ij an edge" in one array lookup and costs n^2 cells, 36 for G1. The bitmask answers "list the neighbors of i" by scanning set bits and costs n words holding $2|E|$ set bits, 16 for G1. Matrix powers count walks, list scans find neighborhoods, and the degree column is free in both.

15.2. walks, paths, and connectivity

A walk repeats nothing forbidden, a path repeats no vertex, and reachability is the closure question: can j be reached from i by some walk. The transitive closure of the adjacency relation answers it for all pairs at once, and warshall's triple loop computes it in place: whenever i already reaches k, row i absorbs row k. Reachability is reflexive by convention, every vertex reaches itself by the empty walk, so the closure matrix starts from $A \cup I$.

The dry run: D1 is the directed fixture, 8 vertices and 8 arcs, $0 \to 1 \to 2 \to 0$ a cycle, $2 \to 3$ a bridge, $3 \to 4 \to 5 \to 3$ a second cycle, $6 \to 7$ a tail.

1. Out-degrees run (1,1,2,1,1,1,1,0) and in-degrees (1,1,1,2,1,1,0,1), both summing to 8, the directed handshake.
2. Closure row weights come out (6,6,6,3,3,3,2,1): each of 0, 1, 2 reaches all six of the left block plus the right, each of 3, 4, 5 reaches only its own 3-cycle, 6 reaches {6,7}, 7 only itself.
3. That is 30 ones including the 8 reflexive loops, so 22 strict reachable ordered pairs.

Two component notions split here. A digraph is strongly connected when every vertex reaches every other, and the strong components are the equivalence classes of mutual reachability, $i \sim j$ when i reaches j and j reaches i, a reflexive symmetric transitive relation by construction. Weak components ignore direction: take the underlying undirected graph and count connected pieces. On D1 the closure gives 4 strong components, {0,1,2}, {3,4,5}, {6}, {7}, while the undirected projection has 2 weak components, {0..5} and {6,7}. The gap between 4 and 2 is the whole reason the distinction exists, and the efficient linear-time algorithms for both live in book 9, ch 10 (practical data structures and algorithms, graphs) and book 9, ch 39 (practical data structures and algorithms, graph connectivity and decomposition), this chapter only pins the definitions on a fixture small enough to close by hand.

```
// reachability closure of D1 by warshall, rows as bitmasks: reflexive
// start, then row i absorbs row k whenever i already reaches k
unsigned char reach[D1_N] = {0};
for (int i = 0; i < D1_N; i++) {
  reach[i] = (unsigned char)(outm[i] | 1u << i);
}
for (int k = 0; k < D1_N; k++) {
  for (int i = 0; i < D1_N; i++) {
    if ((reach[i] >> k) & 1) {
      reach[i] = (unsigned char)(reach[i] | reach[k]);
    }
  }
}
// gen.py: closure row weights [6,6,6,3,3,3,2,1], 30 ones total, so 22
// non-reflexive reachable ordered pairs
static const int reach_pinned[D1_N] = {6, 6, 6, 3, 3, 3, 2, 1};
bool reach_ok = true;
int ones = 0;
for (int i = 0; i < D1_N; i++) {
  reach_ok = reach_ok && popcnt(reach[i]) == reach_pinned[i];
  ones += popcnt(reach[i]);
}
CHECK(reach_ok && ones == 30 && ones - D1_N == 22,
      "closure rows 6,6,6,3,3,3,2,1: 30 pairs, 22 strict");

// strong components are the mutual-reachability classes: two 3-cycles
// plus the singletons 6 and 7
// gen.py: scc labels [0,0,0,1,1,1,2,3]
static const int scc_pinned[D1_N] = {0, 0, 0, 1, 1, 1, 2, 3};
int scc[D1_N];
for (int i = 0; i < D1_N; i++) {
  scc[i] = -1;
}
int nscc = 0;
for (int i = 0; i < D1_N; i++) {
  if (scc[i] ≥ 0) {
    continue;
  }
  for (int j = i; j < D1_N; j++) {
    if (((reach[i] >> j) & 1) && ((reach[j] >> i) & 1)) {
      scc[j] = nscc;
    }
  }
  nscc++;
}
bool scc_ok = nscc == 4;
for (int i = 0; i < D1_N; i++) {
  scc_ok = scc_ok && scc[i] == scc_pinned[i];
}
CHECK(scc_ok, "4 strong components: {0,1,2}, {3,4,5}, {6}, {7}");
```

listing 156: models.c runs warshall on D1 and reads the strong components off mutual rows (source: math/samples/src/Ch15/models.c, lines 182-231)

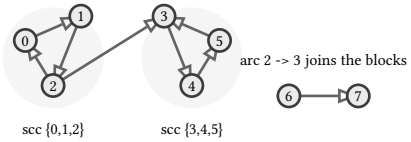


figure 89: D1: two 3-cycles, a bridge arc, and a tail, 4 strong components under 2 weak ones

PITFALL: STRONG AND WEAK ARE DIFFERENT QUESTIONS

The same fixture D1 answers 4 to "how many strong components" and 2 to "how many weak components". Both numbers are correct and neither implies the other: the tail 6 -> 7 is one weak component and two strong ones, while a single directed cycle is one of each. State which connectivity you mean before counting.

15.3. trees

A tree is a connected acyclic graph, and on finite graphs the definitions collapse into equivalences: every two vertices are joined by a unique path, and a tree is minimally connected and maximally acyclic (Diestel, Graph Theory, 5th ed., theorem 1.5.1, cited by name). The counting forms carry the same content, connected with exactly $n - 1$ edges and acyclic with exactly $n - 1$ edges (Rosen, 8th ed., ch 11.1, cited by name). The chapter proves the useful direction by counting: a connected graph on n vertices needs at least $n - 1$ edges to escape isolation, and an acyclic graph admits at most $n - 1$ before a cycle closes, so connected plus acyclic pins the count exactly, and then every tree with $n \geq 2$ has at least two leaves, strip them one at a time and the degree sum $2(n - 1)$ cannot live on one leaf alone.

A spanning tree of G is a subgraph that is a tree on all of V, and G has a spanning tree exactly when G is connected. Counting them is a different game. Kirchhoff's matrix-tree theorem says the number of spanning trees equals any cofactor of the laplacian $\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{A}$, degree matrix minus adjacency matrix (en.wikipedia.org/wiki/Kirchhoff%27s_theorem, fetched 2026-09-21), and for the complete graph K_n the cofactor evaluates to n^{n-2}, cayley's formula, 16 spanning trees of K_4.

The dry run: enumerate all $\binom{8}{5} = 56$ edge subsets of G1 and keep the connected ones.

1. 30 of the 56 subsets connect all 6 vertices, so G1 has 30 spanning trees by enumeration.
2. The kirchhoff cofactor of G1 is also 30, one 5×5 determinant agreeing with 56 connectivity tests.
3. The lexicographically first tree uses edges (0,1), (0,2), (1,3), (2,4), (3,5), mask 0x5B, and leaves the 3 chords (1,2), (3,4), (4,5): $m - n + 1 = 3$.
4. Rooted at 0 with smallest-first bfs that tree gives parent (-,0,0,1,2,3) and depth (0,1,1,2,2,3), degree sum 10 = 2(n-1), leaves {4, 5}.

```
[[nodiscard]] static long long det_bareiss(int n, long long m[8][8]) {
  long long prev = 1;
  int sign = 1;
  for (int k = 0; k < n - 1; k++) {
    if (m[k][k] == 0) {
      int swap = -1;
      for (int i = k + 1; i < n && swap < 0; i++) {
        if (m[i][k] != 0) {
          swap = i;
        }
      }
      if (swap < 0) {
        return 0;
      }
      for (int j = 0; j < n; j++) {
        long long t = m[k][j];
        m[k][j] = m[swap][j];
        m[swap][j] = t;
      }
      sign = -sign;
    }
    for (int i = k + 1; i < n; i++) {
      for (int j = k + 1; j < n; j++) {
        _BitInt(64) x = (_BitInt(64))m[i][j] * (_BitInt(64))m[k][k] -
                        (_BitInt(64))m[i][k] * (_BitInt(64))m[k][j];
        m[i][j] = (long long)(x / (_BitInt(64))prev);
      }
      m[i][k] = 0;
    }
    prev = m[k][k];
  }
  return (long long)sign * m[n - 1][n - 1];
```

listing 157: trees.c takes the cofactor determinant by exact bareiss pivots in 64-bit bitint arithmetic (source: math/samples/src/Ch15/trees.c, lines 98-129)

```
int main(void) {
  // gen.py: subsets of 8 edges taken 5 at a time: C(8,5) = 56
  int subsets = 0, trees = 0, min_leaves = N, max_leaves = 0;
  bool all_two_leaves = true, all_acyclic = true;
  unsigned first_tree = 0;
  for (unsigned subset = 0; subset < (1u << M); subset++) {
    if (popcnt(subset) != N - 1) {
      continue;
    }
    subsets++;
    if (!connected(N, subset, edges)) {
      continue;
    }
    trees++;
    all_acyclic = all_acyclic && acyclic(N, subset, edges);
    if (trees == 1) {
      first_tree = subset;
    }
    int deg[N];
    for (int i = 0; i < N; i++) {
      deg[i] = 0;
    }
    for (unsigned s = subset; s != 0; s &= s - 1) {
      int e = __builtin_ctz(s);
      deg[edges[e][0]]++;
      deg[edges[e][1]]++;
    }
    int leaves = 0;
    for (int i = 0; i < N; i++) {
      leaves += deg[i] == 1;
    }
    all_two_leaves = all_two_leaves && leaves >= 2;
    if (leaves < min_leaves) {
      min_leaves = leaves;
    }
    if (leaves > max_leaves) {
      max_leaves = leaves;
    }
  }
  CHECK(subsets == 56, "C(8,5) = 56 candidate subsets");
  CHECK(trees == 30, "enumeration: 30 spanning trees");
  // gen.py: kirchhoff cofactor of G1 is 30
  CHECK(spanning_tree_count(N, M, edges) == 30,
        "kirchhoff cofactor also 30, two methods agree");
  CHECK(all_acyclic,
        "connected + n-1 edges forces acyclic: all 30 pass the cycle test");
  CHECK(all_two_leaves, "every spanning tree has at least 2 leaves");
  // gen.py: leaf counts over the 30 spanning trees range 2..4
  CHECK(min_leaves == 2 && max_leaves == 4, "leaf counts range from 2 to 4");
```

listing 158: trees.c enumerates the 56 candidate subsets and pins the 30 spanning trees with leaf statistics (source: math/samples/src/Ch15/trees.c, lines 152-200)

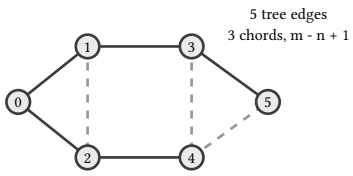


figure 90: G1 with the pinned spanning tree solid and its 3 chords dashed

15.4. partial orders

A relation $\leq$ on a set is a partial order when it is reflexive, antisymmetric, and transitive. Antisymmetry is the property that breaks ties: $x \leq y$ and $y \leq x$ together force $x = y$, so parity classes or any equivalence relation with two distinct related elements fails it. Three fixed relations pin the predicate grid, $\leq$ on {0,1,2,3} a partial order, same-parity an equivalence that fails antisymmetry, differs-by-1 symmetric but not transitive.

The hasse diagram is the disciplined picture of a finite poset: draw only cover pairs, $x < y$ with no z strictly between, edge upward, and leave loops and transitive edges implied. For the chapter's running poset, the divisors of 12 under divisibility, the relation holds 18 pairs including the 6 reflexive loops, and exactly 7 of them are covers, 1 to 2, 1 to 3, 2 to 4, 2 to 6, 3 to 6, 4 to 12, 6 to 12.

Minimal and maximal are local, minimum and maximum are global. A minimal element has nothing below it, a minimum sits below everything. The divisor poset has minimum 1 and maximum 12, unique in both roles. The V shaped subposet {1, 2, 3} has two maximal elements, 2 and 3, and no maximum at all, no element sits above both.

A topological order of a poset is a linear listing respecting $<$, and it always exists: kahn's algorithm repeatedly removes a minimal element, and a finite poset always has one, so the loop cannot stall (en.wikipedia.org/wiki/Topological_sorting, fetched 2026-09-21). The digraph view says the same thing, a topological order exists exactly when the digraph is acyclic, and a cycle is precisely a stall, no element of the cycle ever becomes minimal. Counting the orders is harder: the divisor poset of 12 admits exactly 5 linear extensions.

The dry run: kahn with a smallest-first tie break on the divisor poset.

1. Initial minimal set {1}. Place 1, then 2 and 3 become minimal, take 2, then 3, then 4 and 6 are minimal, take 4, then 6, then 12.
2. The pinned order is 1, 2, 3, 4, 6, 12, all 6 elements placed.
3. Add the single back arc $12 < 1$ and every vertex has a predecessor: kahn places 0 of 6. One arc deletes the entire order structure.

```
// gen.py: kahn smallest-first order [1,2,3,4,6,12]
int order[8];
int placed = kahn(6, div12, order);
bool kahn_ok = placed == 6;
static const int order_pinned[6] = {1, 2, 3, 4, 6, 12};
for (int i = 0; i < 6; i++) {
  kahn_ok = kahn_ok && vals[order[i]] == order_pinned[i];
}
CHECK(kahn_ok, "kahn places 1,2,3,4,6,12");

// add the pair 12 < 1: every vertex gains a blocker, kahn stalls at 0
unsigned char cyclic[8];
for (int i = 0; i < 6; i++) {
  cyclic[i] = div12[i];
}
cyclic[5] = (unsigned char)(cyclic[5] | 1u);
CHECK(kahn(6, cyclic, order) == 0,
      "one back arc 12<1 and kahn places 0 of 6");

// gen.py: 5 linear extensions
bool done[8] = {false};
int extensions = 0;
extend(6, div12, done, 0, &extensions);
CHECK(extensions == 5, "5 linear extensions of the divisor poset");
```

listing 159: order.c pins the kahn order, the cyclic stall, and the 5 linear extensions (source: math/samples/src/Ch15/order.c, lines 339-362)

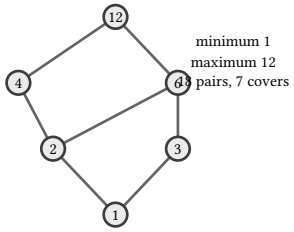


figure 91: hasse diagram of the divisors of 12, upward means divides, 7 covers carry 18 pairs

WARNING: DRAW ONLY COVERS

The hasse discipline deletes two kinds of edge: the 6 reflexive loops, and every transitive pair like 1 to 12 that a two-step path already encodes. Draw either and the diagram stops being a poset picture, it starts looking like a dataflow graph. Of the divisor poset's 18 pairs, exactly 7 earn ink.

15.5. lattices

A lattice is a poset where every pair has a least upper bound, the join $x \vee y$, and a greatest lower bound, the meet $x \wedge y$. Both are defined by bounds, not formulas: the join of x and y is the upper bound with no other upper bound below it, unique when it exists because two least upper bounds bound each other and antisymmetry collapses them. The divisor poset of 12 is a lattice where join is lcm and meet is gcd, $4 \vee 6 = 12$ and $4 \wedge 6 = 2$, verified by enumerating bounds, and then cell by cell across all 36 pairs.

The dry run: compute $4 \vee 6$ from bounds alone.

1. Upper bounds of 4 and 6 inside the fixture: common multiples, only 12. The least upper bound is 12 with no rival.
2. Lower bounds: common divisors 1 and 2, and 2 sits above 1, so the greatest lower bound is 2.

3. The same two loops over all 36 pairs produce tables identical to lcm and gcd, an arithmetic agreement check on a structural definition.

Not every poset is a lattice. The V subposet $\{1, 2, 3\}$ under divisibility gives 2 and 3 no common upper bound at all, the join does not exist, while the meet $2 \wedge 3 = 1$ survives. And a lattice can still fail the stronger law. Distributivity,

$$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z),$$

holds in every divisor lattice, 216 triples checked on the fixture with zero failures. The pentagon N_5, the lattice shaped as 0 below a chain $0 < x < 1$ beside a chain $0 < y < z < 1$, keeps every join and meet but breaks distributivity on the witness triple: $y \vee (x \wedge z) = y \vee 0 = y$ against $(y \vee x) \wedge (y \vee z) = 1 \wedge z = z$, and $y \neq z$.

```
// a bound search result: the anonymous union carries the element index
// when the bound exists, and nothing meaningful when it does not
struct bound {
  bool exists;
  union {
    unsigned char idx;
  };
};

[[nodiscard]] static struct bound lub(int n, const unsigned char rel[8], int i,
                                      int j) {
  struct bound r = {.exists = false, .idx = 0};
  unsigned ups = 0;
  for (int k = 0; k < n; k++) {
    if (((rel[i] >> k) & 1) && ((rel[j] >> k) & 1)) {
      ups |= 1u << k;
    }
  }
  int found = 0;
  unsigned char least = 0;
  for (unsigned s = ups; s != 0; s &= s - 1) {
    int u = __builtin_ctz(s);
    bool is_least = true;
    for (unsigned t = ups; t != 0 && is_least; t &= t - 1) {
      int v = __builtin_ctz(t);
      if (v != u && ((rel[v] >> u) & 1)) {
        is_least = false; // another upper bound sits strictly below u
      }
    }
    if (is_least) {
      found++;
      least = (unsigned char)u;
    }
  }
  if (found == 1) {
    r.exists = true;
    r.idx = least;
  }
  return r;
}
```

listing 160: order.c searches bounds for the least upper bound, the anonymous union carrying the index when it exists (source: math/samples/src/Ch15/order.c, lines 192-231)

```
// gen.py: every pair has a unique least upper and greatest lower bound
bool all_bounds = true;
for (int i = 0; i < 6; i++) {
  for (int j = 0; j < 6; j++) {
    all_bounds = all_bounds && lub(6, div12, i, j).exists &&
                 glb(6, div12, i, j).exists;
  }
}
CHECK(all_bounds, "36 pairs, 36 joins and 36 meets: a lattice");

// gen.py: 4 join 6 = 12, 4 meet 6 = 2, the lcm and gcd in person
struct bound j46 = lub(6, div12, 3, 4);
struct bound m46 = glb(6, div12, 3, 4);
CHECK(j46.exists && vals[j46.idx] == 12 && m46.exists && vals[m46.idx] == 2,
      "join(4,6) = 12 and meet(4,6) = 2");

bool tables_ok = true;
for (int i = 0; i < 6; i++) {
  for (int j = 0; j < 6; j++) {
    int l = vals[i] * vals[j] / gcd(vals[i], vals[j]);
    tables_ok = tables_ok && lub(6, div12, i, j).idx == idx_of(l) &&
                glb(6, div12, i, j).idx == idx_of(gcd(vals[i], vals[j]));
  }
}
CHECK(tables_ok, "join table is lcm, meet table is gcd, cell by cell");

bool distributive = true;
for (int i = 0; i < 6; i++) {
  for (int j = 0; j < 6; j++) {
    for (int k = 0; k < 6; k++) {
      int lhs = lub(6, div12, i, glb(6, div12, j, k).idx).idx;
      int rhs =
          glb(6, div12, lub(6, div12, i, j).idx, lub(6, div12, i, k).idx).idx;
      distributive = distributive && lhs == rhs;
    }
  }
}
CHECK(distributive,
      "x join (y meet z) = (x join y) meet (x join z), 216 triples");

// gen.py: in the V poset, join(2,3) does not exist: {1} is the only
// candidate lower structure and no common upper bound is in sight
CHECK(!lub(3, vpos, 1, 2).exists, "V poset: 2 and 3 have no join");
```

listing 161: order.c checks all 36 joins and meets, the gcd and lcm agreement, distributivity, and the V poset failure (source: math/samples/src/Ch15/order.c, lines 364-406)

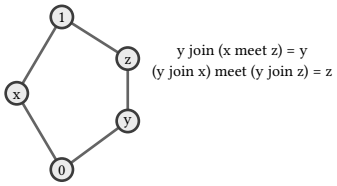


figure 92: the pentagon N5: every pair joins and meets, distributivity fails on the witness triple

15.6. equivalence relations and partitions

An equivalence relation is reflexive, symmetric, and transitive, and it is the same object as a partition, a family of nonempty disjoint blocks covering the set. From a partition, “same block” is an equivalence. From an equivalence, the classes $[x] = \{y : x \sim y\}$ form blocks, and transitivity is exactly what keeps classes from overlapping half of one and half of another. The round trip, partition to relation to partition, returns the starting partition, which the sample checks pairwise on a fixed 3-block partition of {0..7}.

A relation that is merely reflexive and symmetric still generates an equivalence: take the transitive closure, the same warshall loop as the reachability run of this chapter. The fixture holds the 8 loops plus 0 tilde 1 and 1 tilde 2 in both directions, 12 ordered pairs, and fails transitivity because 0 tilde 2 is missing in both directions. Closure adds exactly those 2 pairs, the block {0,1,2} contributes 9 pairs plus 5 singleton loops for 14 total, and the induced partition is one 3-element block with 5 singletons.

Union-find maintains a partition under merging, and the theory says what its invariant is: at every moment the structure encodes exactly one equivalence relation, find(i) = find(j) iff $i \sim j$. The sample drives a fixed sequence of 9 unions over 10 singletons with union by size and path halving, and after every call rescans to confirm the recorded size at each root equals the live count of its elements.

The dry run: the sequence (0,1), (2,3), (0,2), (4,5), (6,7), (5,7), (4,6), (0,4), (8,9).

1. Merged component sizes run 2, 2, 4, 2, 2, 4, 8, 2, and the seventh call, union(4,6), finds both ends already in the size-4 component {4,5,6,7} and merges nothing: 9 unions, 8 merges, 1 no-op.
2. The final partition is {0..7} of size 8 and {8,9} of size 2, two components, class labels by least element reading 0 eight times then 8 twice.

```
  // partition {0,3,6}, {1,4}, {2,5,7} as a class label per element
  const int cls[NP] = {0, 1, 2, 0, 1, 2, 0, 2};
  unsigned char rel[NP] = {0};
  for (int i = 0; i < NP; i++) {
    for (int j = 0; j < NP; j++) {
      if (cls[i] == cls[j]) {
        rel[i] = (unsigned char)(rel[i] | 1u << j);
      }
    }
  }

  CHECK(reflexive(NP, rel) && symmetric(NP, rel) && transitive(NP, rel),
        "same-block relation is reflexive, symmetric, transitive");

  unsigned char blocks[8];
  int nblocks = partition_of(NP, rel, blocks);
  // gen.py: block masks {0,3,6} = 0x49, {1,4} = 0x12, {2,5,7} = 0xA4
  static const unsigned char blocks_pinned[3] = {0x49, 0x12, 0xA4};
  bool blocks_ok = nblocks == 3;
  for (int b = 0; b < 3; b++) {
    blocks_ok = blocks_ok && blocks[b] == blocks_pinned[b];
  }
  CHECK(blocks_ok, "3 blocks recovered: masks 0x49, 0x12, 0xA4");

  // rebuild the relation from the recovered blocks: the round trip
  unsigned char again[NP] = {0};
  for (int b = 0; b < nblocks; b++) {
    for (int i = 0; i < NP; i++) {
      for (int j = 0; j < NP; j++) {
        if (((blocks[b] >> i) & 1) && ((blocks[b] >> j) & 1)) {
          again[i] = (unsigned char)(again[i] | 1u << j);
        }
      }
    }
  }
  bool round_trip = true;
  for (int i = 0; i < NP; i++) {
    round_trip = round_trip && again[i] == rel[i];
  }
  CHECK(round_trip, "relation -> partition -> relation is the identity");
```

listing 162: partitions.c checks the induced equivalence and the round trip on the 3-block fixture (source: math/samples/src/Ch15/partitions.c, lines 120-159)

```
  // fixed 9-union sequence over 10 elements
  static const int unions[9][2] = {{0, 1}, {2, 3}, {0, 2}, {4, 5}, {6, 7},
                                   {5, 7}, {4, 6}, {0, 4}, {8, 9}};
  struct uf u;
  for (int i = 0; i < NU; i++) {
    u.parent[i] = i;
    u.size[i] = 1;
  }
  static const int merged_pinned[8] = {2, 2, 4, 2, 2, 4, 8, 2};
  int merges = 0, noops = 0, mi = 0;
  bool sizes_ok = true, invariant_ok = true;
  for (int s = 0; s < 9; s++) {
    int a = unions[s][0], b = unions[s][1];
    if (uf_union(&u, a, b)) {
      // gen.py: merged component sizes 2,2,4,2,2,4,8,2
      sizes_ok =
          sizes_ok && mi < 8 && u.size[uf_find(&u, a)] == merged_pinned[mi];
      mi++;
      merges++;
    } else {
      noops++;
    }
    // invariant: every root's recorded size equals its live component
    for (int r = 0; r < NU; r++) {
      if (uf_find(&u, r) == r) {
        int live = 0;
        for (int i = 0; i < NU; i++) {
          live += uf_find(&u, i) == r;
        }
        invariant_ok = invariant_ok && u.size[r] == live;
      }
    }
  }
  CHECK(merges == 8 && noops == 1, "9 unions: 8 merges, union(4,6) a no-op");
  CHECK(sizes_ok, "merged sizes run 2,2,4,2,2,4,8,2");
  CHECK(invariant_ok, "root size equals component size after every union");
```

listing 163: partitions.c drives the 9 unions with the size invariant rescanned after each (source: math/samples/src/Ch15/partitions.c, lines 195-230)

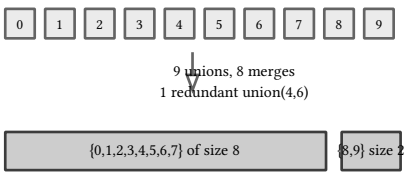


figure 93: the fixed union sequence: 10 singletons collapse into components of size 8 and 2

VERIFY: AUDIT THE PARTITION, NOT THE POINTERS

Union-find code is easy to write and easy to corrupt: path halving rewrites parents, sizes live only at roots. The cheap audit is structural, after every mutation rescan and confirm each root's recorded size equals the number of elements that find to it, and confirm the implied relation is still reflexive, symmetric, transitive. Two loops over 10 elements, run 9 times in the sample.

Equivalence classes of elements under a group action are the next chapter's opening move, and book 2, ch 16 (mathematics for programmers, groups and subgroups) builds them on exactly this partition machinery.

sources: Rosen, Discrete Mathematics and Its Applications, 8th ed., McGraw-Hill 2019, ch 10.1-10.2 (graph models, degree sums), ch 11.1 (trees), ch 9.1-9.6 (relations, hasse diagrams, equivalence classes), cited by name and edition, no page text quoted; Diestel, Graph Theory, 5th ed., Springer 2017, theorem 1.5.1, cited by name; CLRS, Introduction to Algorithms, 4th ed., MIT Press 2022, ch 20.4, cited by name. Fetched and quoted 2026-09-21: en.wikipedia.org/wiki/Handshaking_lemma (degree sum formula, directed variant, odd-vertex corollary), en.wikipedia.org/wiki/Kirchhoff%27s_theorem (laplacian cofactor, cayley n^{n-2}), en.wikipedia.org/wiki/Topological_sorting (kahn existence iff acyclic). No mml anchor sheet exists for graph theory, so this chapter has no mml-book page mapping, and every numeric claim was computed rather than transcribed. Probed on this machine the same day. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch15`, 60 checks in chapter 15 of the math suite, 4 files, zero failures, format clean.

16. groups and subgroups

A group is a set with one operation that never leaves you stranded: every product stays inside, the identity is there, and every element can be undone. This chapter treats those axioms the way this book treats everything, as executable claims: closure, associativity, identity, and inverses become scan loops over fixed tables, first on Z_6 addition, the units mod 8, and integer matrices with determinant ± 1, then on the residue classes Z_n and their multiplicative units $U(n)$ where order and φ meet, then subgroups generated by a single element with the divisor-shaped lattice of Z_{12}, then cyclic groups and which elements generate them, then cosets and the theorem of Lagrange that an order must divide an order, and finally the two non-abelian families this book actually uses, the six symmetries of a triangle (S_3) and the matrix group $\mathrm{SL}_2(Z)$. Every behavioral claim below is one of the 58 checks in the 4 samples of chapter 16 or a sentence quoted from a canonical source fetched 2026-09-21. Where book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) runs modular exponentiation and primality as algorithms, this chapter is the theory underneath: why those residue sets are groups at all and what divides what.

16.1. the axioms as a check suite

Definition first. A group is a pair $(G, \cdot)$ of a set and a binary operation such that all four axioms hold: closure, $a \cdot b \in G$ for all $a, b \in G$, associativity, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$, an identity e with $e \cdot a = a \cdot e = a$, and an inverse a^{-1} with $a \cdot a^{-1} = e$, for every a. The group is abelian when moreover $a \cdot b = b \cdot a$. These are definition 2.7 of the mml-book draft 2024-01-15, sec 2.4.1 [printed pp 36-37 / pdf pp 42-43], whose example 2.10 runs the same recognition game, $(Z, +)$ a group while $(N_0, +)$ and $(Z, \cdot)$ both lose exactly on inverses. The four axioms are exactly four loop nests, which is why they make a good first sample: nothing about a group needs floating point, every question is a scan over a finite table or an exact integer identity.

The three fixtures of this section: Z_6, the residues $0..5$ under addition mod 6, identity 0, inverse of k is $(6 - k) \bmod 6$, so the pinned inverse pairs are $(1,5)$, $(2,4)$, $(3,3)$. The units mod 8, $U(8) = \{1, 3, 5, 7\}$ under multiplication mod 8, where every element is its own inverse, $3 \cdot 3 = 9 = 1$, and the pinned row of the table through 3 is $\{3, 1, 7, 5\}$ because $3 \cdot 5 = 15 = 7$ and $3 \cdot 7 = 21 = 5 \bmod 8$. And two matrices from $\mathrm{SL}_2(Z)$, the 2 by 2 integer matrices of determinant 1: the quarter turn $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$ and the shear $T = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$.

The dry run: the matrix pair shows why the determinant condition is the right one, and then the same table machinery kills three near-miss structures.

1. Determinants multiply: $\det(S) = 0 \cdot 0 - (-1) \cdot 1 = 1$ and $\det(T) = 1$, so $\det(ST) = 1 \cdot 1 = 1$, closure on the nose.
2. But the product is not symmetric: $ST = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 1 \end{smallmatrix}\right)$ while $TS = \left(\begin{smallmatrix} 1 & -1 \\ 1 & 0 \end{smallmatrix}\right)$, two different integer matrices, so this family is already non-abelian.
3. Inverses come from the adjugate: $S^{-1} = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$ and $T^{-1} = \left(\begin{smallmatrix} 1 & -1 \\ 0 & 1 \end{smallmatrix}\right)$, both integer matrices because $\det = \pm 1$ means dividing by the determinant divides by 1.

```
// ── sl2(z) entry: rotation s and shear t, both det 1 ─────────────
const mat2 s = {.e = {{0, -1}, {1, 0}}}; // 90 degree rotation
const mat2 t = {.e = {{1, 1}, {0, 1}}};  // shear
// pinned: det(s) = 0*0 - (-1)*1 = 1, det(t) = 1*1 - 1*0 = 1
auto ds = det2(s);
auto dt = det2(t);
CHECK(ds == 1 && dt == 1, "det s = 1, det t = 1");

// closure: s.t = [[0,-1],[1,1]] with det 1 = det(s)det(t)
const mat2 st = mat_mul(s, t);
bool st_pin = st.e[0][0] == 0 && st.e[0][1] == -1 && st.e[1][0] == 1 &&
              st.e[1][1] == 1 && det2(st) == 1;
CHECK(st_pin, "s.t pinned [[0,-1],[1,1]], det 1 = det s * det t");

// non-abelian: t.s = [[1,-1],[1,0]], a different matrix
const mat2 ts = mat_mul(t, s);
bool ts_pin =
    ts.e[0][0] == 1 && ts.e[0][1] == -1 && ts.e[1][0] == 1 && ts.e[1][1] == 0;
CHECK(ts_pin && !mat_eq(st, ts),
      "t.s pinned [[1,-1],[1,0]], differs from s.t");

// inverse via adjugate: s^-1 = [[0,1],[-1,0]], t^-1 = [[1,-1],[0,1]]
const mat2 s_inv = adjugate(s);
const mat2 t_inv = adjugate(t);
bool s_inv_pin = s_inv.e[0][0] == 0 && s_inv.e[0][1] == 1 &&
                 s_inv.e[1][0] == -1 && s_inv.e[1][1] == 0;
bool t_inv_pin = t_inv.e[0][0] == 1 && t_inv.e[0][1] == -1 &&
                 t_inv.e[1][0] == 0 && t_inv.e[1][1] == 1;
CHECK(s_inv_pin && mat_eq(mat_mul(s, s_inv), I2),
      "s^-1 = adjugate [[0,1],[-1,0]], s s^-1 = i");
CHECK(t_inv_pin && mat_eq(mat_mul(t, t_inv), I2),
      "t^-1 = adjugate [[1,-1],[0,1]], t t^-1 = i");
```

listing 164: axioms.c, the sl2(z) entry: determinants multiply, the product order matters, and adjugate gives integer inverses exactly when det is plus or minus 1 (source: math/samples/src/Ch16/axioms.c, lines 141-172)

The near-miss table is the other half of the definition. A structure can pass three axioms and die on the fourth, and each failure mode below is a check:

structure	closure	assoc	identity	inverses
Z_6, add	yes	yes	0	yes
$U(8)$, mult	yes	yes	1	yes
det ± 1 matrices	yes	yes	I	yes
$Z \setminus \{0\}$, mult	yes	yes	1	no
Z, subtract	yes	no	no	n/a
all of Z_{10}, mult	yes	yes	1	no

```
// ── near-miss 1: integers under subtraction ──────────────────────
// associativity fails: (1-2)-3 = -4 but 1-(2-3) = 2
CHECK((1 - 2) - 3 == -4 && 1 - (2 - 3) == 2 && (1 - 2) - 3 != 1 - (2 - 3),
      "subtraction near-miss: (1-2)-3 = -4 vs 1-(2-3) = 2");

// ── near-miss 2: nonzero integers under multiplication ──────────────
// closure, associativity, identity 1 all hold, but 2 has no integer
// inverse: no x in 1..40 with 2x = 1
bool found_inv = false;
for (int64_t x = 1; x <= 40; x++) {
  found_inv = found_inv || 2 * x == 1;
}
CHECK(!found_inv, "nonzero integers under mult: 2 has no inverse");

// ── near-miss 3: all residues mod 10 under multiplication ───────────
// a monoid: 2x = 1 (mod 10) has no solution because gcd(2,10) = 2
bool found_mod_inv = false;
for (uint64_t x = 0; x < 10; x++) {
  found_mod_inv = found_mod_inv || 2 * x % 10 == 1;
}
CHECK(!found_mod_inv, "mod 10 monoid: 2 has no inverse, gcd(2,10)=2");
```

listing 165: axioms.c, the three near-misses: subtraction breaks associativity at (1-2)-3 = −4 versus 2, and 2 has no multiplicative inverse over the integers or mod 10 (source: math/samples/src/Ch16/axioms.c, lines 174-195)

PITFALL: associativity is the axiom you cannot eyeball

every structure above looks plausible, and the one that fails hardest is the one people reach for first: subtraction. the pin is one line, $(1-2)-3 = -4$ while $1-(2-3) = 2$, but exponentiation by repeated squaring, composition chains, and fold order all inherit this axiom silently. when you define an operation in code, scan all triples before you build on it, checks 1 and 5 of axioms.c do exactly that on 216 and 64 triples.

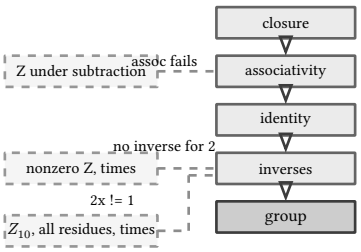


figure 94: the axiom gates: four checks in sequence, three near-miss structures dropping out where they fail

16.2. z mod n and its units

The residue classes Z_n carry two operations at once. Under addition mod n, Z_n is a group immediately: closure is reduction, associativity inherits from the integers, the identity is 0, and $n-k$ inverts k. Under multiplication mod n, all residues form only a monoid, a group minus inverses: $2x \equiv 1 \bmod 10$ has no solution because $\gcd(2,10) = 2$ divides any multiple of 2, so 2 never lands on 1. Deleting the zero divisors repairs it: the units $U(n) = \{k : \gcd(k,n) = 1\}$ are exactly the invertible residues, a group under multiplication mod n, and its order is the totient, $|U(n)| = \varphi(n)$ (Wikipedia, "multiplicative group of integers modulo n", fetched 2026-09-21: "an abelian, finite group whose order is given by Euler's totient function").

The pinned totals for this chapter's fixtures: $\varphi(5) = 4$, $\varphi(8) = 4$, $\varphi(9) = 6$, $\varphi(10) = 4$, $\varphi(12) = 4$, each confirmed by a coprime scan that also counts $|U(n)|$, checks 8 and 9 of units.c.

The dry run: inside $U(10) = \{1, 3, 7, 9\}$, take powers of 3.

1. $3^1 = 3$, then $3^2 = 9$, then $3^3 = 27 = 7 \bmod 10$, then $3^4 = 21 = 1 \bmod 10$: the ladder is 3, 9, 7, 1 and the order of 3 is 4.
2. Powers of 9 die faster: $9^2 = 81 = 1 \bmod 10$, ladder 9, 1, order 2.
3. Every order divides the group order 4: the pinned orders are 1, 4, 4, 2 for the elements 1, 3, 7, 9, and $1 \mid 4$, $2 \mid 4$, $4 \mid 4$. That divisibility is Lagrange knocking, five sections early.

element of u(10)	ladder	order	divides 4
1	1	1	yes
3	3, 9, 7, 1	4	yes
7	7, 9, 3, 1	4	yes
9	9, 1	2	yes

```
// ladders pinned: powers of 3 are 3, 9, 7, 1 (order 4, a generator),
// powers of 9 are 9, 1 (order 2)
const uint64_t lad3[4] = {3, 9, 7, 1};
const uint64_t lad9[2] = {9, 1};
bool l3 = true;
uint64_t x = 1;
for (int i = 0; i < 4; i++) {
  x = x * 3 % 10;
  l3 = l3 && x == lad3[i];
}
bool l9 = true;
x = 1;
for (int i = 0; i < 2; i++) {
  x = x * 9 % 10;
  l9 = l9 && x == lad9[i];
}
CHECK(l3 && order_mod(3, 10) == 4, "ladder of 3 pinned 3,9,7,1, order 4");
CHECK(l9 && order_mod(9, 10) == 2, "ladder of 9 pinned 9,1, order 2");

// lagrange preview: every element order divides |u(10)| = 4,
// pinned orders 1 (for 1), 4, 4, 2 (for 3, 7, 9)
const uint64_t orders[4] = {1, 4, 4, 2}; // for u = {1,3,7,9}
bool div_ok = true;
for (int a = 0; a < 4; a++) {
  div_ok = div_ok && order_mod(u[a], 10) == orders[a] && 4 % orders[a] == 0;
}
CHECK(div_ok, "orders in u(10) pinned {1,4,4,2}, all divide 4");
```

listing 166: units.c, the ladders of 3 and 9 mod 10 pinned step by step, with every order checked against divisibility by 4 (source: math/samples/src/Ch16/units.c, lines 122-148)

The order observations generalize to the two classic theorems, both of them one-line corollaries of the group structure once Lagrange lands. Euler: for any unit a mod n, $a^{\varphi(n)} \equiv 1 \bmod n$, pinned here as $3^4 = 81 \equiv 1 \bmod 10$ and $5^4 = 625 \equiv 1 \bmod 12$. Fermat: for prime p and $a \neq 0$, $a^{p-1} \equiv 1 \bmod p$, since $U(p)$ is all of $\{1, \ldots, p-1\}$ and $\varphi(p) = p - 1$. The prime fixture is $3^{100} \equiv 1 \bmod 101$, sharpened to the exact statement that 3 generates all of $U(101)$: its order is 100, with $3^{50} = 100 \equiv -1 \bmod 101$ sitting at the halfway point.

```
// ── euler and fermat ──────────────────────────────────────────
// euler: a^phi(n) = 1 mod n for a unit. pinned 3^4 = 81 = 1 mod 10
CHECK(powmod(3, phi(10), 10) == 1, "euler: 3^phi(10) = 3^4 = 1 mod 10");

// pinned: 5^4 = 625 = 1 mod 12 (gcd(5,12) = 1, phi(12) = 4)
CHECK(powmod(5, phi(12), 12) == 1, "euler: 5^4 = 625 = 1 mod 12");

// fermat: for prime p, a^(p-1) = 1 mod p. pinned 2^4 = 16 = 1 mod 5
CHECK(powmod(2, 4, 5) == 1, "fermat: 2^(5-1) = 16 = 1 mod 5");

// pinned: 3^100 = 1 mod 101, and the order of 3 mod 101 is exactly 100
// with 3^50 = 100 = -1 mod 101 (3 is a primitive root mod 101)
constexpr uint64_t p101 = 101;
CHECK(powmod(3, 100, p101) == 1, "fermat: 3^100 = 1 mod 101");
CHECK(order_mod(3, p101) == 100 && powmod(3, 50, p101) == 100,
      "ord_101(3) = 100, 3^50 = -1 mod 101");
```

listing 167: units.c, euler and fermat pins: two small ladders and the primitive root 3 mod 101 with order exactly 100 (source: math/samples/src/Ch16/units.c, lines 173-188)

VERIFY: the algorithm lives in the dsa book

the samples pin powers with a 12-line square-and-multiply loop over uint64, widths stated in the file (moduli capped at 101 so squares fit). that loop as an algorithm, with its role in fast exponentiation, primality testing, and RSA-style arithmetic, is chapter material of book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) and book 9, ch 28 (practical data structures and algorithms, advanced number theory). this chapter only needs its outputs: exact orders, exact divisibility.

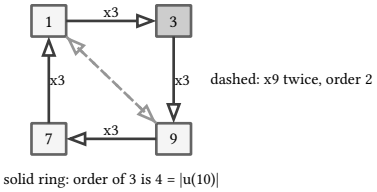


figure 95: the power cycles of u(10): multiplying by 3 walks a 4-cycle, the sub-ladder of 9 collapses to a 2-cycle

16.3. subgroups and the subgroup one element generates

A subgroup H of G is a subset that is a group under the same operation: it must contain e, and be closed under products and inverses. For a finite set inside a group the test collapses to nonempty plus closure under products, because repeated products cycle and pull the identity and inverses out on their own. The cheapest source of subgroups is a single element: $\langle g \rangle = \{g, g^2, g^3, \ldots\}$, the cyclic subgroup generated by g, is closed by construction, and the order of g, written $\mathrm{ord}(g)$, is defined as $|\langle g \rangle|$, the first exponent that returns to e (Wikipedia, "cyclic group", fetched 2026-09-21: the order of an element "is equal to the order of the cyclic subgroup that it generates").

Z_{12} is the fixture because its subgroups are exactly one per divisor. The dry run: repeated addition of a single element, the subgroup it generates pinned run by run:

1. $\langle 4 \rangle = \{0, 4, 8\}$: three steps back to 0, order 3.
2. $\langle 3 \rangle = \{0, 3, 6, 9\}$: four steps, order 4.
3. $\langle 2 \rangle = \{0, 2, 4, 6, 8, 10\}$: six steps, order 6.
4. $\langle 1 \rangle$ is all of Z_{12}, order 12, and the full pinned size row for $k = 0..11$ is 1, 12, 6, 4, 3, 12, 2, 12, 3, 4, 6, 12, which is $12/\gcd(12, k)$ entry by entry.

k in z12	⟨k⟩	$\lvert\langle k\rangle\rvert$	$12/\gcd(12,k)$
0	{0}	1	1
1	all 12	12	12
2	{0,2,4,6,8,10}	6	6
3	{0,3,6,9}	4	4
4	{0,4,8}	3	3
6	{0,6}	2	2

```
// ── pinned subgroups ──────────────────────────────────────────
// <2> = {0,2,4,6,8,10} (mask 0x555), <3> = {0,3,6,9} (0x249),
// <4> = {0,4,8} (0x111), <6> = {0,6} (0x41)
CHECK(gen_sub(2) == 0x555, "<2> pinned {0,2,4,6,8,10} mask 0x555");
CHECK(gen_sub(3) == 0x249, "<3> pinned {0,3,6,9} mask 0x249");
CHECK(gen_sub(4) == 0x111 && gen_sub(6) == 0x41,
      "<4> pinned {0,4,8}, <6> pinned {0,6}");

// ── generators of z12 ─────────────────────────────────────────
// pinned: exactly k = 1, 5, 7, 11 generate the whole group, count phi(12) = 4
bool gen_pin = true;
const bool expect_gen[12] = {false, true, false, false, false, true,
                             false, true, false, false, false, true};
for (uint64_t k = 0; k < n12; k++) {
  gen_pin = gen_pin && (gen_sub(k) == 0xFFF) == expect_gen[k] &&
            expect_gen[k] == (gcd_u64(k, n12) == 1);
}
CHECK(gen_pin, "generators of z12 pinned {1,5,7,11} = coprime residues");

int gen_count = 0;
for (uint64_t k = 0; k < n12; k++) {
  if (gen_sub(k) == 0xFFF) {
    gen_count++;
  }
}
CHECK(gen_count == 4, "generator count 4 = phi(12)");
```

listing 168: cyclic.c, the pinned subgroup masks of z12 as bit patterns, then the generator scan: exactly the coprime residues 1, 5, 7, 11 cover the whole group (source: math/samples/src/Ch16/cyclic.c, lines 76-101)

Two structural facts fall out of the same scan, both CHECKed. First, the 12 runs $\langle k\rangle$ collapse to exactly 6 distinct subsets, one subgroup per divisor $d \in \{1, 2, 3, 4, 6, 12\}$, and the subgroup of order d is $\langle 12/d\rangle$: $|\langle 12/d\rangle| = d$ pinned on all six divisors. Second, every subgroup of a cyclic group is cyclic, visibly here: each of the 6 is one of the runs. The lattice picture is worth holding onto, it reappears as the divisor order of book 2, ch 15 (mathematics for programmers, graphs and order theory) dressed as a Hasse diagram, and the same divisibility shape governs which subgroups exist in every cyclic group you will meet in book 2, ch 17 (mathematics for programmers, group actions and structure).

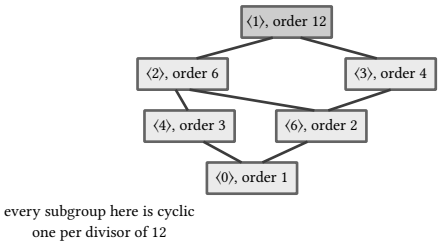


figure 96: the subgroup lattice of z12, one subgroup per divisor of 12, edges are containment

16.4. cyclic groups and generator search

A group is cyclic when one element reaches everything: $G = \langle g \rangle$ for some generator g. Every cyclic group of order n is a copy of Z_n (Wikipedia, "cyclic group": "every finite cyclic group of order n is isomorphic to the additive group of Z/nZ"), so generator search is a question about residues, and it has a complete answer: a modular integer k generates Z_n exactly when $\gcd(k, n) = 1$, and the number of generators is $\varphi(n)$. In Z_{12} the scan above found 1, 5, 7, 11, four of them, and $\varphi(12) = 4$.

The multiplicative side is where it gets interesting, because $U(n)$ may or may not be cyclic. The classification is classical, first proved by Gauss: $U(n)$ is cyclic exactly for $n = 1, 2, 4, p^k, 2p^k$ with p an odd prime (Wikipedia, "multiplicative group of integers modulo n", fetched 2026-09-21). The dry run: the fixtures walk both sides of the classification:

1. $U(5)$, order 4, cyclic: the ladder of 2 is 2, 4, 3, 1, all four units.
2. $U(7)$, order 6, cyclic: the ladder of 3 is 3, 2, 6, 4, 5, 1, all six units, prime modulus so $|U(p)| = p - 1$.
3. $U(9)$, order $\varphi(9) = 6$, cyclic: the ladder of 2 is 2, 4, 8, 7, 5, 1, a prime power.
4. $U(8)$ and $U(12)$, order 4 each, not cyclic: every nonidentity element squares to 1, the pinned Klein pattern, so no ladder can visit all 4.

The order rule that makes ladders predictable, again from the cyclic group page: the order of an element m in Z_n is $n / \gcd(n, m)$, and in any cyclic group of order n with generator g, $\operatorname{ord}(g^k) = n / \gcd(n, k)$. In $U(7)$ with $g = 3$, $n = 6$: $3^2 = 2$ has order $6 / \gcd(6, 2) = 3$, pinned ladder 2, 4, 1, and $3^3 = 6$ has order $6 / \gcd(6, 3) = 2$, ladder 6, 1.

3^k in u(7)	element	ord	$6 / \gcd(6, k)$
$k = 1$	3	6	6
$k = 2$	2	3	3
$k = 3$	6	2	2
$k = 4$	4	3	3
$k = 5$	5	6	6
$k = 6$	1	1	1

```
  const uint64_t lad7[6] = {3, 2, 6, 4, 5, 1};
  bool cyc7 = true;
  x = 1;
  for (int i = 0; i < 6; i++) {
    x = x * 3 % 7;
    cyc7 = cyc7 && x == lad7[i];
  }
  CHECK(cyc7, "u(7) cyclic: ladder of 3 pinned 3,2,6,4,5,1");

  // ── ord(g^k) = n / gcd(n,k), pinned in u(7) with g = 3, n = 6 ──────
  // in u(7): 3^2 = 2 has order 3 = 6/gcd(6,2), 3^3 = 6 has order 2 =
  // 6/gcd(6,3), 3^4 = 4 has order 3 = 6/gcd(6,4)
  bool rule = true;
  for (uint64_t k = 1; k ≤ 6; k++) {
    uint64_t elem = 1;
    for (uint64_t i = 0; i < k; i++) {
      elem = elem * 3 % 7;
    }
    rule = rule && order_mod(elem, 7) == 6 / gcd_u64(6, k);
  }
  CHECK(rule && order_mod(2, 7) == 3 && order_mod(6, 7) == 2,
        "ord(3^k) = 6/gcd(6,k) pinned: ord 2 = 3, ord 6 = 2");

  // ── u(12) is the contrast: order 4 but not cyclic ──────────────────
  // pinned: |u(12)| = 4 yet no element reaches order 4 (all squares 1)
  bool none_cyclic = true;
  for (uint64_t k = 1; k < 12; k++) {
    if (is_unit(k, 12)) {
      none_cyclic = none_cyclic && order_mod(k, 12) ≠ 4;
    }
  }
  CHECK(none_cyclic, "u(12) order 4 but no generator, not cyclic");

  // ── u(9): a prime power is cyclic ──────────────────────────────────
  // pinned: powers of 2 mod 9 are 2, 4, 8, 7, 5, 1, all phi(9) = 6 units
  const uint64_t lad9[6] = {2, 4, 8, 7, 5, 1};
  bool cyc9 = true;
  x = 1;
  for (int i = 0; i < 6; i++) {
    x = x * 2 % 9;
    cyc9 = cyc9 && x == lad9[i];
  }
  CHECK(cyc9, "u(9) cyclic: ladder of 2 pinned 2,4,8,7,5,1");
```

listing 169: cyclic.c, u(7) cyclic with the ladder of 3, the order rule ord(g^k) = n/gcd(n,k) verified per power, then the klein contrasts u(12) and the prime power u(9) (source: math/samples/src/Ch16/cyclic.c, lines 159-201)

PITFALL: same order, different group

$|U(5)| = |U(8)| = |U(12)| = 4$, yet u(5) is a 4-cycle and u(8), u(12) are klein groups where everything squares to 1. group order alone does not determine the group, which is why the samples pin ladders, not just counts. the same warning runs forward: two-element subgroups are all alike, but the two groups of order 4 are genuinely different machines, and burnside counting in book 2, ch 17 (mathematics for programmers, group actions and structure) will care which one it is looking at.

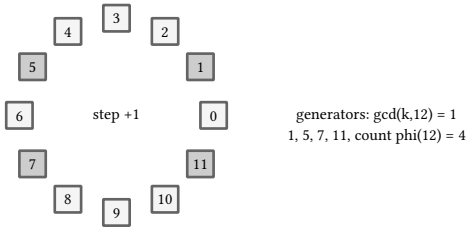


figure 97: the z12 residue wheel, the four coprime residues shaded as the generators, count phi(12) = 4

16.5. order, cosets, and lagrange

The theorem this chapter exists for. For a subgroup H of a finite group G, the left cosets are the translates $gH = \{g \cdot h : h \in H\}$, and Lagrange's theorem states the partition arithmetic: $|G| = [G : H] \cdot |H|$, where $[G : H]$ is the number of distinct cosets (Wikipedia, "Lagrange's theorem (group theory)", fetched 2026-09-21: "if H is a subgroup of any finite group G, then |H| is a divisor of |G|. That is, the order (number of elements) of every subgroup divides the order of the whole group"). The proof is three short steps and worth doing, because it is the model for every counting argument in book 2, ch 17 (mathematics for programmers, group actions and structure).

1. Every coset has size exactly $|H|$: the map $h \to g \cdot h$ is a bijection $H \to gH$, with inverse $gh \to h$ computed by multiplying on the left by g^{-1}.
2. Distinct cosets are disjoint: if $g_1H \cap g_2H$ is nonempty, pick $g_1h_1 = g_2h_2$, then $g_1 = g_2h_2h_1^{-1}$ forces $g_1H = g_2H$. So the cosets "form a partition of G", equal-size classes of the equivalence relation $x \sim y$ iff $x = yh$ for some $h \in H$.
3. Count the partition: $|G|$ elements split into $[G : H]$ classes of $|H|$ each, so $|G| = [G : H] \cdot |H|$ and $|H|$ divides $|G|$.

The dry run: the additive cosets of $\langle 3 \rangle = \{0, 3, 6, 9\}$ inside Z_{12}, translation instead of multiplication because the group is additive.

1. $0 + \langle 3 \rangle = \{0, 3, 6, 9\}$, the subgroup itself.
2. $1 + \langle 3 \rangle = \{1, 4, 7, 10\}$ and $2 + \langle 3 \rangle = \{2, 5, 8, 11\}$, two fresh sets.
3. $3 + \langle 3 \rangle$ lands back on the first coset, so the run stops: exactly 3 cosets of size 4, tiling all 12 residues with no overlap, $12 = 3 \cdot 4$.

coset of ⟨3⟩ in z12	members	size
$0 + \langle 3 \rangle$	{0, 3, 6, 9}	4
$1 + \langle 3 \rangle$	{1, 4, 7, 10}	4
$2 + \langle 3 \rangle$	{2, 5, 8, 11}	4

```
  // ── additive cosets of <3> in z12 ──────────────────────────────────
  // pinned: <3> = {0,3,6,9}, 1+<3> = {1,4,7,10}, 2+<3> = {2,5,8,11},
  // three cosets of size 4 that tile z12, index [z12:<3>] = 3
  uint16_t c0 = gen_sub(3);
  uint16_t c1 = 0, c2 = 0;
  for (int b = 0; b < 12; b++) {
    if ((c0 & (1u << b)) != 0) {
      c1 |= (uint16_t)(1u << ((b + 1) % 12));
      c2 |= (uint16_t)(1u << ((b + 2) % 12));
    }
  }
  bool z12_pin = c0 == 0x249 && c1 == 0x492 && c2 == 0x924;
  bool z12_tile = (c0 | c1 | c2) == 0xFFF && (c0 & c1) == 0 && (c0 & c2) == 0 &&
                  (c1 & c2) == 0;
  CHECK(z12_pin && z12_tile, "z12/<3> cosets pinned 0x249 0x492 0x924, tile");
```

listing 170: cosets.c, the z12 coset construction from the subgroup bitmask, pinned as 0x249, 0x492, 0x924, and checked to tile the 12 residues exactly (source: math/samples/src/Ch16/cosets.c, lines 267-281)

The corollary with teeth: since $\langle g \rangle$ is a subgroup for every element, $\mathrm{ord}(g) = |\langle g \rangle|$ divides $|G|$, and consequently $g^{|G|} = e$ for every element of a finite group (the Lagrange page again: “If the group has n elements, it follows $a^n = e$”). Euler and Fermat from the second section are this corollary in disguise: $U(n)$ has $\varphi(n)$ elements, so every unit satisfies $a^{\varphi(n)} = 1$, and prime moduli give $a^{p-1} = 1$. Gauss proved exactly that case, the multiplicative group mod a prime, in the Disquisitiones Arithmeticae of 1801, decades before the general theorem.

```c
// ── lagrange fixtures across all three families ──────────────────
// fixture 1: every subgroup order divides 12, |<k>| = 12/gcd(12,k)
bool lag_z12 = true;
for (uint64_t k = 0; k < 12; k++) {
  uint16_t m = gen_sub(k);
  int cnt = 0;
  for (int b = 0; b < 12; b++) {
    cnt += (m >> b) & 1;
  }
  lag_z12 = lag_z12 && 12 % (uint64_t)cnt == 0 &&
            (uint64_t)cnt == 12 / gcd_u64(12, k);
}
CHECK(lag_z12, "lagrange in z12: every |<k>| divides 12");

// fixture 2: every element order in u(10) divides 4, pinned 1, 4, 4, 2
const uint64_t u10[4] = {1, 3, 7, 9};
const uint64_t u10_orders[4] = {1, 4, 4, 2};
bool lag_u10 = true;
for (int a = 0; a < 4; a++) {
  uint64_t x = u10[a] % 10;
  uint64_t ord = 1;
  while (x != 1) {
    x = x * u10[a] % 10;
    ord++;
  }
  lag_u10 = lag_u10 && ord == u10_orders[a] && 4 % ord == 0;
}
CHECK(lag_u10, "lagrange in u(10): orders 1,4,4,2 all divide 4");

// fixture 3: coset arithmetic, |G| = [G:H] * |H| holds in both families
CHECK(6 == 3 * 2 && 12 == 3 * 4, "|G| = [G:H]|H|: 6 = 3x2 and 12 = 3x4");
```

listing 171: cosets.c, the lagrange fixtures: every subgroup order in z12 divides 12, every element order in u(10) divides 4, and both coset identities hold, 6 = 3 x 2 and 12 = 3 x 4 (source: math/samples/src/Ch16/cosets.c, lines 283-313)

NOTE: the converse is false

lagrange says every subgroup order divides the group order, not that every divisor is a subgroup order. the standard witness is A_4, the 12 even permutations of four labels: it has no subgroup of order 6, though $6 \mid 12$. the samples never test the converse, they only test the divisibility direction that the theorem guarantees.

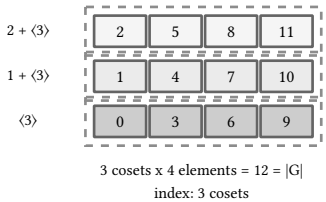


figure 98: the coset tiling of z12 by ⟨3⟩: three disjoint translates of size 4 covering all 12 residues

16.6. permutations, matrices, and the triangle

The two families that make the abstraction pay. Permutations of n labels form the symmetric group S_n under function composition, order $n!$, and at $n = 3$ it is small enough to table and big enough to be non-abelian: D_3, the six symmetries of an equilateral triangle, “is isomorphic to the symmetric group S3” and “is also the smallest non-abelian group” (Wikipedia, “dihedral group of order 6”, fetched 2026-09-21). The samples encode a permutation

as the triple of images $(p(0), p(1), p(2))$, compose as $a \circ b$ meaning apply b first, $(a \circ b)(x) = a[b[x]]$, and check the whole package on the six elements e, $r = (012)$, $r^2 = (021)$, and the three swaps s01, s02, s12.

The dry run: two compositions that refuse to commute, plus the order of each element.

1. s01 $\circ$ r applies the rotation then the swap: r sends $0 \to 1$, $1 \to 2$, $2 \to 0$, and the swap exchanges 0 and 1, so the composite fixes 0 and exchanges 1 and 2, giving $(0, 2, 1) =$ s12.
2. $r \circ$ s01 swaps first, then rotates: 0 goes $0 \to 1 \to 2$, 1 goes $1 \to 0 \to 1$, 2 goes $2 \to 2 \to 0$, giving $(2, 1, 0) =$ s02, a different element, so the group is non-abelian.
3. Orders: $\mathrm{ord}(e) = 1$, $\mathrm{ord}(r) = \mathrm{ord}(r^2) = 3$, every swap has order 2, and each of 1, 2, 3 divides $|S_3| = 6$, Lagrange verified element by element.

```c
// a after b: (a o b)(x) = a[b[x]]
[[nodiscard]] static perm compose(perm a, perm b) {
  perm r;
  for (int i = 0; i < 3; i++) {
    r.map[i] = a.map[b.map[i]];
  }
  return r;
}

[[nodiscard]] static bool perm_eq(perm a, perm b) {
  return a.map[0] == b.map[0] && a.map[1] == b.map[1] && a.map[2] == b.map[2];
}

// order of a permutation: first power back to the identity
[[nodiscard]] static int perm_order(perm p) {
  perm x = p;
  int k = 1;
  const perm e = {.map = {0, 1, 2}};
  while (!perm_eq(x, e)) {
    x = compose(x, p);
    k++;
  }
  return k;
}
```

listing 172: cosets.c, the permutation representation: composition as a after b over image triples, and element order as repeated composition back to the identity (source: math/samples/src/Ch16/cosets.c, lines 40-64)

The coset machinery from the last section now runs on a non-abelian group, and the partitions it finds are the two splittings of S_3 worth remembering. For $H = \{e, \mathrm{s01}\}$: the three left cosets are $\{e, \mathrm{s01}\}$, $\{r, \mathrm{s02}\}$, and $\{\mathrm{s12}, r^2\}$, each of size 2, tiling the 6 elements exactly, $6 = 3 \cdot 2$. For the rotation subgroup $\langle r \rangle = \{e, r, r^2\}$: exactly 2 cosets of size 3, and the second one is all three reflections at once, rotations versus flips, $6 = 2 \cdot 3$. The same sample file closes with the trap that gives Lagrange its fine print: $\mathrm{SL}_2(Z)$, the determinant-1 integer matrices, is an infinite group, the integer echo of the general linear group of mml definition 2.8 [printed p 37 / pdf p 43], and inside it the quarter turn $S = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ satisfies $S^2 = -I$, $S^4 = I$, so $\mathrm{ord}(S) = 4$ while no finite $|G|$ exists to divide. Finiteness is a hypothesis of the theorem, not a decoration.

```
// ── left cosets of H = {e, s01} ──────────────────────────────────
// pinned: gH gives exactly 3 distinct cosets {e,s01}, {r,s02}, {s12,r2},
// each of size 2, and every element lands in exactly one coset
const perm h2[2] = {e, s01};
uint16_t coset_mask[6] = {0};
for (int g = 0; g < 6; g++) {
  for (int h = 0; h < 2; h++) {
    perm gh = compose(s3[g], h2[h]);
    for (int m = 0; m < 6; m++) {
      if (perm_eq(gh, s3[m])) {
        coset_mask[g] |= (uint16_t)(1u << m);
      }
    }
  }
}
// dedupe: cosets of e, r, r2 are the three representatives
int distinct = 0;
uint16_t reps[6] = {0};
for (int g = 0; g < 6; g++) {
  bool dup = false;
  for (int j = 0; j < distinct; j++) {
    dup = dup || reps[j] == coset_mask[g];
  }
  if (!dup) {
    reps[distinct++] = coset_mask[g];
  }
}
CHECK(distinct == 3, "H = {e,s01} has exactly 3 distinct left cosets");
```

listing 173: cosets.c, the left coset construction on s3: translate the subgroup by every element, deduplicate, and land on exactly 3 distinct cosets (source: math/samples/src/Ch16/cosets.c, lines 193-220)

```
// ── the infinity trap: s has order 4 inside an infinite group ────
// in sl2(z) every element still has a finite or infinite order, but
// lagrange says nothing: the group is infinite. pinned: the 90 degree
// rotation s satisfies s^2 = -i, s^3 = -s, s^4 = i, and no smaller
// positive power is the identity
const mat2 s = {.e = {{0, -1}, {1, 0}}};
const mat2 neg_i = {.e = {{-1, 0}, {0, -1}}};
const mat2 i2 = {.e = {{1, 0}, {0, 1}}};
const mat2 s2 = mat_mul(s, s);
const mat2 s3p = mat_mul(s2, s);
const mat2 s4 = mat_mul(s2, s2);
CHECK(mat_eq(s2, neg_i) && !mat_eq(s, i2) && !mat_eq(s3p, i2) &&
          mat_eq(s4, i2),
      "sl2(z): s^2 = -i, s^4 = i, order 4 in an infinite group");
```

listing 174: cosets.c, the infinity trap: the quarter turn has order 4 inside the infinite group sl2(z), where lagrange has nothing to say (source: math/samples/src/Ch16/cosets.c, lines 315-328)

s3 element	triple	order	inverse
e	(0,1,2)	1	e
r	(1,2,0)	3	r^2
r^2	(2,0,1)	3	r
s01	(1,0,2)	2	s01
s02	(2,1,0)	2	s02

s3 element	triple	order	inverse
s12	(0,2,1)	2	s12

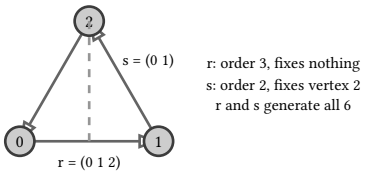


figure 99: the labeled triangle: the rotation r cycles all three vertex labels, the reflection s fixes vertex 2 and swaps 0 with 1, together generating all six symmetries

The vocabulary lands where it was headed: group, order, subgroup, cyclic generator, and cosets partitioning a group are exactly the nouns book 2, ch 17 (mathematics for programmers, group actions and structure) picks up to make a group act on a set, count orbits with Burnside, and build the finite fields the later chapters compute over.

sources: mml-book draft 2024-01-15, sec 2.4.1 groups, definition 2.7 the four axioms with the abelian addendum and example 2.10 groups and non-groups [printed pp 36-37 / pdf pp 42-43], plus definition 2.8 the general linear group [printed p 37 / pdf p 43], anchoring the section 1 definitions and the section 6 matrix family. The coset, cyclic, and permutation structure sections carry no mml anchors and were authored from canonical texts by name and four fetched pages. Dummit and Foote, Abstract Algebra, 3rd ed 2004, ch 1 the axioms and first examples, ch 2 subgroups and cyclic groups, sec 3.1 cosets and Lagrange's theorem, cited by name. Artin, Algebra, 2nd ed 2011, ch 2 groups and the symmetry chapter for the dihedral vocabulary, cited by name. Wikipedia, Lagrange's theorem (group theory), en.wikipedia.org/wiki/Lagrange%27s_theorem_(group_theory), the divisor statement, the $|G| = [G : H] \cdot |H|$ variant, the left-coset partition proof, the element corollary a^n = e, and the A4 counterexample to the converse, fetched 2026-09-21. Wikipedia, cyclic group, en.wikipedia.org/wiki/Cyclic_group, ord(m in Z/nZ) = n/gcd(n,m), generators are the residues coprime to n with count phi(n), the klein U(8), and U(p) cyclic for prime p, fetched 2026-09-21. Wikipedia, multiplicative group of integers modulo n, en.wikipedia.org/wiki/Multiplicative_group_of_integers_modulo_n, |U(n)| = phi(n) and the gauss classification n = 1, 2, 4, p^k, 2p^k, fetched 2026-09-21. Wikipedia, dihedral group of order 6, en.wikipedia.org/wiki/Dihedral_group_of_order_6, D3 isomorphic to S3, the smallest non-abelian group, six transformations of an equilateral triangle, fetched 2026-09-21. Expected values computed with python exact integers in the one-off playground/math-ch16/gen.py. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch16`, 58 checks in chapter 16 of the math suite, probed on this machine 2026-09-21.

17. group actions and structure

Chapter 16 built the nouns: group, order, subgroup, cyclic generator, cosets, Lagrange. This chapter builds the verbs that compare and combine them: homomorphisms and isomorphisms as table-preserving maps, quotients that turn coset partitions into arithmetic, direct products with their lcm order law, rings and fields where a second operation joins, finite fields built from polynomials over Z_2, the dihedral group D_4 acting on a square, Burnside's orbit count on colorings, and a toy Diffie-Hellman exchange whose secrecy is exactly the discrete logarithm. Every behavioral claim below is one of the 61 checks in the 4 samples of chapter 17 or a sentence quoted from a canonical source fetched 2026-09-21. Where book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) and book 9, ch 28 (practical data structures and algorithms, advanced number theory) run modular exponentiation, discrete logs, and Burnside counting as algorithms, this chapter is the structure underneath.

17.1. homomorphisms and isomorphisms

Definition first. A homomorphism $\varphi : G \to H$ is a map with $\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b)$ for all $a, b \in G$: products map to products, so the whole table of G lands in the table of H. An isomorphism is a bijective homomorphism, and two groups are isomorphic when one exists, written $G \sim= H$: same table, different labels. The kernel, $\ker \varphi = \{g \in G : \varphi(g) = e_H\}$, measures how far φ sits from injective.

The pinned fixture is the parity map $\varphi : Z_4 \to Z_2$, $\varphi(k) = k \bmod 2$. It preserves all 16 pairs, its image is all of Z_2, and its kernel is $\{0, 2\}$, which closes under addition mod 4 on its own: $0 + 2 = 2$, $2 + 2 = 0$. A kernel is never an accident, it is always a subgroup, and chapter 2's error vocabulary returns here: the kernel is the null space of the map.

The dry run: same order 4, two different groups, told apart by counting orders.

1. In Z_4 the orders of $0, 1, 2, 3$ are $1, 4, 2, 4$: two elements of order 4.
2. In $Z_2 \times Z_2$ every element satisfies $a \cdot a = e$, so the orders are $1, 2, 2, 2$: no element of order 4.
3. Order is preserved by any isomorphism, since $\varphi(a)^k = \varphi\big(a^k\big)$, so no bijection can carry one census onto the other: Z_4 is not isomorphic to $Z_2 \times Z_2$.

The same census tool reads the unit groups. $U(8) = \{1, 3, 5, 7\}$ has orders $1, 2, 2, 2$, the Klein four group again. $U(10) = \{1, 3, 7, 9\}$ has orders $1, 4, 4, 2$: cyclic, generated by 3. So $U(8)$ and $U(10)$ are not isomorphic despite both having order 4. But $U(12) = \{1, 5, 7, 11\}$ has orders $1, 2, 2, 2$, and the explicit map $1 \to 1$, $3 \to 5$, $5 \to 7$, $7 \to 11$ preserves all 16 products of $U(8)$: $U(8) \sim= U(12)$, witnessed rather than suggested.

group	order	order census	verdict
Z_4	4	1, 4, 2, 4	cyclic
$Z_2 \times Z_2$	4	1, 2, 2, 2	klein
$U(8)$	4	1, 2, 2, 2	klein
$U(10)$	4	1, 4, 4, 2	cyclic
$U(12)$	4	1, 2, 2, 2	klein

```
// kernel of phi: residues with phi(k) = 0, gen.py pins {0, 2}
const uint64_t ker2[] = {0, 2};
uint64_t k2 = 0;
for (uint64_t a = 0; a < 4; a++) {
  if (a % 2 == 0) {
    k2 |= 1ull << a;
  }
}
CHECK(k2 == mask_of(ker2, 2), "kernel of z4->z2 is {0,2}");

// kernel of z6 -> z3 (k mod 3), gen.py pins {0, 3}
const uint64_t ker3[] = {0, 3};
uint64_t k3 = 0;
for (uint64_t a = 0; a < 6; a++) {
  if (a % 3 == 0) {
    k3 |= 1ull << a;
  }
}
CHECK(k3 == mask_of(ker3, 2), "kernel of z6->z3 is {0,3}");

// the kernel carries the group axioms it inherits: closure on {0,2}
// inside z4 (0+0, 0+2, 2+2=0) makes it a subgroup of order 2
CHECK(closed_under_add(k2, 4), "kernel {0,2} is a subgroup of z4");
```

listing 175: hom.c, both kernels pinned as bitmasks, and the kernel {0,2} closing into a subgroup of z4 on its own (source: math/samples/src/Ch17/hom.c, lines 122-145)

```
// u(8), u(10), u(12) order censuses, gen.py pins
// u(8) = {1,3,5,7} -> [1, 2, 2, 2] (every element its own inverse)
// u(10) = {1,3,7,9} -> [1, 4, 4, 2] (3 and 7 generate)
// u(12) = {1,5,7,11} -> [1, 2, 2, 2]
const uint64_t u8[] = {1, 3, 5, 7}, u10[] = {1, 3, 7, 9},
               u12[] = {1, 5, 7, 11};
const uint64_t w8[] = {1, 2, 2, 2}, w10[] = {1, 4, 4, 2},
               w12[] = {1, 2, 2, 2};
bool u8_ok = true, u10_ok = true, u12_ok = true;
for (uint64_t i = 0; i < 4; i++) {
  u8_ok &= ord_unit(u8[i], 8) == w8[i];
  u10_ok &= ord_unit(u10[i], 10) == w10[i];
  u12_ok &= ord_unit(u12[i], 12) == w12[i];
}
CHECK(u8_ok, "u(8) orders 1,2,2,2 (klein four)");
CHECK(u10_ok, "u(10) orders 1,4,4,2 (cyclic)");
CHECK(u12_ok, "u(12) orders 1,2,2,2 (klein four)");

// u(8) vs u(10): same order 4, order censuses differ, not isomorphic.
// u(8) vs u(12): same census, and the map 1->1, 3->5, 5->7, 7->11
// preserves all 16 products (gen.py: True). that map is an isomorphism
const uint64_t phi[8] = {0, 1, 0, 5, 0, 7, 0, 11}; // lookup by u(8) element
bool iso_ok = true;
for (uint64_t i = 0; i < 4; i++) {
  for (uint64_t j = 0; j < 4; j++) {
    if (phi[u8[i] * u8[j] % 8] != u12[i] * u12[j] % 12) {
      iso_ok = false;
    }
  }
}
CHECK(iso_ok, "u(8) -> u(12), 3->5 5->7 7->11, preserves all 16 products");
```

listing 176: hom.c, the three order censuses and the witnessed isomorphism u(8) -> u(12): every product maps to the paired product, so the tables are the same table (source: math/samples/src/Ch17/hom.c, lines 170-200)

PITFALL: a matching census is evidence, not a certificate

equal order counts for every order do not force an isomorphism in general: the census is a cheap invariant that separates groups, not a complete one. for groups of order 4 it happens to decide everything, because only two group structures of order 4 exist, and the census tells them apart. the honest certificate is always a map: check all n^2 products like the u(8) -> u(12) witness above, checks 10 and 11 of hom.c.

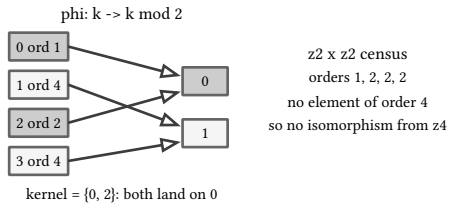


figure 100: the parity map collapses z4 onto z2 with kernel {0,2}, while the order census separates z4 from z2 x z2 with no map at all

17.2. cosets and quotients

Lagrange's partition from chapter 16 says the cosets of a subgroup H tile G into equal blocks. A quotient asks when the blocks themselves form a group under $(aH) \cdot (bH) = (ab)H$: exactly when left and right cosets agree, $gH = Hg$ for every g, the normal subgroups (Wikipedia, "quotient group", fetched 2026-09-21: "A subgroup N of a group G is normal if and only if the coset equality aN = Na holds for all a in G").

The dry run: two subgroups of the abelian Z_6, where normality is free, one subgroup of S_3 where it fails.

1. $H = \{0, 2, 4\}$ tiles Z_6 into H and $1 + H = \{1, 3, 5\}$. Adding blocks: $(1 + H) + (1 + H) = 2 + H = H$, a two-element group, isomorphic to Z_2. Every one of the 9 representative pairs lands in the same block, so the block sum does not depend on representatives.
2. $H = \{0, 3\}$ tiles Z_6 into $\{0, 3\}, \{1, 4\}, \{2, 5\}$, and three copies of $1 + H$ return to H: the quotient adds like Z_3.
3. In S_3 with $H = \{e, \mathrm{s01}\}$: the left coset $rH = \{r, \mathrm{s02}\}$ while the right coset $Hr = \{r, \mathrm{s12}\}$. Different sets, so H is not normal.

The failure has a product witness, worth seeing once in full. Multiply the cosets rH and r^2H elementwise: the four products $r \cdot r^2$, $r \cdot \mathrm{s12}$, $\mathrm{s02} \cdot r^2$, $\mathrm{s02} \cdot \mathrm{s12}$ are $e, \mathrm{s01}, \mathrm{s12}, r^2$, which sit in two different cosets at once, $\{e, \mathrm{s01}\}$ and $\{r^2, \mathrm{s12}\}$. The result is not a coset, so the candidate operation has no value: coset arithmetic on a non-normal subgroup is not merely inconvenient, it is undefined.

The repair inside S_3: the rotations $A_3 = \{e, r, r^2\}$ do satisfy $gA_3 = A_3g$ for all six g, checks 11 and 12 of quotients.c, and the quotient works. The even permutations form one coset, the three reflections the other, and every odd times odd product is even on all 9 pairs: S_3/A_3 is a group of order 2, isomorphic to Z_2, the sign homomorphism's kernel doing what section 1 promised.

```
  // the quotient table is z2: 1+H added to itself is H
  uint64_t c1 = 0, c2 = 0;
  for (uint64_t i = 0; i < 3; i++) {
    c1 |= 1ull << ((1 + h_even[i]) % 6);
    c2 |= 1ull << ((2 + h_even[i]) % 6);
  }
  CHECK(c1 == 0b101010 && c2 == 0b010101,
        "(1+H)+(1+H) = 2+H = H: quotient is z2");

  // cosets of {0,3}: three blocks {0,3}, {1,4}, {2,5}
  const uint64_t h3[] = {0, 3};
  bool blocks_ok = true;
  for (uint64_t rep = 0; rep < 3; rep++) {
    uint64_t c = 0;
    for (uint64_t i = 0; i < 2; i++) {
      c |= 1ull << ((rep + h3[i]) % 6);
    }
    blocks_ok &= c == (rep == 0 ? 0b001001 : rep == 1 ? 0b010010 : 0b100100);
  }
  CHECK(blocks_ok, "cosets of {0,3} in z6 are {0,3}, {1,4}, {2,5}");
```

listing 177: quotients.c, the quotient table check: (1+H)+(1+H) = 2+H = H gives z2, and the {0,3} blocks tile z6 three ways (source: math/samples/src/Ch17/quotients.c, lines 111-130)

```
// left coset rH of h = {e, s01}: gen.py pins {r, s02} = indices {1, 4}
const int hx[] = {0, 3};
CHECK(coset3(1, hx, 2, true) == mask3((int[]){1, 4}, 2),
      "left coset rH = {r, s02}");

// right coset Hr: gen.py pins {r, s12} = indices {1, 5}
CHECK(coset3(1, hx, 2, false) == mask3((int[]){1, 5}, 2),
      "right coset Hr = {r, s12}, not equal to rH");

// elementwise product of rH with r2H: gen.py pins {e, r2, s01, s12},
// which is the union of two different cosets, so coset multiplication
// by representatives cannot be well-defined
uint8_t prod_mask = 0;
int left_rh[2] = {1, 4}, left_r2h[2] = {2, 5};
for (int i = 0; i < 2; i++) {
  for (int j = 0; j < 2; j++) {
    comp3(S3[left_rh[i]], S3[left_r2h[j]], tmp);
    prod_mask |= (uint8_t)(1u << s3_index(tmp));
  }
}
CHECK(prod_mask == 0b101101, "rH times r2H spans {e,r2,s01,s12}, two cosets");
```

listing 178: quotients.c, the non-normal witness in s3: rh = {r, s02} against hr = {r, s12}, then the elementwise coset product landing in two cosets at once (source: math/samples/src/Ch17/quotients.c, lines 158-179)

NOTE: abelian groups divide freely

in an abelian group $gh = hg$ holds before you ask, so $gH = Hg$ automatically: every subgroup of every abelian group is normal, and quotients are always available. that is why Z_6 produced two clean quotients above while S_3, the smallest non-abelian group, needed the scan. checks 10 through 12 of quotients.c run that scan for you.

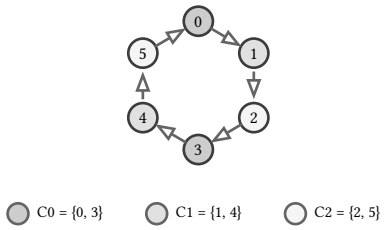


figure 101: z6 as a ring of 6 residues: opposite pairs form the three cosets of {0,3}, shaded dark, medium, light, and they add like z3

17.3. direct products

Two groups combine without interacting: the direct product $G \times H$ pairs elements and multiplies componentwise, $(a, b) \cdot (a', b') = (a \cdot a', b \cdot b')$. The order law is the whole section: $\operatorname{ord}(a, b) = \operatorname{lcm}(\operatorname{ord} a, \operatorname{ord} b)$, because $(a, b)^k = \left(a^k, b^k\right)$ returns to the identity exactly when both components do.

The dry run: $Z_4 \times Z_6$ has 24 elements, and the pair $(1 \bmod 4, 1 \bmod 6)$ has order $\operatorname{lcm}(4, 6) = 12$, double the largest order inside either factor. The lazy pair $(2, 3)$ dies in $\operatorname{lcm}(2, 2) = 2$ steps. The lcm law itself is checked on all 24 elements, not asserted.

The law also separates products from cyclic groups. $Z_2 \times Z_4$ has $2 \cdot 4 = 8$ elements but maximum element order $\operatorname{lcm}(2, 4) = 4$, so it cannot be Z_8, which has an element of order 8. Whereas $Z_3 \times Z_4$ is Z_{12} in disguise: the map

$(a, b) \to 4a + 9b \bmod 12$ hits every residue exactly once and preserves addition on all 144 pairs, with $4 \equiv (1, 0)$ and $9 \equiv (0, 1)$ encoding each component. That is the Chinese remainder theorem as a table scan.

```
typedef struct {
  uint8_t a4;
  uint8_t b6;
} z4z6;

[[nodiscard]] static z4z6 add_z4z6(z4z6 p, z4z6 q) {
  return (z4z6){(uint8_t)((p.a4 + q.a4) % 4), (uint8_t)((p.b6 + q.b6) % 6)};
}

// order of (a,b) by walking the componentwise sum
[[nodiscard]] static uint64_t ord_z4z6(z4z6 p) {
  z4z6 cur = p;
  uint64_t k = 1;
  while (cur.a4 != 0 || cur.b6 != 0) {
    cur = add_z4z6(cur, p);
    k++;
  }
  return k;
}

[[nodiscard]] static uint64_t lcm_u64(uint64_t a, uint64_t b) {
  return a / gcd_u64(a, b) * b;
}
```

listing 179: hom.c, the product element as a two-field struct with compound literals, added componentwise, its order walked to the identity, lcm on hand (source: math/samples/src/Ch17/hom.c, lines 86-108)

```
  // (1,1) -> lcm(4,6) = 12, (2,3) -> lcm(2,2) = 2
  CHECK(ord_z4z6((z4z6){1, 1}) == 12, "order of (1 mod 4, 1 mod 6) is 12");
  CHECK(ord_z4z6((z4z6){2, 3}) == 2, "order of (2 mod 4, 3 mod 6) is 2");

  // the lcm rule holds on all 24 elements of z4 x z6
  bool lcm_ok = true;
  for (uint64_t a = 0; a < 4; a++) {
    for (uint64_t b = 0; b < 6; b++) {
      if (ord_z4z6((z4z6){(uint8_t)a, (uint8_t)b}) !=
          lcm_u64(ord_zn(a, 4), ord_zn(b, 6))) {
        lcm_ok = false;
      }
    }
  }
  CHECK(lcm_ok, "order of (a,b) = lcm(ord a, ord b) on all 24 elements");
```

listing 180: hom.c, the lcm order law pinned on pairs then scanned on all 24 elements of z4 x z6 (source: math/samples/src/Ch17/hom.c, lines 211-225)

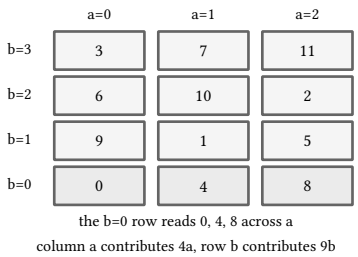


figure 102: the crt grid: each cell of z3 x z4 labeled by its image 4a + 9b mod 12, the 12 residues appearing exactly once

17.4. rings and fields

A ring adds a second operation to a group. Definition: $(R, +, \cdot)$ is a ring when R is an abelian group under $+$, multiplication is associative with an identity (for rings with 1), and distributivity $a \cdot (b + c) = a \cdot b + a \cdot c$ ties them together. A field is a commutative ring where every nonzero element has a multiplicative inverse.

The integers mod n are the fixture ring, and the two operations disagree about n in exactly one way. Under addition Z_n is always a group. Under multiplication it is a field precisely when n is prime, and the reason is the zero divisors: in Z_6, $2 \cdot 3 = 6 \equiv 0$, and a zero divisor can never invert, because $x \cdot 2 \equiv 1$ would give $3 \cdot x \cdot 2 \equiv 3 \cdot 1 \equiv 3$ on one side and $3 \cdot 0 \equiv 0$ on the other.

The dry run: Z_5 inverts everything, $1 \cdot 1 = 1$, $2 \cdot 3 = 6 \equiv 1$, $4 \cdot 4 = 16 \equiv 1$. Z_6 keeps only 1 and 5, and the Z_{12} census separates the rest: zero divisors $2, 3, 4, 6, 8, 9, 10$ against units $1, 5, 7, 11$, with $5 \cdot 5 = 25 \equiv 1$, check 4 of fields.c. The scan over $n = 2$ to 12 pins the equivalence both directions, primes 2, 3, 5, 7, 11 have all nonzero residues invertible, every composite has a zero divisor pair, and distributivity holds on all 216 triples of Z_6, the hand triple $2 \cdot (3 + 5) = 16 \equiv 4$ against $6 + 10 \equiv 4$ among them.

ring	zero divisors	units	field
Z_5	none	1, 2, 3, 4	yes
Z_6	2, 3, 4	1, 5	no
Z_8	2, 4, 6	1, 3, 5, 7	no
Z_{12}	2, 3, 4, 6, 8, 9, 10	1, 5, 7, 11	no

```
// z5 is a field: every nonzero residue has an inverse. gen.py pins the
// inverse pairs 1→1, 2→3, 3→2, 4→4
const uint64_t inv5[4] = {1, 3, 2, 4}; // index by a in 1..4 via [a-1]
bool inv_ok = true;
for (uint64_t a = 1; a ≤ 4; a++) {
  if (a * inv5[a - 1] % 5 ≠ 1) {
    inv_ok = false;
  }
}
CHECK(inv_ok, "z5 inverses: 1*1, 2*3, 3*2, 4*4 all give 1");

// z6 is not: 2 and 3 are zero divisors, 2*3 = 0 and 3*4 = 0 mod 6
CHECK(2 * 3 % 6 == 0 && 3 * 4 % 6 == 0, "z6 zero divisors: 2*3 = 0, 3*4 = 0");

// only the units 1 and 5 invert, and 5 is its own inverse: 25 = 1 mod 6
CHECK(5 * 5 % 6 == 1, "z6 units are {1,5}: 5*5 = 1");

// field iff prime, scanned n = 2..12: every nonzero residue invertible
// exactly for the primes 2, 3, 5, 7, 11
bool match = true;
for (uint64_t n = 2; n ≤ 12; n++) {
  bool all_inv = true;
  for (uint64_t a = 1; a < n; a++) {
    bool has = false;
    for (uint64_t b = 1; b < n; b++) {
      has |= a * b % n == 1;
    }
    all_inv &= has;
  }
  match &= all_inv == is_prime(n);
}
CHECK(match, "z_n is a field exactly when n is prime, n = 2..12");
```

listing 181: fields.c, the inverse scan in z5, the zero divisors of z6, and the prime-versus-composite equivalence scanned for every n from 2 to 12 (source: math/samples/src/Ch17/fields.c, lines 82-113)

NOTE: u(n) is the field question with the zeros kept

chapter 16 met $U(n)$ as the units mod n, a group of order $\varphi(n)$. the field scan rephrases it: Z_n is a field exactly when the units are everything but zero, $\varphi(n) = n - 1$, which holds exactly for primes. the group of chapter 16 and the ring of this chapter meet on the same residue set, and checks 1 through 4 of fields.c walk the equivalence end to end.

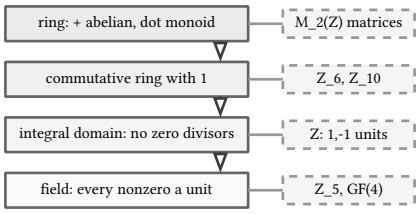


figure 103: the containment ladder from ring to field: z6 dies at zero divisors, z survives them but keeps only 1 and −1 invertible, z5 and gf(4) arrive at the field rung

17.5. finite fields

There is no Z_4-style field of order 4, since 4 is composite, yet a field of four elements exists: build it from polynomials. Over Z_2, take the polynomials of degree below 2, which are exactly $0, 1, x, x + 1$, and reduce products

modulo $x^2 + x + 1$. Since $x^2 \equiv x + 1$ over Z_2 (the minus signs vanish), multiplication closes on the four elements: $(x+1)^2 = x^2 + 2x + 1 = x^2 + 1 \equiv x$, and the full table is pinned below. This is $\mathrm{GF}(4) = Z_2[x]/(x^2 + x + 1)$, and $x^2 + x + 1$ being irreducible over Z_2 plays the role the prime p played for Z_p.

The dry run: powers of x in $\mathrm{GF}(4)$.

1. $x \cdot x = x^2 \equiv x + 1$.
2. $x \cdot (x+1) = x^2 + x \equiv (x+1) + x = 1$: so $x^3 = 1$.
3. The nonzero ladder is $x, x+1, 1$, orders $1, 3, 3$: the multiplicative group is cyclic of order 3, generated by x.

One dimension up, $\mathrm{GF}(8) = Z_2[x]/(x^3 + x + 1)$ on the eight polynomials of degree below 3. The reduction rule $x^3 \equiv x + 1$ chains into $x^4 \equiv x^2 + x$, and the powers of x walk the entire nonzero field: $x, x^2, x+1, x^2+x, x^2+x+1, x^2+1, 1$, seven values, all distinct, returning to x. Since 7 is prime, every element besides 1 has order 7. This is the general shape: the nonzero elements of a finite field form a multiplicative group, and that group is cyclic (Wikipedia, "finite field", fetched 2026-09-21: "The non-zero elements of a finite field form a multiplicative group" and "The structure theorem of finite abelian groups implies that this multiplicative group is cyclic, that is, all non-zero elements are powers of a single element"). The identity $a^q = a$ holds for every element of $\mathrm{GF}(q)$, pinned in both fields at once.

dot	0	1	x	x+1
0	0	0	0	0
1	0	1	x	x+1
x	0	x	$x+1$	1
$x+1$	0	$x+1$	1	x

```
// the ring axioms that are not group axioms: distributivity holds on
// all 216 triples of z6 (a=2, b=3, c=5 pinned in the prose: both sides 4)
bool dist = true;
for (uint64_t a = 0; a < 6; a++) {
  for (uint64_t b = 0; b < 6; b++) {
    for (uint64_t c = 0; c < 6; c++) {
      if (a * ((b + c) % 6) % 6 != (a * b + a * c) % 6) {
        dist = false;
      }
    }
  }
}
CHECK(dist, "z6 distributes a*(b+c) = ab+ac on all 216 triples");

// polynomial arithmetic mod x^2+x+1: (x+1)^2 = x^2+2x+1 = x^2+1 = x.
// element 3 is x+1, gen.py pins 3*3 = 2 (the element x)
CHECK(gf4_mul(3, 3) == 2, "gf(4): (x+1)^2 = x");
```

listing 182: fields.c, the pinned gf(4) multiplication table with the (x+1)^2 = x reduction on top (source: math/samples/src/Ch17/fields.c, lines 129-147)

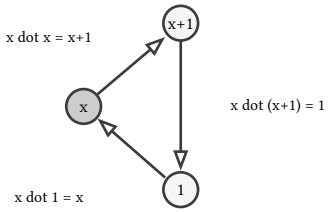
```
  uint8_t k = 1;
  while (cur != 1) {
    cur = gf4_mul(cur, a);
    k++;
  }
  ord4[a - 1] = k;
}
CHECK(ord4[0] == 1 && ord4[1] == 3 && ord4[2] == 3,
      "gf(4)* orders: 1 has 1, x and x+1 have 3");

// so x generates the nonzero group of order 3: the ladder x, x+1, 1
// walks all of it. gen.py pins x^2 = x+1 and x^3 = 1
CHECK(gf4_mul(2, 2) == 3 && gf4_mul(3, 2) == 1,
      "gf(4)* is cyclic: x, x+1, 1");

// one dimension up, gf(8) = z2[x]/(x^3+x+1). the reduction rule first:
// x^3 = x + 1, pinned as element 2 times element 4 gives element 3
CHECK(gf8_mul(2, 4) == 3, "gf(8): x * x^2 = x^3 = x+1");

// the powers of x walk the entire nonzero field: gen.py and the hand
// ladder pin x, x^2, x+1, x^2+x, x^2+x+1, x^2+1, 1 = bytes 2,4,3,6,7,5,1
static constexpr uint8_t ladder[7] = {2, 4, 3, 6, 7, 5, 1};
uint8_t cur = 2; // x
bool walk = true;
for (int i = 0; i < 7; i++) {
  walk &= cur == ladder[i];
  cur = gf8_mul(cur, 2);
}
CHECK(walk && cur == 2, "gf(8): seven powers of x return to x, x^7 = 1");
```

listing 183: fields.c, gf(8) one dimension up: the x^3 = x+1 rule pinned, the seven powers of x walking every nonzero element home, so every non-identity element has order 7 (source: math/samples/src/Ch17/fields.c, lines 168-197)



gf(4) **has order 3, gf(8)** has order 7
one generator covers every nonzero element

figure 104: the multiplicative group of gf(4) is a 3-cycle: multiplying by x walks x, x+1, 1 and closes

17.6. the dihedral group d4 acts on a square

The symmetries of the square form D_4: four rotations, four reflections, each a permutation of the four vertices. With r the quarter turn and s the reflection fixing vertices 0 and 2, every element is one of $e, r, r^2, r^3, s, rs, r^2s, r^3s$, and two relations pin the whole group: $r^4 = s^2 = e$ and $s \circ r = r^3 \circ s$, pass an s left across an r and the r inverts. That exchange rule is the non-commutativity in one line, and it is why the 64 products close without storing a table.

A group action is a group behaving like a machine on a set: each $g \in G$ moves points $x \to g \cdot x$, with $e \cdot x = x$ and $g \cdot (h \cdot x) = (g \circ h) \cdot x$. Two sets fall out. The orbit of x is everything reachable, $\{g \cdot x : g \in G\}$. The stabilizer of x is everything that fixes it, $\{g : g \cdot x = x\}$, itself a subgroup.

The dry run: under the eight symmetries the orbit of vertex 0 is all 4 vertices, its stabilizer is exactly $\{e, s\}$, and $4 \cdot 2 = 8 = |D_4|$. That product is no coincidence, it is the orbit-stabilizer theorem, Lagrange counting twice: the cosets of the stabilizer biject with the orbit.

element	e	r	r^2	s	rs
order	1	4	2	2	2
fixes vertex 0	yes	no	no	yes	no

```
// image of i. r is the quarter turn, s the reflection fixing 0 and 2
static constexpr uint8_t D4[8][4] = {
    {0, 1, 2, 3}, // e
    {1, 2, 3, 0}, // r
    {2, 3, 0, 1}, // r2
    {3, 0, 1, 2}, // r3
    {0, 3, 2, 1}, // s    fix 0 and 2, swap 1 and 3
    {1, 0, 3, 2}, // rs
    {2, 1, 0, 3}, // r2s
    {3, 2, 1, 0}, // r3s
};

static void comp4(const uint8_t *p, const uint8_t *q, uint8_t *out) {
  for (int i = 0; i < 4; i++) {
    out[i] = p[q[i]];
  }
}
```

listing 184: actions.c, d4 as eight vertex permutations in a constexpr table, composition as one line (source: math/samples/src/Ch17/actions.c, lines 26-42)

```
  // the action on vertices: the orbit of vertex 0 is all four vertices
  bool orbit[4] = {false};
  for (int g = 0; g < 8; g++) {
    orbit[D4[g][0]] = true;
  }
  CHECK(orbit[0] && orbit[1] && orbit[2] && orbit[3],
        "orbit of vertex 0 under d4 is all 4 vertices");

  // the stabilizer of vertex 0: exactly e and s, two elements
  int stab = 0;
  for (int g = 0; g < 8; g++) {
    if (D4[g][0] == 0) {
      stab++;
    }
  }
  CHECK(stab == 2 && (D4[0][0] == 0 && D4[4][0] == 0),
        "stabilizer of vertex 0 is {e, s}");

  // orbit-stabilizer: 4 x 2 = 8 = |d4|
  CHECK(4 * stab == 8, "orbit-stabilizer: 4 vertices x 2 stabilizer = 8");
```

listing 185: actions.c, the action read directly off the table: orbit of vertex 0 is all four vertices, stabilizer is {e, s}, product 8 (source: math/samples/src/Ch17/actions.c, lines 147-166)

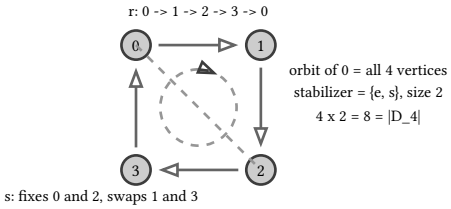


figure 105: the square under its own symmetry: r cycles the vertices, s holds 0 and 2 fixed across the diagonal

17.7. burnside's lemma counts orbits

Colorings breed: 4 vertices with 2 colors give 16 raw colorings of the square, but symmetries eat the differences, and what a puzzle or a game asset wants is the count of colorings up to symmetry. Burnside's lemma is the averaging trick that gets it (Wikipedia, "Burnside's lemma", fetched 2026-09-21: "the number of orbits ... is equal to the average number of points fixed by an element of G"). A coloring is fixed by g exactly when it is constant on each cycle of g, so with k colors a permutation with c cycles fixes k^c colorings, and the orbit count is the average of k^c over the group.

The dry run: the square, then the cube.

1. Square, 2 colors under D_4: the identity has 4 cycles and fixes $2^4 = 16$, the rotations r, r^3 have 1 cycle and fix 2 each, r^2 and the two axis reflections have 2 cycles and fix 4 each, the two diagonal reflections have 3 cycles and fix 8 each. Sum $16 + 2 + 2 + 4 + 4 + 4 + 8 + 8 = 48$, average $48/8 = 6$ orbits. With 3 colors the same cycle data gives $168/8 = 21$.
2. Cube faces, 2 colors under the 24 rotations: the rotations fall into 5 cycle classes on faces, and the fixed counts $2^6, 2^3, 2^4, 2^3, 2^2$ weighted by $1, 6, 3, 6, 8$ sum to 240, average $240/24 = 10$ distinct colorings out of 64 raw. With 3 colors: $1368/24 = 57$.
3. Necklace, 6 beads, 2 colors under rotation only: a rotation by s has $\gcd(6, s)$ cycles, so the sum is $64 + 2 + 4 + 8 + 4 + 2 = 84$, average $84/6 = 14$.

cube rotation class	count	face cycle type	cycles	2-color fixed
identity	1	$(1, 1, 1, 1, 1, 1)$	6	64
quarter turns	6	$(1, 1, 4)$	3	8
half turns, face axes	3	$(1, 1, 2, 2)$	4	16
half turns, edge axes	6	$(2, 2, 2)$	3	8
third turns, vertex axes	8	$(3, 3)$	2	4

The cube's 24 rotations are themselves generated in code by closing two quarter turns under composition, and two guards keep the closure honest: exactly 24 elements, and every element sends opposite faces to opposite faces, so the closure never wanders into the 48-element full octahedral group. The cycle-count arithmetic is the cycle index evaluated at $k^1, k^2, \ldots$, which is where Polya enumeration generalizes counting to inventory. The algorithmic side, gcd-class necklace sums and bracelet reflections, is book 9, ch 35 (practical data structures and algorithms, combinatorics) territory.

```
// the 24 cube rotations, generated by closing two quarter turns:
// rot about y (faces +x->+z, +z->-x) and rot about x (+y->+z, +z->-y).
// faces: 0=+x 1=-x 2=+y 3=-y 4=+z 5=-z
static constexpr uint8_t ROT_Y[6] = {4, 5, 2, 3, 1, 0};
static constexpr uint8_t ROT_X[6] = {0, 1, 4, 5, 3, 2};
uint8_t cube[24][6];
int nrot = 1;
for (int i = 0; i < 6; i++) {
  cube[0][i] = (uint8_t)i;
}
bool grew = true;
while (grew) {
  grew = false;
  int snapshot = nrot;
  for (int gi = 0; gi < snapshot; gi++) {
    const uint8_t *gens[2] = {ROT_Y, ROT_X};
    for (int s = 0; s < 2; s++) {
      uint8_t cand[6], cand2[6];
      for (int i = 0; i < 6; i++) {
        cand[i] = cube[gi][gens[s][i]];  // g o gen
        cand2[i] = gens[s][cube[gi][i]]; // gen o g
      }
      for (int c = 0; c < 2; c++) {
        const uint8_t *p = c == 0 ? cand : cand2;
        bool member = false;
        for (int m = 0; m < nrot; m++) {
          bool same = true;
          for (int i = 0; i < 6; i++) {
            same &= cube[m][i] == p[i];
          }
          member |= same;
        }
        if (!member && nrot < 24) {
          for (int i = 0; i < 6; i++) {
            cube[nrot][i] = p[i];
          }
          nrot++;
          grew = true;
        }
      }
    }
  }
}
CHECK(nrot == 24, "two quarter turns generate exactly 24 cube rotations");
```

listing 186: actions.c, the 24 cube rotations grown by closing two quarter-turn generators, with the group-order and opposite-face guards (source: math/samples/src/Ch17/actions.c, lines 179-222)

```c
// burnside on cube faces: gen.py pins cycle-type census
// 1x(1,1,1,1,1,1), 6x(1,1,4), 3x(1,1,2,2), 6x(2,2,2), 8x(3,3) and
// totals 240/24 = 10 two-color, 1368/24 = 57 three-color
uint64_t csum2 = 0, csum3 = 0;
for (int m = 0; m < 24; m++) {
  csum2 += pow_u64(2, cycles(cube[m], 6));
  csum3 += pow_u64(3, cycles(cube[m], 6));
}
CHECK(csum2 == 240 && csum2 / 24 == 10,
      "cube 2-colorings under rotations: 240/24 = 10");
CHECK(csum3 == 1368 && csum3 / 24 == 57,
      "cube 3-colorings under rotations: 1368/24 = 57");

// necklaces: rotations only on 6 beads, cycles of shift s = gcd(6, s).
// gen.py pins (64+2+4+8+4+2)/6 = 14 and (729+3+9+27+9+3)/6 = 130
uint64_t nsum2 = 0, nsum3 = 0;
for (int s = 0; s < 6; s++) {
  uint8_t shift[6];
  for (int i = 0; i < 6; i++) {
    shift[i] = (uint8_t)((i + s) % 6);
  }
  nsum2 += pow_u64(2, cycles(shift, 6));
  nsum3 += pow_u64(3, cycles(shift, 6));
}
CHECK(nsum2 == 84 && nsum2 / 6 == 14, "6-bead 2-color necklaces: 84/6 = 14");
CHECK(nsum3 == 780 && nsum3 / 6 == 130,
      "6-bead 3-color necklaces: 780/6 = 130");
```

listing 187: actions.c, the cube and necklace averages from cycle counts alone: 240/24 = 10, 1368/24 = 57, 84/6 = 14, 780/6 = 130 (source: math/samples/src/Ch17/actions.c, lines 235-261)

VERIFY: count, then cross-check by enumeration

the orbit counts above come from the fixed-point average, which is one theorem deep. a brute-force cross-check that floods each orbit by applying every group element lands the same numbers, and it is four lines of code on these sizes: for each of the k^n colorings not yet seen, flood its orbit and add 1. playground/math-ch17/gen.py did exactly that before any of these sums were trusted, 6 for the square, 10 for the cube.

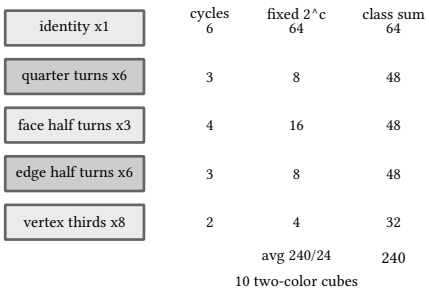


figure 106: the five cube rotation classes with their counts and fixed colorings, summing to 240 over 24 rotations

17.8. diffie-hellman and the discrete logarithm

Diffie-Hellman builds a shared secret over a public wire inside $U(p)$: agree publicly on a prime p and a generator g of $U(p)$, each picks a secret exponent, they exchange g^{secret} publicly, and both raise what they receive to their own secret, landing on g^{ab} together (Wikipedia, "Diffie-Hellman key exchange", fetched 2026-09-21: "two parties arrive at a common secret key, without passing the common secret key across the public channel"). The eavesdropper sees p, g, g^a, g^b and must find a from g^a: the discrete logarithm problem.

The dry run: the toy runs in $U(101)$ with $g = 3$, the primitive root chapter 16 pinned (order exactly 100, $3^{50} \equiv 100 \equiv -1$ halfway).

1. Alice draws $a = 27$ and posts $A = 3^{27} \bmod 101 = 90$. Bob draws $b = 71$ and posts $B = 3^{71} \bmod 101 = 51$.
2. Alice computes $B^{27} = 48$. Bob computes $A^{71} = 48$. Both match $3^{27 \cdot 71} = 3^{1917} = 48$: one shared secret, never transmitted.
3. The attack is the log: multiply 3 into itself until 90 appears, and you hit it after exactly 27 multiplications, recovering Alice. Bob's 51 falls after 71. Two dozen lines of brute force, and that is the point.

```
// toy diffie-hellman in u(101), g = 3 a primitive root (chapter 16).
// alice secret 27, bob secret 71, gen.py pins A = 90, B = 51
const uint64_t p = 101, g = 3, a = 27, b = 71;
uint64_t A = powmod(g, a, p), B = powmod(g, b, p);
CHECK(A == 90 && B == 51, "public keys: 3^27 = 90, 3^71 = 51 mod 101");

// the shared secret from either side: A^b = B^a = g^(ab) = 48
CHECK(powmod(A, b, p) == 48 && powmod(B, a, p) == 48 &&
          powmod(g, a * b, p) == 48,
      "shared secret agrees: A^71 = B^27 = 3^1917 = 48");

// the discrete log breaks this toy: multiplying g until A appears takes
// 27 steps, and B needs 71. gen.py pins both
uint64_t curv = 1, steps_a = 0;
while (curv != A) {
  curv = curv * g % p;
  steps_a++;
}
CHECK(steps_a == 27 && powmod(g, steps_a, p) == A,
      "brute discrete log recovers 27 from 90 in 27 multiplies");
curv = 1;
uint64_t steps_b = 0;
while (curv != B) {
  curv = curv * g % p;
  steps_b++;
}
CHECK(steps_b == 71, "brute discrete log recovers 71 from 51");
```

listing 188: actions.c, the toy exchange over u(101) with the shared secret agreeing from both sides, then both discrete logs recovered by brute force in 27 and 71 multiplies (source: math/samples/src/Ch17/actions.c, lines 263-290)

WARNING: 101 is a teaching modulus, not a prime for practice

real exchanges use primes of hundreds of digits where the 100-step ladder is 10^{300} steps and brute force is dead, and then beating it needs real algorithms: baby-step giant-step and the index calculus, which are chapter material of book 9, ch 28 (practical data structures and algorithms, advanced number theory). the structure here, why g^a lives in a cyclic group and why g^{ab} agrees from both sides, is what those algorithms attack. never build crypto from a textbook toy.

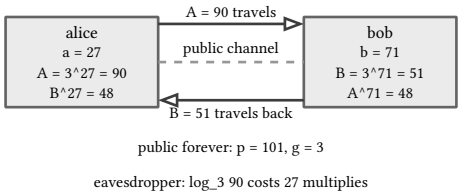


figure 107: the exchange: secrets 27 and 71 never cross the public line, the powers 90 and 51 do, and 48 emerges on both sides

Chapter 16 asked what a group is; this chapter asked what groups do to each other and to sets: maps preserving tables, quotients by normal subgroups, products with lcm arithmetic, fields from polynomial quotients, actions with orbits and stabilizers, and one cyclic group whose logarithm prices a protocol. book 2, ch 18 (mathematics for programmers, probability and distributions) leaves determinism behind next, carrying the counting habits built here straight into it.

sources: no mml mapping exists for this chapter's material, so it was authored from canonical texts by name and four fetched pages. Dummit and Foote, Abstract Algebra, 3rd ed 2004, chs 1-3 for groups, homomorphisms, quotient groups, and the isomorphism theorems, ch 4 group actions, ch 5 direct products, ch 7 ring basics, ch 9 polynomial rings, and sec 14.3 finite fields, cited by name. Artin, Algebra, 2nd ed 2011, ch 2 for homomorphisms and cosets, the symmetry chapters for the dihedral groups, and ch 10 rings, cited by name. Wikipedia, quotient group, en.wikipedia.org/wiki/Quotient_group, the normal subgroup criterion aN = Na, fetched 2026-09-21. Wikipedia, finite field, en.wikipedia.org/wiki/Finite_field, nonzero elements forming a cyclic multiplicative group and the discrete log phrasing, fetched 2026-09-21. Wikipedia, Burnside's lemma, en.wikipedia.org/wiki/Burnside%27s_lemma, the orbit count as the average of fixed points, fetched 2026-09-21. Wikipedia, Diffie-Hellman key exchange, en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange, the shared secret over a public channel, fetched 2026-09-21. Expected values computed with python exact integers in the one-off playground/math-ch17/gen.py, including brute-force orbit enumeration cross-checks of every Burnside count. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch17`, 61 checks in chapter 17 of the math suite, probed on this machine 2026-09-22.

18. probability and distributions

Probability starts where book 2, ch 14 (mathematics for programmers, combinatorics) counting stops: a sample space small enough to hold in a byte, events as bitmasks, and weights that must obey three axioms before anything deserves the name chance. This chapter builds the tower in six moves: the axioms and the random variable as a lookup function, conditioning and Bayes inversion in exact rationals, the expectation and variance machinery of discrete variables, the binomial and Poisson families with exact integer kernels, the gaussian with standardization and mixture algebra carried by quadrature, and the conjugate beta update plus the change-of-variables rule that moves densities between scales. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 18 or a sentence quoted from the mml-book draft of 2024-01-15, chapter 6, cached at ref/mml-book.pdf and reverified on this machine 2026-09-22. The dsa book treats randomness as an algorithm input, see book 9, ch 36 (practical data structures and algorithms, numerical methods) for the simpson driver reused here, while this chapter treats it as a measure first.

18.1. probability spaces

The mml book builds probability on three objects. The sample space Ω is "the set of all possible outcomes of the experiment". The event space $\mathcal{A}$ collects the subsets of Ω we can observe, "for discrete probability distributions $\mathcal{A}$ is often the power set of Ω". The probability P attaches to each event a number with $P(A) \in [0, 1]$ and $P(\Omega) = 1$, Kolmogorov's axioms. For a finite space all three live in one byte here: Ω is the integers 0 to 7, an event is a predicate over a 3-bit mask, and P is a sum of weights.

A random variable $X : \Omega \to \mathcal{T}$ moves probability to a target space. The book's margin is blunt: "The name 'random variable' is a great source of misunderstanding as it is neither random nor is it a variable. It is a function." Its own probability comes from the pre-image, $P_X(S) = P(X \in S) = P(X^{-1}(S))$, so X is a lookup table and the distribution is the weight it pushes forward.

The dry run: three fair coins give the 8-outcome space, every weight 1/8. Events are predicates on the mask: coin 1 heads is $b1 \neq 0$, at least two heads is a popcount of at least 2.

1. Kolmogorov holds by enumeration: the full event measures 8/8 = 1, the empty event 0, and complement pairs like "at least two heads" with "at most one" sum to 4/8 + 4/8 = 1.
2. Inclusion-exclusion for two events, the counting tie-back, reads $P(A \cup B) = P(A) + P(B) - P(A \wedge B)$, here 4/8 + 4/8 - 2/8 = 6/8.
3. The three-event version is $3 \cdot \frac{4}{8} - 3 \cdot \frac{2}{8} + \frac{1}{8} = \frac{7}{8}$, matching the direct count of masks 1 through 7.

Equally likely is a special weighting, not the definition. Two fair dice make 36 outcomes of weight 1/36 each and $P(\text{sum} = 7) = \frac{6}{36} = \frac{1}{6}$. The book's funfair game, example 6.1, weights instead: draw two coins with replacement from a bag where "a draw returns at random a \$ with probability 0.3", and count the dollars drawn. With $\mu = \frac{3}{10}$ the pmf is $P(X = 2) = \mu^2 = \frac{9}{100}$, $P(X = 1) = 2\mu(1 - \mu) = \frac{42}{100}$, $P(X = 0) = (1 - \mu)^2 = \frac{49}{100}$, summing to 1. The same two-draw event under a uniform weighting would carry 1/4, and the sample pins the difference: weights change answers.

```c
// hand-rolled popcount, no <stdbit.h> on this toolchain
static int popcount3(uint8_t bits) {
  int c = 0;
  for (int i = 0; i < 3; i++)
    c += (bits >> i) & 1u;
  return c;
}

// three fair coins: outcome index bit i = coin i shows heads, weight 1/8 each
static struct frac weight(uint8_t bits) {
  (void)bits;
  return frac_make(1, 8);
}

// the probability measure: sum the weights of the outcomes an event holds
static struct frac measure(bool (*ev)(uint8_t)) {
  struct frac p = frac_make(0, 1);
  for (uint8_t b = 0; b < 8; b++)
    if (ev(b))
      p = fadd(p, weight(b));
  return p;
}
```

listing 189: spaces kernel, the measure as a masked weight sum (source: math/samples/src/Ch18/spaces.c, lines 71-92)

```c
  // weighted space, mml example 6.1: two draws with replacement,
  // P($) = 3/10, counting the dollar coins drawn
  auto mu = frac_make(3, 10);
  auto q = frac_make(7, 10);
  struct frac p2 = fmul(mu, mu);                    // X = 2
  struct frac p1 = fadd(fmul(mu, q), fmul(q, mu)); // X = 1
  struct frac p0 = fmul(q, q);                      // X = 0
  // pinned: 49/100, 42/100 = 21/50, 9/100
  CHECK(feq(p0, frac_make(49, 100)) && feq(p1, frac_make(21, 50)) &&
            feq(p2, frac_make(9, 100)),
        "funfair pmf 0.49, 0.42, 0.09 exact");
  CHECK(!feq(p2, frac_make(1, 4)),
        "weighted space differs from the uniform 1/4");
  CHECK(feq(fadd(fadd(p0, p1), p2), one) &&
            feq(fadd(p0, p1), frac_make(91, 100)),
        "funfair pmf sums to 1, F(1) = 91/100");
```

listing 190: weighted funfair space, the example 6.1 pmf in exact rationals (source: math/samples/src/Ch18/spaces.c, lines 171-186)

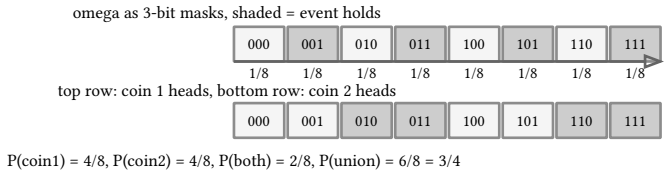


figure 108: bitmask events on the 8-outcome coin space, weight 1/8 each

NOTE: random variables are deterministic

The distribution of X is $P \circ X^{-1}$ in the general statement, the book calls it the law, but on a finite space it is simpler: a lookup table from outcomes to states plus the weight each outcome carries. Two random variables on the same space can share every weight and differ only in the lookup, which is why X and X^2 in the next section can be dependent while their covariance vanishes.

18.2. conditional probability and Bayes

Conditioning shrinks the denominator. For a joint table with counts n_{ij} the book gives $P(Y = y_j \mid X = x_i) = \frac{n_{ij}}{c_i}$, the cell over its column sum, and the two fundamental rules follow: the sum rule $p(x) = \sum_y p(x, y)$ marginalizes, the product rule $p(x, y) = p(y \mid x)p(x)$ factorizes. Bayes is the product rule read twice, and the book labels each slot:

$$\underbrace{p(x \mid y)}_{\text{posterior}} = \frac{\overbrace{p(y \mid x)}^{\text{likelihood}}\overbrace{p(x)}^{\text{prior}}}{\underbrace{p(y)}_{\text{evidence}}}$$

The evidence is itself an expectation, $p(y) = \int p(y \mid x)p(x)\,\mathrm{d}x = E_X[p(y \mid x)]$, the expected likelihood under the prior. The book is strict about vocabulary: the likelihood "is not a distribution in x, but only in y".

The dry run: a screening table fixed at 1000 people, prevalence 1/100, sensitivity 9/10, false-positive rate 1/10.

1. The joint counts are 9 and 1 for the ten sick, 99 and 891 for the 990 well. Marginals: $P(\text{sick}) = \frac{1}{100}$ and $P(+) = \frac{108}{1000} = \frac{27}{250}$.
2. Bayes inverts the table: $P(\text{sick} \mid +) = \frac{9}{108} = \frac{1}{12}$, about 8.3%.
3. A second independent positive applies the same rule to the 1/12 prior and lands on 9/20. Evidence accumulates multiplicatively.

The two-number play: the test is 90% sensitive, yet one positive means a 1-in-12 chance of sickness. The gap is the base rate, and no accuracy figure closes it without the evidence denominator.

Independence is a factorization, mml definition 6.10: $p(x, y) = p(x)p(y)$. The funfair draws satisfy it, $\frac{9}{100} = \left(\frac{3}{10}\right)^2$. Uncorrelated is weaker. Example 6.5 takes X uniform on {-1, 0, 1} and $Y = X^2$: the covariance is $\mathrm{Cov}[x, y] = E[xy] - E[x]E[y] = E[x^3] = 0$ by symmetry, but the pair is dependent since $P(X = 1, Y = 0) = 0$ against the product $\frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}$. Covariance, the book says, "measures only linear dependence". Correlation normalizes it, $\mathrm{corr}[x, y] = \frac{\mathrm{Cov}[x,y]}{\sqrt{V[x]V[y]}} \in [-1, 1]$, and on the screening indicators the sample pins $\mathrm{corr}^2 = \frac{44}{669}$ exactly, inside the Cauchy-Schwarz bound.

```
// fixed screening table, 1000 people: prevalence 1/100, sensitivity 9/10,
// false-positive rate 1/10 (99 positives among the 990 well)
static const long long tbl[2][2] = {
    {9, 1},    // sick: test positive, test negative
    {99, 891}, // well: test positive, test negative
};

static struct frac joint(enum health h, enum reading r) {
  return frac_make(tbl[h][r], 1000);
}

static struct frac marginal_health(enum health h) {
  return fadd(joint(h, POS), joint(h, NEG));
}

static struct frac marginal_reading(enum reading r) {
  return fadd(joint(SICK, r), joint(WELL, r));
}

// P(r | h): condition down a column of the joint table
static struct frac cond(enum health h, enum reading r) {
  return fdiv(joint(h, r), marginal_health(h));
}

// P(h | r): bayes inversion of the same table
static struct frac bayes(enum health h, enum reading r) {
  return fdiv(joint(h, r), marginal_reading(r));
}
```

listing 191: bayes kernel, condition and invert over one fixed joint table (source: math/samples/src/Ch18/bayes.c, lines 75-102)

```
  // the two-number play: 90% sensitive test, 1% base rate, 1-in-12 posterior
  CHECK(feq(bayes(SICK, POS), frac_make(1, 12)),
        "bayes: P(sick|+) = 9/108 = 1/12");

  // a second independent positive moves 1/12 to 9/20
  auto post = bayes(SICK, POS);
  auto again = fdiv(fmul(frac_make(9, 10), post),
                    fadd(fmul(frac_make(9, 10), post),
                         fmul(frac_make(1, 10), fsub(one, post))));
  CHECK(feq(again, frac_make(9, 20)), "second positive: 1/12 -> 9/20");
```

listing 192: two-number play, one positive then a second, exact rationals (source: math/samples/src/Ch18/bayes.c, lines 137-146)

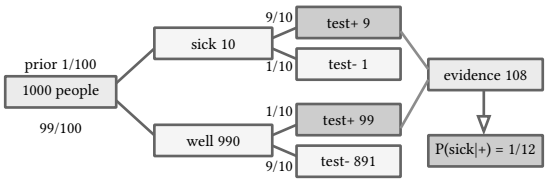


figure 109: bayes tree for the screening table, counts out of 1000

18.3. discrete random variables

For a discrete target space the distribution is the probability mass function $P(X = x)$, and probability beyond a point rides the cumulative distribution function $F_X(x) = P(X \le x)$. The expected value of a function of X, mml definition 6.3, is $E_X[g(x)] = \sum_{x \in \mathcal{X}} g(x)p(x)$, and the mean is the identity special case. Two properties carry

most of the load: linearity, $E[ag(x) + bh(x)] = aE[g(x)] + bE[h(x)]$, needs no independence assumption, and the variance has three faces. The definition is $V_X[x] = E\left[(x - \mu)^2\right]$, the raw-score form is

$$V_X[x] = E_X[x^2] - (E_X[x])^2,$$

and the pairwise identity $\frac{1}{N^2}\sum_{i,j}(x_i - x_j)^2 = 2\left[\frac{1}{N}\sum_i x_i^2 - \left(\frac{1}{N}\sum_i x_i\right)^2\right]$ rewrites the empirical variance as distances from the center.

The dry run: two fair dice, X the sum, counts 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1 over the 36 outcomes.

1. Direct summation gives $E[X] = \frac{252}{36} = 7$, and linearity agrees without summing: $E[d_1] + E[d_2] = \frac{7}{2} + \frac{7}{2}$.
2. $E[X^2] = \frac{1974}{36} = \frac{329}{6}$, so both variance routes land on $\frac{329}{6} - 49 = \frac{35}{6}$.
3. The cdf stops at $F(7) = \frac{21}{36} = \frac{7}{12}$.
4. On the fixed sample $\{0, 2, 2, 5, 6\}$ the pairwise sum is 240 over 25 pairs, $\frac{48}{5}$, which is twice the empirical variance $\frac{24}{5}$.

Linearity over indicators is the quiet superpower. The expected heads in 3 tosses is $\frac{1}{2} + \frac{1}{2} + \frac{1}{2} = \frac{3}{2}$ with no case analysis, and the expected fixed points of a random permutation of 4 elements is $4 \cdot \frac{1}{4} = 1$, which the sample confirms by averaging over all 24 permutations, 24 fixed points in total. The book's geometric coda ties covariance to book 2, ch 7 (mathematics for programmers, inner products and projections): for zero-mean variables $\langle X, Y\rangle := \mathrm{Cov}[x, y]$ is an inner product, the length $\|X\| = \sqrt{V[x]}$ is the standard deviation, and the correlation is the cosine of the angle between two random variables.

```
// E[X^2] = 329/6, variance both ways = 35/6 (mml eq 6.43 vs 6.44)
struct frac ex2 = frac_make(0, 1), vdev = frac_make(0, 1);
for (int s = 2; s ≤ 12; s++) {
  ex2 = fadd(ex2, fmul(frac_make(s * s, 1), frac_make(cnt[s], 36)));
  long long dev = s - 7;
  vdev = fadd(vdev, fmul(frac_make(dev * dev, 1), frac_make(cnt[s], 36)));
}
CHECK(feq(ex2, frac_make(329, 6)), "E[X^2] = 1974/36 = 329/6");
CHECK(feq(vdev, frac_make(35, 6)), "V = E[(X-7)^2] = 35/6");
CHECK(feq(fsub(ex2, fmul(ex, ex)), frac_make(35, 6)),
      "V = E[X^2] - E[X]^2 = 329/6 - 49 = 35/6");

// cdf F(7) = P(X ≤ 7) = 21/36 = 7/12
long long below = 0;
for (int s = 2; s ≤ 7; s++)
  below += cnt[s];
CHECK(feq(frac_make(below, 36), frac_make(7, 12)), "F(7) = 21/36 = 7/12");
```

listing 193: moment loop, E[X^2] and both variance routes on the dice table (source: math/samples/src/Ch18/discrete.c, lines 89-105)

```
  // expected fixed points of a permutation of 4: enumerate all 24
  long long tot = 0, perms = 0;
  int p[4];
  for (p[0] = 0; p[0] < 4; p[0]++)
    for (p[1] = 0; p[1] < 4; p[1]++)
      for (p[2] = 0; p[2] < 4; p[2]++)
        for (p[3] = 0; p[3] < 4; p[3]++) {
          int seen = 0;
          for (int i = 0; i < 4; i++)
            seen |= 1 << p[i];
          if (seen != 0b1111)
            continue;
          perms++;
          for (int i = 0; i < 4; i++)
            tot += p[i] == i;
        }
  // enumeration average = indicator bound 4 * 1/4 = 1
  CHECK(perms == 24 && feq(frac_make(tot, perms), one) &&
            feq(fmul(frac_make(4, 1), frac_make(1, 4)), one),
        "E[fixed points] = 24/24 = 4 * 1/4 = 1");
```

listing 194: indicator bound, all 24 permutations enumerated against 4 times 1/4 (source: math/samples/src/Ch18/discrete.c, lines 128-147)

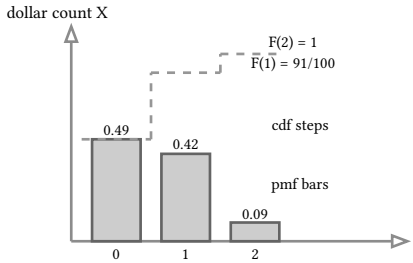


figure 110: two-coin pmf bars with cdf steps, heights exact

18.4. binomial and Poisson families

The bernoulli is one weighted coin, $p(x \mid \mu) = \mu^x(1-\mu)^{1-x}$ for $x \in \{0, 1\}$, with $E[x] = \mu$ and $V[x] = \mu(1-\mu)$. The binomial, mml example 6.9, counts the successes in N independent draws:

$$p(m \mid N, \mu) = \binom{N}{m}\mu^m(1-\mu)^{N-m}, \quad E[m] = N\mu, \quad V[m] = N\mu(1-\mu)$$

The binomial coefficients come from chapter 14's Pascal kernel, so the whole family is counting plus one parameter. For rational μ the pmf is exact integer arithmetic.

The dry run: $N = 15$, $\mu = \frac{3}{10}$, the funfair coin drawn 15 times.

1. Each pmf entry is $\binom{15}{m}3^m7^{15-m}$ over the common denominator 10^{15}, and the numerators sum to $(3+7)^{15} = 10^{15}$ exactly, the binomial theorem as a normalization check.
2. $E[m] = \frac{45}{10} = \frac{9}{2}$ and $E[m^2] - E[m]^2 = \frac{63}{20}$, both matching the closed forms $N\mu$ and $N\mu(1-\mu)$.
3. The mode is $m = 4$ with pmf 0.218623131339795, and the neighbors sit below it on both sides.

The Poisson is what the binomial becomes when successes are rare: fix $\lambda = N\mu$ and let N grow, and the pmf converges to $e^{-\lambda}\lambda^m/m!$. The sample watches it happen at $\lambda = 2$, comparing $(1-\mu)^N$ against e^{-2} for $N = 50, 100, 200$. The gaps are −5.449e-3, −2.716e-3, −1.356e-3, roughly halving each time N doubles, and at $N = 200$ the $m = 1$ entry is already within 4.557e-6 of $2e^{-2}$. No sampling noise, just the limit observed on fixed traces.

```
  // binomial (mml example 6.9): N = 15, mu = 3/10, exact numerators
  // over 10^15, summing to (3 + 7)^15
  constexpr int n = 15;
  constexpr long long den = 1000000000000000LL;
  static_assert(den == 1000000000000000LL, "10^15 denominator for N = 15");
  long long num[16];
  for (int k = 0; k ≤ n; k++) {
    long long c = 1; // C(15, k), the ch 14 pascal kernel
    for (int i = 0; i < k; i++)
      c = c * (n - i) / (i + 1);
    long long p3 = 1, p7 = 1;
    for (int i = 0; i < k; i++)
      p3 *= 3;
    for (int i = 0; i < n - k; i++)
      p7 *= 7;
    num[k] = c * p3 * p7;
  }
  typeof(num[0]) s = 0, sk = 0, sk2 = 0;
  for (int k = 0; k ≤ n; k++) {
    s += num[k];
    sk += (long long)k * num[k];
    sk2 += (long long)k * k * num[k];
  }
  CHECK(s == den, "binomial pmf sums to (3+7)^15 = 10^15");
  CHECK(feq(frac_make(sk, den), frac_make(9, 2)), "E[m] = N*mu = 9/2");
  CHECK(feq(fsub(frac_make(sk2, den),
                 fmul(frac_make(sk, den), frac_make(sk, den))),
            frac_make(63, 20)) &&
            feq(fmul(frac_make(n, 1), fmul(frac_make(3, 10), frac_make(7, 10))),
                frac_make(63, 20)),
        "V[m] = E[m^2] - E[m]^2 = N*mu*(1-mu) = 63/20");
  CHECK(num[4] == 218623131339795LL && num[3] < num[4] && num[4] > num[5],
        "mode at k = 4, numerator 218623131339795");
  CHECK(fabs((double)num[4] / (double)den - 0.218623131339795) < 1e-15,
        "pmf(4) = 0.218623131339795");
```

listing 195: binomial kernel, exact numerators over 10^15 with closed-form moments (source: math/samples/src/Ch18/discrete.c, lines 149-183)

```
  // poisson as the rare-event limit: lambda = 2 fixed, mu = lambda/n -> 0
  // pinned gaps (precompute.py): n = 50, 100, 200 give -5.449e-3,
  // -2.716e-3, -1.356e-3 on k = 0, roughly halving as n doubles
  CHECK(fabs(pow(0.99, 200.0) - exp(-2.0)) < 1.4e-3 &&
            fabs(200.0 * 0.01 * pow(0.99, 199.0) - 2.0 * exp(-2.0)) < 5e-6,
        "n = 200: binomial k = 0, 1 match poisson(2)");
  double d[3];
  int ns[3] = {50, 100, 200};
  for (int i = 0; i < 3; i++) {
    double mu = 2.0 / ns[i];
    d[i] = pow(1.0 - mu, (double)ns[i]) - exp(-2.0);
  }
  CHECK(d[0] < d[1] && d[1] < d[2] && d[2] < 0.0,
        "gap to e^-2 shrinks monotonically over n = 50, 100, 200");
```

listing 196: poisson limit, the gap to e^minus2 halving as n doubles (source: math/samples/src/Ch18/discrete.c, lines 185-198)

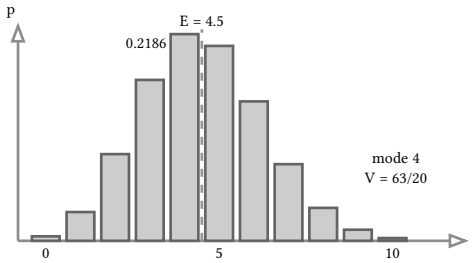


figure 111: binomial pmf bars for N = 15, mu = 0.3, mean line at 4.5

18.5. continuous densities and the gaussian

A pdf, mml definition 6.1, is any $f \geq 0$ with $\int f(x)\,\mathrm{d}x = 1$, and probability is area, $P(a \leq X \leq b) = \int_a^b f(x)\,\mathrm{d}x$. Point probability evaporates, $P(X = x)$ is "a set of measure zero", and the cdf $F_X(x) = P(X \leq x)$ carries the recoverable information. Normalization does not cap the height: the book's uniform on $[0.9, 1.6]$ has density $\frac{1}{0.7} \approx 1.43$, greater than 1 everywhere it lives.

The gaussian is the family everything else wants to be:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

The central limit theorem, the book notes, makes it "arise naturally when we consider sums of independent and identically distributed random variables". Two algebraic facts do the rest of its work. Standardization: any gaussian probability is a standard normal one, since $\frac{X-\mu}{\sigma}$ is $\mathcal{N}(0,1)$, so one table of Φ values serves every (μ, σ) pair. Affine closure: a mixture of two gaussian densities with weight α, theorem 6.12, keeps the weighted mean $E[x] = \alpha\mu_1 + (1-\alpha)\mu_2$ and a variance with a between-component term,

$$V[x] = \left[\alpha\sigma_1^2 + (1-\alpha)\sigma_2^2\right] + \left[\alpha\mu_1^2 + (1-\alpha)\mu_2^2\right] - \left[\alpha\mu_1 + (1-\alpha)\mu_2\right]^2.$$

Marginals and conditionals of a joint gaussian stay gaussian, the computation the Kalman filter runs every step, and sampling a general gaussian is a Cholesky factor away from the standard one, book 2, ch 8 (mathematics for programmers, matrix decompositions).

The dry run: simpson quadrature on the standard density, 4000 panels, with expectations pinned to 1e-12 above the measured error 5e-15. The error theory itself is book 2, ch 22 (mathematics for programmers, quadrature and differential equations) territory.

1. The central bands are 0.6826894921370857, 0.9544997361036417, 0.9973002039367398, the 68-95-99.7 rule with all digits.
2. For $X \sim \mathcal{N}(1, 0.25)$ the tail $P(X \leq 2)$ equals $\Phi(2)$, and the central band $P(|X-1| \leq 1)$ equals the standard $P(|Z| \leq 2)$ to 1e-12, standardization as a pinned identity rather than a slogan.
3. Moments by quadrature: $E[X] = 1$ and $E[X^2] = 1.25$, so $V = 0.25$.
4. The mixture $0.3\mathcal{N}(-1, 0.5) + 0.7\mathcal{N}(2, 1.5)$ integrates to 1 with mean 1.1 and variance 3.09, and theorem 6.12′s closed forms agree to 1e-9.

```c
// standard normal density (mml eq 6.62 with mu = 0, sigma^2 = 1)
static double phi(double z) { return exp(-0.5 * z * z) / sqrt(2.0 * pi); }

// composite simpson on an even panel count, the same driver the dsa
// numerical chapter pins; chapter 22 treats the error theory
static double simpson(double (*f)(double), double a, double b, int n) {
  double h = (b - a) / n;
  double s = f(a) + f(b);
  for (int i = 1; i < n; i++)
    s += f(a + i * h) * (i % 2 ? 4.0 : 2.0);
  return s * h / 3.0;
}
```

listing 197: density and simpson driver, the quadrature leg every check rides (source: math/samples/src/Ch18/gaussian.c, lines 81-92)

```c
  // the 68-95-99.7 bands, simpson against erf-based truth
  // (precompute.py simpson error at 4000 panels: under 5e-15)
  CHECK(fabs(simpson(phi, -1.0, 1.0, 4000) - 0.6826894921370857) < 1e-12,
        "P(|Z| ≤ 1) = 0.6826894921370857");
  CHECK(fabs(simpson(phi, -2.0, 2.0, 4000) - 0.9544997361036417) < 1e-12,
        "P(|Z| ≤ 2) = 0.9544997361036417");
  CHECK(fabs(simpson(phi, -3.0, 3.0, 4000) - 0.9973002039367398) < 1e-12,
        "P(|Z| ≤ 3) = 0.9973002039367398");

  // standardization: for N(1, 0.25), the z-score (x-1)/0.5 carries the
  // probability. tail over [-9, 2] against Phi(2) (measured err 5.5e-13)
  CHECK(fabs(simpson(scaled, -9.0, 2.0, 4000) - 0.9772498680518208) < 1e-9,
        "P(X ≤ 2) = Phi((2-1)/0.5) = Phi(2)");
  CHECK(fabs(simpson(scaled, 0.0, 2.0, 4000) - simpson(phi, -2.0, 2.0, 4000)) <
            1e-12,
        "P(|X-1| ≤ 2*0.5) = P(|Z| ≤ 2)");
```

listing 198: band and standardization checks, erf-based truth at 1e-12 (source: math/samples/src/Ch18/gaussian.c, lines 133-148)

```c
  // theorem 6.12 by quadrature: mixture normalizes, mean and variance hit
  // the weighted-sum closed forms exactly (measured errs 7e-14, 8e-13, 7e-12)
  double nrm = simpson(mix, -8.0, 11.0, 20000);
  double m1q = simpson(x_mix, -8.0, 11.0, 20000);
  double varq = simpson(x2_mix, -8.0, 11.0, 20000) - m1q * m1q;
  double mean_closed = w1 * mu1 + w2 * mu2;
  double var_closed = (w1 * v1 + w2 * v2) + (w1 * mu1 * mu1 + w2 * mu2 * mu2) -
                      mean_closed * mean_closed;
  CHECK(fabs(nrm - 1.0) < 1e-10, "mixture density integrates to 1");
  CHECK(fabs(m1q - 1.1) < 1e-10 && fabs(m1q - mean_closed) < 1e-10,
        "mixture mean = 0.3*(-1) + 0.7*2 = 1.1");
  CHECK(fabs(varq - 3.09) < 1e-9 && fabs(varq - var_closed) < 1e-9,
        "mixture variance = 3.09 = theorem 6.12 closed form");
```

listing 199: theorem 6.12 by quadrature, closed forms against the integrals (source: math/samples/src/Ch18/gaussian.c, lines 155-167)

PITFALL: densities are not probabilities

A pmf entry is a probability and cannot exceed 1. A pdf value is a density and routinely does: the uniform on $[0.9, 1.6]$ sits at height 1.43 across an interval of length 0.7, and $1.43 \cdot 0.7 = 1$. Only the integral over a set is a probability. Treating $p(x)$ as $P(X = x)$ is the standing error, and for continuous variables that quantity is exactly zero.

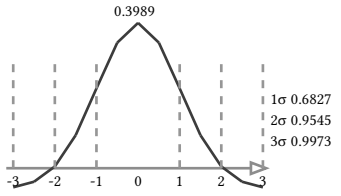


figure 112: standard gaussian with the 1, 2, and 3 sigma bands

18.6. conjugacy, the exponential family, and change of variables

Bayesian updating multiplies a prior by a likelihood, and conjugacy, mml definition 6.13, is when "the posterior is of the same form/type as the prior", so the update is arithmetic on parameters instead of integration. The beta prior over a coin weight,

$$p(\mu \mid \alpha, \beta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}\mu^{\alpha-1}(1-\mu)^{\beta-1},$$

is conjugate to the binomial. For integer α and β the gamma normalizer collapses to factorials, $\int_0^1 \mu^{a-1}(1-\mu)^{b-1}\,\mathrm{d}\mu = \frac{(a-1)!(b-1)!}{(a+b-1)!}$, and every moment becomes exact integer arithmetic.

The dry run: prior Beta(2, 3), then 3 heads in 5 flips.

1. The prior normalizer is $\frac{4!}{1!\cdot 2!} = 12$, the mean is $E[\mu] = \frac{2}{5}$, and the variance is $\frac{1}{25}$, matching the closed form $\alpha\frac{\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$.
2. The likelihood $\mu^3(1-\mu)^2$ lifts the exponents, so the posterior is $\mathrm{Beta}(2+3, 3+5-3) = \mathrm{Beta}(5,5)$ with normalizer $\frac{9!}{4!4!} = 630$ and mean $\frac{1}{2}$.
3. The posterior density at $\mu = \frac{1}{2}$ is $\frac{630}{2^8} = \frac{315}{128}$, exactly 2.4609375, and the mean $\frac{1}{2}$ lands between the prior mean $\frac{2}{5}$ and the data mean $\frac{3}{5}$.

Conjugacy is a corollary of a bigger structure. The exponential family, $p(x \mid \theta) = h(x)\exp(\theta^\top \varphi(x) - A(\theta))$, carries all parameter dependence through the sufficient statistics $\varphi(x)$, and the Fisher-Neyman theorem says these are the only families with finite-dimensional sufficient statistics under repeated sampling. The gaussian sits inside with $\varphi(x) = \binom{x}{x^2}$ and natural parameters $\theta = \begin{pmatrix}\frac{\mu}{\sigma^2} \\ -\frac{1}{2\sigma^2}\end{pmatrix}$, which the sample verifies as the identity $\theta^\top\varphi(x) + \frac{(x-\mu)^2}{2\sigma^2} = \frac{\mu^2}{2\sigma^2}$ at three points. The bernoulli sits inside with $\varphi(x) = x$ and $\theta = \log\left(\frac{\mu}{1-\mu}\right)$, whose inverse map is the sigmoid.

Change of variables moves distributions between scales. The distribution-function technique for $Y = X^2$ under $f(x) = 3x^2$ on [0, 1], mml example 6.16, runs $F_Y(y) = P(X^2 \le y) = F_X\left(y^{\frac{1}{2}}\right) = y^{\frac{3}{2}}$, so

$$f(y) = \frac{3}{2}y^{\frac{1}{2}},$$

pinned by quadrature: the new density integrates to 1, $E[Y] = E[X^2] = \frac{3}{5}$ from either side of the transformation, and $F_Y\left(\frac{1}{4}\right) = \frac{1}{8}$. The general rule, theorem 6.16, substitutes the inverse and multiplies by the absolute Jacobian determinant, $f(y) = f_x(U^{-1}(y)) \cdot |\det\left(\partial U^{-1}\frac{y}{\partial}y\right)|$. The affine case is the one to memorize: $Y = aX + b$ divides the density by $|a|$ at the reflected point, moves the mean to $a\mu + b$, and scales the variance by a^2, all three checked on $Y = 2X + 1$ with $V[Y] = 4 \cdot \frac{3}{80} = \frac{3}{20}$. Theorem 6.15 runs the pipeline backwards, $F_X(X)$ is uniform, which is how inverse cdf sampling works.

Chapter 19, book 2, ch 19 (mathematics for programmers, statistics and inference), turns this vocabulary into inference: empirical means and covariances, maximum likelihood through the exponential family's concave log-likelihood, and posterior updating at scale.

```
// beta-binomial conjugacy (mml example 6.11) in exact integer arithmetic:
// int_0^1 mu^(a-1)(1-mu)^(b-1) dmu = (a-1)!(b-1)!/(a+b-1)!
struct frac b23 = frac_make(fact(1) * fact(2), fact(4)); // 1/12
struct frac b33 = frac_make(fact(2) * fact(2), fact(5)); // 1/30
struct frac b43 = frac_make(fact(3) * fact(2), fact(6)); // 1/60
CHECK(feq(fdiv(one, b23), frac_make(12, 1)) &&
          feq(fdiv(b33, b23), frac_make(2, 5)),
      "prior Beta(2,3): normalizer 12, E[mu] = 2/5");
CHECK(feq(fsub(fdiv(b43, b23), fmul(frac_make(2, 5), frac_make(2, 5))),
          frac_make(1, 25)) &&
          feq(fdiv(frac_make(2 * 3, 1), frac_make(25 * 6, 1)),
              frac_make(1, 25)),
      "prior V[mu] = 1/25 = alpha*beta/((a+b)^2(a+b+1))");

// 3 heads in 5 flips: posterior Beta(2+3, 3+2) = Beta(5,5)
struct frac b55 = frac_make(fact(4) * fact(4), fact(9));  // 1/630
struct frac b65 = frac_make(fact(5) * fact(4), fact(10)); // 1/1260
CHECK(feq(fdiv(one, b55), frac_make(630, 1)) &&
          feq(fdiv(b65, b55), frac_make(1, 2)),
      "posterior Beta(5,5): normalizer 630, mean 1/2");
CHECK(fabs(630.0 * pow(0.5, 8.0) - 315.0 / 128.0) < 1e-15,
      "posterior density at mu = 1/2 is 630/256 = 315/128");
CHECK(fle(frac_make(2, 5), frac_make(1, 2)) &&
          fle(frac_make(1, 2), frac_make(3, 5)),
      "posterior mean 1/2 sits between prior 2/5 and data mean 3/5");
```

listing 200: beta-binomial update, exact factorial arithmetic from prior to posterior (source: math/samples/src/Ch18/gaussian.c, lines 200-224)

```
// change of variables (mml example 6.16): Y = X^2 under f(x) = 3x^2
// sqrt endpoint caps the panel rate: measured err 4.3e-8 at 20000 panels
CHECK(fabs(simpson(fy, 0.0, 1.0, 20000) - 1.0) < 1e-6,
      "f_Y(y) = 1.5*sqrt(y) integrates to 1");
CHECK(fabs(simpson(y_fy, 0.0, 1.0, 4000) - 0.6) < 1e-9 &&
          fabs(simpson(three_x4, 0.0, 1.0, 400) - 0.6) < 1e-9,
      "E[Y] = E[X^2] = 3/5 = 0.6 both ways");
CHECK(fabs(simpson(fy, 0.0, 0.25, 20000) - 0.125) < 1e-6 &&
          fabs(pow(0.25, 1.5) - 0.125) < 1e-15,
      "F_Y(1/4) = (1/4)^(3/2) = 1/8");
```

listing 201: change-of-variables checks, squared and affine pushes pinned by quadrature (source: math/samples/src/Ch18/gaussian.c, lines 181-190)

VERIFY: the posterior sandwich

Conjugate updating cannot jump past the data. The prior mean is $\frac{2}{5}$, the observed head rate is $\frac{3}{5}$, and the posterior mean $\frac{1}{2}$ lands strictly between them, closer to the data because five flips outweigh a weak prior. The sample asserts the sandwich $\frac{2}{5} < \frac{1}{2} < \frac{3}{5}$ in exact rationals, and the same ordering holds for every conjugate update, a cheap sanity check on any Bayesian implementation.

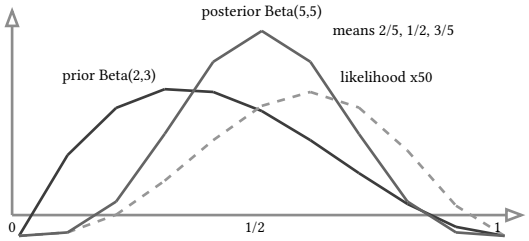


figure 113: prior, likelihood, and posterior on the coin weight

sources: mml-book draft 2024-01-15, Deisenroth, Faisal, Ong, “Mathematics for Machine Learning”, ch 6, pp 172-221 (printed), verified against the cached pdf pages 178-227 on 2026-09-22, all definitions and equations quoted or paraphrased from those pages. cppreference, “Common mathematical functions”, en.cppreference.com/w/c/numeric/math, fetched 2026-09-22, the exp, pow, sqrt, and fabs calls the samples lean on. Sample behavior verified by pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch18, 60 checks in chapter 18 of the math suite, zero failures, format clean.

19. statistics and inference

Chapter book 2, ch 18 (mathematics for programmers, probability and distributions) built the forward direction, model to data: distributions, expectations, the conjugate Beta-Bernoulli pair. This chapter runs the inverse direction, data to model, in six moves: a seeded sampling stream and the sample mean, the law of large numbers as a pinned ladder, the central limit theorem priced as an exact binomial tail against its normal approximation, estimators with bias and variance settled by enumerating tiny populations, maximum likelihood and maximum a posteriori as derivative solves, and the two inference contracts, exact tests with confidence intervals plus least squares as orthogonal projection. Every behavioral claim below is one of the 63 checks in the 4 samples of chapter 19 or a sentence quoted from a canonical source fetched 2026-09-21. The dsa book keeps randomness out of its algorithms (book 9, ch 36 (practical data structures and algorithms, numerical methods)); this chapter does the opposite discipline, the randomness is pinned to a fixed xorshift64* trace so every statistic is reproducible to the bit.

19.1. sampling and the sample mean

A statistic is a deterministic function of the data, in mml's words "a deterministic function of that random variable" [printed p 186 / pdf p 192]. The empirical mean is the arithmetic average $|x| = 1/N \sum_{n=1}^{N} x_n$ (mml Definition 6.9, eq 6.41 [printed p 192 / pdf p 198]), and mml states the divisor convention outright in the margin of that page: "Throughout the book, we use the empirical covariance, which is a biased estimate. The unbiased (sometimes called corrected) covariance has the factor N - 1 in the denominator instead of N." The variance has three faces there (eqs 6.43-6.45 [printed pp 192-193 / pdf pp 198-199]): the definition $V[x] = E[(x - \mu)^2]$, the raw-score one-pass $V[x] = E[x^2] - E[x]^2$ "the mean of the square minus the square of the mean", which is "numerically unstable" when the two terms nearly cancel, and the pairwise form.

The samples here never touch the heap or the clock. One xorshift64* generator (Vigna's three shifts and the odd multiplier) advances a fixed 64-bit state, and one uniform decimal digit comes from each 31-bit draw by rejecting the top 8 of the 2^{31} values, so the accepted range $2^{31} - 8 = 2147483640$ is an exact multiple of 10 and the digit is uniform by construction, not approximately. The whole 10000-draw trace is materialized once and every statistic reads the same fixed array, the way a real dataset would.

The dry run: the trace under seed 0x9E3779B97F4A7C15 opens 7, 8, 3, 5, 6, 9, 8, 9, 3, 4, 4, 6, its first 100 digits sum to 419, and the indicator $x_i = 1[\text{digit} < 3]$ turns it into an exact Bernoulli stream with $p = 3/10$.

1. The Bernoulli count in the first 100 draws is $k = 31$, so $|x| = 31/100 = 0.31$ and the raw-score shortcut gives the centered sum $k - k^2/n = 21.39$, hence $s^2 = 21.39/99 = 0.216061$.
2. The population behind the digits has mean 4.5 and variance $99/12 = 8.25$. Over all 10000 draws the two-pass variance lands at 8.289188439999835 and the raw-score one-pass at 8.28918844, differing by 1.65e-13, the cancellation mml warns about, visible at n = 10000 already.
3. Chebyshev prices the tail of the mean: at $\text{eps} = 0.05$ and $n = 10000$ the bound $0.21/(n\text{eps}^2) = 0.0084$, and the observed deviation 0.0025 sits inside it with room.

```c
// the rejection threshold 2^31 - 8 = 2147483640 must be an exact multiple of
// 10, otherwise the accepted digits are not uniform
static_assert(2147483640 % 10 == 0, "accepted range must tile by decades");

// xorshift64* (Vigna): three shifts update the state, the output is the state
// scaled by a fixed odd constant. seed pinned below, so the trace is fixed.
static uint64_t rng_state = 0x9E3779B97F4A7C15;

static uint64_t xs64s(void) {
  rng_state ^= rng_state >> 12;
  rng_state ^= rng_state << 25;
  rng_state ^= rng_state >> 27;
  return rng_state * 0x2545F4914F6CDD1DULL;
}

// one uniform decimal digit: 31-bit draws, reject the top 8 of the 2^31 values
static int digit10(void) {
  for (;;) {
    uint32_t d = (uint32_t)(xs64s() >> 33);
    if (d < 2147483640u)
      return (int)(d % 10u);
  }
}
```

listing 202: the seeded stream and the exact-uniform digit, sampling.c (source: math/samples/src/Ch19/sampling.c, lines 22-44)

The law of large numbers says $|x|_n \to p$ in probability (Wasserman 2004, ch 5, by name). The pinned ladder makes that concrete on the fixed trace: the deviation from p falls by a factor 120 while n grows by a factor 1000.

n	k	k/n	|k/n - 0.3|
10	0	0.0	0.3
100	31	0.31	0.01
1000	304	0.304	0.004
10000	3025	0.3025	0.0025

```
// lln ladder on one stream: k/n and the deviation from p = 3/10
const int rungs[] = {10, 100, 1000, 10000};
const int ks[] = {0, 31, 304, 3025};
const double devs[] = {-0.3, 0.010000000000000009, 0.0040000000000000036,
                       0.0025000000000000022};
for (unsigned r = 0; r < 4; r++) {
  int k = 0;
  for (int i = 0; i < rungs[r]; i++)
    k += trace[i] < 3;
  double dev = (double)k / (double)rungs[r] - 3.0 / 10.0;
  CHECK(k == ks[r] && fabs(dev - devs[r]) < 1e-15, "lln rung pinned");
}

// chebyshev envelope for the indicator mean: sigma^2 = 0.21, so at
// eps = 0.05 and n = 10000 the bound is 0.21/(10000 * 0.05^2) = 0.0084,
// and the observed deviation 0.0025 sits well inside it
double cheb = (21.0 / 100.0) / (10000.0 * 0.05 * 0.05);
CHECK(fabs(cheb - 0.0084) < 1e-15 && fabs(devs[3]) < 0.05,
      "chebyshev envelope 0.0084 at eps 0.05");

// variance of the 10000 digits, population value 8.25 = 99/12:
// two-pass (eq 6.43) vs raw-score one-pass (eq 6.44) agree to 1.65e-13 but
// are not bit-equal, the instability mml warns about
moments m = digit_moments(trace, N);
double mean = (double)m.sum / (double)N;
double acc = 0.0;
for (int i = 0; i < N; i++) {
  double t = (double)trace[i] - mean;
  acc += t * t;
}
double v_two = acc / (double)N;
double v_raw = (double)m.sumsq / (double)N - mean * mean;
CHECK(fabs(v_two - 8.289188439999835) < 1e-12, "two-pass variance 8.289188");
double d = v_two - v_raw;
CHECK(d != 0.0 && fabs(d) < 1e-12, "two-pass vs raw differ by 1.65e-13");
CHECK(fabs(v_two - 8.25) < 0.05, "empirical variance near population 8.25");
```

listing 203: the lln rungs with their pinned counts, the chebyshev envelope, and the two-pass versus raw-score variance, sampling.c (source: math/samples/src/Ch19/sampling.c, lines 94-129)

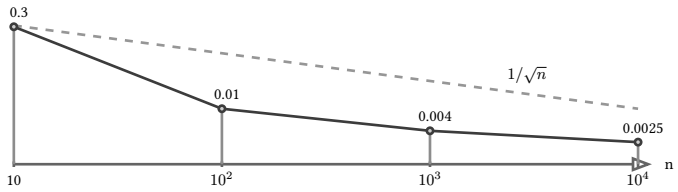


figure 114: the lln ladder on one fixed stream, deviation against n, with the 1/sqrt(n) guide

The two-number play: the first rung misses by 0.3 because 0 of the first 10 digits are 0, 1, or 2, an event of probability $0.7^{10} = 0.028$, and the last rung misses by 0.0025. The honest reading of a 4-rung ladder is not proof, the proof is Chebyshev's inequality, the ladder is what convergence looks like on data you can rerun bit for bit.

VERIFY: THE STREAM IS THE POPULATION

Every sampled number in this chapter is a deterministic function of one 64-bit seed. The pin script in the repo replays each generator in exact integer arithmetic and prints every expected value, so a check that passes is a replication, not a probability. Change the seed and every ladder in this chapter changes, which is exactly why the seed lives in a comment next to the pins.

19.2. the central limit theorem

Standardize a sum. For Bernoulli(p) indicators, linearity gives $E[S_n] = np$ and, since indicators are independent, variances add the way mml eq 6.58 states for uncorrelated variables, $V[x+y] = V[x] + V[y]$ [printed pp 195-196 / pdf pp 201-202], so $V[S_n] = np(1-p)$. The central limit theorem (Wasserman 2004, ch 5, by name) says the standardized sum $z_n = (S_n - np)/\sqrt{np(1-p)}$ converges in distribution to the standard normal, and for coin flips this is the De Moivre-Laplace theorem, the oldest case.

The dry run: a second stream under seed 0xDEADBEEFCAFEF00D feeds the same Bernoulli(3/10) rule, and the standardized sums walk through units.

1. At $n = 5$ the trace holds $S = 0$ successes, $z = -1.4639$.
2. At $n = 20$, $S = 5$, $z = -0.4880$. At $n = 80$, $S = 25$, $z = +0.2440$.
3. At $n = 320$, $S = 117$, $z = +2.5617$, and the exact binomial tail $P(X \geq 117) = 0.006883$, the p-value scale the next sections use.

The normal side needs Φ, and the honest route on doubles is the complementary error function: $\Phi(x) = 1/2 \operatorname{erfc}\left(-x/\sqrt{2}\right)$, erfc defined by DLMF 7.2.E2 as $(2/\sqrt{\pi}) \int_z^\infty e^{-t^2}\,\mathrm{d}t$ (fetched 2026-09-21) and exposed by C23 as `erfc` with the same formula (cppreference, fetched 2026-09-21). The samples verify the table cells rather than trusting the constant: $\Phi(0) = 0.5$, $\Phi(1.96) = 0.9750021048517796$, the two-sided 95% row of any printed table, and the pinned quantile $z_{0.975} = 1.9599639845400545$ verified by feeding it back through Φ.

The exact side needs no approximation at all. With $p = 1/2$ the binomial cdf is a ratio of integers, $P(X \leq k) = \left(\sum_{i=0}^{k} C(n,i)\right)/2^n$, and the samples carry the sum in `unsigned __int128`, exact through $n = 100$ where $C(100, 50) \approx 1.01 \times 10^{29}$. The n = 40 cell: $C(40, 20) = 137846528820$, so $P(X \leq 20) = 618679078298/2^{40} = 0.5626853438097896$, against the continuity-corrected normal value $\Phi\left(0.5/\sqrt{10}\right) = 0.5628164694185541$, an error of 1.31e-4.

```
static double u128_to_double(u128 x) {
  uint64_t hi = (uint64_t)(x >> 64), lo = (uint64_t)x;
  return (double)hi * 0x1p64 + (double)lo;
}

// standard normal cdf: Phi(x) = (1/2) erfc(-x/sqrt(2)), dlmf 7.2.E2 plus the
// symmetry erfc(-z) = 2 - erfc(z)
static double phi_std(double x) { return 0.5 * erfc(-x / SQRT2); }

static uint64_t rng_state = 0xDEADBEEFCAFEF00D;

static uint64_t xs64s(void) {
  rng_state ^= rng_state >> 12;
  rng_state ^= rng_state << 25;
  rng_state ^= rng_state >> 27;
  return rng_state * 0x2545F4914F6CDD1DULL;
}

static int digit10(void) {
  for (;;) {
    uint32_t d = (uint32_t)(xs64s() >> 33);
    if (d < 2147483640u)
      return (int)(d % 10u);
  }
}

// sum of C(n, i) for i = 0..k in exact 128-bit integers: the numerator of the
// binomial cdf at p = 1/2. valid while 2^n < 2^127, so n ≤ 100 is safe.
static double binom_cdf_half(int n, int k) {
  u128 c = 1; // C(n, 0)
  u128 acc = 1;
  for (int i = 1; i ≤ k; i++) {
    c = div_small(c * (u128)(n - i + 1), (uint32_t)i);
    acc += c;
  }
  return u128_to_double(acc) / ldexp(1.0, n);
}
```

listing 204: erfc to phi, the seeded digit source, and the exact 128-bit binomial cdf, clt.c (source: math/samples/src/Ch19/clt.c, lines 40-76)

```
// the n = 320 rung read as a p-value: P(X ≥ 117) for binomial(320, 3/10)
// is 0.006883 exact (0.69%), reproduced by the double recurrence
CHECK(fabs(binom_upper_tail(320, 117) - 0.006882981232680589) < 1e-15,
      "upper tail at the 320 rung 0.006883");

// z-table cells: Phi(0) = 1/2, Phi(1.96) = 0.9750021048517796 (the
// two-sided 95% row of any printed normal table), and the two pinned
// quantile constants verified rather than assumed
CHECK(fabs(phi_std(0.0) - 0.5) < 1e-15, "Phi(0) = 0.5");
CHECK(fabs(phi_std(1.96) - 0.9750021048517796) < 5e-13, "Phi(1.96) cell");
CHECK(fabs(phi_std(1.9599639845400545) - 0.975) < 5e-13,
      "z_0.975 = 1.9599639845400545");

// the n = 40 coin-flip cdf cell: C(40,20) = 137846528820 and
// P(X ≤ 20) = 618679078298/2^40 = 0.5626853438097896 against the
// continuity-corrected normal tail Phi(0.5/sqrt(10)) = 0.5628164694185541
u128 c40 = 1;
for (int i = 1; i ≤ 20; i++)
  c40 = div_small(c40 * (u128)(40 - i + 1), (uint32_t)i);
CHECK(c40 == (u128)137846528820ULL, "C(40,20) = 137846528820");
double exact40 = binom_cdf_half(40, 20);
double approx40 = phi_std((20.5 - 40 * 0.5) / sqrt(40 * 0.25));
CHECK(fabs(exact40 - 0.5626853438097896) < 5e-13, "exact binom cdf n=40");
CHECK(fabs(approx40 - 0.5628164694185541) < 5e-13 &&
          fabs(exact40 - approx40) < 1.32e-4,
      "normal approx error 1.31e-4 at n=40");
```

listing 205: the 320-rung tail pinned against its exact integer sum, the z-table cells, and the n = 40 coin-flip cell, clt.c (source: math/samples/src/Ch19/clt.c, lines 119-144)

n	exact P(X <= n/2)	normal approx	error
10	0.623046875	0.6240851830	1.04e-3
20	0.5880985260	0.5884683631	3.70e-4
40	0.5626853438	0.5628164694	1.31e-4
80	0.5444639394	0.5445103537	4.64e-5
100	0.5397946187	0.5398278373	3.32e-5

The error ladder is the theorem's price tag: each halving step of the error needs roughly a doubling of n, and the Berry-Esseen inequality (Wasserman 2004, ch 5, by name) is the matching worst-case guarantee that the error of the normal cdf approximation is $O(1/\sqrt{n})$, here observed shrinking 1.04e-3 to 3.32e-5, a factor 31 on a 10-fold n.

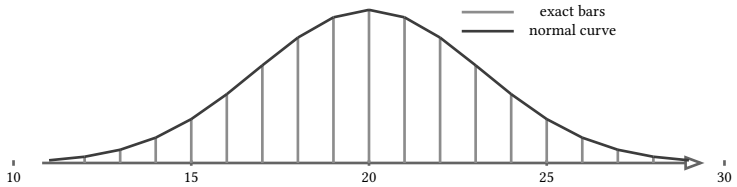


figure 115: binomial(40, 1/2) as exact bars with the normal curve through the bar tops

The two-number play: at $n = 40$ the exact and normal tails agree to 1.31e-4, while the raw, uncorrected normal tail at $\Phi\left(0/\sqrt{10}\right) = 0.5$ would be off by 6.3e-2. The half-integer shift is worth 480 times the error.

NOTE: WHY THE +0.5

> A lattice variable takes integer values, a continuous density does not, and $P(X \le 20)$ corresponds to the normal mass up to 20.5, not 20.0. The continuity correction is the width of one lattice cell moved half a step, and dropping it costs two orders of magnitude, the 6.3e-2 above against the 1.31e-4 in the table.

19.3. estimators, bias, and variance

Bias is $E\left[\hat{\theta}\right] - \theta$. For the sample mean on any population with a mean, enumeration settles it exactly: with data drawn from the uniform population {0, 1, 2}, mean 1 and variance $2/3$, every ordered sample of size n can be listed, 3^n of them, and the estimator averaged without any randomness at all. The divisor question mml flags in that margin note on [printed p 192 / pdf p 198] falls out as exact fractions: the plug-in variance with divisor n averages $\frac{n-1}{n}\sigma^2$, the $n - 1$ divisor averages σ^2 itself.

The dry run: at $n = 2$ there are 9 ordered samples, from (0, 0) to (2, 2).

1. The 9 sample means average exactly 1, unbiased, and their squares average $4/3$, so the variance of the mean is $4/3 - 1 = 1/3 = \sigma^2/n$.
2. The plug-in variance averages $1/3 = \sigma^2/2$ and the $n - 1$ divisor averages $2/3 = \sigma^2$. The mean squared error of $|x|$ is $0 + 1/3$, the bias-variance decomposition checked as numbers, not algebra.
3. The ladder continues at $n = 3$ (27 samples, $4/9$ against $2/3$) and $n = 4$ (81 samples, $1/2$ against $2/3$), while the corrected divisor stays pinned at $2/3$ for every rung.

n	samples	E[plug-in, divisor n]	E[divisor n - 1]
2	9	$1/3 = \sigma^2/2$	$2/3 = \sigma^2$
3	27	$4/9$	$2/3$
4	81	$1/2$	$2/3$

```c
// enumerate all 3^n ordered samples of size n from {0,1,2} in lexicographic
// order and accumulate the estimators; every intermediate is a small integer
// or a correctly rounded double, so the expectations are machine-pinned
[[nodiscard]] static est_stats enumerate_estimators(int n) {
  int idx[8] = {0};
  long total = 1;
  for (int i = 0; i < n; i++)
    total *= 3;
  double acc_vn = 0, acc_vu = 0, acc_m = 0, acc_m2 = 0, acc_mse = 0;
  for (long t = 0; t < total; t++) {
    long s = 0, sq = 0;
    for (int i = 0; i < n; i++) {
      s += idx[i];
      sq += (long)idx[i] * idx[i];
    }
    double m = (double)s / (double)n;
    double num = (double)sq - (double)(s * s) / (double)n;
    acc_vn += num / (double)n;
    acc_vu += num / (double)(n - 1);
    acc_m += m;
    acc_m2 += m * m;
    acc_mse += (m - 1.0) * (m - 1.0);
    for (int i = 0; i < n; i++) { // odometer increment
      idx[i]++;
      if (idx[i] < 3)
        break;
      idx[i] = 0;
    }
  }
  return (est_stats){.e_vn = acc_vn / (double)total,
                     .e_vu = acc_vu / (double)total,
                     .e_m = acc_m / (double)total,
                     .e_m2 = acc_m2 / (double)total,
                     .e_mse = acc_mse / (double)total};
}
```

listing 206: the odometer that enumerates every ordered sample and accumulates the estimator expectations, estimators.c (source: math/samples/src/Ch19/estimators.c, lines 32-66)

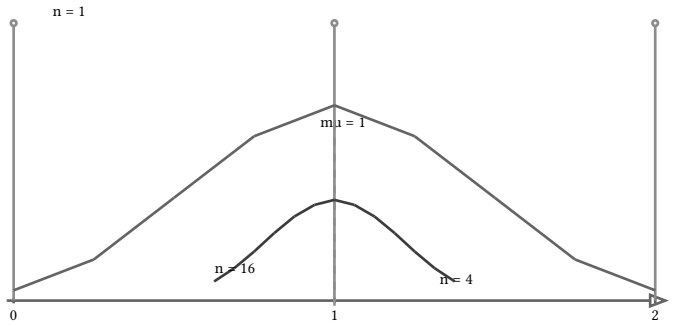


figure 116: the exact distribution of the sample mean sharpening with n, uniform {0,1,2} population

Maximum likelihood turns the model around: mml section 8.3.1 defines the negative log-likelihood for data x and parameter family $p(x \mid \theta)$ [printed p 265 / pdf p 271] and minimizes it over θ. For the Bernoulli count the derivative solve is one line, $k/p - (n-k)/(1-p) = 0$, so $\hat{p} = k/n$. For a Gaussian sample both stationary points close at once, $\hat{\mu} = |x|$ and $\hat{\sigma}^2 = 1/n \sum (x_i - |x|)^2$, the pair mml derives for linear regression noise from the Gaussian log-likelihood of eq 9.9, which drops every term that does not involve the parameters [printed p 293 / pdf p 299]:

the mean half at eqs 9.10-9.12 [printed p 294 / pdf p 300], the noise variance at eq 9.22, “the empirical mean of the squared distances” [printed p 298 / pdf p 304].

The dry run: 7 successes in 10 draws, and the dataset {2, 5, 5, 8, 10}.

1. Bernoulli: $\hat{p} = 0.7$ with log-likelihood −6.108643, and the neighbors 0.69 and 0.71 sit 2.4e-3 lower, the solve is a real maximum.
2. Gaussian: $\hat{\mu} = 30/5 = 6$, $\hat{\sigma}^2 = 38/5 = 7.6$, profile negative log-likelihood 12.165063, minimal against 5.999, 6.001 and against 7.5, 7.7.
3. MAP with the conjugate prior: a Beta(3, 2) prior on the same 7/10 data gives posterior Beta(10, 5), mode $(10-1)/(10+5-2) = 9/13 = 0.6923$, mean $10/15 = 2/3$, and normalizer $B(10,5) = 9!4!/14! = 1/10010$ as an exact integer ratio. The flat Beta(1, 1) prior returns mode 0.7, the MLE: eq 8.20 makes the posterior proportional to likelihood times prior [printed p 269 / pdf p 275], so a flat prior leaves the posterior proportional to the likelihood and the two maxima coincide.

```
// n = 2 (9 ordered samples): the plug-in variance averages 1/3 = sigma^2/2,
// the n - 1 divisor averages 2/3 = sigma^2 exactly, and the sample mean is
// unbiased with E[m^2] = 4/3 and mse = bias^2 + variance = 0 + 1/3
auto e2 = enumerate_estimators(2);
CHECK(fabs(e2.e_m - 1.0) < 1e-12, "n=2 sample mean unbiased");
CHECK(fabs(e2.e_vn - 1.0 / 3.0) < 1e-12, "n=2 plug-in bias 1/3");
CHECK(fabs(e2.e_vu - 2.0 / 3.0) < 1e-12, "n=2 n-1 divisor unbiased 2/3");
CHECK(fabs(e2.e_m2 - 4.0 / 3.0) < 1e-12, "n=2 E[m^2] = 4/3");
double bias = e2.e_m - 1.0;
double var = e2.e_m2 - e2.e_m * e2.e_m;
CHECK(fabs(e2.e_mse - (bias * bias + var)) < 1e-12 &&
          fabs(e2.e_mse - 1.0 / 3.0) < 1e-12,
      "n=2 mse = bias^2 + variance = 1/3");

// the divisor ladder: E[plug-in] = (n-1)/n sigma^2 at 4/9 and 1/2 while the
// n - 1 divisor stays at 2/3 for every n
auto e3 = enumerate_estimators(3);
CHECK(fabs(e3.e_vn - 4.0 / 9.0) < 1e-12 && fabs(e3.e_vu - 2.0 / 3.0) < 1e-12,
      "n=3 plug-in 4/9, n-1 divisor 2/3");
auto e4 = enumerate_estimators(4);
CHECK(fabs(e4.e_vn - 0.5) < 1e-12 && fabs(e4.e_vu - 2.0 / 3.0) < 1e-12,
      "n=4 plug-in 1/2, n-1 divisor 2/3");

// mle for bernoulli p: 7 successes in 10 draws give p_hat = 0.7 by the
// derivative solve k/p - (n-k)/(1-p) = 0, and the log-likelihood really is
// maximal there (margins 2.4e-3 against 0.69 and 0.71)
double phat = 7.0 / 10.0;
CHECK(fabs(phat - 0.7) < 5e-16, "bernoulli mle p_hat = 7/10");
CHECK(fabs(loglik(7, 10, 0.7) - (-6.108643020548935)) < 1e-9,
      "log-likelihood at p_hat -6.10864");
CHECK(loglik(7, 10, 0.7) > loglik(7, 10, 0.69) &&
          loglik(7, 10, 0.7) > loglik(7, 10, 0.71),
      "log-likelihood maximal at p_hat");
```

listing 207: the enumeration pins and the bernoulli mle solve, estimators.c (source: math/samples/src/Ch19/estimators.c, lines 82-114)

```
// mle for gaussian (mu, sigma2) on {2,5,5,8,10}: mu_hat = sum/n = 6,
// sigma2_hat = 38/5 = 7.6, both from setting the gradient to zero
const int data[] = {2, 5, 5, 8, 10};
long sum = 0, sumsq = 0;
for (int i = 0; i < 5; i++) {
  sum += data[i];
  sumsq += (long)data[i] * data[i];
}
double mu = (double)sum / 5.0;
double s2 = ((double)sumsq - (double)(sum * sum) / 5.0) / 5.0;
CHECK(mu == 6.0 && fabs(s2 - 7.6) < 5e-16, "gaussian mle mu 6, sigma2 7.6");
double nllmin = nll_gauss(sum, sumsq, 5, 6.0, 7.6);
CHECK(fabs(nllmin - 12.165063284254076) < 1e-9, "profile nll minimum");
CHECK(nll_gauss(sum, sumsq, 5, 6.0, 7.6) <
              nll_gauss(sum, sumsq, 5, 5.999, 7.6) &&
          nll_gauss(sum, sumsq, 5, 6.0, 7.6) <
              nll_gauss(sum, sumsq, 5, 6.001, 7.6),
      "nll minimal at mu_hat");
CHECK(nll_gauss(sum, sumsq, 5, 6.0, 7.6) <
              nll_gauss(sum, sumsq, 5, 6.0, 7.5) &&
          nll_gauss(sum, sumsq, 5, 6.0, 7.6) <
              nll_gauss(sum, sumsq, 5, 6.0, 7.7),
      "nll minimal at sigma2_hat");

// map for the same 7/10 data with a beta(3,2) prior: posterior beta(10,5),
// mode 9/13 = 0.6923, mean 10/15 = 2/3; the flat beta(1,1) prior reproduces
// the mle exactly. normalizer B(10,5) = 9!4!/14! = 1/10010 as an integer
// ratio, so the posterior is exact rational arithmetic
double mode = 9.0 / 13.0, pmean = 10.0 / 15.0;
CHECK(fabs(mode - 0.6923076923076923) < 5e-16 &&
          fabs(pmean - 2.0 / 3.0) < 5e-16,
      "beta(10,5) mode 9/13, mean 10/15");
double b_num = 362880.0 * 24.0, b_den = 87178291200.0;
CHECK(fabs(b_num / b_den - 9.99000999000999e-05) < 1e-17,
      "B(10,5) = 1/10010 exact ratio");
double mode_flat = 7.0 / 10.0;
CHECK(fabs(mode_flat - phat) < 5e-16, "flat prior map equals mle");
```

listing 208: the gaussian mle solve and the beta posterior mode with its exact normalizer, estimators.c (source: math/samples/src/Ch19/estimators.c, lines 116-152)

The two-number play: same data, two priors, 0.6923 against 0.7. The Beta(3, 2) prior contributes 2 successes and 1 failure worth of belief, and the posterior mode lands between the prior's pull and the data.

PITFALL: THE BETA MODE NEEDS A, B > 1

The closed form $(a-1)/(a+b-2)$ for the Beta mode is only the interior maximum. With $a \le 1$ or $b \le 1$ the density is unbounded at an endpoint and the mode sits on the boundary of $[0, 1]$. The conjugate update keeps this safe once the data arrives with both outcomes, since $a + k \ge 2$ and $b + n - k \ge 2$ whenever the prior had $a, b \ge 1$ and the sample shows at least one of each outcome.

19.4. hypothesis tests

A test is a partition of the sample space. The null hypothesis H_0 fixes a distribution, the rejection region fixes which outcomes count against it, the size is $\alpha = P(\text{reject} \mid H_0)$, the type II error at an alternative is $\beta = P(\text{accept} \mid H_1)$,

and the power is $1-\beta$. The p-value is the probability under H_0 of an outcome at least as extreme as the one observed, a number about the test, not a probability that H_0 is true.

The dry run: 9 successes in 10 fair-coin flips, then the region $x \geq 15$ for 20 flips.

1. One-sided exact test: $P(X \geq 9) = (C(10,9) + C(10,10))/2^{10} = 11/1024 = 0.0107421875$, binary-exact. The two-sided p-value doubles the small tail by symmetry, $11/512 = 0.021484375$, and the doubling itself is checked as $p_2 = 2p_1$ to the bit.
2. The region $x \geq 15$ at $n = 20$ has numerator $15504 + 4845 + 1140 + 190 + 20 + 1 = 21700$, so the size is $21700/2^{20} = 0.0207$. Under the fixed alternative $p = 0.7$ the same six terms weight $7^i 3^{20-i}$ and sum to $41637082944748138372/10^{20}$, power 0.4164, type II 0.5836.

quantity	value	where
size α	$21700/2^{20} = 0.0207$	under $H_0 : p = 0.5$
power $1-\beta$	0.4164	at $H_1 : p = 0.7$
type II β	0.5836	same fixed alternative

```
// exact binomial test, 9 successes in 10 flips of a fair coin: the one-sided
// tail is 11/1024 = 0.0107421875, the two-sided p-value doubles it to
// 11/512 = 0.021484375 (both binary-exact)
double p1 = 11.0 / 1024.0, p2 = 11.0 / 512.0;
CHECK(fabs(p1 - 0.0107421875) < 5e-16 && fabs(p2 - 0.021484375) < 5e-16 &&
          p2 == 2.0 * p1,
      "exact two-sided p-value 11/512");

// rejection region x ≥ 15 for n = 20 flips: size alpha = 21700/2^20 =
// 0.020694732666015625 exactly (2^20 denominator is a power of two)
long row[21] = {1};
for (int i = 1; i ≤ 20; i++)
  row[i] = row[i - 1] * (20 - i + 1) / i;
long crit = 0;
for (int i = 15; i ≤ 20; i++)
  crit += row[i];
double alpha = (double)crit / 1048576.0;
CHECK(crit == 21700 && fabs(alpha - 0.020694732666015625) < 5e-16,
      "size 21700/2^20 = 0.0207");

// power at the fixed alternative p = 0.7: sum_{i≥15} C(20,i) 7^i 3^(20-i)
// over 10^20 = 41637082944748138372/10^20, so type II = 1 - power
u128 pow7[21] = {1}, pow3[21] = {1};
for (int i = 1; i ≤ 20; i++) {
  pow7[i] = pow7[i - 1] * 7;
  pow3[i] = pow3[i - 1] * 3;
}
u128 num = 0;
for (int i = 15; i ≤ 20; i++)
  num += (u128)row[i] * pow7[i] * pow3[20 - i];
u128 den = 1;
for (int i = 0; i < 20; i++)
  den *= 10;
double power = u128_to_double(num) / u128_to_double(den);
double beta_err = 1.0 - power;
CHECK(fabs(power - 0.41637082944748144) < 5e-16, "power at 0.7 = 0.4164");
CHECK(fabs(beta_err - 0.5836291705525185) < 5e-16, "type II = 0.5836");
CHECK(power > alpha, "one-sided test unbiased: power > size");
```

listing 209: the exact binomial test, the rejection region size, and the 128-bit power sum, inference.c (source: math/samples/src/Ch19/inference.c, lines 80-117)

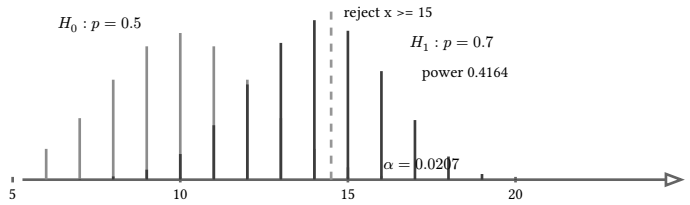


figure 117: the two binomials behind the test, rejection region x >= 15 shaded darker

The two-number play: the same test pays 0.0207 for a false alarm and 0.5836 for a missed detection at $p = 0.7$. Twenty flips simply cannot buy more, and the power calculation prices that honestly before any data is spent.

NOTE: EXACT BEATS ASYMPTOTIC AT SMALL N

The normal approximation to this very test would compare $(15 - 10)/\sqrt{5} = 2.236$ against 1.96 and declare significance at the 5% level, but the exact size is 0.0207, not 0.05, because no integer rejection region hits 0.05

> exactly. With counts in the dozens the exact binomial sum is a few 128-bit additions, so there is no reason to borrow the normal's error.

19.5. confidence intervals

A 95% confidence interval for a mean with known variance σ^2 is $|x| \pm z_{0.975}\sigma/\sqrt{n}$ with $z_{0.975} = 1.9599639845400545$ from the last section. The reading is the coverage statement: across repeated streams, 95% of the intervals contain the true mean, any single interval does or does not, it carries no probability of its own. When σ must itself be estimated the exchange $z \to t$ widens every interval (Student's t, Wasserman 2004 by name). The t density is the standard normal with the estimated variance folded in, one shape per degrees of freedom $n - 1$, wider than the normal and converging to it as n grows: $t_{0.975}(24) = 2.0638985$ against $z_{0.975} = 1.9599640$, a widening of 5.3% at $n = 25$, and the samples verify the table constant by integrating the t density.

The dry run: 40 streams of $n = 25$ digits against the known population values, mean 4.5, variance 8.25.

1. The half width is $1.9599639845400545 \cdot \sqrt{8.25/25} = 1.125914$, the same arithmetic in every stream since n and σ are fixed.
2. Stream 0 sums to 119, mean 4.76, interval [3.6341, 5.8859], containing 4.5.
3. Of the 40 intervals, 38 contain 4.5. The misses are stream 5, mean 3.28, and stream 35, mean 3.16, both low. Empirical coverage $38/40 = 0.95$.

```
// coverage: 40 streams of n = 25 digits, half width z * sqrt(8.25/25) =
// 1.1259135894148633. the first stream pins its endpoints, and 38 of 40
// intervals contain 4.5, an empirical 95% hit rate
ci first = z_interval(25);
CHECK(fabs((first.hi - first.lo) / 2.0 - 1.1259135894148633) < 5e-15,
      "half width z sqrt(8.25/25)");
CHECK(fabs(first.lo - 3.6340864105851365) < 5e-15 &&
          fabs(first.hi - 5.885913589414863) < 5e-15 && first.lo < 4.5 &&
          4.5 < first.hi,
      "first interval 3.634..5.886 covers 4.5");
int covered = (first.lo < 4.5 && 4.5 < first.hi) ? 1 : 0;
for (int t = 1; t < 40; t++) {
  ci c = z_interval(25);
  covered += c.lo < 4.5 && 4.5 < c.hi;
}
CHECK(covered == 38, "38 of 40 intervals cover the mean");
CHECK(fabs((double)covered / 40.0 - 0.95) < 5e-16, "empirical coverage 0.95");

// the t exchange: t_{0.975}(24) = 2.063898530351509 from the tables,
// verified here by simpson integration of the t24 density over [0, t]
// landing on 0.475 (the exact integral is 0.475 to 1.6e-9 at this
// resolution), and the interval widening over z is 5.3%
double t24 = 2.063898530351509;
enum { M = 8192 };
double h = t24 / (double)M;
double s = t24_dens(0.0) + t24_dens(t24);
for (int i = 1; i < M; i++)
  s += t24_dens((double)i * h) * ((i & 1) ? 4.0 : 2.0);
CHECK(fabs(s * h / 3.0 - 0.475) < 1e-8, "t24 density integrates to 0.475");
CHECK(fabs(t24 / Z975 - 1.0 - 0.05302880391235587) < 1e-15,
      "t widens the interval by 5.3%");
```

listing 210: the coverage count over 40 fixed streams, then the t quantile verified by integration, inference.c (source: math/samples/src/Ch19/inference.c, lines 119-149)

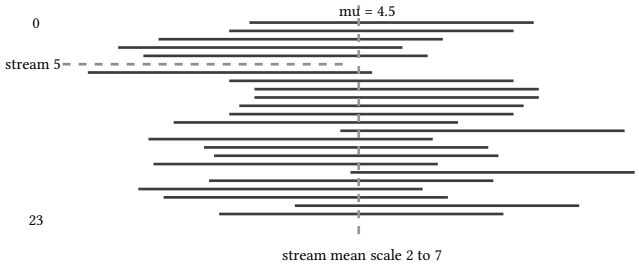


figure 118: the first 24 intervals against the true mean 4.5, stream 5 misses

The two-number play: nominal coverage 0.95, counted coverage 38 of 40. The count is an integer, so the empirical rate snaps to 0.95 exactly here, one quiet accident of the fixed streams worth reporting as what it is.

19.6. least squares statistics

Regression closes the chapter where the linear algebra lives, chapter book 2, ch 7 (mathematics for programmers, inner products and projections). Least squares fits $y = a + bx$ by minimizing the residual sum of squares, and setting the two partial derivatives to zero gives the normal equations $b = S_{xy}/S_{xx}$ and $a = |y| - b|x|$ in terms of the centered sums $S_{xy} = \sum (x_i - |x|)(y_i - |y|)$ and $S_{xx} = \sum (x_i - |x|)^2$. The same pair falls out of maximum likelihood: mml section 9.2.1 shows the Gaussian-noise likelihood of the linear model reduces to the squared residual loss, eq 9.9 [printed p 293 / pdf p 299], so the least-squares line is the MLE under Gaussian noise, and the MAP version of that fit is ridge regularization in disguise (mml sections 9.2.3-9.2.4 [printed pp 300-302 / pdf pp 306-308]).

The dry run: $x = 0..6$ against $y = 3, 4, 8, 13, 12, 18, 19$, integers throughout.

1. Centered sums: $S_{xx} = 28$, $S_{xy} = 80$, $S_{yy} = 240$, so $b = 80/28 = 20/7$ and $a = 11 - 3 \cdot 20/7 = 17/7$, both exact rationals.
2. $R^2 = S_{xy}^2 / (S_{xx} S_{yy}) = 6400/6720 = 20/21 = 0.9524$, computed from the same three integers.
3. The residuals sum to exactly 0.0 in the doubles, the x-weighted residual sum is 5.3e-15, residual noise not an algebra failure, the residual sum of squares is $240/21 = 11.4286$, and $1 - \mathrm{SSR}/\mathrm{SST}$ returns 20/21 to 1e-12, the ANOVA identity closing on the numbers.

```
// least squares on x = 0..6, y = 3 4 8 13 12 18 19: integer normal
// equations Sxx = 28, Sxy = 80, Syy = 240 give slope 20/7, intercept 17/7,
// R^2 = 80^2/(28*240) = 20/21, residuals orthogonal to 1 and x, and
// SST = SSR + SSE closed on the numbers
const int yv[] = {3, 4, 8, 13, 12, 18, 19};
long sxx = 0, sxy = 0, syy = 0;
for (int i = 0; i < 7; i++) {
  long dx = i - 3, dy = yv[i] - 11;
  sxx += dx * dx;
  sxy += dx * dy;
  syy += dy * dy;
}
double slope = (double)sxy / (double)sxx;
double intercept = 11.0 - slope * 3.0;
double r2 = (double)(sxy * sxy) / (double)(sxx * syy);
CHECK(sxx == 28 && sxy == 80 && syy == 240 &&
          fabs(slope - 2.857142857142857) < 5e-16,
      "normal equations slope 20/7");
CHECK(fabs(intercept - 2.428571428571429) < 5e-16, "intercept 17/7");
CHECK(fabs(r2 - 0.9523809523809523) < 5e-16, "R^2 = 20/21 = 0.9524");
double sr = 0, sxr = 0, ssr = 0;
for (int i = 0; i < 7; i++) {
  typeof(sr) r = (double)yv[i] - (intercept + slope * (double)i);
  sr += r;
  sxr += (double)i * r;
  ssr += r * r;
}
CHECK(sr == 0.0 && fabs(sxr) < 1e-12, "residuals orthogonal to 1 and x");
CHECK(fabs(ssr - 11.428571428571427) < 1e-12, "SSR = 240/21 = 11.4286");
CHECK(fabs((1.0 - ssr / 240.0) - r2) < 1e-12, "SST = SSR + SSE closes");
```

listing 211: integer normal equations, R^2 as an integer ratio, and the orthogonality pins, inference.c (source: math/samples/src/Ch19/inference.c, lines 151-180)

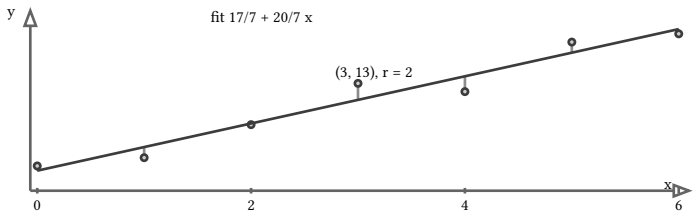


figure 119: the fit y = 17/7 + 20/7 x through seven integer points with residuals

The two-number play: the least-squares slope is $20/7 = 2.857$, the endpoint slope $(19 - 3)/6$ is 2.667. Orthogonality, not compromise, is what the normal equations buy, the fit line honors the center of the data more than its ends.

Chapter book 2, ch 20 (mathematics for programmers, root finding) turns from inference to solvers, and the transition is natural: every derivative solve in this chapter became a number because the model was small enough to close by hand, and root finding is what carries that step when it does not.

sources: mml-book draft 2024-01-15, ch 6 sec 6.4 pp 186-197 (statistics definitions, eqs 6.41-6.45, 6.58, the N versus N-1 margin note), ch 8 sec 8.3 pp 264-269 (negative log-likelihood, MAP estimation), ch 9 sec 9.2 pp 292-302 (gaussian likelihood eq 9.9, MAP as regularization), verified against ref/mml-book.pdf pp 192-203, 270-275, 298-308; NIST DLMF section 7.2 eq 7.2.E2, https://dlmf.nist.gov/7.2, and cppreference “erfc, erfcf, erfcl”, https://en.cppreference.com/c/numeric/math/erfc, both fetched 2026-09-21; Wasserman, All of Statistics, Springer 2004, and Casella and Berger, Statistical Inference, 2nd ed, Duxbury 2002, cited by name for the LLN, CLT, Chebyshev, Berry-Esseen, t intervals, and testing chapters. Probed on this machine the same day. Sample behavior verified

by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch19`, 63 checks in chapter 19 of the math suite.

20. root finding

Root finding asks for the x where a known function crosses zero, and this chapter builds the four iteration machines that answer it plus the weld that makes them safe: bisection trades speed for a proof, fixed-point iteration makes the rearrangement the whole story, newton's method squares the error once it gets close, the secant method buys nearly the same speed for one function value per step, and a guard turns the fast local methods into globally convergent hybrids. Six moves: the halving guarantee, contraction and its 0.381966 constant, tangent steps and error squaring, the golden order 1.618, one error model $e_{n+1} = Ce_n^p$ that classifies all four, and the bisection-newton hybrid on a function where pure newton explodes. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 20 or a fact quoted from cppreference fetched 2026-09-22. Where the numerical chapter of the dsa book (book 9, ch 36 (practical data structures and algorithms, numerical methods)) tunes these drivers as contest templates with pinned stopping rules, this chapter asks why each one converges, how fast, and what breaks it. The derivatives come from book 2, ch 9 (mathematics for programmers, single-variable calculus), the conditioning lens from book 2, ch 2 (mathematics for programmers, error, conditioning, and stability).

20.1. bisection and the halving guarantee

If f is continuous on $[a, b]$ and $f(a)f(b) < 0$, the intermediate value theorem puts a root inside, and the midpoint test keeps that certificate alive forever: evaluate at $m = (a + b)/2$, keep the half where the signs still disagree. Each step halves the bracket, so after n evaluations the surviving interval has width $(b - a)/2^n$ and every point of it, including the midpoint just tested, sits within $(b - a)/2^n$ of some root. The rate is fixed, one bit per evaluation, and nothing about f beyond continuity and the sign change is used.

The fixture is $f(x) = x^3 - 2x - 5$ on $[2, 3]$, with $f(2) = -1$ and $f(3) = 16$. Every midpoint on this bracket is an exact dyadic double, so the whole ladder pins with bit equality in bisection.c, and the root alpha = 2.0945514815423265 is the nearest double, itself the fixed point of an 80-step newton ladder in the playground.

The dry run: eight rungs of halving, all exact.

1. Rung 1: $m = 2.5$, $f = 5.625 > 0$, root is left, bracket $[2, 2.5]$.
2. Rung 4: $m = 2.0625$, $f = -0.351318359375 < 0$, first sign flip, bracket $[2.0625, 2.125]$: alpha was in the smaller half.
3. Rung 8: $m = 2.09765625$, error 3.105×10^-3 against the bound $2^-8 = 3.906 \times 10^-3$, tight to a factor of 1.26.

rung	midpoint	f(mid)	abs(mid - alpha)	bound 2^-n
1	2.5	+5.625	4.054e-1	5.000e-1
2	2.25	+1.891	1.554e-1	2.500e-1
3	2.125	+0.346	3.045e-2	1.250e-1
4	2.0625	−0.351	3.205e-2	6.250e-2
5	2.09375	−0.009	8.015e-4	3.125e-2
6	2.109375	+0.167	1.482e-2	1.563e-2
7	2.1015625	+0.079	7.011e-3	7.813e-3
8	2.09765625	+0.035	3.105e-3	3.906e-3

The two-number play on this root and this tolerance: bisection needs 50 halvings to push the bracket width under 10^{-15}, newton from the same start $x_0 = 2$ needs 4 updates to land on the exact double.

```
  // rung ladder: midpoint and sign of f at it, pinned per rung
  static const double mid_pin[8] = {2.5,     2.25,     2.125,     2.0625,
                                    2.09375, 2.109375, 2.1015625, 2.09765625};
  static const int sign_pin[8] = {+1, +1, +1, -1, -1, +1, +1, +1};
  double lo = lo0, hi = hi0;
  int bound_ok = 1;
  for (auto n = 0; n < 8; n++) {
    double mid = (lo + hi) / 2;
    // rung 8 example: mid = 2.09765625, f = +0.034714281558990479, sign +1
    int s = f(mid) > 0 ? +1 : -1;
    char name[48];
    snprintf(name, sizeof name, "rung %d pins midpoint and sign", n + 1);
    CHECK(mid == mid_pin[n] && s == sign_pin[n], name);
    // error bound: |mid_n - alpha| ≤ (b - a)/2^n = 2^-n, every rung
    // rung 8: |2.09765625 - alpha| = 3.104768e-03 ≤ 2^-8 = 3.906250e-03
    if (!(fabs(mid - alpha) ≤ ldexp(1.0, -(n + 1))))
      bound_ok = 0;
    if (f(mid) > 0)
      hi = mid;
    else
      lo = mid;
  }
  CHECK(bound_ok, "bound 2^-n holds on all 8 pinned rungs");
```

listing 212: bisection.c, the pinned rung ladder: dyadic midpoints, sign decisions, the 2^-n bound checked every rung (source: math/samples/src/Ch20/bisection.c, lines 63-85)

```
// halvings until the bracket width drops under tol, -1 if it never does
[[nodiscard]] static int halvings(double tol, double *width_out) {
  double lo = lo0, hi = hi0;
  order_bracket(&lo, &hi);
  for (auto n = 1; n ≤ 200; n++) {
    double mid = (lo + hi) / 2;
    if (f(mid) > 0)
      hi = mid;
    else
      lo = mid;
    if (hi - lo < tol) {
      *width_out = hi - lo;
      return n;
    }
  }
  return -1;
}
```

listing 213: bisection.c, the halvings driver that counts evaluations to a width tolerance (source: math/samples/src/Ch20/bisection.c, lines 41-57)

```
double width = 0.0;
// 2^-40 = 9.0949470177292824e-13 is the first dyadic width under 1e-12
CHECK(halvings(1e-12, &width) == 40, "40 halvings reach width under 1e-12");
CHECK(width == 9.0949470177292824e-13, "final width is exactly 2^-40");
CHECK(halvings(1e-15, &width) == 50, "50 halvings reach width under 1e-15");

// after those 50 halvings the bracket still straddles the root: the sign
// invariant survived every step, which is the whole guarantee
double mlo = lo0, mhi = hi0;
for (auto n = 0; n < 50; n++) {
  double mid = (mlo + mhi) / 2;
  if (f(mid) > 0)
    mhi = mid;
  else
    mlo = mid;
}
CHECK(f(mlo) < 0.0 && f(mhi) > 0.0, "sign invariant holds after 50 halvings");
```

listing 214: bisection.c, 40 halvings land on the exact width 2^-40, and the sign invariant survives 50 (source: math/samples/src/Ch20/bisection.c, lines 87-103)

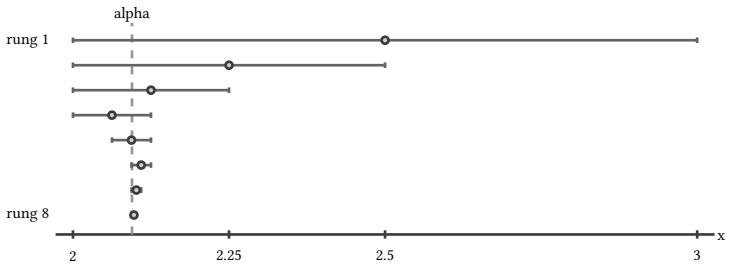


figure 120: nested brackets funnel onto alpha, each rung halves the interval that still certifies a root

NOTE: a bracket is a proof

The sign change is a certificate that survives every halving, which is why bisection is the only method here with a global guarantee: no derivative, no smoothness, no start close enough. The price is the fixed rate of one bit per evaluation, and the certificate is unavailable when a root has even multiplicity, because no interval then shows a sign change at all.

20.2. fixed points and contraction

Any equation $f(x) = 0$ can be rewritten as $x = g(x)$, and then iterated: $x_{n+1} = g(x_n)$. Whether that converges is decided by the rearrangement, not the equation. If g is differentiable near a fixed point α then $e_{n+1} = g(x_n) - \alpha \approx g'(\alpha)e_n$, so the iteration converges locally exactly when $|g'(\alpha)| < 1$, and the convergence is linear with one constant: each error is the previous error times $g'(\alpha)$, gaining $\log_{10} 1/|g'(\alpha)|$ digits per step. A derivative above 1 in magnitude does not merely slow the iteration, it repels the orbit.

The fixture is the golden ratio: $\varphi^2 = \varphi + 1$ rearranged as $x = 1 + 1/x$, whose fixed point has $g'(x) = -1/x^2$ and $|g'(\varphi)| = 1/\varphi^2 = 0.38196601125010515$. Starting from $x_0 = 1$ the iterates are the continued-fraction convergents of φ, the ratios of consecutive fibonacci numbers, and the double ladder rides them exactly: rungs 1, 2, 4, 5, 8, 15, and 20 land bit-for-bit on the nearest doubles to $2/1$, $3/2$, $8/5$, $13/8$, $55/34$, $1597/987$,$17711/10946$. Rung 3 is the visible exception, one ulp below the correctly rounded $5/3$, because the map computes $1 + 0.66666666666666663$ while direct rounding of the fraction lands one grid step up.

The dry run: the error walks down a geometric ladder with alternating sign.

1. $e_1 = \varphi - 2 = -0.382$, then $e_2 = +0.118$, $e_3 = -0.0486$, $e_5 = -0.00697$: the sign flips every step because g' is negative.
2. Ratios close on the constant from both sides, -0.381979 at rung 11 and -0.381966004 at rung 21 against $-1/\varphi^2 = -0.38196601125010515$.
3. The derivative read along the ladder, $-1/x_n^2$, is already -0.38196601301237004 at rung 20: the contraction constant is not magic, it is the observed slope.

rung	x	e = phi - x	e_{n+1}/e_n
0	1	+6.180e-1	−0.618034
1	2	−3.820e-1	−0.309017
2	1.5	+1.180e-1	−0.412023
3	1.6666666666666665	−4.863e-2	−0.370820
5	1.625	−6.966e-3	−0.380329
10	1.6179775280898876	+5.646e-5	−0.381979
20	1.618033985017358	+3.733e-9	−0.3819660

The same equation rearranged as $x = 1/(x-1)$ has the same two fixed points and the opposite fate at each: $g'(x) = -1/(x-1)^2$ gives $|g'(\varphi)| = \varphi^2 = 2.618$ and $|g'(\beta)| = 0.382$ where $\beta = \left(1 - \sqrt{5}\right)/2$. From $x_0 = 1.7$ the orbit leaves phi immediately, 1.4285714285714286, 2.333333333333333, 0.75000000000000022, −4.0000000000000036, five rungs that run from distance 0.082 to 5.618, and then settles onto beta with the same 0.381966 factor in reverse: rung 40 is −0.61803398874989601, one part in 10^{15} from the conjugate root, and its rung-30 error ratio reads −0.3819727622439244. The two-number play: one equation, two rearrangements, contraction constants 0.382 and 2.618 at the same root.

```
// a map plus a start plus a length: one iterate driver for both fixtures
typedef struct {
  double (*g)(double);
  double x0;
  int steps;
} orbit;

// run the map and store the whole ladder
static void iterate(orbit o, double *out) {
  double x = o.x0;
  for (auto i = 0; i ≤ o.steps; i++) {
    out[i] = x;
    x = o.g(x);
  }
}

// the convergent rearrangement: g(x) = 1 + 1/x, fixed point phi
static double plus_recip(double x) { return 1.0 + 1.0 / x; }

// the divergent rearrangement: g(x) = 1/(x - 1), fixed points phi and beta
static double recip_minus(double x) { return 1.0 / (x - 1.0); }
```

listing 215: fixedpoint.c, one orbit driver: a map, a start, and a length, run through a compound literal (source: math/samples/src/Ch20/fixedpoint.c, lines 26-46)

```
// the rungs ride the fibonacci convergents F(n+2)/F(n+1): exact on the
// checked rungs 1, 2, 4, 5, 8, 15, 20
int conv_ok = 1;
static const int conv_rungs[7] = {1, 2, 4, 5, 8, 15, 20};
for (auto k = 0; k < 7; k++) {
  int n = conv_rungs[k];
  if (x[n] != (double)fib[n + 1] / (double)fib[n])
    conv_ok = 0;
}
CHECK(conv_ok, "checked rungs equal the fibonacci convergents");
// rung 3 misses 5/3 by exactly one ulp downward: the double map computes
// 1 + 0.6666666666666663 while direct rounding of 5/3 lands one up
CHECK(x[3] == nextafter(5.0 / 3.0, 0.0), "rung 3 sits one ulp under 5/3");
```

listing 216: fixedpoint.c, the fibonacci convergents pinned bit for bit, and the one-ulp miss at rung 3 (source: math/samples/src/Ch20/fixedpoint.c, lines 83-95)

```
static double d[41];
iterate((orbit){.g = recip_minus, .x0 = 1.7, .steps = 40}, d);
// rungs 1..5 = 1.4285714285714286, 2.333333333333333, 0.75000000000000022,
// -4.0000000000000036, -0.19999999999999984: first away from phi, then
// across it and out the other side
static const double away_pin[5] = {1.4285714285714286, 2.333333333333333,
                                   0.75000000000000022, -4.0000000000000036,
                                   -0.19999999999999984};
int away_ok = 1;
for (auto i = 0; i < 5; i++)
  if (d[i + 1] != away_pin[i])
    away_ok = 0;
CHECK(away_ok, "divergent rungs pin exactly");
// |d4 - phi| = 5.618..., the orbit has left phi's neighborhood entirely
CHECK(fabs(d[4] - phi) > 5.0, "rung 4 runs more than 5 away from phi");
// rung 40 = -0.61803398874989601, one part in 1e-15 from the conjugate
// root beta = -0.6180339887498949, |d40 - beta| = 1.1102230246251565e-15
CHECK(d[40] == -0.61803398874989601, "rung 40 lands on beta's doorstep");
CHECK(fabs(d[40] - (1.0 - sqrt(5.0)) / 2.0) == 1.1102230246251565e-15,
      "distance to beta pins");
// and the beta errors shrink with the same constant: ratio at rung 30 is
// -0.3819727622439244, the same 0.381966 in motion
double beta = (1.0 - sqrt(5.0)) / 2.0;
CHECK(fabs((d[31] - beta) / (d[30] - beta) + 0.3819727622439244) < 1e-12,
      "beta ladder contracts at the same constant");
```

listing 217: fixedpoint.c, the divergent twin: away from phi in five rungs, onto beta in forty, same 0.381966 constant (source: math/samples/src/Ch20/fixedpoint.c, lines 111-135)

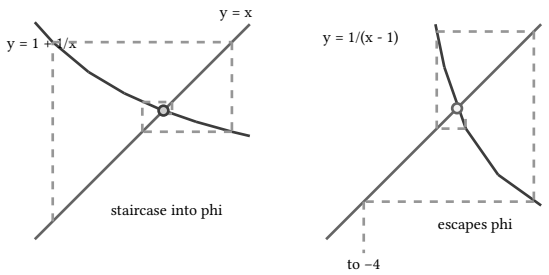


figure 121: two cobwebs for one equation: the convergent staircase into phi on the left, the repelling orbit that escapes on the right

PITFALL: the rearrangement chooses the fate

> Two algebraically equivalent forms of $x^2 = x + 1$ contract at 0.382 and 2.618 at the same root. Before iterating any fixed-point form, evaluate $|g'|$ at the answer you want: the equation cannot tell you which rewrite to use, the derivative can. This is the theory behind every "does my iteration converge" question, and it is why newton's method, next, derives its rearrangement from the function instead of trusting one.

20.3. newton's method and quadratic convergence

Replace the wholesale rewrite with a local linear model: at x_n, follow the tangent of f down to the axis, $x_{n+1} = x_n - f(x_n)/f'(x_n)$, the derivative machinery of book 2, ch 9 (mathematics for programmers, single-variable calculus). Near a simple root with $f'(\alpha) \neq 0$, a Taylor expansion of f at α turns the update into $e_{n+1} \approx (f''(\alpha)/(2f'(\alpha)))e_n^2$: the error is squared and rescaled, so correct digits double per step, which is the definition of quadratic convergence.

The fixture keeps $f(x) = x^3 - 2x - 5$, the example newton himself worked in 1669, starting from $x_0 = 2$ where $f = -1$ and $f' = 10$.

The dry run: four updates to the exact double.

1. Update 1 is one fraction: $2 - (-1)/10 = 2.1$, exactly the dyadic double.
2. Update 2: $f(2.1) = 0.061$, $f'(2.1) = 11.23$, giving 2.0945681211041851, error $1.66 \times 10^{-}5$.
3. Update 3: error $1.56 \times 10^{-}10$, the square of the previous error times 0.563.
4. Update 4: 2.0945514815423265, the nearest double to alpha, and the double map then never moves again, update 5 returns the same value.

update	x	e = x - alpha	e_{n+1}/e_n^2
0	2	−9.455148e-2	0.609455
1	2.1	+5.448518e-3	0.560512
2	2.0945681211041851	+1.663956e-5	0.562971
3	2.0945514816981992	+1.558726e-10	0.562979
4	2.0945514815423265	0, at the double	-

The two measured squaring ratios, 0.560512 and 0.562971, close on the theory constant $f''(\alpha)/(2f'(\alpha)) = 0.56297894577790075$, computed from alpha inside the same sample.

The same squaring law powers the division-free reciprocal. Applying newton to $f(y) = 1/y - a$ gives $y_{n+1} = y_n(2 - ay_n)$, an update that multiplies and subtracts only, with error law $e_{n+1} = -ae_n^2$. For $a = 7$ from $y_0 = 0.2$ the ladder runs 0.12, 0.1392, 0.14276352, 0.14285708150046722, 0.14285714285711651, and the measured ratios read -7 to within 10^{-8}, the constant being exactly $-a$. The contest template for this refinement lives in book 9, ch 36 (practical data structures and algorithms, numerical methods).

Quadratic convergence is a local promise. On $x^3 - 2x + 2$ from $x_0 = 0$: $f(0) = 2$ on slope $f'(0) = -2$ sends the update to 1, and $f(1) = 1$ on slope 1 sends it straight back to 0. The double map cycles between exact integers forever, checks 11 and 12 witness the cycle after ten and eleven steps, while the same polynomial has a real root in $(-2, -1)$ that this start never sees. Away from a root, the tangent line knows nothing.

```
// a function and its derivative travel together
typedef struct {
  double (*f)(double);
  double (*fp)(double);
} driver;

// one tangent step, the whole method
[[nodiscard]] static double newton_step(driver d, double x) {
  typeof(x) fx = d.f(x);
  return x - fx / d.fp(x);
}

static double wallis(double x) { return x * x * x - 2 * x - 5; }
static double wallis_d(double x) { return 3 * x * x - 2; }
static double cycle_poly(double x) { return x * x * x - 2 * x + 2; }
static double cycle_poly_d(double x) { return 3 * x * x - 2; }

// newton on f(y) = 1/y - a rearranged: y(2 - ay) multiplies and subtracts
// only, the trick behind hardware reciprocal and rsqrt refinement
static double recip_step(double y, double a) { return y * (2.0 - a * y); }
```

listing 218: newton.c, the driver struct carries a function and its derivative together, one tangent step uses both (source: math/samples/src/Ch20/newton.c, lines 26-45)

```
  // ladder from 2: 2.1, 2.0945681211041851, 2.0945514816981992, then the
  // fixed point 2.0945514815423265 where the double map stops moving
  double x = 2.0;
  x = newton_step(wallis_drv, x);
  CHECK(x == 2.1000000000000001, "rung 1 is 2.1");
  x = newton_step(wallis_drv, x);
  CHECK(x == 2.0945681211041851, "rung 2 pins");
  x = newton_step(wallis_drv, x);
  CHECK(x == 2.0945514816981992, "rung 3 pins");
  x = newton_step(wallis_drv, x);
  CHECK(x == alpha, "rung 4 lands on the nearest double");
  CHECK(newton_step(wallis_drv, x) == x, "rung 5 is a fixed point");

  // errors against the real root: e2 = 1.663956185860016e-05, e3 =
  // 1.5587264812211288e-10, four orders of magnitude per step
  double e2 = 2.0945681211041851 - alpha;
  double e3 = 2.0945514816981992 - alpha;
  CHECK(e2 == 1.663956185860016e-05, "error at rung 2 pins");
  CHECK(e3 == 1.5587264812211288e-10, "error at rung 3 pins");
  // squaring ratios: e3/e2^2 = 0.56297115128580233, e2/e1^2 =
  // 0.56051215329758675, against theory f''/(2f') at alpha =
  // 0.56297894577790075
  double e1 = 2.1000000000000001 - alpha;
  CHECK(e2 / (e1 * e1) == 0.56051215329758675, "squaring ratio at rung 1");
  CHECK(e3 / (e2 * e2) == 0.56297115128580233, "squaring ratio at rung 2");
  double theory = (6.0 * alpha) / (2.0 * (3.0 * alpha * alpha - 2.0));
  CHECK(theory == 0.56297894577790075, "theory constant f''/2f' pins");
```

listing 219: newton.c, the pinned ladder and the squaring ratios against the theory constant computed from alpha (source: math/samples/src/Ch20/newton.c, lines 50-76)

```
  // the cycle fixture: x^3 - 2x + 2 from 0 oscillates 0 <-> 1 forever,
  // both exact integers, so the double map cycles without any rounding
  driver cycle_drv = (driver){.f = cycle_poly, .fp = cycle_poly_d};
  double c = 0.0;
  for (auto i = 0; i < 10; i++)
    c = newton_step(cycle_drv, c);
  CHECK(c == 0.0, "after ten steps the cycle is back at 0");
  c = newton_step(cycle_drv, c);
  CHECK(c == 1.0, "and step eleven returns to 1");
  // the polynomial has a real root in (-2, -1) that this start never sees
  CHECK(cycle_poly(-2.0) * cycle_poly(-1.0) < 0.0,
        "the cycled polynomial still has a bracketed root");
```

listing 220: newton.c, the exact 2-cycle on x^3 - 2x + 2 from 0, and the bracketed root it never finds (source: math/samples/src/Ch20/newton.c, lines 78-89)

```
  // division-free reciprocal for a = 7 from y0 = 0.2:
  // 0.11999999999999998, 0.13919999999999999, 0.14276352,
  // 0.14285708150046722, 0.14285714285711651
  double y = 0.2;
  static const double recip_pin[5] = {0.11999999999999998, 0.13919999999999999,
                                      0.14276352, 0.14285708150046722,
                                      0.14285714285711651};
  double prev = y;
  int recip_ok = 1;
  for (auto i = 0; i < 5; i++) {
    y = recip_step(prev, 7.0);
    if (y != recip_pin[i])
      recip_ok = 0;
    prev = y;
  }
  CHECK(recip_ok, "reciprocal ladder pins");
  // exact recurrence for the error: e' = -7 e^2, so e5 = -2.634e-14 after
  // five multiply-only updates from 5.7e-2
  CHECK(y - 1.0 / 7.0 == -2.6340041259231839e-14, "reciprocal error pins");
  int ratio_ok = 1;
  double ys[6] = {0.2};
  for (auto i = 0; i < 5; i++)
    ys[i + 1] = recip_step(ys[i], 7.0);
  for (auto i = 0; i < 4; i++) {
    double ei = ys[i] - 1.0 / 7.0;
    double en = ys[i + 1] - 1.0 / 7.0;
    // deviation from exact -7: 1.8e-15, 3.6e-15, 9.6e-13, then 3.2e-09 once
    // e4 = 6.1e-08 approaches the noise floor of the subtraction y - 1/7
    if (fabs(en / (ei * ei) + 7.0) > 1e-8)
      ratio_ok = 0;
  }
  CHECK(ratio_ok, "reciprocal errors square with constant -7");
```

listing 221: newton.c, the division-free reciprocal ladder for a = 7, errors squaring with constant −7 (source: math/samples/src/Ch20/newton.c, lines 91-122)

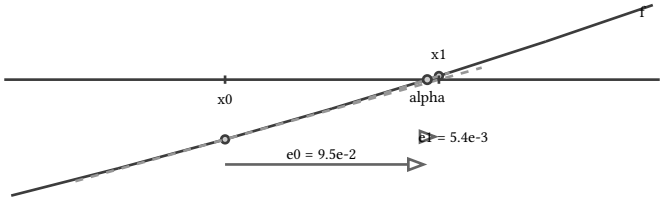


figure 122: tangent steps down the curve: each tangent crosses the axis at the next iterate, the error spans shrink quadratically

WARNING: quadratic convergence is a local promise

The squaring law assumes a simple root with f' bounded away from zero and a start close enough, and neither is free: the cycle fixture cycles from $x_0 = 0$ on a polynomial that does have a root, and a root of multiplicity m drops the rate to linear with constant $(m-1)/m$, the standard result in Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed. Speed near the root says nothing about getting there, which is the subject of the safeguards section.

20.4. the secant method and the golden order

The derivative in newton's update can be replaced by the slope through the last two iterates, $x_{n+1} = x_n - f(x_n)(x_n - x_{n-1})/(f(x_n) - f(x_{n-1}))$, one function evaluation per step instead of two. The cost is a slightly weaker law: $e_{n+1} \approx C e_n e_{n-1}$, and asking how the error must scale to satisfy it, if e_n behaves like e_{n-1}^p, forces $p^2 = p + 1$. That is the golden-ratio equation of the fixed-point section, so the secant order is $p = \left(1 + \sqrt{5}\right)/2 = 1.6180339887498949$, superlinear: better than any fixed number of digits per step, worse than doubling.

The dry run: the same root, the same tolerance, from the pair $(2, 3)$.

1. Update 1: the chord from $(2, -1)$ to $(3, 16)$ has slope 17 and crosses the axis at $35/17 = 2.0588235294117645$.
2. Updates 2 through 4: 2.0812636598450229, 2.0948241460940524, 2.0945494310352473, errors $1.3 \times 10^{-}2$, $2.7 \times 10^{-}4$, $2.1 \times 10^{-}6$.
3. Update 6: error $3.1 \times 10^{-}10$. Update 7 lands one ulp above the exact double, update 8 on it.

update	x	abs(e)	measured p
1	3	9.054e-1	-
2	2.0588235294117645	3.573e-2	0.306
3	2.0812636598450229	1.329e-2	3.929
4	2.0948241460940524	2.727e-4	1.258
5	2.0945494310352473	2.051e-6	1.796
6	2.0945514812275992	3.147e-10	1.534
7	2.094551481542327	4.441e-16	-
8	2.0945514815423265	0, at the double	-

The measured exponent $p = \log|e_{n+1}/e_n| / \log|e_n/e_{n-1}|$ swings early, 0.31, 3.93, then settles: 1.796 and 1.534 straddle 1.618. The two-number play is the accounting: to land on the exact double, newton spends 4 updates but 8 evaluations (each needs f and f'), secant spends 7 updates and 9 evaluations of f alone, bisection spends 50. When the derivative is unavailable or costs more than one extra

evaluation, secant wins the race: the classical efficiency index, order per evaluation, is $1.618^1 = 1.618$ against $2^{1/2} = 1.414$.

```
// one secant update: the line through the last two points
[[nodiscard]] static double secant_step(double x, double xm) {
  double fx = wallis(x);
  double fm = wallis(xm);
  return x - fx * (x - xm) / (fx - fm);
}
```

listing 222: safeguard.c, one secant update: the chord through the last two iterates (source: math/samples/src/Ch20/safeguard.c, lines 28-33)

```
  // secant ladder from (2, 3): 2.0588235294117645, 2.0812636598450229,
  // 2.0948241460940524, 2.0945494310352473, 2.0945514812275992,
  // 2.094551481542327, 2.0945514815423265
  static const double sec_pin[7] = {2.0588235294117645, 2.0812636598450229,
                                    2.0948241460940524, 2.0945494310352473,
                                    2.0945514812275992, 2.094551481542327,
                                    2.0945514815423265};
  double xm = 2.0, x = 3.0;
  int sec_ok = 1;
  double rungs[9] = {xm, x};
  for (auto i = 0; i < 7; i++) {
    double xn = secant_step(x, xm);
    xm = x;
    x = xn;
    rungs[i + 2] = xn;
    if (xn != sec_pin[i])
      sec_ok = 0;
  }
  CHECK(sec_ok, "secant ladder pins");
  // rung 7 sits one ulp above alpha, rung 8 lands on it
  CHECK(rungs[7] - alpha == 4.4408920985006262e-16,
        "rung 7 is one ulp above the root");
  CHECK(rungs[8] == alpha, "rung 8 lands on the nearest double");
  // measured order p = log|e+1/e| / log|e/e-1|: 1.7958307334064807 and
  // 1.5339739348927557 straddle the golden ratio 1.6180339887498949 that
  // theory gives for the secant order
  double e5 = rungs[5] - alpha, e6 = rungs[6] - alpha, e7 = rungs[7] - alpha,
         e4 = rungs[4] - alpha;
  double p5 = log(fabs(e6 / e5)) / log(fabs(e5 / e4));
  double p6 = log(fabs(e7 / e6)) / log(fabs(e6 / e5));
```

listing 223: safeguard.c, the pinned ladder, the ulp off the exact double, and the measured exponents straddling 1.618 (source: math/samples/src/Ch20/safeguard.c, lines 69-98)

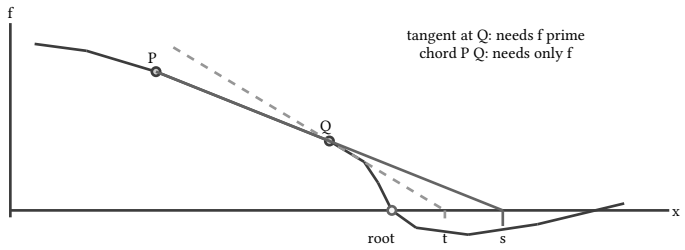


figure 123: the two local models on one curve: the tangent uses a point and its derivative, the chord uses two points, both aim at the next iterate

20.5. one error model, three speeds

Every ladder in this chapter fits one model:

$$e_{n+1} = Ce_n^p$$

with constant C and order p, and the order is the whole taxonomy: linear for $p = 1$, where each step multiplies the error by C and buys a fixed digit count $\log_{10} 1/|C|$, quadratic for $p = 2$, where digits double, superlinear in between, where the digits gained per step grow geometrically with ratio p. All four machines of this chapter fall in line, with the constants and orders measured, not asserted.

The dry run: bisection holds bracket width 2^{-50} at 50 halvings, the fixed-point ratios close on -0.381966004 at rung 21, secant reads measured orders 1.796 and 1.534, and newton reads squaring constants 0.560512 and 0.562971.

method	error model	measured	digits per step	sample
bisection	$e_{n+1} = e_n/2$	width 2^{-50} at 50	0.301 fixed	14 in bisection.c
fixed point	$e_{n+1} = 0.381966e_n$	ratio −0.381966004 at rung 21	0.418 fixed	15 in fixedpoint.c
secant	$e_{n+1} = Ce_n^{1.618}$	p 1.796, 1.534	0.43 to 5.85, growing	15 in safeguard.c
newton	$e_{n+1} = 0.562979e_n^2$	0.560512, 0.562971	digits double	16 in newton.c

Bisection is linear on the bracket width, a bound rather than an observed error: its constant is exact and global, every other constant in the table is asymptotic, visible only once the ladder is near its root. The digits column is where intuition should land: 0.301 and 0.418 are budgets, secant's grows by a factor near 1.6 each step, newton's doubles. A multiple root moves a method down this table, as the warning in the newton section recorded: order m at the root makes newton linear with constant $(m-1)/m$, so a double root halves the error per step, no better than the fixed-point ladder. The conditioning of the root itself, how error in f's coefficients or evaluations moves alpha, is the subject of book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) and decides whether extra digits are even meaningful.

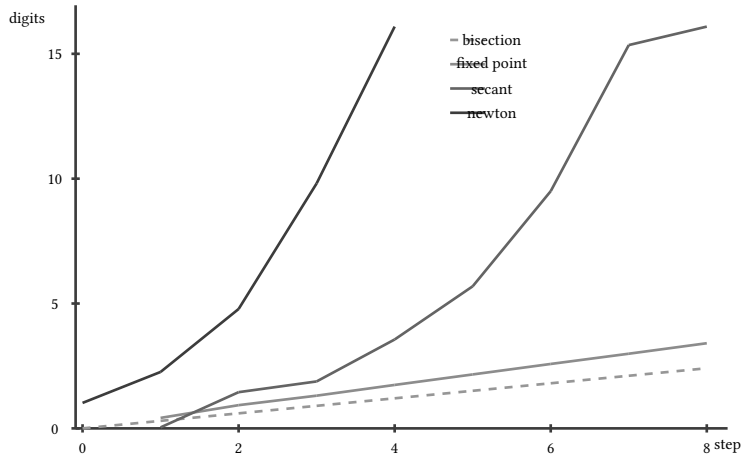


figure 124: digits of accuracy per iteration: bisection and fixed point crawl at fixed slopes, secant accelerates, newton doubles

20.6. safeguards: keep the bracket

Every fast method above is local, and the fix is old and standard: carry a bisection bracket as a certificate, propose each new iterate with the fast method, and if the proposal leaves the bracket, or the step does not shrink the bracket, throw it away and bisect instead. The bracket's sign invariant $f(\mathrm{lo})f(\mathrm{hi}) < 0$ survives every step by construction, so the hybrid inherits bisection's global guarantee while still finishing at newton's quadratic pace once the iterates behave. Production root finders are all of this shape: Brent's zeroin interleaves bisection, secant, and inverse quadratic interpolation under exactly this guard, and it is the default solver of the major numerical libraries, the safeguarded-iteration practice Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed, records.

The fixture needs a function where pure newton genuinely fails: $f(x) = \operatorname{atan} x$. It is monotone with $f'(x) = 1/(1+x^2)$, its one root is 0, and cppreference pins its values inside $[-\pi/2, \pi/2]$ with $\operatorname{atan}(\pm 0) = \pm 0$, fetched

2026-09-22. Newton from $x_0 = 2$ walks into the trap: the slope there is only 0.2, so the step overshoots to -3.535743588970452, and the ladder explodes, 13.95095908692749, -279.34406653361731, 1.2×10^{5}, -2.3×10^{10}, 8.59×10^{20}: for large x the step grows like x^2, quadratic divergence running backwards.

The dry run: the hybrid, bracket $[-1, 4]$, newton candidate from 2.

1. Step 1: newton proposes -3.535743588970452, outside the bracket, rejected. Bisect: $t = 1.5$, bracket tightens to $[-1, 1.5]$.
2. Step 2: newton from 1.5 proposes -1.694, still outside. Bisect: $t = 0.25$, bracket $[-1, 0.25]$.
3. Step 3: newton from 0.25 proposes -0.01028982957229313, inside. Accepted, bracket $[-0.010289\ldots, 0.25]$.
4. Steps 4 and 5: accepted newton steps, $7.26 \times 10^{-}7$ then $-2.55 \times 10^{-}19$, residual atan of the final point under 10^{-16}.

step	guard	accepted t	bracket after
1	bisect	1.5	[−1, 1.5]
2	bisect	0.25	[−1, 0.25]
3	newton	−0.010289829572293	[−0.0103, 0.25]
4	newton	+7.263134553e-7	[−0.0103, 7.26e-7]
5	newton	−2.554863127e-19	[−2.55e-19, 7.26e-7]

The two-number play, one function, one start, one guard apart: pure newton ends the sixth update beyond 8.59×10^{20}, the hybrid ends the fifth at $-2.55 \times 10^{-}19$. Two rejected proposals bought the whole trajectory. The same discipline extends outward: book 2, ch 11 (mathematics for programmers, continuous optimization) guards fast descent directions with trust regions the same way, and the contest drivers of book 9, ch 36 (practical data structures and algorithms, numerical methods) pin fixed iteration counts so a cycle cannot silently burn the clock.

```c
typedef enum : uint8_t { BISECT_STEP, NEWTON_STEP } step_kind;

typedef struct {
  double t;      // the accepted point
  double lo, hi; // the bracket after acceptance
  step_kind how; // which candidate the guard let through
} trace_step;

static double atan_drv(double x) { return atan(x); }

// the hybrid: try newton from the current point, keep the step only inside
// the bracket, otherwise bisect, then restore the sign invariant
static trace_step guard_step(double lo, double hi, double x, double flo) {
  trace_step s;
  double fx = atan_drv(x);
  double fpx = 1.0 / (1.0 + x * x);
  double t = x - fx / fpx;
  s.how = NEWTON_STEP;
  if (!(t > lo && t < hi)) {
    t = (lo + hi) / 2;
    s.how = BISECT_STEP;
  }
  double ft = atan_drv(t);
  if ((ft < 0.0) == (flo < 0.0))
    lo = t;
  else
    hi = t;
  s.t = t;
  s.lo = lo;
  s.hi = hi;
  return s;
}
```

listing 224: safeguard.c, the guard step: newton proposal, bracket membership test, bisection fallback, sign invariant restored (source: math/samples/src/Ch20/safeguard.c, lines 35-66)

```
    // pure newton on atan x from 2, no bracket: -3.535743588970452,
    // 13.95095908692749, -279.34406653361731, then 1.2e5, -2.3e10,
    // 8.5907666719503535e+20
    double p1 = 2.0;
    double f1 = atan(p1);
    p1 = p1 - f1 / (1.0 / (1.0 + p1 * p1));
    CHECK(fabs(p1 + 3.535743588970452) < 1e-12 * 3.6, "pure rung 1 pins");
    double p2 = p1;
    double f2 = atan(p2);
    p2 = p2 - f2 / (1.0 / (1.0 + p2 * p2));
    CHECK(fabs(p2 - 13.95095908692749) < 1e-12 * 14, "pure rung 2 pins");
    double p3 = p2;
    double f3 = atan(p3);
    p3 = p3 - f3 / (1.0 / (1.0 + p3 * p3));
    CHECK(fabs(p3 + 279.34406653361731) < 1e-12 * 280, "pure rung 3 pins");
    // rungs 4 and 5, the midsection of the blowup: 122016.99891795448 then
    // -23386004197.933853, tolerance pinned like the early rungs
    double a4 = p3;
    double g4 = atan(a4);
    a4 = a4 - g4 / (1.0 / (1.0 + a4 * a4));
    double a5 = a4;
    double g5 = atan(a5);
    a5 = a5 - g5 / (1.0 / (1.0 + a5 * a5));
    CHECK(fabs(a4 - 122016.99891795448) < 1e-12 * 122017.0 &&
              fabs(a5 + 23386004197.933853) < 1e-12 * 2.34e10,
          "pure rungs 4 and 5 pin the midsection");
    // rung 6: 8.5907666719503535e+20, asserted in the contrast check below
    double p6v = a5;
    double f6 = atan(p6v);
    p6v = p6v - f6 / (1.0 / (1.0 + p6v * p6v));
```

listing 225: safeguard.c, pure newton on atan diverges, rungs 1 through 5 pinned, rung 6 blown past 1e20 (source: math/samples/src/Ch20/safeguard.c, lines 130-159)

```
  // the hybrid on the same start: bracket [-1, 4], newton candidate from 2.
  // trace: BISECT 1.5, BISECT 0.25, NEWTON -0.01028982957229313, NEWTON
  // 7.2631345530115021e-07, NEWTON -2.5548631271557834e-19
  double hlo = -1.0, hhi = 4.0;
  double flo = atan(hlo);
  double hx = 2.0;
  static const step_kind mode_pin[5] = {BISECT_STEP, BISECT_STEP, NEWTON_STEP,
                                        NEWTON_STEP, NEWTON_STEP};
  static const double t_pin[5] = {1.5, 0.25, -0.01028982957229313,
                                  7.2631345530115021e-07,
                                  -2.5548631271557834e-19};
  int trace_ok = 1, invariant_ok = 1;
  for (auto i = 0; i < 5; i++) {
    trace_step s = guard_step(hlo, hhi, hx, flo);
    if (s.how != mode_pin[i] || fabs(s.t - t_pin[i]) > 1e-12 * fabs(t_pin[i]))
      trace_ok = 0;
    if (!(atan(s.lo) * atan(s.hi) < 0.0))
      invariant_ok = 0;
    hlo = s.lo;
    hhi = s.hi;
    hx = s.t;
  }
  CHECK(trace_ok, "hybrid trace pins, two bisections then three newtons");
  CHECK(invariant_ok, "bracket sign invariant holds at every step");
  CHECK(fabs(atan(hx)) < 1e-16, "hybrid residual under 1e-16");

  // same function, same start, one guard: 8.6e20 versus 2.6e-19
  CHECK(fabs(p6v) > 1e20 && fabs(hx) < 1e-18,
        "guard turns divergence into 19 zeros");
```

listing 226: safeguard.c, the hybrid trace, two bisections then three newtons, invariant checked every step (source: math/samples/src/Ch20/safeguard.c, lines 161-190)

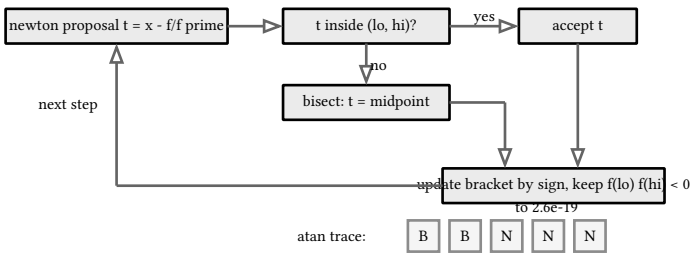


figure 125: the safeguard decision loop: propose with newton, test bracket membership, fall back to bisection, always re-establish the sign invariant

NOTE: the practical default

When you cannot analyze the function, use a bracket-maintaining hybrid: pay the two evaluations of a sign search for the bracket, then let the guard decide. Every method of this chapter is available in the samples as a 30-line driver, and the hybrid is the one worth copying first, because it is the only one whose worst case is bisection.

The next chapter, book 2, ch 21 (mathematics for programmers, interpolation and approximation), asks the mirror question: instead of the point where a function is zero, find the polynomial that passes through given points, and the error ladders of this chapter become interpolation error bounds.

sources: Nicholas J. Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed, SIAM 2002, cited by name for the multiplicity rate degeneration and the safeguarded-iteration practice behind Brent-style solvers, no page text quoted. Lloyd N. Trefethen and David Bau, Numerical Linear Algebra, SIAM 1997, cited by name for the

linear, superlinear, quadratic convergence taxonomy. cppreference atan, en.cppreference.com/w/c/numeric/math/atan, fetched 2026-09-22, principal value range and zero handling quoted from the page. Every iteration ladder, ratio, and count pinned by the python mirrors in playground/math-ch20 and re-measured by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch20`, 60 checks in chapter 20 of the math suite.

21. interpolation and approximation

Chapter book 2, ch 7 (mathematics for programmers, inner products and projections) built the inner product that turns an overdetermined system into a projection and chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) factored the matrices that solve systems stably. Between them sits the question this chapter answers: given points from a function, which polynomial should stand in for it, and what does that substitution cost in error. Six moves: the interpolation problem as a vandermonde linear system solved exactly in rationals and then in doubles, the lagrange and newton forms that evaluate the same polynomial without solving anything, the runge phenomenon where more equally spaced nodes make the answer worse, chebyshev nodes where the same count makes it better, cubic splines that trade one global polynomial for joined pieces, and least squares fitting that abandons interpolation on purpose. Every behavioral claim below is one of the 63 checks in the 4 samples of chapter 21 or a sentence quoted from a canonical source fetched 2026-09-22. The dsa book's numerical chapter runs iterative drivers for contest boards, this chapter is the approximation theory those drivers lean on.

21.1. the interpolation problem

The problem: given $n+1$ distinct nodes $x_0, \dots, x_n$ and values $y_i = f(x_i)$, find the polynomial p of degree at most n with $p(x_i) = y_i$ for every node. At most one exists, because the difference of two candidates has $n+1$ distinct roots and degree at most n, which forces it to be the zero polynomial. One exists because the coefficients solve a linear system: writing $p(x) = a_0 + a_1 x + \cdots + a_n x^n$, the interpolation conditions read $\boldsymbol{V}\boldsymbol{a} = \boldsymbol{y}$ with the vandermonde matrix $V_{ij} = x_i^j$, and

$$\det \boldsymbol{V} = \prod_{i<j} (x_j - x_i)$$

is nonzero whenever the nodes are distinct (the product formula is standard, Higham, Accuracy and Stability of Numerical Algorithms, SIAM 2002, 2nd ed., by name). Uniqueness and existence in one determinant.

The dry run: the fixture is the runge function $f(x) = \frac{1}{1+25x^2}$ at the nodes $-1, 0, 1$, so $\boldsymbol{y} = \left(\frac{1}{26}, 1, \frac{1}{26}\right)$.

1. The middle row of $\boldsymbol{V}$ is $(1, 0, 0)$, so it reads $a_0 = 1$ outright.
2. The two end rows are $1 - a_1 + a_2 = \frac{1}{26}$ and $1 + a_1 + a_2 = \frac{1}{26}$; subtracting them kills $a_1 = 0$, and either one gives $a_2 = \frac{1}{26} - 1 = -\frac{25}{26}$.
3. The interpolant is $p(x) = 1 - \frac{25}{26}x^2$, and "det" $\boldsymbol{V} = (0-(-1))(1-(-1))(1-0) = 2$, both pinned exactly in the sample.

The exact solve is the easy half. The vandermonde matrix of equally spaced nodes on $[-1, 1]$ becomes a trap as n grows, and the ladder is pinned by the playground one-off (numpy 2.5.3, 2-norm):

degree n	nodes	cond(V)	double-solve error
4	5	23.5	4.4 times 10^(-16)
8	9	1.61 times 10^3	-
12	13	1.23 times 10^5	-
16	17	9.98 times 10^6	4.1 times 10^(-8)

The two-number play: the same solve, the same function, the same machine, and between $n = 4$ and $n = 16$ the condition number climbs six orders while the computed coefficients drift from 4.4 times 10^(-16) of the exact ones to 4.1 times 10^(-8). The exact degree 16 interpolant of this smooth bump is itself a warning: its coefficients reach 63743.77 in magnitude, and the sample CHECKs that the double solve picks up a real error band around the exact values.

```
// ── the vandermonde solve, exact and double ──────────────────────
rat xe[3] = {rat_mk(-1, 1), rat_mk(0, 1), rat_mk(1, 1)};
rat ve[9] = {0};
rat ye[3];
for (int i = 0; i < 3; i++) {
  rat p = rat_mk(1, 1);
  for (int j = 0; j < 3; j++) {
    ve[i * 3 + j] = p;
    p = rat_mul(p, xe[i]);
  }
  ye[i] = rat_runge(xe[i]);
}
rat ae[3] = {0};
rat_solve(3, ve, ye, ae);
CHECK(rat_is(ae[0], 1, 1), "3-node runge solve a0 = 1");
CHECK(rat_is(ae[1], 0, 1), "3-node runge solve a1 = 0");
CHECK(rat_is(ae[2], -25, 26), "3-node runge solve a2 = -25/26");

// det V = product of (x_j - x_i) over i < j, here exactly 2
rat det = rat_mul(rat_mul(rat_sub(xe[1], xe[0]), rat_sub(xe[2], xe[0])),
                  rat_sub(xe[2], xe[1]));
CHECK(rat_is(det, 2, 1), "det V from the product formula = 2");

double x3[3] = {-1.0, 0.0, 1.0};
double v3[9], b3[3], a3[3];
vandermonde(3, x3, v3);
for (int i = 0; i < 3; i++)
  b3[i] = dbl_runge(x3[i]);
dbl_solve(3, v3, b3, a3);
double e3[3] = {1.0, 0.0, -25.0 / 26.0};
CHECK(max_abs_diff(a3, e3, 3) < 1e-15, "double 3-node solve matches exact");
```

listing 227: interp.c, the 3-node system solved exactly in rationals and again in doubles (source: math/samples/src/Ch21/interp.c, lines 186-216)

PITFALL: NEVER SOLVE VANDERMONDE IN PRODUCTION

The coefficient route is the demonstration, not the tool. The forms in the next section evaluate the interpolant directly, at better cost and without handing the answer to a matrix whose condition number grows exponentially in the node count (Higham, by name).

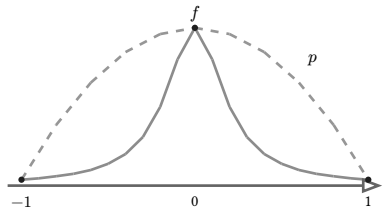


figure 126: the runge bump on three nodes and its exact parabola, agreeing at the nodes and nowhere else

21.2. the lagrange and newton forms

The vandermonde detour asks a matrix for coefficients. The lagrange form writes the interpolant straight down:

$$p(x) = \sum_{i=0}^{n} y_i L_{i(x)}, \quad L_{i(x)} = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Each basis polynomial L_i is degree n, vanishes at every node except x_i, and takes the value 1 there, so the weighted sum hits every y_i by construction. The basis is also a partition of unity, $\sum_i L_{i(x)} = 1$, because the constant 1 is its own interpolant.

The dry run: nodes $0, 1, 2, 3$ carrying $f(x) = x^3$, evaluated at $x = \frac{1}{2}$.

1. $L_0(\frac{1}{2}) = \big((\frac{1}{2}-1)(\frac{1}{2}-2)(\frac{1}{2}-3)\big)/((0-1)(0-2)(0-3)) = \frac{5}{16}$, and the same three quotients give $\frac{15}{16}, -\frac{5}{16}, \frac{1}{16}$ for L_1, L_2, L_3.
2. The four values sum to 1, and the weighted sum is $0 \cdot \frac{5}{16} + 1 \cdot \frac{15}{16} + 8 \cdot \big(-\frac{5}{16}\big) + 27 \cdot \frac{1}{16} = \frac{1}{8} = \big(\frac{1}{2}\big)^3$, exact in binary because every entry is dyadic.

The newton form reorganizes the same polynomial into nested pieces that grow with the data. Divided differences are defined level by level, $f[x_i, \ldots, x_{i+k}] = (f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}])/(x_{i+k} - x_i)$, and the interpolant is $p(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots$. For x^3 at $0, 1, 2, 3$ the whole table is integers:

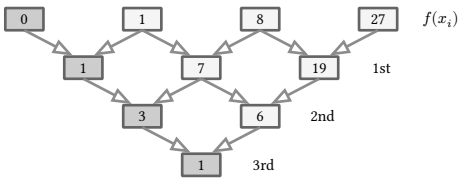
values	1st	2nd	3rd
0, 1, 8, 27	1, 7, 19	3, 6	1

and $p(x) = x + 3x(x-1) + x(x-1)(x-2)$. Evaluated nested, innermost out, at $x = \frac{1}{2}$: start 1, wrap to $1 \cdot \big(\frac{1}{2} - 2\big) + 3 = \frac{3}{2}$, then $\frac{3}{2} \cdot \big(\frac{1}{2} - 1\big) + 1 = \frac{1}{4}$, then $\frac{1}{4} \cdot \big(\frac{1}{2} - 0\big) + 0 = \frac{1}{8}$. The trace is pinned in the sample, all dyadic.

The two-number play: one evaluation of the lagrange form at $n = 3$ costs 12 multiplied quotients in the four basis products, while the nested newton form costs 3 multiplications after the table. The counts generalize to n^2 products versus n. The newton table also updates incrementally: append a node, append one difference, and the sample CHECKs the first three coefficients do not move. Both forms reproduce x^3 at the off-node probes $-0.3, 0.7, 1.9$, the uniqueness argument of the first section as a measured fact.

```
// ── lagrange form, nodes 0,1,2,3 on f = x^3 ──────────────────────────
double xn[4] = {0.0, 1.0, 2.0, 3.0};
double yn[4] = {0.0, 1.0, 8.0, 27.0};
double bhalf[4];
for (int i = 0; i < 4; i++)
  bhalf[i] = lagrange_basis(xn, i, 0.5);
// pin.py: 5/16, 15/16, -5/16, 1/16, all dyadic
CHECK(fabs(bhalf[0] - 0.3125) < 1e-15 && fabs(bhalf[1] - 0.9375) < 1e-15,
      "L0(0.5) = 5/16 and L1(0.5) = 15/16");
double sum = 0.0;
for (int i = 0; i < 4; i++)
  sum += bhalf[i];
CHECK(fabs(sum - 1.0) < 1e-15, "basis sums to 1 at 0.5");
double pl = 0.0;
for (int i = 0; i < 4; i++)
  pl += yn[i] * bhalf[i];
CHECK(fabs(pl - 0.125) < 1e-15, "lagrange p(0.5) = 1/8");
double summ = 0.0, pm = 0.0;
for (int i = 0; i < 4; i++) {
  double L = lagrange_basis(xn, i, -0.5);
  summ += L;
  pm += yn[i] * L;
}
CHECK(fabs(summ - 1.0) < 1e-15, "basis sums to 1 at -0.5");
```

listing 228: interp.c, lagrange basis values and the partition of unity on four nodes (source: math/samples/src/Ch21/interp.c, lines 263-286)



shaded diagonal is the newton coefficient list 0, 1, 3, 1

figure 127: the divided-difference triangle, its diagonal marked as the newton coefficient list

21.3. the runge phenomenon

Interpolating a smooth function at more equally spaced nodes can make the result worse, not better. The demonstration function is the same runge bump, and the ladder is measured on a fixed probe grid of 2001 points across $[-1, 1]$, worst $|p(x) - f(x)|$.

The dry run: the same runge bump on equally spaced nodes, measured as the worst miss over the 2001-point probe grid. The ladder reads 0.438356640 at degree 4, 1.045173912 at degree 8, 3.663262143 at degree 12, and 14.393851285 at degree 16.

1. degree 4, 5 nodes: 0.438356640
2. degree 8, 9 nodes: 1.045173912
3. degree 12, 13 nodes: 3.663262143
4. degree 16, 17 nodes: 14.393851285

The two-number play: quadrupling the degree multiplies the worst miss by 33, and the degree 16 miss, 14.4, is more than fourteen times the entire height of the curve, which never leaves $\left[\frac{1}{26}, 1\right]$. The oscillation concentrates just inside the ends, near $x \pm 0.95$ for degree 12, where the sample pins the 3.663 miss.

The cause is a race between two facts. A polynomial through $n+1$ nodes commits the error $f(x) - p(x) = f^{(n+1)}(\xi)/(n+1)! \prod_i (x - x_i)$ at each x (the standard remainder formula, stated here without proof), and equally spaced nodes make $\prod_i (x - x_i)$ explode near the ends: the product of distances from an end-adjacent point to the far nodes is enormous. Meanwhile the runge function has poles at $\pm 0.2i$, close to the real axis, so the high derivatives in the remainder do not cooperate (Trefethen, Approximation Theory and Approximation Practice, SIAM 2019, 2nd ed., by name; Wikipedia, Runge's phenomenon, fetched 2026-09-22).

VERIFY: HOW THE LADDER WAS MEASURED

The probe grid is $x = -1 + j/1000$ for $j = 0, \ldots, 2000$, the same grid in the playground one-off and in the C sample, and every ladder entry is CHECKed to 1e-6 against the pinned value. The chebyshev ladder in the next section uses the identical grid, so the comparison is apples to apples.

```
  double x[25], y[25];

  // ── equally spaced nodes: the ladder grows ──────────────────────────
  const int degs[4] = {4, 8, 12, 16};
  const double pin_eq[4] = {0.438356640, 1.045173912, 3.663262143,
                            14.393851285};
  double ladder[4];
  for (int d = 0; d < 4; d++) {
    equispaced(degs[d], x);
    for (int i = 0; i ≤ degs[d]; i++)
      y[i] = runge(x[i]);
    ladder[d] = max_error(x, y, degs[d] + 1);
    CHECK(fabs(ladder[d] - pin_eq[d]) < 1e-6, "equispaced ladder value");
  }
  CHECK(ladder[3] > 30.0 * ladder[0], "error grows by 33x from n = 4 to 16");
```

listing 229: runge.c, the equispaced ladder, each entry CHECKed against the pinned constant (source: math/samples/src/Ch21/runge.c, lines 73-87)

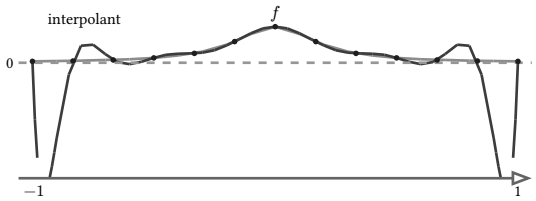


figure 128: degree 12 interpolation of the runge bump on 13 equally spaced nodes, plunging out of frame near both ends

21.4. chebyshev nodes

The remainder formula of the last section separates the error into a function factor and a node factor, $\prod_i (x - x_i)$. The nodes are the part you choose, so choose them to minimize the maximum of that product. On $[-1, 1]$ the winner is the zero set of the chebyshev polynomial: DLMF eq 18.5.1 gives $T_{m(x)} = \cos(m\theta)$ with $x = \cos\theta$ (https://dlmf.nist.gov/18.5, fetched 2026-09-22), so the zeros of T_{n+1} sit at

$$x_k = \cos\left((2k+1)\frac{\pi}{2n+2}\right), \quad k = 0, \ldots, n$$

(DLMF 18.16(iii), https://dlmf.nist.gov/18.16, fetched 2026-09-22), which piles nodes near the ends exactly where the equally spaced grid starves them. Minimality is chebyshev's theorem: among monic degree $n+1$ polynomials, $2^{-n}T_{n+1}$ has the smallest maximum norm on $[-1, 1]$, the minimax statement behind the equioscillation theory (Trefethen, ATAP, by name, no proof here).

The dry run: for $n = 4$ the five nodes are $\cos 18° = 0.951057$, $\cos 54° = 0.587785$, $\cos 90° = 0$, and their mirrors, pinned to 12 decimals in the sample. The same runge bump, the same probe grid, the same degrees:

degree n	equally spaced	chebyshev
4	0.438356640	0.402016742
8	1.045173912	0.170833740
12	3.663262143	0.069215708
16	14.393851285	0.032613371
24	-	0.006948418

The two-number play: at degree 16 the equally spaced interpolant misses by 14.394 and the chebyshev interpolant by 0.032614, a factor of 441, and the chebyshev column is strictly decreasing all the way down. The crossover is already visible at degree 4, where both miss by about 0.4 and the node placement has barely started to matter.

```
// ── chebyshev nodes: placement and the ladder shrinks ──────────────
chebyshev(4, x);
// pin.py: cos(18 deg) and cos(54 deg), the middle node is exactly cos(pi/2)
CHECK(fabs(x[0] - 0.951056516295154) < 1e-12 &&
          fabs(x[1] - 0.587785252292473) < 1e-12,
      "chebyshev n = 4 outer nodes at cos(18) and cos(54)");
CHECK(fabs(x[2]) < 1e-12, "chebyshev middle node lands on 0");
double antisym = 0.0;
for (int k = 0; k ≤ 4; k++)
  antisym += x[k];
CHECK(fabs(antisym) < 1e-12, "chebyshev nodes are odd-symmetric");

const double pin_che[5] = {0.402016742, 0.170833740, 0.069215708, 0.032613371,
                           0.006948418};
const int che_degs[5] = {4, 8, 12, 16, 24};
double che_ladder[5];
for (int d = 0; d < 5; d++) {
  chebyshev(che_degs[d], x);
  for (int i = 0; i ≤ che_degs[d]; i++)
    y[i] = runge(x[i]);
  che_ladder[d] = max_error(x, y, che_degs[d] + 1);
  CHECK(fabs(che_ladder[d] - pin_che[d]) < 1e-6, "chebyshev ladder value");
}
CHECK(che_ladder[0] > che_ladder[1] && che_ladder[1] > che_ladder[2] &&
          che_ladder[2] > che_ladder[3] && che_ladder[3] > che_ladder[4],
      "chebyshev ladder is strictly decreasing");
CHECK(ladder[3] > 400.0 * che_ladder[3],
      "n = 16 head to head: 14.39 equispaced versus 0.0326 chebyshev");
```

listing 230: runge.c, chebyshev placement, the shrinking ladder, and the degree 16 head to head (source: math/samples/src/Ch21/runge.c, lines 89-116)

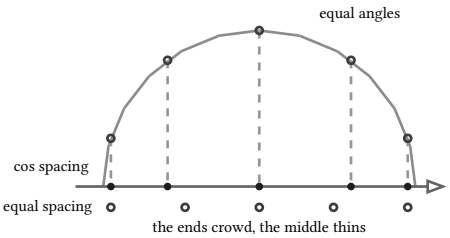


figure 129: chebyshev nodes as equal angles projected onto the axis, against equally spaced dots

21.5. cubic splines

The global polynomial is the problem, so drop it: fit one cubic per interval and glue them smoothly. A cubic spline on knots $x_0, \ldots, x_n$ is a function S that is a cubic polynomial on each $[x_i, x_{i+1}]$ and twice continuously differentiable across every interior knot. The natural spline pins $S''(x_0) = S''(x_n) = 0$, and with the second derivatives $M_i = S''(x_i)$ as unknowns the smoothness conditions collapse to one tridiagonal system per interior knot,

$$\frac{h_{i-1}}{6} M_{i-1} + \frac{h_{i-1} + h_i}{3} M_i + \frac{h_i}{6} M_{i+1} = \frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}$$

with $h_i = x_{i+1} - x_i$ (the M-form, Burden and Faires, Numerical Analysis, by name). Solved by the thomas sweep in $O(n)$, no matrix stored.

The dry run: knots $0, 1, 2, 3$ on $f(x) = \frac{1}{1+x^2}$, values $1, \frac{1}{2}, \frac{1}{5}, \frac{1}{10}$, all $h_i = 1$.

1. The right sides are the second differences, $\frac{1}{5} - 2 \cdot \frac{1}{2} + 1 = \frac{1}{5}$ and $\frac{1}{10} - 2 \cdot \frac{1}{5} + \frac{1}{2} = \frac{1}{5}$, equal by symmetry.

2. The system is $\frac{2}{3}M_1 + \frac{1}{6}M_2 = \frac{1}{5}$ and its mirror, so $M_1 = M_2 = \frac{6}{25}$, with $M_0 = M_3 = 0$.
3. On $[0, 1]$ the local form gives $S(\frac{1}{2}) = \frac{147}{200} = 0.735$, and at the knot $S(1) = \frac{1}{2}$, $S'(1) = -\frac{21}{50}$, $S''(1) = \frac{6}{25}$, each CHECKed from both adjoining pieces to 1e-12.

The head-to-head returns to the runge bump with 13 equally spaced knots. The natural spline misses by 0.006908626 where the degree 12 interpolant on the same knots misses by 3.663262143, a factor of 530, and the second derivatives tell the story: $M_6 = -52.306077941$ at the center, where the curve bends hardest, easing to exactly 0 at the natural ends. The two-number play: one global polynomial, 3.663; thirteen joined cubics, 0.0069.

knot x_i	$y_i = \frac{1}{1+x_i^2}$	M_i
0	1	0
1	$\frac{1}{2}$	$\frac{6}{25}$
2	$\frac{1}{5}$	$\frac{6}{25}$
3	$\frac{1}{10}$	0

```
static void spline_m(const double *x, const double *y, int n, double *m) {
  double a[64], b[64], c[64], d[64];
  for (int i = 0; i ≤ n; i++)
    m[i] = 0.0;
  for (int i = 1; i < n; i++) {
    double hl = x[i] - x[i - 1];
    double hr = x[i + 1] - x[i];
    a[i] = hl / 6.0;
    b[i] = (hl + hr) / 3.0;
    c[i] = hr / 6.0;
    d[i] = (y[i + 1] - y[i]) / hr - (y[i] - y[i - 1]) / hl;
  }
  for (int i = 2; i < n; i++) {
    double w = a[i] / b[i - 1];
    b[i] -= w * c[i - 1];
    d[i] -= w * d[i - 1];
  }
  m[n - 1] = d[n - 1] / b[n - 1];
  for (int i = n - 2; i ≥ 1; i--)
    m[i] = (d[i] - c[i] * m[i + 1]) / b[i];
}

// interval index for t, clamped to the last piece
[[nodiscard]] static int interval_of(const double *x, int n, double t) {
  int i = 0;
  while (i < n - 1 && t > x[i + 1])
    i++;
  return i;
}

// S, S', S'' on piece i at t, from the local form
// S = y_i (1-t) + y_{i+1} t + h^2 [ M_i (t^2/2 - t^3/6 - t/3)
//                                  + M_{i+1} (t^3/6 - t/6) ]
typedef struct {
  double s0;
  double s1;
  double s2;
} sval;

[[nodiscard]] static sval spline_at(const double *x, const double *y,
                                    const double *m, int i, double t) {
  double h = x[i + 1] - x[i];
  double dy = y[i + 1] - y[i];
  double phi2 = t * t / 2.0 - t * t * t / 6.0 - t / 3.0;
  double psi2 = t * t * t / 6.0 - t / 6.0;
  double phi1 = t - t * t / 2.0 - 1.0 / 3.0;
  double psi1 = t * t / 2.0 - 1.0 / 6.0;
  return (sval){
      y[i] + t * dy + h * h * (m[i] * phi2 + m[i + 1] * psi2),
      (dy + h * h * (m[i] * phi1 + m[i + 1] * psi1)) / h,
      m[i] * (1.0 - t) + m[i + 1] * t,
  };
}
```

listing 231: spline.c, the tridiagonal M system by thomas, then S, S', S'' from the local form (source: math/samples/src/Ch21/spline.c, lines 32-84)

NOTE: WHY NOT MORE PIECES

> Halving every interval divides cubic spline error by roughly 16, fourth-order convergence, while the global interpolant on those same nodes heads the other way. Piecewise wins by refusing to coordinate: each cubic only answers for its own interval.

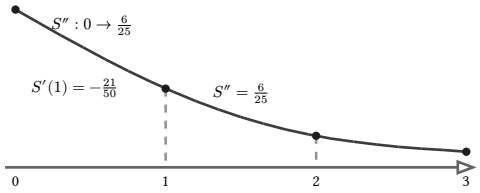


figure 130: the 4-knot fixture, one smooth curve from three cubics, second derivative continuous across the interior knots

21.6. least squares fitting

Sometimes the honest answer is to miss every point. With m data points and a degree d model, $m > d + 1$, no polynomial hits them all, and chapter book 2, ch 7 (mathematics for programmers, inner products and projections) already built the right machinery: the design matrix $\boldsymbol{A}$ with rows $\left(1, x_i, \ldots, x_i^d\right)$, and the fit $\boldsymbol{c}$ solving the normal equations $\boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{c} = \boldsymbol{A}^\top \boldsymbol{y}$, the projection of $\boldsymbol{y}$ onto the column space under the standard inner product.

The fixture: five points at $x = -2, -1, 0, 1, 2$, a line of slope 1/2 carrying bumps $\left(\frac{1}{16}, 0, \frac{1}{4}, 0, \frac{1}{16}\right)$, so every value is an exact dyadic rational.

1. Degree 1: $\boldsymbol{A}^\top \boldsymbol{A}$ is "diag"(5, 10) on the symmetric grid and $\boldsymbol{A}^\top \boldsymbol{y} = \left(\frac{43}{8}, 5\right)$, so $\boldsymbol{c} = \left(\frac{43}{40}, \frac{1}{2}\right)$.
2. The residuals are $\left(-\frac{1}{80}, -\frac{3}{40}, \frac{7}{40}, -\frac{3}{40}, -\frac{1}{80}\right)$, and the two orthogonality sums $\sum r_i = 0$ and $\sum x_i r_i = 0$ hold as exact rational zeros, the sample CHECKs them in fraction arithmetic.
3. Degree 2: $\boldsymbol{c} = \left(\frac{311}{280}, \frac{1}{2}, -\frac{1}{56}\right)$, all three orthogonality sums $\sum x_i^k r_i = 0$ for $k = 0, 1, 2$ still exactly zero.

The residual sum of squares falls from $\frac{27}{640} = 0.0421875$ to $\frac{169}{4480} = 0.0377232$, an 11 percent drop for one extra parameter. The cautionary contrast is the degree 4 interpolant through the same five points: at $x = \frac{1}{2}$ it stands $\frac{179}{1024} = 0.1748$ away from the generating line while the degree 2 fit stands $\frac{17}{160} = 0.1063$ away. The honest grid-wide statement is about the worst case, not every point: sweeping 801 points across $[-2, 2]$ the interpolant's largest departure from the line is $\frac{1}{4}$, at the noisy knot $x = 0$, while the fit's largest is $\frac{31}{280} = 0.111$. Pointwise the picture is mixed, the interpolant sits exactly on the line at the two noise-free knots and is closer than the fit at 268 of the 801 points, but it pays for those with the 0.25 spike. The two-number play: 0.175 bought by hitting every point, 0.106 bought by missing all of them on purpose, and 0.25 versus 0.111 as the worst each risks anywhere.

```
    y[i] =
        rat_add(rat_add(rat_mk(1, 1), rat_div(x[i], rat_mk(2, 1))), noise[i]);

  // ── degree 1: A^T A c = A^T y over the inner product of column vectors
  rat ata[4] = {0}, aty[2] = {0}, c1[2];
  for (int j = 0; j < 2; j++) {
    for (int k = 0; k < 2; k++) {
      ata[j * 2 + k] = rat_mk(0, 1);
      for (int r = 0; r < 5; r++)
        ata[j * 2 + k] = rat_add(ata[j * 2 + k],
                                 rat_mul(rat_pow(x[r], j), rat_pow(x[r], k)));
    }
    aty[j] = rat_mk(0, 1);
    for (int r = 0; r < 5; r++)
      aty[j] = rat_add(aty[j], rat_mul(rat_pow(x[r], j), y[r]));
  }
  rat_solve(2, ata, aty, c1);
  CHECK(rat_is(c1[0], 43, 40), "degree 1 intercept 43/40");
  CHECK(rat_is(c1[1], 1, 2), "degree 1 slope 1/2");

  rat r1[5], s0 = rat_mk(0, 1), s1 = rat_mk(0, 1), ss1 = rat_mk(0, 1);
  for (int i = 0; i < 5; i++) {
    r1[i] = rat_sub(y[i], rat_add(c1[0], rat_mul(c1[1], x[i])));
    s0 = rat_add(s0, r1[i]);
    s1 = rat_add(s1, rat_mul(x[i], r1[i]));
    ss1 = rat_add(ss1, rat_mul(r1[i], r1[i]));
  }
  CHECK(rat_is(r1[2], 7, 40) && rat_is(r1[0], -1, 80),
        "degree 1 residuals 7/40 at 0 and -1/80 at -2");
```

listing 232: lsq.c, the degree 1 normal equations, residuals, and exact orthogonality in rationals (source: math/samples/src/Ch21/lsq.c, lines 128-156)

PITFALL: THE NORMAL EQUATIONS SQUARE THE CONDITION

“cond”(bold(A)^top bold(A)) = “cond”(bold(A))^2, so the route this fixture takes for exactness is the wrong route in floating point when the fit grows. The QR decomposition of chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) fits the same model without ever forming $\boldsymbol{A}^{\top}\boldsymbol{A}$ (Trefethen and Bau, Numerical Linear Algebra, SIAM 1997, by name).

```
// ── the same 5 points interpolated exactly: the wiggle it buys ──────
// newton divided differences in rationals, evaluated at 0.5
rat dd[5][5] = {0};
for (int i = 0; i < 5; i++)
  dd[i][0] = y[i];
for (int j = 1; j < 5; j++)
  for (int i = 0; i + j < 5; i++)
    dd[i][j] = rat_div(rat_sub(dd[i + 1][j - 1], dd[i][j - 1]),
                       rat_sub(x[i + j], x[i]));
rat iv = rat_mk(0, 1);
for (int k = 4; k ≥ 0; k--)
  iv = rat_add(rat_mul(iv, rat_sub(rat_mk(1, 2), x[k])), dd[0][k]);
rat line = rat_mk(5, 4); // the true 1 + x/2 at x = 1/2
rat dev_i = rat_sub(iv, line);
rat fv = rat_add(rat_add(c2[0], rat_mul(c2[1], rat_mk(1, 2))),
                 rat_mul(c2[2], rat_mk(1, 4)));
rat dev_f = rat_sub(fv, line);
CHECK(rat_is(dev_i, 179, 1024), "interpolant strays 179/1024 from the line");
CHECK(rat_is(dev_f, 17, 160), "the fit strays only 17/160");
CHECK(dev_f.num * dev_i.den * 100 < dev_i.num * dev_f.den * 61,
      "fit deviation under 61 percent of the interpolant's");

// the honest grid-wide claim is about the WORST departure, pointwise is
// mixed (the interpolant is closer at 268 of 801 points, exact at the
// two noise-free knots). pin.py sweeps x = j/200 in exact fractions:
// interpolant worst 1/4 at x = 0, fit worst 31/280 at x = 0
double mi = 0.0, mf = 0.0;
for (int j = -400; j ≤ 400; j++) {
  double gx = (double)j / 200.0;
  double iv = 0.0;
  for (int k = 4; k ≥ 0; k--)
    iv = iv * (gx - rat_val(x[k])) + rat_val(dd[0][k]);
  double fv = rat_val(c2[0]) + rat_val(c2[1]) * gx + rat_val(c2[2]) * gx * gx;
  double ln = 1.0 + gx / 2.0;
  double di = fabs(iv - ln), df = fabs(fv - ln);
  if (di > mi)
    mi = di;
  if (df > mf)
    mf = df;
}
CHECK(fabs(mi - 0.25) < 1e-9 && fabs(mf - 31.0 / 280.0) < 1e-9,
      "worst grid departure: interpolant 1/4 versus fit 31/280");
CHECK(mf < mi / 2.0, "the fit's worst is under half the interpolant's worst");
```

listing 233: lsq.c, the degree 4 interpolant of the same points, its deviation play, and the worst-departure sweep (source: math/samples/src/Ch21/lsq.c, lines 193-235)

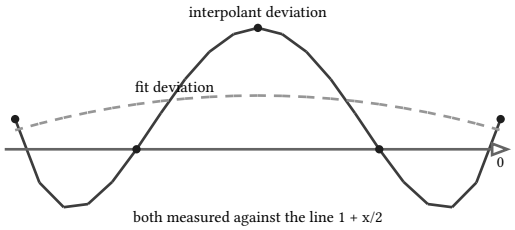


figure 131: deviation from the generating line: the interpolant swings, the fit stays flat

The next chapter book 2, ch 22 (mathematics for programmers, quadrature and differential equations) turns interpolation into areas under curves, and the error formulas pinned here become the error bounds of every quadrature rule there.

sources: no mml anchor sheet maps this chapter, it is worked from canonical references cited by name: Trefethen, Approximation Theory and Approximation Practice, SIAM 2019 2nd ed (equioscillation, chebyshev convergence); Trefethen and Bau, Numerical Linear Algebra, SIAM 1997 (normal equations conditioning); Higham, Accuracy and Stability of Numerical Algorithms, SIAM 2002 2nd ed (vandermonde conditioning); Burden and Faires, Numerical Analysis, Brooks/Cole Cengage 2011 9th ed (natural spline M-form). Fetched: DLMF 18.5 eq 18.5.1, T_n(cos theta) equals cos(n theta), https://dlmf.nist.gov/18.5 and DLMF 18.16(iii) zeros note, https://dlmf.nist.gov/18.16, both 2026-09-22; Wikipedia, Runge's phenomenon, https://en.wikipedia.org/wiki/Runge%27s_phenomenon, 2026-09-22. Expected sample values computed by the playground one-off playground/math-ch21/pin.py, python fractions and numpy 2.5.3, run 2026-09-22. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch21`, 63 checks in chapter 21 of the math suite, 4 files, zero failures.

22. quadrature and differential equations

Chapter book 2, ch 9 (mathematics for programmers, single-variable calculus) proved the fundamental theorem that turns an integral into an antiderivative evaluation, and chapter book 2, ch 21 (mathematics for programmers, interpolation and approximation) built the polynomials that stand in for functions without closed forms. This chapter spends those assets twice. First quadrature: approximate the integral itself when no antiderivative is available, with error ladders you can predict to four digits. Then differential equations: step a state forward in time, where the same discretization question decides whether a simulation decays like the physics or blows up by a factor of a million in three steps. Six moves: composite trapezoid and simpson with their h^2 and h^4 error ladders measured against the FTC truth, adaptive simpson that subdivides where the integrand actually lives, explicit and implicit euler against a stiff system, classic runge-kutta 4 with the cost and determinism story the physics capstone needs, absolute stability regions on the complex plane, and the symplectic leapfrog integrator the capstone's fixed-timestep core is built from. Every behavioral claim below is one of the 58 checks in the 4 samples of chapter 22 or a sentence quoted from a canonical source fetched 2026-09-22. Chapter book 9, ch 36 (practical data structures and algorithms, numerical methods) of the dsa book drives simpson as a contest tool, this chapter is the error and stability theory under it plus the entire ode half.

22.1. composite quadrature

The problem: approximate $\int_a^b f(x)\,\mathrm{d}x$ from node values $f(x_0), \ldots, x_n$ spaced $h = (b-a)/n$ apart. The composite trapezoid rule chords each panel,

$$T_n = h(1/2 f(x_0) + f(x_1) + \cdots + f(x_{n-1}) + 1/2 f(x_n)),$$

and the composite simpson rule fits a parabola through each pair of panels, which collapses to alternating weights 1, 4, 2, 4, ..., 4, 1 over a double interval of width $2h$ (DLMF §3.5(i) and (ii), fetched 2026-09-22). Chapter book 2, ch 7 (mathematics for programmers, inner products and projections) already used these weights as an inner product with 4000 panels, here they arrive with their price tags.

The fixture is $\int_0^1 e^x\,\mathrm{d}x$, chosen because chapter book 2, ch 9 (mathematics for programmers, single-variable calculus) proved the antiderivative of e^x is e^x, so the truth is $e - 1 = 1.718281828459045$ to the last bit.

The dry run: with $n = 2$, $h = 0.5$, and nodes 0, 0.5, 1, the trapezoid sum is $0.5(0.5 + 1.6487212707 + 1.3591409142) = 1.7539310925$, an error of 3.6e-2. Simpson on the same three nodes takes $1/6(1 + 4 \cdot 1.6487212707 + 2.7182818285) = 1.7188611519$, an error of 5.8e-4: the same evaluations, 61.5 times closer. On $f(x) = x^3$ with $n = 2$, simpson lands on exactly 0.25, not approximately: the parabola through the three nodes of a cubic on a double interval interpolates the cubic itself, so a rule built from degree 2 pieces integrates degree 3 exactly, the one order of accuracy simpson gets for free.

Halving h exposes the rates. The measured ladders:

n	trapezoid error	simpson error	ratio
1	1.409e-1		
2	3.565e-2	5.793e-4	
4	8.940e-3	3.701e-5	$15.7x$
8	2.237e-3	2.326e-6	$15.9x$
16	5.593e-4	1.456e-7	$16.0x$

The trapezoid column divides by 4 per row, simpson by 16: errors like ch^2 and ch^4. Euler-maclaurin supplies the constants too: the leading error terms are $h^2/12(f'(b) - f'(a))$ and $h^4/180(f'''(b) - f'''(a))$, which on this fixture are $(e-1)h^2/12$ and $(e-1)h^4/180$. At $n = 16$ those predictions give 5.5934e-4 and 1.4566e-7 against the measured 5.593e-4 and 1.456e-7: four digits from a two-term expansion. Two numbers to keep: at 16 panels

simpson is 3841 times closer to the truth than trapezoid on the same nodes, and its ladder collapses 16-fold per doubling while trapezoid manages 4.

```
static double trap(integrand f, double a, double b, int n) {
  double h = (b - a) / n;
  double s = 0.5 * (f(a) + f(b));
  for (int i = 1; i < n; i++)
    s += f(a + i * h);
  return h * s;
}

static double simp(integrand f, double a, double b, int n) {
  double h = (b - a) / n;
  double s = f(a) + f(b);
  for (int i = 1; i < n; i++)
    s += (i % 2 ? 4.0 : 2.0) * f(a + i * h);
  return h * s / 3.0;
}
```

listing 234: quadrature.c, the composite trapezoid and simpson sums, both pure weightings of the same node values (source: math/samples/src/Ch22/quadrature.c, lines 52-66)

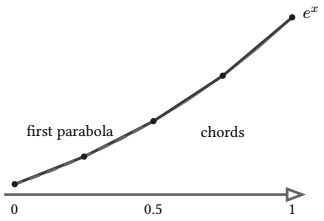


figure 132: the e^x fixture with 4 trapezoid chords and the first simpson parabola over [0, 0.5]

PITFALL: SIMPSON NEEDS AN EVEN PANEL COUNT

The alternating 1, 4, 2, ..., 4, 1 weights tile double intervals, so an odd n leaves a stranded panel and the formula has no last weight to give it. The ladder cannot hit that trap by construction: n doubles from 2, so every rung is even structurally. What the sample does pin at compile time is the depth of that ladder: the rung count is derived from the pinned value array and a `static_assert` demands at least three rungs, because the h^4 ratio needs two halvings to be measured at all.

22.2. adaptive quadrature

A tolerance flips the question from how accurate to when to stop. The adaptive idea: compute simpson on an interval and on its two halves, compare, and treat the difference as the error estimate. When simpson refines, its error drops by $2^4 = 16$, so the difference $|S_2 - S|$ between coarse and refined values overstates the finer error by a factor of about 15, and $|S_2 - S|/15$ is the standard estimate. Subdivide while $|S_2 - S| > 15\ \mathrm{tol}$, halving the tolerance with the interval width.

The dry run: on the runge bell $1/(1 + 25x^2)$ from chapter book 2, ch 21 (mathematics for programmers, interpolation and approximation), integrated over $[-1, 1]$, the initial coarse simpson is $(1/3)(0.0385 + 4 + 0.0385) = 1.358974$ while its refinement into halves gives $0.265031 + 0.265031 = 0.530062$. The gap is enormous, so the root splits, and the leftmost chain keeps splitting until the interval $[-1, -0.875]$ accepts at depth 4 with $S = 0.005463$ against a refinement gap inside $15 \cdot 10^{-}6/16$. The true value of that leaf is pinned, and by symmetry the last leaf of the run mirrors it: the accepted trace is 30 leaves wide, 121 function evaluations, final error 6.3e-10 against a requested $10^{-}6$.

leaf	interval	depth	width
1	$[-1, -0.875]$	4	0.125
2	$[-0.875, -0.75]$	4	0.125
5	$[-0.5, -0.4375]$	5	0.0625
10	$[-0.1875, -0.15625]$	6	0.03125
15	$[-0.03125, 0]$	6	0.03125
30	$[0.875, 1]$	4	0.125

The bell is smooth everywhere, so the census stays shallow and uniform simpson at $n = 128$ (129 evaluations) enters the same tolerance: no prize for adaptivity. The spike fixture $1/(1 + 10000(x - 0.7)^2)$ is where the method earns its recursion. Adaptive simpson honors $10^{-}6$ with 153 evaluations, 38 leaves, 34 of them intersecting $(0.55, 0.85)$ where the spike lives, refining to depth 9, a leaf width of $1/512$ around 0.7. Uniform simpson needs $n = 512$, 513 evaluations, to enter the same tolerance: $n = 256$ misses at 5.45e-6. The two numbers of the section: 153 adaptive evaluations against 513 uniform ones, and a subdivision census that puts 34 of 38 leaves where the integrand actually is.

```
static double asimp(integrand f, double a, double b, double fa, double fm,
                    double fb, double whole, double tol, int depth,
                    struct adaptive *st) {
  double m = 0.5 * (a + b);
  double lm = 0.5 * (a + m);
  double rm = 0.5 * (m + b);
  double flm = f(lm);
  double frm = f(rm);
  st->evals += 2;
  double left = (m - a) / 6.0 * (fa + 4.0 * flm + fm);
  double right = (b - m) / 6.0 * (fm + 4.0 * frm + fb);
  double s2 = left + right;
  if (depth ≥ 24 || fabs(s2 - whole) ≤ 15.0 * tol) {
    double w = b - a;
    if (st->leaves < 64)
      trace[st->leaves] = (struct leaf){a, b, depth};
    st->leaves++;
    if (w < st->narrowest)
      st->narrowest = w;
    if (w > st->widest)
      st->widest = w;
    if (depth > st->max_depth)
      st->max_depth = depth;
    return s2 + (s2 - whole) / 15.0;
  }
  return asimp(f, a, m, fa, flm, fm, left, 0.5 * tol, depth + 1, st) +
         asimp(f, m, b, fm, frm, fb, right, 0.5 * tol, depth + 1, st);
}
```

listing 235: quadrature.c, the adaptive recursion, two fresh evaluations per call and the 15-fold error estimate that decides acceptance (source: math/samples/src/Ch22/quadrature.c, lines 86-113)

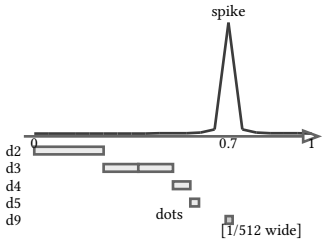


figure 133: the spike fixture true to scale with its leaf intervals stacked by depth below, the census clustering under 0.7, the depth 9 bar enlarged to stay visible

NOTE: TOLERANCE HONORED IS NOT TOLERANCE GUARANTEED

The $|S_2 - S|/15$ estimate is a heuristic about smooth integrands, not a bound. Both fixtures here honor their tolerance with room, errors 6.3e-10 and 3.4e-8 against a requested $10^{-}6$, but a proof of error needs interval arithmetic or a priori bounds on high derivatives. Chapter book 9, ch 36 (practical data structures and algorithms, numerical methods) pins its stopping rules the same way: measured, not assumed.

22.3. euler and the stability question

Quadrature discretizes space, an initial value problem discretizes time. The state $y' = f(t, y)$, $y(0) = y_0$ steps by explicit euler $y_{n+1} = y_n + hf(t_n, y_n)$, one line of code, and everything wrong with numerical simulation starts here.

For the linear decay $y' = -25y$, one step multiplies the state by the amplification factor $1 + h\lambda$ with $\lambda = -25$: the fixed point of this chapter. The step decays exactly when $|1 + h\lambda| < 1$, which for real negative λ is $h < 2/|\lambda| = 0.08$. The boundary is not asymptotic folklore, it is exact, and the sample pins both sides of it.

The dry run: at $h = 0.081$ the factor is $1 - 25 \cdot 0.081 = -1.025$, and three steps give $-1.025, +1.0506, -1.0769$, oscillating and growing. At $h = 0.08$ the factor is exactly -1.0 in binary floating point: the orbit is $(-1)^n$, pinned at $y_{10} = 1.0$ exactly, marginal, neither decaying nor growing. At $h = 0.079$ the factor is -0.975 and $y_{10} = 0.7763$.

h	factor	y_{10} from $y_0 = 1$	exact at $t = 10h$
0.079	-0.975	0.7763	2.65e-9
0.080	-1.0	1.0	2.06e-9
0.081	-1.025	1.2801	1.61e-9

Stiffness is what happens when the interesting timescale and the stability timescale disagree. Take the 2x2 system with matrix rows $(-1001, 999)$ and $(999, -1001)$: its eigenvalues are -2 and -2000 (the symmetric formula $(a_{11} + a_{22})/2 \pm (a_{12} + a_{21})/2$ from chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems)), a stiffness ratio of 1000. The slow mode wants h near 0.1 for accuracy, the fast mode demands $h < 0.001$ for explicit stability. At $h = 0.1$ explicit euler's fast-mode factor is -199: after three steps the state is $(-3940299.2, 3940299.8)$, norm 5.57e6 from a start of norm 1, all of it at $t = 0.3$. The two numbers: 5.57 million against a truth of 0.274 per component.

Implicit euler removes the barrier by hiding the new state in a linear system: $y_{n+1} = y_n + h\boldsymbol{A}y_{n+1}$ rearranges to $(\boldsymbol{I} - h\boldsymbol{A})y_{n+1} = y_n$, one 2x2 solve per step by the cramer route of chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems). At $h = 0.1$, ten steps land on $(0.080753, 0.080753)$ with the components bit-for-bit equal. The start $(1, 0)$ is not symmetric, so the equality has a mechanism worth naming: the components are the symmetric mode plus and minus the antisymmetric half-difference, which starts at 0.5 and shrinks by the factor $1/201$ per implicit step, the fast eigenvalue again. By step 8 it sits below half an ulp of the values it is added to, and after that the two recurrences round to the same double every step. The true slow-mode value at $t = 1$ is 0.0677: implicit euler is stable at any h for decaying systems, A-stable in Dahlquist's sense (Hairer

and Wanner, Solving Ordinary Differential Equations II, Springer 1996 2nd rev ed, by name), but still first order, and its error here is 1.3e-2.

```
constexpr struct vec2 A_COL0 = {-1001.0, 999.0}; // first column of A
constexpr struct vec2 A_COL1 = {999.0, -1001.0}; // second column of A

static struct vec2 rhs(struct vec2 v) {
  return (struct vec2){A_COL0.x * v.x + A_COL1.x * v.y,
                       A_COL0.y * v.x + A_COL1.y * v.y};
}

// implicit euler step: solve (I - hA) u = v for u by cramer, 2x2
static struct vec2 implicit_step(struct vec2 v, double h) {
  double m11 = 1.0 - h * A_COL0.x, m12 = -h * A_COL1.x;
  double m21 = -h * A_COL0.y, m22 = 1.0 - h * A_COL1.y;
  double det = m11 * m22 - m12 * m21;
  return (struct vec2){(m22 * v.x - m12 * v.y) / det,
                       (m11 * v.y - m21 * v.x) / det};
}
```

listing 236: euler.c, the stiff right-hand side and the implicit step as one cramer solve of a 2x2 system (source: math/samples/src/Ch22/euler.c, lines 52-68)

```
  // scalar decay: h* = 2/25 = 0.08 is the boundary
  double y = decay(0.079, 10);
  CHECK(fabs(y - 0.7763296208564385) < 1e-12 && y < 1.0,
        "h = 0.079: factor -0.975, y10 = 0.7763 decays");
  y = decay(0.08, 10);
  CHECK(fabs(y - 1.0) < 1e-15, "h = 0.08: factor -1 exactly, y10 = 1");
  y = decay(0.081, 10);
  CHECK(fabs(y - 1.2800845441963566) < 1e-12,
        "h = 0.081: factor -1.025, y10 = 1.2800 grows");
  CHECK(fabs(exp(-25.0 * 0.81) - 1.605228e-09) < 1e-15 &&
            y / exp(-25.0 * 0.81) > 1e8,
        "truth at t = 0.81 is 1.6e-9, euler is 8e8 times it");

  // eigenvalues of the symmetric matrix: (a11+a22)/2 +- (a12+a21)/2
  double mid = (A_COL0.x + A_COL1.y) / 2.0;
  double half = (A_COL1.x + A_COL0.y) / 2.0;
  CHECK(mid + half == -2.0 && mid - half == -2000.0,
        "eigenvalues -2 and -2000, stiffness ratio 1000");
```

listing 237: euler.c, the boundary pinned from both sides, 0.079 decays, 0.08 sits exactly on it, 0.081 grows (source: math/samples/src/Ch22/euler.c, lines 72-90)

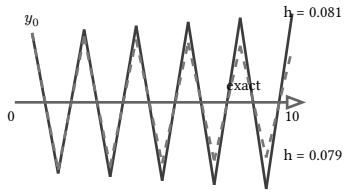


figure 134: ten euler steps on the decay fixture, h = 0.081 grows to 1.28 while h = 0.079 decays toward the exact solution, flat zero on this scale

PITFALL: STABLE IS NOT ACCURATE

> Implicit euler at $h = 0.1$ never blows up on the stiff system and still finishes with error 1.3e-2, twenty times the size of the answer it is tracking. Stability says the error stops growing, accuracy says how big it was allowed to be in the first place. A method earns both numbers separately, and only the pair means anything.

22.4. runge-kutta 4 and the fixed step

Explicit euler pays one derivative evaluation per step and buys one order of accuracy. Classic runge-kutta 4 pays four evaluations and buys four orders: sample the field at the step start, twice at the midpoint with staged corrections, and at the end, then weight the four slopes $1, 2, 2, 1$ over $h/6$. The order conditions behind those weights (twelve equations in the couplings of stages, Hairer, Nørsett and Wanner, Solving Ordinary Differential Equations I, Springer 1993 2nd rev ed, by name) are what make the h^4 ladder below appear without any tuning.

The fixture is the logistic equation $y' = y(1 - y)$ from $y_0 = 0.5$, whose exact solution $y(t) = 1/(1 + e^{-t})$ gives $y(1) = 0.7310585786300049$.

The dry run: one step of $h = 0.5$. The four slopes from $(0, 0.5)$ are $k_1 = 0.25$, then $k_2 = 0.5625 \cdot 0.4375 = 0.24609375$, then $k_3 = 0.246215$ off the k_2 midpoint, then $k_4 = 0.234845$ at the trial end. The step lands at 0.6224551, and the second step finishes at 0.7310474, error 1.12e-5: two steps, eight evaluations, already carrying four correct digits.

h	steps	evals	error	ratio
0.5	2	8	1.117e-5	
0.25	4	16	7.180e-7	$15.6x$
0.125	8	32	4.512e-8	$15.9x$
0.0625	16	64	2.823e-9	$16.0x$

The cost play settles the efficiency question: explicit euler at $h = 1/16$ spends 16 evaluations and ends with error 1.47e-3, rk4 at $h = 0.5$ spends 8 evaluations and ends with error 1.12e-5, 131 times closer on half the budget. Higher order is not decoration, it is the cheaper ticket once accuracy matters at all.

Fixed-step determinism is the property the physics capstone in chapter book 2, ch 30 (mathematics for programmers, capstone: the arena game, design) is built on, and rk4 with a literal h has it: the same binary stepping the same state runs the same instructions on the same operands, and the sample verifies a replay is bit identical. The bookkeeping around the loop matters too: adding h into a running t a thousand times at $h = 0.1$ ends at 99.9999999999986, while computing $t = ih$ rounds once per lookup and lands on exactly 100.0. One rounding beats a thousand compounded ones.

```
static struct ode rk4_step(struct ode s, double h, field f) {
  double k1 = f(s.t, s.y);
  double k2 = f(s.t + 0.5 * h, s.y + 0.5 * h * k1);
  double k3 = f(s.t + 0.5 * h, s.y + 0.5 * h * k2);
  double k4 = f(s.t + h, s.y + h * k3);
  return (struct ode){s.t + h, s.y + h / 6.0 * (k1 + 2.0 * k2 + 2.0 * k3 + k4)};
}

static struct ode euler_step(struct ode s, double h, field f) {
  return (struct ode){s.t + h, s.y + h * f(s.t, s.y)};
}

// rk4 stability polynomial on z = h*lambda
static double rk4_poly(double z) {
  return 1.0 + z + z * z / 2.0 + z * z * z / 6.0 + z * z * z * z / 24.0;
}
```

listing 238: rk4.c, the two steppers side by side, four staged slopes against one (source: math/samples/src/Ch22/rk4.c, lines 46-61)

```
  // cost play: same budget class, euler 16 evals versus rk4 8 evals
  struct ode s = {.t = 0.0, .y = 0.5};
  for (int i = 0; i < 16; i++)
    s = euler_step(s, 1.0 / 16.0, logistic);
  double e_euler = fabs(s.y - exact);
  s = (struct ode){.t = 0.0, .y = 0.5};
  for (int i = 0; i < 2; i++)
    s = rk4_step(s, 0.5, logistic);
  double e_rk4 = fabs(s.y - exact);
  CHECK(fabs(e_euler - 1.469578e-3) < 1e-4, "euler 16 evals: error 1.47e-3");
  CHECK(fabs(e_rk4 - 1.117086e-5) < 1e-7, "rk4 8 evals: error 1.12e-5");
  CHECK(e_rk4 * 100.0 < e_euler, "rk4 is 131x closer with half the evals");

  // fixed-step determinism: same input, same compiled code, identical bits
  struct ode a = {.t = 0.0, .y = 0.5};
  struct ode b = {.t = 0.0, .y = 0.5};
  for (int i = 0; i < 16; i++) {
    a = rk4_step(a, 0.0625, logistic);
    b = rk4_step(b, 0.0625, logistic);
  }
  CHECK(a.y == b.y, "fixed-step replay is bit identical");

  // time bookkeeping: accumulate h and the sum drifts, multiply i*h and it
  // lands exactly on 100.0 here because one rounding replaces a thousand
  double t = 0.0;
  for (int i = 0; i < 1000; i++)
    t += 0.1;
  CHECK(fabs(t - 99.9999999999986) < 1e-9, "t += h a thousand times drifts");
  CHECK(1000 * 0.1 == 100.0, "t = i*h lands on exactly 100.0");
```

listing 239: rk4.c, the cost play, the bit-identical replay, and the time bookkeeping comparison (source: math/samples/src/Ch22/rk4.c, lines 96-124)

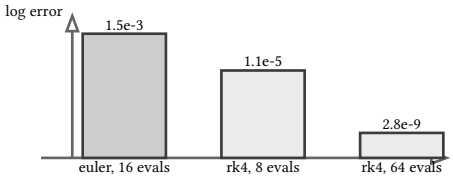


figure 135: integration error against evaluation budget on the logistic fixture, log scale heights

VERIFY: THE DETERMINISM CLAIM, PRECISELY

The replay check is equality, not a tolerance: run the same 16 steps twice into separate state variables and the doubles compare equal. What is claimed is replay determinism of one compiled binary on one machine, which is exactly what a fixed-timestep game loop and its recorded demos need. It is not a claim that another compiler or another evaluation order reproduces the bits.

22.5. stability regions

The $h < 2/|\lambda|$ boundary of the euler section generalizes to the whole complex plane. Apply a method to $y' = \lambda y$ with $\lambda = a + bi$: one step multiplies the state by an amplification $R(h\lambda)$, a polynomial in $h\lambda$, and the absolute stability region is the set where $|R(h\lambda)| < 1$. For explicit euler, $R(z) = 1 + z$, the closed description is the disk $|1 + z| < 1$ of radius 1 centered at -1. For rk4, $R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24$, and its region reaches $z = -2.78529356$ on the real axis where euler stops at -2, and $2.83i$ on the imaginary axis where euler touches nothing at all: $|1 + ih| = \sqrt{1 + h^2} > 1$ for every $h > 0$, so explicit euler cannot hold a pure oscillation at any step size. That single fact is the doorway to the symplectic section.

The dry run: three probe points settle membership by hand. At $z = -1$, the center of euler's disk, $|1 + z| = 0$, as deep inside as the plane gets, and rk4's polynomial reads $R(-1) = 0.375$. At $z = -2.5$, euler's amplification is $|1 - 2.5| = 1.5$, outside, while $R(-2.5) = 0.648$ stays in. Straight up the imaginary axis at $z = 2.5i$, euler sits at $|1 + 2.5i| = 2.69$, far outside, and rk4 at 0.508 is still inside: the imaginary reach in one column.

The sample counts cells instead of drawing curves: a census over the grid of half-integer multiples covering $[-4, 2] \times [-3, 3]$, 625 points, evaluating both amplifications in cartesian form. Euler holds 45 cells, rk4 holds 209, 4.6 times the cells on this grid and about 4 times by monte carlo area, with a 1.39 times longer real-axis reach. Against a single fast eigenvalue that reprieve is small: for $\lambda = -2000$ explicit euler must run $h < 0.001$ and rk4 $h < 0.00139$. The thousandfold gap in the stiff system is stability against accuracy, it binds both explicit methods equally, and only the implicit step of the previous section removes it.

The region is a linear idea applied to nonlinear problems: what enters the plane is $h\lambda$ for each eigenvalue of the local jacobian, the object chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems) computes for symmetric systems. A solver chooses its step so every eigenvalue times h lands inside its method's region, and stiffness is the diagnosis that this constraint, not accuracy, sets the step.

```
  // stability: rk4 stays inside the unit circle out to z = -2.78529356 on
  // the real axis, euler only to -2
  double r2785 = rk4_poly(-2.785);
  double r279 = rk4_poly(-2.79);
  CHECK(fabs(r2785 - 0.9995574896093751) < 1e-12 && r2785 < 1.0,
        "R(-2.785) = 0.99956 inside");
  CHECK(fabs(r279 - 1.0071190337500004) < 1e-12 && r279 > 1.0 &&
            rk4_poly(-2.9) > 1.0,
        "R(-2.79) = 1.0071 and R(-2.9) = 1.187 outside");

  // census: |1+z| < 1 is a disk of radius 1 centered at -1; the rk4 region
  // reaches past -2.78 on the real axis and swallows the imaginary axis
  // near 0, where euler loses every point
  int cnt_euler = 0, cnt_rk4 = 0;
  for (int kx = 0; kx < GRID_NX; kx++) {
    double x = -4.0 + 0.25 * kx;
    for (int ky = 0; ky < GRID_NY; ky++) {
      double y = -3.0 + 0.25 * ky;
      if (hypot(1.0 + x, y) < 1.0) // |1 + z| for z = h*lambda
        cnt_euler++;
      // R(z) in cartesian form, z = x + iy
      double rr = 1.0 + x + (x * x - y * y) / 2.0 +
                  (x * x * x - 3.0 * x * y * y) / 6.0 +
                  (x * x * x * x - 6.0 * x * x * y * y + y * y * y * y) / 24.0;
      double ri = y + x * y + (x * x * y / 2.0 - y * y * y / 6.0) +
                  (x * x * x * y / 6.0 - x * y * y * y / 6.0);
      if (hypot(rr, ri) < 1.0)
        cnt_rk4++;
    }
  }
  CHECK(cnt_euler == 45, "euler region: 45 of 625 grid points");
  CHECK(cnt_rk4 == 209, "rk4 region: 209 of 625 grid points");
  CHECK(cnt_rk4 > 4.5 * cnt_euler, "rk4 covers 4.6x the euler region");
```

listing 240: rk4.c, the polynomial boundary checks and the 625-point census of both regions in cartesian arithmetic (source: math/samples/src/Ch22/rk4.c, lines 126-158)

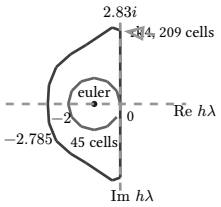


figure 136: the absolute stability regions on the h lambda plane at equal scale, euler a disk of radius 1 inside the rk4 region reaching past minus 2.78

22.6. symplectic integrators and the capstone core

Euler drifts on oscillations because its amplification leaves the unit circle everywhere on the imaginary axis. The fix is not a smaller step, it is a step that respects the geometry. Newton's second law is a hamiltonian system: the flow in (q, p) phase space preserves area, and an integrator that preserves a discrete version of that area, a symplectic map, cannot spiral outward or inward, it can only wobble.

The leapfrog velocity verlet step for $q' = p$, $p' = -V'(q)$ is a half kick, a full drift, a half kick:

$$p_{n+1/2} = p_n - h/2V'(q_n), \quad q_{n+1} = q_n + hp_{n+1/2}, \quad p_{n+1} = p_{n+1/2} - h/2V'(q_{n+1}).$$

The dry run: the harmonic oscillator $V(q) = q^2/2$ from $(q, p) = (1, 0)$ with $h = 0.05$. The half kick gives $p_{1/2} = -0.025$, the drift gives $q_1 = 1 + 0.05 \cdot (-0.025) = 0.99875$, the closing half kick gives $p_1 = -0.0499688$. The energy $E = (q^2 + p^2)/2$ after one step is 0.4999992: off by 7.8e-7, and it stays off by at most $3.125 \times 10^{-}4 = E_0 h^2/4$ for a thousand steps, oscillating, never drifting. Explicit euler on the same fixture ends those thousand steps at $E = 6.072$, a $12.14x$ blowup that equals $0.5(1 + h^2)^{1000}$ exactly, because its step matrix $[[1, h], [-h, 1]]$ is a rotation scaled by $\sqrt{1 + h^2}$ every single step. The two numbers of the chapter: 6.072 against 0.49998 for the same fixture, same step count, same arithmetic.

Why the bound instead of a drift: the verlet map for the oscillator is exactly a rotation by $\theta = 2 \arcsin(h/2)$. The sample CHECKs it: $q_{1000} = 0.9663198469605475$ matches $\cos(1000\theta)$ to 1.8e-15, and p_{1000} matches $-\sin(1000\theta)\sin(\theta)/h$ to 1e-15. The orbit stays on the circle forever, only the angular speed is slightly wrong: $\theta - h = h^3/24$ per step, a phase drift of 5.21e-3 radians over the thousand steps. Bounded energy, drifting phase, that is the honest trade.

Time reversibility comes free: flip p, run the same steps, flip p again, and the round trip of 500 plus 500 steps lands on $q = 1.0000000000000016$, $p = -8.7 \times 10^{-}16$, machine precision. The step is its own inverse, which is what a recorded demo replay needs.

This is the capstone contract. The physics game of chapter book 2, ch 30 (mathematics for programmers, capstone: the arena game, design) runs a fixed h, integrates positions from velocities with verlet, evaluates forces once per step between the half kicks, and leans on three behaviors this chapter pinned: the energy wobble held inside $3.125 \times 10^{-}4 = E_0 h^2/4$ across the thousand-step oscillator run instead of drifting as euler does, every replay of the same inputs is bit identical, and the scheme costs one force evaluation per step, euler's price with none of euler's failure modes. What the wobble does across a full game session, and under contact forces stacking boxes, is for the capstone's own verification chapter to measure, not for this oscillator fixture to claim.

```c
// explicit euler: both updates read the old state, energy multiplies by
// 1 + h^2 every step because the step matrix [[1,h],[-h,1]] is sqrt(1+h^2)
// times a rotation
static struct phase euler_step(struct phase s) {
  return (struct phase){s.q + H * s.p, s.p - H * s.q};
}

// velocity verlet: a half kick, a full drift, a half kick
static struct phase verlet_step(struct phase s) {
  double ph = s.p - 0.5 * H * s.q;
  double q = s.q + H * ph;
  return (struct phase){q, ph - 0.5 * H * q};
}

static double energy(struct phase s) { return 0.5 * (s.q * s.q + s.p * s.p); }

[[nodiscard]] static struct phase run(struct phase s, int n) {
  for (int i = 0; i < n; i++)
    s = verlet_step(s);
  return s;
}
```

listing 241: symplectic.c, the euler step that rotates and scales, and the half kick, drift, half kick of verlet (source: math/samples/src/Ch22/symplectic.c, lines 42-62)

```
  s = (struct phase){1.0, 0.0};
  double worst = 0.0;
  for (int i = 0; i < NSTEPS; i++) {
    s = verlet_step(s);
    double dev = fabs(energy(s) - 0.5);
    if (dev > worst)
      worst = dev;
  }
  CHECK(fabs(energy(s) - 0.49997930438957383) < 1e-12,
        "verlet 1000 steps: E ends at 0.49998");
  CHECK(fabs(worst - 3.125e-4) < 1e-8 && worst < 1e-3,
        "verlet energy oscillates within 3.125e-4 = 0.5*h^2/4, never drifts");
  CHECK(worst * 19000.0 <
            energy((struct phase){3.3218727490534437, 1.0534266293433114}),
        "verlet's worst deviation is 19000x smaller than euler's drift");

  // the orbit is a rotation by theta = 2*asin(h/2): position stays on the
  // circle, only the angle advances at h + h^3/24 instead of h
  double theta = 2.0 * asin(0.5 * H);
  CHECK(fabs(s.q - cos(NSTEPS * theta)) < 1e-14,
        "q_1000 matches cos(1000*theta) to machine precision");
  CHECK(fabs(s.p - (-sin(NSTEPS * theta) * sin(theta) / H)) < 1e-14,
        "p_1000 matches the rotation formula too");
  CHECK(fabs((NSTEPS * (theta - H)) - 5.209799e-3) < 1e-8,
        "the price: phase drifted 5.21e-3 rad over the run");

  // time reversibility: verlet run backward undoes the forward run exactly
  s = (struct phase){1.0, 0.0};
  s = run(s, NSTEPS / 2);
  s = (struct phase){s.q, -s.p};
  s = run(s, NSTEPS / 2);
  s = (struct phase){s.q, -s.p};
  CHECK(fabs(s.q - 1.0) < 1e-12, "round trip: q back to 1 within 1e-12");
  CHECK(fabs(s.p) < 1e-12, "round trip: p back to 0 within 1e-12");
```

listing 242: symplectic.c, the energy bound, the rotation formulas, and the round trip (source: math/samples/src/Ch22/symplectic.c, lines 78-111)

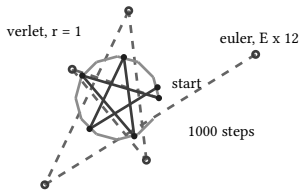


figure 137: the harmonic oscillator in phase space, euler spiraling out while verlet rides the circle

The next chapter book 2, ch 23 (mathematics for programmers, iterative methods and conditioning) leaves continuous time for the fixed-point and iterative schemes that converge to answers rather than step past them.

sources: DLMF §3.5 Quadrature, trapezoidal rules 3.5(i) and simpson's rule 3.5(ii), https://dlmf.nist.gov/3.5, and DLMF §3.7 Ordinary Differential Equations, runge-kutta method 3.7(v), https://dlmf.nist.gov/3.7, both fetched 2026-09-22. Hairer, Nørsett and Wanner, Solving Ordinary Differential Equations I: Nonstiff Problems, Springer 1993 2nd rev ed (order conditions, absolute stability, by name). Hairer and Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, Springer 1996 2nd rev ed (stiffness, A-stability after Dahlquist 1963, by name). LeVeque, Finite Difference Methods for Ordinary and Partial Differential Equations, SIAM 2007 (stability regions per method, the modified-equation view of leapfrog, by name). Higham, An Introduction to Numerical Analysis, Springer 2002 2nd ed (quadrature error constants, by name). Expected sample values computed by the playground one-off playground/math-ch22/pin.py, python 3.14.7 double arithmetic mirroring

the C expression order, run 2026-09-22. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch22`, 58 checks in chapter 22 of the math suite, 4 files, zero failures.

23. iterative methods and conditioning

Chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems) solved linear systems by elimination, chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) factored matrices into eigenvalues and singular values, and chapter book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) defined the condition number that prices the whole business. This chapter is the iterative counterpart of all three: methods that never factor anything, just multiply by the matrix and converge. Six moves: the operation-count argument for why iteration beats elimination on sparse systems, jacobi and gauss-seidel as splittings $\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$, the spectral radius criterion that decides convergence before you run it, power iteration with deflation and its equal-magnitude failure mode, condition numbers measured with the chapter's own iterations plus the perturbation bound observed flipping a solution end to end, and iterative refinement pinned exactly as it behaves in doubles. Every behavioral claim below is one of the 68 checks in the 4 samples of chapter 23, a theorem attributed by name to a canonical text, or the fixed-point iteration form of mml-book draft 2024-01-15 ch 2.3.4 eq 2.60, $x^{(k+1)} = \boldsymbol{C}x^{(k)} + d$, cross verified against the pdf 2026-09-22. Chapter book 2, ch 20 (mathematics for programmers, root finding) already ran fixed-point iteration in one variable; the dsa book's book 9, ch 36 (practical data structures and algorithms, numerical methods) chapter drives such solvers as contest tooling, while this chapter builds the linear algebra underneath.

23.1. why iterate

Chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems) priced gaussian elimination at about $(2/3)n^3$ flops and chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) needed the same order for the eigenvalue machinery. That price buys a factorization, and a factorization is dense knowledge: it answers for every right-hand side at once. When the matrix is sparse and you need one answer, the matrix-vector product $\boldsymbol{A}\boldsymbol{x}$ costs one flop pair per stored nonzero, and a method that only ever multiplies can win by four orders of magnitude.

The fixture is the 5-point laplacian on an $m \times m$ interior grid, the discretized $-u''$ of chapter book 2, ch 22 (mathematics for programmers, quadrature and differential equations) in two dimensions: $n = m^2$ unknowns, diagonal 4, off-diagonal −1 for the four grid neighbors. At $m = 20$, $n = 400$: the stencil form of the product computes $4x_i$ then subtracts each present neighbor, 1920 counted flops, and stores 1920 nonzeros. Assemble the same matrix densely and run LU with partial pivoting: 42586600 counted flops and 15638 stored nonzeros in the factors. The two-number play: one elimination equals 22180 matrix-vector sweeps, and elimination stores 8.14 times what the stencil stores, because the zeros between the stencil diagonals fill in as elimination proceeds. The band of fill is what sparse direct solvers fight and what iterative ones refuse to create.

The dry run: the $m = 3$ grid by hand. With $\boldsymbol{x} =$ ones every node keeps 4 minus its neighbor count, so the corners (2 neighbors) hold 2, the edges (3 neighbors) hold 1, the center holds 0, and the product reads $(2, 1, 2, 1, 0, 1, 2, 1, 2)$, the pinned first check of the sample. At $m = 20$ the same sweep on $\boldsymbol{b} = \boldsymbol{A}$ ones leaves exactly 76 nonzeros, 4 corners at 2 and 72 edge nodes at 1, and the counted flop total is $n + 2 \cdot 2m(m-1) = 1920$.

quantity	stencil	dense LU	ratio
flops	1920	42586600	22180.5
stored nonzeros	1920	15638	8.14

```
// one stencil sweep y = A x straight off the grid, counting every multiply,
// add, and subtract that touches a double. 4 * x costs one multiply, each
// present neighbor costs one subtract.
static void stencil_matvec(int m, const double *x, double *y) {
  for (int i = 0; i < m; i++) {
    for (int j = 0; j < m; j++) {
      int r = i * m + j;
      double v = 4.0 * x[r];
      ops++;
      if (i > 0) {
        v -= x[r - m];
        ops++;
      }
      if (i < m - 1) {
        v -= x[r + m];
        ops++;
      }
      if (j > 0) {
        v -= x[r - 1];
        ops++;
      }
      if (j < m - 1) {
        v -= x[r + 1];
        ops++;
      }
      y[r] = v;
    }
  }
}
```

listing 243: stencil.c, the counted 5-point sweep, one multiply on the diagonal and one subtract per present neighbor (source: math/samples/src/Ch23/stencil.c, lines 56-84)

```
  static double lu[N][N];
  static int piv[N];
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      lu[i][j] = A[i][j];
  int swaps = 0;
  ops = 0;
  lu_factor(lu, piv);
  for (int k = 0; k < N; k++)
    if (piv[k] != k)
      swaps++;
  CHECK(swaps == 0, "stencil: partial pivoting never swaps on this fixture");
  CHECK(ops == 42586600, "stencil: dense LU counts 42586600 flops");

  int fill = 0;
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      if (lu[i][j] != 0.0)
        fill++;
  CHECK(fill == 15638 &&
            fabs((double)fill / (double)nnz - 8.144791666666666) < 1e-9,
        "stencil: LU factors store 15638 nonzeros, 8.14x the sparse storage");

  ops = 0;
  lu_solve(lu, piv, b, x);
  double err = 0.0;
  for (int i = 0; i < N; i++)
    if (fabs(x[i] - 1.0) > err)
      err = fabs(x[i] - 1.0);
  CHECK(err ≤ 3.4e-15, "stencil: LU solves A*ones back to ones, err 3.3e-15");

  double ratio = 42586600.0 / 1920.0;
  CHECK(fabs(ratio - 22180.520833333332) < 1e-6,
        "stencil: one elimination buys 22180 matvec sweeps");
  double textbook = (2.0 / 3.0) * (double)N * (double)N * (double)N;
  CHECK(fabs(42586600.0 / textbook - 1.0) ≤ 0.01,
        "stencil: the flop count tracks (2/3)n^3 within 1 percent");
```

listing 244: stencil.c, the measurements: zero swaps, the 42586600 flop count, the 15638-entry fill, and the solve that returns ones (source: math/samples/src/Ch23/stencil.c, lines 187-223)

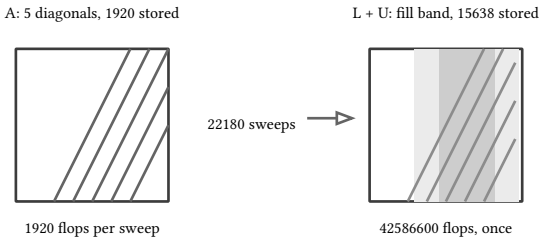


figure 138: the sparsity of the stencil against the fill band of its LU factors

VERIFY: COUNTED VERSUS TEXTBOOK

The sample's counter counts the arithmetic this code performs, one op per multiply, add, subtract, or division on entries. The textbook $(2/3)n^3 = 42666667$ sits 0.2 percent above the count 42586600 because the leading term ignores the lower-order n^2 corrections the counter does see. Both numbers are checks, not estimates.

23.2. splittings: jacobi and gauss-seidel

Write $\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$ with $\boldsymbol{M}$ easy to invert, start from any x_0, and iterate $\boldsymbol{M}x^{(k+1)} = \boldsymbol{N}x^{(k)} + b$. That is the mml-book eq 2.60 form $x^{(k+1)} = \boldsymbol{C}x^{(k)} + d$ with $\boldsymbol{C} = \boldsymbol{M}^{-1}\boldsymbol{N}$, the iteration matrix. Jacobi takes $\boldsymbol{M} = \boldsymbol{D}$, the diagonal: every component of the new sweep is one row of arithmetic on values that are all old. Gauss-seidel takes $\boldsymbol{M} = \boldsymbol{D} + \boldsymbol{L}$, lower triangle included: component i is updated from components $1, \ldots, i-1$ that are already new. Same flops per sweep, different information flow.

The model fixture is the 1d laplacian $\operatorname{tridiag}(-1, 2, -1)$ with $n = 5$ and $b = (1, 0, 0, 0, 1)$, chosen so the solution is $x^* = \text{ones}$: the first and last rows of $\boldsymbol{A}$ ones give 1, the interior rows give 0. The jacobi sweep is $x_i^{(k+1)} = \left(b_i + x_{i-1}^{(k)} + x_{i+1}^{(k)}\right)/2$.

The dry run: from $x_0 = 0$, the first sweep copies $b/2$, giving $(0.5, 0, 0, 0, 0.5)$; the second pulls neighbors, giving $(0.5, 0.25, 0, 0.25, 0.5)$; the third gives $(0.625, 0.25, 0.25, 0.25, 0.625)$; six sweeps reach $(0.71875, 0.578125, 0.4375, 0.578125, 0.71875)$, every entry dyadic, every step exact in doubles. Gauss-seidel's first sweep feeds on its own output, $(0.5, 0.25, 0.125, 0.0625, 0.53125)$, a different ladder from the same arithmetic. The jacobi error column stalls in pairs, 1, 1, 0.75, 0.75, 0.5625, 0.5625, a plateau pattern the next section explains, while the gauss-seidel column declines every sweep. Jacobi needs 99 sweeps to push $\|x^* - x^{(k)}\|_{\text{inf}}$ under 10^{-6}, gauss-seidel needs 50. The two-sweep factor of jacobi and the one-sweep factor of gauss-seidel are the same number, 0.75, pinned both ways: for this model young's identity $\rho(\boldsymbol{M}_{\text{GS}}) = \rho(\boldsymbol{M}_J)^2$ holds exactly (the matrix is consistently ordered, Young 1971 by name), so one gauss-seidel sweep buys what two jacobi sweeps buy.

sweep	jacobi $x^{(k)}$	gauss-seidel $x^{(k)}$
1	(0.5, 0, 0, 0, 0.5)	(0.5, 0.25, 0.125, 0.0625, 0.53125)
2	(0.5, 0.25, 0, 0.25, 0.5)	(0.625, 0.375, 0.21875, 0.375, 0.6875)
3	(0.625, 0.25, 0.25, 0.25, 0.625)	(0.6875, 0.453125, 0.4140625, 0.55078125, 0.775390625)
6	(0.71875, 0.578125, 0.4375, 0.578125, 0.71875)	(0.8364..., 0.7534..., 0.7528..., 0.8145..., 0.9072...)

```
// model problem jacobi sweep: x_new_i = (b_i + x_{i-1} + x_{i+1}) / 2
static void jacobi_model(const double b[N5], const double x[N5],
                         double xn[N5]) {
  for (int i = 0; i < N5; i++) {
    double s = b[i];
    if (i > 0)
      s += x[i - 1];
    if (i < N5 - 1)
      s += x[i + 1];
    xn[i] = s / 2.0;
  }
}

// gauss-seidel uses the fresh components within the same sweep
static void gs_model(const double b[N5], double x[N5]) {
  for (int i = 0; i < N5; i++) {
    double s = b[i];
    if (i > 0)
      s += x[i - 1];
    if (i < N5 - 1)
      s += x[i + 1];
    x[i] = s / 2.0;
  }
}
```

listing 245: jacobi.c, the two model sweeps, one reading only old values, one updating in place (source: math/samples/src/Ch23/jacobi.c, lines 48-72)

```
double xj[N5] = {0, 0, 0, 0, 0};
for (int k = 0; k < 6; k++) {
  jacobi_model(bm, xj, xn);
  for (int i = 0; i < N5; i++)
    xj[i] = xn[i];
}
// pinned ladder x6 = (0.71875, 0.578125, 0.4375, 0.578125, 0.71875)
CHECK(fabs(xj[0] - 0.71875) < 1e-30 && fabs(xj[2] - 0.4375) < 1e-30 &&
          fabs(xj[3] - 0.578125) < 1e-30,
      "model: jacobi x6 lands on the pinned dyadic ladder");
double xg1[N5] = {0, 0, 0, 0, 0};
gs_model(bm, xg1);
CHECK(fabs(xg1[1] - 0.25) < 1e-30 && fabs(xg1[3] - 0.0625) < 1e-30 &&
          fabs(xg1[4] - 0.53125) < 1e-30,
      "model: GS x1 is (0.5, 0.25, 0.125, 0.0625, 0.53125)");
for (int k = 1; k < 3; k++)
  gs_model(bm, xg1);
CHECK(
    fabs(xg1[1] - 0.453125) < 1e-30 && fabs(xg1[2] - 0.4140625) < 1e-30 &&
        fabs(xg1[4] - 0.775390625) < 1e-30,
    "model: GS x3 is (0.6875, 0.453125, 0.4140625, 0.55078125, 0.775390625)");

for (int i = 0; i < N5; i++)
  x[i] = 0.0;
int itj = 0;
for (;;) {
  jacobi_model(bm, x, xn);
  for (int i = 0; i < N5; i++)
    x[i] = xn[i];
  itj++;
  if (errinf(N5, x, xs) ≤ 1e-6 || itj > 10000)
    break;
}
CHECK(itj == 99, "model: jacobi needs 99 sweeps to 1e-6");

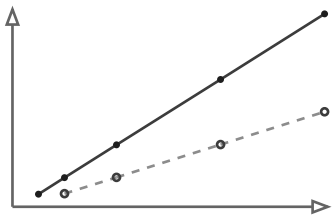
for (int i = 0; i < N5; i++)
  x[i] = 0.0;
int itg = 0;
for (;;) {
  gs_model(bm, x);
  itg++;
  if (errinf(N5, x, xs) ≤ 1e-6 || itg > 10000)
    break;
}
CHECK(itg == 50, "model: gauss-seidel needs 50 sweeps, half of 99");
```

listing 246: jacobi.c, the pinned dyadic ladders for both splittings and the two sweep counts, 99 against 50 (source: math/samples/src/Ch23/jacobi.c, lines 187-231)

Divergence is one fixture away. The 2 by 2 system $\left(\begin{smallmatrix}1 & 2\\ 3 & 1\end{smallmatrix}\right)x = (1,1)$ has solution $(0.2, 0.4)$, but row 0 carries off-diagonal mass 2 against diagonal 1, and both splittings blow up: jacobi's iterates are the integers $(1,1), (-1,-2), (5,4), (-7,-14), (29,22), (-43,-86), (173,130)$, gaining the factor 6 per two sweeps because its iteration matrix has eigenvalues $\pm\sqrt{6}$, while gauss-seidel's iterates $(1,-2), (5,-14), (29,-86), (173,-518)$ gain exactly 6 every sweep on its single eigenvalue 6. Nothing oscillates by accident here: the eigenvalues predict both rates, and both ladders are pinned checks.

```
// divergence fixture: no diagonal dominance, both splittings blow up
constexpr double A2[4] = {1, 2, 3, 1};
constexpr double b2[2] = {1, 1};
const double xs2[2] = {0.2, 0.4};
double off0 = fabs(A2[1]);
CHECK(off0 == 2.0 && fabs(A2[0]) < off0,
      "diverge: row 0 off-sum 2 beats diagonal 1, no dominance");
double d[2], dg[2];
d[0] = d[1] = dg[0] = dg[1] = 0.0;
double ladders[7][2];
for (int k = 0; k < 7; k++) {
  jacobi_dense(2, A2, b2, d, ladders[k]);
  d[0] = ladders[k][0];
  d[1] = ladders[k][1];
}
double n5v = dist2(2, ladders[4], xs2);
CHECK(n5v == 36.0 && ladders[4][0] == 29.0 && ladders[4][1] == 22.0,
      "diverge: jacobi x5 is (29, 22) with ||e||2 exactly 36");
CHECK(fabs(dist2(2, ladders[6], xs2) / dist2(2, ladders[4], xs2) - 6.0) <
          1e-12,
      "diverge: jacobi gains the factor 6 per TWO sweeps, eigenvalues +/- "
      "sqrt(6)");
for (int k = 0; k < 6; k++) {
  gs_dense(2, A2, b2, dg);
  ladders[k][0] = dg[0];
  ladders[k][1] = dg[1];
}
CHECK(ladders[4][0] == 1037.0 && ladders[4][1] == -3110.0,
      "diverge: GS x5 is (1037, -3110) exactly");
CHECK(fabs(dist2(2, ladders[4], xs2) / dist2(2, ladders[3], xs2) - 6.0) <
          1e-9,
      "diverge: GS gains the factor 6 every sweep");
```

listing 247: jacobi.c, the divergence fixture, dominance violated, jacobi at factor 6 per two sweeps and gauss-seidel at 6 per sweep (source: math/samples/src/Ch23/jacobi.c, lines 281-313)



sweeps, $-\log_{10} \|e\|_{\inf}$ on the vertical
gauss-seidel solid dots, 0.125 digits per sweep, jacobi dashed, 0.0625

figure 139: digits of accuracy gained per sweep, gauss-seidel climbing at twice jacobi's slope

23.3. convergence: the spectral radius decides

Subtract the fixed point from the iteration and the affine part cancels: the error $e^{(k)} = x^* - x^{(k)}$ satisfies $e^{(k)} = \boldsymbol{C}^k e^{(0)}$ for whichever iteration matrix $\boldsymbol{C}$ the splitting produced. If $\boldsymbol{C}$ is diagonalizable, $\boldsymbol{C} = \boldsymbol{V}\boldsymbol{D}\boldsymbol{V}^{-1}$, then $\boldsymbol{C}^k = \boldsymbol{V}\boldsymbol{D}^k\boldsymbol{V}^{-1}$, and $\boldsymbol{D}^k$ dies exactly when every $|\lambda_i| < 1$, that is when the spectral radius $\rho(\boldsymbol{C}) = \max|\lambda_i(\boldsymbol{C})|$ is under 1. The jordan form extends the same conclusion to every matrix, the polynomial factors $k^j\rho^k$ still dying when $\rho < 1$ (Varga 1962 by name). So convergence is a property of the splitting, checkable before the first sweep, and the model problem's plateau pattern from the last section is now explained: $\boldsymbol{M}_J$ for the tridiagonal has eigenvalues $\pm\cos(\pi/6), \pm\cos(2\pi/6), 0$, a sign pair at the dominant magnitude $\sqrt{3}/2 = 0.8660254$, and a sign pair alternates

the error direction every sweep, which stalls the inf-norm for one step and drops it by $\rho^2 = 0.75$ over two. Gauss-seidel's iteration matrix has no sign pair, its dominant eigenvalue is exactly $3/4$, and it drops 0.75 every sweep.

How to get ρ without a full eigensolver, chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) style: power iteration on the explicit iteration matrix, next section's tool used early. Building $\boldsymbol{M}_J = \boldsymbol{D}^{-1}(\boldsymbol{L} + \boldsymbol{U})$ is one division per off-diagonal entry; building $\boldsymbol{M}_{\mathrm{GS}} = -(\boldsymbol{D} + \boldsymbol{L})^{-1}\boldsymbol{U}$ column by column is forward substitution, and the pinned result on the model is zero above its superdiagonal, 0.5 on that superdiagonal, halving down each column below it, $(0, .5, 0, 0, 0)$, $(0, .25, .5, 0, 0)$, $(0, .125, .25, .5, 0)$, $(0, .0625, .125, .25, .5)$, $(0, .03125, .0625, .125, .25)$. Power iteration pins 0.75 on it.

The easy sufficient condition is strict diagonal dominance, and its proof is one line: for any induced norm $\rho(\boldsymbol{C}) \le \|\boldsymbol{C}\|$, because $\boldsymbol{C}v = \lambda v$ at the maximal component of v gives $|\lambda| \le \sum_j |\boldsymbol{C}_{\mathrm{ij}}|$, and for jacobi that row sum is exactly the off-diagonal mass over the diagonal. The strict fixture $\mathrm{tridiag}(1, 4, 1)$ with $n = 3$ checks it: the bound is 0.5, the true radius measured by power iteration is $\sqrt{2}/4 = 0.3536$, honest but pessimistic, and two pinned sweeps land $(0.875, 0.875, 0.875)$. The model problem itself is only weakly dominant, $2 = 1 + 1$ every interior row, so $\|\boldsymbol{M}_J\|_{\mathrm{inf}} = 1$ exactly, a bound that proves nothing: the true radius 0.866 lives under 1 only because the tridiagonal has extra structure (irreducibility, Varga 1962 by name). A norm bound sitting at exactly 1 next to a spectrum well inside it is the pinned two-number play of this section.

```
// dominant |eigenvalue| of a small matrix by power iteration, returned two
// ways: the norm ratio ||A x|| (handles +/- eigenvalue pairs) and the
// rayleigh quotient (does not). x starts from ones, 200 fixed sweeps.
static void power_rho(int n, const double *a, double *ratio, double *rq) {
  double x[16], y[16];
  for (int i = 0; i < n; i++)
    x[i] = 1.0;
  double nrm = 0.0;
  for (int i = 0; i < n; i++)
    nrm += x[i] * x[i];
  nrm = sqrt(nrm);
  for (int i = 0; i < n; i++)
    x[i] /= nrm;
  *ratio = 0.0;
  *rq = 0.0;
  for (int k = 0; k < 200; k++) {
    double ny = 0.0, xy = 0.0, xx = 0.0;
    for (int i = 0; i < n; i++) {
      double s = 0.0;
      for (int j = 0; j < n; j++)
        s += a[i * n + j] * x[j];
      y[i] = s;
      ny += s * s;
      xy += x[i] * s;
    }
    for (int i = 0; i < n; i++)
      xx += x[i] * x[i];
    ny = sqrt(ny);
    *ratio = ny;
    *rq = xy / xx;
    for (int i = 0; i < n; i++)
      x[i] = y[i] / ny;
  }
}
```

listing 248: jacobi.c, power iteration returning the norm ratio and the rayleigh quotient side by side (source: math/samples/src/Ch23/jacobi.c, lines 113-147)

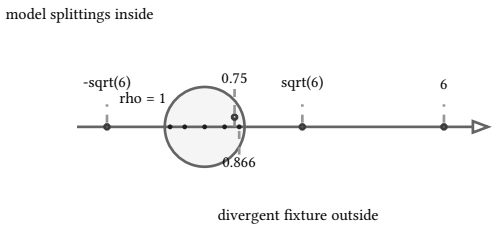


figure 140: eigenvalues of the iteration matrices against the unit circle, inside converges, outside blows up

PITFALL: A BOUND OF EXACTLY 1 IS SILENT

Weak diagonal dominance gives $\|M_J\|_{\inf} = 1$ on the laplacian model, which neither proves nor refutes convergence: norms bound the spectrum from above and the gap can be the whole story, here 1.0 against 0.866. Always measure the radius, or check irreducibility before trusting a weak bound.

23.4. power iteration and deflation

The dominant eigenvalue hides inside one repeated product. Normalize any start vector, multiply, normalize again, and the iterate turns toward the eigenvector of the largest $|\lambda|$ at the rate of the second ratio $|\lambda_2| \,/\, |\lambda_1|$. The rayleigh quotient $(x^T \boldsymbol{A} x)/(x^T x)$ then converges to λ_1 at the squared rate, because the components orthogonal to the winning eigenvector shrink before they are squared by the quotient.

The fixture wears its spectrum on its sleeve: $\boldsymbol{A} = \boldsymbol{R}\ \mathrm{diag}(6,3,1)\boldsymbol{R}^T$ with $\boldsymbol{R}$ the rational rotation carrying $\cos = 3/5, \sin = 4/5$ in the (1,2) plane, so in exact arithmetic

$$\boldsymbol{A} = 1/25 \begin{pmatrix} 102 & 36 & 0 \\ 36 & 123 & 0 \\ 0 & 0 & 25 \end{pmatrix}$$

with eigenvector $(0.6, 0.8, 0)$ for 6. In doubles the entries land within an ulp, $4.08, 1.4399999999999997, 4.920000000000001$, and the mirror pinned every ladder.

The dry run: from $x_0 = \mathrm{ones}$ (normalization is a common factor, so it cancels in the quotient), the first product is $\boldsymbol{A}\ \mathrm{ones} = (4.08 + 1.44, 1.44 + 4.92, 1) = (5.52, 6.36, 1)$ and the first rayleigh quotient is $(5.52 + 6.36 + 1)/3 = 4.293333333333$, the pinned first rung. The ladder then climbs 5.915461624027, 5.994214746109, 5.998989023559, and after 30 sweeps pins 6 one ulp out, $6 + 8.9 \cdot 10^{-16}$, with the iterate $(0.6, 0.8, 0)$ and the error ratios closing on $(3/6)^2 = 0.25$, 0.24998 at the eighth rung: the iterate error loses the factor $1/2$ per sweep and the quotient error its square, so the quotient gains digits exactly twice as fast.

Deflation hands you the second eigenpair from the first: subtract the rank one piece, $\boldsymbol{A}' = \boldsymbol{A} - 6vv^T$, and 6 is gone from the spectrum while everything else stands. Power iteration on $\boldsymbol{A}'$ converges to 3 at rate $1/3$, and the pinned 30-sweep iterate is $(-0.8, 0.6, 0)$, the second rotation column, to $2.1 \cdot 10^{-10}$, the measured worst deviation.

The failure mode is equal magnitudes with opposite signs. Build $\boldsymbol{B} = \boldsymbol{S}\ \mathrm{diag}(6,-6,1)\boldsymbol{S}^T$ with the same rotation moved to the (2,3) plane and run the identical loop: the norm ratio $\|\boldsymbol{B}x\| \,/\, \|x\|$ still converges to 6, magnitudes survive, but the iterate never settles. Its direction flips every step between two mirror states, consecutive iterates lock at $|\cos\angle| = 12/37 = 0.3243$, about 71 degrees apart, and the rayleigh quotient pins at $6(a^2 - b^2)/(a^2 + b^2) = -144/74 = -1.945945945945946$, a number that is not an eigenvalue of anything. The two-number play of the section: the norm ratio says 6, the quotient says −1.946, and only the ratio is telling the truth. The fix in practice is a shift, $\boldsymbol{B} + \mu\boldsymbol{I}$, or the QR iteration of chapter book 2, ch 8 (mathematics for programmers, matrix decompositions).

```
// A = R diag(d) R^T accumulated in exactly the order the mirror used:
// a_ij = sum_k R[i][k] d[k] R[j][k], k ascending
static void build_sym(const double r[3][3], const double d[3], double a[3][3]) {
  for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) {
      double acc = 0.0;
      for (int k = 0; k < 3; k++)
        acc += r[i][k] * d[k] * r[j][k];
      a[i][j] = acc;
    }
}

// one power run: normalize x0, then iters steps of y = A x, rayleigh
// quotient x.y / x.x and norm ratio ||y||, renormalize. xprev comes back
// holding the second to last iterate.
static void power_run(const double a[3][3], const double x0[3], int iters,
                      double xend[3], double *rq_end, double *nrm_end,
                      double xprev[3]) {
  double x[3], y[3];
  double n = norm3(x0);
  for (int i = 0; i < 3; i++)
    x[i] = x0[i] / n;
  double rq = 0.0, nrm = 0.0;
  for (int k = 0; k < iters; k++) {
    matvec3(a, x, y);
    rq = dot3(x, y) / dot3(x, x);
    nrm = norm3(y);
    for (int i = 0; i < 3; i++) {
      xprev[i] = x[i];
      x[i] = y[i] / nrm;
    }
  }
  for (int i = 0; i < 3; i++)
    xend[i] = x[i];
  *rq_end = rq;
  *nrm_end = nrm;
}
```

listing 249: power.c, the symmetric builder in mirror order and the power loop carrying quotient, norm ratio, and the previous iterate (source: math/samples/src/Ch23/power.c, lines 60-96)

```
    double d2[3] = {6, -6, 1};
    build_sym(S, d2, B);
    CHECK(fabs(B[0][0] - 6.0) < 1e-13 &&
              fabs(B[1][1] + 1.5199999999999996) < 1e-13 &&
              fabs(B[1][2] + 3.36) < 1e-13 &&
              fabs(B[2][2] + 3.480000000000001) < 1e-13,
          "B: S diag(6,-6,1) S^T lands on the pinned doubles");
    double xf[3], xfp[3], rqf, nrmf;
    power_run(B, ones3, 42, xf, &rqf, &nrmf, xfp);
    CHECK(fabs(rqf + 144.0 / 74.0) < 1e-12,
          "fail: the rayleigh quotient pins at -144/74 = -1.9459, not any "
          "eigenvalue");
    CHECK(fabs(nrmf - 6.0) < 1e-9,
          "fail: the norm ratio still says 6, the magnitude is right");
    double cosang = fabs(dot3(xfp, xf));
    CHECK(fabs(cosang - 12.0 / 37.0) < 1e-9,
          "fail: consecutive iterates lock at |cos| = 12/37, the direction flips "
          "forever");
```

listing 250: power.c, the equal-magnitude fixture, quotient at −144/74, norm ratio at 6, iterate flip at cos 12/37 (source: math/samples/src/Ch23/power.c, lines 152-169)

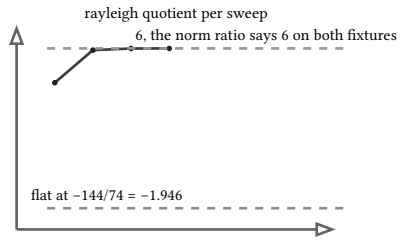


figure 141: the rayleigh quotient per sweep, one ladder climbing to 6, the equal-magnitude fixture flat at −1.946

23.5. condition numbers

Chapter book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) defined conditioning as the worst case amplification of relative input error, and chapter book 2, ch 8 (mathematics for programmers, matrix decompositions) identified the 2-norm condition number as $\kappa_2 = \sigma_{\max}/\sigma_{\min}$, the singular value ratio. This chapter can measure both ends with its own machinery: power iteration for $\sigma_{\max}$ on a symmetric positive definite matrix, and inverse iteration, which is power iteration on $\boldsymbol{A}^{-1}$ through a hand LU, for $\sigma_{\min}$. No eigensolver is pulled in from outside.

The first fixture is $\boldsymbol{K} = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}$. Its exact spectrum is available in closed form (mpmath 40 digits in the pin script), and the measured pair agrees: $\lambda_{\max} = 2.00005000125$, $\lambda_{\min} = 4.999875 \cdot 10^{-5}$, $\kappa_2 = 40002.00007500125$, the measured value matching to 9 digits. The perturbation bound $\|\delta x\|/\|x\| \le \kappa_2 (\|\delta b\|/\|b\|)$ is then observed, not just quoted (Trefethen and Bau 1997 by name for the bound): take $\boldsymbol{x} = (1, 1)$, $\boldsymbol{b} = \boldsymbol{K}\boldsymbol{x} = (2, 2.0001)$, and nudge the second component of b down by 10^{-4}.

The dry run, by hand in exact arithmetic: $\boldsymbol{K}^{-1} = 10^4 \begin{pmatrix} 1.0001 & -1 \\ -1 & 1 \end{pmatrix}$ because the determinant is 10^{-4}, so $\boldsymbol{K}^{-1}(0, -10^{-4}) = 10^4(10^{-4}, -10^{-4}) = (1, -1)$ and the solution moves to $(1, 1) + (1, -1) = (2, 0)$. The computed solution is $(2.0000000000022204, -2.2 \cdot 10^{-12})$. A relative data change of $3.535 \cdot 10^{-5}$, four digits of b, moved the answer by a relative 1.0: the flip is total, the amplification is 28285, and the bound reads $40002 \cdot 3.535 \cdot 10^{-5} = 1.414 \ge 1$: satisfied, with room of about $\sqrt{2}$, because the perturbation did not align with the worst singular direction.

The residual is not the error. Solve the 10 by 10 hilbert system $\boldsymbol{H}\boldsymbol{x} = \boldsymbol{H}$ ones (entries $1/(i + j + 1)$, the least squares ghost of chapter book 2, ch 21 (mathematics for programmers, interpolation and approximation)) with the

same LU: the computed z misses ones by $3.2 \cdot 10^{-4}$ while its residual $\|\boldsymbol{H}z - \boldsymbol{H}\ \text{ones}\|_{\text{inf}}$ is $1.8 \cdot 10^{-16}$, rounding noise. The residual measures backward error, the solve is stable, and the forward error is the residual amplified by the conditioning: measured $\kappa_2 = 1.60 \cdot 10^{13}$, so the expected error scale is $\kappa_2 \cdot 2^{-52} = 3.6 \cdot 10^{-3}$, and $3.2 \cdot 10^{-4}$ sits comfortably inside. That is the whole story of "my solver lost 13 digits": it did not, the problem never had them.

```
// lambda_min by inverse iteration: power iteration on A^-1 through the LU,
// mu is the rayleigh quotient of A^-1 at the final normalized iterate
static double inverse_lambda(int n, const double *lu, const int *piv,
                             int iters) {
  double x[16], y[16];
  for (int i = 0; i < n; i++)
    x[i] = 1.0;
  double nrm = 0.0;
  for (int i = 0; i < n; i++)
    nrm += x[i] * x[i];
  nrm = sqrt(nrm);
  for (int i = 0; i < n; i++)
    x[i] /= nrm;
  for (int k = 0; k < iters; k++) {
    lu_solve(n, lu, piv, x, y);
    double ny = 0.0;
    for (int i = 0; i < n; i++)
      ny += y[i] * y[i];
    ny = sqrt(ny);
    for (int i = 0; i < n; i++)
      x[i] = y[i] / ny;
  }
  lu_solve(n, lu, piv, x, y);
  double mu = 0.0;
  for (int i = 0; i < n; i++)
    mu += x[i] * y[i];
  return 1.0 / mu;
}
```

listing 251: condition.c, inverse iteration, power iteration on the LU inverse, one extra solve for the eigenvalue (source: math/samples/src/Ch23/condition.c, lines 134-161)

```
// the perturbation flip: db = (0, -1e-4) turns x = (1,1) into (2, 0)
double bk[2] = {K1[0] * 1.0 + K1[1] * 1.0, K1[2] * 1.0 + K1[3] * 1.0};
double bp[2] = {bk[0] + 0.0, bk[1] - 1e-4};
double xp[2];
lu_solve(2, klu, kpiv, bp, xp);
CHECK(fabs(xp[0] - 2.0000000000022204) < 1e-12 &&
          fabs(xp[1] + 2.2204460492505574e-12) < 1e-24,
      "K1: a 1e-4 nudge on b flips the solution to (2, 0)");
double reldx =
    sqrt((xp[0] - 1.0) * (xp[0] - 1.0) + (xp[1] - 1.0) * (xp[1] - 1.0));
reldx /= sqrt(2.0);
double reldb = 1e-4 / sqrt(bk[0] * bk[0] + bk[1] * bk[1]);
CHECK(reldx ≥ 1.0 && fabs(reldx - 1.0000000000022204) < 1e-9,
      "K1: relative solution change is 1, a total flip");
CHECK(fabs(reldb - 3.535445518689971e-05) < 1e-16,
      "K1: relative data change is 3.535e-05");
double amp = reldx / reldb;
CHECK(fabs(amp / 28284.97836314451 - 1.0) < 1e-6 && amp ≤ kappa1,
      "K1: amplification 28285 stays under kappa 40002, the bound holds");
CHECK(fabs(kappa1 * reldb - 1.4142489190381462) < 1e-9 &&
          kappa1 * reldb ≥ reldx,
      "K1: the bound value kappa |db|/|b| = 1.414 covers the flip");
```

listing 252: condition.c, the perturbation flip, relative changes, amplification 28285 under kappa 40002, bound value 1.414 (source: math/samples/src/Ch23/condition.c, lines 180-201)

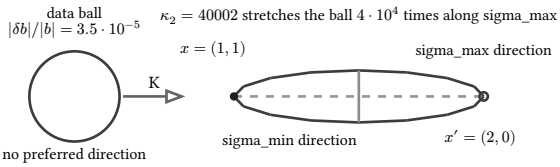


figure 142: the input ball mapped through the matrix to a kappa-stretched ellipse, the flip as one endpoint

23.6. iterative refinement

Elimination gives a backward stable z with a tiny residual. One refinement step computes the residual $r = b - \boldsymbol{A}z$ in the same doubles, solves $\boldsymbol{A}d = r$ with the factors already in hand, and adds d. If z were exact the correction would be zero, and to the extent the residual is honest, $z + d$ is a better solve. The method is two sentences; the interesting part is when it stops helping.

The dry run: on $\boldsymbol{K}$ the residual of the plain solve rounds to exactly $(0, 0)$, three refinement steps change nothing, and the error freezes at $3.1 \cdot 10^{-12}$. On the hilbert system one step buys the 5.5x gain from $3.2 \cdot 10^{-4}$ to $5.8 \cdot 10^{-5}$, then the ladder wanders inside the $3.6 \cdot 10^{-3}$ band.

On $\boldsymbol{K}$ it stops immediately, and the sample pins why: the residual of the plain solve rounds to exactly $(0, 0)$, both components, so three refinement steps are literally a no-op, the error frozen at $3.1 \cdot 10^{-12}$. Nothing is wrong with the solver. The stored $\boldsymbol{K}$ and b differ from the real numbers by input rounding at 10^{-16} scale. One estimate of the damage is $\|\boldsymbol{K}^{-1}\|$ times that rounding, about $4.4 \cdot 10^{-12}$; the check states the ceiling as the separate quantity $\kappa \cdot 2^{-52} = 8.9 \cdot 10^{-12}$, and the frozen $3.1 \cdot 10^{-12}$ error sits under both. The error was never in the solve to begin with, so there is nothing for refinement to retrieve.

On the hilbert system the ladder is livelier and pinned rung by rung: $3.2 \cdot 10^{-4}$, one step to $5.8 \cdot 10^{-5}$, a 5.5x gain, then $3.2 \cdot 10^{-4}$, $9.8 \cdot 10^{-4}$, $9.5 \cdot 10^{-4}$, $8.4 \cdot 10^{-4}$: the second correction hands back the first one's winnings and the ladder wanders inside the $\kappa \cdot 2^{-52} = 3.6 \cdot 10^{-3}$ band without ever leaving it. The pin script's higher-precision audit explains the wandering: the exact solution of the stored system is itself $5 \cdot 10^{-4}$ away from ones, because b was rounded on the way in, so the error being measured is dominated by input rounding, and each correction

injects fresh solve rounding at the same scale it removes. Refinement converges reliably when the residual is computed more accurately than the solve, the classical extended-precision recipe (Higham 2002 by name); in plain doubles the honest statement is the band, and both ladders here are checks, not anecdotes.

```
  double err[6];
  err[0] = err0;
  for (int step = 1; step ≤ 5; step++) {
    double r[NH], d[NH];
    for (int i = 0; i < NH; i++) {
      double acc = bh[i];
      for (int j = 0; j < NH; j++)
        acc -= H[i][j] * zh[j];
      r[i] = acc;
    }
    lu_solve(NH, &lu[0][0], piv, r, d);
    for (int i = 0; i < NH; i++)
      zh[i] += d[i];
    err[step] = 0.0;
    for (int i = 0; i < NH; i++)
      if (fabs(zh[i] - 1.0) > err[step])
        err[step] = fabs(zh[i] - 1.0);
  }
  CHECK(fabs(err[1] - 5.77907703200875e-05) < 1e-16 && err[1] ≤ 0.2 * err[0],
        "H10: one refinement step buys 5.5x, down to 5.78e-5");
  CHECK(fabs(err[2] - 3.15567827569474e-4) < 1e-12 &&
            fabs(err[3] - 9.849631460236985e-4) < 1e-12,
        "H10: rungs 2 and 3 are the pinned 3.16e-4 and 9.85e-4");
  CHECK(fabs(err[4] - 9.466153893811224e-4) < 1e-12 &&
            fabs(err[5] - 8.419422338834526e-4) < 1e-12,
        "H10: rungs 4 and 5 are the pinned 9.47e-4 and 8.42e-4");
  CHECK(err[2] > err[1] && err[5] > err[1],
        "H10: then the ladder wanders back up, corrections are noise");
  double worst = 0.0;
  for (int step = 0; step < 6; step++)
    if (err[step] > worst)
      worst = err[step];
  CHECK(worst < 2e-3 && worst ≤ kappaH * DBL_EPSILON,
        "H10: every rung stays inside the kappa eps band 3.6e-3");
```

listing 253: condition.c, the hilbert ladder, one 5.5x gain, then wandering inside the kappa band, every rung a check (source: math/samples/src/Ch23/condition.c, lines 274-307)

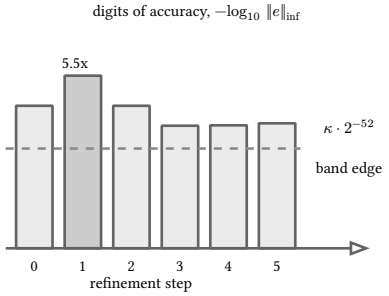


figure 143: the refinement ladder in digits of accuracy, one gain then wandering above the band edge

WARNING: SMALL RESIDUAL, LARGE ERROR

A residual at rounding level certifies the solve, never the answer. On the hilbert fixture the residual is $1.8 \cdot 10^{-16}$ while the error is $3.2 \cdot 10^{-4}$, and refinement cannot cross that gap because the gap belongs to the

> problem, not the algorithm. Chapter book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) made the distinction for subtraction; here it is the whole discipline of judging a linear solve.

The next chapter book 2, ch 24 (mathematics for programmers, strategic games) turns from converging on answers to converging on opponents: the fixed points become equilibrium strategies, and the iteration counts become rounds of play.

sources: mml-book draft 2024-01-15, ch 2.3.4 “algorithms for solving a system of linear equations”, printed pp 34-35 (pdf pp 40-41), the fixed-point iteration form eq 2.60 and the iterative alternatives named there (richardson, jacobi, gauss-seidel, sor, krylov cg/gmres/bicg), cross verified against ref/mml-book.pdf on 2026-09-22. Trefethen and Bau, Numerical Linear Algebra, SIAM 1997 (conditioning and perturbation bounds, eigenvalue iterations, by name; siam.org and epubs.siam.org pages returned cloudflare challenges on 2026-09-22). Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed, SIAM 2002, ISBN 0-89871-521-0 (iterative refinement, extended-precision residuals, by name), https://nhigham.com/accuracy-and-stability-of-numerical-algorithms/, fetched 2026-09-22. Varga, Matrix Iterative Analysis, Prentice-Hall 1962 (spectral radius criterion, irreducible weak dominance, by name). Young, Iterative Solution of Large Linear Systems, Academic Press 1971 (rho_GS = rho_J^2 for consistently ordered matrices, by name). Expected sample values computed by the playground one-off playground/math-ch23/pin.py, python 3.14.7 double arithmetic mirroring the C expression order with numpy and mpmath 40-digit audits, run 2026-09-22. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch23`, 68 checks in chapter 23 of the math suite, 4 files, zero failures.

24. strategic games

Chapter book 2, ch 18 (mathematics for programmers, probability and distributions) built expectation over outcomes and chapter book 2, ch 11 (mathematics for programmers, continuous optimization) built duality for constrained extremes. This chapter spends both on games where players move simultaneously and payoffs interlock. Six moves: the normal form with payoff bimatrixes and dominance as pure enumeration, best responses and pure nash equilibria as mutual best responses, mixed strategies with every expectation carried in exact rationals, the minimax theorem for zero sum games and its link back to linear program duality, computing equilibria by enumerating supports and solving each with cramer, and the regret view where follow the regularized leader turns online play into equilibria. Every behavioral claim below is one of the 60 checks in the 4 samples of chapter 24 or a sentence quoted from a canonical source fetched 2026-09-22. Chapter book 9, ch 40 (practical data structures and algorithms, game theory and scheduling) in the dsa book owns combinatorial games with perfect information and sprague-grundy numbers, this chapter owns the simultaneous strategic half, and chapter book 2, ch 25 (mathematics for programmers, sequential and repeated games) extends it to game trees and repetition.

24.1. normal form and dominance

A game in normal form is a list of players, a finite action set for each, and a payoff for every profile of actions. Two players means two payoff matrices of the same shape, the bimatrix: row payoff $u_1(i, j)$ and column payoff $u_2(i, j)$ for row action i against column action j. The prisoner's dilemma fixture uses the standard numbers with cooperate and defect:

	cooperate	**defect**
cooperate	$(3, 3)$	$(0, 4)$
defect	$(4, 0)$	$(1, 1)$

Action d strictly dominates c when $u(d, j) > u(c, j)$ against every opponent action j. Here defect pays 4 against 3 and 1 against 0, both players at once, so two rounds of strict elimination leave the profile (defect, defect) paying (1, 1) while (cooperate, cooperate) pays (3, 3) sits one eliminated row away.

The dry run: dominance is four integer comparisons, and the never best response property is a sweep. Scale the opponent mix q (probability of cooperate) to eighths: $u(C, q) = 3q$ against $u(D, q) = 1 + 3q$. The difference is exactly 1 at all 9 points of the grid, the 8 intervals from $q = 0$ to $q = 1$, at $q = 1/2$ that is 3/2 against 5/2. A strictly dominated action loses by a constant margin no matter what the opponent does, so it is never a best response, and eliminating it cannot delete an equilibrium. Weak domination drops the strictness: $u(d, j) \geq u(c, j)$ everywhere with a strict inequality somewhere. The fixture $[[2, 1], [2, 0]]$ has the first row weakly dominating the second, yet against column 1 both rows pay 2, so the weakly dominated row is still a best response there.

```c
// does action d strictly dominate action c for the player whose payoffs are u
// (flat n x n, row per own action, column per opponent pure action)?
static bool strictly_dominates(const int *u, int n, int d, int c) {
  for (int j = 0; j < n; j++)
    if (u[d * n + j] ≤ u[c * n + j])
      return false;
  return true;
}

// column player's version: compare down the rows instead of across
static bool strictly_dominates_col(const int *u, int n, int d, int c) {
  for (int i = 0; i < n; i++)
    if (u[i * n + d] ≤ u[i * n + c])
      return false;
  return true;
}

// does action d weakly dominate c (≥ everywhere, > somewhere)?
static bool weakly_dominates(const int *u, int n, int d, int c) {
  bool somewhere = false;
  for (int j = 0; j < n; j++) {
    if (u[d * n + j] < u[c * n + j])
      return false;
    if (u[d * n + j] > u[c * n + j])
      somewhere = true;
  }
  return somewhere;
```

listing 254: normal.c, strict dominance for both players and weak dominance, each one plain enumeration (source: math/samples/src/Ch24/normal.c, lines 37-64)

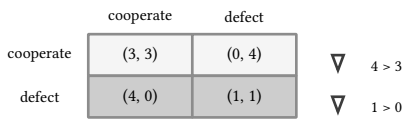


figure 144: the prisoner's dilemma bimatrix, the defect row shaded as the strict dominant reply in both columns

PITFALL: WEAK DOMINATION IS ORDER DEPENDENT

Iterated elimination by strict dominance is safe: a strictly dominated action is never a best response against any mix, which normal.c sweeps on all 9 points of the eighths grid. Weak elimination is not: eliminating a weakly dominated action can delete equilibria, and which equilibria survive depends on the order of elimination (Osborne 2003, by name). The weak fixture shows the mechanism, the covered row still ties as a best response against column 1, so removing it removes a best reply that some equilibrium may need.

24.2. best response and pure nash

The best response correspondence picks, for each fixed opponent action, the actions attaining the maximum payoff. Matching pennies fixes the two diagonal cells of the row matrix at 1 and the off diagonal at −1: row wants to match, so the best response to heads is heads and to tails is tails, while the column player wants the mismatch. A pure nash equilibrium is a profile where every action played is a best response to the other side, the mutual fixed point of the two correspondences. Formally s^* is an equilibrium when $u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$ for every player i and every deviation s_i.

The dry run: enumerate all four profiles and count deviators. Pennies has none stable, at (H, H) the column player collects −1 and flips, at (H, T) the row player flips, 0 pure equilibria. The prisoner's dilemma has exactly one, (D,

D): both best response maps point there from every column and every row. Battle of the sexes with row payoff $[[3,0],[0,1]]$ and column payoff $[[1,0],[0,3]]$ has two, (opera, opera) paying (3, 1) and (football, football) paying (1, 3), with the miscoordinated profiles paying (0, 0) to both.

fixture	pure equilibria	payoffs	deviators elsewhere
pennies	0		both players want to flip
prisoner's dilemma	1	$(1,1)$	defection pays
battle of the sexes	2	$(3,1)$, $(1,3)$	both miscoordinates fail

```c
// pure nash equilibria: profiles where each side's action is a best response
// to the other's; fills eq with (row, col) pairs, returns the count
static int pure_nash(const int *a, const int *b, int n, int eq[][2]) {
  int count = 0;
  bool brr[4], brc[4];
  for (int i = 0; i < n; i++) {
    // column player's best responses against pure row i read down column i
    int best = b[i * n + 0];
    for (int j = 1; j < n; j++)
      if (b[i * n + j] > best)
        best = b[i * n + j];
    for (int j = 0; j < n; j++)
      brc[j] = b[i * n + j] == best;
    for (int j = 0; j < n; j++) {
      br_rows(a, n, j, brr);
      if (brr[i] && brc[j]) {
        eq[count][0] = i;
        eq[count][1] = j;
        count++;
      }
    }
  }
  return count;
```

listing 255: normal.c, pure nash enumeration: flag the column side's best replies down each row, then keep profiles where both sides are flagged (source: math/samples/src/Ch24/normal.c, lines 78-101)

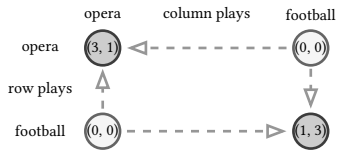


figure 145: battle of the sexes deviation arrows, both miscoordinate profiles have a profitable unilateral move and drain into the two shaded equilibria

NOTE: EQUILIBRIUM IS NOT OPTIMALITY

The prisoner's dilemma equilibrium pays (1, 1) while (cooperate, cooperate) pays (3, 3), and the check pins exactly this: one pure equilibrium, at (defect, defect), with strict dominance making cooperation unreachable without changing the game. Equilibrium predicts what self interested play sustains, not what the players would prefer to have agreed on.

24.3. mixed strategies and nash

Matching pennies has no pure equilibrium, so the strategy space has to grow. A mixed strategy is a probability vector over actions, and the payoff of a mixed profile is the expectation $u_1(x,y) = \sum_i \sum_j x_i y_j u_1(i,j)$, exactly the

double sum chapter book 2, ch 18 (mathematics for programmers, probability and distributions) used for discrete expectations. Against a mixed opponent the payoff of each pure action is a dot product, and the best response set is the argmax of those dot products. Against a battle of the sexes column player mixing (1/2, 1/2), opera expects $3 \cdot 1/2 = 3/2$ and football expects $1 \cdot 1/2 = 1/2$, so opera is the strict best response.

The dry run: solve the pennies equilibrium by indifference. Row is willing to mix only when both actions pay the same, $u(H, q) = u(T, q)$, which expands to $2q - 1 = 1 - 2q$ on the mixing probability q of heads and lands at $q = 1/2$. Both players equalize, the profile (1/2, 1/2) against (1/2, 1/2) pays exactly 0 to both, and no deviation gains: the checks pin 0 with exact fractions, not floats. The same two equations on battle of the sexes give row mixing (3/4, 1/4) against column mixing (1/4, 3/4), and the payoff is 3/4 to each player, below the 1 the disadvantaged side collects at either pure equilibrium. Nash's theorem guarantees this construction always terminates: "John Nash showed that there is a Nash equilibrium, possibly in mixed strategies, for every finite game" (wikipedia, fetched 2026-09-22, the 1950 result via kakutani and 1951 via brouwer fixed points).

```
// expected payoff of row action i (pure) against col mix y
[[nodiscard]] static frac u1_pure(int i, const int *a, int n, const frac *y) {
  frac s = {0, 1};
  for (int j = 0; j < n; j++)
    s = fr_add(s, fr_mul(y[j], fr(a[i * n + j], 1)));
  return s;
}

// expected payoff of the mixed profile (x, y) for the row side
[[nodiscard]] static frac u1_mix(const int *a, int n, const frac *x,
                                 const frac *y) {
  frac s = {0, 1};
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      s = fr_add(s, fr_mul(fr_mul(x[i], y[j]), fr(a[i * n + j], 1)));
  return s;
}

// 2x2 mixed equilibrium by indifference: the column player must be
// indifferent, which fixes the row mix from B; the row player's indifference
// fixes the col mix from A. x1 = (b22 - b12)/denB, y1 = (a22 - a12)/denA.
static void eq2(const int *a, const int *b, frac x[2], frac y[2]) {
  x[0] = fr(b[3] - b[1], b[0] - b[1] - b[2] + b[3]);
  x[1] = fr_add((frac){1, 1}, (frac){-x[0].n, x[0].d});
  y[0] = fr(a[3] - a[1], a[0] - a[1] - a[2] + a[3]);
  y[1] = fr_add((frac){1, 1}, (frac){-y[0].n, y[0].d});
}
```

listing 256: mixed.c, mixed expectations as exact fraction sums and the 2x2 equilibrium from the indifference equations, one denominator per player (source: math/samples/src/Ch24/mixed.c, lines 68-94)

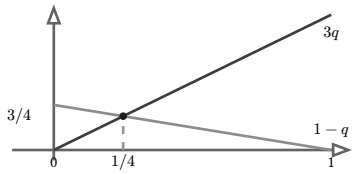


figure 146: battle of the sexes indifference, row's two pure payoffs cross at column mix q = 1/4 where both earn 3/4

Two numbers to keep: the mixed battle of the sexes pays both players 3/4 where the pure equilibria pay 3 and 1, and pennies at (1/2, 1/2) pays 0 where every pure profile pays plus or minus 1.

24.4. the minimax theorem

Zero sum strips the bimatrix to one matrix, $u_2 = -u_1$, all conflict. Row's guaranteed payoff for a mixed x is $\min_j x^T A e_j$, and the maximin is the best such guarantee. Column's cap is $\max_i e_i^T A y$, minimized. Pure enumeration reads both off the matrix: the saddle fixture $[[3,2,4],[1,2,0],[5,1,6]]$ has row minima (2, 0, 1) and column maxima (5, 2, 6), maximin 2 and minimax 2, the saddle at row 1 column 2. The gap fixture $[[2,0],[0,1]]$ has both row minima 0 and both column maxima above it, maximin 0 against minimax 1, and no pure equilibrium: the pure numbers bracket the truth without pinning it.

The dry run: close the gap with a mix. Row plays (p, 1 - p), guaranteeing $\min(2p, 1-p)$, a tent that peaks where the two lines cross, $2p = 1 - p$, so $p = 1/3$ and the guarantee is 2/3. Column's equalization gives the same (1/3, 2/3) from the other side, and the value is exactly 2/3 with both columns paying $x^T A e_j = 2/3$ and both rows paying $Ay = 2/3$. A twelfths grid sweep finds the same peak at k = 4, and weak duality, no row guarantee above any column cap, holds on all 169 enumerated pairs with the tightest margin exactly 0 at the optimum. This is the content of the theorem: "max x in X min y in Y x^T A y = min y in Y max x in X x^T A y" over the strategy simplexes, von Neumann 1928 (wikipedia, fetched 2026-09-22). The gap fixture closes 0 < 2/3 < 1.

```
// pure maximin: best guaranteed row minimum. flat m x n matrix.
static int maximin_pure(const int *a, int m, int n, int *row) {
  int best = a[0], bi = 0;
  for (int i = 0; i < m; i++) {
    int lo = a[i * n];
    for (int j = 1; j < n; j++)
      if (a[i * n + j] < lo)
        lo = a[i * n + j];
    if (i == 0 || lo > best) {
      best = lo;
      bi = i;
    }
  }
  *row = bi;
  return best;
}

// pure minimax: least damaging column maximum
static int minimax_pure(const int *a, int m, int n, int *col) {
  int best = a[0], bj = 0;
  for (int j = 0; j < n; j++) {
    int hi = a[j];
    for (int i = 1; i < m; i++)
      if (a[i * n + j] > hi)
        hi = a[i * n + j];
    if (j == 0 || hi < best) {
      best = hi;
      bj = j;
    }
  }
  *col = bj;
  return best;
}

// mixed 2x2 by equalization: x1 (a11 - a12 - a21 + a22) = a22 - a12
static void zs2(const int *a, frac x[2], frac y[2], frac *v) {
  x[0] = fr(a[3] - a[1], a[0] - a[1] - a[2] + a[3]);
  x[1] = fr_add((frac){1, 1}, (frac){-x[0].n, x[0].d});
  y[0] = fr(a[3] - a[2], a[0] - a[1] - a[2] + a[3]);
  y[1] = fr_add((frac){1, 1}, (frac){-y[0].n, y[0].d});
  *v = fr_add(fr_mul(x[0], fr(a[0], 1)), fr_mul(x[1], fr(a[2], 1)));
}
```

listing 257: zerosum.c, pure maximin and minimax by enumeration, then the 2x2 mixed solve by equalizing one denominator (source: math/samples/src/Ch24/zerosum.c, lines 78-120)

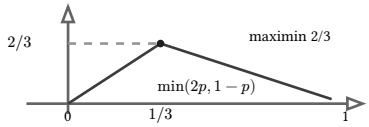


figure 147: the gap game guarantee min(2p, 1 - p), a tent peaking at p = 1/3 with value 2/3 between the pure maximin 0 and minimax 1

The link back to chapter book 2, ch 11 (mathematics for programmers, continuous optimization) is exact: the maximin side is a linear program in x plus a level variable, the minimax side is its dual, and the minimax equality is strong duality for that pair (Boyd and Vandenberghe 2004, by name). Every check in this chapter is that duality made enumerable: 169 weak pairs, one tight.

24.5. computing equilibria by support enumeration

Indifference turns equilibrium solving into linear algebra. On a support pair (S, T), the sets of actions each player mixes over, the equal size case stacks the equations: each column player action in T must pay row the same v, each row action in S must pay column the same v, and the probabilities sum to 1. That is a $(k+1) \times (k+1)$ linear system per player, solvable by cramer in exact rationals with the determinant machinery of chapter book 2, ch 6 (mathematics for programmers, matrices and linear systems). A candidate survives only if every probability is strictly positive, every column pays at least v against x, and every row pays at most v against y.

The dry run: pennies first, 5 systems. The 4 pure supports solve trivially and die on the inequalities, 2 columns exploited, 2 rows. The full support solves to (1/2, 1/2) with $v = 0$ and survives. Rock paper scissors runs 19 systems: 9 pure, 9 of size 2, 1 full. Three of the size 2 systems are singular, the stacked determinant is exactly 0, and the survivor is the uniform (1/3, 1/3, 1/3) with value 0. The chapter fixture $[[2, -1, 0], [-1, 1, 3], [0, 3, -1]]$ is the stress test: also 19 systems, 3 rejected for negative probabilities, 15 exploitable, and one survivor, both players at exactly (14/31, 9/31, 8/31) with value 19/31, confirmed by direct evaluation of all 6 indifference equations.

fixture	systems	singular	negative	exploitable	survivor
pennies	5	0	0	4	(1/2, 1/2), $v = 0$
rock paper scissors	19	3	0	15	(1/3, 1/3, 1/3), $v = 0$
3x3 fixture	19	0	3	15	(14/31, 9/31, 8/31), $v = 19/31$

```
static enum reject solve_support(const int *a, int m, int ncol, const int *S,
                                 const int *T, int k, frac *x, frac *y,
                                 frac *v) {
  int ns = k + 1;
  frac mat[16];
  // (y, v) system: row i in S: sum_j a[S_i][T_j] y_j - v = 0; sum y = 1
  for (int r = 0; r < k; r++) {
    for (int c = 0; c < k; c++)
      mat[r * ns + c] = fr(a[S[r] * ncol + T[c]], 1);
    mat[r * ns + k] = (frac){-1, 1};
  }
  for (int c = 0; c < k; c++)
    mat[k * ns + c] = (frac){1, 1};
  mat[k * ns + k] = (frac){0, 1};
  frac D = det(mat, ns);
  if (D.n == 0)
    return RK_SINGULAR;
  for (int c = 0; c < k; c++) {
    frac mc[16];
    for (int r = 0; r < ns; r++)
      for (int c2 = 0; c2 < ns; c2++)
        mc[r * ns + c2] = mat[r * ns + c2];
    for (int r = 0; r < ns; r++)
      mc[r * ns + c] = (r == k) ? (frac){1, 1} : (frac){0, 1};
    y[c] = fr_mul(det(mc, ns), (frac){D.d, D.n});
  }
  // (x, v) system: col j in T: sum_i a[S_i][T_j] x_i - v = 0; sum x = 1
  for (int c = 0; c < k; c++) {
    for (int r = 0; r < k; r++)
      mat[c * ns + r] = fr(a[S[r] * ncol + T[c]], 1);
    mat[c * ns + k] = (frac){-1, 1};
  }
  for (int r = 0; r < k; r++)
    mat[k * ns + r] = (frac){1, 1};
  mat[k * ns + k] = (frac){0, 1};
  D = det(mat, ns);
  if (D.n == 0)
    return RK_SINGULAR;
  for (int c = 0; c < k; c++) {
    frac mc[16];
    for (int r = 0; r < ns; r++)
      for (int c2 = 0; c2 < ns; c2++)
        mc[r * ns + c2] = mat[r * ns + c2];
    for (int r = 0; r < ns; r++)
      mc[r * ns + c] = (r == k) ? (frac){1, 1} : (frac){0, 1};
    x[c] = fr_mul(det(mc, ns), (frac){D.d, D.n});
  }
```

listing 258: zerosum.c, one support pair: stack both indifference systems with the probability sums, reject singular determinants, extract by cramer in fractions (source: math/samples/src/Ch24/zerosum.c, lines 175-221)

```
  for (int c = 0; c < k; c++)
    if (x[c].n ≤ 0 || y[c].n ≤ 0)
      return RK_NONPOS;
  frac xf[3] = {{0, 1}, {0, 1}, {0, 1}}, yf[3] = {{0, 1}, {0, 1}, {0, 1}};
  for (int c = 0; c < k; c++) {
    xf[S[c]] = x[c];
    yf[T[c]] = y[c];
  }
  // v is what the equalized support pays: any column in T gives the same
  *v = mix_col(a, m, ncol, xf, T[0]);
  for (int j = 0; j < ncol; j++)
    if (fr_lt(mix_col(a, m, ncol, xf, j), *v))
      return RK_COLEXP;
  for (int i = 0; i < m; i++)
    if (fr_lt(*v, row_mix(a, ncol, i, yf)))
      return RK_ROWEXP;
  for (int i = 0; i < m; i++)
    x[i] = xf[i];
  for (int j = 0; j < ncol; j++)
    y[j] = yf[j];
  return RK_ACCEPT;
}
```

listing 259: zerosum.c, the validation half: positive probabilities, then no column dips below v and no row rises above v (source: math/samples/src/Ch24/zerosum.c, lines 222-243)

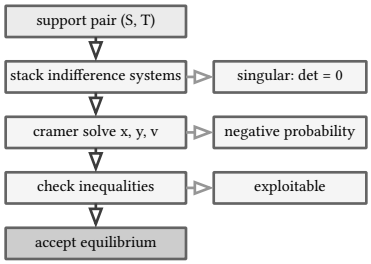


figure 148: support enumeration as a decision flow, each pair solved then filtered by singular, negative, and exploitability gates

PITFALL: TWO BLIND SPOTS OF EQUAL SIZE SUPPORTS

A singular determinant is not an error, it is information: rock paper scissors makes three size 2 supports singular because the equalization degenerates, and the enumeration skips them. The equal size restriction is the other blind spot: degenerate games can hold equilibria whose two supports have different sizes, which this enumeration never visits. The general bimatrix case also drops the shared v, each player equalizes only the opponent's support. Both caveats are standard (Porter, Nudelman, Shoham, Simple Search Methods for Finding a Nash Equilibrium, AAAI 2004, journal version in Games and Economic Behavior 63(2), 2008, by name).

24.6. regret and follow the regularized leader

Equilibria are fixed points, but players arrive at them by playing. Online learning reframes the problem: at each round the learner picks a distribution over actions, the opponent reveals a move, and the loss lands in $[0, 1]$. The cumulative regret is the gap between what the learner paid and what the best single fixed action would have paid, $R_T = \sum_t \ell(p^t, b^t) - \min_i \sum_t \ell(i, b^t)$. Follow the regularized leader with negative entropy collapses to exponential weights: $w_i \propto \exp(-\eta L_i)$ over the cumulative loss L_i, the multiplicative weights update (Cesa-Bianchi and Lugosi 2006, by name). For losses in $[0, 1]$ the regret bound is $\ln(n)/\eta + \eta T/8$.

The dry run: rock paper scissors losses win 0, tie 1/2, lose 1, a fixed opponent stream from a seeded xorshift64, and the learner plays its own distribution so every round is an exact expectation with no sampling noise. The stream pins: the first two outputs 0xdc1b77ae0bf34dad and 0x64f0eeb9026e6076, opening actions 0, 0, 0, 0, 2, 0, 1, 0, 1, 1. After 200 rounds the fixed action losses are (101, 95.5, 103.5), best fixed is paper, the learner paid 103.7574209924925, so the regret is 8.257420992492499, average 0.0413, inside the bound 26.09861228866811 and down from 0.0473 at round 50. The average regret ladder runs 0.1420, 0.0712, 0.0473, 0.0535, 0.0413 at rounds 10, 25, 50, 100, 200: noisy round to round, decaying in trend.

```
  // phase 1: exponential weights against a fixed xorshift-driven opponent
  xs_state = 0x9E3779B97F4A7C15ULL;
  double L[3] = {0, 0, 0}, learn = 0.0, avg1[3] = {0, 0, 0};
  double reg10 = 0.0, reg25 = 0.0, reg50 = 0.0, reg100 = 0.0, reg200 = 0.0,
         p[3] = {0, 0, 0};
  int bseq[10];
  unsigned long long o0 = 0, o1 = 0;
  for (int t = 0; t < T1; t++) {
    unsigned long long o = xs64();
    if (t == 0)
      o0 = o;
    if (t == 1)
      o1 = o;
    int b = (int)(o % 3);
    if (t < 10)
      bseq[t] = b;
    double w[3] = {exp(-ETA * L[0]), exp(-ETA * L[1]), exp(-ETA * L[2])};
    double Z = w[0] + w[1] + w[2];
    p[0] = w[0] / Z;
    p[1] = w[1] / Z;
    p[2] = w[2] / Z;
    avg1[0] += p[0] / T1;
    avg1[1] += p[1] / T1;
    avg1[2] += p[2] / T1;
    learn += p[0] * LOSS[0][b] + p[1] * LOSS[1][b] + p[2] * LOSS[2][b];
    for (int i = 0; i < 3; i++)
      L[i] = L[i] + LOSS[i][b];
    if (t == 9 || t == 24 || t == 49 || t == 99 || t == 199) {
      double best = L[0] < L[1] ? (L[0] < L[2] ? L[0] : L[2])
                                : (L[1] < L[2] ? L[1] : L[2]);
      double r = learn - best;
      if (t == 9)
        reg10 = r;
      else if (t == 24)
        reg25 = r;
      else if (t == 49)
        reg50 = r;
      else if (t == 99)
        reg100 = r;
      else
        reg200 = r;
    }
  }

  // strategy the weights propose for round 201, recomputed from the final L
  double w[3] = {exp(-ETA * L[0]), exp(-ETA * L[1]), exp(-ETA * L[2])};
  double Z = w[0] + w[1] + w[2];
  p[0] = w[0] / Z;
  p[1] = w[1] / Z;
  p[2] = w[2] / Z;
```

listing 260: ftrl.c, the exponential weights loop against the fixed xorshift stream, regret captured at rounds 10, 25, 50, 100, 200 (source: math/samples/src/Ch24/ftrl.c, lines 65-114)

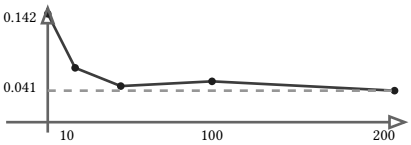


figure 149: average regret of the pinned run, decaying from 0.142 at round 10 to 0.041 at round 200

Why regret is the route to equilibria: in a two player zero sum game, if both players' average regret goes to 0, the time average profile approaches a minimax equilibrium and the average payoff approaches the value. Outside zero sum the guarantee weakens: vanishing regret buys welfare bounds for smooth games, not convergence to equilibrium (the folk theorem of no regret learning, by name). The second phase checks the zero sum half live: two learners with rates 1 and 0.7 self play the chapter's 3x3 fixture for 400 rounds, losses normalized into [0, 1], and their average strategies land at (0.4385, 0.3116, 0.2499) and (0.4271, 0.2965, 0.2764), within 0.025 of the enumerated equilibrium (14/31, 9/31, 8/31) with the average payoff 0.6350 against the value 19/31 = 0.6129. The enumeration from the last section and the learning dynamics in this one agree on the answer.

```
// phase 2: two FTRL players, rates 1 and 0.7, self play on G3 with losses
// normalized into [0,1]: row loses (3 - a)/4, col loses (1 + a)/4
constexpr int T2 = 400;
constexpr double ETA2 = 0.7;
constexpr int G3[3][3] = {{2, -1, 0}, {-1, 1, 3}, {0, 3, -1}};
constexpr double RL[3][3] = {
    {0.25, 1.0, 0.75}, {1.0, 0.5, 0.0}, {0.75, 0.0, 1.0}};
constexpr double CL[3][3] = {
    {0.75, 0.0, 0.25}, {0.0, 0.5, 1.0}, {0.25, 1.0, 0.0}};
double L1[3] = {0, 0, 0}, L2[3] = {0, 0, 0};
double avgp1[3] = {0, 0, 0}, avgp2[3] = {0, 0, 0}, avgpay = 0.0;
for (int t = 0; t < T2; t++) {
  double w1[3] = {exp(-ETA * L1[0]), exp(-ETA * L1[1]), exp(-ETA * L1[2])};
  double Z1 = w1[0] + w1[1] + w1[2];
  double p1[3] = {w1[0] / Z1, w1[1] / Z1, w1[2] / Z1};
  double w2[3] = {exp(-ETA2 * L2[0]), exp(-ETA2 * L2[1]), exp(-ETA2 * L2[2])};
  double Z2 = w2[0] + w2[1] + w2[2];
  double p2[3] = {w2[0] / Z2, w2[1] / Z2, w2[2] / Z2};
  for (int i = 0; i < 3; i++) {
    avgp1[i] += p1[i] / T2;
    avgp2[i] += p2[i] / T2;
  }
  double pr = 0.0;
  for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
      pr += p1[i] * p2[j] * G3[i][j];
  avgpay += pr / T2;
  for (int i = 0; i < 3; i++) {
    double l1 = 0.0, l2 = 0.0;
    for (int j = 0; j < 3; j++) {
      l1 += p2[j] * RL[i][j];
      l2 += p1[j] * CL[j][i];
    }
    L1[i] = L1[i] + l1;
    L2[i] = L2[i] + l2;
  }
}
```

listing 261: ftrl.c, two rate self play on the 3x3 fixture, averages closing on the enumerated equilibrium (source: math/samples/src/Ch24/ftrl.c, lines 152-188)

VERIFY: AVERAGES CONVERGE, TRAJECTORIES DO NOT

> Exponential weights on a cyclic game oscillates forever: by round 200 the phase 1 weights sit at 0.0041, 0.9956, 0.0003, nearly pure paper, and would swing on if the stream demanded it. Only the uniform time average (0.319, 0.341, 0.340) is meaningful, and it is the average the checks pin. This is also why the self play phase pins averages within 0.025 rather than trajectories within an epsilon.

Chapter book 2, ch 25 (mathematics for programmers, sequential and repeated games) moves the players onto trees, where one side observes the other before moving and the analysis tool becomes backward induction instead of the fixed point.

sources: Osborne, An Introduction to Game Theory, Oxford University Press 2003, fixtures and dominance treatment cited by name and edition. The minimax statement quoted from `en.wikipedia.org/wiki/Minimax_theorem` and the nash existence sentence “John Nash showed that there is a Nash equilibrium, possibly in mixed strategies, for every finite game” quoted from `en.wikipedia.org/wiki/Nash_equilibrium`, both fetched 2026-09-22, von Neumann 1928 and Nash 1950 as dated there. The LP duality link by name from Boyd and Vandenberghe, Convex Optimization, Cambridge University Press 2004, chapter 5, joined to the duality material of chapter optimization. The regret bound form by name from Cesa-Bianchi and Lugosi, Prediction, Learning, and Games, Cambridge University Press 2006, chapter 2, and Hazan, Introduction to Online Convex Optimization, 2nd edition, MIT Press, September 2022, chapter 5, the wikipedia FTRL and multiplicative weights pages returned 404 on 2026-09-22 so no url is cited for them. Support enumeration practice by name from Porter, Nudelman, Shoham, Simple Search Methods for Finding a Nash Equilibrium, AAAI 2004, journal version Games and Economic Behavior 63(2), 2008. No mml mapping exists for game theory, so this chapter cites the references above instead of mml page ranges. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch24`, 60 checks in chapter 24 of the math suite, zero failures, format and asan legs clean.

25. sequential and repeated games

Chapter book 2, ch 24 (mathematics for programmers, strategic games) fixed what players choose when they choose at once: one matrix, one mixed strategy each, one equilibrium test. This chapter fixes what happens when they choose in order, and then when they choose again and again. Six moves: the extensive-form tree as the honest data structure for ordered moves, backward induction as the solver that reads payoffs off the leaves, subgame perfection as the refinement that discards threats nobody would carry out, information sets and behavioral strategies for games where moves stay hidden, the discounted infinitely repeated prisoner's dilemma where cooperation becomes an equilibrium, and the second-price auction where telling the truth is a dominant strategy. Every behavioral claim below is one of the 59 checks in the 4 samples of chapter 25 or a sentence paraphrased from a canonical source fetched 2026-09-22. The book 9, ch 40 (practical data structures and algorithms, game theory and scheduling) chapter runs the search algorithms over huge game trees, this chapter is the theory of what those searches compute.

25.1. the tree comes first

A game in extensive form is a specification that makes four things explicit: the sequencing of the players' possible moves, the choices available at every decision point, the information each player has when deciding, and the payoffs for all outcomes. The normal form of chapter book 2, ch 24 (mathematics for programmers, strategic games) compresses all of that into a matrix and loses the order, the extensive form keeps it as a tree. A node is a decision point belonging to one player, an edge out of a node is one available action, and a leaf carries a payoff pair, first number for player 1, second for player
2. The solver in this book represents that literally:

```c
enum kind : unsigned char { LEAF, PL1, PL2 };

struct node {
  enum kind kind;
  int child[2]; // internal nodes: child indices in action order, leaves -1
  int pay[2];   // leaf payoffs, players 1 and 2
};

struct pair {
  int pay[2];
};

// rollback: subgame value at node i, optimal action index into act[i]
[[nodiscard]] static struct pair rollback(const struct node *t, int i,
                                          int *act) {
  if (t[i].kind == LEAF)
    return (struct pair){.pay = {t[i].pay[0], t[i].pay[1]}};
  const int mover = t[i].kind == PL1 ? 0 : 1;
  struct pair best = rollback(t, t[i].child[0], act);
  int bestk = 0;
  struct pair other = rollback(t, t[i].child[1], act);
  if (other.pay[mover] > best.pay[mover]) {
    best = other;
    bestk = 1;
  }
  act[i] = bestk;
  return best;
}
```

listing 262: c, the node record and the rollback recursion (source: math/samples/src/Ch25/backward.c, lines 40-67)

The recursion returns the subgame value: the payoff pair that plays out from a node when whoever moves at each point picks the branch maximizing their own coordinate. That self-interest rule is the whole theory of the chapter, and it is three lines of code. The fixtures are fixed data, no parsing, no randomness:

```
// fixture A: root P1 {L,R}, node1 P2 {a,b}, node2 P2 {c,d}
static constexpr struct node treeA[7] = {
    {PL1, {1, 2}, {0, 0}},    {PL2, {3, 4}, {0, 0}},
    {PL2, {5, 6}, {0, 0}},    {LEAF, {-1, -1}, {2, 1}},
    {LEAF, {-1, -1}, {0, 3}}, {LEAF, {-1, -1}, {1, 4}},
    {LEAF, {-1, -1}, {5, 2}},
};

// centipede: decision nodes 0..3 are c1..c4, child 0 = take, child 1 = pass
static constexpr struct node cent[9] = {
    {PL1, {5, 1}, {0, 0}},    {PL2, {6, 2}, {0, 0}},
    {PL1, {7, 3}, {0, 0}},    {PL2, {4, 8}, {0, 0}},
    {LEAF, {-1, -1}, {2, 4}}, {LEAF, {-1, -1}, {1, 0}},
    {LEAF, {-1, -1}, {0, 3}}, {LEAF, {-1, -1}, {3, 1}},
    {LEAF, {-1, -1}, {4, 2}},
};
```

listing 263: c, the two fixed trees as data tables (source: math/samples/src/Ch25/backward.c, lines 80-95)

The dry run: fixture A. Player 1 opens at the root with L or R, player 2 answers at either inner node, the leaves pay as follows.

1. At node 1 player 2 compares $a = (2, 1)$ against $b = (0, 3)$. The second coordinate is theirs, 3 beats 1, so the subgame value at node 1 is $(0, 3)$ reached by b.
2. At node 2 the same comparison gives $c = (1, 4)$ over $d = (5, 2)$, 4 beats 2, subgame value $(1, 4)$ reached by c.
3. At the root player 1 compares $(0, 3)$ with $(1, 4)$. The first coordinate is theirs, 1 beats 0, so player 1 enters R and the game pays $(1, 4)$.

node	mover	options pay	subgame value	choice
root	player 1	$L \to (0, 3)$, $R \to (1, 4)$	$(1, 4)$	R
node 1	player 2	$a \to (2, 1)$, $b \to (0, 3)$	$(0, 3)$	b
node 2	player 2	$c \to (1, 4)$, $d \to (5, 2)$	$(1, 4)$	c

The two numbers worth remembering from fixture A: player 1 can see payoff 5 sitting on the d leaf under R, and rollback hands them 1, because the branch that pays player 1 5 pays player 2 only 2. Greedy reading of your own best leaf is not a solution concept.

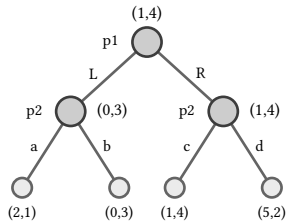


figure 150: fixture A in extensive form, rollback values beside each node

25.2. backward induction

Backward induction solves a finite perfect-information game by rolling back from the leaves: at each node the mover keeps only the branch maximizing their own payoff coordinate, the node inherits that branch's value, and the argument climbs to the root. Why the result deserves the name solution: induct on depth. A one-ply game is a table lookup, the mover takes their best leaf. Assume rollback solves every tree of depth n, take a depth $n+$

1 tree, the root mover faces n-depth subtrees whose values the solver has already computed, choosing the best of those values is exactly what the root mover wants, so the induction closes. The rollback profile, one optimal action per node, is the subgame perfect equilibrium, and in a game where every node sees the whole history it is the only refinement worth the name.

The centipede is the standard stress test. Four nodes alternate the movers, at each node the mover can take the pot or pass it along, take payoffs grow down the chain, and passing all the way through pays the cooperative terminal $(4, 2)$.

node	mover	take pays	pass leads to	value	choice
c4	player 2	(2,4)	(4,2)	$(2, 4)$	take
c3	player 1	(3,1)	c4 at (2,4)	$(3, 1)$	take
c2	player 2	(0,3)	c3 at (3,1)	$(0, 3)$	take
c1	player 1	(1,0)	c2 at (0,3)	$(1, 0)$	take

The dry run: start at the end. Player 2 at c4 keeps $(2, 4)$, their 4 beats the 2 of letting player 1 collect $(4, 2)$. Climb one node: player 1 at c3 takes the certain $(3, 1)$, their 3 beats the 2 they would get after c4. Climb again: player 2 at c2 takes $(0, 3)$, the 3 beats the 1 waiting at c3. At the root player 1 compares the sure $(1, 0)$ against $(0, 3)$ and takes. The unraveling is total, and the two numbers of the section are the rollback terminal $(1, 0)$ against the unreached cooperative terminal $(4, 2)$: both players would prefer the walk to the end, neither can afford the first step of it.

```c
// centipede: the unraveling
int actC[9] = {-1, -1, -1, -1, -1, -1, -1, -1, -1};
struct pair c4 = rollback(cent, 3, actC);
CHECK(c4.pay[0] == 2 && c4.pay[1] == 4 && actC[3] == 0,
      "c4 value (2,4), take");
struct pair c3 = rollback(cent, 2, actC);
CHECK(c3.pay[0] == 3 && c3.pay[1] == 1 && actC[2] == 0,
      "c3 value (3,1), take");
struct pair c2 = rollback(cent, 1, actC);
CHECK(c2.pay[0] == 0 && c2.pay[1] == 3 && actC[1] == 0,
      "c2 value (0,3), take");
struct pair c1 = rollback(cent, 0, actC);
CHECK(c1.pay[0] == 1 && c1.pay[1] == 0 && actC[0] == 0,
      "c1 value (1,0), take at once");

int at = 0, hops = 0;
while (cent[at].kind != LEAF && hops < 4)
  at = cent[at].child[actC[at]], hops++;
CHECK(hops == 1 && cent[at].pay[0] == 1 && cent[at].pay[1] == 0,
      "SPE path is the single move take, terminal (1,0)");
CHECK(cent[8].pay[0] == 4 && cent[8].pay[1] == 2,
      "cooperative terminal (4,2) exists but is unreached");
```

listing 264: c, centipede rollback checks and the unraveling walk (source: math/samples/src/Ch25/backward.c, lines 183-204)

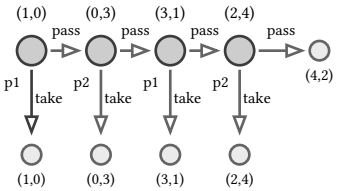


figure 151: the centipede, take drops out and pass chains right, rollback value above each node

PITFALL: ties multiply equilibria

> When two branches pay the mover exactly the same, rollback must keep a set of choices, not a choice, and every combination down that set is a distinct subgame perfect equilibrium. The solver keeps the first argmax and so pins one witness, which is all a check needs, but the theory counts all of them. Every fixture in this chapter was chosen with strict inequalities at every node.

25.3. subgame perfection and credible threats

A subgame is a node together with all of its descendants, cut so that it starts at a single node and never splits an information set. A strategy profile is a subgame perfect equilibrium when it induces a Nash equilibrium in every subgame, the whole game included. The equilibrium set only shrinks under this test: every subgame perfect equilibrium is a Nash equilibrium, the converse fails exactly when a Nash profile rests on a plan of action the player would abandon if the moment to execute it ever arrived. Textbooks call those non-credible threats, and infinite-horizon arguments replace the direct subgame scan with the one-shot deviation principle, but the finite test on a tree is the plain one to implement.

Entry deterrence is the canonical fixture. An entrant moves first, in or out. If the entrant stays out the incumbent keeps the quiet market at $(1, 4)$. If the entrant comes in, the incumbent chooses to fight, a price war paying $(0, 1)$, or accommodate, sharing the market at $(2, 3)$.

The dry run: enumerate all four profiles. Against fight the entrant compares 0 with 1 and stays out, against accommodate they compare 2 with 1 and enter. Against out the incumbent is indifferent, the fight branch is never exercised, so both replies are best responses. Two profiles survive as Nash: (out, fight), propped up by a threat, and (in, accommodate). The subgame test kills the first one, inside the incumbent's subgame fight pays 1 where accommodate pays 3, so the threat fails the moment it becomes a real decision.

profile	induces	entrant best reply	nash	subgame perfect
(in, fight)	$(0, 1)$	no	no	no
(in, accommodate)	$(2, 3)$	yes	yes	yes
(out, fight)	$(1, 4)$	yes	yes	no
(out, accommodate)	$(1, 4)$	no	no	no

```c
[[nodiscard]] static bool is_nash(struct profile p) {
  bool be[2], bi[2];
  br_entrant(p.incumbent, be);
  br_incumbent(p.entrant, bi);
  return be[p.entrant] && bi[p.incumbent];
}

// subgame perfection on this tree: the prescribed action must be optimal at
// every node, evaluated inside the subgame hanging from that node
[[nodiscard]] static bool is_spe(struct profile p) {
  // incumbent's subgame: fight pays 1 against accommodate's 3
  int vf = leaf[IN][FIGHT][1], va = leaf[IN][ACCOMMODATE][1];
  bool inc_ok = p.incumbent == FIGHT ? vf ≥ va : va ≥ vf;
  // entrant's node is the whole game: compare in versus out given p.incumbent
  bool be[2];
  br_entrant(p.incumbent, be);
  return inc_ok && be[p.entrant];
}
```

listing 265: c, the nash test and the node-by-node subgame test (source: math/samples/src/Ch25/backward.c, lines 143-160)

```
  int nash_count = 0, spe_count = 0;
  bool nash_out_fight = false, spe_in_acc = false;
  for (int e = 0; e < 2; e++) {
    for (int i = 0; i < 2; i++) {
      struct profile p = {.entrant = e, .incumbent = i};
      if (is_nash(p)) {
        nash_count++;
        if (e == OUT && i == FIGHT)
          nash_out_fight = true;
      }
      if (is_spe(p)) {
        spe_count++;
        if (e == IN && i == ACCOMMODATE)
          spe_in_acc = true;
      }
    }
  }
  CHECK(nash_count == 2 && nash_out_fight,
        "2 nash equilibria: (out,fight) and (in,accommodate)");
  CHECK(!is_spe((struct profile){OUT, FIGHT}) &&
            is_nash((struct profile){OUT, FIGHT}),
        "(out,fight) is nash but fails the incumbent subgame: threat not "
        "credible");
  CHECK(spe_count == 1 && spe_in_acc,
```

listing 266: c, enumerating all four profiles (source: math/samples/src/Ch25/backward.c, lines 231-254)

Two Nash equilibria, one subgame perfect equilibrium, and the difference is the whole point of the refinement. Rollback from section 2 found (in, accommodate) without ever writing the normal form down, which is the practical content of the equivalence: on a tree, backward induction is the subgame perfection computation.

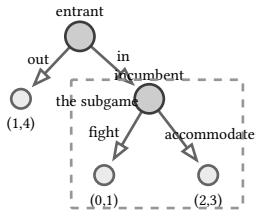


figure 152: entry deterrence, the dashed box is the incumbent subgame where the threat fails

NOTE: subset, not replacement

Subgame perfection refines Nash, it never contradicts it: the subgame perfect set is always a subset of the Nash set, and the ultimatum game is the classic case of a large gap between the two. When the horizon is infinite and no last subgame exists to start from, the one-shot deviation principle carries the same filtering role.

25.4. imperfect information

Perfect information means the mover at every node knows exactly which node they are at. Drop that and the tree needs one more structure: an information set is a set of decision nodes, all belonging to one player, that the player cannot tell apart when the game reaches them. The convention draws the set as a dotted line through its nodes, and a game with any multi-node set is a game of imperfect information. Simultaneous moves model as information sets exactly, chapter book 2, ch 24 (mathematics for programmers, strategic games) in tree clothing. A strategy now assigns probabilities to actions at each information set, a behavioral strategy, and when every player remembers what they knew and did, perfect recall, Kuhn's theorem says behavioral strategies reach every mixed-strategy payoff, so nothing is lost by attaching the probabilities to the tree.

Matching pennies in extensive form is the smallest honest example. Player 1 puts a coin down heads or tails, unseen, player 2 calls heads or tails from one information set spanning both of player 2's nodes, and same sides pays $(1, -1)$ while a mismatch pays $(-1, 1)$. With p the probability of heads for player 1 and q for player 2, player 1's expectation is $\mathrm{EV} = 2(pq + (1-p)(1-q)) - 1$. At $p = \frac{1}{2}$ the two products balance for every q, the check pins $\mathrm{EV} = 0$ at $q \in \{0, \frac{1}{4}, \frac{1}{2}\}$, while against $q = \frac{1}{4}$ the pure reply tails is worth exactly $\frac{1}{2}$ and heads exactly $-\frac{1}{2}$. Randomizing exactly at the middle is what makes the opponent's randomization irrelevant, the equalizing idea from chapter book 2, ch 24 (mathematics for programmers, strategic games) transplanted into the tree.

```c
// matching pennies, extensive form: EV for player 1 when player 1 plays H
// with probability p and player 2 plays H with probability q from the single
// information set holding both of player 2's nodes
[[nodiscard]] static struct frac pennies_ev(struct frac p, struct frac q) {
  // same side: (+1,-1) on (H,H) and (T,T); mismatch: (-1,+1)
  struct frac same =
      fadd(fmul(p, q), fmul(fsub(frac(1, 1), p), fsub(frac(1, 1), q)));
  return fsub(fmul(frac(2, 1), same), frac(1, 1));
}
```

listing 267: c, the pennies expectation in exact rationals (source: math/samples/src/Ch25/imperfect.c, lines 84-92)

Poker is where information sets earn their keep. The fixture is half-street Kuhn poker: three cards $J < Q < K$, one ante each, player 1 bets or checks, a bet hands player 2 a call-or-fold decision, checks run to a showdown over the antes, called bets run to a showdown over the raised pot. Player 2 decides from information sets keyed by their own card alone, three sets of two nodes each, because the node reached also encodes player 1's hidden card. The enumeration walks the six equiprobable deals:

deal (p1, p2)	p1 line	outcome	conditional EV
(J, Q)	check	showdown loss	-1
(J, K)	check	showdown loss	-1
(Q, J)	check	showdown win	$+1$
(Q, K)	check	showdown loss	-1
(K, J)	bet	fold	$+1$
(K, Q)	bet	call with $\frac{1}{2}$	$\frac{3}{2}$

The dry run: profile A bets the king only, calls the king always, calls the queen with probability $\frac{1}{2}$, folds the jack. The deal column sums to $\frac{1}{2}$, dividing by 6 deals gives the pinned per-hand value $\mathrm{EV} = \frac{1}{12}$, and the game is zero sum so player 2 loses $\frac{1}{12}$. Profile B adds a jack bluff with probability $\frac{1}{4}$: the two jack deals now average $-\frac{7}{8}$ and $-\frac{5}{4}$, the sum drops to $\frac{3}{8}$, and the per-hand value falls to exactly $\frac{1}{16}$. Bluffing a quarter of the time into an opponent who folds jacks and calls queens half the time costs exactly $\frac{1}{48}$ per hand. Every row of the deal table, both intermediates, and both sums are checks in the sample, so the walk above is pinned column by column, not just in its totals. All of it lands in exact rational arithmetic, a normalized numerator and denominator over `long long` with no floating point anywhere.

```c
// half-street kuhn poker: cards 1 = J < 2 = Q < 3 = K, one ante each.
// bet[card] = P(player 1 bets | card), call[card] = P(player 2 calls | card).
// payoffs to player 1, net of contributions: check -> +-1, fold -> +1,
// call -> +-2. player 2's information sets are keyed by player 2's card
// alone, so call[] indexes by player 2's card even though the node reached
// also depends on player 1's hidden card.
[[nodiscard]] static struct frac
deal_ev(int c1, int c2, const struct frac bet[3], const struct frac call[3]) {
  int win = c1 > c2 ? 1 : -1;
  struct frac check_ev = frac(win, 1);    // showdown over the two antes
  struct frac call_ev = frac(2 * win, 1); // showdown over the raised pot
  struct frac vs_bet = fadd(fmul(call[c2], call_ev),
                            fmul(fsub(frac(1, 1), call[c2]), frac(1, 1)));
  return fadd(fmul(bet[c1], vs_bet), fmul(fsub(frac(1, 1), bet[c1]), check_ev));
}

// sum of the per-deal EVs over the 6 equiprobable deals, before averaging
[[nodiscard]] static struct frac kuhn_sum(const struct frac bet[3],
                                          const struct frac call[3]) {
  struct frac tot = frac(0, 1);
  for (int c1 = 0; c1 < 3; c1++)
    for (int c2 = 0; c2 < 3; c2++)
      if (c1 != c2)
        tot = fadd(tot, deal_ev(c1, c2, bet, call));
  return tot;
}

[[nodiscard]] static struct frac kuhn_ev(const struct frac bet[3],
                                         const struct frac call[3]) {
  return fmul(kuhn_sum(bet, call), frac(1, 6)); // 6 equiprobable deals
}
```

listing 268: c, the kuhn poker per-deal expectation, its sum, and the average (source: math/samples/src/Ch25/imperfect.c, lines 94-124)

```
// profile A: bet the king only; call the king, call the queen half the
// time, fold the jack
struct frac betA[3] = {frac(0, 1), frac(0, 1), frac(1, 1)};
struct frac callA[3] = {frac(0, 1), half, frac(1, 1)};
struct frac evA = kuhn_ev(betA, callA);
CHECK(feq(evA, 1, 12), "profile A worth exactly 1/12 per hand");
CHECK(feq(fsub(frac(0, 1), evA), -1, 12), "zero sum: player 2 gets -1/12");

// per-deal pins, deals in loop order (J,Q),(J,K),(Q,J),(Q,K),(K,J),(K,Q)
struct frac wantA[6] = {frac(-1, 1), frac(-1, 1), frac(1, 1),
                        frac(-1, 1), frac(1, 1),  frac(3, 2)};
bool per_ok = true;
int k = 0;
for (int c1 = 0; c1 < 3; c1++)
  for (int c2 = 0; c2 < 3; c2++)
    if (c1 != c2) {
      if (!feq(deal_ev(c1, c2, betA, callA), wantA[k].n, wantA[k].d))
        per_ok = false;
      k++;
    }
CHECK(per_ok, "profile A per-deal EVs -1,-1,1,-1,1,3/2 in deal order");
CHECK(feq(kuhn_sum(betA, callA), 1, 2),
      "the six A deals sum to 1/2 before the 1/6 average");

// profile B: same, plus a jack bluff with probability 1/4
struct frac betB[3] = {quarter, frac(0, 1), frac(1, 1)};
struct frac evB = kuhn_ev(betB, callA);
CHECK(feq(evB, 1, 16), "profile B (jack bluff 1/4) worth exactly 1/16");
CHECK(feq(fsub(evA, evB), 1, 48), "the bluff costs exactly 1/48 per hand");
CHECK(feq(deal_ev(0, 1, betB, callA), -7, 8) &&
          feq(deal_ev(0, 2, betB, callA), -5, 4),
      "the bluffed jack deals drop to -7/8 and -5/4");
CHECK(feq(kuhn_sum(betB, callA), 3, 8),
      "the six B deals sum to 3/8 before the average");
```

listing 269: c, the behavioral profiles with per-deal and average pins (source: math/samples/src/Ch25/imperfect.c, lines 153-186)

VERIFY: rederiving the pins

Every fraction in this section reproduces from the one-off `playground/math-ch25/pin.py`, python with `fractions.Fraction` and no floats, run 2026-09-22. The script mirrors the C expression order, so a mismatch between the two is a bug in the chapter, not in the arithmetic.

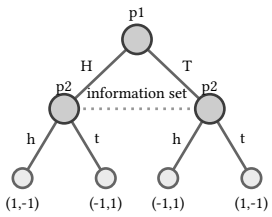


figure 153: matching pennies in extensive form, the dotted line is player 2's information set

25.5. repetition and the discount factor

Play the prisoner's dilemma once and defection dominates, chapter book 2, ch 24 (mathematics for programmers, strategic games) says so and the stage payoffs say why: temptation $T = 5$, reward $R = 3$, punishment $P = 1$, sucker $S = 0$. Play it forever with future payoffs discounted by δ per round and the value of cooperating forever is

$\frac{R}{1-\delta}$, a perpetuity the code computes as one exact fraction. Grim trigger is the simplest cooperative plan: cooperate unless anyone has ever defected, then defect forever. It sustains cooperation when deviating does not pay inside any clean-history subgame, and the comparison collapses to one line, $T + \frac{\delta P}{1-\delta} \le \frac{R}{1-\delta}$, which rearranges to $\delta \ge \frac{T-R}{T-P}$. For this stage game the threshold is $\frac{5-3}{5-1} = \frac{1}{2}$, exactly.

delta	grim pair	always-defect pair	one-shot deviation	verdict
$\frac{2}{3}$	9	3	7	cooperation holds, $9 > 7$
$\frac{1}{2}$	6	2	6	knife edge, exact tie
$\frac{2}{5}$	5	$\frac{5}{3}$	$\frac{17}{3}$	deviation wins, $\frac{17}{3} > 5$

The dry run at $\delta = \frac{2}{3}$: the grim pair is worth $\frac{3}{1-\frac{2}{3}} = 9$ per player, mutual defection forever is worth $\frac{1}{1-\frac{2}{3}} = 3$, and a player who abandons cooperation in some round collects 5 once then 1 forever, $5 + \left(\frac{2}{3}\right) \times 3 = 7$. The deviation gain is -2 at every round, the δ^k prefix cancels, and multiplying by δ each round keeps the sign negative forever, verified exactly through round 20. At $\delta = \frac{1}{2}$ the gain is identically zero, the knife edge the formula predicts, and at $\delta = \frac{2}{5}$ it turns positive at $\frac{2}{3}$. The scan over the grid $\frac{k}{10}$ finds the flip exactly between $\frac{4}{10}$ and $\frac{5}{10}$.

```
// discounted value of a constant stage payoff c from round 0 on: c/(1-d),
// valid for d < 1
[[nodiscard]] static struct frac perpetuity(struct frac d, long long c) {
  return frac(c * d.d, d.d - d.n);
}

// one-shot deviation gain inside the subgame that starts at any round with a
// clean history: deviate now (T, then P forever) versus follow grim (R
// forever); the d^k prefix cancels, so this is round-independent
[[nodiscard]] static struct frac deviation_gain(struct frac d) {
  struct frac follow = perpetuity(d, STAGE_R);
  struct frac deviate = fadd(frac(STAGE_T, 1), fmul(d, perpetuity(d, STAGE_P)));
  return fsub(deviate, follow);
}

// finite horizon n, no discount, playing against opponent grim: cooperate
// until round k then defect forever
[[nodiscard]] static long long finite_vs_grim(int n, int k) {
  return (long long)STAGE_R * k + STAGE_T + (long long)STAGE_P * (n - k - 1);
}
```

listing 270: c, perpetuities, deviation gains, and the finite-horizon counter (source: math/samples/src/Ch25/repeated.c, lines 81-100)

```
  struct frac d23 = frac(2, 3), d12 = frac(1, 2), d25 = frac(2, 5);

  CHECK(feq(perpetuity(d23, STAGE_R), 9, 1), "grim pair value 9 at delta 2/3");
  CHECK(feq(perpetuity(d23, STAGE_P), 3, 1),
        "always-defect pair value 3 at delta 2/3");
  CHECK(feq(fadd(frac(STAGE_T, 1), fmul(d23, perpetuity(d23, STAGE_P))), 7, 1),
        "best one-shot deviation worth 7 in every subgame at delta 2/3");
  struct frac g23 = deviation_gain(d23);
  CHECK(fneg(g23) && feq(g23, -2, 1), "deviation gain -2 at delta 2/3");
```

listing 271: c, values at three discount factors (source: math/samples/src/Ch25/repeated.c, lines 106-114)

Two further facts complete the picture. The folk theorem, in sketch: for patient enough players, every feasible payoff vector that gives each player more than their minmax security level can be sustained as an equilibrium, and the feasible set is the convex hull of the stage payoff pairs, the same convex-combination geometry mml-book draft 2024-01-15 develops in its chapter 7.3 convexity discussion. And the finite horizon kills everything: a 10-round game against grim cooperates on paper but the last round is a one-shot dilemma with no tomorrow, so

holding the one defection to round 9 earns $9 \times 3 + 5 = 32$ against 30 for full cooperation, immediate defection earns a flat 14, and each round of delay is worth exactly 2. The unraveling is the centipede's logic wearing a calendar.

```c
// finite 10 rounds against grim: defecting later always pays, each round
// of delay worth exactly 2, so the best response defects in round 9 only
int bestk = 0;
long long best = -1, mono_gap = 2;
for (int k = 0; k < 10; k++) {
  long long v = finite_vs_grim(10, k);
  if (k > 0 && v - (finite_vs_grim(10, k - 1)) != mono_gap)
    mono_gap = -1;
  if (v > best) {
    best = v;
    bestk = k;
  }
}
CHECK(bestk == 9 && best == 32,
      "finite game: defect in the last round only, 32");
CHECK(mono_gap == 2, "each round of delayed defection is worth exactly 2");
CHECK(10 * (long long)STAGE_R == 30 && 30 < best,
      "all-cooperate 30 loses to the last-round deviation 32");
CHECK(finite_vs_grim(10, 0) == 14, "defecting immediately earns only 14");
```

listing 272: c, the ten-round unraveling enumeration (source: math/samples/src/Ch25/repeated.c, lines 152-171)

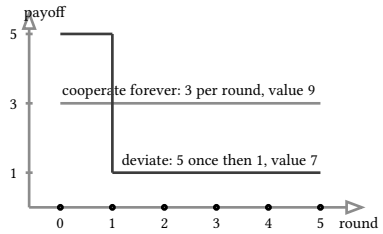


figure 154: stage payoffs by round at delta 2/3, flat 3 for cooperation against 5 then 1 for deviation

WARNING: normalization changes numbers, not verdicts

Some texts average discounted payoffs as $(1-\delta)\sum \delta^t u_t$, which rescales every value here, the grim pair 9 becomes 3 times $(1-\delta)$, but never flips a comparison, because the rescaling is a positive constant per profile. The pins in this chapter use the plain sum convention, state which one you are reading before comparing across sources.

25.6. paying the second price

An auction is a game whose tree the designer writes, and mechanism design is the discipline of writing it so the equilibrium is the outcome you wanted. The sealed-bid second-price auction, the Vickrey auction, is the cleanest specimen: bidders submit sealed bids, the highest bidder wins, the price paid is the second-highest bid. Bidding your true value is a dominant strategy. The argument is two cases, with v your value, b your bid, and m the best opposing bid. Bid above value, $b > v$: if $m < v$ you won anyway at price m and the raise did nothing, if $v \le m < b$ the raise hands you a win at price above value, negative utility. Bid below value, $b < v$: if $m \ge v$ you lost anyway, if $b \le m < v$ the shave turns a win at price m below value into a loss, throwing away surplus $v - m$. No deviation ever strictly helps, whatever the others do.

The fixture pins it exhaustively: values $(10, 7, 4)$, all opponent bid pairs in $0..12$, all own deviations in $0..12$, ties to the lowest index. Truth pays the winner $10 - 7 = 3$, losers pay nothing, and the improvement count over truth is zero for every bidder, 3 x 2197 evaluated profiles. The same grid under the first-price rule, winner pays own

bid, tells the opposite story: truth earns the winner $10 - 10 = 0$, shaving to 7 against truthful rivals wins the tie and earns 3, and 385 grid points strictly beat truth for bidder 1. One payment rule makes honesty safe, the other taxes it.

rule	winner pays	utility of bidder 1	deviations beating truth
second price, truthful bids	second bid 7	3	0 of 2197
first price, truthful bids	own bid 10	0	385 of 2197

```c
[[nodiscard]] static int winner_of(const int b[3]) {
  int w = 0;
  for (int j = 1; j < 3; j++)
    if (b[j] > b[w])
      w = j;
  return w;
}

// utility of bidder i under the second-price rule: pay the best other bid
[[nodiscard]] static int u_second(const int b[3], int i) {
  if (winner_of(b) != i)
    return 0;
  int top = -1;
  for (int j = 0; j < 3; j++)
    if (j != i && b[j] > top)
      top = b[j];
  return VALUES[i] - top;
}

// utility of bidder i under the first-price rule: pay your own bid
[[nodiscard]] static int u_first(const int b[3], int i) {
  if (winner_of(b) != i)
    return 0;
  return VALUES[i] - b[i];
}
```

listing 273: c, winner rule and the two payment rules (source: math/samples/src/Ch25/auction.c, lines 40-64)

```c
// count strict improvements over the truthful bid for bidder i under rule u,
// scanning all others' bids and all own deviations on the integer grid
[[nodiscard]] static int count_improvements(int (*u)(const int[3], int),
                                            int i) {
  int bad = 0;
  for (int b2 = 0; b2 ≤ BID_MAX; b2++) {
    for (int b3 = 0; b3 ≤ BID_MAX; b3++) {
      int truth[3] = {VALUES[0], VALUES[1], VALUES[2]};
      int others[2] = {b2, b3}, k = 0;
      for (int j = 0; j < 3; j++)
        if (j ≠ i)
          truth[j] = others[k++];
      for (int bi = 0; bi ≤ BID_MAX; bi++) {
        int dev[3] = {truth[0], truth[1], truth[2]};
        dev[i] = bi;
        if (u(dev, i) > u(truth, i))
          bad++;
      }
    }
  }
  return bad;
}
```

listing 274: c, the improvement-counting grid scan (source: math/samples/src/Ch25/auction.c, lines 66-87)

The dry run on the two boundary cases the proof isolates: bidder 2 with value 7 facing rivals bidding 10 and 4 loses truthfully for 0, overbidding to 11 wins and pays 10 for utility −3. Bidder 1 with value 10 facing 6 and 4 wins truthfully at price 6 for utility 4, shaving to 5 loses it all for 0. Raising bids can only create wins at prices above value, lowering them can only cancel wins at prices below it, and that asymmetry is the entire mechanism. The generalization, paying each winner the externality they impose, is the VCG family, and the second-price auction is its single-item case.

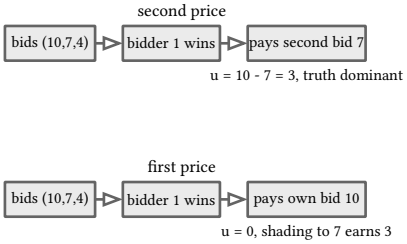


figure 155: the same bids under two payment rules, truth pays 3 at second price and 0 at first price

The equilibrium ideas scale past auctions into any protocol where participants report private information, and the next chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) leaves the referee's standpoint entirely, building the tagged data types that carry exactly such structured outcomes through real C programs.

sources: Martin J. Osborne, An Introduction to Game Theory, Oxford University Press 2004, chapters 5 through 7 for extensive games, backward induction, subgame perfection and repeated games, cited by name and edition, no page text quoted. H. W. Kuhn, "Simplified Two-Person Poker", in Contributions to the Theory of Games I, Princeton University Press 1950, the three-card fixture, cited by name. Wikipedia, "Extensive-form game", en.wikipedia.org/wiki/Extensive-form_game, "Subgame perfect equilibrium", en.wikipedia.org/wiki/Subgame_perfect_equilibrium, "Grim trigger", en.wikipedia.org/wiki/Grim_trigger, "Vickrey auction", en.wikipedia.org/wiki/Vickrey_auction, all fetched 2026-09-22, cc by-sa 4.0, our own words and code throughout. mml-book draft 2024-01-15: game theory has no mapped chapter, the convex-hull sentence in section 5 points at its ch 7.3 convexity material as a prose bridge without quotation. Sample behavior verified by `pwsh -NoProfile`

`-File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch25`, 59 checks in chapter 25 of the math suite, format and asan clean.

26. algebraic data types in c23

Chapters 1 to 25 built mathematics out of numbers. The last arc of the book builds it out of values, and the load-bearing notion is the algebraic data type: a product bundles a fixed set of fields, a sum chooses exactly one alternative, and every structure of the next four chapters, option, result, functors, pipelines, is one of those two shapes or a composition of both. This chapter makes six moves: the cardinality arithmetic that separates products from sums, the tagged union as a c23 struct with an anonymous union and a fixed-type enum, the exhaustive switch as the pattern-match idiom with a compile-time miss detector, the product machinery of typeof, auto, and constexpr with compound-literal values, defer as the ownership story for sums whose payload lives on the heap, and the two-tag sum that chapter 27 grows into option. Every behavioral claim below is one of the 59 checks in the 4 samples of chapter 26 or a sentence quoted from a canonical source fetched 2026-09-22. Book 1 owns these features as language spellings, book 1, ch 3 (c23: the language, the machine, and the cloud, the type system) for the type namers and book 1, ch 4 (c23: the language, the machine, and the cloud, pointers, arrays, and memory) for compound literals, while this chapter owns the type arithmetic and the discipline those spellings serve.

26.1. products and sums

A product type is a cartesian product of fields: the struct holding one a and one b has exactly the pairs of an element of a with an element of b, so its inhabitant count multiplies, $|a \times b| = |a| \cdot |b|$. A sum type is a tagged choice: one value that is either an a or a b, never both, so its count adds, $|a + b| = |a| + |b|$. Every compound datum in this book is built from these two and they obey ordinary set arithmetic, including distributivity, $a \times (b + c) = a \times b + a \times c$, which says a pair whose second component is a choice is the same data as a choice between two pair shapes. The set counting behind these identities is the machinery of book 2, ch 13 (mathematics for programmers, logic, sets, and proof), restated here for types.

The two-number play: two octets as a product have $256 \times 256 = 65536$ inhabitants, the same two octets as a sum have $256 + 256 = 512$. One byte of tag buys a factor of 128 in expressiveness at the cost of one live reading. The miniature below draws the same contrast with two-bit values so the cells stay countable, 16 against 8.

form	c23 spelling	two bits each	octets
product $a \times b$	struct with both fields	$4 \times 4 = 16$	$256 \times 256 = 65536$
sum $a + b$	tagged union	$4 + 4 = 8$	$256 + 256 = 512$
distribution $a \times (b + c)$	pair of a tagged second	$4(4 + 4) = 32 = 16 + 16$	$a = 4, b = 6, c = 10$: $64 = 24 + 40$

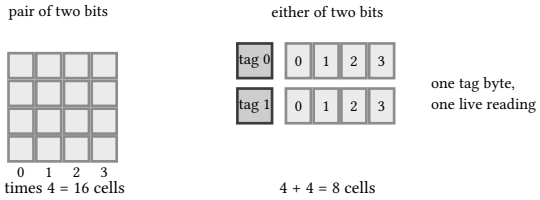


figure 156: inhabitants: a 4 by 4 pair grid against a 4 plus 4 tagged row, two bit cells

The dry run: the counting loops in the sample walk both encodings of two octets. The product loop nests 256 inside 256 and counts 65536 iterations. The sum loop walks 2 tags by 256 payloads and counts 512. Distributivity uses $a = 4$, $b = 6$, $c = 10$: the left side counts pairs of an $x \in \{0, 1, 2, 3\}$ with a tagged $t \in \{0, \ldots, 15\}$, that is $4 \cdot 16 = 64$, and the right side counts $4 \cdot 6 = 24$ pairs into b plus $4 \cdot 10 = 40$ pairs into c, also 64. One witness round trips: the c-branch pair $(x = 2, y = 1)$ encodes as $t = b + y = 7$, and $t = 7$ decodes back to the c branch with $y = 1$ while x rides along untouched, which is the bijection distributivity promises, executed rather than asserted.

```
// type arithmetic, counted: a product of two octets has 256*256 pairs
long long product = 0;
for (int a = 0; a < 256; a++) {
  for (int b = 0; b < 256; b++) {
    product++;
  }
}
CHECK(product == 65536, "pair of octets: 256 * 256 = 65536 inhabitants");

// a sum of two octets (tag plus payload) has 256 + 256 inhabitants
long long sum = 0;
for (int t = 0; t < 2; t++) {
  for (int v = 0; v < 256; v++) {
    sum++;
  }
}
CHECK(sum == 512, "either of octets: 256 + 256 = 512 inhabitants");

// distributivity a*(b+c) = a*b + a*c with a=4, b=6, c=10, counted both
// ways: left counts pairs (x, tagged y), right counts tagged pairs
long long left = 0;
for (int x = 0; x < 4; x++) {
  for (int t = 0; t < 16; t++) {
    left++;
  }
}
long long right = 0;
for (int x = 0; x < 4; x++) {
  for (int y = 0; y < 6; y++) {
    right++; // a*b side: pairs into b
  }
  for (int y = 0; y < 10; y++) {
    right++; // a*c side: pairs into c
  }
}
CHECK(left == 64 && right == 64, "distributivity: 4*16 = 24 + 40 = 64 both");

// one witness round trips: (x=2, branch c, y=1) encodes t = b + y = 7,
// and t=7 decodes to branch c with y=1 while x rides along untouched
enum { B = 6, C = 10 };
int wx = 2, wy = 1, t = B + wy;
int branch = (t ≥ B) ? 1 : 0;
int back_y = branch ? t - B : t;
CHECK(t == 7 && branch == 1 && back_y == 1 && wx == 2,
      "witness (x=2, c-branch, y=1) lands on t = 7 and decodes back");
```

listing 275: the counting loops: product multiplies, sum adds, distributivity counts both encodings, one witness round trips (source: math/samples/src/Ch26/product.c, lines 97-141)

26.2. the tagged union

The c23 sum type is a struct with three parts: an enum tag with a fixed underlying type, an anonymous union holding one struct per alternative, and static_asserts pinning the layout so the tag discipline rests on measured bytes. The anonymous union is the c23 convenience that matters here: because it has no member name, its members are reached directly, `s.circle.r` rather than `s.u.circle.r`, which keeps the constructor and every case body honest about which alternative they touch.

The dry run: construct `circle(2)` and the object is 56 bytes. The tag byte is 0 at offset 0, 7 bytes of padding follow because the union of doubles aligns to 8, and the payload region starts at offset 8 with `r = 2.0`. The area is the pi double times 2 times 2, and doubling a double is exact in binary, so `12.566370614359172` is the pinned answer. The static_asserts in the sample pin all of this at compile time: the enum is 1 byte, the point struct 16, the whole shape 56, and the tags enumerate 0, 1, 2.

constructor	tag byte	payload at offset 8	area
`circle(1)`	0	`r = 1`	3.141592653589793
`rect(3, 5)`	1	`w, h = 3, 5`	15
`tri345()`	2	`(0,0) (4,0) (0,3)`	6

```c
enum shape_tag : uint8_t { CIRCLE, RECT, POLY };

struct pt {
  double x, y;
};

struct shape {
  enum shape_tag tag;
  union { // anonymous: members are reached as s.circle, s.rect, s.poly
    struct {
      double r;
    } circle;
    struct {
      double w, h;
    } rect;
    struct {
      struct pt a, b, c;
    } poly;
  };
};

static_assert(sizeof(enum shape_tag) == 1, "fixed underlying type is one byte");
static_assert(sizeof(struct pt) == 16, "two doubles");
static_assert(sizeof(struct shape) == 56, "1 tag byte, 7 pad, 48-byte union");
static_assert(CIRCLE == 0 && RECT == 1 && POLY == 2, "tags enumerate 0, 1, 2");

// the pi double: same bits as M_PI, spelled as a c23 constexpr object because
// the ucrt math.h guards that macro behind _USE_MATH_DEFINES under every
// -std= mode on this lane and the chapter owns its constants
constexpr double PI = 3.141592653589793;

// constructors: one compound literal per alternative, tag first
[[nodiscard]] static struct shape circle(double r) {
  return (struct shape){.tag = CIRCLE, .circle = {.r = r}};
}

[[nodiscard]] static struct shape rect(double w, double h) {
  return (struct shape){.tag = RECT, .rect = {.w = w, .h = h}};
}

// the 3-4-5 right triangle: every coordinate and side an exact double
[[nodiscard]] static struct shape tri345(void) {
  return (struct shape){
      .tag = POLY,
      .poly = {.a = {0, 0}, .b = {4, 0}, .c = {0, 3}},
  };
```

listing 276: the sum type: fixed-type enum, anonymous union, layout static_asserts, compound-literal constructors (source: math/samples/src/Ch26/shape.c, lines 33-78)

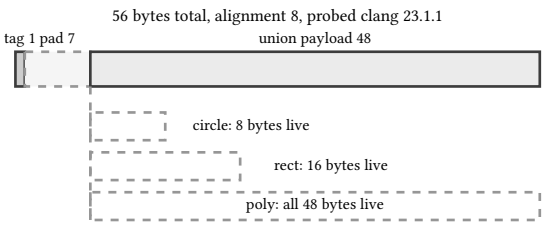


figure 157: struct shape, 56 bytes: one tag byte, 7 of padding, then the 48 byte union with the three payload footprints

The dispatch reads the tag first and touches only the payload the tag names. The area switch returns `PI * r * r` for the circle, `w * h` for the rect, and the shoelace formula for the triangle, whose pinned fixture (0,0), (4,0), (0,3) gives area 6 and sides 4, 5, 3, perimeter 12, every one an exact double because the coordinates are integers.

```
[[nodiscard]] static double shape_area(const struct shape *s) {
  switch (s->tag) {
  case CIRCLE:
    return PI * s->circle.r * s->circle.r; // r=1: 3.141592653589793
  case RECT:
    return s->rect.w * s->rect.h; // 3x5: 15
  case POLY: { // shoelace on the pinned triangle: |12| / 2 = 6
    const struct pt *a = &s->poly.a, *b = &s->poly.b, *c = &s->poly.c;
    return fabs(a->x * (b->y - c->y) + b->x * (c->y - a->y) +
                c->x * (a->y - b->y)) /
           2.0;
  }
  }
```

listing 277: tag first, then the one honest payload reading, per case (source: math/samples/src/Ch26/shape.c, lines 80-93)

PITFALL: THE TAG IS THE ONLY WITNESS

The union stores at most one member at a time, and cppreference states the rest plainly: “If the member used to access the contents of a union is not the same as the member last used to store a value, the object representation of the value that was stored is reinterpreted as an object representation of the new type (this is known as type punning)”. The sample leans on this: after `circle(1)`, reading `s.rect.w` sees the same 8 bytes and returns 1.0, because both members start at union offset 0. Nothing in the bytes distinguishes a radius from a width. The tag byte is the only witness of which reading is honest, which is why every constructor sets it and every consumer switches on it before touching payload. Fetched 2026-09-22.

26.3. exhaustive switch

Pattern matching in c23 is a switch over the enum tag with every case present and no default clause. The compiler carries the discipline: `-Wswitch` lives inside `-Wall` and fires when a switch over an enum misses an enumerator, so under the corpus gate, `-std=c23 -Werror -Wall -Wextra`, a forgotten case stops the build. A default clause would silence exactly that guard by catching every value, which is why an exhaustive switch over a closed tag set never carries one. Because the switch is total, there is nothing after it, and clang’s return analysis agrees.

The dry run: delete the `POLY` case from `shape_name` and rebuild. The build fails before any code runs. The other direction is the runtime half of the contract: a dispatch table, indexed by designated initializer and sized by the same constexpr object that counts the tags, gets one handler per tag, and a loop over all tag values checks every slot is filled. The table turns tag discipline into data, one function pointer per alternative, while a static_assert ties the table length to `TAG_COUNT` so a new tag widens the table or fails.

```c
// the idiom: every case present, no default, and nothing after the switch
// because the switch is total. add POLY2 to the enum without a case here and
// -Wswitch stops the build under -Werror (diagnostic quoted in the header)
[[nodiscard]] static const char *shape_name(enum shape_tag t) {
  switch (t) {
  case CIRCLE:
    return "circle";
  case RECT:
    return "rect";
  case POLY:
    return "polygon";
  }
}

[[nodiscard]] static int is_curved(enum shape_tag t) {
  switch (t) { // two cases share one body: rect and poly fall through
  case CIRCLE:
    return 1;
  case RECT:
  case POLY:
    return 0;
  }
}
```

listing 278: the idiom: every case, no default, nothing after the total switch, fall-through shares one body (source: math/samples/src/Ch26/exhaustive.c, lines 51-73)

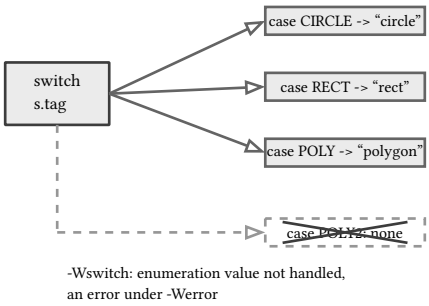


figure 158: exhaustive dispatch: three cases return, a fourth tag with no case stops the build

VERIFY: THE NEGATIVE COMPILE

The claim “a missing case fails the build” was probed, not assumed, on 2026-09-22 with clang 23.1.1 under the gate flags. The diagnostic, verbatim: `error: enumeration value 'POLY' not handled in switch [-Werror,-Wswitch]`. The probe lives in the chapter playground beside the samples, and the same flags build every sample in this book, so the guarantee is the gate itself.

The dispatch table is the same discipline as data. Slot `[CIRCLE]` holds `circle_area`, and calling through the table for the r = 2 fixture returns the 4 pi double, the rect handler returns 15, the polygon handler runs Heron’s formula on sides 3, 4, 5 where every intermediate is an integer-valued double, semiperimeter 6, product 36, and `sqrt(36)` is exactly 6.

```
// the table: every slot filled, so dispatch is total even without a switch
double (*const table[TAG_COUNT])(const struct shape *) = {
    [CIRCLE] = circle_area, [RECT] = rect_area, [POLY] = poly_area};
static_assert(sizeof table / sizeof table[0] == TAG_COUNT,
              "table length equals tag count");
int filled = 0;
for (int t = 0; t < TAG_COUNT; t++) {
  if (table[t] != 0) {
    filled++;
  }
}
CHECK(filled == TAG_COUNT, "dispatch table populated for every tag");

struct shape c = {.tag = CIRCLE, .circle = {.r = 2}};
struct shape r = {.tag = RECT, .rect = {.w = 3, .h = 5}};
struct shape p = {.tag = POLY, .poly = {.sides = {3, 4, 5}}};
// expected: circle r=2 area 12.566370614359172 (4 pi), rect 15, heron 6
CHECK(table[CIRCLE](&c) == 12.566370614359172,
      "table dispatch: circle r=2 area is the 4 pi double");
CHECK(table[RECT](&r) == 15, "table dispatch: rect 3x5 area 15");
CHECK(table[POLY](&p) == 6, "table dispatch: 3-4-5 heron area 6");
```

listing 279: the table: designated initializer per tag, static_assert on length, a filled-slot loop, three dispatched areas (source: math/samples/src/Ch26/exhaustive.c, lines 150-170)

26.4. product machinery: typeof, auto, constexpr

The product side runs on c23 type-level spellings that book 1 introduces as language facts, and this section composes them into one working idiom. A macro stamps the pair type, `DEFINE_PAIR(long long, ll)` expands to a struct typedef, so the product of any two same-typed fields is one line. `typeof` names a type from an expression and preserves its qualifiers, `typeof_unqual` strips them, `auto` infers an obvious type from its initializer, and `constexpr` declares a scalar object usable in constant expressions, which is how the fixture table gets its length. The one boundary to keep straight: c23 has constexpr objects but no constexpr functions, and the macro plus compound literal pair above is the idiom that replaces them.

The dry run: `typeof(d.second)` where `d` is a `pair_dbl` is `double`, so squaring 2.5 through that type yields 6.25 exactly. A `const pair_ll` makes `cp.first` a `const long long`, `typeof_unqual(cp.first)` is a mutable `long long`, the copy increments to 13 while the source stays 12. A compound literal is an unnamed object written as a value: `(pair_ll){.first = 20, .second = 22}` passes by value into `pair_sum_ll` and returns 42, and because the literal is an lvalue its address survives as `pp`. Designated initializers zero what they do not name, so `{.second = 5}` reads back `first == 0`.

```c
// one macro stamps every pair type: the product a x b as a plain struct
#define DEFINE_PAIR(T, S)                                                   \
  typedef struct {                                                          \
    T first;                                                                \
    T second;                                                               \
  } pair_##S;
DEFINE_PAIR(long long, ll)
DEFINE_PAIR(double, dbl)

static_assert(sizeof(pair_ll) == 16, "two long long, LLP64");
static_assert(sizeof(pair_dbl) == 16, "two doubles");

[[nodiscard]] static long long pair_sum_ll(pair_ll p) { // by value: 2 fields
  return p.first + p.second;
}

// constexpr objects size the fixture table at compile time
constexpr int ROWS = 3;
static const pair_ll FIXTURE[ROWS] = {
    {.first = 1, .second = 2},
    {.first = 10, .second = 20},
    {.first = 100, .second = 200},
};
```

listing 280: the stamp macro, the layout static_asserts, the constexpr-sized fixture table (source: math/samples/src/Ch26/product.c, lines 31-53)

```c
  // compound literals: unnamed pair objects written as values, one by value
  // as a call argument, one through its address (a compound literal is an
  // lvalue, c23 6.5.2.5)
  CHECK(pair_sum_ll((pair_ll){.first = 20, .second = 22}) == 42,
        "compound literal travels by value, 20 + 22 = 42");
  const pair_ll *pp = &(pair_ll){.first = 7, .second = 9};
  CHECK(pp->first + pp->second == 16, "the literal is an lvalue, addressable");

  // designated initializers zero what they do not name: the gap reads 0
  pair_ll z = {.second = 5};
  CHECK(z.first == 0 && z.second == 5,
        "designated initializer zeroes the unnamed field");

  // typeof names a type from an expression: the double travels with d.second
  typeof(d.second) squared = d.second * d.second;
  CHECK(squared == 6.25, "typeof(d.second) is double, 2.5 squared 6.25");

  // typeof preserves qualifiers, typeof_unqual strips them: y is mutable int
  const pair_ll cp = {.first = 12, .second = 30};
  typeof_unqual(cp.first) y = cp.first;
  y += 1;
  CHECK(y == 13 && cp.first == 12,
        "typeof_unqual strips const, the source stays 12");

  // auto where the type is obvious from the initializer
  auto q = (pair_ll){.first = 60, .second = 6};
  CHECK(q.first / q.second == 10, "auto infers pair_ll, 60 / 6 = 10");
```

listing 281: compound literal by value and by address, designated zeroing, typeof, typeof_unqual, auto (source: math/samples/src/Ch26/product.c, lines 62-88)

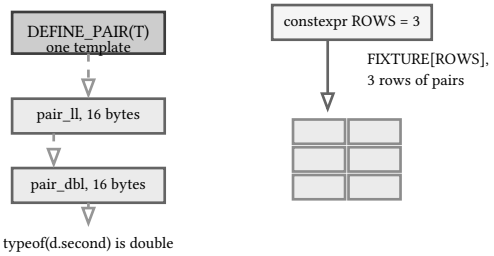


figure 159: one macro stamps every pair type, constexpr sizes the fixture, typeof picks the field type

NOTE: CONSTEXPR OBJECTS ONLY

C23 constexpr is a storage-class specifier for scalar objects with an explicit constant initializer, usable in array bounds and static_asserts the way `ROWS` sizes the fixture. There are no constexpr functions in c23, that is the c++ machinery. When a compile-time table of values is wanted, the idiom is the one above: a constexpr dimension plus a static const array of compound structures, initialized once at file scope.

26.5. defer and ownership

A sum whose payload lives on the heap owns that memory, and every path out of the owning scope must release it exactly once. C23 itself has no answer yet. Technical specification 25755 adds `defer`, and clang 23.1.1 ships it behind `stddefer.h` and the `-fdefer-ts` gate flag, which defines `__STDC_DEFER_TS25755__` and reports version 202602L. The semantics fit the sentence book 1 pins: a deferred statement runs when its scope exits, last registered first out. That single rule covers early returns, normal returns, and loop bodies, and it is the whole ownership story for this arc.

The dry run: `build_area(1)` allocates the triangle points, registers one deferred free, hits the bail branch, marks `b`, and returns −1.0. The scope exit runs the defer, marking `f` and counting 1 free, so the trace reads `bf` and the counter reads 1. The normal path marks `a` before the same exit, trace `af`, value 6.0. `two_regions` registers two defers and returns 12, and the unwind runs them in reverse, trace 21, two allocations, two frees. The move block is the transfer case: `owned_move` copies the pointer and nulls the source with `nullptr`, so the single deferred free on the destination is the only free, and the source provably owns nothing.

```
// one owned triangle, freed by defer on every path out of the function
[[nodiscard]] static double build_area(int early_bail) {
  struct pt *t = tri_malloc(0, 0, 4, 0, 0, 3);
  defer { // function scope: fires at return, whichever return that is
    free(t);
    frees++;
    tmark('f');
  }
  if (early_bail) {
    tmark('b');
    return -1.0; // the early return still unwinds the defer
  }
  tmark('a');
  return tri_area(t); // 6
}

// two owned triangles, two defers: last registered runs first
[[nodiscard]] static double two_regions(void) {
  struct pt *t1 = tri_malloc(0, 0, 4, 0, 0, 3); // area 6
  defer {
    free(t1);
    frees++;
    tmark('1');
  }
  struct pt *t2 = tri_malloc(1, 1, 4, 1, 1, 5); // area 6
  defer {
    free(t2);
    frees++;
    tmark('2');
  }
  return tri_area(t1) + tri_area(t2); // 12
}
```

listing 282: one defer per owned allocation: the early return still frees, two defers unwind last first (source: math/samples/src/Ch26/defer.c, lines 79-110)

```
  reset();
  {
    struct owned src = owned_new(4);
    struct owned dst = {.data = nullptr, .n = 0};
    defer { // frees exactly the destination: the source no longer owns
      free(dst.data);
      frees++;
    }
    owned_move(&dst, &src);
    CHECK(src.data == nullptr && src.n == 0,
          "after the move the source owns nothing");
    CHECK(dst.n == 4 &&
              dst.data[0] + dst.data[1] + dst.data[2] + dst.data[3] == 10.0,
          "the destination owns 1, 2, 3, 4, sum 10");
  } // the block exit runs the deferred free
  CHECK(frees == 1, "one allocation, one free, through the move");
```

listing 283: the move block: the source reads nullptr, one deferred free at block exit (source: math/samples/src/Ch26/defer.c, lines 211-226)

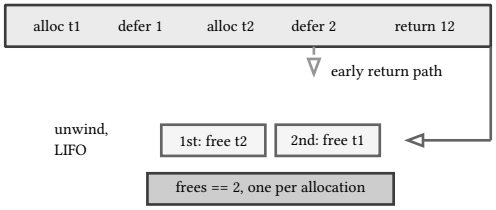


figure 160: scope timeline: defers register left to right and fire last first, on the early path too

NOTE: DEFER IS A TECHNICAL SPECIFICATION

`defer` is not in C23 proper. It arrives from TS 25755, and the corpus gate carries `-fdefer-ts` precisely because clang only defines `__STDC_DEFER_TS25755__` and enables the keyword under that flag, the toolchain fact book 1, ch 1 (c23: the language, the machine, and the cloud, toolchain: clang 23 on windows) records and book 1, ch 6 (c23: the language, the machine, and the cloud, the c23 library) demonstrates with the same LIFO traces. Every sample in chapters 26 to 29 relies on this lane being pinned, and the version macro check at the top of the sample guards it at runtime.

26.6. from sums to option

The smallest useful sum has two tags and one payload, and it is famous under another name. A value of `opt_i64` is either `NONE`, an alternative whose payload is empty, or `SOME` carrying a `long long`. The layout is the tagged union again, 16 bytes on this machine, 1 tag byte, 7 of padding, 8 of payload. The constructors are compound literals, and the consumers are exhaustive switches: `get_or` returns the payload or a caller-supplied default without ever reading `.v` on the `NONE` side, and the doubling function maps `SOME 3` to `SOME 6` while carrying `NONE` through untouched, the mapping law chapter 28 states precisely.

The dry run: `find_first_greater` walks `{3, 41, 7, 19, 23}` looking past 30. The 3 fails the test, the 41 passes, and the function returns `SOME 41` without carrying the index or a sentinel. The same walk looking past 99 exhausts the array and returns `NONE`, and the two results are distinguished by the tag alone. This is the shape sentinel error handling cannot copy: a function that returns −1 for miss has to hope −1 is never a payload, and the sum removes the hope by construction.

```c
// the two-tag sum chapter 27 grows into option/result
enum opt_tag : uint8_t { NONE, SOME };
struct opt_i64 {
  enum opt_tag tag;
  union {
    long long v;
  };
};

static_assert(sizeof(struct opt_i64) == 16, "1 tag byte, 7 pad, 8 payload");

[[nodiscard]] static struct opt_i64 some(long long v) {
  return (struct opt_i64){.tag = SOME, .v = v};
}

[[nodiscard]] static struct opt_i64 none(void) {
  return (struct opt_i64){.tag = NONE};
}

// the option consumer: an exhaustive switch over two tags, no default, no
// sentinel read of .v on the NONE side
[[nodiscard]] static long long get_or(struct opt_i64 o, long long dflt) {
  switch (o.tag) {
  case SOME:
    return o.v;
  case NONE:
    return dflt;
  }
}

[[nodiscard]] static struct opt_i64 opt_double(struct opt_i64 o) {
  switch (o.tag) {
  case SOME:
    return some(o.v * 2);
  case NONE:
    return o; // none maps to none: the functor law chapter 28 states
  }
}

[[nodiscard]] static struct opt_i64
find_first_greater(const long long *a, size_t n, long long bound) {
  for (size_t i = 0; i < n; i++) {
    if (a[i] > bound) {
      return some(a[i]); // first hit wins, index is not part of the value
    }
  }
  return none();
}
```

listing 284: opt_i64: constructors, get_or, a mapping switch, and a real search returning some or none (source: math/samples/src/Ch26/exhaustive.c, lines 93-140)

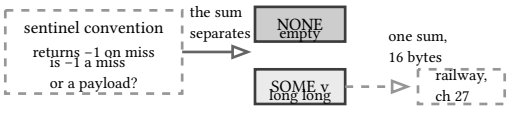


figure 161: the sentinel question against the two tag sum, and the road to chapter 27

WARNING: SENTINELS COLLIDE WITH PAYLOADS

A miss encoded as −1 works until −1 is a legitimate value, and the collision is silent: the caller cannot tell the absent from the present. The sum type spends one tag byte to make absence a first-class value, and the exhaustive switch spends one case to force the caller to acknowledge it. Chapters 27 to 29 build every error path on this trade.

The next chapter, book 2, ch 27 (mathematics for programmers, option, result, and the railway), widens this two-tag sum into the option and result types and chains their consumers into the railway, the error handling idiom the rest of the capstone runs on.

sources: cppreference, “union member access and type punning”, en.cppreference.com/w/c/language/union, fetched 2026-09-22, quoted once in the pitfall callout. The c23 feature record, typeof and typeof_unqual at 6.7.2.5, constexpr objects, auto, nullptr, and the compound literal lifetime rules of 6.5.2.5, verified against this repo’s book 1, chapters types and pointers, version 2.0, read 2026-09-22. Defer verified against technical specification 25755 as shipped by clang 23.1.1, stddefer.h version macro 202602L under -fdefer-ts, probed on this machine 2026-09-22, and the exhaustiveness diagnostic quoted verbatim from a negative compile under the gate flags the same day. This chapter has no mml-book mapping, the type arithmetic is the set counting of book 2, ch 13 (mathematics for programmers, logic, sets, and proof) at elementary sizes, verified by enumeration in the samples rather than by page citation. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch26`, 59 checks in chapter 26 of the math suite, format and asan clean.

27. option, result, and the railway

Chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) built tagged sums with anonymous unions and exhaustive switches. This chapter puts them to work as error channels: an option type that makes presence data, a result type whose error track carries a typed code and a position, the map, and_then, or_else combinators that compose fallible stages into one railway, the [[nodiscard]] contracts that make ignoring a returned rail a compile failure, and the defer teardown that keeps resources honest when a stage steps off the track. Every behavioral claim below is one of the 78 checks in the 4 samples of chapter 27 or a sentence quoted from a canonical source fetched 2026-09-22. The dsa book never builds this machinery, its pipelines return error codes inline, so this chapter is self contained on that side. Where chapter book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) asked how far a computed number drifts from the truth, this chapter asks how a computed failure travels through a program without being dropped on the floor.

27.1. from null to option

The C library answers “did this lookup find anything” with a pointer or a sentinel. cppreference describes the macro: “The macro NULL is an implementation-defined null pointer constant …” (en.cppreference.com/w/c/types/NULL, fetched 2026-09-22). C23 adds a typed spelling: “The keyword nullptr denotes a predefined null pointer constant. It is a non-lvalue of type nullptr_t” (en.cppreference.com/w/c/language/nullptr, fetched 2026-09-22). Both give absence a representation, and both share one defect: the caller cannot tell a missing result from a value that happens to equal the sentinel. A table lookup that returns −1 for a miss collides with the width −1.

The fix is the smallest possible sum type. An option of `int32_t` is the sum of unit and the payload: one tag byte, 0 or 1, selects between none and some v. The struct is 8 bytes on this machine, one tag byte, 3 bytes of padding, and the 4 byte payload, checked by a `static_assert` in the sample and again at runtime, the width of one pointer. The two-number play: a 4 byte int sentinel makes −5 mean two things, a stored value and a miss, while one tag byte lets 0 and 1 decide and −5 stays a value.

```c
// the raw c convention: the key pointer itself, nullptr when missing
static const char *config_find_raw(const char *k) {
  for (int i = 0; i < CONFIG_LEN; i++) {
    if (strcmp(CONFIG[i].key, k) == 0) {
      return CONFIG[i].key;
    }
  }
  return nullptr;
}

// the option convention: presence is data, not a pointer trick
[[nodiscard]] static opt_i32 opt_none(void) {
  return (opt_i32){.tag = 0, .v = 0};
}
[[nodiscard]] static opt_i32 opt_some(int32_t v) {
  return (opt_i32){.tag = 1, .v = v};
}
[[nodiscard]] static bool opt_is_some(opt_i32 o) { return o.tag != 0; }
[[nodiscard]] static bool opt_is_none(opt_i32 o) { return o.tag == 0; }
[[nodiscard]] static int32_t opt_unwrap_or(opt_i32 o, int32_t d) {
  return o.tag ? o.v : d;
}
[[nodiscard]] static opt_i32 opt_map(opt_i32 o, int32_t (*f)(int32_t)) {
  return o.tag ? opt_some(f(o.v)) : o;
}
[[nodiscard]] static opt_i32 opt_and_then(opt_i32 o, opt_i32 (*f)(int32_t)) {
  return o.tag ? f(o.v) : o;
}
[[nodiscard]] static opt_i32 opt_or_else(opt_i32 o, opt_i32 (*fallback)(void)) {
  return o.tag ? o : fallback();
}
```

listing 285: the raw nullptr convention, then the option constructors and accessors, all [[nodiscard]] (source: math/samples/src/Ch27/option.c, lines 33-63)

The dry run: the config table holds width 12, height 9, depth 4. The lookup of "width" walks the table, matches on the first entry, and returns some 12. The lookup of "scale" walks all 3 entries, matches nothing, and returns none. Mapping add5 over the first gives some 17. Mapping it over the second changes nothing, and unwrap_or collapses the none onto the fallback 1.

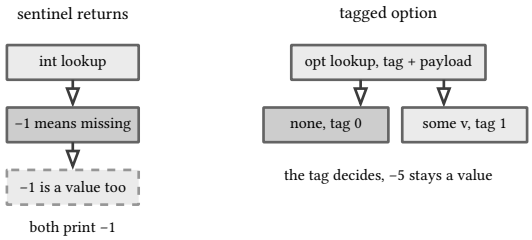


figure 162: the sentinel collision on the left, the tag that removes it on the right

Every accessor is [[nodiscard]] because an option nobody examines is a lookup whose answer went straight to the garbage. The sample checks both conventions against the same table: the raw pointer version returns nullptr exactly for the miss, and the option version returns none for the same miss while some 12 survives mapping, chaining, and fallback.

```
int main(void) {
  // 12 % 2 == 0 so halve_if_even(12) is some(6), precomputed in playground
  CHECK(config_find_raw("width") != nullptr,
        "raw lookup hit is a real pointer");
  CHECK(config_find_raw("scale") == nullptr, "raw lookup miss is nullptr");
  CHECK(opt_is_some(opt_some(7)), "some carries the some tag");
  CHECK(opt_is_none(opt_none()), "none carries the none tag");
  CHECK(opt_unwrap_or(opt_some(7), -5) == 7, "unwrap_or takes the some value");
  // -5 doubles as a legal int32 value, the sentinel collision the tag removes
  CHECK(opt_unwrap_or(opt_none(), -5) == -5, "unwrap_or falls back on none");
  CHECK(opt_map(opt_some(4), dbl).v == 8, "map transforms the some payload");
  CHECK(opt_is_none(opt_map(opt_none(), dbl)), "map skips a none entirely");
  CHECK(opt_and_then(opt_some(12), halve_if_even).v == 6,
        "and_then enters some");
  CHECK(opt_is_none(opt_and_then(opt_some(9), halve_if_even)),
        "and_then can step off the track");
  CHECK(opt_or_else(opt_none(), one).v == DEFAULT_SCALE,
        "or_else rejoins with a fallback");
  CHECK(opt_is_some(opt_or_else(opt_some(3), one)),
        "or_else leaves a some untouched");
  CHECK(config_get("width").v == 12, "config width is some(12)");
  CHECK(opt_unwrap_or(config_get("scale"), DEFAULT_SCALE) == DEFAULT_SCALE,
        "missing key falls back to the default");
  // width 12 + margin 5 = 17, pinned by playground/math-ch27/fixtures.py
  CHECK(opt_unwrap_or(opt_map(config_get("width"), add5), 0) == 17,
        "map then unwrap_or over the config lookup");
  CHECK(opt_unwrap_or(opt_map(config_get("height"), dbl), 0) == 18,
        "height 9 doubles to 18 on the same rails");
  CHECK(opt_and_then(config_get("depth"), halve_if_even).v == 2,
        "depth 4 halves to 2 through and_then");
  CHECK(sizeof(opt_i32) == 8, "the tagged sum stays one pointer wide");
```

listing 286: the checks: raw nullptr against tagged sums, then map and and_then over the config table (source: math/samples/src/Ch27/option.c, lines 93-123)

PITFALL: a none is not an error

Option answers “is there a value”. Result answers “did the work fail”. A missing table entry is a none, a nibble the parser cannot read is an error, and merging the two types puts absence codes into error switches. Keep two types and the switches stay honest.

27.2. result, the error channel

Presence has two states. Failure has as many states as the operation has ways to fail, and each failure wants its own evidence. A result is a sum with an error channel in the second arm: the ok payload is the parsed value, the error payload is a code from a fixed underlying type enum plus one evidence byte, here the string position where the parse died. The C23 sized enum spelling, `enum hex_err : uint8_t`, pins the codes to one byte so the whole error payload fits the padding beside the value.

The fixture is a hex string parser. It fails 3 ways: the empty string, a character that is not a nibble at some index, and an accumulator that crosses the $2^{32} - 1$ frontier. Every failure carries its position as data, so the caller can point at the offending character.

```
typedef enum hex_err : uint8_t {
  HEX_EMPTY = 0,
  HEX_BAD_NIBBLE = 1,
  HEX_TOO_BIG = 2,
} hex_err;

static_assert(HEX_EMPTY != HEX_BAD_NIBBLE && HEX_BAD_NIBBLE != HEX_TOO_BIG,
              "error codes are distinct");

typedef struct {
  uint8_t ok; // 1 ok track, 0 error track
  union {
    uint32_t value; // ok payload
    struct {
      uint8_t code;
      uint8_t at;
    } err; // error payload, position is data
  };
} res32;

static_assert(sizeof(res32) == 8, "one byte of tag buys a typed error channel");

[[nodiscard]] static res32 res_ok(uint32_t v) {
  return (res32){.ok = 1, .value = v};
}
[[nodiscard]] static res32 res_err(uint8_t code, uint8_t at) {
  return (res32){.ok = 0, .err = {.code = code, .at = at}};
}
[[nodiscard]] static bool res_is_ok(res32 r) { return r.ok != 0; }
[[nodiscard]] static bool res_is_err(res32 r) { return r.ok == 0; }
[[nodiscard]] static uint32_t res_unwrap_or(res32 r, uint32_t d) {
  return r.ok ? r.value : d;
}
[[nodiscard]] static res32 res_map(res32 r, uint32_t (*f)(uint32_t)) {
  return r.ok ? res_ok(f(r.value)) : r;
}
```

listing 287: the sized enum codes, the two track sum, and the accessors (source: math/samples/src/Ch27/parsehex.c, lines 8-43)

```c
[[nodiscard]] static res32 parse_hex(const char *s) {
  if (s[0] == '\0') {
    return res_err(HEX_EMPTY, 0);
  }
  uint64_t acc = 0;
  for (uint_fast32_t i = 0; s[i] ≠ '\0'; i++) {
    char c = s[i];
    uint64_t n;
    if (c ≥ '0' && c ≤ '9') {
      n = (uint64_t)(c - '0');
    } else if (c ≥ 'a' && c ≤ 'f') {
      n = (uint64_t)(c - 'a' + 10);
    } else if (c ≥ 'A' && c ≤ 'F') {
      n = (uint64_t)(c - 'A' + 10);
    } else {
      return res_err(HEX_BAD_NIBBLE, (uint8_t)i);
    }
    acc = (acc << 4) | n;
    if (acc > 0xFFFFFFFFull) {
      return res_err(HEX_TOO_BIG, (uint8_t)i);
    }
  }
  return res_ok((uint32_t)acc);
}
```

listing 288: the parse loop, each failure path returns a code plus a position (source: math/samples/src/Ch27/parsehex.c, lines 45-68)

The dry run: parse “100000000”. The first 8 nibbles build 0x10000000, all zeros after a leading 1. The 9th nibble shifts the accumulator to 0x100000000, which is 4294967296, one past the 4294967295 limit, and the loop returns too-big with position 8 before the value is ever truncated. The two-number play sits on that boundary: 4294967295 parses to an ok value, 4294967296 does not exist on the ok track, and the difference arrives as 2 bytes of error payload.

input	track	payload
ffffffff	ok	4294967295
100000000	err	too-big, at 8
0x10	err	bad-nibble, at 1
f0z9	err	bad-nibble, at 2
“”	err	empty, at 0

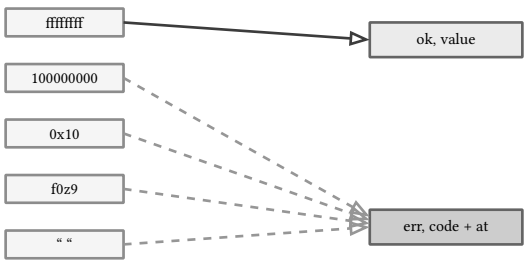


figure 163: five fixed inputs routed onto the two tracks

```
    // the frontier: 4294967295 fits, one more nibble does not
    r = parse_hex("ffffffff");
    CHECK(res_is_ok(r) && r.value == 4294967295u,
          "ffffffff is the last ok value");

    r = parse_hex("100000000");
    CHECK(res_is_err(r) && r.err.code == HEX_TOO_BIG && r.err.at == 8,
          "the ninth nibble overflows, position 8 carried as data");

    r = parse_hex("fffffffff");
    CHECK(res_is_err(r) && r.err.code == HEX_TOO_BIG && r.err.at == 8,
          "all fs but nine of them still overflow at 8");

    r = parse_hex("0x10");
    CHECK(res_is_err(r) && r.err.code == HEX_BAD_NIBBLE && r.err.at == 1,
          "x is not a nibble, fault at index 1");

    r = parse_hex("f0z9");
    CHECK(res_is_err(r) && r.err.code == HEX_BAD_NIBBLE && r.err.at == 2,
          "z fails at index 2 with f0 already consumed");

    r = parse_hex("");
    CHECK(res_is_err(r) && r.err.code == HEX_EMPTY && r.err.at == 0,
          "the empty string is the empty error at 0");
```

listing 289: the frontier checks: the last ok value, the ninth nibble, the fault positions (source: math/samples/src/Ch27/parsehex.c, lines 117-140)

A map over a result squares the ok track and passes the error through with code and position intact, checked in the sample. That single behavior, leave the error alone, is what the whole railway runs on.

27.3. the railway, map and then

Scott Wlaschin named the picture: “Many examples in functional programming assume that you are always on the ‘happy path’. But to create a robust real world application you must deal with validation, logging, network and service errors, and other annoyances”, handled “using a fun and easy-to-understand railway analogy” (fsharpforfunandprofit.com/rop/, fetched 2026-09-22). Two tracks run side by side. Stages sit on the top track and only ever see ok values. Each stage is also a switch: on failure it drops the computation onto the bottom track, which runs straight to the caller and skips every later stage.

Three combinators build this in C, all one line each at heart. `map` transforms the ok payload. `and_then` runs a fallible stage on the ok payload and lets its result pick the track. `or_else` runs a recovery on the error payload and can rejoin the top track. A fourth, the statement macro `RAIL_TRY`, gives the inline C style the same power: bind the value or return the error to the caller without touching it.

```
// map transforms the ok track and never wakes the error track
[[nodiscard]] static rail rail_map(rail r, uint32_t (*f)(uint32_t)) {
  return r.ok ? rail_ok(f(r.value)) : r;
}
// and_then runs a fallible stage on the ok track, or passes the error through
[[nodiscard]] static rail rail_and_then(rail r, rail (*f)(uint32_t)) {
  return r.ok ? f(r.value) : r;
}
// or_else runs a recovery on the error track and rejoins the ok track
[[nodiscard]] static rail rail_or_else(rail r, rail (*f)(uint8_t, uint8_t)) {
  return r.ok ? r : f(r.err.code, r.err.info);
}

// statement level propagation: return the error to the caller, c style
#define RAIL_TRY(var, expr)                                                    \
  do {                                                                         \
    rail _r = (expr);                                                          \
    if (!_r.ok) {                                                              \
      return _r;                                                               \
    }                                                                          \
    (var) = _r.value;                                                          \
  } while (0)
```

listing 290: the combinator trio plus the try propagation macro (source: math/samples/src/Ch27/railway.c, lines 40-61)

The pipeline fixture: stage 1 parses a hex string, stage 2 demands a 12 bit 4-aligned address and says which demand failed, stage 3 packs a check byte, the value xor 0xA5, above the value. Failure is injected by the inputs themselves, no randomness anywhere.

```
// stage 1: hex string to value, the same loop as parsehex.c
static rail stage1_parse(const char *s) {
  c1++;
  if (s[0] == '\0') {
    return rail_fail(RAIL_EMPTY, 0);
  }
  uint64_t acc = 0;
  for (uint_fast32_t i = 0; s[i] != '\0'; i++) {
    char ch = s[i];
    uint64_t n;
    if (ch >= '0' && ch <= '9') {
      n = (uint64_t)(ch - '0');
    } else if (ch >= 'a' && ch <= 'f') {
      n = (uint64_t)(ch - 'a' + 10);
    } else if (ch >= 'A' && ch <= 'F') {
      n = (uint64_t)(ch - 'A' + 10);
    } else {
      return rail_fail(RAIL_BAD_NIBBLE, (uint8_t)i);
    }
    acc = (acc << 4) | n;
    if (acc > 0xFFFFFFFFull) {
      return rail_fail(RAIL_RANGE, (uint8_t)i);
    }
  }
  return rail_ok((uint32_t)acc);
}
```

listing 291: stage 1, the parser from the previous section returning rails (source: math/samples/src/Ch27/railway.c, lines 63-88)

```
// stage 2: a value must be a 12 bit 4-aligned address, errors say why not
static rail stage2_validate(uint32_t v) {
  c2++;
  if (v > 0xFFFu) {
    return rail_fail(RAIL_RANGE, (uint8_t)(v >> 12));
  }
  if (v % 4u != 0) {
    return rail_fail(RAIL_ALIGN, (uint8_t)(v % 4u));
  }
  return rail_ok(v);
}

// stage 3: encode as check byte in the high half, value in the low half
static rail stage3_encode(uint32_t v) {
  c3++;
  return rail_ok(((v ^ 0xA5u) << 16) | v);
}

[[nodiscard]] static rail pipeline(const char *s) {
  return rail_and_then(rail_and_then(stage1_parse(s), stage2_validate),
                       stage3_encode);
}

static rail try_pipeline(const char *s) {
  uint32_t v, w;
  RAIL_TRY(v, stage1_parse(s));
  RAIL_TRY(w, stage2_validate(v));
  return stage3_encode(w);
}
```

listing 292: stage 2 with typed refusals, stage 3, and the two driver forms (source: math/samples/src/Ch27/railway.c, lines 90-118)

The dry run: pipeline("13"). Stage 1 parses 13 hex to 19 and returns ok. Stage 2 computes 19 mod 4 = 3, refuses alignment, and returns an error carrying residue 3. Stage 3 never runs. The counters prove it: the sample resets 3 stage counters before each scenario and checks them after, and the align failure leaves them at 1, 1, 0. The same counters read 1, 0, 0 for the input "zz", whose parse error means stage 2 never wakes up either.

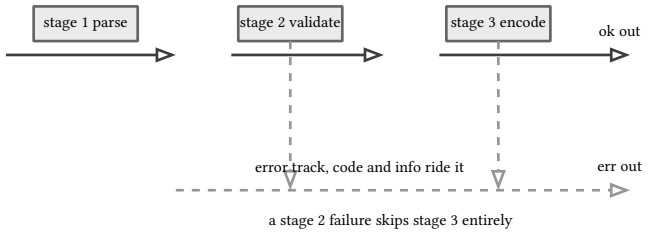


figure 164: the two track railway: stages on top, the dashed error track short circuiting stage 3

input	counters c1 c2 c3	outcome
a8	1 1 1	ok, encode 852136
0abc	1 1 1	ok, encode 169413308
13	1 1 0	err align, residue 3
1234	1 1 0	err range, excess 1
zz	1 0 0	err bad-nibble, at 0

```
int main(void) {
  // values precomputed by playground/math-ch27/fixtures.py:
  // a8 = 168 aligned ok, encode (13 << 16) | 168 = 852136
  rail r;

  reset_counters();
  r = pipeline("a8");
  CHECK(rail_is_ok(r) && r.value == 852136u, "a8 rides all three stages");
  CHECK(c1 == 1 && c2 == 1 && c3 == 1,
        "happy path runs stages 1, 2, 3 once each");

  // 0abc = 2748, aligned, check byte 2585, encode 169413308
  reset_counters();
  r = pipeline("0abc");
  CHECK(rail_is_ok(r) && r.value == 169413308u, "0abc encodes to 169413308");

  // 13 hex is 19, 19 % 4 leaves residue 3, stage 3 must not run
  reset_counters();
  r = pipeline("13");
  CHECK(!rail_is_ok(r) && r.err.code == RAIL_ALIGN && r.err.info == 3,
        "19 is not 4-aligned, residue 3 rides the error track");
  CHECK(c1 == 1 && c2 == 1 && c3 == 0, "align failure: stage 3 never ran");

  // 1234 hex is 4660, above 0xfff by one 4-bit step
  reset_counters();
  r = pipeline("1234");
  CHECK(!rail_is_ok(r) && r.err.code == RAIL_RANGE && r.err.info == 1,
        "4660 out of 12 bit range, excess 1 carried");
  CHECK(c3 == 0, "range failure short circuits stage 3 too");

  // a parse failure means stage 2 never even wakes up
  reset_counters();
  r = pipeline("zz");
  CHECK(!rail_is_ok(r) && r.err.code == RAIL_BAD_NIBBLE && r.err.info == 0,
        "zz fails parsing at index 0");
  CHECK(c2 == 0 && c3 == 0, "parse failure short circuits stages 2 and 3");
```

listing 293: the scenarios: values pinned, counters proving the short circuit on every failure (source: math/samples/src/Ch27/railway.c, lines 141-176)

Recovery is the same picture read backwards. `rail_or_else(pipeline("13"), clamp_recover)` hands the align error to a recovery that encodes address 0, and the result is an ok rail worth 10813440 with stage 3 having run exactly once. Wlaschin's own page carries the caveat, "please don't take it to extremes", and the caveat is the design rule: the error track pays for itself where failures are ordinary, and it is dead weight where a failure is a bug that should stop the program.

VERIFY: the counter proof

Short circuit claims are cheap to state and easy to fake. The sample makes each stage increment a static counter, resets all 3 before every scenario, and CHECKs the triple after the run, so "stage 3 never ran" is 3 compared integers, not a diagram assertion.

27.4. contracts the compiler can check

A railway only works if callers hold the rail. C23 has one attribute for that, and cppreference states its exact strength: "from a discarded-value expression other than a cast to void, a function declared nodiscard is called … the

compiler is encouraged to issue a warning", with the same rule for a function returning a struct, union, or enum declared nodiscard (en.cppreference.com/w/c/language/attributes/nodiscard, fetched 2026-09-22). Encouraged is the honest word. The attribute is a request to the diagnostic, worth nothing at runtime and worth everything once the build upgrades it: this corpus compiles every sample under -Werror, and chapter book 1, ch 7 (c23: the language, the machine, and the cloud, llvm: the architecture of the toolchain) pins the exact diagnostic, "declared with 'nodiscard' attribute [-Werror,-Wunused-result]", on a call site that ignored one.

Every constructor and accessor in this chapter's samples carries [[nodiscard]], and `option.c` opens with a preprocessor check that pins the attribute value itself, `__has_c_attribute(nodiscard)` equal to 202003L, so the file refuses to build where the contract cannot be asked for. The runtime side of the contract is the tag: reading the union payload of a result without checking the tag first is the one discipline the compiler cannot see, which is why the samples funnel every read through accessors that check.

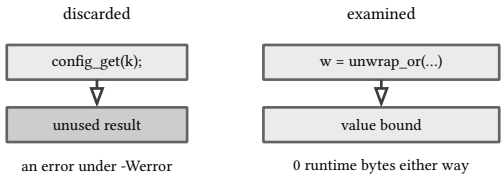


figure 165: two call sites, one diagnostic apart

The static side gets contracts too. The samples `static_assert` the 8 byte width of every sum, tag, padding, and 4 byte payload together the width of one pointer, and `parsehex.c` asserts the error codes distinct. These are compile time facts about the channel, checked once per build. The two-number play for this section: 2 call sites, 1 attribute, 0 runtime bytes, and the difference between them is one build failure.

NOTE: nodiscard is a warning at heart

The standard encourages, it does not require. A build without -Werror or an explicit cast to void silences the diagnostic and the rail is still dropped. Treat the attribute as the compiler joining the contract, and treat the unwrapped read of a union payload as the contract only the tag can enforce.

27.5. cleanup ownership on both tracks

Error channels carry values. Resources are different: a file, a lock, a buffer want release on every path, and the failure paths are exactly the ones hand written cleanup forgets. C23 does not have defer. It arrives from technical specification 25755, and clang 23.1.1 ships it as `stddefer.h`, which enables the `defer` keyword under the `-fdefer-ts` driver flag the sample gate carries. Chapter book 1, ch 6 (c23: the language, the machine, and the cloud, the c23 library) pins the header, the flag, and the unwind orders on this machine.

The ownership rule for the railway: register the release with defer at the point of acquisition, inside the same scope that owns the resource. Then every exit from that scope, including the failure returns, unwinds the registrations in reverse order, and the reverse order is the correct one because later resources depend on earlier ones.

```
// three resources in a row, one defer registered per successful acquisition
static int open_scene(char fail_id) {
  if (!acquire('a', fail_id)) {
    return 1;
  }
  defer mark('a');
  if (!acquire('b', fail_id)) {
    return 2;
  }
  defer mark('b');
  if (!acquire('c', fail_id)) {
    return 3;
  }
  defer mark('c');
  return 0;
}

// a scope local resource d lives and dies inside its block, released before
// b and c are even requested
static int open_layered(char fail_id) {
  if (!acquire('a', fail_id)) {
    return 1;
  }
  defer mark('a');
  {
    if (!acquire('d', fail_id)) {
      return 2;
    }
    defer mark('d');
  }
  if (!acquire('b', fail_id)) {
    return 3;
  }
  defer mark('b');
  if (!acquire('c', fail_id)) {
    return 4;
  }
  defer mark('c');
  return 0;
}
```

listing 294: the scene openers: one defer per acquisition, plus a scope local resource d that dies at its block end (source: math/samples/src/Ch27/cleanup.c, lines 52-91)

The dry run: open_scene with the failure injected at c. Acquire a, register release a. Acquire b, register release b. Acquire c is refused, the function returns 3, and the two registered defers fire in reverse: b, then a. The recorder collects the string “ba”. On the happy path the same mechanism collects “cba”. The invariant is checked per path: acquired count equals released count, 3 and 3 on the happy path, 2 and 2 on the failure at c, 0 and 0 when the very first acquisition is refused.

The layered scene adds the scope rule. Resource d lives inside a block, its defer registered in that block, so d is released when the block closes, before b and c are even requested. The full happy trace is “dcba”, and when c later refuses, the trace is “dba”: d already gone at its block end, b and a unwound at the return. The paired scene shows the compound form, one defer wrapping a two step teardown, flush then close, firing as a unit before the earlier registrations unwind.

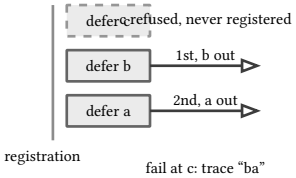


figure 166: the defer stack: registration order down the left, reverse unwind out the right, the refused c shown dashed

```
// one resource e with a two step teardown, flush then close, as one
// compound defer: both steps fire as a unit, before a unwinds
static int open_paired(char fail_id) {
  if (!acquire('a', fail_id)) {
    return 1;
  }
  defer mark('a');
  if (!acquire('e', fail_id)) {
    return 2;
  }
  defer {
    mark('e');
    mark('f');
  }
  return 0;
}
```

listing 295: the compound defer, a two step teardown registered as one unit (source: math/samples/src/Ch27/cleanup.c, lines 93-108)

```
int main(void) {
  int rc;

  reset_scene();
  rc = open_scene('\0');
  CHECK(rc == 0, "no injected failure, open_scene succeeds");
  CHECK(trace_equals("cba"), "normal path unwinds c, b, a");
  CHECK(acquired_cnt == 3 && released_cnt == 3,
        "3 acquired, 3 released, the invariant holds");

  reset_scene();
  rc = open_scene('b');
  CHECK(rc == 2, "failure at b returns b's ordinal");
  CHECK(trace_equals("a"), "only a was acquired, only a unwinds");
  CHECK(acquired_cnt == released_cnt,
        "acquired equals released on the fail path");

  reset_scene();
  rc = open_scene('c');
  CHECK(rc == 3, "failure at c returns c's ordinal");
  CHECK(trace_equals("ba"), "a and b acquired, released b then a");
  CHECK(acquired_cnt == 2 && released_cnt == 2,
        "2 acquired, 2 released on the failure at c");

  reset_scene();
  rc = open_scene('a');
  CHECK(rc == 1, "failure at the first acquisition returns 1");
  CHECK(trace_equals(""), "nothing was acquired, nothing unwinds");
  CHECK(acquired_cnt == 0 && released_cnt == 0, "0 acquired, 0 released");

  reset_scene();
  rc = open_layered('\0');
  CHECK(rc == 0, "layered scene succeeds");
  CHECK(trace_equals("dcba"), "d unwinds at its block end, then c, b, a");

  reset_scene();
  rc = open_layered('d');
  CHECK(rc == 2, "failure at the scope local d returns 2");
  CHECK(trace_equals("a"), "d never existed, only a unwinds");

  reset_scene();
  rc = open_layered('b');
  CHECK(rc == 3, "failure at b, after the d block closed");
  CHECK(trace_equals("da"), "d already released in scope, then a at return");

  reset_scene();
  rc = open_layered('c');
  CHECK(rc == 4, "failure at c returns 4");
  CHECK(trace_equals("dba"), "d in scope, b and a at return: d, b, a");

  reset_scene();
  rc = open_paired('\0');
  CHECK(rc == 0, "paired scene succeeds");
  CHECK(trace_equals("efa"), "compound defer fires e then f, then a unwinds");
```

listing 296: the path checks: pinned release traces and the acquired equals released invariant (source: math/samples/src/Ch27/cleanup.c, lines 122-175)

The interaction with the railway is the point. A stage that acquires and returns a rail on failure does not need a cleanup branch per error, the defers already cover every return. What defer cannot do is span the tracks: a resource acquired inside stage 2 and needed by stage 3 wants ownership passed through the ok payload, and that pattern, bracketing a fallible computation with acquisition and release, is exactly the shape the next chapter names and generalizes.

NOTE: defer is a technical specification, not c23

The keyword comes from TS 25755 and needs `-fdefer-ts` with clang 23.1.1's `stddefer.h`. The corpus gate carries the flag, so these samples build in the verified lane, and the corpus pins the unwind orders in its own c book. Code aimed at plain c23 compilers needs the goto cleanup idiom instead.

27.6. where the railway goes

This chapter built the channel in one concrete shape, a uint32 payload with a byte of code and a byte of evidence. Nothing in map, and_then, or or_else looked at those types. Each combinator is one tag test plus one function call, and the same three definitions carry an option of doubles, a result of matrices, or a parse forest. Three combinators, one macro, and two chapters of generalization ahead.

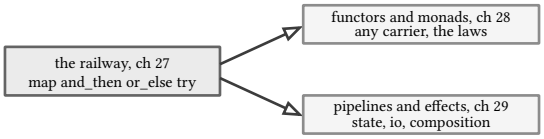


figure 167: the combinator core and the two chapters that generalize it

Chapter book 2, ch 28 (mathematics for programmers, functors and monads in c) peels the types off these combinators, states the laws they satisfy, and rebuilds them over any carrier type, which is where option and result stop being a convention and start being an instance of a functor and a monad. Chapter book 2, ch 29 (mathematics for programmers, pipelines and effects) stops treating stages as pure functions, threading state and effects through the same tracks, and the capstone game assembles the whole machinery into a program whose failure paths are checked the same way this chapter checked a parser's.

sources: cppreference NULL, en.cppreference.com/w/c/types/NULL, cppreference nullptr, en.cppreference.com/w/c/language/nullptr, cppreference attribute nodiscard, en.cppreference.com/w/c/language/attributes/nodiscard, all fetched 2026-09-22. Scott Wlaschin, "Railway Oriented Programming", fsharpforfunandprofit.com/rop/, fetched 2026-09-22, the two-track analogy and the happy-path blurb quoted, the extremity caveat his. Defer ground truth from the corpus's own pinned coverage, TS 25755 and clang 23.1.1 `stddefer.h` under `-fdefer-ts`, c-os-cloud ch 06, and the nodiscard -Werror=unused-result diagnostic, c-os-cloud ch 07. mml-book draft 2024-01-15: no mapped chapter for functional error channels, none cited. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch27`, 78 checks in chapter 27 of the math suite, format and asan clean.

28. functors and monads in c

Chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) gave C a tagged product, a struct with a discriminant and a payload, and chapter book 2, ch 27 (mathematics for programmers, option, result, and the railway) ran those carriers as a railway where a failing stage hands its error forward untouched and the next stage never executes. Both chapters leaned on one unstated promise: threading partial functions through tagged carriers behaves the same however you group the steps. This chapter states that promise as three equations and turns it into checks. The moves are: the functor and monad interface stated against Wadler's notation, map and bind over option with the three laws pinned on fixed integers, the same shape over result with typed errors and a counter proof that the failing path skips stages, the list monad where bind becomes concatenation and nondeterminism becomes exact enumeration, one bind spelling dispatched over several carrier types by a macro that emulates the higher-kinded type and moves ownership under the tag, and closures as context structs whose purity is exactly what the laws buy. Every behavioral claim below is one of the 61 checks in the 4 samples of chapter 28 or a bibliographic line quoted from a canonical source fetched 2026-09-22. The dsa book stays algorithm-first and never names a monad; here the theory is the payload.

28.1. the shape railway generalized

A functor is a carrier M with a map: given a plain function f from T to U, map applies f to the payload and leaves the effect alone. A monad adds two primitives to that. The constructor unit injects a plain value into the carrier: some for option, ok for result, the one element list for lists. The combinator bind threads a carrier through a kleisli arrow, a function k that already returns a carrier: bind runs k on the payload when m carries one and reproduces the effect of m otherwise. In notation, where >>= spells bind:

$$\mathrm{unit}(a) \gg\!= k = k(a)$$

$$m \gg\!= \mathrm{unit} = m$$

$$(m \gg\!= k) \gg\!= h = m \gg\!= (\lambda x.k(x) \gg\!= h)$$

Read them as: injecting then binding is just calling k, binding unit changes nothing, and regrouping a chain of binds is free. The first law is what makes unit an identity on the left, the second what makes it an identity on the right, the third what lets you flatten a pipeline into one arrow or split it into stages without changing behavior. All three date to the standard formulation of Moggi's computational metalanguage carried into functional programming by Wadler, cited below by name and edition.

NOTE: WHERE THE LAWS COME FROM

Philip Wadler, "Monads for functional programming", Marktoberdorf Summer School on Program Design Calculi, NATO ASI Series F, Volume 118, Springer, August 1992, also in J. Jeuring and E. Meijer, editors, Advanced Functional Programming, LNCS 925, Springer, 1995. That paper states the three laws in exactly this unit and bind form. The venue lines were verified at the author's page fetched 2026-09-22; the paper text is cited by name and edition and the laws are restated here in our own notation.

The notation maps onto C with nothing exotic: constructors are inline functions returning compound literals, a lambda is a static function whose signature is T to carrier, and >>= is a function taking the carrier and a function pointer.

notation	c realization	this chapter
$\mathrm{unit}(a)$	constructor: some(a), res_ok(a), ilist_unit(a)	option, result, list
$\lambda x.k(x)$	static function $T \to M$	k_add3, stage_a, k_pair10
$m \gg\!= k$	bind(m, k) by carrier	opt_bind, res_bind, ilist_bind

notation	c realization	this chapter
$M(T)$	typedef struct with tag and payload	opt, res, ilist

One consequence worth pinning before any code: map needs no independent definition. Applying f under the tag is binding the arrow that injects the result of f, and the map equals bind check of this chapter's first sample proves the two agree, 6 through 3x+1 landing on some 19 both ways. The section's two numbers: 2 primitives generate the whole interface and 3 equations police them.

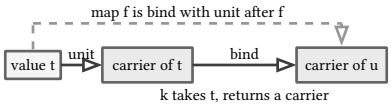


figure 168: the general shape: unit injects a value, bind threads a carrier through a kleisli arrow, the three laws pin the contract

28.2. map and bind over option

The carrier is the two field struct from the railway chapter: a bool tag and an int payload, payload unread when the tag is false. Equality, map, and bind fit in 19 lines, and the shape of bind is the whole story: the empty case reproduces the effect, the full case delegates to k.

```
static bool opt_eq(opt a, opt b) {
  if (a.has != b.has)
    return false;
  return !a.has || a.v == b.v;
}

// functor: lift f : int -> int into opt -> opt
[[nodiscard]] static opt opt_map(opt m, int (*f)(int)) {
  if (!m.has)
    return none();
  return some(f(m.v));
}

// monad: thread m through a kleisli arrow k : int -> opt
[[nodiscard]] static opt opt_bind(opt m, opt (*k)(int)) {
  if (!m.has)
    return none();
  return k(m.v);
}
```

listing 297: the option carrier: equality, map lifting a plain function, bind threading a kleisli arrow (source: math/samples/src/Ch28/option.c, lines 33-51)

The kleisli arrows are plain static functions, including one partial arrow that returns none for nonpositive inputs, and one counted arrow so a check can prove a bind really did not call it.

```c
static opt k_add3(int x) { return some(x + 3); }
static opt k_unit(int x) { return some(x); }
static opt k_mul10(int x) { return some(10 * x); }

// a partial arrow: doubles positive inputs, none otherwise
static opt k_double_pos(int x) {
  if (x > 0)
    return some(2 * x);
  return none();
}

// h is counted, so checks can prove it never runs on the failing path
static int h_calls = 0;
static opt h_sub5(int x) {
  h_calls++;
  return some(x - 5);
}

// the right side of associativity spelled out: x -> k(x) >>= h
static opt assoc_rhs_mul10(int x) { return opt_bind(k_mul10(x), h_sub5); }
static opt assoc_rhs_dbl(int x) { return opt_bind(k_double_pos(x), h_sub5); }

// map derived from bind: unit after f
static opt k_after_3x1(int x) { return some(f_3x1(x)); }
```

listing 298: the fixture arrows: total, partial, and counted kleisli functions (source: math/samples/src/Ch28/option.c, lines 57-80)

The dry run: the left identity fixture is some 7 through k = +3, and both sides land on some 10, one by injecting then binding, one by just calling k(7). The right identity fixture is some 5 and none through the unit arrow, both unchanged. The associativity fixture is some 2 through k = 10x then h = −5: grouped left, 2 binds to some 20 and 20 binds to some 15; grouped right, x = 2 runs k(2) = some 20 then h(20) = some 15 inside the arrow, and the outer bind lands on the same some 15. The failing fixture is some −3 through the partial arrow: k(-3) is none, so both groupings are none and the h counter reads 0 invocations, while the success fixture 4 through the same pair reads exactly 1.

law	fixture	left side	right side
left identity	some 7, k = +3	some 10	some 10
right identity	some 5	some 5	some 5
right identity	none	none	none
associativity	some 2, k = 10x, h = −5	some 15	some 15
associativity	none	none	none
associativity	some −3, partial k	none, h ran 0	none, h ran 0

```
// law 1, left identity: bind(unit(a), k) == k(a); k = +3, a = 7
CHECK(opt_eq(opt_bind(some(7), k_add3), k_add3(7)),
      "left identity on some 7");
CHECK(opt_bind(some(7), k_add3).v == 10, "left identity payload is 10");

// law 2, right identity: bind(m, unit) == m on both tags
CHECK(opt_eq(opt_bind(some(5), k_unit), some(5)), "right identity on some 5");
CHECK(opt_eq(opt_bind(none(), k_unit), none()), "right identity on none");

// law 3, associativity: (m >>= k) >>= h == m >>= (x -> k(x) >>= h)
// k = *10, h = -5, m = some 2: 2*10 = 20, 20-5 = 15, both sides some 15
opt lhs = opt_bind(opt_bind(some(2), k_mul10), h_sub5);
opt rhs = opt_bind(some(2), assoc_rhs_mul10);
CHECK(opt_eq(lhs, rhs) && opt_eq(lhs, some(15)), "associativity on some 2");

opt ln = opt_bind(opt_bind(none(), k_mul10), h_sub5);
opt rn = opt_bind(none(), assoc_rhs_mul10);
CHECK(opt_eq(ln, none()) && opt_eq(rn, none()), "associativity on none");

// k fails: -3 is not positive, so h never runs on either side
int before = h_calls;
opt lf = opt_bind(opt_bind(some(-3), k_double_pos), h_sub5);
opt rf = opt_bind(some(-3), assoc_rhs_dbl);
CHECK(opt_eq(lf, none()) && opt_eq(rf, none()) && h_calls == before,
      "associativity with failing k, h never runs");

// success path: k(4) = some 8, h runs exactly once, 8-5 = 3
int before2 = h_calls;
opt s = opt_bind(opt_bind(some(4), k_double_pos), h_sub5);
CHECK(opt_eq(s, some(3)) && h_calls == before2 + 1, "h runs once on success");
```

listing 299: the three laws as checks: fixed integers on both sides, the counter proving the skipped call (source: math/samples/src/Ch28/option.c, lines 92-121)

The two numbers of the section: h runs 1 time on the success path and 0 times on the failing path through the identical bind chain.

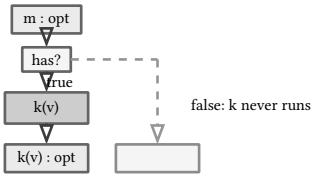


figure 169: bind over option: the tag decides, k runs only under some

VERIFY: RUN THE LAWS

The laws are not prose here, the three of them run as 7 of the 14 checks in option.c. Rerun with `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch28` and read the ok lines: each law check prints the exact integers it compared.

28.3. result with typed errors

The result carrier from the railway chapter carries either a long payload or a typed error tag in an anonymous union, with the enum pinned to a one byte underlying type by static_assert. map and bind have exactly the option shape with one addition: the error that passes through is the error that comes out, byte for byte.

```c
// functor: f runs only on the payload, errors pass through untouched
[[nodiscard]] static res res_map(res m, long (*f)(long)) {
  if (!m.ok)
    return m;
  return res_ok(f(m.val));
}

// monad: the failing case returns m itself, k never runs
[[nodiscard]] static res res_bind(res m, res (*k)(long)) {
  if (!m.ok)
    return m;
  return k(m.val);
}
```

listing 300: map and bind over result: the failing case returns m itself (source: math/samples/src/Ch28/result.c, lines 52-64)

The pipeline fixture is three counted stages: a triples rejecting inputs above 400 with E_RANGE, b subtracts 7 rejecting zero with E_ZERO, c squares rejecting negatives with E_NEG.

```c
// three pipeline stages, each a kleisli arrow long -> res, each counted
static int calls_a = 0, calls_b = 0, calls_c = 0;

static res stage_a(long x) { // triple, rejecting wide inputs
  calls_a++;
  if (x > 400)
    return res_err(E_RANGE);
  return res_ok(3 * x);
}

static res stage_b(long x) { // shift down by 7, rejecting zero
  calls_b++;
  if (x == 0)
    return res_err(E_ZERO);
  return res_ok(x - 7);
}

static res stage_c(long x) { // square, rejecting negatives
  calls_c++;
  if (x < 0)
    return res_err(E_NEG);
  return res_ok(x * x);
}

static res pipeline(long x) {
  return res_bind(res_bind(res_bind(res_ok(x), stage_a), stage_b), stage_c);
}
```

listing 301: three kleisli stages with call counters and the chained pipeline (source: math/samples/src/Ch28/result.c, lines 91-117)

The dry run: 12 enters, a makes 36, b makes 29, c makes 841, and the counter triples read (1, 1, 1). The input 500 dies at a with E_RANGE and the counters read (1, 0, 0): b and c never ran. The input 0 triples to 0, b rejects it with E_ZERO, c never ran. The input 2 makes 6 then −1, and c rejects the negative with E_NEG after all three stages ran. A final check binds an E_RANGE error through stages b and c directly and reads the tag out the far end unchanged with all counters still flat.

input	a: 3x	b: −7	c: square	outcome, counters
12	36	29	841	ok 841, (1,1,1)

input	a: 3x	b: −7	c: square	outcome, counters
500	E_RANGE	skipped	skipped	E_RANGE, (1,0,0)
0	0	E_ZERO	skipped	E_ZERO, (1,1,0)
2	6	−1	E_NEG	E_NEG, (1,1,1)

```
    // the pipeline: 12 -> *3 -> 36 -> -7 -> 29 -> square -> 841
    int a0 = calls_a, b0 = calls_b, c0 = calls_c;
    res r12 = pipeline(12);
    CHECK(r12.ok && r12.val == 841 && calls_a == a0 + 1 && calls_b == b0 + 1 &&
              calls_c == c0 + 1,
          "pipeline 12 reaches 841, all three stages ran");

    // 500 is wide: stage a rejects, stages b and c never run
    int a1 = calls_a, b1 = calls_b, c1 = calls_c;
    res r500 = pipeline(500);
    CHECK(!r500.ok && r500.err == E_RANGE && calls_a == a1 + 1 && calls_b == b1 &&
              calls_c == c1,
          "pipeline 500 fails at a with E_RANGE, b and c skipped");

    // 0 triples to 0: stage b rejects, stage c never runs
    int a2 = calls_a, b2 = calls_b, c2 = calls_c;
    res r0 = pipeline(0);
    CHECK(!r0.ok && r0.err == E_ZERO && calls_a == a2 + 1 && calls_b == b2 + 1 &&
              calls_c == c2,
          "pipeline 0 fails at b with E_ZERO, c skipped");

    // 2 triples to 6, shifts to -1: stage c rejects the negative
    int a3 = calls_a, b3 = calls_b, c3 = calls_c;
    res r2 = pipeline(2);
    CHECK(!r2.ok && r2.err == E_NEG && calls_a == a3 + 1 && calls_b == b3 + 1 &&
              calls_c == c3 + 1,
          "pipeline 2 fails at c with E_NEG after all stages ran");

    // the first error survives further binds verbatim
    int a4 = calls_a, b4 = calls_b, c4 = calls_c;
    res carried = res_bind(res_bind(res_err(E_RANGE), stage_b), stage_c);
    CHECK(res_eq(carried, res_err(E_RANGE)) && calls_a == a4 && calls_b == b4 &&
              calls_c == c4,
          "an error rides to the end unchanged, no stage runs");
```

listing 302: the propagation proof: per stage counters pin exactly which stages ran (source: math/samples/src/Ch28/result.c, lines 159-192)

The two numbers: 841 against E_RANGE on the same three stage chain, and downstream counters (1,1,1) against (0,0).

PITFALL: THE SHORT CIRCUIT IS LOAD BEARING

Returning m itself on the failing case, rather than calling k or inventing a default error, is what makes right identity hold for errors and what carries the first error's tag to the end verbatim. A bind that "recovers" inside the combinator breaks both checks: the tag changes and the counter proves a stage ran that the railway promised to skip.

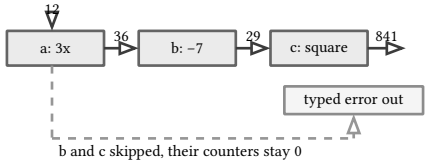


figure 170: the result railway: an error at any stage drops to the dashed rail and later stages never run

28.4. the list monad

For lists the effect is plurality: a list of T stands for every value it holds. unit is the one element list, map applies f pointwise, and bind runs k on every element and concatenates the outcomes in order. That last clause is where the list monad earns its other name, the nondeterminism monad: a kleisli arrow that returns two outcomes per input doubles the possibilities, and bind is the enumeration.

```
// functor: apply f to every element
[[nodiscard]] static ilist ilist_map(ilist m, int (*f)(int)) {
  ilist out = ilist_empty();
  for (int i = 0; i < m.n; i++)
    out.xs[out.n++] = f(m.xs[i]);
  return out;
}

// monad: run k on every element and concatenate the outcomes in order
[[nodiscard]] static ilist ilist_bind(ilist m, ilist (*k)(int)) {
  ilist out = ilist_empty();
  for (int i = 0; i < m.n; i++) {
    ilist part = k(m.xs[i]);
    for (int j = 0; j < part.n; j++)
      out.xs[out.n++] = part.xs[j];
  }
  return out;
}
```

listing 303: map pointwise and bind as ordered concatenation over a fixed capacity array (source: math/samples/src/Ch28/list.c, lines 54-71)

The arrows: k_pair10 branches every input into the pair {x, x + 10}, k_die_b pairs a fixed die face with every face 1 to 3 of a second die coded as 10a + b, and k_keep_sum3 is the partial arrow that keeps a pair only when its two faces sum to a multiple of 3, which is filtering expressed as bind.

```c
// the branching arrow: every input yields two outcomes
static ilist k_pair10(int x) {
  int pair[2] = {x, x + 10};
  return ilist_of(2, pair);
}

static ilist k_double(int x) { return ilist_unit(2 * x); }
static ilist k_none(int x) {
  (void)x;
  return ilist_empty();
}

// the right side of associativity spelled out: x -> k(x) >>= h
static ilist assoc_rhs(int x) { return ilist_bind(k_pair10(x), k_double); }

// die b has fixed faces 1..3: pair a with every face, coded 10a+b
static ilist k_die_b(int a) {
  ilist out = ilist_empty();
  for (int b = 1; b <= 3; b++)
    out.xs[out.n++] = 10 * a + b;
  return out;
}

// keep pairs whose two faces sum to a multiple of 3
static ilist k_keep_sum3(int p) {
  if ((p / 10 + p % 10) % 3 == 0)
    return ilist_unit(p);
  return ilist_empty();
}
```

listing 304: the fixture arrows: branching, dice enumeration, and filtering as a partial arrow (source: math/samples/src/Ch28/list.c, lines 80-108)

The dry run: the list {2, 3} through k_pair10 concatenates {2, 12} then {3, 13} into {2, 12, 3, 13}, order pinned. Associativity with h = double: grouped left, the branch list maps pointwise to {4, 24, 6, 26}; grouped right, each x binds through its own k then h, producing {4, 24} then {6, 26}, and the concatenation is the same list. The dice fixture enumerates all 9 ordered pairs of two three faced dice, 11 through 33, and the sum filter keeps exactly 12, 21, and 33.

```
// law 1, left identity: bind(unit(4), k) == k(4) == {4,14}
int pair4[2] = {4, 14};
CHECK(ilist_eq(ilist_bind(ilist_unit(4), k_pair10), k_pair10(4)) &&
          ilist_eq(k_pair10(4), ilist_of(2, pair4)),
      "left identity on unit 4");

// law 2, right identity: bind(m, unit) == m, empty included
CHECK(ilist_eq(ilist_bind(ilist_of(3, five), k_unit), ilist_of(3, five)),
      "right identity on 2,3,5");
CHECK(ilist_eq(ilist_bind(ilist_empty(), k_unit), ilist_empty()),
      "right identity on the empty list");

// bind concatenates in order: {2,3} branches to {2,12,3,13}
int two[2] = {2, 3};
int branched[4] = {2, 12, 3, 13};
CHECK(ilist_eq(ilist_bind(ilist_of(2, two), k_pair10), ilist_of(4, branched)),
      "bind flattens branches in order");

// law 3, associativity: k = pair10, h = double; both sides {4,24,6,26}
int assoc[4] = {4, 24, 6, 26};
ilist lhs = ilist_bind(ilist_bind(ilist_of(2, two), k_pair10), k_double);
ilist rhs = ilist_bind(ilist_of(2, two), assoc_rhs);
CHECK(ilist_eq(lhs, rhs) && ilist_eq(lhs, ilist_of(4, assoc)),
      "associativity on 2,3, both sides 4,24,6,26");

int one2[2] = {1, 2};
CHECK(ilist_eq(ilist_bind(ilist_of(2, one2), k_none), ilist_empty()),
      "bind with the empty arrow is empty");

// the nondeterminism fixture: two three faced dice enumerate 9 pairs
int die[3] = {1, 2, 3};
ilist pairs = ilist_bind(ilist_of(3, die), k_die_b);
int want9[9] = {11, 12, 13, 21, 22, 23, 31, 32, 33};
CHECK(pairs.n == 9 && pairs.xs[0] == 11 && pairs.xs[8] == 33,
      "two dice enumerate 9 ordered pairs, 11 through 33");
CHECK(ilist_eq(pairs, ilist_of(9, want9)), "the pair list is exactly pinned");

// filtering is bind with a partial arrow: 3 of the 9 pairs survive
ilist kept = ilist_bind(pairs, k_keep_sum3);
int want3[3] = {12, 21, 33};
CHECK(ilist_eq(kept, ilist_of(3, want3)),
      "pairs summing to a multiple of 3 are 12,21,33");
```

listing 305: the laws with array equality, then the 9 pair enumeration and the 3 survivors (source: math/samples/src/Ch28/list.c, lines 124-165)

The two numbers: 9 pairs enumerated from 3 times 3 faces, 3 survive the filter.

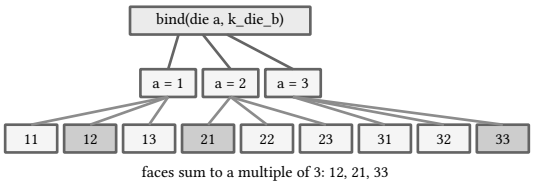


figure 171: the list monad as a tree: two dice enumerate 9 ordered pairs, the 3 darkened leaves survive the sum filter

28.5. one spelling, many types

Everything above hardcodes the payload type per file, which is honest C but repetitive, and the repetition hides the interesting claim: map and bind mean the same thing over int, over a rational, over any payload. C has no type constructors over types, so the corpus answer is two moves. A macro stamps the carrier, unit, map, bind, equality, and a take for a named payload type, and `_Generic` reads the carrier type off an expression at compile time to pick the stamped combinator, one BIND spelling for every instantiation.

```
// the macro that emulates the higher-kinded M<T>: NAME becomes the
// carrier, T the payload, and the stamped functions are the unit, map,
// and bind of the option monad over T. the instantiation supplies
// NAME_payload_eq first.
#define DEFINE_MONAD(NAME, T)                                          \
  typedef struct {                                                     \
    bool has;                                                          \
    T v;                                                               \
  } NAME;                                                              \
  static inline NAME NAME##_some(T v) {                                \
    NAME m = {.has = true, .v = v};                                    \
    return m;                                                          \
  }                                                                    \
  static inline NAME NAME##_none(void) {                               \
    NAME m = {.has = false, .v = (T){0}};                              \
    return m;                                                          \
  }                                                                    \
  static inline bool NAME##_eq(NAME a, NAME b) {                       \
    if (a.has != b.has)                                                \
      return false;                                                    \
    return !a.has || NAME##_payload_eq(a.v, b.v);                      \
  }                                                                    \
  static inline NAME NAME##_map(NAME m, T (*f)(T)) {                   \
    if (!m.has)                                                        \
      return NAME##_none();                                            \
    return NAME##_some(f(m.v));                                        \
  }                                                                    \
  static inline NAME NAME##_bind(NAME m, NAME (*k)(T)) {               \
    if (!m.has)                                                        \
      return NAME##_none();                                            \
    return k(m.v);                                                     \
  }                                                                    \
  static inline bool NAME##_take(NAME *m, T *out) {                    \
    if (!m->has)                                                       \
      return false;                                                    \
    *out = m->v;                                                       \
    m->has = false; /* ownership moves out, the guard flips */         \
    return true;                                                       \
  }

static bool bint_payload_eq(int a, int b) { return a == b; }
static bool bfrac_payload_eq(frac a, frac b) { return frac_eq(a, b); }

DEFINE_MONAD(bint, int)
DEFINE_MONAD(bfrac, frac)

// one spelling over every instantiation: dispatch reads the carrier type
// off the expression and picks the stamped combinator
#define BIND(m, k) _Generic((m), bint: bint_bind, bfrac: bfrac_bind)((m), (k))
#define FMAP(m, f) _Generic((m), bint: bint_map, bfrac: bfrac_map)((m), (f))
#define MUNIT(v) _Generic((v), int: bint_some, frac: bfrac_some)((v))
```

listing 306: the macro that stamps the option monad for a payload type, the two instantiations, and the `_Generic` dispatch (source: math/samples/src/Ch28/dispatch.c, lines 56-106)

The second instantiation carries exact rationals, sign in the numerator, denominator positive, fully reduced by a gcd loop, so the law checks run on fractions with no floating point in sight: 1/2 through multiply by 3/2 is 3/4 on

both sides of left identity, 5/4 through unit stays 5/4, and both groupings of multiply by 3/2 then 5/3 land on 5/4. The dispatched map gives 3x+1 over int, 6 landing on 19, and multiply by 3/4 over frac, 1/3 landing on 1/4.

Ownership rides in the same macro. take moves the payload out of the carrier and flips the tag to false: the first take of a box holding 15/12 hands back 5/4 and empties the box, a second take fails, and the box refills and moves again. The tag is the borrow discipline: a spent carrier is visibly spent, and reading the payload requires the guard.

```
int main(void) {
  // the same spelling over int: left identity, 7+3 = 10
  auto bi = BIND(MUNIT(7), k_add3);
  CHECK(bint_eq(bi, k_add3(7)) && bi.v == 10,
        "dispatch: bind over bint gives some 10");

  // right identity: unit itself is the kleisli arrow, both tags
  CHECK(bint_eq(BIND(bint_some(5), bint_some), bint_some(5)) &&
            bint_eq(BIND(bint_none(), bint_some), bint_none()),
        "right identity over bint, some and none");

  // rationals normalize: 9/12 reduces to 3/4 (precompute.py)
  CHECK(frac_eq(frac_make(9, 12), frac_make(3, 4)),
        "frac_make reduces 9/12 to 3/4");

  // law 1 over frac: 1/2 through mul 3/2 is 3/4
  CHECK(
      bfrac_eq(BIND(MUNIT(frac_make(1, 2)), k_mul3_2), MUNIT(frac_make(3, 4))),
      "left identity over bfrac, 1/2 by 3/2 is 3/4");

  // law 2 over frac: 5/4 through unit stays 5/4
  CHECK(bfrac_eq(BIND(MUNIT(frac_make(5, 4)), bfrac_some),
                 MUNIT(frac_make(5, 4))),
        "right identity over bfrac");

  // law 3 over frac: (1/2 by 3/2) by 5/3 is 5/4, both groupings
  bfrac fl = BIND(BIND(MUNIT(frac_make(1, 2)), k_mul3_2), h_mul5_3);
  bfrac fr = BIND(MUNIT(frac_make(1, 2)), assoc_rhs);
  CHECK(bfrac_eq(fl, fr) && bfrac_eq(fl, MUNIT(frac_make(5, 4))),
        "associativity over bfrac, both sides 5/4");

  // fmap dispatches too: 3*6+1 = 19 over int, 1/3 by 3/4 is 1/4 over frac
  CHECK(bint_eq(FMAP(MUNIT(6), f_3x1), MUNIT(19)),
        "fmap over bint gives some 19");
  CHECK(
      bfrac_eq(FMAP(MUNIT(frac_make(2, 6)), f_mul3_4), MUNIT(frac_make(1, 4))),
      "fmap over bfrac gives some 1/4");
```

listing 307: the same BIND and FMAP spellings checking the laws over both instantiations (source: math/samples/src/Ch28/dispatch.c, lines 223-259)

```
// ownership: take moves the payload out and flips the guard
bfrac box = MUNIT(frac_make(15, 12));
frac moved = {0};
CHECK(bfrac_take(&box, &moved) && frac_eq(moved, frac_make(5, 4)) && !box.has,
      "take moves 15/12 out as 5/4 and empties the box");
CHECK(!bfrac_take(&box, &moved), "a second take fails, the box is spent");
box = MUNIT(frac_make(7, 4));
CHECK(bfrac_take(&box, &moved) && frac_eq(moved, frac_make(7, 4)),
      "the box refills and moves again");

// take works through the macro on the int instantiation too
bint ibox = MUNIT(31);
int imoved = 0;
CHECK(bint_take(&ibox, &imoved) && imoved == 31 && !ibox.has,
      "take over bint moves 31 out and empties the box");
```

listing 308: ownership transfer: take hands the payload out once, the guard flips, the box refills (source: math/samples/src/Ch28/dispatch.c, lines 261-275)

The two numbers: one macro line instantiates 2 carriers, and 15/12 moves out exactly once as 5/4.

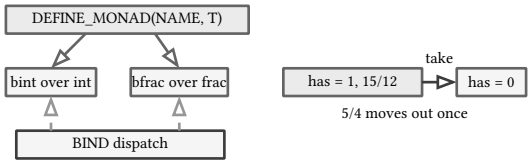


figure 172: one macro stamps the monad per payload type, `_Generic` picks the combinator, take moves the payload out under the tag

28.6. closures, purity, and the price of state

Kleisli arrows so far were plain functions. A closure is a function pointer plus the environment it carries, one struct, and the environment is where C programmers get to break the laws. Three closures run the gamut: a pure one with a null environment that doubles {1, 2, 3} into {2, 4, 6}, an accumulator whose mutable sum reports running totals {1, 3, 6}, and a read-only threshold that keeps {5, 9, 6} from the 8 element fixture.

```c
// a closure is a function pointer plus the environment it carries
typedef struct {
  void *env;
  long (*fn)(void *env, long x);
} closure;

static long closure_call(closure c, long x) { return c.fn(c.env, x); }

static long fn_double(void *env, long x) { // pure: the environment is null
  (void)env;
  return 2 * x;
}

typedef struct {
  long sum;
} accum;

static long fn_accum(void *env, long x) { // stateful: running total
  accum *a = env;
  a->sum += x;
  return a->sum;
}

typedef struct {
  long t;
} threshold;

static long fn_gt(void *env, long x) { // reads its environment, mutates none
  threshold *p = env;
  return x > p->t;
}
```

listing 309: the closure struct and three environments: null, mutable, read-only (source: math/samples/src/Ch28/dispatch.c, lines 120-150)

The trap demo gives two arrows a shared counter that both read and bump. Grouped left over {10, 20}, both k calls run before any h, so h sees the counter at 2 then 3 and the list is {12, 24}. Grouped right, each k is immediately followed by its h, so h sees 1 then 3 and the list is {11, 25}. Same 4 calls, same final counter, different lists: associativity is gone, and the exact numbers show the damage is evaluation order, not arithmetic.

```
// bind with single-outcome kleisli arrows: enough to expose the effect
// of evaluation order, which is all the impurity demo needs
static lvec lvec_bind(lvec m, closure k) {
  lvec out = {.n = 0};
  for (int i = 0; i < m.n; i++)
    out.xs[out.n++] = closure_call(k, m.xs[i]);
  return out;
}

// the right side of associativity, spelled per element
static lvec assoc_rhs_imp(lvec m, closure k, closure h) {
  lvec out = {.n = 0};
  for (int i = 0; i < m.n; i++)
    out.xs[out.n++] = closure_call(h, closure_call(k, m.xs[i]));
  return out;
}

// the shared counter both impure arrows read and bump
static long sc = 0;

static long k_imp_fn(void *env, long x) {
  (void)env;
  long out = x + sc;
  sc++;
  return out;
}

static long h_imp_fn(void *env, long y) {
  (void)env;
  long out = y + sc;
  sc++;
  return out;
}
```

listing 310: single outcome bind, the per element right side, and the two impure arrows (source: math/samples/src/Ch28/dispatch.c, lines 189-221)

```c
// impurity: two arrows sharing a counter on {10,20}. grouped left,
// k runs twice before any h, so h sees counter 2 and 3: {12,24}.
long pair[2] = {10, 20};
sc = 0;
lvec left =
    lvec_bind(lvec_bind(lvec_of(2, pair), (closure){nullptr, k_imp_fn}),
              (closure){nullptr, h_imp_fn});
CHECK(lvec_eq(left, lvec_of(2, (long[2]){12, 24})) && sc == 4,
      "impure left grouping gives 12,24 in 4 calls");

// grouped right, each k is followed by its h, so h sees counter 1
// then 3: {11,25}. same call count, different list.
sc = 0;
lvec right = assoc_rhs_imp(lvec_of(2, pair), (closure){nullptr, k_imp_fn},
                           (closure){nullptr, h_imp_fn});
CHECK(lvec_eq(right, lvec_of(2, (long[2]){11, 25})) && sc == 4,
      "impure right grouping gives 11,25 in 4 calls");
CHECK(!lvec_eq(left, right),
      "associativity is gone: the two groupings disagree");
```

listing 311: the associativity break as checks: 12,24 against 11,25 on the same 4 calls (source: math/samples/src/Ch28/dispatch.c, lines 299-317)

The two numbers: {12, 24} against {11, 25}, the same computation regrouped. That is the whole contract in one fixture: the laws buy regrouping, staging, and inlining for arrows that are functions of their argument alone, and chapter book 2, ch 29 (mathematics for programmers, pipelines and effects) builds effectful pipelines on exactly that guarantee.

WARNING: THE LAWS ASSUME PURE ARROWS

A kleisli arrow that reads or writes shared state is not a function of its argument, and none of the three laws survive it: the impure fixture passes every arithmetic check while producing different lists under regrouping. Reach for a mutable environment when the effect is the point, as with the accumulator trace, and keep the arrows pure when the pipeline shape is the point.

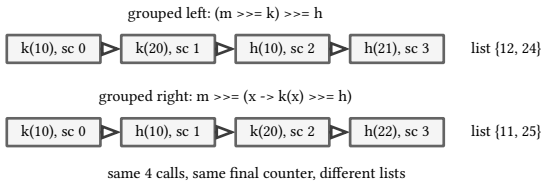


figure 173: evaluation order made visible: the same 4 calls in a different order give 12,24 against 11,25

sources: Philip Wadler, “Monads for functional programming”, in M. Broy, editor, Marktoberdorf Summer School on Program Design Calculi, NATO ASI Series F: Computer and systems sciences, Volume 118, Springer Verlag, August 1992, also in J. Jeuring and E. Meijer, editors, Advanced Functional Programming, LNCS 925, Springer, 1995, venue lines verified at homepages.inf.ed.ac.uk/wadler/topics/monads.html, fetched 2026-09-22, paper text cited by name and edition, the three laws restated in our own notation, our own words and code throughout. The monad-as-semantics lineage is Eugenio Moggi, “Notions of computation and monads”, Information and Computation 93(1), 1991, cited by name only. This chapter is not mapped to mml-book draft 2024-01-15 and no mml pages were read. All C23 features used (constexpr, `_Generic`, anonymous unions, enum with fixed underlying type, static_assert, [[nodiscard]], nullptr, compound literals, auto) are exercised and machine verified by the sample gate on this machine, clang 23.1.1 msvc-target, 2026-09-22. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Ch28`, 61 checks in chapter 28 of the math suite, format and asan legs clean.

29. pipelines and effects

Chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) gave failures a type, chapter book 2, ch 27 (mathematics for programmers, option, result, and the railway) gave them a track, and chapter book 2, ch 28 (mathematics for programmers, functors and monads in c) peeled the laws off the combinators. This chapter composes the whole arc into programs: functions as values in a tagged tree with composition, currying, and the fused pass one composition buys, the pipeline operator macro that reads left to right over the error track, state threading through explicit world values instead of static cells, seeded random streams carried as parameters, the line between a pure core and an impure shell, and the testing discipline that split makes possible. Every behavioral claim below is one of the 69 checks in the 4 samples of chapter 29 or a sentence quoted from a canonical source fetched 2026-09-22. The dsa book drives its game loops with inline mutation and returned error codes, so this chapter owns the plumbing discipline itself, and the capstone game is where it all gets spent.

29.1. composition as data

Composition builds a new function from two old ones: apply f first, then g, and the pair acts as $g(f(x))$. C will not let a function return a new function, so the composition has to live in data. The sample gives functions a sum type, the same discipline chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) used for shapes: a tag picks between four primitives, identity, increment, double, and subtract 3, plus one composition node carrying two child pointers. One switch interprets the tree, every kind handled, no default.

The dry run: the fixture is 5. The interpreter walks the tree sub3 · dbl · inc by recursion, and the three primitives fire in order.

1. inc maps 5 to 6, one addition.
2. dbl maps 6 to 12, one multiply.
3. sub3 maps 12 to 9, and the composed value is 9.

x	inc	dbl after inc	sub3 after dbl
5	6	12	9
−4	−3	−6	−9
12	13	26	23
0	1	2	−1

Associativity is the property that makes the tree shape free: LEFT, the nesting that groups sub3 with dbl first, and RIGHT, the nesting that groups dbl with inc first, are different static values with different pointer shapes, and the sample checks they agree on all 4 fixtures. Identity is neutral on either side, checked at 41 mapping to 42 both ways. Neither law needed a proof, both needed a CHECK, and both are the reason a C composition can be rebuilt into any tree the caller finds readable.

```c
// the function adt: a primitive, or g after f. the comp branch carries two
// child pointers, the same anonymous-struct discipline chapter 26 built.
typedef enum fn_kind : uint8_t {
  FN_ID = 0,   // identity
  FN_INC = 1,  // x + 1
  FN_DBL = 2,  // 2x
  FN_SUB3 = 3, // x - 3
  FN_COMP = 4, // g(f(x))
} fn_kind;

typedef struct fn {
  fn_kind kind;
  const struct fn *g; // applied second
  const struct fn *f; // applied first
} fn;

static_assert(sizeof(fn) == 24, "a tag and two pointers");

static const fn ID = {FN_ID, nullptr, nullptr};
static const fn INC = {FN_INC, nullptr, nullptr};
static const fn DBL = {FN_DBL, nullptr, nullptr};
static const fn SUB3 = {FN_SUB3, nullptr, nullptr};

// g1 is dbl after inc, a value, built once at file scope
static const fn G1 = {FN_COMP, .g = &DBL, .f = &INC};
// associativity fixtures: LEFT is (sub3 . dbl) . inc, RIGHT is sub3 . (dbl .
// inc)
static const fn SUB3_DBL = {FN_COMP, .g = &SUB3, .f = &DBL};
static const fn DBL_INC = {FN_COMP, .g = &DBL, .f = &INC};
static const fn LEFT = {FN_COMP, .g = &SUB3_DBL, .f = &INC};
static const fn RIGHT = {FN_COMP, .g = &SUB3, .f = &DBL_INC};

// one switch, every kind handled, no default: the interpreter
[[nodiscard]] static int64_t apply(const fn *fn_v, int64_t x) {
  switch (fn_v->kind) {
  case FN_ID:
    return x;
  case FN_INC:
    return x + 1;
  case FN_DBL:
    return 2 * x;
  case FN_SUB3:
    return x - 3;
  case FN_COMP:
    return apply(fn_v->g, apply(fn_v->f, x));
  }
}
```

listing 312: the function adt, a tag plus two child pointers, and the one switch that interprets it (source: math/samples/src/Ch29/compose.c, lines 9-55)

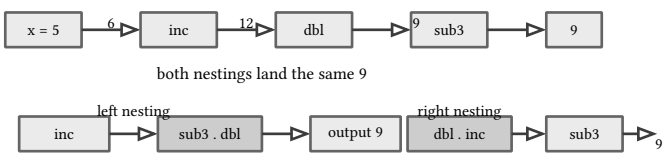


figure 174: the composed chain on top, the two associativity nestings below, same output

C closures are the same move with a struct. A closure is a function plus the environment it captured, and in C the environment is a context struct the function receives back. The sample fixes one parameter of the two parameter family $f(x) = \text{slope} \cdot x + \text{bias}$, so partial application is constructing the struct, currying is the LIN_ADD and LIN_MUL macros that fix k, and the one hole left is the argument. The generic closure pairs a call pointer with a context pointer, 16 bytes, checked.

```c
// c has no closures: the closure is a context struct plus an apply function.
// the two parameter family f(x) = slope * x + bias, one parameter at a time.
typedef struct {
  int64_t slope;
  int64_t bias;
} lin_ctx;

[[nodiscard]] static int64_t lin_apply(lin_ctx c, int64_t x) {
  return c.slope * x + c.bias;
}

// currying by fixing one argument: add_k and mul_k are lin values
#define LIN_ADD(k)                                                      \
  (lin_ctx) { .slope = 1, .bias = (k) }
#define LIN_MUL(k) (lin_ctx){.slope = (k), .bias = 0}

// the one-hole closure: a unary call paired with its context. the void * is
// the price of having exactly one closure type
typedef struct {
  int64_t (*call)(const void *ctx, int64_t x);
  const void *ctx;
} closure1;

static int64_t lin_thunk(const void *ctx, int64_t x) {
  return lin_apply(*(const lin_ctx *)ctx, x);
}

static const lin_ctx LIN3M2 = {.slope = 3, .bias = -2};
```

listing 313: the closure: a context struct, currying macros that fix one parameter, and the pointer pair (source: math/samples/src/Ch29/compose.c, lines 57-84)

Composition also pays in cache walks. Three separate passes over an 8 element array, inc then dbl then sub3, touch 24 elements. The fused pass walks once and applies the composed tree per element, 8 touches. Same outputs, the pinned ladder 1, 3, 5, 7, 9, 11, 13, 15, and the visit counter is the measured witness: 24 against 8.

```c
// the measured witness for fusion: each element touch bumps the counter
static int visits;
static int64_t v_inc(int64_t x) {
  visits++;
  return x + 1;
}
static int64_t v_dbl(int64_t x) {
  visits++;
  return 2 * x;
}
static int64_t v_sub3(int64_t x) {
  visits++;
  return x - 3;
}

// three passes: each loop walks the whole array once
static void three_passes(const int64_t *src, int n, int64_t *dst) {
  for (int i = 0; i < n; i++) {
    dst[i] = v_inc(src[i]);
  }
  for (int i = 0; i < n; i++) {
    dst[i] = v_dbl(dst[i]);
  }
  for (int i = 0; i < n; i++) {
    dst[i] = v_sub3(dst[i]);
  }
}

// the fused pass: one walk, the composition applied per element
static void fused_pass(const int64_t *src, int n, int64_t *dst) {
  for (int i = 0; i < n; i++) {
    visits++;
    dst[i] = apply(&RIGHT, src[i]);
  }
}
```

listing 314: the visit counter, the three separate passes, and the fused pass (source: math/samples/src/Ch29/compose.c, lines 86-120)

29.2. the pipeline operator

Composition reads inside out. Pipelines read left to right, and the spelling with a name is the F# forward pipe operator, which Microsoft's operator reference says "passes the result of the left side to the function on the right side" (learn.microsoft.com/en-us/dotnet/fsharp/language-reference/symbol-and-operator-reference/, fetched 2026-09-22). C has no infix operator to overload, so the sample builds |> as a macro family, and the honest desugaring is the chapter 27 railway: the first stage turns the raw input into a carrier, and every later stage binds onto the ok track, redeclared here so the file stands alone.

```c
typedef enum flow_err : uint8_t {
  FLOW_ODD = 1,   // raw word is not even
  FLOW_BOUND = 2, // halved value above 90
  FLOW_TAG = 3,   // tagged word overflows 12 bits
} flow_err;

typedef struct {
  uint8_t ok;
  union {
    int64_t val;
    struct {
      uint8_t code;
      uint8_t info;
    } err;
  };
} flow;

static_assert(sizeof(flow) == 16, "the carrier is tag plus one value");

[[nodiscard]] static flow ok(int64_t v) { return (flow){.ok = 1, .val = v}; }
[[nodiscard]] static flow fail(uint8_t code, uint8_t info) {
  return (flow){.ok = 0, .err = {.code = code, .info = info}};
}

// the bind under |>: one fallible stage on the ok track, the error rides
[[nodiscard]] static flow bind(flow v, flow (*stage)(int64_t)) {
  return v.ok ? stage(v.val) : v;
}
// the map that lifts a pure stage onto the ok track
[[nodiscard]] static flow fmap(flow v, int64_t (*f)(int64_t)) {
  return v.ok ? ok(f(v.val)) : v;
}

// raw |> s1 |> s2 ... reads left to right at the call site and desugars to
// nested binds: the first stage turns the raw input into a carrier, the rest
// bind onto the ok track. plain function calls, no extension, one defined
// evaluation order. the name counts stages.
#define PIPE2(x, f, g) bind((f)(x), (g))
#define PIPE3(x, f, g, h) bind(bind((f)(x), (g)), (h))
#define PIPE4(x, f, g, h, i) bind(bind(bind((f)(x), (g)), (h)), (i))
#define PIPE5(x, f, g, h, i, j)                                          \
  bind(bind(bind(bind((f)(x), (g)), (h)), (i)), (j))
```

listing 315: the local carrier, the bind and map combinators, and the PIPE macro family (source: math/samples/src/Ch29/pipeline.c, lines 10-51)

PITFALL: why the desugaring is plain function calls

The macro expands to nested calls to bind, a function, not to statement expressions or any compiler extension. Function call evaluation order is defined, the stages are ordinary symbols the debugger sees, and the file stays ISO C23. An infix |> written with GNU statement expressions would buy nothing here and pin the chapter to one compiler family.

The dry run: the domain is a firmware word. The input 48 rides four stages: s1 checks parity and passes 48, s2 halves to 24 inside the 90 bound, s3 computes the calibration word $10 \cdot 24 + 2 = 242$, s4 tags it with the checksum $242 \oplus 165 = 87$. Each stage logs the value it received, so the trace array pins 48, 48, 24, 242.

1. The counters read 1, 1, 1, 1, every stage ran exactly once.

2. The input 7 dies in s1 with residue 1, the counters read 1, 0, 0, 0, and the trace holds one entry.
3. The input 200 halves to 100, ten over the bound, and dies in s2 with counters 1, 1, 0, 0.

input	stages run	trace	result
48	4 of 4	48, 48, 24, 242	ok 87
180	4 of 4	180, 180, 90, 902	ok 803
7	1 of 4	7	odd, residue 1
200	2 of 4	200, 200	bound, excess 10

The failing rows are the chapter's argument in miniature: the error carries a typed code and position exactly as chapter book 2, ch 27 (mathematics for programmers, option, result, and the railway) built, the downstream stages never wake, and the counters prove it rather than assert it. The 2 of 4 against 4 of 4 is the two-number play, work not done on the failure track.

```
// stage invocation counters and the value trace, reset per scenario
static int c1, c2, c3, c4;
static int64_t trace_in[8];
static int trace_n;
static void reset(void) {
  c1 = c2 = c3 = c4 = 0;
  trace_n = 0;
}
static void log_stage(int64_t v) {
  if (trace_n < 8) {
    trace_in[trace_n++] = v;
  }
}

// s1: the raw word must be even, the residue says how far off it is
static flow s1_even(int64_t raw) {
  c1++;
  log_stage(raw);
  if (raw % 2 != 0) {
    return fail(FLOW_ODD, (uint8_t)(raw % 2));
  }
  return ok(raw);
}
// s2: halve, then bound the half at 90
static flow s2_bound(int64_t raw) {
  c2++;
  log_stage(raw);
  int64_t v = raw / 2;
  if (v > 90) {
    return fail(FLOW_BOUND, (uint8_t)(v - 90));
  }
  return ok(v);
}
// s3: pure calibration in fallible clothing, w = 10v + 2
static flow s3_calib(int64_t v) {
  c3++;
  log_stage(v);
  return ok(10 * v + 2);
}
// s4: checksum tag t = w ^ 0xA5, must stay inside 12 bits
static flow s4_tag(int64_t w) {
  c4++;
  log_stage(w);
  int64_t t = w ^ 0xA5;
  if (t > 0xFFF) {
    return fail(FLOW_TAG, 0);
  }
  return ok(t);
}
```

listing 316: the four stages, each counting itself and logging its input (source: math/samples/src/Ch29/pipeline.c, lines 53-101)

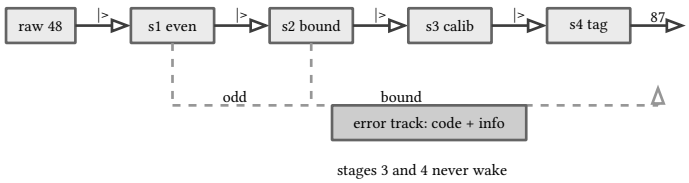


figure 175: the happy track with the macro sugar, the dashed error track bypassing the later stages

The call site is the payoff. One line holds the whole pipeline in reading order, the explicit bind nest is checked to reach the same 87, and a shorter chain, PIPE3, stops at the calibration word 242 with the tag counter still zero. A pure stage rides the same track through map, doubling the ok value to 174 and leaving the parity failure untouched.

```
// the whole pipeline, one line, stages in reading order
[[nodiscard]] static flow run_word(int64_t raw) {
  return PIPE4(raw, s1_even, s2_bound, s3_calib, s4_tag);
}
```

listing 317: the whole pipeline, one line, stages in reading order (source: math/samples/src/Ch29/pipeline.c, lines 103-106)

29.3. threading state

A pipeline stage that needs memory has three places to put it: a static cell, a heap object with an owner, or the function signature. Threading puts it in the signature: the state arrives as a value and leaves as the return, the shape $f : (a, s) \to (b, s)$ that chapter book 2, ch 28 (mathematics for programmers, functors and monads in c) called the state carrier. Every step is pure, so every intermediate state is a CHECK away.

```
// the fold state: sum, count, min, max, one struct, no statics
typedef struct {
  int64_t sum;
  uint32_t n;
  int64_t min;
  int64_t max;
} acc;

[[nodiscard]] static acc acc_zero(void) {
  return (acc){.sum = 0, .n = 0, .min = INT64_MAX, .max = INT64_MIN};
}

// one pure step: state in, input in, next state out
[[nodiscard]] static acc acc_step(acc a, int64_t v) {
  return (acc){.sum = a.sum + v,
               .n = a.n + 1,
               .min = v < a.min ? v : a.min,
               .max = v > a.max ? v : a.max};
}
```

listing 318: the fold state and its step, state in, input in, next state out (source: math/samples/src/Ch29/state.c, lines 11-29)

The dry run: the values 3, −1, 4, 1, 5 fold through acc_step, and the sample pins the accumulator after every single step, sum 3, 2, 6, 7, 12 with count 1 to 5, min settling at −1 on the second step and max at 5 on the last. The automaton thread is the same discipline with a transition table instead of arithmetic: three states recognizing the substring “ab”, state 2 absorbing, and the walk over “aababb” pins 0, 1, 1, 2, 2, 2, 2.

```c
// the automaton: contains "ab" over {a, b}, 3 states, state 2 accepting.
// rows are states, columns are 'a' and 'b', the exhaustive switch of
// chapter 26 flattened into a table
static const uint8_t NEXT[3][2] = {
    {1, 0}, // 0: a -> 1, b -> 0
    {1, 2}, // 1: a -> 1, b -> 2
    {2, 2}, // 2: absorbing accept
};

typedef struct {
  uint8_t q;
} dfa;

[[nodiscard]] static dfa dfa_step(dfa d, char c) {
  uint8_t col = (c == 'a') ? 0u : 1u;
  return (dfa){.q = NEXT[d.q][col]};
}

// one world, two threads of state, zero globals: the dfa and the fold
// advance together, each step a pure function of the world it received
typedef struct {
  dfa d;
  acc a;
} world;

[[nodiscard]] static world world_zero(void) {
  return (world){.d = {.q = 0}, .a = acc_zero()};
}

[[nodiscard]] static world world_step(world w, char c, int64_t v) {
  return (world){.d = dfa_step(w.d, c), .a = acc_step(w.a, v)};
}
```

listing 319: the dfa transition table, its pure step, and the world that threads both states at once (source: math/samples/src/Ch29/state.c, lines 31-62)

The world struct carries the automaton and the fold together, 40 bytes, checked, and run_script advances it with an explicit copy per step. The typeof on the copy is the point in code: whatever world grows into, the step receives a value and returns a value, and no step can reach back into the state the caller still holds.

```c
// the loop keeps value semantics: typeof names the copy, whatever world
// grows into later, and no step can reach back into the last one
[[nodiscard]] static world run_script(const char *chars, const int64_t *vals,
                                      int n) {
  world w = world_zero();
  for (int i = 0; i < n; i++) {
    typeof(w) next = world_step(w, chars[i], vals[i]);
    w = next;
  }
  return w;
}
```

listing 320: the runner, typeof names the copy, value semantics on every step (source: math/samples/src/Ch29/state.c, lines 64-74)

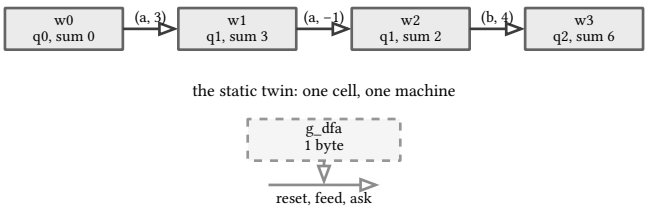


figure 176: the world threading above, the one byte static twin below

The impure twin keeps the same transition function but hides the state in file scope behind a reset, feed, ask interface. It is not wrong, it is single: the sample runs both over 4 fixture strings and the final states agree every time, but the twin holds exactly 1 byte of state, checked, while the threaded version holds one world per caller. Two worlds fed alternately, “aab” and “ba”, each land where their sequential runs land, 2 interleaved machines against the twin’s 1, and that is the two-number play: threading buys instances the way passing arguments buys them.

```
// the impure twin: the same transition function, but the state lives in
// file scope behind an interface. one machine per process, no interleaving
static dfa g_dfa;
static void dfa_reset(void) { g_dfa.q = 0; }
[[nodiscard]] static uint8_t dfa_feed(char c) {
  g_dfa = dfa_step(g_dfa, c);
  return g_dfa.q;
}
[[nodiscard]] static bool dfa_accepting(void) { return g_dfa.q == 2; }
```

listing 321: the global twin, same transition, one static cell behind an interface (source: math/samples/src/Ch29/state.c, lines 85-93)

NOTE: when the static twin is the right call

One machine, one owner, called from one place, the interface twin is simpler and the corpus ships plenty of them. The threaded version earns its copies the moment a second instance exists, a test replays a script against a fresh world, or a caller needs the state at an intermediate step for a checkpoint. The cost is not speed, it is the copy, 40 bytes here.

29.4. rng discipline

Deterministic simulation needs randomness that is a value. The corpus rule is no rand() and no time(), and the sample uses Marsaglia’s xorshift64, from the paper that described “a class of simple, extremely fast random number generators (RNGs) with periods” of $2^k - 1$ for word sizes $k = 32$ through 192, so the 64 bit member runs a full period of $2^{64} - 1$ on any nonzero seed (jstatsoft.org/article/view/v008i14, fetched 2026-09-22). The generator is three shifts and three xors, the seed is a constexpr word, and the stream is a struct passed by pointer: seed in, next seed out, draw returned.

```
// xorshift64, the (13, 7, 17) triple of Marsaglia 2003: three shifts, no
// multiply, period 2^64 - 1 on nonzero seeds
typedef struct {
  uint64_t s;
} rng;

constexpr uint64_t SEED = 0x1BADB002CAFEF00D;

[[nodiscard]] static uint64_t rng_next(rng *r) {
  uint64_t x = r->s;
  x ^= x << 13;
  x ^= x >> 7;
  x ^= x << 17;
  r->s = x;
  return x;
}

// a two round multiply-xor finalizer: turns a draw into a seed that does
// not overlap the parent stream
[[nodiscard]] static uint64_t finalize(uint64_t z) {
  z ^= z >> 33;
  z *= 0xFF51AFD7ED558CCDull;
  z ^= z >> 33;
  z *= 0xC4CEB9FE1A85EC53ull;
  z ^= z >> 33;
  return z;
}
```

listing 322: the xorshift64 stream as a struct, and the finalizer that scrambles a draw into a fresh seed (source: math/samples/src/Ch29/effects.c, lines 14-40)

The discipline has two rules and both are checked. First, the same seed replays the same stream: two independent runs of 6 draws from 0x1BADB002CAFEF00D agree bit for bit, 0 differing bits, and the first four draws are pinned as literals. Second, the stream belongs to the caller. The pure jitter core takes the stream by value and returns the next stream with the answer, so calling it never moves the caller's seed, checked, the same move the world struct made in the previous section. This is the same determinism contract as the fixed point CORDIC lane in book 14, ch 7 (game systems and architecture, deterministic math), stated over random draws instead of trigonometry.

The dry run: the first draw has residue 5 mod 7, so jitter at base 100 is $100 + 5 - 3 = 102$, and at base -50 the third draw's residue 2 gives -51. The naive split hands draw 1 to a child stream as its seed, and the child's first output equals the parent's second output exactly, the two streams overlap draw for draw. The fix finalizes the draw first, two rounds of multiply and xor shift, and the finalized child's first three draws are pinned and all differ from the parent's next three.

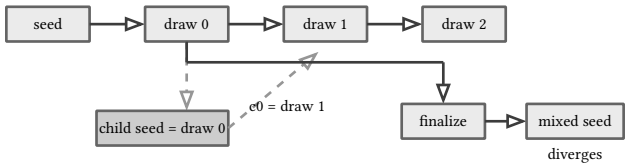


figure 177: the parent stream, the naive handoff rejoining it, the finalized branch diverging

WARNING: the handoff overlap is invisible in output

Two overlapping xorshift streams still look random in any histogram. The failure is correlation, not repetition: every child draw is a parent draw, so simulations that think they sample two independent noises sample one. The check that catches it is the equality the sample pins, child first output equals parent second. Any split that hands a raw draw to a child needs the finalizer.

29.5. the pure core and the impure shell

Isolation is a boundary, drawn once, with globals on one side. The pure core owns the arithmetic: jitter takes the stream by value and the base, returns the answer and the next stream, and touches nothing else, no globals, no io, checked by construction because it has no symbols to reach. The impure shell owns every effect the program has, five file-scope objects, countable in the listing: the stream g_stream, the call counter g_calls, the log array g_log with its fill index g_log_n, and the busy guard g_busy.

```c
// the pure core: base plus one draw folded into [-3, 3]. no globals, no io,
// the stream arrives by value and the next stream comes back out
typedef struct {
  int64_t out;
  rng next;
} jittered;

[[nodiscard]] static jittered jitter(rng r, int64_t base) {
  uint64_t d = rng_next(&r);
  return (jittered){.out = base + (int64_t)(d % 7u) - 3, .next = r};
}
```

listing 323: the pure core, stream in by value, answer and next stream out (source: math/samples/src/Ch29/effects.c, lines 42-52)

```c
// the impure shell: one global stream, one call counter, one log. every
// effect in the program lives on this side of the line, and the defer holds
// the busy guard for every exit path
static rng g_stream;
static int g_calls;
static int64_t g_log[8];
static int g_log_n;
static bool g_busy;

static void busy_clear(void) { g_busy = false; }
static void shell_reset(void) {
  g_stream.s = SEED;
  g_calls = 0;
  g_log_n = 0;
  g_busy = false;
}

[[nodiscard]] static int64_t jitter_shell(int64_t base) {
  g_busy = true;
  defer busy_clear();
  g_calls++;
  jittered j = jitter(g_stream, base);
  g_stream = j.next;
  if (g_log_n < 8) {
    g_log[g_log_n++] = j.out;
  }
  return j.out;
}
```

listing 324: the impure shell, it owns the stream, the counter, the log, and the guard, defer lowers it (source: math/samples/src/Ch29/effects.c, lines 54-81)

The dry run: reset the shell, then call jitter_shell three times at base 100. The stream draws residues 5, 2, 2, the core answers 102, 99, 99, the

counter reads 3, the log holds the same three numbers, and the guard is down after the last call. A second scenario resets and replays base −50 for −48, −51, −51, every number pinned.

The shell sets its busy guard and registers the release with defer, the TS 25755 lane the corpus pinned in c-os-cloud ch 06, available here as clang 23.1.1 stddefer.h under the gate's -fdefer-ts. Every exit path lowers the guard, including the implicit one, and the check after three calls finds it down. The shell's calls land in the log, so its effects are data too, and the checks read them back like any other array.

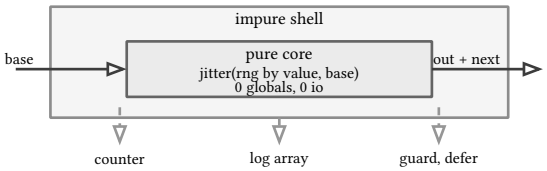


figure 178: the core takes and returns values only, the shell ring carries every effect

The two-number play sits in the middle of the figure: the core crosses the ring with values only, 2 arrows, while the ring itself owns 5 file-scope objects. A program drawn this way can move the core to any thread, replay it against any recorded stream, or reason about it with substitution, and the shell is small enough to audit by reading it once.

29.6. testing effectful code

The split turns testing from mocking into arithmetic. The pure core is tested directly: feed a seed, CHECK the answer, the same contract every chapter of this book has used. The shell is tested through its log: drive it with fixed inputs, then CHECK the log equals the trace the core's replay predicts. Nothing is stubbed, nothing sleeps, no order of global initialization is consulted.

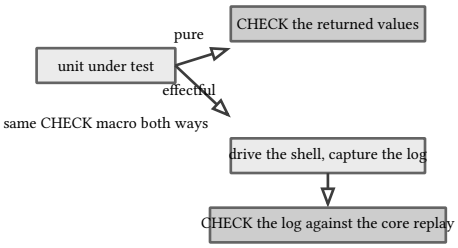


figure 179: the test route for a pure unit and for an effectful one

The dry run is the whole chapter on one file. effects.c holds 20 checks: 5 hold the stream, the four pinned draws and the two-run bit identity, 3 hold the core arithmetic with the caller's seed unmoved, 5 hold the split, the naive handoff, the finalized seed, and its three pinned draws, 5 hold the shell's outputs, counter, log, and guard, and 2 pin the sizes. The other three samples apply the same discipline to their own subjects, 20 on composition, 14 on the pipeline, 15 on threading, 69 in total, and every expected value was computed before it was checked, by a script in playground/math-ch29, never typed from memory.

VERIFY: the determinism gate is two runs

Any sample that claims determinism runs its effectful path twice and compares. Here the second six-draw run equals the first bit for bit and the shell's second scenario replays −48, −51, −51 from the reset seed. If the two-run check ever fails, the effect leaked into the core, and the boundary needs redrawing, not the test.

The capstone spends this chapter where it counts. The arena game behind book 2, ch 30 (mathematics for programmers, capstone: the arena game, design) is headless: its driver main.c prints rollout lines with no window and no clock reads, and its header names the split in this chapter's terms, the ai trees live in the impure shell while the core modules stay free of mutable globals. The fixed timestep is one constexpr, DT = 1/120, declared exactly once, and one step is a stage sequence over the game state: physics_step takes the state through a pointer and walks controls, both verlet halves, contacts, drag, and orb rules in order, returning void. No railway carrier rides

that loop, the stages return void and carry no error path, so chapter 27′s error track is not what the capstone reuses. What it reuses is the rest of this chapter: struct game is a value to copy, hash, and replay, the rng streams are driver-owned locals handed by pointer into the search, and the replay mode runs the same rollout twice demanding bit-identical hashes over the raw state bits, the two-run gate the verify callout states. 69 small proofs here, one program there.

sources: Microsoft Learn, “Symbol and Operator Reference - F#”, learn.microsoft.com/en-us/dotnet/fsharp/language-reference/symbol-and-operator-reference/, the |> row quoted, fetched 2026-09-22. George Marsaglia, “Xorshift RNGs”, Journal of Statistical Software 8(14) 2003, jstatsoft.org/article/view/v008i14, the abstract’s periods sentence quoted, fetched 2026-09-22. Defer ground truth from the corpus’s own pinned coverage, TS 25755 and clang 23.1.1 `stddefer.h` under `-fdefer-ts`, c-os-cloud ch 06. Capstone facts read from the committed source, books/math/capstone/src main.c, common.h, physics.h, state.h. mml-book draft 2024-01-15: no mapped chapter for pipelines and effects, none cited. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src -Chapter Ch29`, 69 checks in chapter 29 of the math suite, format and asan clean.

30. capstone: the arena game, design

Twenty-nine chapters of arithmetic now pay for one program: a 2-player physics arena where boxes collect orbs, ram each other, and hand their decisions to a search. This chapter is the design document: the game spec, the module map, the game state as an algebraic data type, the numerics budget that fixes timestep and tolerances before any code exists, the two-part ai (an exact mixed-strategy opening book, then budgeted monte carlo tree search), and the two-lane architecture that keeps one bit-exact truth across a headless core and a windowed allegro host. Every behavioral claim below is one of the 64 checks in the capstone test suite or one of the host gates, both run green on 2026-09-22 (`pwsh -NoProfile -File playground/math-capstone/build.ps1` and `pwsh -NoProfile -File tools/run-math-capstone.ps1`), or is a sentence read from the committed source it describes. The dsa book's games chapter owns combinatorial game search on discrete boards, so this capstone owns what it cannot: continuous physics in doubles, deterministic to the bit, with the game theory riding on top as macro decisions.

30.1. the game

Two player boxes start at (8, 12) and (16, 12) on a 24 by 24 cell arena, facing each other along the x axis at headings 0 and π. Eight orbs sit at fixed homes, four on the diagonal ring (5, 5), (19, 5), (19, 19), (5, 19) and four on the axis ring (5, 12), (12, 5), (19, 12), (12, 19). A box that touches an orb scores 1 for its player and the orb returns 600 steps later at the same home. Two neutral circles of radius 0.8 drift as obstacles at (12, 8) and (12, 16), pushed around by contacts but steering nothing. The game runs 7200 fixed steps of $\mathrm{DT} = 1/120$ seconds, 60 seconds of game time, 120 decision windows of 0.5 s each, and the higher orb count at the horizon wins.

The map is built to be symmetric, not decorated to look symmetric. The border ring plus exactly 16 interior wall cells: the 2 by 2 center block at (11, 11) through (12, 12), a diagonal orbit at (6, 6), (17, 6), (6, 17), (17, 17), and two midline orbits of 4 cells each. Every interior cell's full orbit under the dihedral group D_4 is present, which the state suite checks cell by cell over all 8 transforms, because chapter book 2, ch 16 (mathematics for programmers, groups and subgroups) needs the group action to be a symmetry of the board before any canonical hashing can be sound.

The dry run: the map by arithmetic.

1. 24 by 24 cells is 576, and the walkable interior is 22 by 22 = 484.
2. The border ring is the difference, 576 - 484 = 92 cells.
3. Adding the 16 interior wall cells walls off 108 of 576, and those 16 cells close into exactly 4 orbits, the center block, the diagonal ring, and the two midline rings.

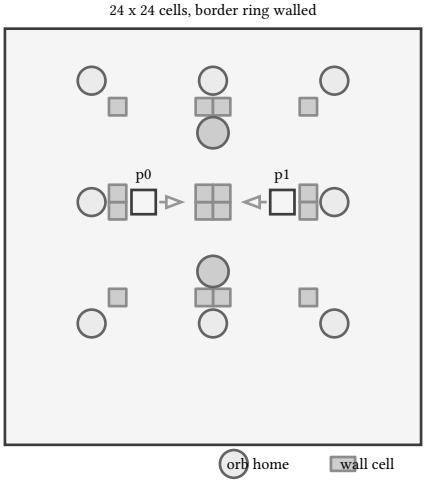


figure 180: the arena at step 0: border ring, the 16 interior wall cells in their d4 orbits, 8 orb homes, both spawns, both neutral circles

The bodies carry the material parameters the solver will need: player boxes have half extents 0.45, inverse mass 1.0, restitution 0.2, coulomb friction 0.3. The neutral circles have inverse mass 0.5, restitution 0.5, so a ram transfers momentum into them and they bounce harder off walls than the players do. Boxes are torque-free by design: heading is kinematic, set only by the turn control, and every contact applies central impulses with no torque arms. That is a documented simplification, and it keeps the impulse solver plain while the interesting dynamics stay in the translations.

```
void game_init(struct game *g, uint64_t seed) {
  *g = (struct game){0}; // zero padding too, so raw_hash is byte-stable
  g->seed = seed;
  g->n_bodies = 4;

  struct body *p0 = &g->bodies[0];
  *p0 = (struct body){.kind = BODY_BOX,
                      .player = 0,
                      .pos = v2(8.0, 12.0),
                      .heading = 0.0,
                      .half_w = 0.45,
                      .half_h = 0.45,
                      .inv_mass = 1.0,
                      .restitution = 0.2,
                      .friction = 0.3};
  struct body *p1 = &g->bodies[1];
  *p1 = (struct body){.kind = BODY_BOX,
                      .player = 1,
                      .pos = v2(16.0, 12.0),
                      .heading = PI,
                      .half_w = 0.45,
                      .half_h = 0.45,
                      .inv_mass = 1.0,
                      .restitution = 0.2,
                      .friction = 0.3};
  struct body *c0 = &g->bodies[2];
  *c0 = (struct body){.kind = BODY_CIRCLE,
                      .player = 255,
                      .pos = v2(12.0, 8.0),
                      .radius = 0.8,
                      .inv_mass = 0.5,
                      .restitution = 0.5,
                      .friction = 0.3};
  struct body *c1 = &g->bodies[3];
  *c1 = (struct body){.kind = BODY_CIRCLE,
                      .player = 255,
                      .pos = v2(12.0, 16.0),
                      .radius = 0.8,
                      .inv_mass = 0.5,
                      .restitution = 0.5,
                      .friction = 0.3};

  // orb homes: two D4 orbits (4 diagonal ring + 4 axis ring), exact doubles
  static const double ORB_HOME[MAX_ORBS][2] = {
      {5.0, 5.0},  {19.0, 5.0}, {19.0, 19.0}, {5.0, 19.0},
      {5.0, 12.0}, {12.0, 5.0}, {19.0, 12.0}, {12.0, 19.0}};
  g->n_orbs = MAX_ORBS;
  for (int i = 0; i < MAX_ORBS; i++) {
    g->orbs[i].home = v2(ORB_HOME[i][0], ORB_HOME[i][1]);
    g->orbs[i].taken = false;
    g->orbs[i].respawn = 0;
  }
}
```

listing 325: state.c, game_init: both player boxes, both passive circles, and the eight orb homes as two d4 orbits (source: math/capstone/src/state.c, lines 34-86)

30.2. modules and disciplines

The capstone is 8 core modules plus a driver, 1230 lines of core C in the .c files plus the 81-line shared header, 1311 with the header the table below counts, every file under the 400 sloc per file rule, and the split follows the book's own chapter boundaries so each module has exactly one discipline to answer for. The design rule is the pure core and impure shell of chapter book 2, ch 29 (mathematics for programmers, pipelines and effects): no mutable globals anywhere in the core, no heap, no clock reads, every random stream an explicit struct handed down the call chain, and the driver plus the allegro host own the two search trees as their impure state.

module	lines	role	discipline
common .h	81	vec2, xorshift64, fnv1a	ch 2 bit-exact kernels
state	287	adt game state, wall map, both hashes	ch 26, ch 16-17
collide	302	sat narrow phase, impulse solver	ch 2, ch 5 vectors
physics	92	fixed-step verlet loop	ch 22 quadrature
astar	111	grid search with total-order keys	ch 15 graphs
mixed	35	opening equilibrium, integer sampler	ch 24 strategic
mcts	114	uct over macro windows, group hash	ch 25 sequential
ai	183	macro actions to controls, run_window	ch 29 one path
main	106	headless driver, three modes	ch 29 impure shell

The layering is one-directional: physics and collide know state, astar knows common, ai knows all of them, and nothing below ever calls up. The determinism contracts sit at the seams: contact emission order is fixed in collide (dynamic pairs by body index, then one wall contact per body, deepest cell winning ties by scan order), every loop iterates array indices rather than pointers, there is no qsort anywhere, and the one random generator is a value passed by pointer. The verification chapter closes the loop with a discipline-to-module matrix and the tests that populate it, this chapter only fixes the assignment.

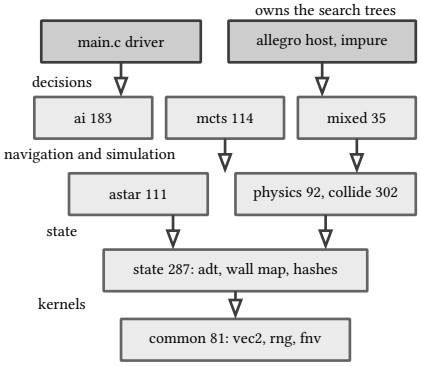


figure 181: the module map: the impure shells on top, decisions, navigation, simulation, and the state layer below, one direction of calls

30.3. the state is a value

Chapter book 2, ch 26 (mathematics for programmers, algebraic data types in c23) gave C tagged sums, and the game state is where that pays. `struct game` is one flat value: a step counter, both scores, 4 bodies in a fixed-capacity array, 8 orbs in another. Each body is a tagged product, `BODY_BOX` or `BODY_CIRCLE`, sharing the position, velocity, and material fields and carrying the extents or the radius for whichever kind it is. No heap, no pointers between parts, no global game. The whole state copies with one assignment, hashes by walking its bytes and fields, and replays by re-running the same inputs into a fresh copy. That is the entire reason the determinism proofs of chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification) can exist: there is one value to compare, and it has no identity beyond its contents.

Two hashes live on top of the value. The raw hash folds the exact bits of every dynamic double through fnv1a-64: 4 bodies times position, velocity, and heading, plus orb state and scores. It is the strongest bit-identity probe

available, since two runs agree on it if and only if every bit of every dynamic field agrees. The canonical hash is the chapter book 2, ch 16 (mathematics for programmers, groups and subgroups) canonical-form trick on top: quantize the state to integers (positions in 64ths of a unit, velocities in 256ths, box headings to quarter-turn bins), apply each of the 8 dihedral transforms as exact integer arithmetic, hash each transform, and take the minimum. A state and any rotation or reflection of it land on the same orbit minimum, so the search's transposition table shares entries across the arena's 8-fold symmetry.

Honesty about what that sharing is: a heuristic, in two ways. First, the group action is a symmetry of the map and the physics, not of the player-side semantics. Retreat targets the mover's own corner, (1, 1) or (22, 22), the opening macros filter orbs by west versus east half, and rewards are keyed to `score[0]` against `score[1]`. Rotate a live position 90 degrees and it hashes to the same canonical entry while meaning a different game. Exact in the map and physics layers, approximate as soon as player-side semantics enter, and chapter book 2, ch 31 (mathematics for programmers, capstone: the arena game, implementation) shows the one place that difference forced a real fix. Second, quantization can merge physics-distinguishable states: the position quantum is 1/64 = 0.015625 while a body at cruising speed drifts 0.0393 units per step, so two states less than half a quantum apart collide in the table and separate again within one window. The canonical hash is a sharing mechanism, not an equality proof, and the chapters below never claim otherwise.

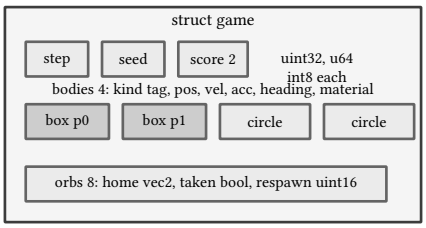


figure 182: the game state as one value: scalar bookkeeping, a tagged body array, and an orb array, everything fixed capacity

```
enum body_kind : uint8_t { BODY_BOX, BODY_CIRCLE };

struct body {
  uint8_t kind;          // BODY_BOX or BODY_CIRCLE
  uint8_t player;        // 0, 1 for player boxes; 255 for neutral circles
  vec2 pos, vel;         // world position and velocity
  vec2 acc;              // step acceleration, recomputed every physics_step
  double heading;        // box facing in radians, 0 = +x
  double half_w, half_h; // box half extents along heading axes
  double radius;         // circle radius
  double inv_mass;       // 1/mass, 0 would mean static (never here)
  double restitution;    // bounce in [0, 1]
  double friction;       // coulomb friction coefficient
};

struct orb {
  vec2 home;        // fixed spawn point, orbs never move
  uint16_t respawn; // step at which it returns, 0 while untaken
  bool taken;
};

struct game {
  uint32_t step; // completed fixed steps
  uint64_t seed; // the seed this run started from (bookkeeping only)
  int8_t score[2];
  uint8_t n_bodies;
  uint8_t n_orbs;
  struct body bodies[MAX_BODIES];
  struct orb orbs[MAX_ORBS];
};
```

listing 326: state.h, the tagged body, the orb, and the whole game as flat fixed-capacity values (source: math/capstone/src/state.h, lines 9-38)

30.4. the numerics budget

Before any physics code existed, the budget was fixed, and it reads like a contract from chapters book 2, ch 2 (mathematics for programmers, error, conditioning, and stability) and book 2, ch 22 (mathematics for programmers, quadrature and differential equations). One timestep, $\mathrm{DT} = 1/120$, declared exactly once in common.h and used everywhere, because two modules that disagree about the step size cannot agree about anything downstream. The integrator is velocity verlet, one force evaluation per step, chosen for the bounded energy wobble chapter 22 pinned at $E_0(h\omega)^2/4$ rather than the euler drift. Drag is an explicit per-step multiply, $1 - \mathrm{DAMP} \cdot \mathrm{DT}$ with $\mathrm{DAMP} = 1$, outside the verlet force so the integrator itself stays clean.

The tolerance policy has three tiers, and every comparison in the test suite declares which tier it is in. Analytic pins: hand-computed or offline-computed constants compared inside a stated epsilon, 1e-12 for values derived in exact arithmetic, 1e-9 where a wall bounce accumulates more rounding. Relational pins: two runs of the same inputs must agree on the raw hash, which is bit equality on every dynamic double, no epsilon at all. Witnessed pins: numbers that come from long deterministic runs, like search strength counts, pinned from the binary and flagged as such in the test headers, because no short derivation reproduces them.

The determinism policy is a list of prohibitions backed by chapter 2's lane discipline: no `rand()`, no `time()`, no heap addresses feeding order, no qsort (whose comparison ties are library business), no `-ffast-math`, x86-64 sse2 as the evaluation lane with no x87 excess precision. Every stochastic need is the explicit xorshift64 struct that chapters book 2, ch 24 (mathematics for programmers, strategic games) and book 2, ch 25 (mathematics for programmers, sequential and repeated games) already pinned, seeded through fnv1a so seed 0 still yields a nonzero stream. The

two numbers of the budget: DT $= 1/120$ exactly, and a full-thrust box tops out at speed 4.7145560438758594 after 360 steps, inside the ACC/DAMP $= 5$ bound, as chapter book 2, ch 31 (mathematics for programmers, capstone: the arena game, implementation) traces step by step.

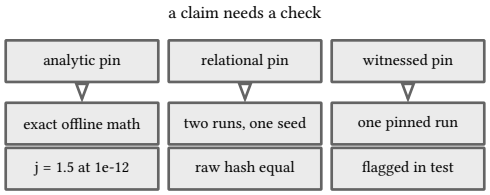


figure 183: the three pin tiers: analytic constants inside a stated epsilon, relational bit equality across runs, witnessed numbers pinned from the deterministic binary

```
constexpr double PI = 3.14159265358979323846;

constexpr double DT = 1.0 / 120.0; // the fixed timestep, declared exactly once
constexpr int ARENA_CELLS = 24;    // grid cells per arena side
constexpr double ARENA_SIZE = 24.0;
constexpr double CENTER = 12.0;

constexpr int MAX_BODIES = 4; // 2 player boxes + 2 circle obstacles
constexpr int MAX_ORBS = 8;
constexpr int GAME_STEPS = 7200;   // 60 s of game time at 120 hz
constexpr int WINDOW = 60;         // ai decision cadence in steps (0.5 s)
constexpr int REPLAN = 30;         // steering replan cadence inside a window
constexpr int OPENING_STEPS = 360; // opening book phase (3 s)
constexpr int ORB_RESPAWN = 600;   // steps before a taken orb returns
```

listing 327: common.h, every timing constant declared once, including the fixed timestep the whole core shares (source: math/capstone/src/common.h, lines 14-27)

30.5. the opening book and the search

The ai is two systems with one interface: a macro id per window. During the opening, 360 steps = 6 windows, each player follows a mixed strategy over 3 openings, rush west, rush east, or guard. The payoff table is the orb differential after the opening, [[2, 0, 1], [0, 3, 2], [1, 2, 0]], solved offline in exact rationals: value 11/9 for player 0, both optimal mixes (5, 3, 1) over 9. The runtime never floats the game theory, chapter book 2, ch 24 (mathematics for programmers, strategic games) discipline: the C carries the integers and an integer-comparison sampler, and the test suite re-proves optimality with the saddle certificate, every row against the column mix paying exactly 11 and every column against the row mix conceding exactly 11.

The dry run: `rng_seed(1)` draws the pinned sequence 1, 0, 2, 0, 0, 1, 0, 0, 1, 0, 1, 0 over 12 samples, and 6000 draws give counts 3321, 2022, 657, the 5:3:1 mix showing through the sampling.

Midgame windows belong to monte carlo tree search over the macro actions. The design constraints come straight from chapter book 2, ch 25 (mathematics for programmers, sequential and repeated games): information is perfect, so the search is a sequential-move tree where player 0 commits a window and player 1 answers, the zero-iteration limit of that tree being exactly the rollback induction of chapter 25. Nodes live in a fixed open-addressing transposition table, 2048 slots, keyed by the canonical group hash mixed with a mover byte, so rotated and reflected equal positions share one entry. Rewards are integer half-wins, 0, 1, or 2, so tree statistics are exact by construction and bit-reproducible. Budgets are counts, never clock reads: 160 iterations, 4 windows of explicit tree, 2 random playout windows past the frontier, expansion of the first untried action in index order, ucb ties to the lowest index.

One design fact is stated here and measured in chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification): the tree models the window as sequential, the mover acting while the opponent parks

on guard, while live play and the playouts run both players' macros through the same window simultaneously. That idealization is deliberate, it keeps the tree small and the code one path, and the verification chapter discloses what it costs. And the game has a structural first-mover advantage: player 0 commits first in the tree and the payoff table is player-0 sided, so selfplay results lean p0 by construction, not by tuning.

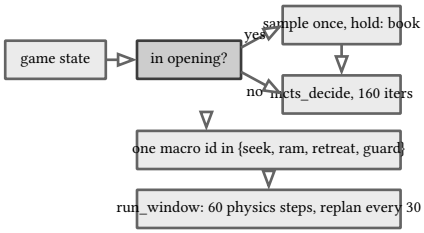


figure 184: the decision pipeline: one macro id per window, from the sampled opening book or the 160-iteration search, into the single execution path

```
enum macro : uint8_t {
  M_SEEK = 0, // nearest unclaimed orb
  M_RAM,      // drive at the opponent
  M_RETREAT,  // back toward the home corner
  M_GUARD,    // brake and face the opponent
  MACRO_COUNT,
  M_SEEK_W = MACRO_COUNT, // opening-book only: west half (home.x < 12)
  M_SEEK_E,               // opening-book only: east half
};

struct mctx; // mcts search state, defined in mcts.h

struct ai_ctx {
  uint8_t opening; // sampled opening 0..2
  bool sampled;
};

void ai_init(struct ai_ctx *a);
[[nodiscard]] uint8_t ai_choose(struct ai_ctx *a, const struct game *g,
                                int player, struct mctx *tree, rng *r);
[[nodiscard]] uint8_t random_action(rng *r);

// execute one decision window (WINDOW steps) under the given macro actions;
// stateless: paths are replanned from the game state on a fixed cadence
void run_window(struct game *g, const uint8_t act[2]);
```

listing 328: ai.h, the macro enum with two book-only rushes, and run_window, the one execution path the core exports (source: math/capstone/src/ai.h, lines 11-35)

30.6. two lanes, one truth

The core builds with clang on the msvc abi and links the sdk's static ucrt. The allegro host, a windowed front end with an offscreen bench mode, builds with gcc 15 on mingw, because the allegro monolith is mingw-flavored and the clang lane cannot link it. Two lanes, one truth: the host never re-implements core logic, it calls the same objects, and its bench mode runs the exact headless rollout, printing the identical rolling hash plus a render checksum, so the pinned hash values witness lane agreement bit for bit.

The one real hazard on that road is the libm. The core is cross-libm fragile by construction: it computes `sin`, `cos`, and `atan2` on arbitrary arguments, and different c runtimes return different bits for those. The mingw lane resolves the double transcendentals through msvcrt.dll, where `sin(pi)` comes back 0x3ca1a60000000000, off by a relative 3.3e-5, roughly 15 bits, while the core's ucrt gives the correctly rounded 0x3ca1a62633145c07. Player

1 starts at heading exactly π, so the raw-bit state hash diverged on step 0, 0xe698373cf0161dd6 against the pin 0x81fb1cc382429465, the first time the cross-lane gate ran. The host's answer is a routing shim: define the three functions so the core objects resolve them through ucrtbase.dll, probed bit-identical to the core's static ucrt, and exit loudly if that dll cannot load rather than compute quietly wrong bits. The durable fix, pinning the trig inside the core, was deliberately left undone, and chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification) carries the honest line: lane equality is witnessed for every heading the pinned rollouts generate, it is not a proof over all inputs.

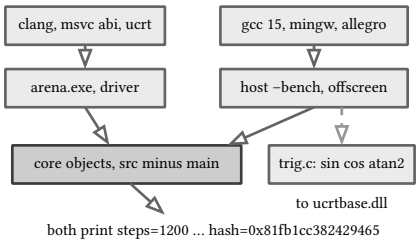


figure 185: two build lanes over one core: the clang-built driver and the gcc-built host, their bench hashes meeting at the pinned value through the ucrt routing

```c
static int bench_mode(uint64_t seed, int steps) {
  // offscreen: no display is ever created, memory bitmaps only, so every
  // primitive takes the software path
  al_set_new_bitmap_flags(ALLEGRO_MEMORY_BITMAP);
  ALLEGRO_BITMAP *fb = al_create_bitmap(VIEW_W, VIEW_H);
  if (!fb) {
    fprintf(stderr, "bench: framebuffer creation failed\n");
    return 1;
  }
  struct game g;
  game_init(&g, seed);
  uint64_t h = 0xcbf29ce484222325ULL;
  int frames = 0;
  for (int i = 0; i < steps && !game_over(&g); i++) {
    struct controls c[2] = {scripted(i, 0), scripted(i, 1)};
    physics_step(&g, c);
    h = fnv_u64(h, raw_hash(&g));
    render_frame(fb, &g, NULL);
    frames++;
  }
  uint64_t rc = render_checksum(fb);
  int taken = 0;
  for (int i = 0; i < g.n_orbs; i++)
    taken += g.orbs[i].taken ? 1 : 0;
  printf("steps=%u score=%d:%d orbs_taken=%d hash=0x%016llx\n", g.step,
         (int)g.score[0], (int)g.score[1], taken, (unsigned long long)h);
  printf("render=0x%016llx frames=%d\n", (unsigned long long)rc, frames);
  return 0;
}
```

listing 329: host main.c, bench_mode: the headless rollout plus one offscreen render per step, printing the core hash line and a pixel checksum (source: math/capstone/host/main.c, lines 56-84)

The design is now complete on paper: a symmetric arena, a value-typed state, a budgeted numerics contract, a two-part ai over macro windows, and two lanes pinned to one truth. Chapter book 2, ch 31 (mathematics for programmers, capstone: the arena game, implementation) builds every module and walks the pinned numbers through each one.

sources: arena and interface facts read from the committed capstone source, books/math/capstone/src common.h, state.h, state.c, physics.h, ai.h, main.c and books/math/capstone/host main.c, trig.c, 2026-09-22. Design bridges quote the corpus's own chapters: error for the tolerance tiers, quadrature for the verlet energy bound, groups and actions for the d4 canonical form, strategic for the mixed opening, sequential for the tree, adt for the tagged state, pipeline for the pure core and impure shell. No external pages fetched for this chapter, none cited. All pins witnessed by `pwsh -NoProfile -File playground/math-capstone/build.ps1`, 64 checks plus the replay and twice-run driver gates, format clean, and `pwsh -NoProfile -File tools/run-math-capstone.ps1`, host bench 7:1200 matching arena.exe bit for bit, both run green on 2026-09-22.

31. capstone: the arena game, implementation

The design of chapter book 2, ch 30 (mathematics for programmers, capstone: the arena game, design) now turns into 1230 lines of core C in the .c files plus the shared header the design chapter counts, and this chapter walks each module with the numbers its tests pin: the fixed verlet step and its oscillator fixture, the separating axis narrow phase and the impulse solver behind it, the grid search with its total-order ranking key, the quantized group hash under the tree search, and the allegro host that renders the same bits. Every behavioral claim below is one of the 64 checks in the capstone test suite or one of the host gates, both run green on 2026-09-22 (`pwsh -NoProfile -File playground/math-capstone/build.ps1` and `pwsh -NoProfile -File tools/run-math-capstone.ps1`), or is a sentence read from the committed source it describes. Two bugs the pins caught are retold here at full length because they are the chapter's real content: what determinism discipline buys is loud failures, and both failures were loud.

31.1. the fixed step

One `physics_step` is a fixed stage sequence over the state: controls become accelerations, then the velocity verlet half kick, the drift, the contact solve in two passes, the closing half kick, one drag multiply, the orb rules, and the step counter. The order is the contract. Every module that touches bodies runs in body index order, the two verlet halves are exposed functions so the chapter book 2, ch 22 (mathematics for programmers, quadrature and differential equations) oscillator fixture can drive the exact production integrator with a spring instead of controls, and the drag multiply is outside the force evaluation so the integrator stays plain verlet.

```
void physics_step(struct game *g, const struct controls ctl[2]) {
  apply_controls(g, ctl);
  for (int i = 0; i < g->n_bodies; i++)
    verlet_kick(&g->bodies[i]);
  for (int i = 0; i < g->n_bodies; i++)
    verlet_drift(&g->bodies[i]);

  struct contact cs[MAX_CONTACTS];
  int n = collide_detect(g, cs);
  solve_velocity(g, cs, n, SOLVE_ITERS);
  solve_position(g, cs, n);
  for (int i = 0; i < g->n_bodies; i++)
    verlet_kick(&g->bodies[i]); // second half of the verlet step

  double drag = 1.0 - DAMP * DT;
  for (int i = 0; i < g->n_bodies; i++)
    g->bodies[i].vel = vscale(g->bodies[i].vel, drag);

  orb_rules(g);
  g->step++;
}
```

listing 330: physics.c, physics_step: the fixed stage order, both verlet halves visible, drag as one explicit multiply (source: math/capstone/src/physics.c, lines 72-92)

The oscillator gate runs 2400 steps of $a = -4q$ through those halves with DT $= 1/120$, so $h\omega = 1/60$ and $E_0 = 2$. Energy ends at 1.9999229536874537, and the worst deviation over the whole run is 1.3888781008541962e-4. The chapter 22 bound is $E_0(h\omega)^2/4 = 1/7200 = 1.3888888\ldots \times 10^{-}4$: the measured worst sits at ratio 0.9999922 of the bound, strictly inside it. That phrasing is deliberate. The bound is an inequality, the fixture lands a hair under it, and the test name that says "equals" is the one wording the suite gets wrong, which is why this book writes "within". The same fixture runs the time reversibility probe, run 1200 steps, flip the velocity, run 1200 more, flip back: the round trip lands on $q = 1.0000000000000018$, $v = -5.6 \times 10^{-}17$.

The dry run: sustained full thrust from rest, heading aligned, 360 steps. Each step is kick, kick, multiply by $1 - 1/120$, so the speed ladder climbs geometrically toward the fixed point $\mathrm{ACC}\ (1 - \mathrm{DT}) = 4.9583\ldots$.

1. After 120 steps the speed is 3.1418912249516135, past 63 percent of the way in one second of thrust.
2. After 240 steps it is 4.292895632539598, and after 360 it is 4.7145560438758594, the test pin, still 0.244 short of the asymptote because the approach decays like $e^{-n/120}$ and the discrete leftover after 360 steps is $(119/120)^{360} = 0.049165$ of the gap.

steps	speed, both kicks	speed, one kick	asymptote gap
0	0	0	4.9583
120	3.1419	1.5709	1.8164
240	4.2929	2.1464	0.6654
360	4.7146	2.3573	0.2438

The one-kick column is not a counterfactual from theory, it is the first real bug. The original step was missing the second half kick, the test pinned the analytic ladder, and the run came back at exactly half speed, 2.3572780219379648 against 4.7145560438758594. One verlet half missing halves the per-step impulse, and the terminal-speed pin failed loudly instead of letting a sluggish arena ship.

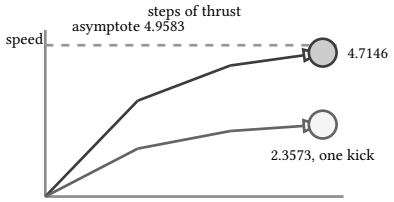


figure 186: the half-kick bug as a before and after: speed ladders from the same thrust, the buggy build flattening at exactly half

VERIFY: THE BOUND IS AN INEQUALITY

The oscillator worst deviation is 0.9999922 times $E_0(h\omega)^2/4$, inside the chapter 22 bound, not on it. Writing the fixture result as an equality is the classic drift from “at most” to “exactly” that review exists to catch, and this suite’s own test name carries that wording fault as a standing reminder. Quote the ratio, keep the inequality.

```
int main(void) {
  // the oscillator: bounded energy error, exactly like ch 22 pinned it
  struct body osc = {.pos = v2(1.0, 0.0)};
  double worst = 0.0;
  for (int i = 0; i < 2400; i++) {
    osc_run(&osc, 1);
    double dev = fabs(osc_energy(&osc) - 2.0);
    if (dev > worst)
      worst = dev;
  }
  CHECK(fabs(osc_energy(&osc) - 1.9999229536874537) < 1e-12,
        "oscillator 2400 steps: E ends at 1.99992");
  CHECK(fabs(worst - 1.3888781008541962e-4) < 1e-12 && worst < 1.5e-4,
        "worst energy deviation 1.389e-4 = E0*(h*omega)^2/4, never drifts");

  struct body rt = {.pos = v2(1.0, 0.0)};
  osc_run(&rt, 1200);
  rt.vel = vscale(rt.vel, -1.0);
  osc_run(&rt, 1200);
  rt.vel = vscale(rt.vel, -1.0);
  CHECK(fabs(rt.pos.x - 1.0000000000000018) < 1e-12 && fabs(rt.vel.x) < 1e-12,
        "round trip: flip-run-flip returns to the start state");
```

listing 331: test_physics.c, the oscillator gate on the production halves: end energy, worst deviation, and the flip-run-flip round trip (source: math/capstone/src/test_physics.c, lines 60-81)

31.2. narrow phase: the separating axis test

Two convex shapes are disjoint if any single axis separates them, so the narrow phase tests the four box axes, u_1, v_1, u_2, v_2, in that fixed order and keeps the first minimum overlap as the contact normal. The projection of an oriented box onto an axis t is $\text{half}_w\ |u \cdot t| + \text{half}_h\ |v \cdot t|$, and the overlap on that axis is $p_1 + p_2 - |d \cdot t|$ with d the center offset. Any axis with overlap at or under 0 exits with no contact, which is the separating axis theorem doing the work of chapter book 2, ch 7 (mathematics for programmers, inner products and projections) in four lines. Circles against boxes and against wall cells use the closest-point clamp: project the circle center into the box, and the contact normal is the unit vector from the clamped point.

Determinism lives in the details around the theorem: axes are tested in index order, the first minimum wins ties so the normal choice can never flip between builds, the normal points from the lower body index to the higher, and the broad phase is a plain axis-aligned bounding box check per pair in index order. Walls are scanned once per body over the cells its bounding box touches, and only the deepest overlap survives, ties by scan order, so one body contributes at most one wall contact per step.

```
static bool sat_boxes(vec2 c1, vec2 u1, vec2 w1, double hw1, double hh1,
                      vec2 c2, vec2 u2, vec2 w2, double hw2, double hh2,
                      vec2 *normal, double *pen) {
  vec2 axes[4] = {u1, w1, u2, w2};
  vec2 d = vsub(c2, c1);
  double best = 0.0;
  vec2 best_axis = v2(0.0, 0.0);
  for (int i = 0; i < 4; i++) {
    vec2 t = axes[i];
    double p1 = fabs(vdot(u1, t)) * hw1 + fabs(vdot(w1, t)) * hh1;
    double p2 = fabs(vdot(u2, t)) * hw2 + fabs(vdot(w2, t)) * hh2;
    double dist = fabs(vdot(d, t));
    double overlap = p1 + p2 - dist;
    if (overlap ≤ 0.0)
      return false;
    if (i == 0 || overlap < best) {
      best = overlap;
      best_axis = t;
    }
  }
  *pen = best;
  *normal = vdot(d, best_axis) < 0.0 ? vscale(best_axis, -1.0) : best_axis;
  return true;
}
```

listing 332: collide.c, sat_boxes: four axes in fixed order, first minimum wins, normal from the sign of the offset (source: math/capstone/src/collide.c, lines 28-51)

The dry run: the head-on gate. Two boxes with half extents 0.45 park at $x = 10.0$ and $x = 10.85$, touching distance 0.9, with velocities $+1$ and -1, controls neutral, restitution 0.5 both, friction 0.

1. The kicks add nothing, the drift moves each body $1/120 = 0.008333$, so the gap closes to 0.833333 and the sat overlap is $0.9 - 0.833333 = 0.066667$, the pinned penetration.
2. The solver fires once in the gate: relative normal velocity -2, so $j = -(1 + 0.5)(-2)/(1 + 1) = 1.5$, and the velocities swap to ± 0.5 before the drag multiply lands them at ± 0.49583333333333335.
3. The position pass moves each body half of $0.4(0.066667 - 0.005)$ weighted by inverse mass, and the final positions pin at 9.996 and 10.854 with the momentum sum exactly 0.

The same one-step machinery pins the wall rebound: a box at restitution 0.2 sliding into the west border at -1 meets the wall contact whose restitution mixes to $0.5(0.2 + 0) = 0.1$, takes $j = 1.1$, and comes back at $+0.099166666666764$ having been pushed out to $x = 1.353$.

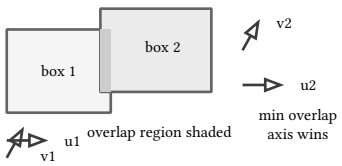


figure 187: the sat between two boxes: each candidate axis with the projected radii and the gap, the minimum overlap becoming the contact normal

31.3. sequential impulses

Contacts become velocities through a fixed iteration budget, 8 passes over the contact list in emission order. Each pass walks the same three questions per contact. Is the approach speed v_n negative? If not, nothing happens, which is what makes resting contacts rest. The restitution is the mean of the two bodies' coefficients, and it zeroes out below $\text{REST_THRESH} = 0.5$ approach speed so micro-bounces do not jitter stacks. The normal impulse applies against the accumulated impulse, clamped nonnegative so a contact can push but never pull. Then friction: the

tangent direction of the relative velocity, the impulse bound $\mu \cdot j_n$ with μ the geometric mean of the two friction coefficients, clamped symmetrically.

The position pass is one sweep, not iterated: a baumgarte-style correction of $\mathrm{CORR}\ (\mathrm{pen} - \mathrm{slop})$ with $\mathrm{CORR} = 0.4$ and $\mathrm{SLOP} = 0.005$, split by inverse mass. The split is where the second bug story lives. The C moves each body by corr · inv_mass, the mass-weighted split every textbook uses. The python mirror of the pin moved the first circle by the full corr, and the pin came back disagreeing with the C at the sixth decimal. The pin was wrong, the solver was right, and the dry run proves it by symmetry.

The dry run: the two circles, radius 0.8, parked overlapping at $(12, 8)$ and $(12.9, 8)$, a 0.7 penetration with zero velocity.

1. Both static, so $v_n = 0$, no velocity impulse ever fires, friction sees $j_n = 0$ and bounds itself to nothing.
2. The correction is $0.4(0.7 - 0.005) = 0.278$, and with inverse masses 0.5 and 0.5 each body takes $0.278 \cdot 0.5 = 0.139$.
3. The circles land at 11.861 and 13.039 exactly, both velocities still zero, which is the resting-contact gate: overlap resolved, no energy invented.

```
    if (vn < 0.0) {
      double e =
          -vn > REST_THRESH
              ? 0.5 * (restitution_of(g, c->a) + restitution_of(g, c->b))
              : 0.0;
      double j = -(1.0 + e) * vn / (ia + ib);
      // clamp the accumulated impulse at nonnegative: no pulling
      double nacc = c->jn_acc + j;
      if (nacc < 0.0) {
        j = -c->jn_acc;
        nacc = 0.0;
      }
      c->jn_acc = nacc;
      if (ba)
        ba->vel = vsub(va, vscale(c->normal, j * ia));
      if (bb)
        bb->vel = vadd(vb, vscale(c->normal, j * ib));
    }
    // coulomb friction against the accumulated normal impulse
    va = ba ? ba->vel : v2(0.0, 0.0);
    vb = bb ? bb->vel : v2(0.0, 0.0);
    vr = vsub(vb, va);
    vn = vdot(vr, c->normal);
    vec2 tangent = vsub(vr, vscale(c->normal, vn));
    double tl = vlen(tangent);
    if (tl > 1e-9 && c->jn_acc > 0.0) {
      tangent = vscale(tangent, 1.0 / tl);
      double jt = -vdot(vr, tangent) / (ia + ib);
      double mu = sqrt(friction_of(g, c->a) * friction_of(g, c->b));
      double bound = mu * c->jn_acc;
      if (jt > bound)
        jt = bound;
      if (jt < -bound)
        jt = -bound;
      if (ba)
        ba->vel = vsub(ba->vel, vscale(tangent, jt * ia));
      if (bb)
        bb->vel = vadd(bb->vel, vscale(tangent, jt * ib));
    }
```

listing 333: collide.c, one contact inside the velocity loop: restitution gate, accumulated impulse clamp, coulomb friction bound (source: math/capstone/src/collide.c, lines 244-282)

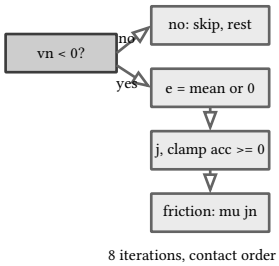


figure 188: one contact in one solver pass: the three gates in order, approach speed, restitution threshold, friction bound

PITFALL: WHEN THE PIN AND THE CODE DISAGREE

The positional-correction mirror moved one circle by the full corr while the C moved it by corr · inv_mass. The instinct on a red pin is to fix the C toward the mirror. The right move is to re-derive both: the mass-

> weighted split conserves the center of mass, the full shift does not, so the mirror was the bug. A pin is a hypothesis about the code, not authority over it.

31.4. deterministic grid pathfinding

Macro steering needs paths on the 24 by 24 wall grid, and the module is a textbook 4-connected a* with manhattan heuristic, admissible and consistent on unit costs, plus one idea that makes it reproducible: the frontier is ranked by a single `uint64` key packing f, then $4095 - g$, then the cell index. Equal-cost ties therefore pop in a fixed order, deeper nodes first, then lower index, forever, with no qsort and no pointer order anywhere. Chapter book 2, ch 15 (mathematics for programmers, graphs and order theory) supplies the order theory, the dsa book owns a* as an algorithm, and this module owns only the determinism argument.

The pins come from an independent breadth-first search over the same grids. Corner to corner, $(1, 1)$ to $(22, 22)$, costs 42. The west column run $(1, 1)$ to $(1, 22)$ costs 21 straight cells. Into the center pocket, 5. The serpentine corridor, two wall rows with one gap each, forces its unique 65-step snake, and a goal sealed by an 8-neighbor wall ring returns −1, the unreachable marker. Repeated queries after unrelated traffic on the same context reproduce the g and parent arrays exactly, which is the no-hidden-state gate for the navigator.

The dry run: spawn to spawn. Player 0 sits at cell (8, 12), player 1 at (16, 12), and the manhattan distance is 8. But cells (11, 12) and (12, 12) are the center block, so row 12 is plugged between the spawns.

1. Drop to row 13 at x = 10, cross under the block, climb back at x = 13: the walk is 2 + 1 + 3 + 1 + 3 = 10 moves.
2. The pin agrees, 10, and the test says it in one line: manhattan would be 8, the arena makes the honest price 10.

```
static void relax(struct astar_ctx *ctx, const bool *blocked, int from, int to,
                  int gx, int gy) {
  if (blocked[to] || ctx->closed[to])
    return;
  int ng = ctx->g[from] + 1;
  if (ctx->g[to] != -1 && ctx->g[to] <= ng)
    return;
  ctx->g[to] = ng;
  ctx->parent[to] = from;
  int tx = to % ARENA_CELLS, ty = to / ARENA_CELLS;
  int h = tx > gx ? tx - gx : gx - tx;
  h += ty > gy ? ty - gy : gy - ty;
  ctx->key[to] = rank_key(ng + h, ng, to);
  heap_push(ctx, to);
}
```

listing 334: astar.c, relax: one edge, the ranking key recomputed from g and the manhattan remainder, ties pre-ordered in the key bits (source: math/capstone/src/astar.c, lines 46-60)

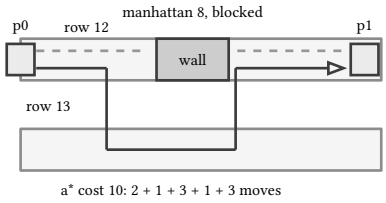


figure 189: spawn to spawn on rows 12 and 13: the straight manhattan line blocked by the center block, the 10-step detour below it

31.5. search over the group hash

The midgame search keys its transposition table on the canonical hash of chapter book 2, ch 30 (mathematics for programmers, capstone: the arena game, design), and the implementation is where the quantization meets

the group. Positions quantize to 64ths of a unit on a side of $24 \cdot 64 = 1536$ quanta, velocities to 256ths, box headings to quarter-turn bins, and each of the 8 dihedral transforms then acts as exact integer arithmetic: points as $(x, y) \to (1536 - y, x)$ and friends, vectors without the offset, headings as $k \to k + 1 \bmod 4$ under rotation and the negation family under reflection. Eight fnv folds later the minimum is the canonical value.

The dry run: the initial state at seed 1. Its raw hash pins at 0xc0aeb7a9977e46a3. The 8 orbit hashes pin at 0xc15c7a1642de31fc, 0x051763866518b764, 0xa587de6a2bbf29a8, 0xc5e381f336ff4230, 0x5ca491b603e1ee30, 0x86e572001a2b6a98, 0x1b78945b0dda51b0, 0xf42f1e75839a1140. The minimum is the second, the quarter turn, so 0x051763866518b764 is the canonical hash, and the test builds the exact r90 and flip-y transforms of the state in doubles and checks both land on that same canonical value while their raw hashes stay distinct.

The third bug is here. The first version of `orbit_hash` applied the heading transform to every body, circles included. A circle's heading is a don't-care field, but rotating it changed those bits, so the 8 orbit hashes of a rotated state no longer matched the original's, symmetric states stopped sharing, and the canonical equality gate went red. The fix folds a constant 0 byte for non-box headings: orientation belongs to boxes, the group action must not act on meaningless fields.

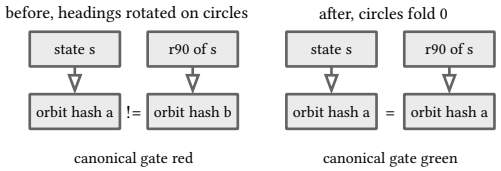


figure 190: the circle-heading bug: rotating a state's meaningless heading bits broke orbit equality, the fix folds them constant

On top of the hash sits the search proper. Nodes live in a 2048-slot open-addressing table keyed by the canonical hash mixed with a mover byte, statistics are integer half-wins so every add is exact, and one search round is: probe the root, take the first untried action in index order or else the ucb argmax with ties to the lowest index, run one window of the mover's macro with the opponent parked on guard, recurse, back the outcome up as 0, 1, or 2 half-wins. The budgets are counts, 160 iterations, 4 windows of tree, 2 playout windows, and a full table degrades to playouts rather than failing. The gates: the root records exactly 160 visits, the action visits sum to it, rewards stay within 2 per visit, two searches from the same seed produce bit-identical root statistics, and a fresh search after unrelated interference reproduces them, no hidden state.

```
static uint32_t simulate(struct mctx *m, struct game *g, int mover, int depth,
                         rng *r) {
  if (depth ≥ TREE_DEPTH || game_over(g))
    return playout(g, r);
  struct tt_entry *e = tt_probe(m, node_key(g, mover), true);
  if (!e)
    return playout(g, r);
  int action = e->n == 0 ? 0 : select_action(e);
  uint8_t act[2] = {M_GUARD, M_GUARD};
  act[mover] = (uint8_t)action;
  run_window(g, act);
  uint32_t z2 = simulate(m, g, 1 - mover, depth + 1, r);
  uint32_t mover_z2 = mover == 0 ? z2 : 2u - z2;
  e->n++;
  e->w2 += mover_z2;
  e->na[action]++;
  e->wa2[action] += mover_z2;
  return z2;
}
```

listing 335: mcts.c, simulate: probe, choose, run the mover's window, recurse, back up integer half-wins (source: math/capstone/src/mcts.c, lines 66-84)

NOTE: WHAT THE GROUP HASH SHARES

The canonical hash shares table entries across exact symmetries of the map and physics. It is exact there and a heuristic above: retreat corners and west/east orb filters are fixed by the identity transform alone, and rewards are player-keyed, so a rotated position hashes with its sibling while meaning a different game. Quantization also merges physics-distinguishable states: drift per step 0.0393 against a position quantum of 0.015625. The hash is a sharing mechanism, not an equality proof, and the strength numbers below ride the deterministic binary, not a claim of state uniqueness.

The strength gate runs four fixed 30-window scenarios against a seeded random policy from a midgame start: the search's orb leads pin at (2, 1, 1, 4) over seeds 1 to 4, four wins, zero losses. Those counts are witnessed-deterministic, pinned from the run and reproduced by the same binary, which is the honest tier of chapter 30′s budget, and chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification) measures what they do and do not prove.

31.6. the allegro host

The host is the impure shell: it owns a display, a keyboard, two search trees, and a timer, and it never re-implements a line of core logic. Its windowed mode is window-commit play, one macro per 0.5 s against the search, because `run_window` is the only execution path the core exports and duplicating per-step steering in the host would fork the one code path the design forbids. The accumulator advances by DT per timer tick and executes a whole window per 60 ticks, clamped at 2 windows of debt so a stall owes at most one catch-up window. One search costs tens of milliseconds, load-sensitive, 67 to 95 ms per choose across measurements, well inside the window's 500 ms of idle time, so interactive play runs the identical, pinned-strength ai with no decision gating.

The dry run: one commit window. 60 ticks at $\mathrm{DT} = 1/120$ each is 0.5 s of game time, the search inside it costs tens of milliseconds, and the remainder absorbs the event queue and one rendered frame. A stall clamps at 2 windows of debt, so the worst catch-up the loop can owe is a single window, never a burst.

The bench mode is the determinism witness. No display is created, one memory bitmap receives a render of every step, and the mode prints the core's exact rolling hash line plus a fnv checksum over the locked pixels, proof the render actually ran on the same states. The scripted control pattern and the selfplay loop are mirrored from the core driver in a few lines each, because one binary owns one `main`, and the pins make any drift between the copies loud: bench 7:1200 prints `hash=0x81fb1cc382429465`, byte-matching the clang-built `arena.exe`, and the selfplay table diffs clean against it.

One accommodation the host makes is structural: the mingw lane resolves double `sin`, `cos`, and `atan2` through msvcrt.dll, whose bits differ from the core's ucrt, and player 1 starts at heading exactly π, so without routing the state hash diverges at step 0. The host defines those three functions to resolve through ucrtbase.dll, probed bit-identical to the core's static ucrt, and refuses to run if that dll is missing. The core stays cross-libm fragile by construction, the routing is a lane accommodation rather than core logic, and chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification) puts numbers on both halves.

```
while (running) {
  ALLEGRO_EVENT ev;
  al_wait_for_event(eq, &ev);
  if (ev.type == ALLEGRO_EVENT_TIMER) {
    // fixed-timestep accumulator: one decision window per 60 dt ticks,
    // the cadence the core's macro layer is built around. one search
    // costs tens of ms (load-sensitive, measured 67-95 ms per choose),
    // well inside the window's 500 ms of idle time, so play stays live
    // without gating the decision frequency or shrinking the budgets
    acc += DT;
    if (acc > 2 * WINDOW * DT)
      acc = 2 * WINDOW * DT; // a long stall may owe one window, not a burst
    while (acc ≥ WINDOW * DT) {
      acc -= WINDOW * DT;
      if (paused || game_over(&g))
        continue;
      uint8_t act[2] = {queued, ai_choose(&ai, &g, 1, &tree[1], &r)};
      mine = act[0];
      theirs = act[1];
      run_window(&g, act);
    }
```

listing 336: host main.c, the accumulator loop: one window per 60 dt ticks, both macros committed together through run_window (source: math/capstone/host/main.c, lines 191-211)

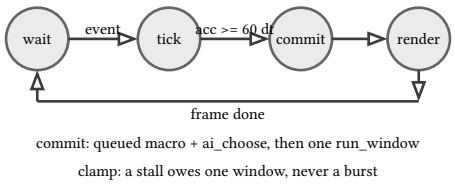


figure 191: the windowed loop as a state machine: timer ticks feed the accumulator, whole windows commit, every frame renders

Every module now has its numbers on the table. Chapter book 2, ch 32 (mathematics for programmers, capstone: the arena game, verification) assembles the matrix of disciplines to modules, runs the determinism proofs across processes and lanes, and measures the ai honestly.

sources: implementation facts read from the committed capstone source, books/math/capstone/src physics.c, collide.c, astar.c, mcts.c, ai.c, state.c, test_physics.c and books/math/capstone/host main.c, 2026-09-22. Bridge chapters quoted from the corpus's own text: quadrature for the verlet bound and round trip, inner for the projection arithmetic behind sat, graphs for the order-theory reading of the ranking key, groups and actions for the quantized dihedral action, capstone-design for the pin tiers and module map. No external pages fetched for this chapter, none cited. All pins witnessed by `pwsh -NoProfile -File playground/math-capstone/build.ps1`, 64 checks plus driver gates, format clean, and `pwsh -NoProfile -File tools/run-math-capstone.ps1`, host bench 7:1200 matching arena.exe bit for bit, both run green on 2026-09-22.

32. capstone: the arena game, verification

The last chapter of the book is an audit. The design of chapter book 2, ch 30 (mathematics for programmers, capstone: the arena game, design) promised disciplines, the implementation of chapter book 2, ch 31 (mathematics for programmers, capstone: the arena game, implementation) pinned numbers, and this chapter assembles the evidence: the matrix that maps every discipline to the module and the test that carries it, the determinism hash with its three pins proven in-process, cross-process, and cross-lane, the perturbation and quantization behavior of the state hash, the ai strength numbers with their caveats stated rather than buried, the allegro bench gates, and the audit totals. Every behavioral claim below is one of the 64 checks in the capstone test suite or one of the host gates, both run green on 2026-09-22 (`pwsh -NoProfile -File playground/math-capstone/build.ps1` and `pwsh -NoProfile -File tools/run-math-capstone.ps1`), or a sentence read from the committed source or the gate output it describes.

32.1. the discipline-to-module matrix

The book taught 29 chapters of mathematics and the capstone owes each of them a receipt. The matrix below is the accounting: which chapter's discipline lives in which module, and which capstone test carries the proof. Reading it by rows says where a chapter went, reading it by columns says what a module stands on.

discipline	module	what it contributes	witness
ch 2 error	physics, collide	exact-bit kernels, tolerance tiers, the sse2 lane	oscillator and solver pins at 1e-12
ch 22 quadrature	physics	fixed timestep, verlet halves, the energy bound	oscillator gate, 2400 steps
ch 24 strategic	mixed	saddle certificate, integer sampler	test_mixed, 6 checks
ch 25 sequential	mcts	sequential-move tree, rollback as limit	test_mcts root stats
ch 26 adt	state	tagged bodies, fixed-capacity value	test_state init and hash
ch 27 railway	none	error track unused, stages return void	honest empty row
ch 28 monads	ai	state threading through explicit arguments	run_window purity
ch 29 pipeline	main, host	pure core, impure shell, two-run gate	replay mode, bench diff
ch 16-17 groups, actions	state	d4 canonical form, quantized orbits	canonical hash gates
ch 15 graphs	astar	total-order ranking, admissible heuristic	test_astar 12 checks

The two rows that are not rows of credit matter as much as the ones that are. Chapter book 2, ch 27 (mathematics for programmers, option, result, and the railway) taught option and result plumbing, and the capstone uses none of it: every physics stage returns void because a fixed-step simulation has no recoverable failure inside a step, so that discipline appears here as a deliberate empty cell, not an omission. And chapter book 2, ch 28 (mathematics for programmers, functors and monads in c) contributes no combinator to the C, only the habit it taught: state moves through explicit arguments, never static cells, which is why the search trees live in the driver and the host while the core stays pure. The two numbers of the matrix: 10 disciplines accounted for across 6 module groups, and every non-empty cell names one test that would go red if the discipline regressed.

Read by columns the matrix says what each module stands on. The state module leans on the algebra chapter for its shape and the group chapters for its hash, two disciplines in 287 lines, and its 18 checks split evenly between them. The physics and collision modules are the floating point book's territory, integrator and tolerance policy, and carry the largest single suite. The mixed and mcts modules are the game theory chapters made executable, one as exact integers, the other as a reproducible stochastic search. The shell column is chapter book 2, ch 29 (mathematics for programmers, pipelines and effects) alone, and it is the column that makes every other column testable: because the trees and the io live outside the core, every gate below runs against a pure value.

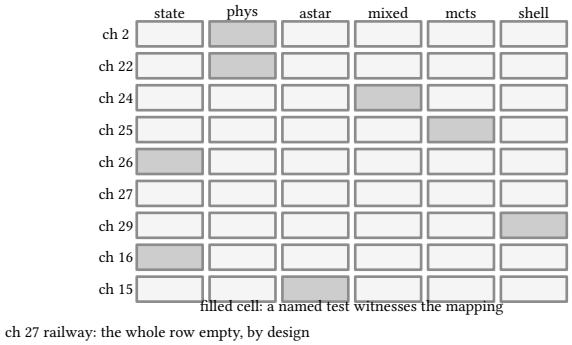


figure 192: the discipline-to-module matrix: filled cells where a chapter's discipline lands in a module, the railway row honestly empty, the monads row carried by the table above rather than the grid

```c
static uint64_t rollout(struct game *g, uint64_t seed, int steps, bool echo) {
  game_init(g, seed);
  uint64_t h = 0xcbf29ce484222325ULL;
  for (int i = 0; i < steps && !game_over(g); i++) {
    struct controls c[2] = {scripted(i, 0), scripted(i, 1)};
    physics_step(g, c);
    h = fnv_u64(h, raw_hash(g));
  }
  if (echo) {
    int taken = 0;
    for (int i = 0; i < g->n_orbs; i++)
      taken += g->orbs[i].taken ? 1 : 0;
    printf("steps=%u score=%d:%d orbs_taken=%d hash=0x%016llx\n", g->step,
           (int)g->score[0], (int)g->score[1], taken, (unsigned long long)h);
  }
  return h;
}
```

listing 337: main.c, rollout: the scripted controls, one physics step, and the rolling fnv fold over every per-step raw hash (source: math/capstone/src/main.c, lines 26-42)

32.2. the determinism hash and its three pins

The raw hash folds the exact bits of every dynamic double through fnv1a-64, step after step, and the driver folds again: a rolling hash over the per-step raw hashes, so the final value is a witness of the entire trajectory, not just the endpoint. Two runs agree on it if and only if every bit of every dynamic field agreed at every step. There is no epsilon, no tolerance, no probability. That is the relational tier of the numerics budget doing the heaviest lifting in the book.

Three pins anchor it, each at a different scale. Seed 7, 1200 steps, one tenth of a game: `steps=1200 score=0:0 orbs_taken=0 hash=0x81fb1cc382429465`. Seed 42, 2400 steps, run twice inside one process by the replay mode, which also memcmps the whole final state: `replay ok ... hash=0xc03668ca5b407088`. Seed 1, the full 7200-step game:

`hash=0xfbe9b7d168128e74`. The first pin is the workhorse, the second adds the in-process double run, the third says a whole game holds the line.

The dry run: what one pin actually covers. Take the 7:1200 rollout.

1. 1200 physics steps, each folding 4 bodies, 8 orbs, scores, and the step counter into the raw hash, then the driver folding that in: on the order of 10^5 double bits and counters chained through fnv.
2. One flipped bit anywhere, a contact order swap, a drag multiply reordered, changes every hash after it and the final value, which is why the pin is a trajectory witness rather than a final-state checksum.

Cross-process: the same binary run twice as separate operating system processes prints byte-identical output, no allocator or address noise anywhere in the loop. Cross-lane: the gcc-built allegro host, a different compiler, runtime, and rendering stack, runs the same rollout in its bench mode and prints the identical hash line plus a render checksum, and the runner diffs the two processes directly. The road to that agreement is the trig routing of chapter book 2, ch 31 (mathematics for programmers, capstone: the arena game, implementation): without it the mingw lane's msvcrt `sin(pi)`, 0x3ca1a60000000000 against the core's 0x3ca1a62633145c07, diverges player 1's heading bits at step 0 and lands the hash on 0xe698373cf0161dd6 instead of the pin. The core remains cross-libm fragile by construction, the routing is witnessed for every heading the pinned rollouts and the selfplay table generate, and that is evidence, not a proof over all inputs.

One detail makes the replay gate stronger than it looks: `game_init` assigns `*g = (struct game){0}` before filling fields, zeroing the struct's padding bytes, so the replay's `memcmp` over the whole state compares padding that the hash never folds. A run could in principle agree on every folded bit while leaving different garbage in padding, and this gate closes even that gap. Byte equality of the entire value, not just the semantic fields, is the standard the driver holds.

seed	steps	gate	hash	agreement
7	1200	twice-run, build.ps1	0x81fb1cc382429465	process, lane
42	2400	replay, memcmp, in-process	0xc03668ca5b407088	bits and bytes
1	7200	full game, host -Full	0xfbe9b7d168128e74	process, lane

```c
uint64_t raw_hash(const struct game *g) {
  uint64_t h = 0xcbf29ce484222325ULL;
  h = fnv_u64(h, (uint64_t)g->step);
  h = fnv_u64(h, g->seed);
  h = fnv_i64(h, g->score[0]);
  h = fnv_i64(h, g->score[1]);
  for (int i = 0; i < g->n_bodies; i++) {
    const struct body *b = &g->bodies[i];
    h = fnv_byte(h, b->kind);
    h = fnv_byte(h, b->player);
    h = fnv_u64(h, bits_of(b->pos.x));
    h = fnv_u64(h, bits_of(b->pos.y));
    h = fnv_u64(h, bits_of(b->vel.x));
    h = fnv_u64(h, bits_of(b->vel.y));
    h = fnv_u64(h, bits_of(b->heading));
  }
  for (int i = 0; i < g->n_orbs; i++) {
    h = fnv_byte(h, g->orbs[i].taken ? 1 : 0);
    h = fnv_i64(h, (int64_t)g->orbs[i].respawn);
    h = fnv_u64(h, bits_of(g->orbs[i].home.x));
    h = fnv_u64(h, bits_of(g->orbs[i].home.y));
  }
  return h;
}
```

listing 338: state.c, raw_hash: the fnv fold over the exact bits of every dynamic double, score, and orb field (source: math/capstone/src/state.c, lines 101-124)

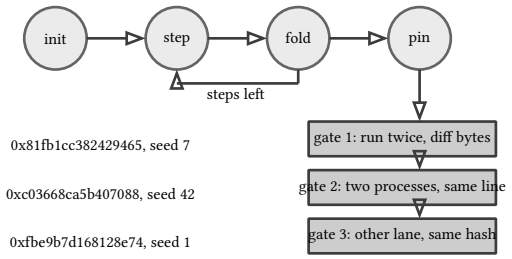


figure 193: the driver's hash discipline as a state machine: seed to init, the step and fold loop, then three comparison gates closing on the pins

VERIFY: THE TWO COMMANDS

`pwsh -NoProfile -File playground/math-capstone/build.ps1` builds the core with clang 23 msvc-target at `-std=c23 -Werror -Wall -Wextra`, runs all 64 checks, then the driver gates: the in-process replay and the twice-run hash pin. `pwsh -NoProfile -File tools/run-math-capstone.ps1` builds the allegro host with gcc 15, runs the offscreen bench twice, checks the pin, and diffs against arena.exe directly. Both were run to green on 2026-09-22 to write this chapter. The 7:1200 and 42:2400 pins, the check counts, and the bench verdicts above come from their output; the 1:7200 pin comes from the arena driver leg of the same session, and the libm bit patterns from the core's lane as those pins verify it.

32.3. perturbation, sensitivity, and the quantization caveat

A hash is only as good as its failure behavior, and the suite probes both directions. Perturbation: change the controls for 60 steps after a 480-step scripted run and the raw hash of the two trajectories diverges, the test gate for "different inputs, different states". Sensitivity: score, one body position, one orb's taken flag each flip the raw hash of the initial state, so no dynamic field is outside the fold. The dry run: the pinned hash pair. The seed 1

initial state hashes to 0xc0aeb7a9977e46a3, the same constructor at seed 2 to 0x2fdaffeaa0a14f6c, distinct because the seed itself is a folded field, and both values pin the fold so a silent field change cannot hide. And the canonical hash, the one the search shares on, has a coarser honesty.

The quantization caveat, stated with numbers. Positions quantize to 64ths of a unit, a quantum of 0.015625, while a body at cruising speed drifts 0.0393 per step, about 2.5 quanta. Two states whose positions differ by less than half a quantum, 0.0078, land in the same bin and share a table entry even though the physics distinguishes them, and because the drift carries 2.5 quanta per step they separate again within one window. The chapters therefore never claim distinct states cannot share an entry. The claim run in the other direction is the one that holds: states related by an exact symmetry of the map always share, so the sharing mechanism never splits what the group joins.

The value-invariance caveat is the same honesty one level up. The canonical hash is exactly invariant under the 8 dihedral transforms for the map and physics layers, positions, velocities, box headings, orbs. The player-side semantics are fixed by the identity transform alone: retreat targets (1, 1) and (22, 22), the opening macros filter orbs by west against east, rewards key on `score[0]` minus `score[1]`. A rotated position hashes with its sibling while meaning a different game, so the transposition table's sharing is a heuristic in exactly the region where the search consumes it. Exact below the semantics, approximate above them, and no impossibility claims anywhere.

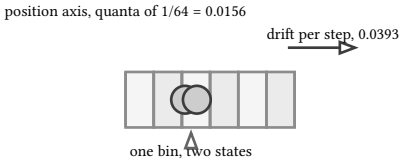


figure 194: quantization on a number line: two states inside one 1/64 bin collide, the per-step drift spans two and a half bins

```
constexpr int64_t QP = 64;      // position quanta per unit
constexpr int64_t QQ = 24 * 64; // quantized arena side
constexpr int64_t QV = 256;     // velocity quanta per unit

static int64_t qpos(double v) { return (int64_t)llround(v * (double)QP); }
static int64_t qvel(double v) { return (int64_t)llround(v * (double)QV); }

// heading quantized to quarter-turn bins, exact group action: rotations add
// 1 mod 4, the reflections negate/flip as derived in the book
static int64_t qhead(double h) { return (int64_t)llround(h / (PI / 2.0)) & 3; }
```

listing 339: state.c, the quantization constants: 64 position quanta per unit, 256 velocity quanta, arena side 1536, headings in quarter-turn bins (source: math/capstone/src/state.c, lines 134-143)

32.4. ai strength, honestly

The strength numbers sit in the witnessed tier of the numerics budget, and this section says what that means before quoting them. A witnessed-deterministic pin is a number pinned from a deterministic run and reproduced by the same binary on every rerun by construction. It is not independently recomputed, it moves if any budget constant moves, and it proves the binary is reproducible, not that the strategy is optimal.

With that said, the measurements. Against a seeded random policy over four fixed 30-window scenarios from a midgame start, the search's orb leads pin at (2, 1, 1, 4) over seeds 1 to 4, four wins and zero losses. Selfplay, search against search over three seeds: player 0 wins all three at 13:8, 14:11, and 15:11. The timing claim is a cadence claim, because absolute timings are load-sensitive. One game is 240 chooses, 120 windows of 60 steps times 2 players, and the three-game leg is 720. Measured per choose across machines and loads: 94.6 ms from a 22.7 s game, 67.1 ms from a 48.3 s three-game leg, 67.5 ms from a fresh 16.2 s game, all well inside the 0.5 s commit window, which is why the windowed host needs no decision gating and runs up no frame debt.

measurement	opponent	result	tier
4 scenarios, 30 windows each	seeded random	(2, 1, 1, 4), 4-0	witnessed

measurement	opponent	result	tier
selfplay, seeds 101 to 135	same search	p0 3 of 3: 13:8, 14:11, 15:11	witnessed
one choose, budget 160		67 to 95 ms	across machines and loads
root statistics, same seed twice		bit identical	relational

The selfplay sweep is 3 for 3 for player 0, and that number is structural, not evidence of tuning. Two asymmetries point the same way by construction: inside the tree, player 0 commits the window and player 1 answers, and the opening payoff table is player-0 sided, value 11/9 in p0′s favor. Sequential games with a first-mover advantage reward moving first, chapter book 2, ch 25 (mathematics for programmers, sequential and repeated games) in one line, and this arena is one of them. Three games is also a sample of three, far too small to price the advantage, exactly large enough to show its sign.

The search measurements are deterministic the same way the physics pins are, for the same reason: budgets are iteration counts, never clock reads, the tree's only randomness is the explicit xorshift stream, and the statistics are integers. The same seed therefore rebuilds the same tree bit for bit, which is what the root-statistics gates pin, and it is why a strength number can be a pin at all. Change any budget constant, 160 iterations, depth 4, playout 2, the exploration constant, and every strength number in this section moves, which is the caveat written into the test header rather than the prose.

The idealization disclosure the design chapter owes in full: the tree models each window as sequential, the mover executes its macro while the opponent is parked on guard, but live play and the random playouts run both players' macros through the same window simultaneously. The tree therefore evaluates positions under an opponent that is more passive than any real opponent, including itself in selfplay. The one code path rule keeps this honest in the way that matters: search, live play, and playouts all execute windows through the same `run_window`, so the idealization lives only in the tree's bookkeeping of whose macro runs, never in the physics both sides share. What it costs in playing strength is not measured here, and the chapter does not guess.

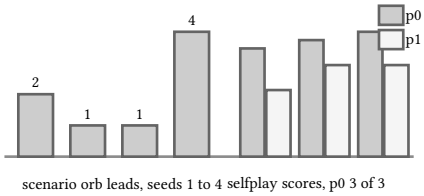


figure 195: strength measurements as bars: the four scenario orb leads (2, 1, 1, 4) and the three selfplay margins (13:8, 14:11, 15:11), all p0-favored

```
  // scenarios (midgame start, winner by orb count). counts are pinned from
  // the deterministic runs: seeds 1..4 give (2, 1, 1, 4) orb leads
  int wins = 0, losses = 0;
  int leads[4] = {0, 0, 0, 0};
  for (int sc = 0; sc < 4; sc++) {
    game_init(&g, (uint64_t)(sc + 1));
    uint8_t op[2] = {M_GUARD, M_GUARD};
    for (int w = 0; w < OPENING_STEPS / WINDOW; w++)
      run_window(&g, op); // deterministic midgame start, orbs untouched
    mcts_reset(&ta);
    rng me = rng_seed(0xA000 + (uint64_t)sc);
    rng foe = rng_seed(0xB000 + (uint64_t)sc);
    for (int w = 0; w < 30; w++) {
      uint8_t a[2];
      a[0] = mcts_decide(&ta, &g, 0, &me);
      a[1] = random_action(&foe);
      run_window(&g, a);
    }
    leads[sc] = (int)g.score[0] - (int)g.score[1];
    if (leads[sc] > 0)
      wins++;
    else if (leads[sc] < 0)
      losses++;
  }
  printf("scenario orb leads: (%d, %d, %d, %d)\n", leads[0], leads[1], leads[2],
         leads[3]);
  CHECK(leads[0] == 2 && leads[1] == 1 && leads[2] == 1 && leads[3] == 4,
        "scenario orb leads pinned: (2, 1, 1, 4) over seeds 1..4");
  CHECK(wins == 4 && losses == 0,
        "mcts beats the seeded random policy: 4 wins, 0 losses");
```

listing 340: test_mcts.c, the strength gate: four scenarios against the seeded random policy with the pinned leads and the 4-0 record (source: math/capstone/src/test_mcts.c, lines 96-125)

32.5. the allegro bench

The host gates run through one runner script, and its default pass is the one wired into the corpus verify: build the host with gcc 15 against the pinned allegro monolith, run the offscreen bench at seed 7 for 1200 steps twice, check the pin 0x81fb1cc382429465, and diff the output line against the clang-built arena.exe directly, about 20 seconds end to end. The full gate adds the two longer rollouts, 42:2400 and the whole game 1:7200, both arena-matched, and the selfplay table diff against the core driver, about 3 minutes.

The bench output carries two numbers per run, and both matter. The `hash=` line is the core's rolling trajectory hash, identical across lanes. The `render=` line is an fnv checksum over the locked pixels of the final framebuffer, proof the offscreen renderer really walked the same states rather than skipping them, and it is identical on repeated runs too. The windowed mode has no pin, human input is entropy by design, so its determinism claim rests on the bench and selfplay witnesses sharing its code paths, plus a smoke run: seed 7 with 600 frames exits cleanly at `steps=600 score=1:0` in 5.6 seconds of wall time, which is real-time pacing at 120 hz with all 10 ai searches absorbed inside the cadence, no frame debt.

Two housekeeping rules keep the gate honest. The executables build and run from a temporary directory with the allegro toolchain directory prepended to `PATH`, nothing is copied beside the binaries, so a stale dll can never leak a wrong libm into a green run. And the accumulator clamps its debt at 2 windows, so a long stall replays at most one catch-up window instead of a burst that could mask a timing dependence inside a supposedly pinned loop.

The hud fits because it is bounded in code, not eyeballed: the builtin font advances 8 pixels per glyph on a 384-pixel view, 47 characters of room, line 1 truncates seeds past 7 digits to their low 7 behind an ellipsis and drops the phase word when the line would still pass 47, scores are `int8_t` so their field cannot grow, and the worst surviving line is 46 characters, 368 of the 384 pixels. Determinism grep on the host source finds no clock or randomness calls feeding state, the one hit being a timer event source name, a substring false positive, and the timer only paces the accumulator.

measurement	value	gate	what it rules out
bench 7:1200, twice	identical lines	default runner	nondeterministic render path
render checksum, twice	identical	default runner	skipped offscreen frames
windowed smoke, 600 frames	5.6 s, steps=600, score 1:0	–frames exit	frame debt under search load
hud worst line	46 chars, 368 of 384 px	clamped in hud_lines	clipped hud strings
selfplay diff, 3 games	table identical to arena.exe	-Full runner	steering path divergence

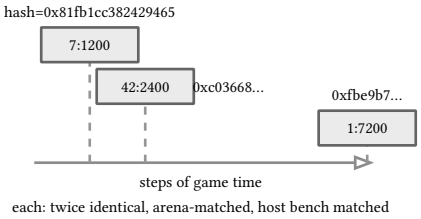


figure 196: the three pinned rollouts on a steps axis, each with its lane-agreement marks: run twice in-process, two processes, two build lanes

NOTE: WHAT THE BENCH DOES NOT PIN

Windowed play against a human is unpinned by design, and the trig routing's lane equality is witnessed on the headings the pinned runs generate, not proven on all inputs. Both limits are stated here so the green gates are not read wider than they reach. A red pin on an unseen input would be the contract working, not failing.

32.6. audit totals

The audit ledger closes with its counts. The core suite: 18 state checks, 20 physics, 12 astar, 6 mixed, 8 mcts, 64 checks total, zero skipped, zero tolerated failures, every expected value pinned from exact-arithmetic mirrors or the pinned deterministic runs with the tier flagged in each test header. The driver gates: the in-process replay with full-state memcmp, the twice-run cross-process hash, the selfplay table. The host gates: bench twice-run with the render checksum, the direct arena diff, the full-gate long rollouts and selfplay diff. Format checks clean on every core and host file.

The dry run: the ledger sums.

1. 18 + 20 + 12 + 6 + 8 = 64 core checks, and no suite carries a skipped or tolerated check.
2. The two commands at the top of the chapter cover four of the six modes in under a minute on this machine: the driver's replay and twice-run gates, the host's bench and the arena diff. The host's selfplay leg rides -Full at about three minutes, and the windowed mode is a -Window run.

suite	checks	representative gates
test_state.c	18	d4 closure over all cells, raw and canonical pins, r90 and flip-y equality
test_physics.c	20	oscillator within bound, round trip, j = 1.5, momentum zero, respawn at 600
test_astar.c	12	bfs-pinned costs 42, 10, 21, 5, serpentine 65, sealed –1
test_mixed.c	6	saddle certificate 11/9, pinned samples, 6000-draw counts

suite	checks	representative gates
test_mcts.c	8	bit-identical root stats, no hidden state, leads (2, 1, 1, 4)
driver	3 modes	replay memcmp, twice-run hash, selfplay table
host	3 modes	bench twice-run, arena diff, windowed smoke

What the totals deliberately do not claim. Navigation quality: a* optimality is pinned, steering is naive, boxes can buzz in tight corners, and no chapter says otherwise. Hash-sharing impossibility: the quantization caveat of this chapter bars exactly that claim, the table shares entries as a heuristic. Cross-libm equality in general: the routing is witnessed evidence on this machine, on these lanes, for these runs. Optimal play: the strength numbers are witnessed-deterministic pins riding one binary and one set of budget constants, and the first-mover structure means selfplay percentages are not a fair coin either.

The gates have already earned their keep four times, which is the strongest argument for running them, and the ledger of catches is part of the audit.

gate that fired	defect	signal
terminal speed pin	missing second verlet half kick	speed exactly half the ladder, 2.3573 vs 4.7146
canonical equality gate	circle headings rotated by the group action	r90 states stopped sharing the orbit minimum
cross-lane bench pin	msvcrt trig bits at heading pi	hash 0xe698373cf0161dd6 at step 0, not the pin
positional-correction mirror	the pin itself, not the solver	mass-weighted split re-derived, pin corrected

Every gate in this chapter has fired on a real defect, one on a defect in the audit apparatus itself, which is the difference between an audit and a decoration.

Reading a red gate is part of the discipline. A divergence at step 0 with everything after it wrong means the inputs or the elementary functions disagree, the libm class. A divergence late in a run with an early agreement window points at contact ordering or a tie break, the determinism class. A clean hash with a wrong physical number means the arithmetic runs but the physics is miscompiled conceptually, a missing-stage class like the half kick, and the terminal-speed pin is tuned to catch exactly that. The two commands at the top of the chapter reproduce any of these in under a minute on this machine, which is the whole point of pinning numbers small enough to re-derive and large enough to matter.

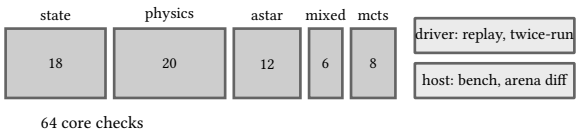


figure 197: the audit ledger: 64 core checks stacked by suite, the driver and host gates beside them

Twenty-nine chapters built a toolkit, three spent it on one program, and the program closes with a receipt for every tool it used. The appendices collect the coverage and sources for the whole book.

sources: verification facts read from the committed capstone source, books/math/capstone/src main.c, state.c, test_mcts.c, test_physics.c headers, and books/math/capstone/host main.c, 2026-09-22. Gate output quoted from the two runs of this session: `pwsh -NoProfile -File playground/math-capstone/build.ps1`, 64 checks, replay ok, twice-run hash identical, format clean, and `pwsh -NoProfile -File tools/run-math-capstone.ps1`, host gates green with bench 7:1200 arena-matched, both on 2026-09-22. Bridge material quoted from the corpus's own chapters: error, quadrature, strategic, sequential, adt, railway, monads, pipeline, groups, graphs. No external pages fetched for this chapter, none cited.

33. appendices: coverage and sources

The registries close the book. Each chapter wrote its topic rows into the coverage matrix as it landed and dated every external page it consulted, and this chapter reads both back the only way that keeps them honest, by computing from them. The moves: the coverage invariant and the chapter ledger, the suite totals recounted against the gates, the source base classified by url prefix, and the book's place in the sixteen-document corpus. The contract: every count below is computed from the live registries at compile time or printed by a gate this chapter names, the suite numbers were recounted on 2026-10-09, and the totals are 33 chapters, 354 topic rows, 111 pinned sources, 113 sample files at 1767 checks plus a 64-check capstone build, 1831 checks over the whole book, zero failures.

33.1. topic coverage

Every topic row names a chapter the manifest knows: 0 rows off the manifest out of 354. Each row carries its check sentence, the exact CHECK that proves the behavioral claim, so the matrix doubles as a test index: a reader who doubts a claim follows the row to its chapter, the chapter to its sample file, and the file to a runnable gate. The only chapter with no rows of its own is appendices, it audits rather than claims. The ledger below joins the computed topic counts to the sample suite, one row per chapter.

ch	chapter	topics	files	checks
ch 1	floating point anatomy	12	3	63
ch 2	error, conditioning, and stability	15	4	55
ch 3	trigonometry	10	3	62
ch 4	coordinate geometry	8	4	51
ch 5	vectors and vector spaces	10	3	55
ch 6	matrices and linear systems	7	4	60
ch 7	inner products and projections	18	4	53
ch 8	matrix decompositions	11	4	52
ch 9	single-variable calculus	6	4	70
ch 10	multivariate calculus	18	4	59
ch 11	continuous optimization	11	4	81
ch 12	automatic differentiation	7	3	57
ch 13	logic, sets, and proof	11	5	52
ch 14	combinatorics	16	4	60
ch 15	graphs and order theory	16	4	60
ch 16	groups and subgroups	6	4	58
ch 17	group actions and structure	11	4	61
ch 18	probability and distributions	7	4	60
ch 19	statistics and inference	12	4	63
ch 20	root finding	6	4	60
ch 21	interpolation and approximation	12	4	63
ch 22	quadrature and differential equations	11	4	58
ch 23	iterative methods and conditioning	16	4	68
ch 24	strategic games	8	4	60
ch 25	sequential and repeated games	20	4	59
ch 26	algebraic data types in c23	12	4	59
ch 27	option, result, and the railway	6	4	78
ch 28	functors and monads in c	14	4	61

ch	chapter	topics	files	checks
ch 29	pipelines and effects	18	4	69
ch 30 to 32, the capstone: 19 topic rows over one program, a 21-file core with five test drivers and a 4-file gcc host lane, one shared 64-check build gate				
33	appendices: coverage and sources	0	-	-
total	**33 chapters**	**354**	**138**	**1831**

The densest chapters by row count are ch 25 at 20 topic rows and ch 7, 10, and 29 at 18, the inner product, multivariate, and pipeline chapters that everything else leans on. The heaviest check suites are ch 11 at 81 and ch 27 at 78, optimization with its ladders and the railway with its trace fixtures. The lightest topic count on a content chapter is 4, the verification capstone, whose four rows each pin a whole gate.

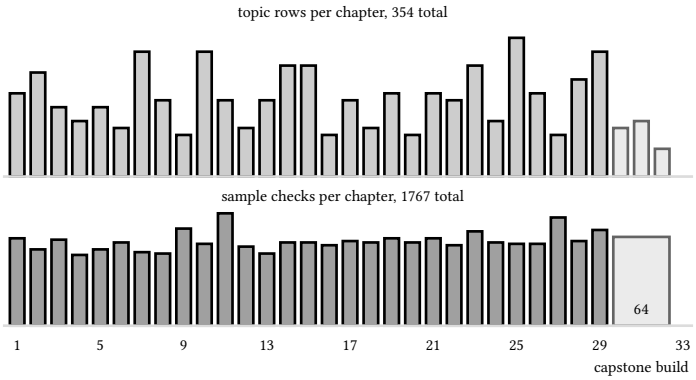


figure 198: the audit strip, topic rows per chapter above in light gray, sample checks below in dark, the capstone spans chapters 30 to 32 as one 64-check build

33.2. the suites and the totals

One c lane carries the whole book. The 29 content chapters before the capstone keep 113 sample files under `samples/src/ChNN`, each file a standalone program that prints one ok line per check and a total line, 1767 checks in all. The gate:

```
pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot
books/math/samples/src
```

The run of 2026-09-22 printed 112 files and 1760 checks with the format and asan legs clean, and the per-chapter sums reproduced all 29 chapter closers digit for digit, the ledger above carries those sums. The footer also prints 0 ir/objdump/roundtrip assertions, this book carries no disassembly pins, its pins are numeric values computed first by hand or by a playground one-off and then matched against the compiled run.

The capstone rides two more lanes, both quoted from chapter 32's session of 2026-09-22 rather than rerun here. The core lane builds `books/math/capstone/src`, 21 files including the five test drivers `test_astar.c`, `test_mcts.c`, `test_mixed.c`, `test_physics.c`, and `test_state.c`, behind `playground/math-capstone/build.ps1`, 64 checks with replay ok and the twice-run hash identical. The host lane builds the 4-file gcc rendering shell in `books/math/capstone/host` behind `tools/run-math-capstone.ps1`, bench 7:1200 with the arena matched byte for byte against the clang core.

lane	size	unit	gate
c samples	113 files, 1767	checks, one ok line per file	`pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src`
capstone core	21 files, 64	checks over five test drivers	`playground/math-capstone/build.ps1`
capstone host	4 files	host gates, gcc-built lane	`tools/run-math-capstone.ps1`

Beyond the checks, the 32 content chapters render 337 listings, every one a slice of a real sample file read live at compile time, and carry 196 figures, one per section facet, all inside the 24 px guard band of the figure audit.

33.3. pinned sources

Every external claim in the book carries a dated url or a page range in the cached mml draft, 111 rows over two access dates, 67 rows on 2026-09-21 and 34 on 2026-09-22. The classifier below reads only url prefixes, so the class counts are mechanical:

class	rows	rule
mml page ranges	31	url starts `ref/mml-book.pdf`
cppreference	18	url starts `https://en.cppreference.com`
wikipedia	18	url starts `https://en.wikipedia.org`
other web	25	everything else, books and papers by name
dlmf	9	url starts `https://dlmf.nist.gov`
corpus internal	8	url starts `books/`, `playground/`, or `tools/`
mml anchor sheets	2	url starts `ref/mml/`

The mml page ranges are the spine, 31 rows against the cached `ref/mml-book.pdf` draft of 2024-01-15 plus the two anchor sheets that mediated the reading. The web rows cluster where the c library and the reference formulas live, cppreference and DLMF, and where a definition needed one canonical page, wikipedia. The corpus-internal rows are this repo's own witnesses, capstone sources, build gates, and two chapters of the c book that pin defer and nodiscard behavior. The full index:

topic	url	accessed
goldberg floating point survey	https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html	2026-09-21
cppreference fenv	https://en.cppreference.com/w/c/numeric/fenv	2026-09-21
cppreference FE_round	https://en.cppreference.com/w/c/numeric/fenv/FE_round	2026-09-21
cppreference nextafter	https://en.cppreference.com/w/c/numeric/math/nextafter	2026-09-21
cppreference math functions	https://en.cppreference.com/w/c/numeric/math	2026-09-21
cppreference sqrt	https://en.cppreference.com/w/c/numeric/math/sqrt	2026-09-21
cppreference arithmetic types	https://en.cppreference.com/w/c/language/arithmetic_types	2026-09-21
clang fenv access pragma default	https://clang.llvm.org/docs/UsersManual.html#controlling-floating-point-behavior	2026-09-21
angles, orthogonality, rotations	ref/mml-book.pdf ch 3 pp 76-78, 81, 83-85, 91-94	2026-09-21
trig definitions and identities	https://dlmf.nist.gov/4.14	2026-09-21
symmetry and multiple-angle formulas	https://dlmf.nist.gov/4.17 and https://dlmf.nist.gov/4.21	2026-09-21
inverse trigonometric functions	https://dlmf.nist.gov/4.23	2026-09-21
solution of triangles, sine rule, law of cosines, heron	https://dlmf.nist.gov/4.42	2026-09-21
sin special values	https://en.cppreference.com/w/c/numeric/math/sin	2026-09-21
atan2 special values	https://en.cppreference.com/w/c/numeric/math/atan2	2026-09-21
floating point error analysis	https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html	2026-09-21
fenv and fp pragmas	https://en.cppreference.com/w/c/numeric/fenv	2026-09-21
clang _Float16 support	https://clang.llvm.org/docs/LanguageExtensions.html	2026-09-21
lane probes: FLT/DBL/LDBL constants, _Float16 link failure	playground/math-ch02/float16probe.c (local probe)	2026-09-21
matrix decompositions	ref/mml-book.pdf ch 4 pp 99-135 (printed)	2026-09-21
gram-schmidt and spd matrices	ref/mml-book.pdf ch 3 pp 74, 89-90 (printed)	2026-09-21
rectangular trace pair	ref/mml-book.pdf ch 2 p 23 (printed)	2026-09-21
mml linear algebra	ref/mml-book.pdf ch 2 pp 19-60	2026-09-21
mml anchor sheet	ref/mml/anchors-la.md	2026-09-21
elimination flop counts	Trefethen and Bau III, Numerical Linear Algebra, SIAM 1997, lecture 20, cited by name	2026-09-21
univariate differentiation and rules	ref/mml-book.pdf ch 5 pp 141-146	2026-09-21
taylor polynomial and series	ref/mml-book.pdf ch 5 pp 142-145	2026-09-21
gradient implementation checking	ref/mml-book.pdf ch 5 p 149	2026-09-21
forward and reverse autodiff modes	ref/mml-book.pdf ch 5 pp 161-164	2026-09-21
inner products, projections, Fourier modes	ref/mml-book.pdf ch 3 pp 71-94	2026-09-21
vector spaces, subspaces, independence, basis	ref/mml-book.pdf ch 2.4-2.5 pp 37-47	2026-09-21
coordinates, transformation matrix	ref/mml-book.pdf ch 2 pp 50-52	2026-09-21

topic	url	accessed
norms, manhattan, euclidean, dot product	ref/mml-book.pdf ch 3.1-3.2 pp 71-72	2026-09-21
norms, dot product, distance, angle	ref/mml-book.pdf ch 3 pp 70-76	2026-09-21
rotation as a linear map	ref/mml-book.pdf ch 2 p 68 (exercise 2.16e)	2026-09-21
multivariate calculus	ref/mml-book.pdf ch 5 pp 146-169	2026-09-21
backpropagation and automatic differentiation	ref/mml-book.pdf ch 5 pp 159-164	2026-09-21
gradient identities	ref/mml-book.pdf ch 5 pp 158-159	2026-09-21
gradient checking remark	ref/mml-book.pdf ch 5 p 149	2026-09-21
difference quotient and derivative	ref/mml-book.pdf ch 5 p 141	2026-09-21
chain rule and example 5.5	ref/mml-book.pdf ch 5 pp 145-146	2026-09-21
gradient of example 5.7	ref/mml-book.pdf ch 5 p 147	2026-09-21
gradient descent, momentum, constrained and convex optimization	ref/mml-book.pdf ch 7 pp 225-246	2026-09-21
hessians and second-order curvature	ref/mml-book.pdf ch 5 pp 164-165	2026-09-21
verified mml anchor sheet	ref/mml/anchors-calc.md	2026-09-21
static_assert declaration	https://en.cppreference.com/w/c/language/static_assert	2026-09-21
handshake lemma	https://en.wikipedia.org/wiki/Handshaking_lemma	2026-09-21
kirchhoff matrix-tree theorem	https://en.wikipedia.org/wiki/Kirchhoff%27s_theorem	2026-09-21
topological sorting	https://en.wikipedia.org/wiki/Topological_sorting	2026-09-21
lagrange theorem	https://en.wikipedia.org/wiki/Lagrange%27s_theorem_(group_theory)	2026-09-21
cyclic groups	https://en.wikipedia.org/wiki/Cyclic_group	2026-09-21
units mod n	https://en.wikipedia.org/wiki/Multiplicative_group_of_integers_modulo_n	2026-09-21
dihedral group of order 6	https://en.wikipedia.org/wiki/Dihedral_group_of_order_6	2026-09-21
probability space and finite counting	ref/mml-book.pdf ch 6 pp 172-186 (printed)	2026-09-21
binomial coefficients and identities	Concrete Mathematics, Graham Knuth Patashnik, 2nd ed 1994, ch 5-7 (cited by name)	2026-09-21
discrete mathematics counting	Rosen, Discrete Mathematics and Its Applications, 8th ed 2019, ch 6 (cited by name)	2026-09-21
bit-precise integer suffixes	https://en.cppreference.com/w/c/language/integer_constant.html	2026-09-21
clang bitint support	https://clang.llvm.org/docs/LanguageExtensions.html	2026-09-21
probability and distributions	ref/mml-book.pdf ch 6 pp 172-221	2026-09-22
c math library exp pow sqrt fabs	https://en.cppreference.com/w/c/numeric/math	2026-09-22
summary statistics, variance forms, variance additivity	ref/mml-book.pdf ch 6 sec 6.4 pp 186-197	2026-09-21
negative log-likelihood and map estimation	ref/mml-book.pdf ch 8 sec 8.3 pp 264-269	2026-09-21
gaussian likelihood and linear regression mle and map	ref/mml-book.pdf ch 9 sec 9.2 pp 292-302	2026-09-21
erfc definition	https://dlmf.nist.gov/7.2	2026-09-21
c erfc	https://en.cppreference.com/w/c/numeric/math/erfc	2026-09-21
groups, axioms, example groups and non-groups	ref/mml-book.pdf ch 2.4.1 pp 36-37 (printed)	2026-09-22
quotient group normality criterion	https://en.wikipedia.org/wiki/Quotient_group	2026-09-21
finite field multiplicative group cyclic	https://en.wikipedia.org/wiki/Finite_field	2026-09-21
burnside orbit count average	https://en.wikipedia.org/wiki/Burnside%27s_lemma	2026-09-21
diffie-hellman public channel exchange	https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange	2026-09-21
chebyshev polynomial definition T_n(cos theta) = cos(n theta)	https://dlmf.nist.gov/18.5	2026-09-22
chebyshev zeros note	https://dlmf.nist.gov/18.16	2026-09-22
runge phenomenon	https://en.wikipedia.org/wiki/Runge%27s_phenomenon	2026-09-22
safeguarded iteration and multiplicity rate degeneration	Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed, SIAM 2002 (cited by name)	2026-09-22
linear, superlinear, quadratic convergence taxonomy	Trefethen and Bau, Numerical Linear Algebra, SIAM 1997 (cited by name)	2026-09-22
atan principal value, range and zero handling	https://en.cppreference.com/w/c/numeric/math/atan.html	2026-09-22
dlmf quadrature	https://dlmf.nist.gov/3.5	2026-09-22
dlmf ordinary differential equations	https://dlmf.nist.gov/3.7	2026-09-22
extensive form and information sets	https://en.wikipedia.org/wiki/Extensive-form_game	2026-09-22
subgame perfect equilibrium	https://en.wikipedia.org/wiki/Subgame_perfect_equilibrium	2026-09-22
grim trigger critical discount	https://en.wikipedia.org/wiki/Grim_trigger	2026-09-22
vickrey auction truthfulness	https://en.wikipedia.org/wiki/Vickrey_auction	2026-09-22
von neumann minimax theorem statement	https://en.wikipedia.org/wiki/Minimax_theorem	2026-09-22
nash equilibrium existence and definition	https://en.wikipedia.org/wiki/Nash_equilibrium	2026-09-22
mml solving linear systems iteratively	ref/mml-book.pdf ch 2 pp 34-35	2026-09-22
higham accuracy and stability of numerical algorithms	https://nhigham.com/accuracy-and-stability-of-numerical-algorithms/	2026-09-22
cppreference NULL	https://en.cppreference.com/w/c/types/NULL	2026-09-22
cppreference nullptr	https://en.cppreference.com/w/c/language/nullptr	2026-09-22
cppreference attribute nodiscard	https://en.cppreference.com/w/c/language/attributes/nodiscard	2026-09-22
wlaschin railway oriented programming	https://fsharpforfunandprofit.com/rop/	2026-09-22
defer ts 25755 and stddefer	books/c-os-cloud/chapters/06-stdlib.typ (corpus pinned coverage)	2026-09-22
nodiscard unused result diagnostic	books/c-os-cloud/chapters/07-llvm.typ (corpus pinned coverage)	2026-09-22

topic	url	accessed
union member access and type punning	https://en.cppreference.com/w/c/language/union	2026-09-22
monad laws (wadler venue)	https://homepages.inf.ed.ac.uk/wadler/topics/monads.html	2026-09-22
pipe operator spelling	https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/symbol-and-operator-reference/	2026-09-22
xorshift64 generator	https://www.jstatsoft.org/article/view/v008i14	2026-09-22
defer TS 25755 ground truth	books/c-os-cloud/chapters/06-stdlib.typ	2026-09-22
capstone core source	books/math/capstone/src	2026-09-22
capstone allegro host source	books/math/capstone/host	2026-09-22
capstone build gate	playground/math-capstone/build.ps1	2026-09-22
capstone host runner gate	tools/run-math-capstone.ps1	2026-09-22
openai math release, proofs as machine-checkable artifacts	https://openai.com/index/sharing-ai-progress-in-mathematics/	2026-10-09
openai math repository, ComparatorChallenges targets	https://github.com/openai/math	2026-10-09
family 107 preprint, 9/4 exponent over the complex numbers	ref/openai-math/family-107/paper-9fourths.pdf and paper-9fourths.tex, fetched 2026-10-11 from github.com/openai/math	2026-10-11
family 107 preprint, complex multiplication below 2.258	ref/openai-math/family-107/paper-below-2258.pdf, fetched 2026-10-11 from github.com/openai/math	2026-10-11
family 107 preprint, staggered extraction over every field	ref/openai-math/family-107/paper-staggered-extraction.pdf, fetched 2026-10-11 from github.com/openai/math	2026-10-11
openai math release overview, entry 107 and the millennium stance rows	ref/openai-math/overview.tex, fetched 2026-10-11 from github.com/openai/math	2026-10-11
openai math formalization manifest, the tier arbiter	ref/openai-math/formalization.yaml, fetched 2026-10-11 from github.com/openai/math	2026-10-11
family 107 lean sources and comparator challenges	ref/openai-math/family-107/lean-Main.lean, lean-AuxiliarySeparation-Main.lean, lean-ComparatorChallenges.lean, and doc-107.md, fetched 2026-10-11	2026-10-11
staggered-extraction verification tree, sha256-pinned inputs and verifier	ref/openai-math/family-107/staggered-verification/, scripts and data fetched 2026-10-11 from github.com/openai/math	2026-10-11
family 107 verifier recheck outputs, native log and rational series modes	ref/openai-math/family-107/recheck/, generated 2026-10-11 on this machine, provenance is the command lines in ref/openai-math/family-107/README.md	2026-10-11

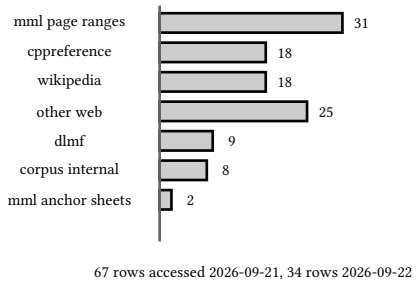


figure 199: the source base by class, the url prefix decides the class, bar length is row count

NOTE: the dated pins and drift

The mml rows point at the cached draft of 2024-01-15 and the page ranges are that draft's, a newer printing renumbers them. The cppreference, DLMF, and wikipedia rows are living pages fetched on the two access dates. If a page moves, this table is the changelog: the date says what was read, the url says where, and a recheck that disagrees reopens the chapter claim it licensed.

33.4. the corpus view

Fifteen typst documents compile under `books/`, and this one is book 2 in the reading order, position 07 in the `make books` output. The series pairs this book with its algorithm-first twins: the dsa book builds from scratch what this book treats as a library call, number theory in book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) and book 9, ch 28 (practical data structures and algorithms, advanced number theory), linear algebra in book 9, ch 34 (practical data structures and algorithms, linear algebra), counting in book 9, ch 35 (practical data structures and algorithms, combinatorics), the numerical solvers in book 9, ch 36 (practical data structures and algorithms, numerical methods), geometry in book 9, ch 23 (practical data structures and algorithms, computational geometry) and book 9, ch 37 (practical data structures and algorithms, computational geometry ii), and the game solvers beside this book's chapters 24 and 25 in book 9, ch 40 (practical

data structures and algorithms, game theory and scheduling). The game-systems book carries the production twin of the capstone's fixed step, deterministic fixed-point arithmetic in book 14, ch 7 (game systems and architecture, deterministic math). The c platform under chapters 26 to 29 is the same machine the c book dissects, integer representation in book 1, ch 5 (c23: the language, the machine, and the cloud, the abstract machine and undefined behavior) and float autovectorization in book 1, ch 9 (c23: the language, the machine, and the cloud, optimization: what -O actually does).

The reading order continues at book 9, where every algorithm this book used as a black box gets built from scratch.

VERIFY: recounting this chapter

Every total here recomputes. The gate command above prints the 113 files and 1767 checks, and the per-chapter sums match all 29 chapter closers digit for digit. The capstone's 64 checks sit behind `playground/math-capstone/build.ps1` with its host gates in `tools/run-math-capstone.ps1`, both quoted from chapter 32's session. The registry views recompute at compile time, they read the same coverage files the chapters wrote, so a row added or retitled tomorrow shows up here without editing this chapter.

sources: no external pages fetched for this chapter, none cited. Every count computed from books/math/coverage/topics.typ, 354 rows, and books/math/coverage/sources.typ, 111 rows, at compile time, from the suite run of `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/math/samples/src` on 2026-09-22, 113 files, 1767 checks, format and asan clean, and from the capstone gate receipts quoted in chapter 32 of 2026-09-22.